

SDMx Technical Specifications

Version 3.0

SDMx Technical Standards Working Group (TWG)

Copyright © SDMx TWG 2020

Table of contents

1. Getting Started 8
1.1 SDMx Technical Documentation 8
1.1.1 Project Description 8
1.1.2 Documentation Layout 8
1.1.3 Contents of the Documentation 8
1.1.4 Feedback 9
1.1.5 Matrix of SDMx and REST / Transmission Format Versions 9
2. Framework 10
2.1 Introduction 10
2.2 Change History 11
2.2.1 Major Changes from 1.0 to 2.0 11
2.2.2 Major Changes from 2.0 to 2.1 11
2.2.3 Major Changes from 2.1 to 3.0 12
2.3 Processes and Business Scope 14
2.3.1 Process Patterns 14
2.3.2 SDMX and Process Automation 15
2.3.3 Statistical Data and Metadata 15
2.3.4 The SDMX View of Statistical Exchange 18
2.3.5 SDMX Registry Services 21
2.3.6 RESTful Web services 22
2.4 The SDMX Information Model 23
2.5 The SDMX Transmission Formats 24
2.6 Dependencies on SDMX content-oriented guidelines 26
2.6.1 Cross-Domain Concepts 26
2.6.2 Metadata Common Vocabulary 26
2.6.3 Statistical Subject-Matter Domains 26
2.6.4 SDMX Concept Roles 26
2.7 Validation and Transformation Language 28
3. Information Model 29
3.1 Introduction 29
3.1.1 Related Documents 29
3.1.2 Modelling Technique and Diagrammatic Notes 29
3.1.3 Overall Functionality 30
3.2 Change History 33
3.2.1 Version 1.0 – initial release September 2004 33

Copyright © SDMx TWG 2020

3.2.2 Version 2.0 – release November 2005 33
3.2.3 Version 2.1 – release April 2011 34
3.2.4 Version 2.1 (Revision 2.0) – release June 2020 37
3.2.5 Version 3.0 – release September 2021 38
3.3 Actors and Use Cases 39
3.3.1 Introduction 39
3.3.2 Use Case Diagrams 39
3.4 SDMX Base Package 45
3.4.1 Introduction 45
3.4.2 Base Structures - Identification, Versioning, and Maintenance 46
3.4.3 Basic Inheritance 51
3.4.4 Data Types 52
3.4.5 The Item Scheme Pattern 53
3.4.6 The Structure Pattern 56
3.5 Specific Item Schemes 66
3.5.1 Introduction 66
3.5.2 Inheritance View 66
3.5.3 `Codelist` 67
3.5.4 `ValueList` 74
3.5.5 Concept Scheme and Concepts 76
3.5.6 `Category` Scheme 80
3.5.7 Organisation Scheme 85
3.5.8 Reporting Taxonomy 92
3.6 Data Structure Definition and Dataset 94
3.6.1 Introduction 94
3.6.2 Inheritance View 95
3.6.3 Data Structure Definition – Relationship View 97
3.6.4 Data Set – Relationship View 107
3.7 Cube 115
3.7.1 Context 115
3.7.2 Support for the Cube in the Information Model 115
3.8 Metadata Structure Definition and Metadata Set 116
3.8.1 Context 116
3.8.2 Inheritance 116
3.8.3 Metadata Structure Definition 118
3.8.4 Metadata Set 121
3.9 Hierarchy 127
3.9.1 Scope 127

Copyright © SDMx TWG 2020

3.9.2 Inheritance 128
3.9.3 Relationship 129
3.10 Structure Map 132
3.10.1 Scope 132
3.11 RepresentationMap 137
3.11.1 Scope 137
3.12 ItemSchemeMap 140
3.12.1 Scope 140
3.13 Constraints 142
3.13.1 Scope 142
3.13.2 Inheritance 142
3.13.3 Constraints 143
3.14 Data Provisioning 152
3.14.1 Class Diagram 152
3.14.2 Explanation of the Diagram 153
3.15 Process 158
3.15.1 Introduction 158
3.15.2 Model – Inheritance and Relationship view 158
3.16 Validation and Transformation Language 161
3.16.1 Introduction 161
3.16.2 Model - Inheritance view 162
3.16.3 Model - Relationship View 164
3.17 Appendix 1: A Short Guide to UML in the SDMX Information Model 174
3.17.1 Scope 174
3.17.2 Use Cases 174
3.17.3 Classes and Attributes 175
3.17.4 Associations 176
4. Registry Specification and Logical Interfaces 179
4.1 Registry Specification 179
4.1.1 Introduction 179
4.1.2 Scope and Normative Status 180
4.1.3 Scope of the SDMX Registry/Repository 181
4.1.4 SDMX Registry/Repository Architecture 185
4.1.5 Registry Interfaces and Services 187
4.2 Logical Interfaces 192
4.2.1 Identification of SDMX Objects 192
4.2.2 Implementation Notes 210

Copyright © SDMx TWG 2020

5. Technical Notes 231
5.1 Purpose and Structure 231
5.1.1 Purpose 231
5.1.2 Structure 231
5.2 General Notes on This Document 232
5.3 Guide for SDMX Format Standards 233
5.3.1 Introduction 233
5.3.2 SDMX Information Model for Format Implementers 233
5.3.3 SDMX Formats: Expressive Capabilities and Function 233
5.3.4 SDMX Best Practices 234
5.4 General Notes for Implementers 238
5.4.1 Representations 238
5.4.2 Time and Time Format 240
5.4.3 Versioning 250
5.4.4 Structural Metadata Querying Best Practices 252
5.5 Reference Metadata 253
5.5.1 Scope of the Metadata Structure Definition (MSD) 253
5.5.2 Identification of the Object(s) to which the Metadata is to be attached 253
5.5.3 Metadata Structure Definition 254
5.5.4 Metadata Set 255
5.5.5 Reference Metadata in Data Structure Definition and Dataset 255
5.6 Codelist 256
5.6.1 Codelist extension and discriminated unions 256
5.6.2 Linking Hierarchies 262
5.7 Geospatial information support 264
5.7.1 Indirect Reference to Geospatial Information 264
5.7.2 Geographic Coordinates 264
5.7.3 A Geographic Codelist 266
5.8 Maintenance Agencies and Metadata Providers 268
5.9 Concept Roles 270
5.9.1 Overview 270
5.9.2 Information Model 270
5.9.3 Technical Mechanism 271
5.9.4 SDMX-ML Examples in a DSD 271
5.9.5 SDMX standard roles Concept Scheme 272
5.10 Constraints 273
5.10.1 Introduction 273
5.10.2 Types of Constraint 273

Copyright © SDMx TWG 2020

5.10.3 Rules for a Constraint 273
5.11 Transforming between versions of SDMX 284
5.11.1 Scope 284
5.11.2 Compatibility and new DSD features 284
5.12 Validation and Transformation Language (VTL) 285
5.12.1 Introduction 285
5.12.2 References to SDMX artefacts from VTL statements 285
5.12.3 Mapping between SDMX and VTL artefacts 289
5.12.4 Mapping between SDMX and VTL Data Types 299
5.13 Structure Mapping 311
5.13.1 Introduction 311
5.13.2 1-1 structure maps 311
5.13.3 N-n structure maps 312
5.13.4 Ambiguous mapping rules 312
5.13.5 Representation maps 312
5.13.6 Regular expression and substring rules 314
5.13.7 Mapping non-SDMX time formats to SDMX formats 315
5.13.8 Using `TIME_PERIOD` in mapping rules 319
5.13.9 Time span mapping rules using validity periods 320
5.13.10 Mapping examples 320
5.14 ANNEX Semantic Versioning 323
5.14.1 Introduction to Semantic Versioning 323
5.14.2 Semantic Versioning Specification for SDMX 3.0(.0) 323
5.14.3 Backus–Naur Form Grammar for Valid SDMX 3.0(.0) Semantic Versions 324
5.14.4 Dependency Management in SDMX 3.0(.0) 326
5.14.5 Upgrade and conversions of artefacts defined with previous SDMX standard versions to Semantic Versioning 326
5.14.6 FAQ for Semantic Versioning 327
6. REST API (v2.2) 330
6.1 REST API 330
6.1.1 SDMX REST API: Developers' documentation 330
6.1.2 Data queries 332
6.1.3 Structure queries 340
6.1.4 Metadata queries 351
6.1.5 Schema queries 361
6.1.6 Data availability queries 364
6.1.7 Query syntax for specific, latest or all artefact versions 373
6.1.8 HTTP Content-Negotiation 376
6.1.9 Error handling and status information 377

Copyright © SDMx TWG 2020

6.1.10 Structure maintenance 378
6.1.11 Useful tips 389
6.1.12 Extending the SDMX REST API 392
7. Transmission Formats 394
7.1 SDMx-ML (v3.1) 394
7.1.1 Introduction 394
7.1.2 Message 399
7.1.3 Common Namespace 414
7.1.4 Structure Namespace 463
7.1.5 Data 743
7.1.6 Registry 769
7.1.7 Samples 795
7.2 SDMx-JSON (v2.1) 800
7.2.1 SDMX-JSON (v2.0) 800
7.2.2 Structure Message 801
7.2.3 Data Message 853
7.2.4 Metadata Message 890
7.3 SDMx-CSV (v2.1) 906
7.3.1 SDMX-CSV 906
7.3.2 Data Message Field Guide 907
7.3.3 Metadata Message Field Guide 915

Copyright © SDMx TWG 2020

1. Getting Started

1.1 SDMx Technical Documentation

This page is work in progress

The official documentation is available on https://www.sdmx.org

1.1.1 Project Description

This project provides a new, easier-to-browse and searchable version of the SDMx Technical Documentation. It brings together all active specifications from the official SDMx website and the GitHub repositories maintained by the SDMx Technical Standards Working Group (TWG), now shown centrally here.

1.1.2 Documentation Layout

- **Top-level navigation**: available through the horizontal bar at the top.
- **Second-level navigation**: available in the left sidebar.
- **Within-page navigation**: available in the right sidebar.

This structure allows quick access to high-level sections, detailed subsections, and in-page anchors.

1.1.3 Contents of the Documentation

The SDMx initiative (https://www.sdmx.org) sets standards for statistical data and metadata using modern information technology.

The SDMx Technical Specifications are organised into several discrete sections, which are all available in this documentation site.

Framework and Information Model

- Section 1 – Framework for SDMx Technical Standards Provides an introduction to the technical standards.
- Section 2 – SDMx Information Model A standardised object model for modelling statistical domains, covering the structure of data and metadata sets, coding schemes, and exchange rules. Includes UML specifications and narrative.

Registries and Technical Notes

- Section 5 – SDMx Registry Specification Specification of the SDMx Registry, a repository for structural metadata and provisioning information.
- Section 6 – SDMx Technical Notes Detailed technical guidance for implementors of the SDMx standard.

Transmission Formats and APIs

- REST API Technical specifications for the SDMx RESTful web services API.
- SDMx-ML Technical specifications for the XML transmission format, including schemas, documentation, and samples.
- SDMx-JSON Technical specifications for the JSON transmission format, including documentation, schemas, and samples.
- SDMx-CSV Technical specifications for the CSV transmission format for data and reference metadata.

Copyright © SDMx TWG 2020

VTL (Validation and Transformation Language)

Since 2020, the SDMx specifications include support for the **Validation and Transformation Language (VTL)**. For SDMx 3.0, the VTL specification has been updated to align with changes to the Information Model and the introduction of Semantic Versioning.

- Information Model covers the “Transformation and Expressions” package for defining and managing VTL 2.0 programs.
- Technical Notes provides detailed guidance on implementing and using VTL with SDMx.

Obsolete Sections (not included here)

- **Section 3 – SDMx-ML** (replaced by SDMx-ML repository)
- **Section 4 – SDMx-EDI**
- **Section 7 – API** (replaced by SDMx REST API)

1.1.4 Feedback

We welcome feedback to improve this documentation. Please provide your input via:

- GitHub Issues
- E-Mail: twg@sdmx.org

1.1.5 Matrix of SDMx and REST / Transmission Format Versions

The SDMx versions do not coincide with the versions of all sections of the standard. This page includes for each SDMx version the documentation of REST-API, SDMx-ML, SDMx-JSON, and SDMx-CSV that were released together with the standard:

SDMx version	REST-API version	SDMx-ML version	SDMx-JSON version	SDMx-CSV version
3.1	2.2	3.1	2.1	2.1
3.0	2.0	3.0	2.0	2.0
2.1	1.5	2.1	1.0	1.0

Please refer to the individual repositories for other versions of the packages:

- REST-API,
- SDMx-ML,
- SDMx-JSON, and
- SDMx-CSV.

Copyright © SDMx TWG 2020

2. Framework

2.1 Introduction

The Statistical Data and Metadata Exchange (SDMX) initiative (https://www.sdmx.org) sets standards that can facilitate the exchange of statistical data and metadata using modern information technology.

The SDMX Technical Specifications are organised into several discrete sections.

The following are published on the SDMX website (https://www.sdmx.org):

- **Section 1 – Framework for SDMX Technical Standards** — this document providing an introduction to the technical standards.
- **Section 2 – SDMX Information Model** — the SDMX information model is a standardised object model for modelling statistical domains centring on the structure of their data and metadata sets, the coding schemes used for classification, and the rules for controlling the exchange of data and metadata between organisations. This document provides a UML specification with supporting narrative.
- **Section 5 – SDMX Registry Specification** — an SDMX 'registry' acts as a repository for structural metadata and provisioning information, and a registry of data and metadata sources. This document sets out the specification.
- **Section 6 – SDMX Technical Notes** — detailed technical guidance for implementors of the SDMX standard.

The following are published on the GitHub repository of the SDMX Technical Working Group (TWG):

- **sdmx-twg/sdmx-rest – REST API** — Technical specifications for the SDMX RESTful web services application programming interfaces (API).
- **sdmx-twg/sdmx-ml – SDMX-ML** — Technical specifications for the XML transmission format including XSD schemas, documentation and samples for data, structure and reference metadata messages.
- **sdmx-twg/sdmx-json – SDMX-JSON** — Technical specifications for the JSON transmission format including documentation, schemas and samples for data, structure and reference metadata messages.
- **sdmx-twg/sdmx-csv – SDMX-CSV** — Technical specifications for the SDMX-CSV transmission format for 'comma-separated values' (CSV) data and reference metadata.

The following sections are obsolete:

- **Section 3 – SDMX-ML** — replaced by the sdmx-twg/sdmx-ml GitHub repository
- **Section 4 – SDMX-EDI**
- **Section 7 – API** — replaced by the sdmx-twg/sdmx-rest GitHub repository
- **VTL** — In July 2020 the SDMX 2.1 specifications were revised to add support for the Validation and Transformation Language (VTL). For 3.0, the VTL specification has been updated to align with changes to the information model and other modifications to the Standard such as the introduction of Semantic Versioning for the versioning of structural metadata artefacts. Section 2 (Information Model) sets out details of the 'Transformation and Expressions' package for defining and managing VTL 2.0 programs and Section 6 (Technical Notes) provides detailed guidance on implementing and using VTL with SDMX.

Copyright © SDMx TWG 2020

2.2 Change History

The 2.0 version of this standard represented a significant increase in scope, and also provided more complete support in those areas covered in the version 1.0 specification. Version 2.0 of this standard is backward-compatible with version 1.0, so that existing implementations can be easily migrated to conformance with version 2.0.

The 2.1 version of this standard represents a set of changes resulting from several years of implementation experience with the 2.0 standard. The changes do not represent a major increase in scope or functionality, but do correct some bugs, and add functionalities in some cases. Major changes in SDMX-ML include a much stronger alignment of the XML Schemas with the Information Model, to emphasize inheritance and object-oriented features, and increased precision and flexibility in the attachment of metadata reports to specific objects in the SDMX Information Model.

The 3.0 version incorporates new features, improvements and changes arising from the collective knowledge gained from a decade of operating experience with the 2.1 standard. In pursuit of modernisation and simplification, features considered obsolete have been deprecated – in particular the EDI transmission format, the lesser-used XML data messages and the SOAP web services API. Many areas remain backwardly compatible with 2.1, but there are some breaking changes where the information model has been redesigned to better support practical use case. Structure mapping and reference metadata are examples. The opportunity has been taken to revise the RESTful web services API which is also not backwardly compatible, but benefits from a rationalisation and better organisations of resources, and a much richer data query URL syntax.

2.2.1 Major Changes from 1.0 to 2.0

- **Reference Metadata:** In addition to describing and specifying data structures and formats (along with related structural metadata), the version 2.0 specification also provides for the exchange of metadata which is distinct from the structural metadata in the 1.0 version. This category includes “reference” metadata (regarding data quality, methodology, and similar types – it can be configured by the user to include whatever concepts require reporting); metadata related to data provisioning (release calendar information, description of the data and metadata provided, etc.); and metadata relevant to the exchange of categorization schemes.
- **SDMX Registry:** Provision is made in the 2.0 standard for standard communication with registry services, to support a data-sharing model of statistical exchange. These services include registration of data and metadata, querying of registered data and metadata, and subscription/notification.
- **Structural Metadata:** The support for exchange of statistical data and related structural metadata has been expanded. Some support is provided for qualitative data; data cube structures are described; hierarchical code lists are supported; relationships between data structures can be expressed, providing support for extensibility of data structures; and the description of functional dependencies within cubes are supported.

2.2.2 Major Changes from 2.0 to 2.1

- **Web-Services-Oriented Changes:** Several organizations have been implementing web services applications using SDMX, and these implementations have resulted in several changes to the specifications. Because the nature of SDMX web services could not be anticipated at the time of the original drafting of the specifications, the web services guidelines have been completely re-developed.
- **Presentational Changes:** Much work has gone into using various technologies for the visualization of SDMX data and metadata, and some changes have been proposed as a result, to better leverage this graphical visualization. These changes are largely to leverage the Cross-domain Concepts of the Content Oriented Guidelines.
- **Consistency Issues:** There have been some areas where the draft specifications were inconsistent in minor ways, and these have been addressed.
- **Clarifications in Documentation:** In some cases, it has been identified that the documentation of specific fields within the standard needed clarification and elaboration, and these issues have been addressed.
- **Optimization for XML Technologies:** Implementation has shown that it is possible to better organize the XML schemas for use within common technology development tools which work with XML. These changes are primarily focused on leveraging the object-oriented features of W3C XML Schema to allow for easier processing of SDMX data and metadata.

Copyright © SDMx TWG 2020

- **Consistency between the SDMX-ML and the SDMX Information Model:** Certain aspects of the XML schemas and UML model have been more closely aligned, to allow for easier comprehension of the SDMX model.
- **Technical Bugs:** Some minor technical bugs have been identified in the registry interfaces and elsewhere. These bugs have been addressed.
- **Support for Non-Time-Series Data in the Generic Format:** One area which has been extended is the ability to express non-time-series data as part of the generic data message.
- **Simplification of the data structure definition - specific message types:** Both time series (version 2.0 Compact) and non-time series data sets (version 2.0 Cross Sectional) use the same underlying structure for a structure-specific formatted message, which is specific to the Data Structure Definition of the data set.
- **Simplification and better support for the metadata structure:** New use cases have been reported and these are now supported by a re-modelled metadata structure definition.
- **Support for partial item schemes such as a code list:** The concept of a partial (sub-set) item scheme such as a partial code list for use in exchange scenarios has been introduced**.**

2.2.3 Major Changes from 2.1 to 3.0

SDMX version 3.0 introduces new features, improvements and changes to the Standard in the following key areas:

Information Model

- Simplification and improvement of the reference metadata model
- Support for microdata
- Support for geospatial data
- Support for code list extension and discriminated union of code lists
- Improvements to structure mapping
- Improvements to code hierarchies for data discovery
- Improvements to constraints

Versioning of Structural Metadata Artefacts

- Adoption of the three-number semantic versioning standard for structural metadata artefacts (https://semver.org)

REST Web Services Application Programming Interface (API)*

- Change to a single ‘structure’ resource for structure queries simplifying the REST API specification by reducing the number of resources to five
- Improvements to data queries
- Improvements to reference metadata queries
- Support for structural metadata maintenance using HTTP PUT, POST and DELETE verbs

SOAP Web Services API

- The SOAP web services API has been deprecated with version 3.0 standardising on REST

XML, JSON, CSV and EDI Transmission formats

- The SDMX-ML, SDMX-JSON and SDMX-CSV specifications have been extended and modified where needed to support the new features and changes such as reference metadata and microdata
- Obsolete SDMX-ML data message variants including Generic, Compact, Utility and Cross-sectional have been deprecated standardising on Structure Specific Data as the sole XML format for data exchange
- The SDMX-EDI transmission format for structures and data has been deprecated

Copyright © SDMx TWG 2020

- The organisation of structures into ‘collections’ in SDMX-ML and SDMX-JSON structure messages has been flattened and simplified
- The option to reference structures in SDMX-ML and SDMX-JSON messages using Agency, ID and Version has been deprecated with URN now exclusively used for all non-local referencing purpose

Several of the changes are ‘breaking’ meaning that, in specific cases, the version 3.0 specification is not backwardly compatible with earlier versions of the Standard.

The principle breaking changes are:

- REST API – The REST API is not backwardly compatible due to modifications to the URLs and query parameters resulting in breaking changes in four of the five main resources.
- SOAP API – Deprecation of the SOAP API means that existing systems designed to use SOAP will not work with version 3.0 registries.
- SDMX-ML – SDMX 2.1 and earlier structure, data and metadata XML messages are not valid in version 3.0. Specifically: legacy data messages including Generic, Compact and Utility are no longer supported. The remaining Structure Specific data message has been changed to support new features such as reporting of reference metadata as part of the dataset, Structure messages have a number of breaking changes, principally modification to the information model, removal of the agency-version-id option for referencing artefacts and changes to the way the structures are organised into ‘collections’ within the message.
- SDMX-JSON – SDMX 2.1 structure, data and metadata JSON messages are not valid in version 3.0. The data message has been changed to support the improved REST data queries, in particular the ability to retrieve in one operation data from multiple datasets with potentially different Data Structure Definitions. Breaking changes similar to those for the SDMX-ML transmission format have been made to the structure message.
- SDMX-CSV – The CSV data and reference metadata messages are not backwardly compatible with those under version 2.1 due to changes to the structure of the messages needed to support new features such as the improved REST API data queries.
- SDMX-EDI – Deprecation of the EDI transmission format means that existing systems designed to send or receive structures or data in EDI will not work with version 3.0 registries.
- Information Model – Several structures have been changed in the version 3.0 model and three removed. For these reasons the version 3.0 model is not directly compatible with version 2.1 or earlier, although conversion of specific artefacts is possible under some circumstances. Loss of information during the conversion process however means that in cases like structure mapping, the conversion is not reversible i.e. it is not possible to recreate the 2.1 structure once it has been converted to the 3.0 model.

The SDMX 3.0 Major Changes document provides more information including an analysis of the breaking changes.

Copyright © SDMx TWG 2020

2.3 Processes and Business Scope

2.3.1 Process Patterns

SDMX identifies three basic process patterns regarding the exchange of statistical data and metadata. These can be described as follows:

1. **Bilateral exchange:** All aspects of the exchange process are agreed between counterparties, including the mechanism for exchange of data and metadata, the formats, the frequency or schedule, and the mode used for communications regarding the exchange. This is perhaps the most common process pattern.
2. **Gateway exchange:** Gateway exchanges are an organized set of bilateral exchanges, in which several data and metadata collecting organizations or individuals agree to exchange the collected information with each other in a single, known format, and according to a single, known process. This pattern has the effect of reducing the burden of managing multiple bilateral exchanges (in data and metadata collection) across the sharing organizations/individuals. This is also a very common process pattern in the statistical area, where communities of institutions agree on ways to gain efficiencies within the scope of their collective responsibilities.
3. **Data-sharing exchange:** Open, freely available data formats and process patterns are known and standard. Thus, any organization or individual can use any counterparty's data and metadata (assuming they are permitted access to it). This model requires no bilateral agreement, but only requires that data and metadata providers and consumers adhere to the standards.

This document specifies the SDMX standards designed to facilitate exchanges based on any of these process patterns, and shows how SDMX offers advantages in all cases. It is possible to agree bilaterally to use a standard format (such as SDMX-ML or SDMX-JSON); it is possible for data senders in a gateway process to use a standard format for data exchange with each other, or with any data providers who agree to do so; it is possible to agree to use the full set of SDMX standards to support a common data-sharing process of exchange, whether based on an SDMX-conformant registry or some other architecture.

The standards specified here specifically support a data-sharing process based on the use of central registry services. Registry services provide visibility into the data and metadata existing within the community, and support the access and use of this data and metadata by providing a set of triggers for automated processing. The data or metadata itself is not stored in a central registry – these services merely provide a useful set of metadata about the data (and additional metadata) in a known location, so that users/applications can easily locate and obtain whatever data and/or metadata is registered. The use of standards for all data, metadata, and the registry services themselves is ubiquitous, permitting a high level of automation within a data-sharing community.

It should be pointed out that these different process models are not mutually exclusive – a single system capable of expressing data and metadata in SDMX-conformant formats could support all three scenarios. Different standards may be applicable to different processes (for example, many registry services interfaces are used only in a data-sharing scenario) but all have a common basis in a shared information model.

In addition to looking at collection and reporting, it is also important to consider the dissemination of data. Data and metadata – no matter how they are exchanged between counterparties in the process of their development and creation – are all eventually supplied to an end user of some type. Often, this is through specific applications inside of institutions. But more and more frequently, data and metadata are also published on websites in various formats. The dissemination of data and its accompanying metadata on the web is a focus of the SDMX standards. Standards for statistical data and metadata allow improvements in the publication of data – it becomes more easily possible to process a standard format once the data is obtained, and the data and metadata are linked together, making the comprehension and further processing of the data easier.

In discussions of statistical data, there are many aspects of its dissemination which impact data quality: data discovery, ease of use, and timeliness. SDMX standards provide support for all of these aspects of data dissemination. Standard data formats promote ease of use, and provide links to relevant metadata. The concept of registry services means that data and metadata can more easily be discovered. Timeliness is improved throughout the data lifecycle by increases in efficiency, promoted through the availability of metadata and ease of use.

It is important to note that SDMX is primarily focused on the *exchange* and *dissemination* of statistical data and metadata. There may also be many uses for the standard model and formats specified here in the context of internal processing of data that are not concerned with the exchange between organizations and users, however. It is felt that a clear, standard formatting of data and

Copyright © SDMx TWG 2020

metadata for the purposes of exchange and dissemination can also facilitate internal processing by organizations and users, but this is not the focus of the specification.

2.3.2 SDMX and Process Automation

Statistical data and metadata exchanges employ many different automated processes, but some are of more general interest than others. There are some common information technologies that are nearly ubiquitous within information systems today. SDMX aims to provide standards that are most useful for these automated processes and technologies.

Briefly, these can be described as:

1. **Batch Exchange of Data and Metadata:** The transmission of whole or partial databases between counterparties, including incremental updating.
2. **Provision of Data and Metadata on the Internet:** Internet technology - including its use in private or semi-private TCP/IP networks - is extremely common. This technology includes XML, JSON and REST web services as primary mechanisms for automating data and metadata provision, as well as the more traditional static HTML and database-driven publishing.
3. **Generic Processes:** While many applications and processes are specific to some set of data and metadata, other types of automated services and processes are designed to handle any type of statistical data and metadata whatsoever. This is particularly true in cases where portal sites and data feeds are made available on the Internet.
4. **Presentation and Transformation of Data:** In order to make data and metadata useful to consumers, they must support automated processes that transform them into application-specific processing formats, other standard formats, and presentational formats. Although not strictly an aspect of exchange, this type of automated processing represents a set of requirements that must be supported if the information exchange between counterparties is itself to be supported.

The SDMX standards specified here are designed to support the requirements of all of these automation processes and technologies.

2.3.3 Statistical Data and Metadata

To avoid confusion about which “data” and “metadata” are the intended content of the SDMX formats specified here, a statement of scope is offered. Statistical “data” are sets of often numeric observations which typically have time associated with them. They are associated with a set of metadata values, representing specific concepts, which act as identifiers and descriptors of the data. These metadata values and concepts can be understood as the named dimensions of a multi-dimensional co-ordinate system, describing what is often called a “cube” of data.

SDMX identifies a standard technique for modelling, expressing, and understanding the structure of this multi-dimensional “cube”, allowing automated processing of data from a variety of sources. This approach is widely applicable across types of data and attempts to provide the simplest and most easily comprehensible technique that will support the exchange of this broad set of data and related metadata.

The term “metadata” is very broad indeed. A distinction can be made between “structural” metadata – those concepts used in the description and identification of statistical data and metadata – and “reference” metadata – the larger set of concepts that describe and qualify statistical data sets and processing more generally, and which are often associated not with specific observations or series of data, but with entire collections of data or even the institutions which provide that data.

The SDMX Information Model provides for the structuring not only of data, but also of “reference” metadata. While these reference metadata structures exist independent of the data and its structural metadata, they are often linked. The SDMX Information Model provides for the attachment of reference metadata to any part of the data or structural metadata, as well as for the reporting and exchange of the reference metadata and its structural descriptions. This function of the SDMX standards supports many aspects of data quality initiatives, allowing as it does for the exchange of metadata in its broadest sense, of which quality-related metadata is a major part.

Metadata are associated not only with data, but also with the process of providing and managing the flow of data. The SDMX Information Model provides for a set of metadata concerned with “data provisioning” – metadata which are useful to those who need to understand the content and form of a data provider’s output. Each data provider can describe in standard fashion the content of and dependencies within the data and metadata sets which they produce, and supply information about the scheduling

Copyright © SDMx TWG 2020

and mechanism by which their data and metadata are provided. This allows for automation of some validation and control functions, as well as supporting management of data reporting.

SDMX also recognizes the importance of classification schemes in organizing and managing the exchange and dissemination of data and metadata. It is possible to express information about classification schemes and domain categories in SDMX, along with their relationships to data and metadata sets, as well as to categorize other objects in the model.

The SDMX standards offer a common model, a choice of syntax and, for XML, a choice of data formats which support the exchange of any type of statistical data meeting the definition above; several optimized formats are specified based on the specific requirements of each implementation, as described below in the SDMX-ML section.

The formal objects in the information model are presented schematically in Figure 1, and are discussed in more detail elsewhere in this document.

Copyright © SDMx TWG 2020

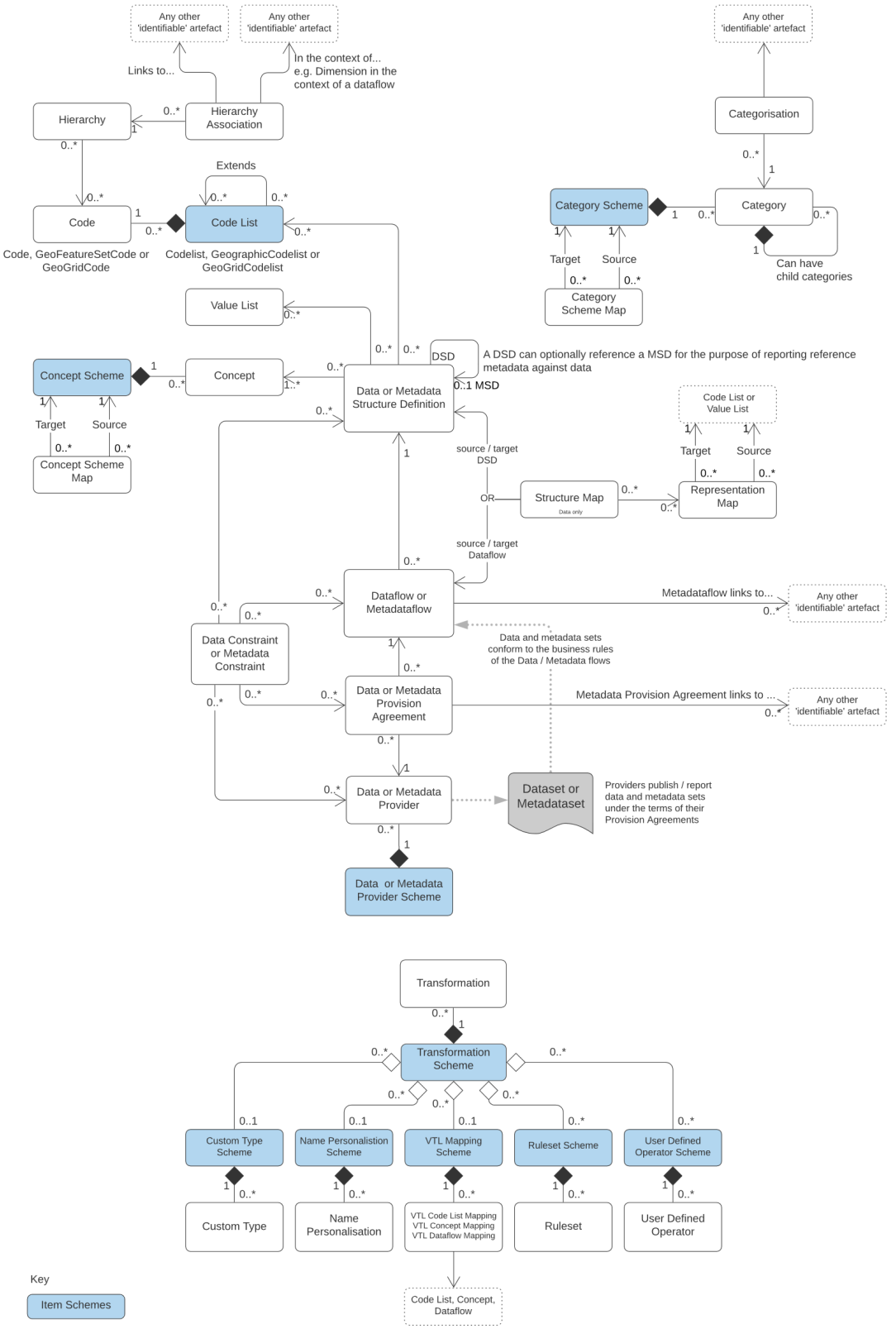


Figure 1. High Level Schematic of Major Artefacts in the SDMX 3.0 Information Model

Copyright © SDMx TWG 2020

2.3.4 The SDMX View of Statistical Exchange

Version 1.0 of ISO/TS 17369 SDMX covered statistical data sets and the metadata related to the structure of these data sets. This scope was useful in supporting the different models of statistical exchange (bilateral exchange, gateway exchange, and data-sharing) but was not by itself sufficient to support them completely. Versions 2.0 and 2.1 provide a much more complete view of statistical exchange, so that an open data-sharing model can be fully supported, and other models of exchange can be more completely automated. In order to produce technical standards that will support this increased scope, the SDMX Information Model provides a broader set of formal objects which describe the actors, processes, and resources within statistical exchanges.

It is important to understand the set of formal objects not only in a technical sense, but also in terms of what they represent in the real-world exchange of statistical data and metadata.

The first version of SDMX provided for data sets - specific statistical data reported according to a specific structure, for a specific time range - and for data structure definitions - the metadata which describes the structure of statistical data sets. These are important objects in statistical exchanges, and are retained and enhanced in the second version of the standards in a backward-compatible form. A related object in statistical exchanges is the “data flow” - this supports the concept of data reporting or dissemination on an ongoing basis. “Data flows” can be understood as data sets which are not bounded by time. Data structures are owned and maintained by agencies - in a similar fashion, data flows are owned by maintenance agencies.

SDMX allows for the publication of statistical data (and the related structural metadata) but also provided for the standard, systematic representation of reference metadata. In version 2.1, reference metadata were reported independent of the statistical data. However, in 3.0 reference metadata associated directly with data such as footnotes are reported as attributes of the data set. For other reference metadata, principally that linked to structures like “concepts”, SDMX provides reference “metadata sets”, “metadata structure definitions”, and “metadata flows”. These objects are very similar to data sets, data structure definitions, and data flows, but concern reference metadata rather than statistical observations. In the same way that data providers may publish statistical data, they may also publish reference metadata. Metadata structural definitions are maintained by agencies in a fashion similar to the way that agencies maintain data structure definitions, the structural definitions of data sets.

The structural definitions of both data and reference metadata associate specific statistical concepts with their representations, whether textual, coded, etc. These concepts are taken from a “concept scheme” which is maintained by a specific agency. Concept schemes group a set of concepts, provide their definitions and names, and allow for semantic relationships to be expressed, when some concepts are specializations of others. It is possible for a single concept scheme to be used both for data structures - key families - and for reference metadata structures.

Inherent in any statistical exchange – and in many dissemination activities – is a concept of “service level agreement”, even if this is not formalized or made explicit. SDMX incorporates this idea in objects termed “provision agreements”. Data providers may provide data to many different data flows. Data flows may incorporate data coming from more than one data provider. Provision agreements are the objects which tell you which data providers are supplying what data to which data flows. Similarly, metadata provision agreements for metadata flows.

Provision agreements allow for a variety of information to be made available: the schedule by which statistical data or metadata is reported or published, the specific topics about which data or metadata is reported within the theoretically possible set of data (as described by a data structure definition or reference metadata structure definition), and the time period covered by the statistical data and metadata. This set of information is termed “constraint” in the SDMX Information Model.

Copyright © SDMx TWG 2020

A brief summary of the objects described in the information model includes:

Copyright © SDMx TWG 2020

- **Data Set:** Data is organized into discrete sets, which include particular observations for a specific period of time. A data set can be understood as a collection of similar data, sharing a structure, which covers a fixed period of time.
- **Data Structure Definition (DSD, also known as Key Family in Version 2.0):** Each data set has a set of structural metadata. These descriptions are referred to in SDMX as Data Structure Definitions, which include information about how concepts are associated with the measures, dimensions, and attributes of a data "cube," along with information about the representation of data and related identifying and descriptive (structural) metadata. In Version 2.1, the term "Key Family" was replaced by "Data Structure Definition" (DSD) both in XML Schemas and the Information Model. The DSD has been modified in version 3.0 to better support microdata by providing the option to define multiple measures and for attributes and measures to take arrays of values. An optional reference to a Metadata Structure Definition has also been added for describing the reference metadata associated with the data. When reported, these reference metadata are included as part of the dataset.
- **Code list:** Code lists enumerate a set of codes to be used in the representation of dimensions, attributes, and other structural parts of SDMX. Codes can be organised into simple hierarchies within a code list, and more complex hierarchies potentially involving multiple code lists using hierarchy and hierarchy association structures.
- **Value list:** Value lists introduced in version 3.0 are similar to codelists with the exception that the items do not need to conform to the usual SDMX rules for identifiable objects. That allows the values to include characters such as currency symbols (e.g. ¥) which would otherwise make illegal codes. However, unlike codes, values are not individually identifiable. Value lists find application in concepts and data structures definitions for less structured data and microdata enumerations and can be mapped to other value or code lists using representation maps.
- **Organisation Scheme:** Organisations and organisation structure can be defined in an Organisation Scheme. Specific Organisation Schemes exist for Maintenance Agency, Data Provider, Metadata Provider, Data Consumer, and Organisation Unit.
- **Category Scheme and Categorisation:** Category schemes are made up of a hierarchy of categories, which in SDMX may include any type of useful classification for the organization of data and metadata. A Categorisation links a category to an identifiable object. In this way sets of objects can be categorised. A statistical subject-matter domain scheme is implemented in SDMX as a Category Scheme.
- **Concept Scheme:** A concept scheme is a maintained list of concepts that are used in data structure definitions and metadata structure definitions. There can be many such concept schemes. A "core" representation of the concept can be specified (e.g. a core code list, or other representation such as "date"). Note that this core representation can be overridden in the data structure definition or metadata structure definition that uses the concept. Indeed, organisations wishing to remain with version 1.0 key family schema specifications will continue to declare the representation in the key family definition.
- **Metadata Set:** A reference metadata set is a set of information pertaining to an object within the formal SDMX view of statistical exchange: they may describe the maintainers of data or structural definitions; they may describe the schedule on which data is released; they may describe the flow of a single type of data over time; they may describe the quality of data, etc. In SDMX, the creators of reference metadata may take whatever concepts they are concerned with, or obliged to report, and provide a reference metadata set containing that information.
- **Metadata Structure Definition:** A reference metadata set also has a set of structural metadata which describes how it is organized. This metadata set identifies what reference metadata concepts are being reported, how these concepts relate to each other (typically as hierarchies), what their presentational structure is, how they may be represented (as free text, as coded values, etc.), and with which formal SDMX object types they are associated.
- **Dataflow Definition:** In SDMX, data sets are reported or disseminated according to a data flow definition. The data flow definition identifies the data structure definition and may be associated with one or more subject matter domains via a Categorisation (this facilitates the search for data according to organised category schemes). Constraints, in terms of reporting periodicity or sub set of possible keys that are allowed in a data set, may be attached to the data flow definition.
- **Metadataflow Definition:** A metadata flow definition is very similar to a data flow definition, but describes, categorises, and constrains metadata sets.
- **Data Provider:** An organization which produces data is termed a data provider.
- **Metadata Provider:** An organization which produces reference metadata is termed a metadata provider.
- **Provision Agreement (Metadata Provision Agreement):** The set of information which describes the way in which data sets and metadata sets are provided by a data/metadata provider. A provision agreement can be constrained in much the same way as a data or metadata flow definition. Thus, a data provider can express the fact that it provides a particular data flow covering a specific set of countries and topics, Importantly, the actual source of registered data or metadata is attached to the provision agreement (in terms of a URL). The term "agreement" is used because this information can be understood as the basis of a "service-level agreement". In SDMX, however, this is informational metadata to support the technical systems, as opposed to

Copyright © SDMx TWG 2020

any sort of contractual information (which is outside the scope of a technical specification). In version 3.0, metadata provision agreement and data provision agreement are two separate artefacts.

- **Constraint:** Data and Metadata Constraints describe a subset of a data source or metadata source, and may also provide information about scheduled releases of data. They are associated with data / metadata providers, provision agreements, data flows, metadataflows, data structure definitions and metadata structure definitions.
- **Structure Map:** Structure maps describes a mapping between data structure definitions or dataflows for the purpose of transforming a data set into a different structure. The mapping rules are defined using one or more component maps which each map in turn describes how one or more components from the source data structure definition map to one or more components in that of the target. Represent maps act as lookup tables and specific provision is made for mapping dates and times.
- **Representation Map:** Representation maps describe mappings between source value(s) and target value(s) where the values are restricted to those in a code list, value list or be of a certain type such as integer or string.
- **Item Scheme Map:** An item scheme map describes mapping rules between any item scheme with the exception of code lists and value lists which use representation maps. The version 3.0 information model provides four item scheme maps: organisation scheme map, concept scheme map, category scheme map and reporting taxonomy map. Organisation scheme map and reporting scheme map have been omitted from the information model schematic in Figure 1.
- **Reporting Taxonomy:** A reporting taxonomy allows an organisation to link (possibly in a hierarchical way) a number of cube or data flow definitions which together form a complete “report” of data or metadata. This supports primary reporting which often comprises multiple cubes of heterogeneous data, but may also support other collection and reporting functions. It also supports the specification of publications such as a yearbook, in terms of the data or metadata contained in the publication.
- **Process:** The process class provides a way to model statistical processes as a set of interconnected *process steps.* Although not central to the exchange and dissemination of statistical data and metadata, having a shared description of processing allows for the interoperable exchange and dissemination of reference metadata sets which describe processes-related concepts.
- **Hierarchy:** Describes complex code hierarchies principally for data discovery purposes. The codes themselves are referenced from the code lists in which they are maintained.
- **Hierarchy Association:** A hierarchy association links a hierarchy to something that needs it like a dimension. Furthermore, the linking can be specified in the context of another object such as a dimension in the context of a dataflow. Thus, a dimension in a data structure definition could have different hierarchies depending on the dataflow.
- **Transformation Scheme:** A transformation scheme is a set of Validation and Transformation Language (VTL) transformations aimed at obtaining some meaningful results for the user (e.g., the validation of one or more data sets). The set of transformations is meant to be executed together (in the same run) and may contain any number of transformations in order to produce any number of results. Thus, a transformation scheme can be considered as a VTL ‘program’.

2.3.5 SDMX Registry Services

In order to provide visibility into the large amount of data and metadata which exists within the SDMX model of statistical exchange, it is felt that an architecture based on a set of registry services is potentially useful. A “registry” – as understood in webservices terminology – is an application which maintains and stores metadata for querying, and which can be used by any other application in the network with sufficient access privileges (though note that the mechanism of access control is outside of the scope of the SDMX standard). It can be understood as the index of a distributed database or metadata repository which is made up of all the data provider’s data sets and reference metadata sets within a statistical community, located across the Internet or similar network.

Note that the SDMX registry services are not concerned with the storage of data or reference metadata. The assumption is that data and reference metadata lives on the sites of its data and metadata providers. The SDMX registry services concern themselves with providing visibility of the data and reference metadata, and information needed to access the data and reference metadata. Thus, a registered data set will have its URL available in the registry, but not the data itself. An application which wishes to access that data would query the registry, perhaps by drilling down via a Category Scheme and Dataflow, for the URL of a registered data source, and then retrieve the data directly from the data provider (using an SDMX REST API query message or other mechanism).

SDMX does not require a particular technology implementation of the registry – instead, it specifies the standard interfaces which may be supported by a registry. Thus, users may implement an SDMX-conformant registry in any fashion they choose, provided the interfaces are supported as specified in the Registry Specification Section. These interfaces are expressed as XML documents, but also REST API request/response messages

Copyright © SDMx TWG 2020

The registry services discussed here can be briefly summarized:

- **Maintenance of Structural Metadata:** This registry service allows users with maintenance agency access privileges to submit and modify structural metadata. In this aspect the registry is acting as a structural metadata repository. However, it is permissible in an SDMX structure to submit just the “stub” of the structural object, such as a code list, and for this stub to reference the actual location from where the metadata can be retrieved, either from a file or a structural metadata resource, such as another registry.
- **Registration of Data and Metadata Sources:** This registry service allows users with maintenance agency access privileges to inform the registry of the existence and location (for retrieval) of data sets and reference metadata sets. The registry stores metadata about these objects, and links it to the structural metadata that give sufficient structural information for an application to process it, or for an application to discover its existence. Objects in the registry are organized and categorized according to one or more category schemes.
- **Querying:** The registry services have interfaces for querying the metadata contained in a registry, so that applications and users can discover the existence of data sets and reference metadata sets, structural metadata, the providers/agencies associated with those objects, and the provider agreements which describe how the data and metadata are made available, and how they are categorized.
- **Subscription/Notification:** It is possible to “subscribe” to specific objects in a registry, so that a notification will be sent to all subscribers whenever the registry objects are updated.

2.3.6 RESTful Web services

Web services allow computer applications to exchange data directly over the Internet, essentially allowing modular or distributed computing in a more flexible fashion than ever before. In order to allow web services to function, however, many standards are required: for requesting and supplying data; for expressing the enveloping data which is used to package exchanged data; for describing web services to one another, to allow for easy integration into applications that use other web services as data resources.

Version 3.0 has standardized on RESTful web services with a OpenAPI specification published on the SDMX Technical Working Group’s GitHub repository. There are five ‘resources’:

- structure — retrieval and maintenance of structural metadata
- data — retrieval of data
- schema — retrieval of XML schemas to validate specific data or metadata sets
- availability — retrieval of information on the data available for a Dataflow
- metadata — retrieval of reference metadata

The following conceptual example uses the ‘data’ resource to query a data repository for a series identified by the key ‘M.USD.EUR.SP00.A’ in the EXR (ECB exchange rates) Dataflow:

```
https://ws-entry-point/data/dataflow/ECB/EXR/1.0.0/M.USD.EUR.SP00.A
```


Copyright © SDMx TWG 2020

2.4 The SDMX Information Model

SDMX provides a way of modelling statistical data, and defines the set of metadata constructs used for this purpose. Because SDMX specifies a number of transmission formats for expressing data and structural metadata, the model is used as a mechanism for guaranteeing that transformation between the different formats is lossless. In this sense, all of the formats are syntax-bound expressions of the common information model.

SDMX recognizes that statistical data is structured; in SDMX this structure is termed a Data Structure Definition. “Data sets” are made up of one or more lower-level “groups”, based on their degrees of similarity. Each group is in turn comprised of one or more “series” of data. Each series or section has a “key” - values for each of a cluster of concepts, also called “dimensions” - which identifies it, and one or more “observations”, which typically combine the time of the observation, and the value of the observation (e.g., measurement). Additionally, metadata may be attached at any level of this structure as descriptive “attributes”. Code lists (enumerations) and other patterns for representation of data and metadata are also modelled.

There is some similarity between “cube” structures commonly used to process statistical data, and the Data Structure Definition idea in the SDMX Information Model. It is important to note that the data as structured according to the SDMX Information Model is optimized for exchange, potentially with partners who may have no ability to process a “cube” of data coming from complex statistical systems. SDMX time series can be understood as “slices” of the cube. Such a slice is identified by its key. A “series” key consists of the values for all dimensions specified by the key family except time. Thus, it is possible to reconstruct and describe data cubes from SDMX-structured data, and to exchange such databases using the interfaces and formats provided for that purpose in the standard. Additional objects such as hierarchical code lists, constraints and structure maps make it possible to more fully model the structure of cubes.

The information model also provides a view of reference metadata: a mechanism for referencing the meaningful “objects” within the SDMX view of statistical exchange processes (data providers, structures, provisioning agreements, dataflows, metadata flows, etc.) to which metadata is attached; a mechanism for describing a set of meaningful concepts, of organizing them into a presentational structure, and of indicating how their values are represented. This is based on a simple, hierarchical view of reference metadata which is common to many metadata systems and classification/categorization schemes. SDMX provides a model (and XML and JSON formats) for both describing reference metadata structures, and of reporting reference metadata according to those structures.

Version 2.0/2.1 introduced support for metadata related to the process aspects of statistical exchange. A step-by-step process can be modelled; information about who is providing data and reference metadata and how they are providing it can be expressed; and the technical aspects of service-level agreements (and similar types of provisioning agreements) can be represented.

Support for the Validation and Transformation Language (VTL) in the SDMX Information Model was introduced in the July 2020 revision of 2.1 and is retained in version 3.0 with minimal changes. This allows reusable VTL ‘programs’ (a cohesive set of transformation statements designed to be executed together) and their associated constructs such as validation rulesets and user-defined operators to be managed and exchanged as SDMX structural metadata. Mappings between objects such as data sets referenced in VTL programs and the actual SDMX artefacts to which they relate is essential when it comes to actually executing programs, and this information can also be defined. Chapter 7 has more information on VTL and its integration with SDMX.

A full UML conceptual design of the information model is set out in Section 2 of the Technical Specifications.

Copyright © SDMx TWG 2020

2.5 The SDMX Transmission Formats

SDMX-ML

SDMX-ML is the XML transmission format specification for exchanging structural metadata, data and reference metadata, and interacting with SDMX registry services. It is designed as a general-purpose format for all automation and data / metadata exchange tasks, and provides the most complete coverage.

There are four distinct types of message:

1. **Structure Definition:** For the exchange of structural metadata. A SDMX-ML structure message can carry details of any number and combination of structural metadata artefacts like DSDs, code lists and constraints.
2. **Structure-specific Data:** For the exchange of data. This format is specific to the Data Structure Definitions of the data sets (in other terms, it is DSD-specific) and is created by following mappings between the metadata constructs defined in the Structure Definition message and the technical specification of the format. It supports the exchange of large data sets in XML format, provides strict validation of conformance with the DSD using a generic XML parser, and supports the transmission of partial data sets (incremental updates) as well as whole data sets.

   Many XML tools and technologies have expectations about the functions performed by an XML schema, one of which is a very direct relationship between the XML constructs described in the XML schema and the tagged data in the XML instance. Strong data typing is also considered normal, supporting full validation of the tagged data. These message types are designed to support validation and other expected XML schema functions.
3. **Generic Metadata:** For the exchange of reference metadata sets. ‘Generic’ means the XML elements and XML attributes are the same regardless of the metadata set.
4. **Registry:** All of the possible interactions with the SDMX registry services are supported using SDMX-ML interfaces and REST API calls. Submission of structural metadata content, data / metadata registrations and subscriptions is performed by a synchronous exchange of documents – a “request” message answered by a “response” message.

SDMX-JSON

SDMX-JSON is the JSON transmission format specification for exchanging structural metadata, data and reference metadata. It provides an alternative to SDMX-ML and is most suited to applications like web data dissemination.

SDMX-JSON messages serve the same function as those of the XML formats but have a different structure. For data, an important distinction is that they carry both component codes and labels which provides all the information needed to display the content in a single JSON response. The XML Structure-specific Data format by contrast carries only code IDs thus requiring applications obtain and hold structural metadata about the data set in order to display the content in human-readable form.

SDMX-JSON does not include messages for subscription / notification or registration registry services - SDMX-ML must be used for those purposes.

There are three distinct message types:

1. **Structure:** For the exchange structural metadata. SDMX-JSON structure messages follow the same principles as for SDMX-ML in that a single message can transmit any number and combination of structural metadata artefacts. While the SDMX-ML and SDMX-JSON messages are structured differently, it is possible to freely convert between them.
2. **Data:** For the exchange of data. Unlike SDMX-ML, the structure of a SDMX-JSON data message is not specific to the DSDs of the data sets so schema validation will not check for compliance of the data with the DSDs.
3. **Metadata:** For the exchange of reference metadata sets.

SDMX-CSV

SDMX-CSV is the CSV transmission format specification for exchanging data and reference metadata only.

Copyright © SDMx TWG 2020

SDMX-CSV provides a simple columnar format for data and metadata that can be readily created and interpreted by standard software tools such as Microsoft Excel. Nevertheless, data and metadata can still be converted between the CSV and the JSON / XML formats without loss.

There are two distinct message types:

1. **Data:** For the exchange of data. Like SDMX-JSON, SDMX-CSV can include both code IDs and labels which is helpful when using the data to create human readable charts and dashboards.
2. **Metadata:** For the exchange of reference metadata sets.

Formats and Messages Deprecated in Version 3.0

The following formats and messages have been deprecated in version 3.0 to simplify, modernise and rationalise the standard.

- SDMX-EDI
- SDMX-ML 1.0/2.0 Generic (time-series) data message
- SDMX-ML 1.0/2.0 Compact (time-series) data message
- SDMX-ML 1.0/2.0 Utility (time-series) data message
- SDMX-ML 1.0/2.0 Cross-Sectional data message
- SDMX-ML 2.1 Generic (Time Series) data messages (for observations, time-series and cross-sectional data)
- SDMX-ML 2.1 Structure Specific Time Series data message

The following messages were deprecated in version 3.0 as a consequence of the deprecation of the SOAP web services:

- SDMX-ML Query messages
- SDMX-ML Submit Structure Request messages

Copyright © SDMx TWG 2020

2.6 Dependencies on SDMX content-oriented guidelines

The technical standards proposed here are designed so that they can be used in conjunction with other SDMX guidelines which are more closely tied to the content and semantics of statistical data exchange. The SDMX Information Model works equally well with any statistical concept, but to encourage interoperability, it is also necessary to standardize and harmonize the use of specific concepts and terminology. To achieve this goal, SDMX creates and maintains guidelines for cross-domain concepts, terminology, and structural definitions. There are three major parts to this effort.

2.6.1 Cross-Domain Concepts

The SDMX Cross-Domain Concepts is a content guideline concerning concepts which are used across statistical domains. This list is expected to grow and to be subject to revision as SDMX is used in a growing number of domains. The use of the SDMX Cross-Domain Concepts, where appropriate, provides a framework to further promote interoperability among organisations using the technical standards presented here. The harmonization of statistical concepts includes not only the definitions of the concepts, and their names, but also, where appropriate, their representation with standard code lists, and the role they play within data structure definitions and metadata structure definitions.

The intent of this guideline is two-fold: to provide a core set of concepts which can be used to structure statistical data and metadata, to promote interoperability between systems (“structural metadata”, as described above); and to promote the exchange of metadata more widely, with a set of harmonized concept names and definitions for other types of metadata (“reference metadata”, as defined above.)

2.6.2 Metadata Common Vocabulary

The Metadata Common Vocabulary is an SDMX guideline which provides definition of terms to be used for the comparison and mapping of terminology found in data structure definitions and in other aspects of statistical metadata management. Essentially, it provides ISO-compliant definitions for a wide range of statistical terms, which may be used directly, or against which other terminology systems may be mapped. This set of terms is inclusive of the terminology used within the SDMX Technical Standards.

The MCV provides definitions for terms on which the SDMX Cross-Domain Metadata Concepts work is built.

2.6.3 Statistical Subject-Matter Domains

The Statistical Subject-Matter Domains is a listing of the breadth of statistical information for the purposes of organizing widespread statistical exchange and categorization. It acts as a standard scheme against which the categorization schemes of various counterparties can be mapped, to facilitate interoperable data and metadata exchange. It serves another useful purpose, however, which is to allow an organization of corresponding “domain groups”, each of which could define standard data structure definitions, concepts, etc. within their domains. Such groups already exist within the international community. SDMX would use the Statistical Subject-Matter Domains list to facilitate the efforts of these groups to develop the kinds of content standards which could support the interoperation of SDMX-conformant technical systems within and across statistical domains. The organisation of the content of such schemes is supported in SDMX as a Category Scheme.

SDMX Statistical Subject-Matter Domains will be listed and maintained by the SDMX Initiative and will be subject to adjustment.

2.6.4 SDMX Concept Roles

These guidelines define the standard set of SDMX Concept Roles and their use. This set of standard SDMX Concepts are implemented as a cross-domain Concept Scheme that defines the set of concept roles and gives examples on concept role implementation in SDMX 2.0, 2.1 and 3.0. A concept role gives a particular context to a concept for easy and systematic interpretation by machine processing and visualization tools.

For example, the concepts `REPORTING_AREA` and `COUNTERPART_AREA` are different concepts but they are both geographical characteristics, therefore they can be associated with the same concept role ID: `GEO`. This allows visualization systems to interpret these concepts as geographical data in order to generate maps. The implementation of concept roles is different in versions 2.0 and 2.1/3.0 of the SDMX technical standard. Specifically for SDMX 3.0, this set of roles is considered a normative list that must be interpreted in the same way by all organisations. Additional roles may be provided via the standard roles’ mechanism in SDMX 3.0,

Copyright © SDMx TWG 2020

i.e., via Concept Schemes; the semantics of these roles have to be agreed bilateraly in data exchanges. The Concept Roles are available as an SDMX Concept Scheme on the SDMX Global Registry.

Copyright © SDMx TWG 2020

2.7 Validation and Transformation Language

For many years the SDMX initiative has been fostering and supporting the development of a standard calculation language, called Validation and Transformation Language (VTL). A blueprint for defining calculations was already described in the original SDMX 2.1 specifications (package 13 of the Information Model - “Transformations and Expressions”). It was just a basic framework that required further developments to became operational in order to achieve a calculation language able to manipulate SDMX artefacts.

These developments started in late 2012 and were put in charge of the Validation and Transformation Language Task Force (VTL TF), which included members of the SDMX Technical Working Group (TWG) and Statistical Working Group (SWG), besides experts coming from the DDI and GSIM communities. The intent was to define a standard language to be implemented in SDMX and applicable also to GSIM and DDI. This brought to the publication of the VTL 1.0 in 2015. Then new requirements came from a number of proofs of concepts and tests of VTL 1.0 made by several organisations and triggered a large improvement of the language. A new provisional version, the VTL 1.1, was released in public consultation in 2017. The high number of comments received triggered another phase of intensive work, with the main goal of achieving a more robust and forward compatible version. Finally, the VTL 2.0 was published between April and July 2018 (see the SDMX website).

The implementation of the VTL 2.0 in SDMX started in late 2018 and was published as an incremental revision to the SDMX 2.1 standards in July 2020. It allows users to write VTL 2.0 programs for validating and transforming SDMX data, to store these programs in a SDMX metadata registry and to exchange them through SDMX messages, also together the definition of the data structures of the involved data.

The Transformations and Expressions package for modelling VTL programs in the SDMX information model is explained in Section 2 of the Technical Specifications with further detailed usage and implementation guidance given in Section 6.

Copyright © SDMx TWG 2020

3. Information Model

3.1 Introduction

This document is not normative but provides a detailed view of the information model on which the normative SDMX specifications are based. Those new to the UML notation or to the concept of Data Structure Definitions may wish to read the appendixes in this document as an introductory exercise.

3.1.1 Related Documents

This document is one of two documents concerned with the SDMX Information Model. The complete set of documents is:

- SDMX SECTION 02 INFORMATION MODEL: UML CONCEPTUAL DESIGN (this document): This document comprises the complete definition of the information model, with the exception of the registry interfaces. It is intended for technicians wishing to understand the complete scope of the SDMX technical standards in a syntax neutral form.
- SDMX SECTION 05 REGISTRY SPECIFICATION: LOGICAL INTERFACES: This document provides the logical specification for the registry interfaces, including subscription/notification, registration/submission of data and metadata, and querying.

3.1.2 Modelling Technique and Diagrammatic Notes

The modelling technique used for the SDMX Information Model (SDMX-IM) is the Unified Modelling Language (UML). An overview of the constructs of UML that are used in the SDMX-IM can be found in the Appendix “A Short Guide to UML in the SDMX Information Model”

UML diagramming allows a class to be shown with or without the compartments for one or both of attributes and operations (sometimes called methods). In this document the operations compartment is not shown as there are no operations.

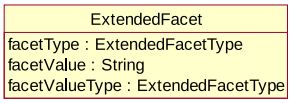


Figure 1. Class with operations suppressed

In some diagrams for some classes the attribute compartment is suppressed even though there may be some attributes. This is deliberate and is done to aid clarity of the diagram. The method used is:

- The attributes will always be present on the class diagram where the class is defined and its attributes and associations are defined.
- On other diagrams, such as inheritance diagrams, the attributes may be suppressed from the class for clarity.



Figure 2. Class with attributes also suppressed

Note that, in any case, attributes inherited from a super class are not shown in the sub class.

Copyright © SDMx TWG 2020

The following table structure is used in the definition of the classes, attributes, and associations.

Class	Feature	Description
`ClassName`		
	`attributeName`	
	`associationName`	
	`+roleName`	

The content in the “Feature” column comprises or explains one of the following structural features of the class:

- Whether it is an abstract class. Abstract classes are shown in *italic Courier* font.
- The superclass this class inherits from, if any.
- The sub classes of this class, if any.
- Attribute – the `attributeName` is shown in Courier font.
- Association – the `associationName` is shown in Courier font. If the association is derived from the association between super classes, then the format is `/associationName`.
- Role – the `+roleName` is shown in Courier font.

The Description column provides a short definition or explanation of the Class or Feature. UML class names may be used in the description and if so, they are presented in normal font with spaces between words. For example, the class `ConceptScheme` will be written as Concept Scheme.

3.1.3 Overall Functionality

Information Model Packages

The SDMX Information Model (SDMX-IM) is a conceptual metamodel from which syntax specific implementations are developed. The model is constructed as a set of functional packages which assist in the understanding, re-use and maintenance of the model.

In addition to this, in order to aid understanding each package can be considered to be in one of three conceptual layers:

- the SDMX Base layer comprises fundamental building blocks which are used by the Structural Definitions layer and the Reporting and Dissemination layer
- the Structural Definitions layer comprises the definition of the structural artefacts needed to support data and metadata reporting and dissemination
- the Reporting and Dissemination layer comprises the definition of the data and metadata containers used for reporting and dissemination

In reality the layers have no implicit or explicit structural function as any package can make use of any construct in another package.

Version 1.0

In version 1.0 the metamodel supported the requirements for:

- Data Structure Definition including (domain) category scheme, (metadata) concept scheme, and code list
- Data and related metadata reporting and dissemination

The SDMX-IM comprises a number of packages. These packages act as convenient compartments for the various sub models in the SDMX-IM. The diagram below shows the sub models of the SDMX-IM that were included in the version 1.0 specification.

Copyright © SDMx TWG 2020



Figure 3. SDMX Information Model Version 1.0 package structure

Version 2.0/2.1

The version 2.0/2.1 model extends the functionality of version 1.0. principally in the area of metadata, but also in various ways to define structures to support data analysis by systems with knowledge of cube type structures such as OLAP[1] systems. The following major constructs have been added at version 2.0/2.1

- Metadata structure definition
- Metadata set
- Hierarchical Codelist
- Data and Metadata Provisioning
- Process
- Mapping
- Constraints
- Constructs supporting the Registry

Furthermore, the term Data Structure Definition replaces the term Key Family: as both of these terms are used in various communities, they are synonymous. The term Data Structure Definition is used in the model and this document.

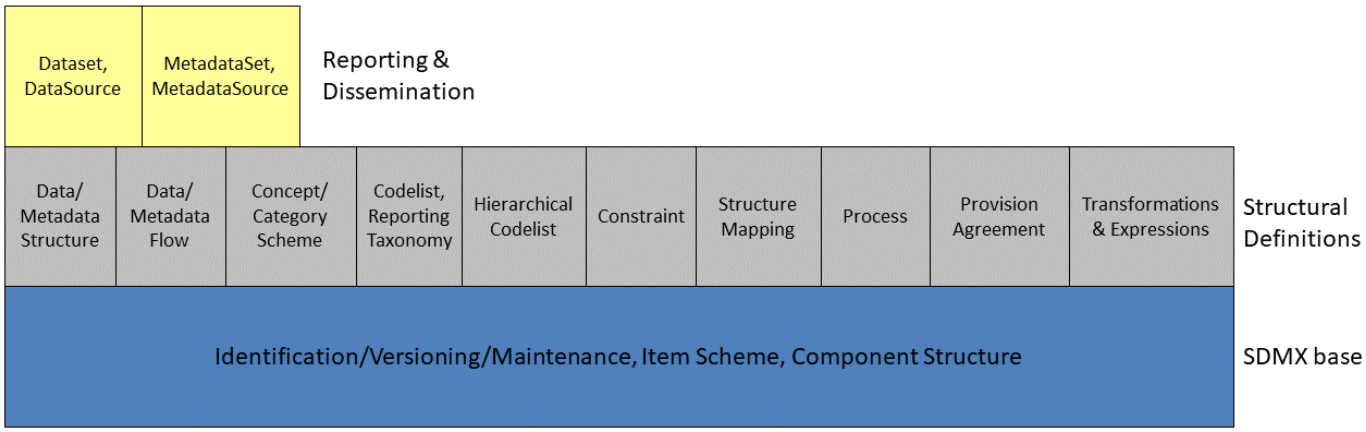


Figure 4. SDMX Information Model Version 2.0/2.1 package structure

Copyright © SDMx TWG 2020

Additional constructs that are specific to a registry-based scenario can be found in the Specification of Registry Interfaces. For information these are shown on the diagram below and comprise:

- Subscription and Notification
- Registration
- Discovery

Note that the data and metadata required for registry functions are not confined to the registry, and the registry also makes use of the other packages in the Information Model.

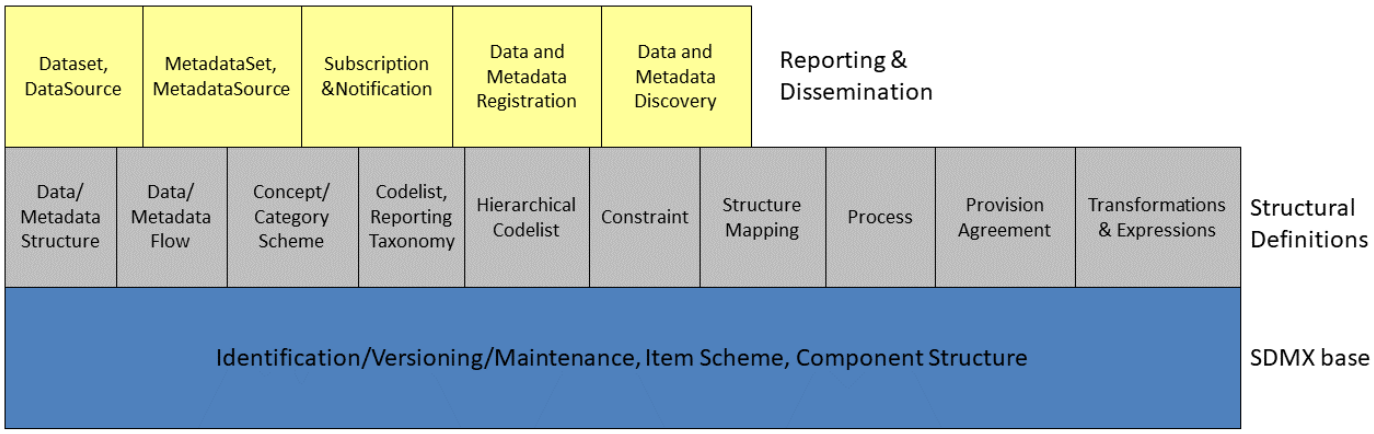


Figure 5. SDMX Information Model Version 2.0/2.1 package structure including the registry

Version 3.0

The version 3.0 model introduces changes in the way reference metadata are handled. In addition, it includes a few more artefacts. Finally, a few abstractions have been added, as shown in section “Basic Inheritance” in “Figure 11: Basic Inheritance from the Base Structures”.

The IM packages are largely the same.

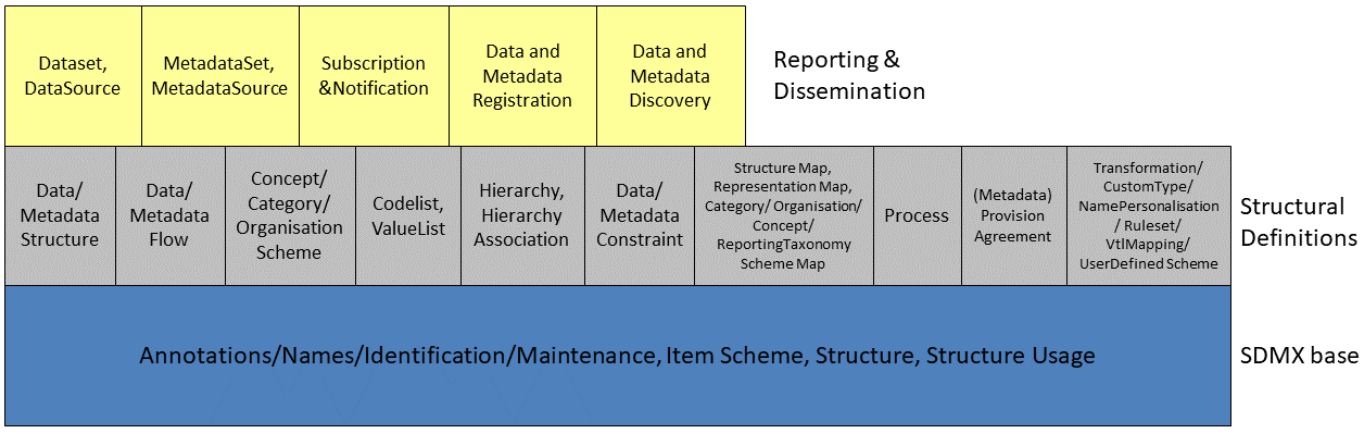


Figure 6. SDMX Information Model version 3.0 package structure

1. OLAP: On line analytical processing ↩

Copyright © SDMx TWG 2020

3.2 Change History

3.2.1 Version 1.0 – initial release September 2004

3.2.2 Version 2.0 – release November 2005

Major functional enhancements by addition of new packages:

- Metadata Structure Definition
- Metadata Set
- Hierarchical Code Scheme
- Data and Metadata Provisioning
- Structure Set and Mappings
- Transformations and Expressions
- Process and Transitions

Re-engineering of some SDMX Base structures to give more functionality:

- Item Scheme and Item can have properties – this gives support for complex hierarchical code schemes (where the property can be used to sequence codes in scheme), and Item Scheme mapping tables (where the property can give additional information about the map between the two schemes and the between two Items)
- revised Organisation pattern to support maintained schemes of organisations, such as a data provider
- modified Component Structure pattern to support identification of roles played by components and the attachment of attributes
- change to inheritance to enable more artefacts to be identifiable and versionable

Introduction of new types of Item Scheme:

- Object Type Scheme to specify object types in support of the Metadata Structure Definition (principally the object types (classes) in this Information Model)
- Type Scheme to specify types other than object type
- A generic Item Scheme Association to specify the association between Items in two or more Item Schemes, where such associations cannot be described in the Structure Set and Transformation.

The Data Structure Definition is introduced as a synonym for Key Family though the term Key Family is retained and used in this specification.

Modification to Data Structure Definition (DSD) to

- align the cross sectional structures with the functionality of the schema
- support Data Structure Definition extension (i.e. to derive and extend a Data Structure Definition from another Data Structure Definition), thus supporting the definition of a related “set” of key families
- distinguish between data attributes (which are described in a Data Structure Definition) from metadata attributes (which are described in a metadata structure definition)
- attach data attributes to specific identifiable artefacts (formally this was supported by attachable artefact)
- Domain Category Scheme re-named Category Scheme to better reflect the multiple usage of this type of scheme (e.g. subject matter domain, reporting taxonomy).
- Concept Scheme enhanced to allow specification of the representation of the Concept. This specification is the default (or core) representation and can be overridden by a construct that uses it (such as a Dimension in a Data Structure Definition).
- Revision of cross sectional data set to reflect the functionality of the version 1.0 schema.
- Revision of Actors and Use Cases to reflect better the functionality supported.

Copyright © SDMx TWG 2020

3.2.3 Version 2.1 – release April 2011

The purpose of this revision is threefold:

- To introduce requested changes to functionality
- To align the model and syntax implementations more closely (note, however, that the model remains syntax neutral)
- To correct errors in version 2.0

SDMX Base

BASIC INHERITANCE AND PATTERNS

1. The following attributes are added to Maintainable:
   a. `isExternalReference`
   b. structure URL
   c. `serviceURL`
2. Added Nameable Artefact and moved the Name and Description associations from Identifiable Artefact to Nameable Artefact. This allows an artefact to be identified (with id and urn) without the need to specify a Name.
3. Removed any inheritance from Versionable Artefact with the exception of Maintainable Artefact – this means that only Maintainable objects can be versioned, and objects contained in a maintainable object cannot be independently versioned.
4. Renamed `MaintenanceAgency` to Agency 0 this is its name in the schema and the URN.
5. Removed abstract class Association as a subclass of Item (as these association types are not maintained in Item Schemes). Specific associations are modelled explicitly (e.g. `Categorisation`, `ItemScheme`, `Item`).
6. Added `ActionType` to data types.
7. Removed Coded Artefact and Uncoded Artefact and all subclasses (e.g. Coded Data Attribute and Uncoded Data Attribute) as the “Representation” is more complex than just a distinction between coded and uncoded.
8. Added Representation to the Component. Removed association to Type.
9. Removed concept role association (to Item) as roles are identified by a relationship to a Concept.
10. Removed abstract class Attribute as both Data Attribute and Metadata Attribute have different properties. Data Attribute and Metadata Attribute inherit directly from Component.
11. `isPartial` attribute added to Item Scheme to support partial Item Schemes (e.g. partial Code list).

REPRESENTATION

1. Removed interval and enumeration from Facet.
2. added `facetValueType` to Facet.
3. Re-named `DataType` to `facetValueType`.
4. Added `observationalTimePeriod`, `inclusiveValueRange` and `exclusiveValueRange` to `facetValueType`.
5. Added `ExtendedFacetType` as a sub class of `FacetType`. This includes Xhtml as a facet type to support this as an allowed representation for a Metadata Attribute

ORGANISATIONS

1. Organisation Role is removed and replaced with specific Organisation Schemes of Agency, Data Provider, Data Consumer, Organisation Unit.

Copyright © SDMx TWG 2020

MAPPING (STRUCTURE MAPS)

Updated Item Scheme Association as follows:

1. Renamed to Item Scheme Map to reflect better the sub classes and relate better to the naming in the schema.
2. Removed inheritance of Item Scheme Map from Item Scheme, and inherited directly from Nameable Artefact.
3. Item Association inherits from Identifiable Artefact.
4. Removed Property from the model as this is not supported in the schema.
5. Removed association type between Item Scheme Map and Item, and Association and Item.
6. Removed Association from the model.
7. Made Item Association a sub class of Identifiable, was a sub class Item.
8. Removed association to Property from both Item Scheme Map and Item.
9. Added attribute alias to both Item Scheme Association and Item Association.
10. Made Item Scheme Map and Item Association abstract.
11. Added sub-classes to Item Scheme Map – there is a subclass for each type of Item Scheme Association (e.g. Code list Map).
12. Added mapping between Reporting Taxonomy as this is an Item Scheme and can be mapped in the same way as other Item Schemes.
13. Added Hybrid Code list Map and Hybrid Code Map to support code mappings between a Code list and a Hierarchical Code list.

MAPPING: STRUCTURE MAP

1. This is a new diagram. Essentially removed inherited /hierarchy association between the various maps, as these no longer inherit from Item, and replaced the associations to the abstract Maintainable and Versionable Artefact classes with the actual concrete classes.
2. Removed associations between Code list Map, Category Scheme Map, and Concept Scheme Map and made this association to Item Scheme Map.
3. Removed hierarchy of Structure Map.

CONCEPT

1. Added association to Representation.

DATA STRUCTURE DEFINITION

1. Added Measure Dimension to support structure-specific renderings of the DSD. The Measure Dimension is associated to a Concept Scheme that specifies the individual measures that are valid.
2. The three types of “Dimension”, - Dimension, Measure Dimension, Time Dimension – have a super class – Dimension Component
3. Added association to a Concept that defines the role that the component (Dimension, Data Attribute, Measure Dimension) plays in the DSD. This replaces the Boolean attributes on the components.
4. Added Primary Measure and removed this as role of Measure.
5. Deleted the derived Data Structure Definition association from Data Structure Definition to itself as this is not supported directly in DSD.
6. Deleted attribute `GroupKeyDescriptor.isAttachmentConstraint` and replaced with an association to an Attachment Constraint.
7. Replaced association from Data Attribute to Attachable Artefact with association to Attribute Relationship.
8. Added a set of classes to support Attribute Relationship.
9. Renamed `KeyDescriptor` to `DimensionDescriptor` to better reflect its purpose.
10. Renamed `GroupKeyDescriptor` to `GroupDimensionDescriptor` to better reflect its purpose.

CODE LIST

1. `CodeList` classname changed to `Codelist`.
2. Removed `codevalueLength` from `Codelist` as this is supported by Facet.
3. Removed `hierarchyView` association between Code and Hierarchy as this association is not implemented.

Copyright © SDMx TWG 2020

METADATA STRUCTURE DEFINITION(MSD)

1. Full Target Identifier, Partial Target Identifier, and Identifier Component are replaced by Metadata Target and Target Object. Essentially this eliminates one level of specification and reference in the MSD, and so makes the MSD more intuitive and easier to specify and to understand.
2. Re-named Identifiable Object Type to Identifiable Object Target and moved to the MSD package.
3. Added sub classes to Target Object as these are the actual types of object to which metadata can be attached. These are Identifiable Object Target (allows reporting of metadata to any identifiable object), Key Descriptor Values Target (allows reporting of metadata for a data series key, Data Set Target (allows reporting of metadata to a data set), and Reporting Period Target (allows the metadata set to specify a reporting period).
4. Allowed Target Object can have any type of Representation, this was restricted in version 2.0 to an enumerated representation in the model (but not in the schemas).
5. Removed Object Type Scheme (as users cannot maintain their own list of object types), and replaced with an enumeration of Identifiable Objects.
6. Removed association between Metadata Attribute and Identifiable Artefact and replaced this with an association between Report Structure and Metadata Target, and allowed one Report Structure to reference more than on Metadata Target. This allowing a single Report Structure to be defined for many object types.
7. Added the ability to specify that a Metadata Attribute can be repeated in a Metadata Set and that a Metadata Attribute can be specified as “presentational” meaning that it is present for structural and presentational purposes, and will not have content in a Metadata Set.
8. The Representation of a Metadata Attribute uses Extended Facet (to support Xhtml).

METADATA SET

1. Added link to Data Provider - 0..1 but note that for metadata set registration this will be 1.
2. Removed Attribute Property as the underlying Property class has been removed.
3. One Metadata Set is restricted to reporting metadata for a single MSD.
4. The Metadata Report classes are re-structured and re-named to be consistent with the renaming and restructuring of the MSD.
5. Metadata Attribute Value is renamed Reported Attribute to be consistent with the schemas.
6. Deleted XML attribute and Contact Details from the inheritance diagram.

CATEGORY SCHEME

1. Added Categorisation. Category no longer has a direct association to Dataflow and Metadataflow.
2. Changed Reporting Taxonomy inheritance from Category Scheme to Maintainable Artefact.
3. Added Reporting Category and associated this to Structure Usage.

DATA SET

1. Removed the association to Provision Agreement from the diagram.
2. Added association to Data Structure Definition. This association was implied via the dataflow but this is optional in the implementation whereas the association to the Data Structure Definition is mandatory.
3. Added attributes to Data Set.
4. There is a single, unified, model of the Data Set which supports four types of data set:
    - Generic Data Set – for reporting any type of data series, including time series and what is sometimes known as “cross sectional data”. In this data set, the value of any one dimension (including the Time Dimension) can be reported with the observation (this must be for the same dimension for the entire data set)
    - Structure-specific Data Set – for reporting a data series that is specific to a DSD
    - Generic Time Series Data Set – this is identical to the Generic Data Set except it must contain only time series, which means that a value for the Time Dimension is reported with the Observation
    - Structure-specific Time Series Data Set - this is identical to the Structure-specific Data Set except it must contain only time series, which means that a value for the Time Dimension is reported with the Observation.

Copyright © SDMx TWG 2020

5. Removed Data Set as a sub class of Identifiable – but note that Data Set has a `setId` attribute.
6. Added coded and uncoded variants of Key Value, Observation, and Attribute Value in order to show the relationship between the coded values in the data set and the Codelist in the Data Structure Definition.
7. Made Key Value abstract with sub classes for coded, uncoded, measure (`MeasureKeyValue`) ads time (`TimeKeyValue`) The Measure Key Value is associated to a Concept as it must take its identify from a Concept.

XSDATASET

1. This is removed and replaced with the single, unified data set model.

CONSTRAINT

1. Constraint is made Maintainable (was Identifiable).
2. Added artefacts that better support and distinguish (from data) the constraints for metadata.
3. Added Constraint Role to specify the purpose of the Constraint. The values are allowable content (for validation of sub set code code lists), and actual content (to specify the content of a data or metadata source).

PROCESS

1. Removed inheritance from Item Scheme and Item: Process inherits directly from Maintainable and Process Step from Identifiable.
2. Removed specialisation association between Transition and Association.
3. Removed Transition Scheme - transitions are explicitly specified and not maintained as Items in a Item Scheme.
4. Removed Expression and replaced with Computation.
5. Transition is associated to Process Step and not Process itself. Therefore the source association to Process Step is removed.
6. Removed Expressions as these are not implemented in the schemas. But note that the Transformations and Expressions model is retained, though it is not implemented in the schemas.

HIERARCHICAL CODELIST

1. Renamed `HierarchicalCodeList` to `HierarchicalCodelist` .
2. This is re-modelled to reflect more accurately the way this is implemented: this is as an actual hierarchy rather than a set of relational associations from which the hierarchy can be derived.
3. Code Association is re-named Hierarchical Code and the association type association to Code is removed (as these association types are not maintained in an Item Scheme).
4. Hierarchical Code is made an aggregate of Hierarchy, and not of Hierarchical Codelist.
5. Removed root node in the Hierarchy – there can be many top-level codes in Hierarchical Code.
6. Added reference association between Hierarchical Code and Level to indicate the Level if the Hierarchy is a level based hierarchy.

PROVISIONING AND REGISTRATION

1. Data Provider and Provision Agreement have an association to Datasource (was Query Datasource), as the association is to any of Query Datasource and Simple Datasource.
2. Provision Agreement is made Maintainable and indexing attributes moved to Registration
3. Registration has a registry assigned Id and indexing attributes.

3.2.4 Version 2.1 (Revision 2.0) – release June 2020

The package 13, previously named “Expressions and Transformations” is completely reformulated, renamed as “Validation and Transformation Language” and implemented also in the other Sections of the SDMX standards for actual use.

Copyright © SDMx TWG 2020

3.2.5 Version 3.0 – release September 2021

New Maintainable Artefacts

- Structure Map
- Representation Map
- Organisation Scheme Map
- Concept Scheme Map
- Category Scheme Map
- Reporting Taxonomy Map
- Value List
- Hierarchy
- Hierarchy Association
- Metadata Constraint
- Data Constraint
- Metadata Provision Agreement
- Metadata Provider Scheme
- Metadataset

New Identifiable Artefacts

- `GeoFeatureSetCode`
- `GeoGridCode`
- Metadata Provider

Removed Maintainable Artefacts

- Structure Set – replaced by Structure Map and the four item scheme maps
- Hierarchical Codelist – replaced by Hierarchy and Hierarchy Association
- Constraint – replaced by Data Constraint and Metadata Constraint

Changed Maintainable Artefacts

- Data Structure Definition – support for microdatasets and reference metadata linked to data
- Metadataflow – simplifies exchange of reference metadata, in particular those linked to structures
- Metadata Structure Definition – simplified model for reference metadata
- Codelist – support for codelist extension and geospatial specialised codelists (`GeographicCodelist` , `GeoGridCodelist`)
- VTL Mapping Scheme – VTL Concept Mapping Scheme removed to align the VTL / SDMX interface with the 3.0 model

New Component Representation Types

- `GeospatialInformation` – a string type where the value is an expression defining a set of geographical features using a purpose-designed syntax

Copyright © SDMx TWG 2020

3.3 Actors and Use Cases

3.3.1 Introduction

In order to develop the data models it is necessary to understand the functions to be supported resulting from the requirements definition. These are defined in a use case model. The use case model comprises actors and use cases and these are defined below.

Actor: “An actor defines a coherent set of roles that users of the system can play when interacting with it. An actor instance can be played by either an individual or an external system”

Use case: “A use case defines a set of use-case instances, where each instance is a sequence of actions a system performs that yields an observable result of value to a particular actor”

The overall intent of the model is to support data and metadata reporting, dissemination, and exchange in the field of aggregated statistical data and related metadata. In order to achieve this, the model needs to support three fundamental aspects of this process:

- Maintenance of structural and provisioning definitions
- Data and reference metadata publishing (reporting), and consuming (using)
- Access to data, reference metadata, and structural and provisioning definitions

This document covers the first two aspects, whilst the document on the Registry logical model covers the last aspect.

3.3.2 Use Case Diagrams

Maintenance of Structural and Provisioning Definitions

USE CASES

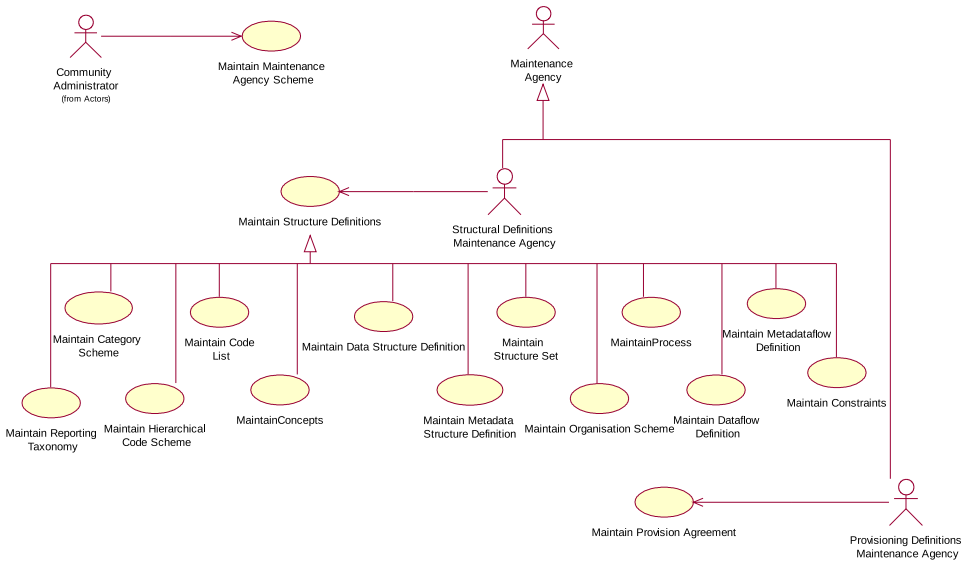


Figure 7. Use cases for maintaining data and metadata structural and provisioning definitions

EXPLANATION OF THE DIAGRAM

In order for applications to publish and consume data and reference metadata it is necessary for the structure and permitted content of the data and reference metadata to be defined and made available to the applications, as well as definitions that support the actual process of publishing and consuming. This is the responsibility of a Maintenance Agency.

Copyright © SDMx TWG 2020

All maintained artefacts are maintained by a Maintenance Agency. For convenience the Maintenance Agency actor is sub divided into two actor roles:

- maintaining structural definitions
- maintaining provisioning definitions

Whilst both these functions may be carried out by the same person, or at least by the same maintaining organization, the purpose of the definitions is different and so the roles have been differentiated: structural definitions define the format and permitted content of data and reference metadata when reported or disseminated, whilst provisioning definitions support the process of reporting and dissemination (who reports what to whom, and when).

In a community-based scenario where at least the structural definitions may be shared, it is important that the scheme of maintenance agencies is maintained by a responsible organization (called here the Community Administrator), as it is important that the Id of the Maintenance Agency is unique.

Copyright © SDMx TWG 2020

DEFINITIONS

Copyright © SDMx TWG 2020

Actor	Use Case	Description
 Community Administrator		Responsible organisation that administers structural definitions common to the community as a whole.
	 Maintain Maintenance Agency Scheme	Creation and maintenance of the top-level scheme of maintenance agencies for the Community.
 Maintenance Agency		Responsible agency for maintaining structural artefacts such as code lists, concept schemes, Data Structure Definition structural definitions, metadata structure definitions, data and metadata provisioning artefacts such as provision agreement, and sub-maintenance agencies. Sub roles are: Structural Definitions Maintenance Agency, Provisioning Definitions Maintenance Agency
 Structural Definitions Maintenance Agency		Responsible for maintaining structural definitions.
	 Maintain Structure Definitions	The maintenance of structural definitions. This use case has subclass use cases for each of the structural artefacts that are maintained.
	 Maintain Code List ,  MaintainConcepts ,  Maintain Category Scheme ,  Maintain Data Structure Definition ,  Maintain Metadata Structure Definition ,  Maintain Hierarchical Code Scheme ,  Maintain Reporting Taxonomy ,  Maintain Organisation Scheme ,	Creation and maintenance of the Data Structure Definition, Metadata Structure Definition, and the supporting artefacts that they use, such as code list and concepts. This includes Agency, Data Provider, Data Consumer, and Organisation Unit Scheme.

Copyright © SDMx TWG 2020

Actor	Use Case	Description
		
		Responsible for maintaining data and metadata provisioning definitions.
		The maintenance of provisioning definitions.

Figure 8. Table of Actors and Use Cases for Maintenance of Structural and Provisioning Definitions

Publishing and Using Data and Reference Metadata

USE CASES

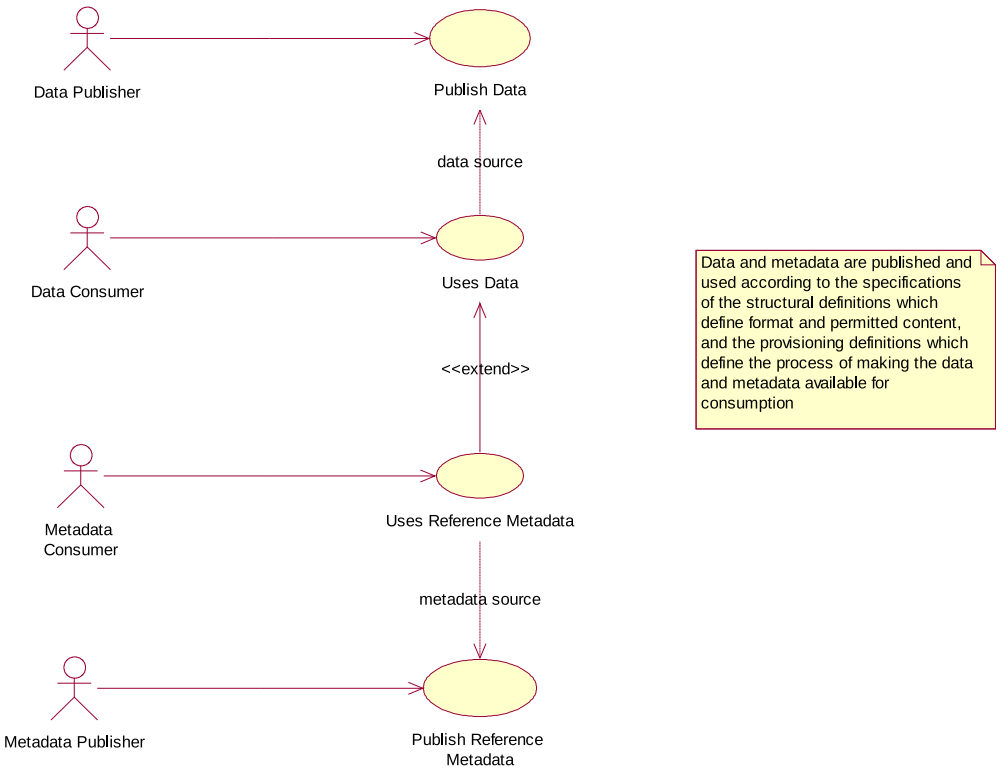


Figure 9. Actors and use cases for data and metadata publishing and consuming

Copyright © SDMx TWG 2020

EXPLANATION OF THE DIAGRAM

Note that in this diagram “publishing” data and reference metadata is deemed to be the same as “reporting” data and reference metadata. In some cases the act of making the data available fulfils both functions. Aggregated data is published and in order for the Data Publisher to do this and in order for consuming applications to process the data and reference metadata its structure must be known. Furthermore, consuming applications may also require access to reference metadata in order to present this to the Data Consumer so that the data is better understood. As with the data, the reference metadata also needs to be formatted in accordance with a maintained structure. The Data Consumer and Metadata Consumer cannot use the data or reference metadata unless it is “published” and so there is a “data source” or “metadata source” dependency between the “uses” and “publish” use cases.

In any data and reference metadata publishing and consuming scenario both the publishing and the consuming applications will need access to maintained Provisioning Definitions. These definitions may be as simple as who provides what data and reference metadata to whom, and when, or it can be more complex with constraints on the data and metadata that can be provided by a particular publisher, and, in a data sharing scenario where data and metadata are “pulled” from data sources, details of the source.

DEFINITIONS

Actor	Use Case	Description
 Data Publisher		Responsible for publishing data according to a specified Data Structure Definition (data structure) definition, and relevant provisioning definitions.
	 Publish Data	Publish a data set. This could mean a physical data set or it could mean to make the data available for access at a data source such as a database that can process a query.
 Data Consumer		The user of the data. It may be a human consumer accessing via a user interface, or it could be an application such as a statistical production system.
	 Uses Data	Use data that is formatted according to the structural definitions and made available according to the provisioning definitions. Data are often linked to metadata that may reside in a different location and be published and maintained independently.
 Metadata Publisher		Responsible for publishing reference metadata according to a specified metadata structure definition, and relevant provisioning definitions.
	 Publish Reference Metadata	Publish a reference metadata set. This could mean a physical metadata set or it could mean to make the reference metadata available for access at a metadata source such as a metadata repository that can process a query.
 Metadata Consumer		The user of the reference metadata. It may be a human consumer accessing via a user interface, or it could be an application such as a statistical production or dissemination system.
	 Uses Reference Metadata	Use reference metadata that is formatted according to the structural definitions and made available according to the provisioning definitions.

Copyright © SDMx TWG 2020

3.4 SDMX Base Package

3.4.1 Introduction

The constructs in the SDMX Base package comprise the fundamental building blocks that support many of the other structures in the model. For this reason, many of the classes in this package are abstract (i.e., only derived sub-classes can exist in an implementation).

The motivation for establishing the SDMX Base package is as follows:

- it is accepted “Best Practise” to identify fundamental archetypes occurring in a model
- identification of commonly found structures or “patterns” leads to easier understanding
- identification of patterns encourages re-use

Each of the class diagrams in this section views classes from the SDMX Base package from a different perspective. There are detailed views of specific patterns, plus overviews showing inheritance between classes, and relationships amongst classes.

Copyright © SDMx TWG 2020

3.4.2 Base Structures - Identification, Versioning, and Maintenance

Class Diagram

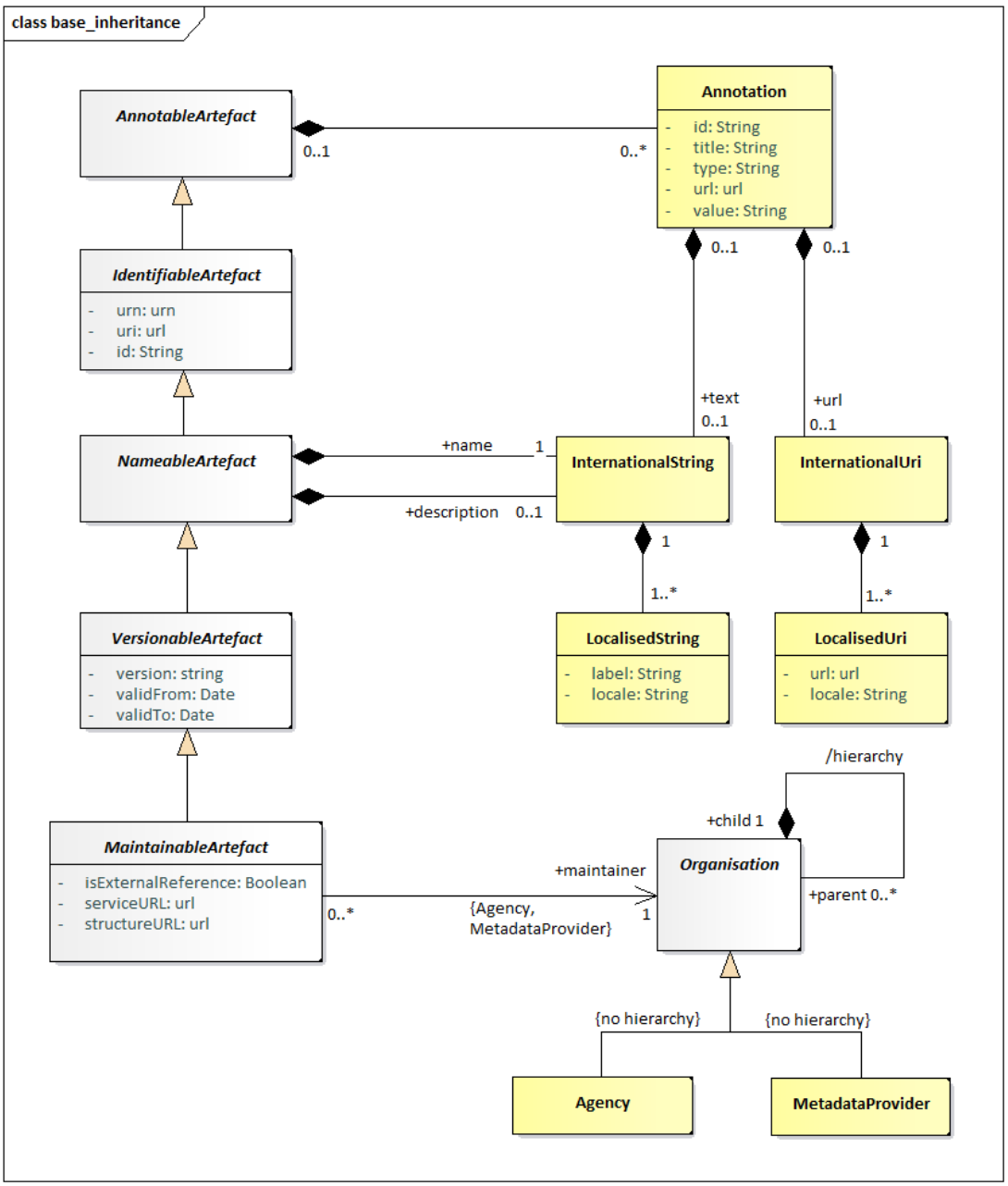


Figure 10. SDMX Identification, Maintenance and Versioning

Explanation of the Diagram

NARRATIVE

This group of classes forms the nucleus of the administration facets of SDMX objects. They provide features which are reusable by derived classes to support horizontal functionality such as identity, versioning etc.

All classes derived from the abstract class `AnnotableArtefact` may have Annotations (or notes): this supports the need to add notes to all SDMX-ML elements. The Annotation is used to convey extra information to describe any SDMX construct. This information may be in the form of a URL reference and/or a multilingual text (represented by the association to `InternationalString`).

Copyright © SDMx TWG 2020

The `IdentifiableArtefact` is an abstract class that comprises the basic attributes needed for identification. Concrete classes based on `IdentifiableArtefact` all inherit the ability to be uniquely identified.

The `NamableArtefact` is an abstract class that inherits from `IdentifiableArtefact` and in addition the `+description` and `+name` roles support multilingual descriptions and names for all objects based on `NameableArtefact`. The `InternationalString` supports the representation of a description in multiple locales (locale is similar to language but includes geographic variations such as Canadian French, US English etc.). The `LocalisedString` supports the representation of a description in one locale.

`VersionableArtefact` is an abstract class which inherits from `NameableArtefact` and adds versioning ability to all classes derived from it, as explained in the SDMX versioning rules in the Section "Versioning" of the SDMX Technical Notes.

`MaintainableArtefact` further adds the ability for derived classes to be maintained via its association to an `Organisation`, and adds locational information (i.e., from where the object can be retrieved).

The inheritance chain from `AnnotableArtefact` through to `MaintainableArtefact` allows SDMX classes to inherit the features they need, from simple annotation, through identity, naming, to versioning and maintenance.

Copyright © SDMx TWG 2020

DEFINITIONS

Copyright © SDMx TWG 2020

Class	Feature	Description
`AnnotableArtefact`	Base inheritance sub classes are: `IdentifiableArtefact`	Objects of classes derived from this can have attached annotations.
`Annotation`		Additional descriptive information attached to an object.
	`id`	Identifier for the Annotation. It can be used to disambiguate one Annotation from another where there are several Annotations for the same annotated object.
	`title`	A title used to identify an annotation.
	`type`	Specifies how the annotation is to be processed.
	`url`	A link to external descriptive text.
	`value`	A non-localised version of the Annotation content.
	`+url`	An International URI provides a set of links that are language specific, via this role.
	`+text`	An International String provides the multilingual text content of the annotation via this role.
`InternationalUri`		The International Uri is a collection of Localised URIs and supports linking to external descriptions in multiple locales.
`LocalisedUri`		The Localised URI supports the link to an external description in one locale (locale is similar to language but includes geographic variations such as Canadian French, US English etc.).
`IdentifiableArtefact`	Superclass is `AnnotableArtefact`. Base inheritance sub classes are: `NameableArtefact`	Provides identity to all derived classes. It also provides annotations to derived classes because it is a subclass of `AnnotableArtefact`.
	`id`	The unique identifier of the object.
	`uri`	Universal resource identifier that may or may not be resolvable.
	`urn`	Universal resource name – this is for use in registries: all registered objects have a urn.
`NameableArtefact`	Superclass is `IdentifiableArtefact`. Base inheritance sub classes are: `VersionableArtefact`	Provides a Name and Description to all derived classes in addition to identification and annotations.
	`+description`	A multi-lingual description is provided by this role via the International String class.
	`+name`	A multi-lingual name is provided by this role via the International String class.
`InternationalString`		The International String is a collection of Localised Strings and supports the representation of text in multiple locales.
`LocalisedString`		The Localised String supports the representation of text in one locale (locale is similar to language but

Copyright © SDMx TWG 2020

Class	Feature	Description
		includes geographic variations such as Canadian French, US English etc.).
	`label`	Label of the string.
	`locale`	The geographic locale of the string e.g French, Canadian French.
`VersionableArtefact`	Superclass is `NameableArtefact`. Base inheritance sub classes are: `MaintainableArtefact`	Provides versioning information for all derived objects.
	`version`	A version string following SDMX versioning rules.
	`validFrom`	Date from which the version is valid.
	`validTo`	Date from which version is superseded.
`MaintainableArtefact`	Inherits from `VersionableArtefact`	An abstract class to group together primary structural metadata artefacts that are maintained by an Agency.
	`isExternalReference`	If set to “true” it indicates that the content of the object is held externally.
	`structureURL`	The URL of an SDMX-ML document containing the external object.
	`serviceURL`	The URL of an SDMX-compliant web service from which the external object can be retrieved.
	`+maintainer`	Association to the Maintenance Agency responsible for maintaining the artefact.
`Agency`		See section on “Organisations”

Copyright © SDMx TWG 2020

3.4.3 Basic Inheritance

Class Diagram – Basic Inheritance from the Base Inheritance Classes

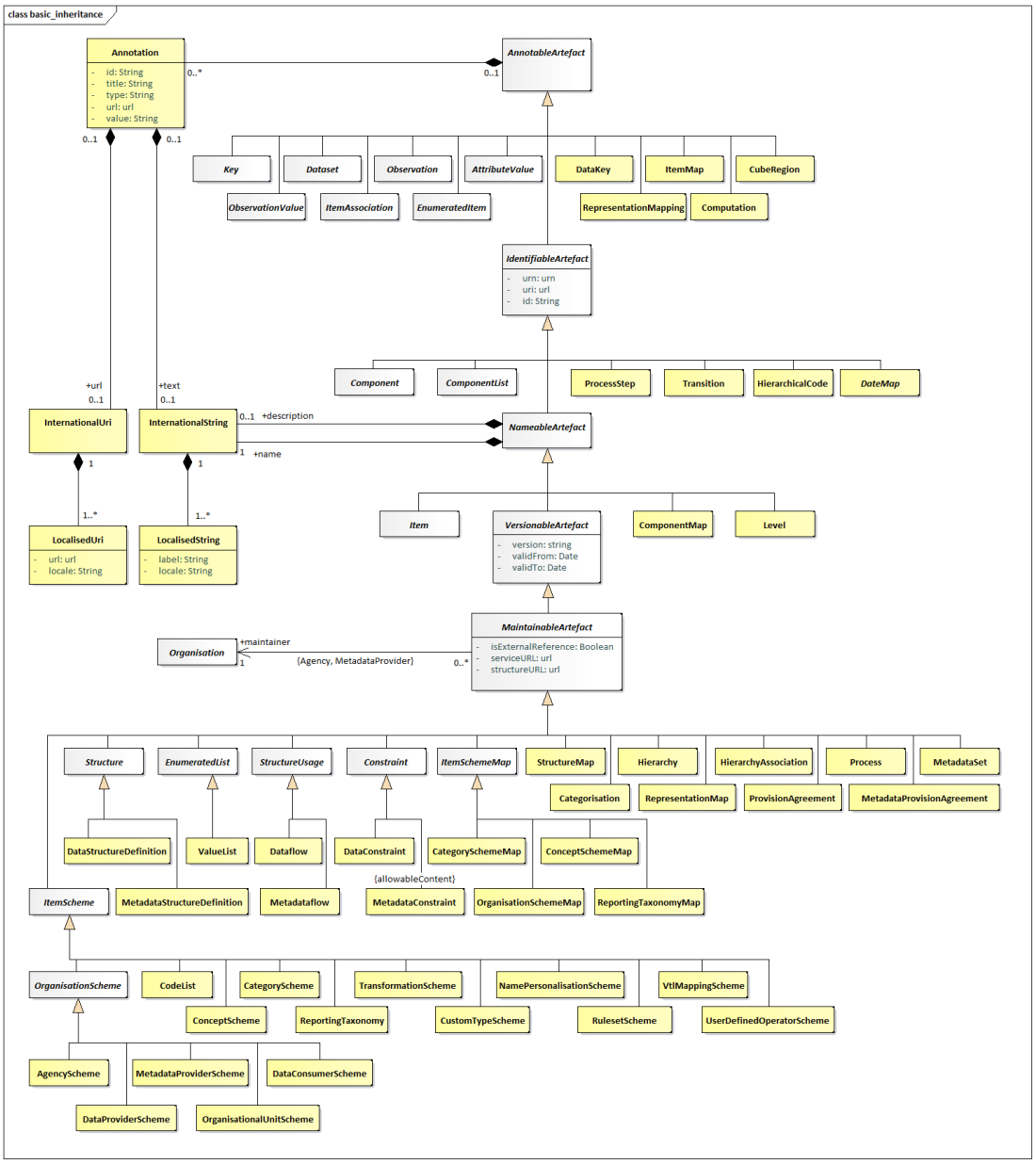


Figure 11. Basic Inheritance from the Base Structures

Explanation of the Diagram

NARRATIVE

The diagram above shows the inheritance within the base structures. The concrete classes are introduced and defined in the specific package to which they relate.

Copyright © SDMx TWG 2020

3.4.4 Data Types

Class Diagram

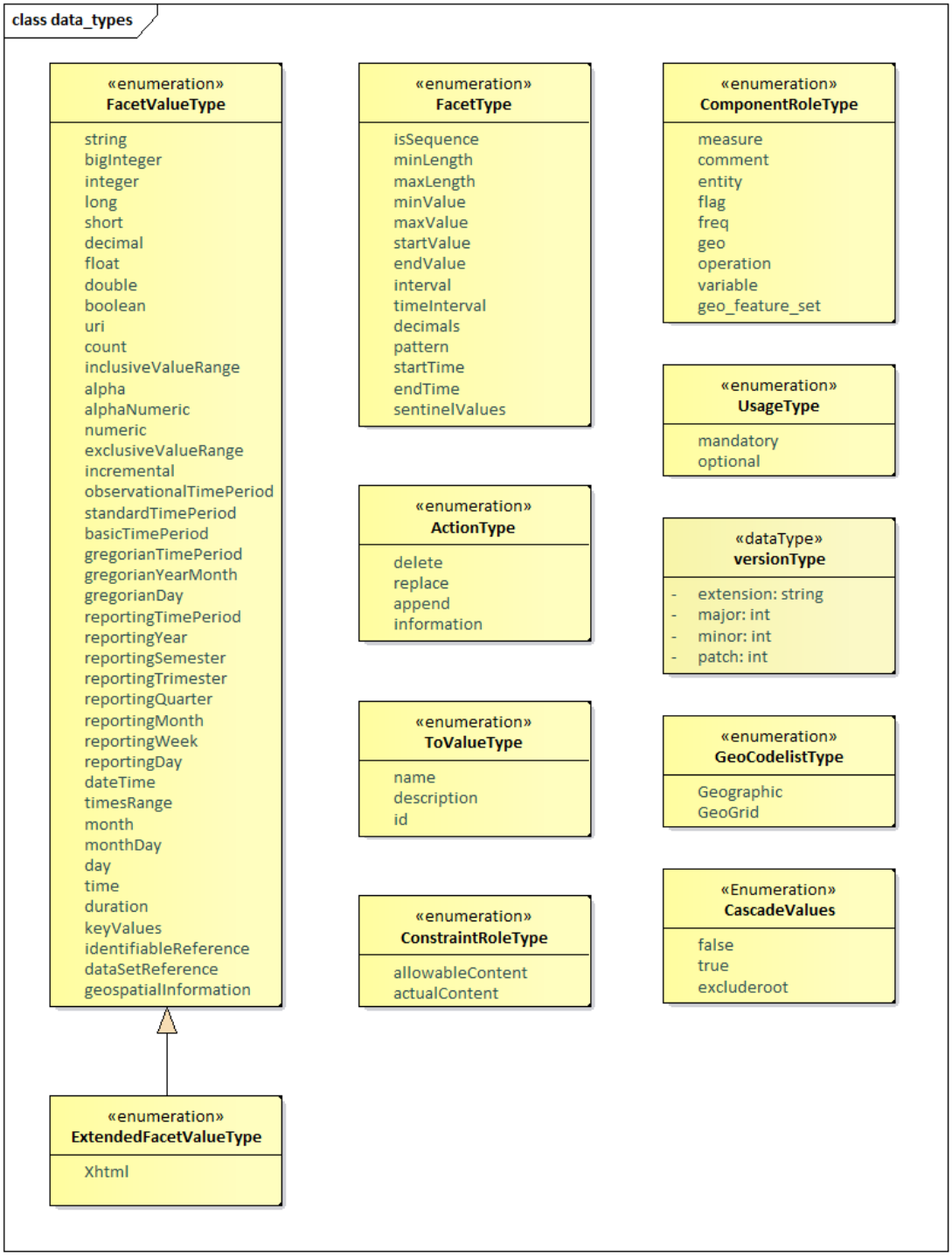


Figure 12. Class Diagram of Basic Data Types

Copyright © SDMx TWG 2020

Explanation of the Diagram

NARRATIVE

The `FacetType` and `FacetValueType` enumerations are used to specify the valid format of the content of a non-enumerated Concept or the usage of a Concept when specified for use on a `Component` on a `Structure` (such as a Dimension in a `DataStructureDefinition`). The description of the various types can be found in the chapter on `ConceptScheme` (section 4.5).

The `ActionType` enumeration is used to specify the action that a receiving system should take when processing the content that is the object of the action. It is enumerated as follows:

- `Append`: Data or metadata is an incremental update for an existing data/metadata set or the provision of new data or documentation (attribute values) formerly absent. If any of the supplied data or metadata is already present, it will not replace that data or metadata. This corresponds to the “Update” value found in version 1.0 of the SDMX Technical Standards.
- `Replace`: Data/metadata is to be replaced and may also include additional data/metadata to be appended.
- `Delete`: Data/Metadata is to be deleted.
- `Information`: Data and metadata are for information purposes.

The `ToValueType` data type contains the attributes to support transformations defined in the `StructureMap` (see Section 0).

The `ConstraintRoleType` data type contains the attributes that identify the purpose of a Constraint (`allowableContent`, `actualContent`).

The `ComponentRoleType` data type contains the predefined Concept roles that can be assigned to any Component.

The `CascadeValues` data type contains the possible values for a `MemberValue` within a `CubeRegion`, in order to enable cascading to all children Codes of a selected Code, while including/excluding the latter in the selection.

The `VersionType` data types provides the details for versioning according to SDMX versioning rules, as explained in the Section “Versioning” of the SDMX Technical Notes.

3.4.5 The Item Scheme Pattern

Context

The Item Scheme is a basic architectural pattern that allows the creation of list schemes for use in simple taxonomies, for example.

The `ItemScheme` is the basis for `CategoryScheme`, `Codelist`, `ConceptScheme`, `ReportingTaxonomy`, `OrganisationScheme`, `TransformationScheme`, `CustomTypeScheme`, `NamePersonalisationScheme`, `RulesetScheme`, `VtlMappingScheme` and `UserDefinedOperatorScheme`.

Copyright © SDMx TWG 2020

Class Diagram

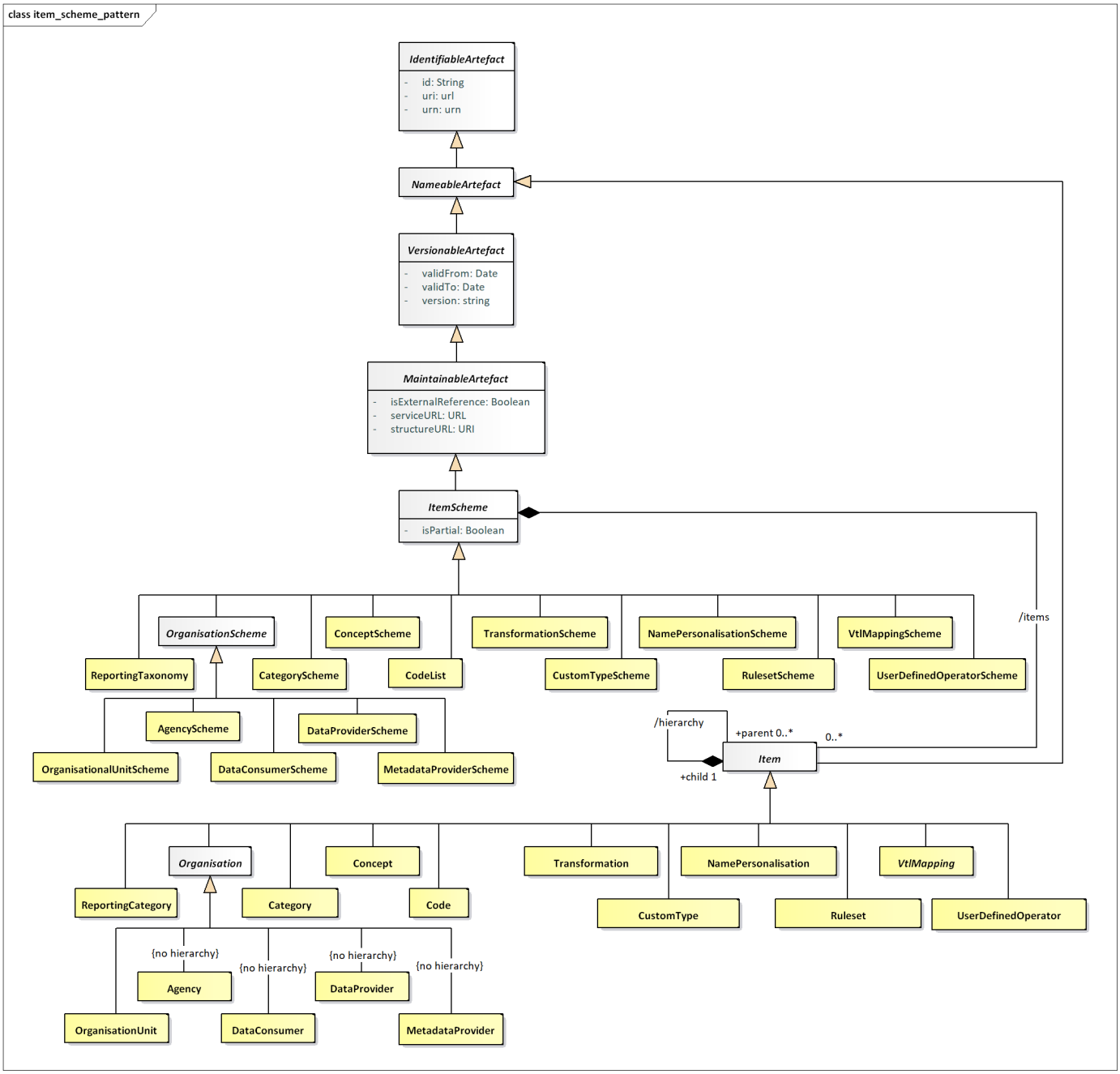


Figure 13. The Item Scheme pattern

Explanation of the Diagram

NARRATIVE

The `ItemScheme` is an abstract class which defines a set of `Item` (this class is also abstract). Its main purpose is to define a mechanism which can be used to create taxonomies which can classify other parts of the SDMX Information Model. It is derived from `MaintainableArtefact` which gives it the ability to be annotated, have identity, naming, versioning and be associated with an Agency. An example of a concrete class is a `ConceptScheme`. The associated Concepts are `Item`s.

Copyright © SDMx TWG 2020

In an exchange environment an `ItemScheme` is allowed to contain a sub-set of the `Item`s in the maintained `ItemScheme`. If such an `ItemScheme` is disseminated with a sub-set of the Items then the fact that this is a sub-set is denoted by setting the `isPartial` attribute to “true”.

A “partial” `ItemScheme` cannot be maintained independently in its partial form i.e., it cannot contain `Item`s that are not present in the full `ItemScheme` and the content of any one `Item` (e.g., names and descriptions) cannot deviate from the content in the full `ItemScheme`. Furthermore, the `id` of the `ItemScheme` where `isPartial` is set to `true` is the same as the `id` of the full `ItemScheme` (`agencyId`, `id`, `version`). This is important as this is the `id` that that is referenced in other structures (e.g., a Codelist referenced in a DSD) and this `id` is always the same, regardless of whether the disseminated `ItemScheme` is the full `ItemScheme` or a partial `ItemScheme`.

The purpose of a partial `ItemScheme` is to support the exchange and dissemination of a sub-set `ItemScheme` without the need to maintain multiple `ItemScheme`s which contain the same `Item`s. For instance, when a Codelist is used in a `DataStructureDefinition` it is sometimes the case that only a sub-set of the Codes in a Codelist are relevant. In this case a partial Codelist can be constructed using the Constraint mechanism explained later in this document.

`Item` inherits from `NameableArtefact` which gives it the ability to be annotated and have identity, and therefore has id, uri and urn attributes, a name and a description in the form of an `InternationalString`. Unlike the parent `ItemScheme`, the `Item` itself is not a `MaintainableArtefact` and therefore cannot have an independent Agency (i.e., it implicitly has the same `agencyId` as the `ItemScheme`).

The `Item` can be hierarchic and so one `Item` can have child `Item`s. The restriction of the hierarchic association is that a child `Item` can have only parent `Item`.

Copyright © SDMx TWG 2020

DEFINITIONS

Class	Feature
`ItemScheme`	Inherits from: `MaintainableArtefact` Direct subclasses: `CategoryScheme`, `ConceptScheme`, `Codelist`, `ReportingTaxonomy`, `OrganisationScheme`, `TransformationScheme`, `UserDefinedOperatorScheme`
	`isPartial`
	`/items`
`Item`	Inherits from: `NameableArtefact` Direct subclasses: `Category`, `Concept`, `Code`, `ReportingCategory`, `Organisation`, `Transformation`, `CustomType`, `NamePersonalisa`
	`hierarchy`

3.4.6 The Structure Pattern

Context

The Structure Pattern is a basic architectural pattern which allows the specification of complex tabular structures which are often found in statistical data (such as Data Structure Definition, and Metadata Structure Definition). A Structure is a set of ordered lists. A pattern to underpin this tabular structure has been developed, so that commonalities between these structure definitions can be supported by common software and common syntax structures.

Copyright © SDMx TWG 2020

Class Diagrams

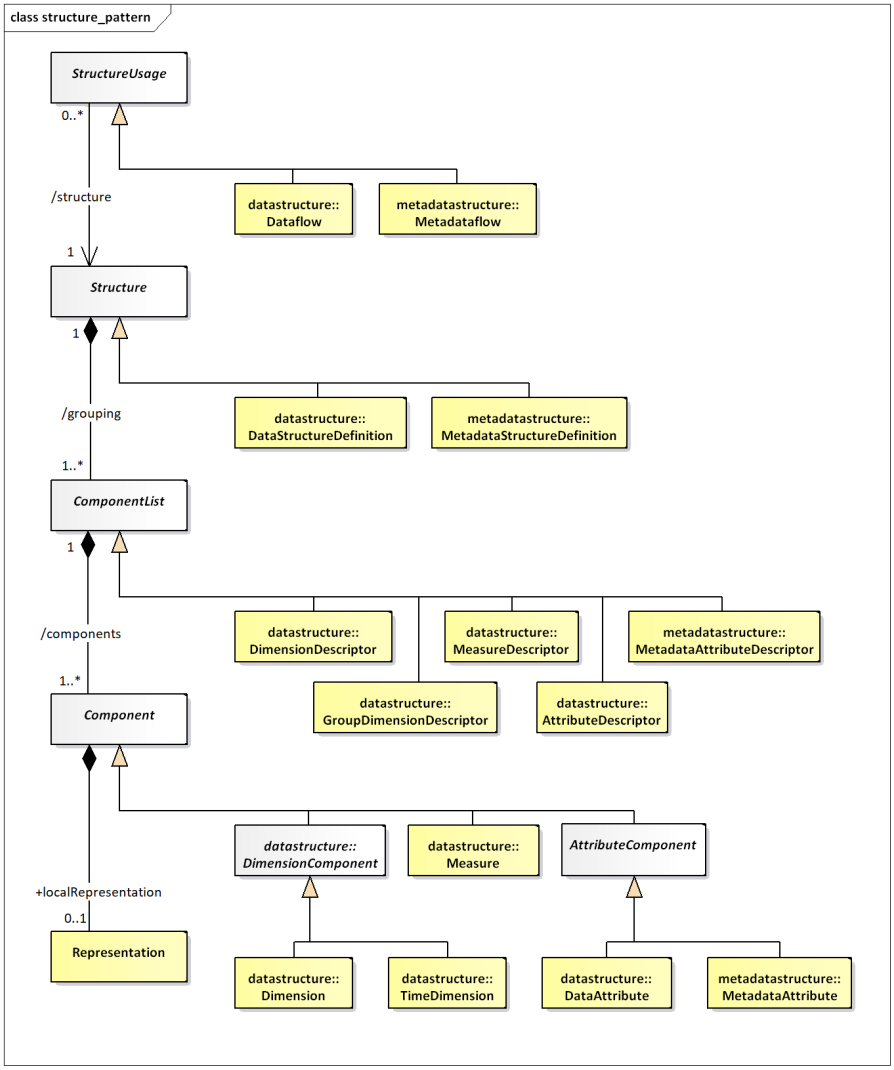


Figure 14. The Structure Pattern

Copyright © SDMx TWG 2020

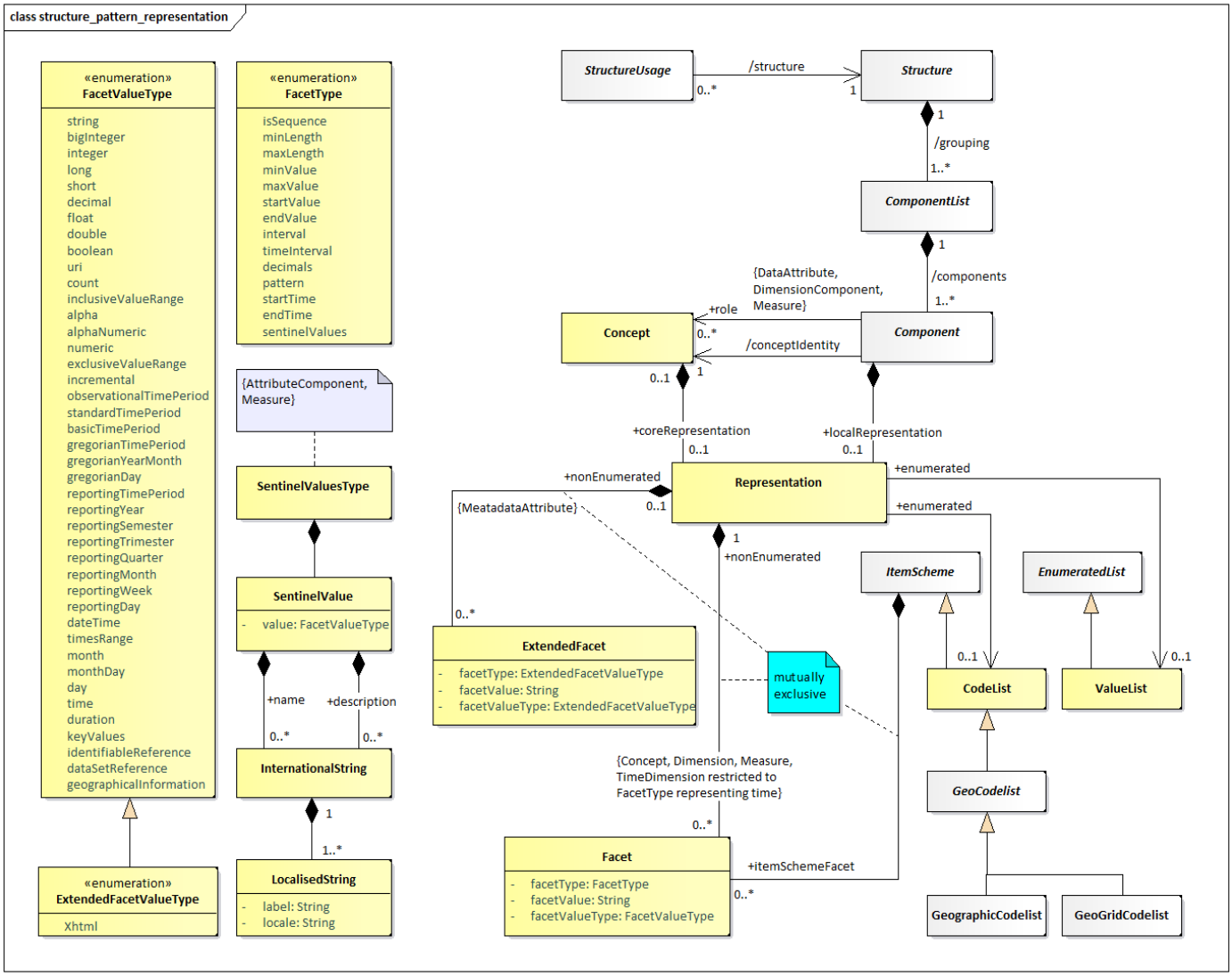


Figure 15. Representation within the Structure Pattern

Explanation of the Diagrams

NARRATIVE

The `Structure` is an abstract class which contains a set of one or more `ComponentList` (s) (this class is also abstract). An example of a concrete `Structure` is `DataStructureDefinition`.

The `ComponentList` is a list of one or more `Component` (s). The `ComponentList` has several concrete descriptor classes based on it: `DimensionDescriptor`, `GroupDimensionDescriptor`, `MeasureDescriptor`, and `AttributeDescriptor` of the `DataStructureDefinition` and `MetadataAttributeDescriptor` of the `MetadataStructureDefinition`.

The `Component` is contained in a `ComponentList`. The type of `Component` in a `ComponentList` is dependent on the concrete class of the `ComponentList` as follows:

- `DimensionDescriptor`: `Dimension`, `TimeDimension`
- `GroupDimensionDescriptor`: `Dimension`, `TimeDimension`
- `MeasureDescriptor`: `Measure`
- `AttributeDescriptor`: `DataAttribute`, `MetadataAttributeRef`
- `MetadataAttributeDescriptor`: `MetadataAttribute`

Copyright © SDMx TWG 2020

Each `Component` takes its semantic (and possibly also its representation) from a Concept in a `ConceptScheme`. This is represented by the `conceptIdentity` association to Concept.

The `Component` may also have a `localRepresentation`. This allows a concrete class, such as Dimension, to specify its representation which is local to the `Structure` in which it is contained (for Dimension this will be `DataStructureDefinition`), and thus overrides any `coreRepresentation` specified for the Concept.

The Representation can be enumerated or non-enumerated. The valid content of an enumerated representation is specified either in an `ItemScheme` which can be one of `Codelist`, `ValueList` or `GeoCodelist`. The valid content of a non-enumerated representation is specified as one or more Facet(s) (for example, these may specify minimum and maximum values). For any Attribute this is achieved by one of more `ExtendedFacet`(s), which allow the additional representation of XHTML.

The types of representation that are valid for specific components is expressed in the model as a constraint on the association:

- The `Dimension`, `DataAttribute`, `Measure`, `MetadataAttribute` may be enumerated and, if so, use an `EnumeratedList`.
- The Dimension and Measure may be non-enumerated and, if so, use one or more Facet(s), note that the `FacetValueType` applicable to the `TimeDimension` is restricted to those that represent time.
- The `MetadataAttribute` and `DataAttribute` may be non-enumerated and, if so, use one or more `ExtendedFacet`(s).

The `Structure` may be used by one or more `StructureUsage`(s). An example of this, in terms of concrete classes, is that a Dataflow (sub class of `StructureUsage`) may use a particular `DataStructureDefinition` (sub class of `Structure`), and similar constructs apply for the Metadataflow (link to `MetadataStructureDefinition`).

Copyright © SDMx TWG 2020

DEFINITIONS

Copyright © SDMx TWG 2020

Class	Feature
`StructureUsage`	Inherits from: `MaintainableArtefact` Sub classes are: `Dataflow`, `Metadataflow`
	`structure`
`Structure`	Inherits from: `MaintainableArtefact` Sub classes are: `DataStructureDefinition`, `MetadataStructureDefinition`
	`grouping`
`ComponentList`	Inherits from: `IdentifiableArtefact` Sub classes are: `DimensionDescriptor`, `GroupDimensionDescriptor`, `MeasureDescriptor`, `AttributeDescriptor`, `MetadataAttributeDescriptor`
	`components`

Copyright © SDMx TWG 2020

Class	Feature
`Component`	Inherits from: `IdentifiableArtefact` Sub classes are: `Measure`, `AttributeComponent`, `DimensionComponent`
	`conceptIdentity`
	`localRepresentation`
`Representation`	
	`+enumerated`
	`+nonEnumerated`

Copyright © SDMx TWG 2020

Class	Feature
`Facet`	
	`facetType`
	`facetValueType`
	`+itemSchemeFacet`
`ExtendedFacet`	

The specification of the content and use of the sub classes to `ComponentList` and `Component` can be found in the section in which they are used (`DataStructureDefinition` and `MetadataStructureDefinition`). Moreover, the `FacetType` `SentinelValues` is explained in the datastructure representation diagram (see Paragraph Definitions), since it only concerns `DataStructureDefinitions` .

Copyright © SDMx TWG 2020

REPRESENTATION CONSTRUCTS

The majority of SDMX `FacetValueTypes` are compatible with those found in XML Schema, and have equivalents in most current implementation platforms:

SDMX Facet Value Type	XML Schema Data Type	JSON Schema Data Type	.NET Framework Type	Java Data Type
`String`	`xsd:string`	`string`	`System.String`	`java.lang.String`
`Big Integer`	`xsd:integer`	`integer`	`System.Decimal`	`java.math.BigInteger`
`Integer`	`xsd:int`	`integer`	`System.Int32`	`int`
`Long`	`xsd:long`	`integer`	`System.Int64`	`long`
`Short`	`xsd:short`	`integer`	`System.Int16`	`short`
`Decimal`	`xsd:decimal`	`number`	`System.Decimal`	`java.math.BigDecimal`
`Float`	`xsd:float`	`number`	`System.Single`	`float`
`Double`	`xsd:double`	`number`	`System.Double`	`double`
`Boolean`	`xsd:boolean`	`boolean`	`System.Boolean`	`boolean`
`URI`	`xsd:anyURI`	`string:uri`	`System.Uri`	`Java.net.URI` or `java.lang.String`
`DateTime`	`xsd:dateTime`	`string:date-time`	`System.DateTime`	`javax.xml.datatype.XMLGregorianCalendar`
`Time`	`xsd:time`	`string:time`	`System.DateTime`	`javax.xml.datatype.XMLGregorianCalendar`
`GregorianYear`	`xsd:gYear`	`string` [1]	`System.DateTime`	`javax.xml.datatype.XMLGregorianCalendar`
`GregorianMonth`	`xsd:gYearMonth`	`string`	`System.DateTime`	`javax.xml.datatype.XMLGregorianCalendar`
`GregorianDay`	`xsd:date`	`string`	`System.DateTime`	`javax.xml.datatype.XMLGregorianCalendar`
`Day, MonthDay, Month`	`xsd:g*`	`string`	`System.DateTime`	`javax.xml.datatype.XMLGregorianCalendar`
`Duration`	`xsd:duration`	`string`	`System.TimeSpan`	`javax.xml.datatype.Duration`

There are also a number of SDMX data types which do not have these direct correspondences, often because they are composite representations or restrictions of a broader data type. These are detailed in Section 6 of the standards.

Copyright © SDMx TWG 2020

The Representation is composed of Facets, each of which conveys characteristic information related to the definition of a value domain. Often a set of Facets are needed to convey the required semantic. For example, a sequence is defined by a minimum of two Facets: one to define the start value, and one to define the interval.

Facet Type	Explanation
`isSequence`	The `isSequence` facet indicates whether the values are intended to be ordered, and it may work in combination with the `interval`, `startValue`, and `endValue` facet or the `timeInterval`, `startTime`, and `endTime`, facets. If this attribute holds a value of true, a start value or time and a numeric or time interval must be supplied. If an end value is not given, then the sequence continues indefinitely.
`interval`	The interval attribute specifies the permitted interval (increment) in a sequence. In order for this to be used, the `isSequence` attribute must have a value of true.
`startValue`	The `startValue` facet is used in conjunction with the `isSequence` and interval facets (which must be set in order to use this facet). This facet is used for a numeric sequence and indicates the starting point of the sequence. This value is mandatory for a numeric sequence to be expressed.
`endValue`	The `endValue` facet is used in conjunction with the `isSequence` and interval facets (which must be set in order to use this facet). This facet is used for a numeric sequence and indicates that ending point (if any) of the sequence.
`timeInterval`	The `timeInterval` facet indicates the permitted duration in a time sequence. In order for this to be used, the `isSequence` facet must have a value of true.
`startTime`	The `startTime` facet is used in conjunction with the `isSequence` and `timeInterval` facets (which must be set in order to use this facet). This attribute is used for a time sequence and indicates the start time of the sequence. This value is mandatory for a time sequence to be expressed.
`endTime`	The `endTime` facet is used in conjunction with the `isSequence` and `timeInterval` facets (which must be set in order to use this facet). This facet is used for a time sequence and indicates that ending point (if any) of the sequence.
`minLength`	The `minLength` facet specifies the minimum and length of the value in characters.
`maxLength`	The `maxLength` facet specifies the maximum length of the value in characters.
`minValue`	The `minValue` facet is used for inclusive and exclusive ranges, indicating what the lower bound of the range is. If this is used with an inclusive range, a valid value will be greater than or equal to the value specified here. If the inclusive and exclusive data type is not specified (e.g., this facet is used with an integer data type), the value is assumed to be inclusive.
`maxValue`	The `maxValue` facet is used for inclusive and exclusive ranges, indicating what the upper bound of the range is. If this is used with an inclusive range, a valid value will be less than or equal to the value specified here. If the inclusive and exclusive data type is not specified (e.g., this facet is used with an integer data type), the value is assumed to be inclusive.
`decimals`	The decimals facet indicates the number of characters allowed after the decimal separator.
`pattern`	The pattern attribute holds any regular expression permitted in the implementation syntax (e.g., W3C XML Schema).

1. In the JSON schemas, more complex data types are complemented with regular expressions, whenever no direct mapping to a standard type exists. ↩

Copyright © SDMx TWG 2020

3.5 Specific Item Schemes

3.5.1 Introduction

The structures that are an arrangement of objects into hierarchies or lists based on characteristics, and which are maintained as a group inherit from `ItemScheme`. These concrete classes are:

- `Codelist`
- `ConceptScheme`
- `CategoryScheme`
- `AgencyScheme`, `DataProviderScheme`, `MetadataProviderScheme`, `DataConsumerScheme`, `OrganisationUnitScheme`, which all inherit from the abstract class `OrganisationScheme`
- `ReportingTaxonomy`
- `TransformationScheme`
- `RulesetScheme`
- `UserDefinedOperatorScheme`
- `NamePersonalisationScheme`
- `CustomTypeScheme`
- `VtlMappingScheme`

Note that the VTL related schemes (the last 6 of the above list) are detailed in a dedicated section below.

3.5.2 Inheritance View

The inheritance and relationship views are shown together in each of the diagrams in the specific sections below.

Copyright © SDMx TWG 2020

3.5.3 `Codelist`

Class Diagram

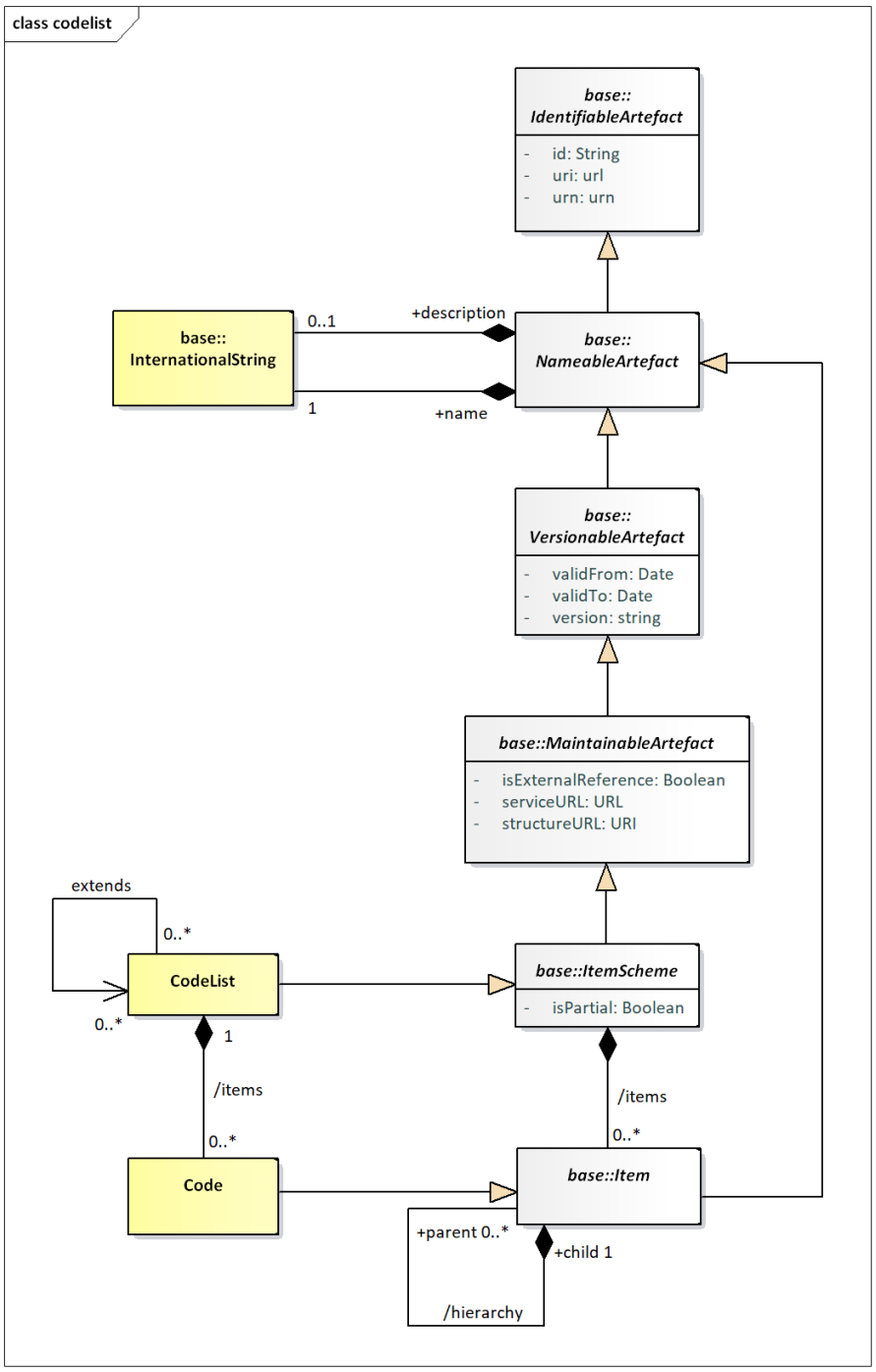


Figure 16. Class diagram of the `Codelist`

Copyright © SDMx TWG 2020

Explanation of the Diagram

NARRATIVE

The `Codelist` inherits from the `ItemScheme` and therefore has the following attributes:

- `id`
- `uri`
- `urn`
- `version`
- `validFrom`
- `validTo`
- `isExternalReference`
- `serviceURL`
- `structureURL`
- `isPartial`

The `Code` inherits from `Item` and has the following attributes:

- `id`
- `uri`
- `urn`

Both `Codelist` and `Code` have the association to `InternationalString` to support a multi-lingual name, an optional multi-lingual description, and an association to Annotation to support notes (not shown).

Through the inheritance the `Codelist` comprise one or more `Code`s, and the `Code` itself can have one or more child `Code`s in the (inherited) hierarchy association. Note that a child `Code` can have only one parent `Code` in this association. A more complex Hierarchy, which allows multiple parents is described later.

A partial `Codelist` (where `isPartial` is set to `true`) is identical to a `Codelist` and contains the `Code` and associated names and descriptions, just as in a normal `Codelist`. However, its content is a subset of the full `Codelist`. The way this works is described in Section The Item Scheme Pattern, paragraph “Explanation of the Diagram” - “Narrative”.

DEFINITIONS

Class	Feature	Description
`Codelist`	Inherits from: `ItemScheme`	A list from which some statistical concepts (coded concepts) take their values.
`Code`	Inherits from: `Item`	A language independent set of letters, numbers or symbols that represent a concept whose meaning is described in a natural language.
	hierarchy	Associates the parent and the child codes.
	extends	Associates a `Codelist` with any `Codelist`s that it may extend.

Copyright © SDMx TWG 2020

Class Diagram – `Codelist` Extension

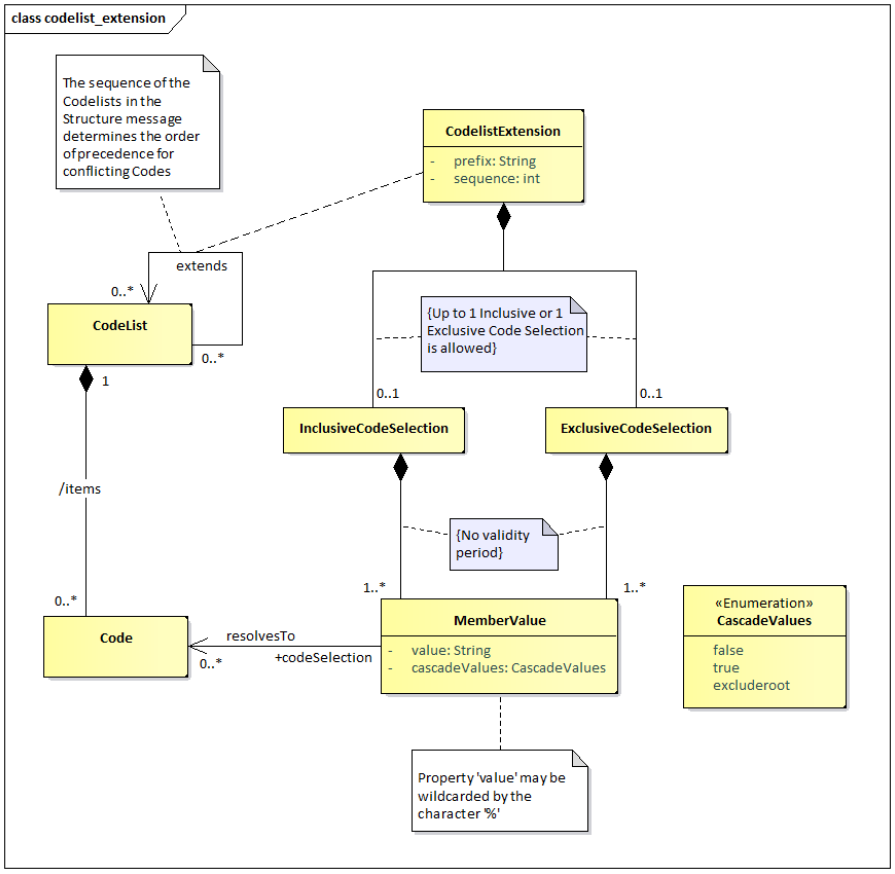


Figure 17. Class diagram for `Codelist` Extension

NARRATIVE

A `Codelist` may extend other `Codelist`s via the `CodelistExtension` class. The latter, via the sequence, indicates the order of precedence of the extended `Codelist`s for conflict resolution of `Code`s. Besides that, the prefix property is used to ensure uniqueness of inherited `Code`s in the extending[1] `Codelist` in case conflicting `Code`s must be included in the latter. Each `CodelistExtension` association may include one `Inclusive`Code`Selection` or one `Exclusive`Code`Selection`; those allow including or excluding a specific selection of `Code`s from the extended `Codelist`s.

The code selection classes may have `MemberValues` in order to specify the subset of the `Code`s that should be included or excluded from the extended `Codelist`. A `MemberValue` may have a value that corresponds to a `Code`, including its children `Code`s (via the `cascadeValues` property), or even include instances of the wildcard character '%' in order to point to a set of `Code`s with common parts in their identifiers.

Copyright © SDMx TWG 2020

DEFINITIONS

Class	Feature	Description
`CodelistExtension`		The association between `Codelist`s that may extend other `Codelist`s.
	`prefix`	A prefix to be used for a `Codelist` used in an extension, in order to avoid `Code` Conflicts.
	`sequence`	The order that will be used when extending a `Codelist`, for resolving `Code` conflicts. The latest `Codelist` used overrides any previous `Codelist`.
`Inclusive` Code `Selection`		The subset of `Code`s to be included when extending a `Codelist`.
`Exclusive` Code `Selection`		The subset of `Code`s to be excluded when extending a `Codelist`.
`MemberValue`	Inherits from: `SelectionValue`	A collection of values based on `Code`s and their children.
	`cascadeValues`	A property to indicate if the child `Code`s of the selected `Code` shall be included in the selection. It is also possible to include children and exclude the `Code` by using the `excluderoot` value.
	`value`	The value of the `Code` to include in the selection. It may include the `%` character as a wildcard.

Class Diagram – Geospatial `Codelist`

The geospatial support is implemented via an extension of the normal `Codelist`. This is illustrated in the following diagrams.

Copyright © SDMx TWG 2020

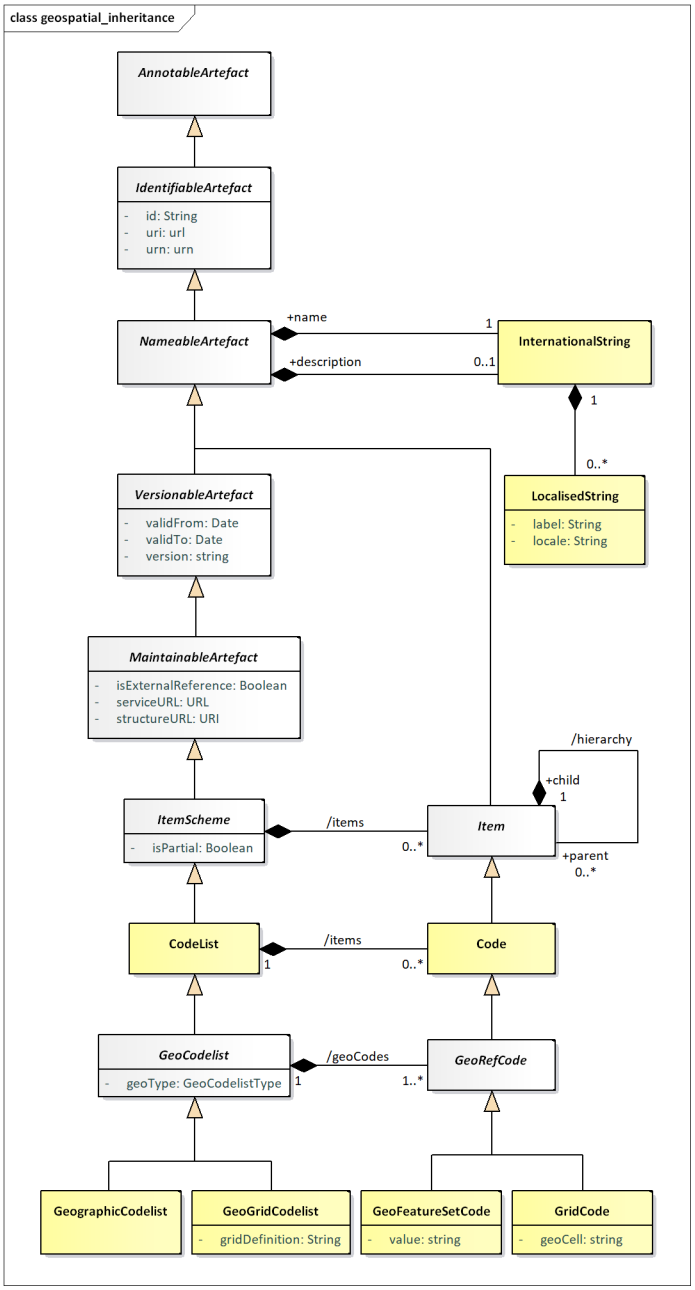


Figure 18. Inheritance for the `GeoCodelist`

Copyright © SDMx TWG 2020

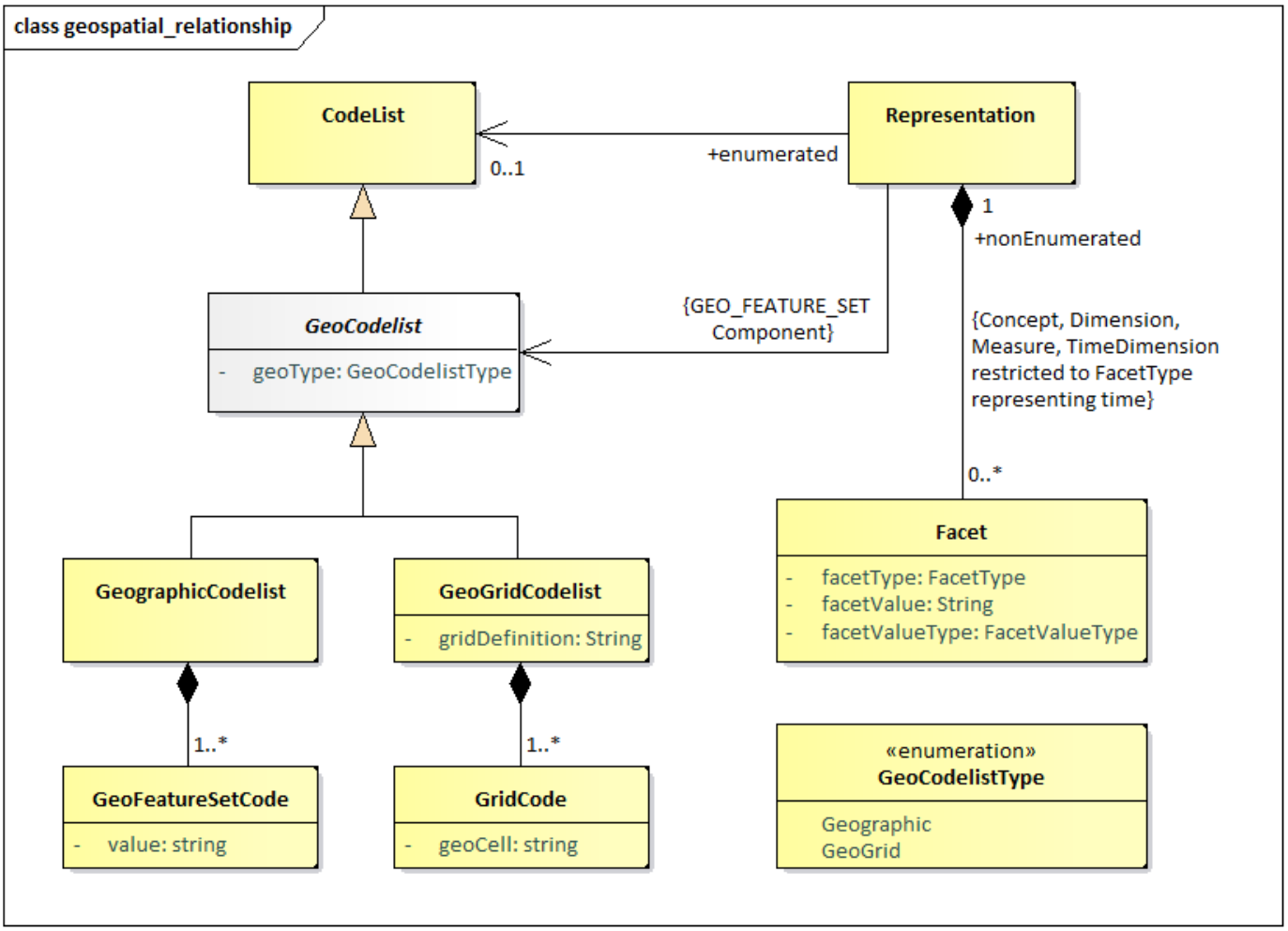


Figure 19. Class diagram for Geospatial `Codelist`

NARRATIVE

A `GeoCodelist` is a specialisation of `Codelist` that includes geospatial information, by comprising a set of special `Code`s, i.e., `GeoRefCode`s. A `GeoCodelist` may be implemented by any of the two following classes, via the `geoType` property:

- `GeographicCodelist`
- `GeoGridCodelist`

The former, i.e., `GeographicCodelist`, comprises a set of `GeoFeatureSet` Code `s`, by adding a value in the `Code` that follows a pattern to represent a geo feature set.

The latter, i.e., `GeoGridCodelist`, comprises a set of `Grid` Code `s`, which are related to the `gridDefinition` specified in the `GeoGridCodelist`.

Copyright © SDMx TWG 2020

DEFINITIONS

Class	Feature	Description
`GeoCodelist`	Abstract Class Sub Classes: `GeographicCodelist`, `GeoGridCodelist`	The abstract class that represents a special type of `Codelist`, which includes geospatial information.
	`geoType`	The type of Geo `Codelist` that the `Codelist` will become.
`GeoRefCode`	Abstract Class Sub Classes: `GeoFeatureSetCode`, `GeoGridCode`	The abstract class that represents a special type of `Code`, which includes geospatial information.
`GeographicCodelist`		A special `Codelist` that has been extended to add a geographical feature set to each of its items, typically including all types of administrative geographies.
`GeoGridCodelist`		A code list that has defined a geographical grid composed of cells representing regular squared portions of the Earth.
	`gridDefinition`	Contains a regular expression string corresponding to the grid definition for the `GeoGrid` `Codelist`.
`GeoFeatureSetCode`		A `Code` that has a geo feature set.
	value	The geo feature set of the `Code`, which represents a set of points defining a feature in a format defined by a predefined pattern (see section 6).
`GeoGridCode`		A `Code` that represents a Geo Grid Cell belonging in a specific grid definition.
	`geoCell`	The value used to assign the `Code` to one cell in the grid.

Copyright © SDMx TWG 2020

3.5.4 `ValueList`

Class Diagram

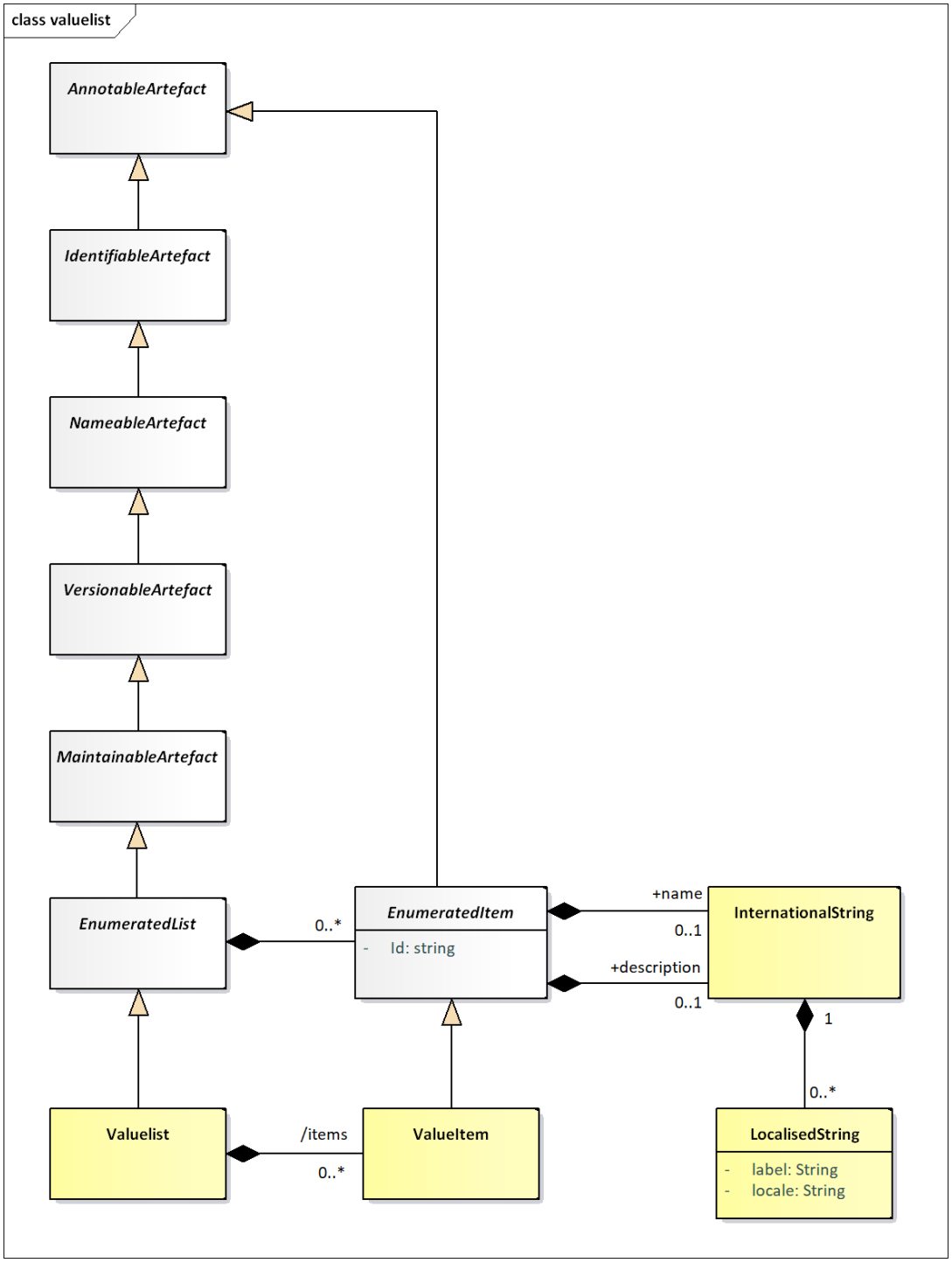


Figure 20. Class diagram of the `ValueList`

Copyright © SDMx TWG 2020

Explanation of the Diagram

NARRATIVE

A `ValueList` inherits from `EnumeratedList` (and hence the `MaintenableArtefact`) and thus has the following attributes:

- `id`
- `uri`
- `urn`
- `version`
- `validFrom`
- `validTo`
- `isExternalReference`
- `registryURL`
- `structureURL`
- `repositoryURL`
- `ValueItem` inherits from `EnumeratedItem`, which adds an id, with relaxed constraints, to the former.

Through the inheritance from `NameableArtefact` the `ValueList` has the association to `InternationalString` to support a multi-lingual name, an optional multi-lingual description, and an association to Annotation to support notes (not shown). Similarly, the `ValueItem`, inherits the association to `InternationalString` and to the Annotation from the `EnumeratedItem`.

The `ValueList` can have one or more `ValueItems`.

DEFINITIONS

Class	Feature	Description
`ValueList`	Inherits from: `EnumeratedList`	A list from which some statistical concepts (enumerated concepts) take their values.
`ValueItem`	Inherits from: `EnumeratedItem`	A language independent set of letters, numbers or symbols that represent a concept whose meaning is described in a natural language.

Copyright © SDMx TWG 2020

3.5.5 Concept Scheme and Concepts

Class Diagram - Inheritance

Copyright © SDMx TWG 2020

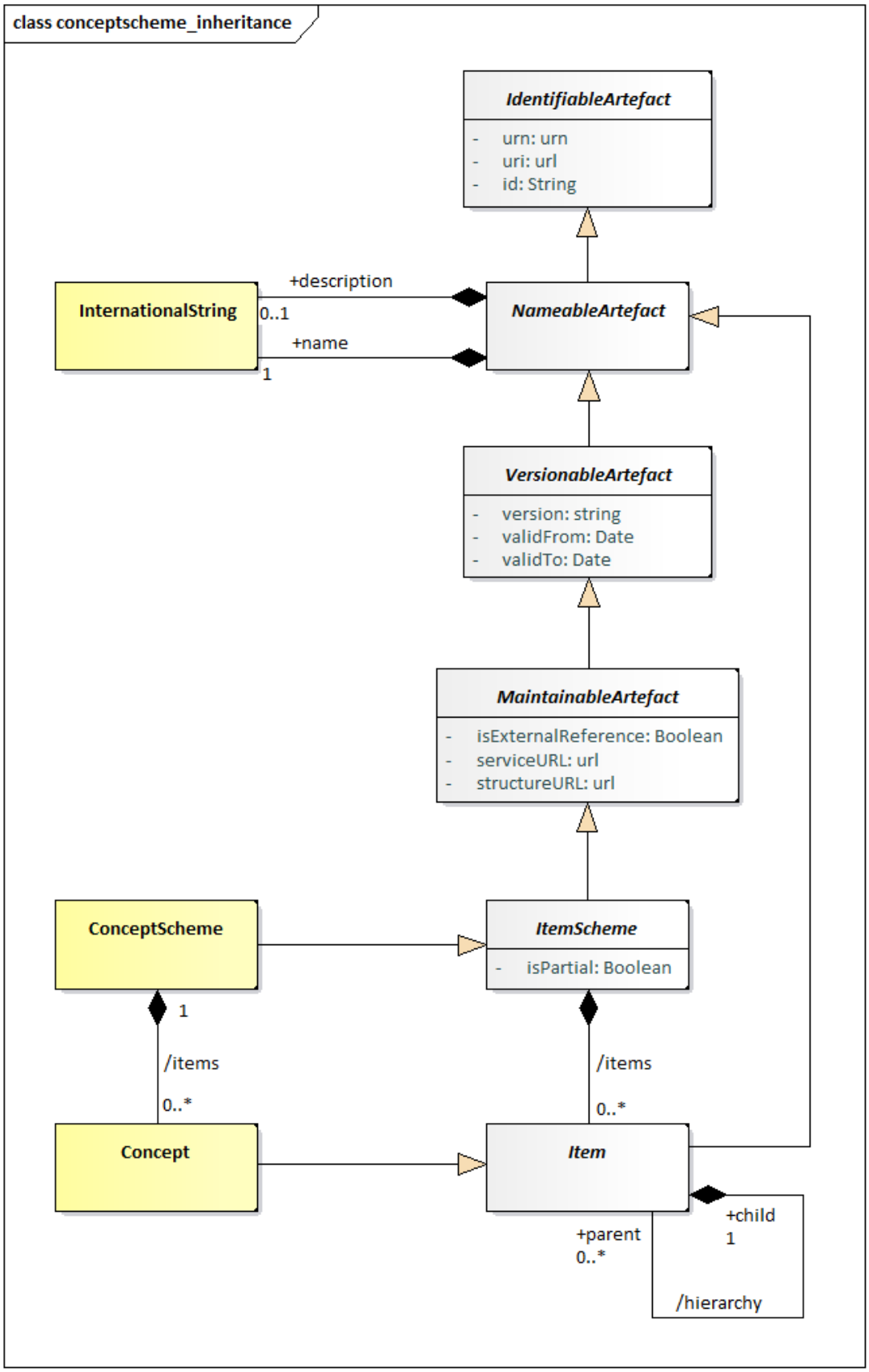


Figure 21. Class diagram of the Concept Scheme

Copyright © SDMx TWG 2020

Explanation of the Diagram

The `ConceptScheme` inherits from the `ItemScheme` and therefore has the following attributes:

- `id`
- `uri`
- `urn`
- `version`
- `validFrom`
- `validTo`
- `isExternalReference`
- `registryURL`
- `structureURL`
- `repositoryURL`
- `isPartial`

Concept inherits from Item and has the following attributes:

- id
- uri
- urn

Through the inheritance from `NameableArtefact` both `ConceptScheme` and Concept have the association to `InternationalString` to support a multi-lingual name, an optional multi-lingual description, and an association to Annotation to support notes (not shown).

Through the inheritance from `ItemScheme` the `ConceptScheme` comprise one or more Concepts, and the Concept itself can have one or more child Concepts in the (inherited) hierarchy association. Note that a child Concept can have only one parent Concept in this association.

A partial `ConceptScheme` (where `isPartial` is set to `true`) is identical to a `ConceptScheme` and contains the Concept and associated names and descriptions, just as in a normal `ConceptScheme`. However, its content is a subset of the full `ConceptScheme`. The way this works is described in Section The Item Scheme Pattern, paragraph “Explanation of the Diagram” - “Narrative”.

Copyright © SDMx TWG 2020

Class Diagram - Relationship

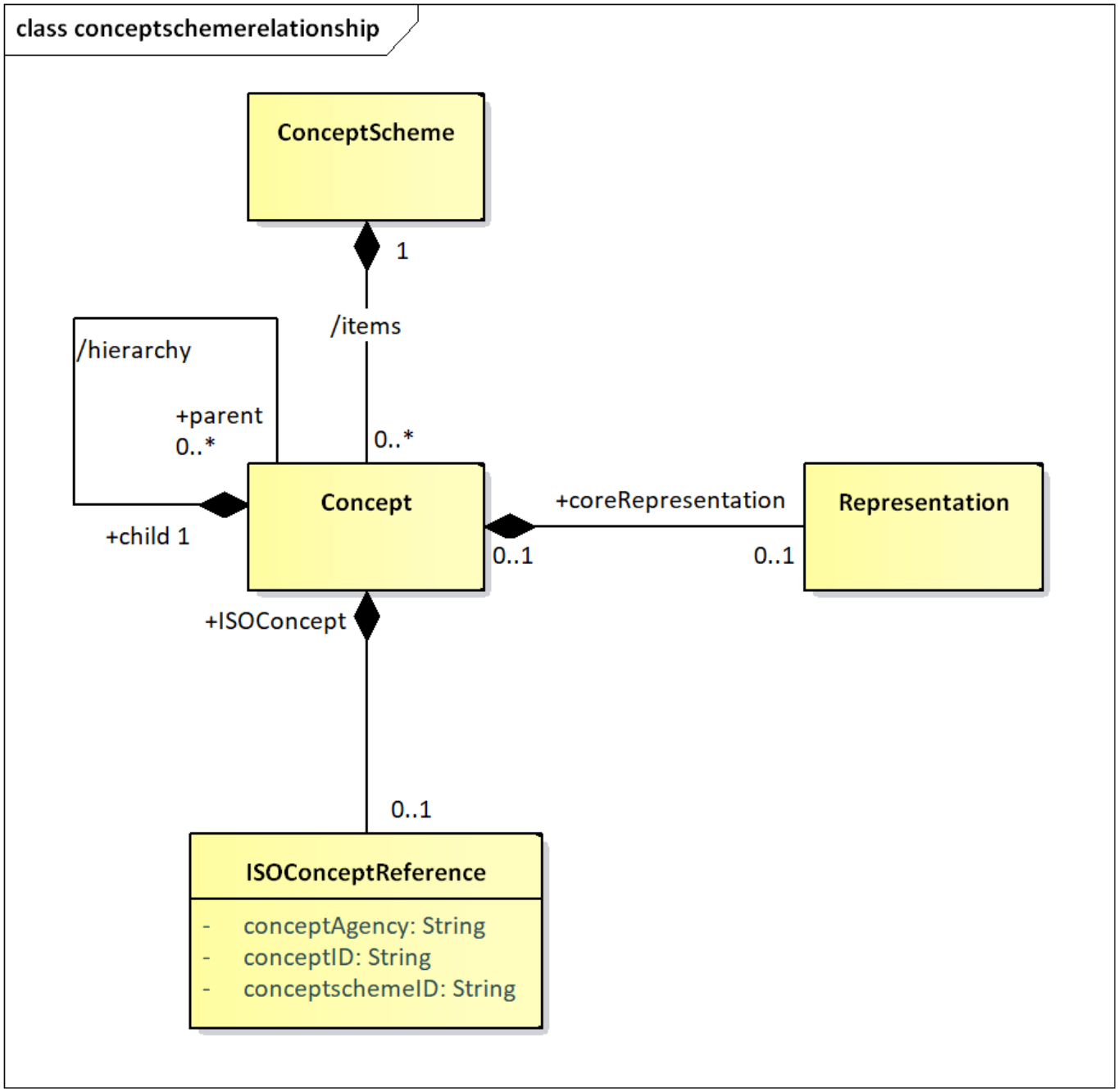


Figure 22. Relationship class diagram of the Concept Scheme

Explanation of the diagram

NARRATIVE

The `ConceptScheme` can have one or more Concepts. A Concept can have zero or more child Concepts, thus supporting a hierarchy of Concepts. Note that a child Concept can have only one parent Concept in this association. The purpose of the hierarchy is to relate concepts that have a semantic relationship: for example, a Reporting_Country and Vis_a_Vis_Country may both have Country as a parent concept, or a CONTACT may have a PRIMARY_CONTACT as a child concept. It is not the purpose of such schemes to define reporting structures: these reporting structures are defined in the `MetadataStructureDefinition`.

Copyright © SDMx TWG 2020

The Concept can be associated with a `coreRepresentation`. The `coreRepresentation` is the specification of the format and value domain of the Concept when used on a structure like a `DataStructureDefinition` or a `MetadataStructureDefinition`, unless the specification of the Representation is overridden in the relevant structure definition. In a hierarchical `ConceptScheme` the Representation is inherited from the parent Concept unless overridden at the level of the child Concept.

The Representation is documented in more detail in the section on the SDMX Base.

The Concept may be related to a concept described in terms of the ISO/IEC 11179 standard. The `ISOConceptReference` identifies this concept and concept scheme in which it is contained.

DEFINITIONS

Class	Feature	Description
`ConceptScheme`	Inherits from `ItemScheme`	The descriptive information for an arrangement or division of concepts into groups based on characteristics, which the objects have in common.
`Concept`	Inherits from `Item`	A concept is a unit of knowledge created by a unique combination of characteristics.
	`/hierarchy`	Associates the parent and the child concept.
	`coreRepresentation`	Associates a `Representation`.
	`+ISOConcept`	Association to an ISO concept reference.
`ISOConceptReference`		The identity of an ISO concept definition.
	`conceptAgency`	The maintenance agency of the concept scheme containing the concept.
	`conceptSchemeID`	The identifier of the concept scheme.
	`conceptID`	The identifier of the concept.

3.5.6 `Category` Scheme

Context

This package defines the structure that supports the definition of and relationships between categories in a category scheme. It is similar to the package for concept scheme. An example of a category scheme is one which categorises data – sometimes known as a subject matter domain scheme or a data category scheme. Importantly, as will be seen later, the individual nodes in the scheme (the “categories”) can be associated to any set of `IdentiableArtefacts` in a Categorisation.

Copyright © SDMx TWG 2020

Class diagram - Inheritance

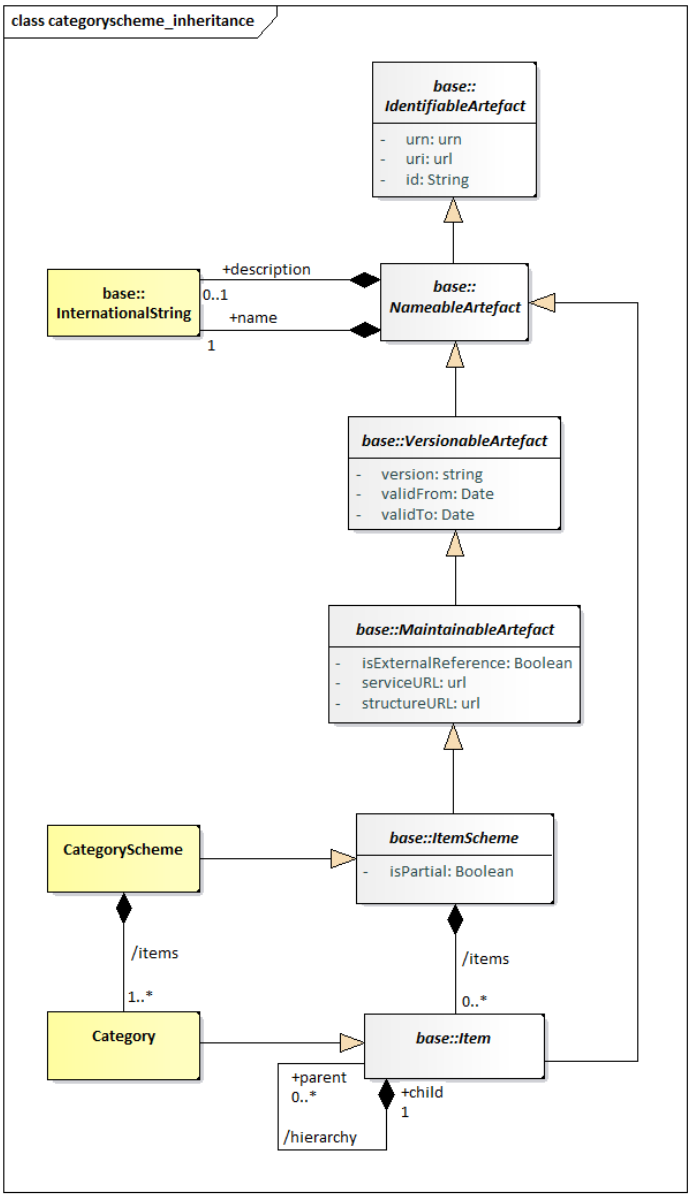


Figure 23. Inheritance Class diagram of the `Category` Scheme

Copyright © SDMx TWG 2020

Explanation of the Diagram

NARRATIVE

The categories are modelled as a hierarchical `ItemScheme`. The `CategoryScheme` inherits from the `ItemScheme` and has the following attributes:

- `id`
- `uri`
- `urn`
- `version`
- `validFrom`
- `validTo`
- `isExternalReference`
- `structureURL`
- `serviceURL`
- `isPartial`

`Category` inherits from `Item` and has the following attributes:

- `id`
- `uri`
- `urn`

Both `CategoryScheme` and `Category` have the association to `InternationalString` to support a multi-lingual name, an optional multi-lingual description, and an association to Annotation to support notes (not shown on the model).

Through the inheritance the `CategoryScheme` comprise one or more `Category`s, and the `Category` itself can have one or more child `Category` in the (inherited) hierarchy association. Note that a child `Category` can have only one parent `Category` in this association.

A partial `CategoryScheme` (where `isPartial` is set to `true`) is identical to a `CategoryScheme` and contains the `Category` and associated names and descriptions, just as in a normal `CategoryScheme`. However, its content is a subset of the full `CategoryScheme`. The way this works is described in Section The Item Scheme Pattern, paragraph “Explanation of the Diagram” - “Narrative”.

Copyright © SDMx TWG 2020

Class diagram - Relationship

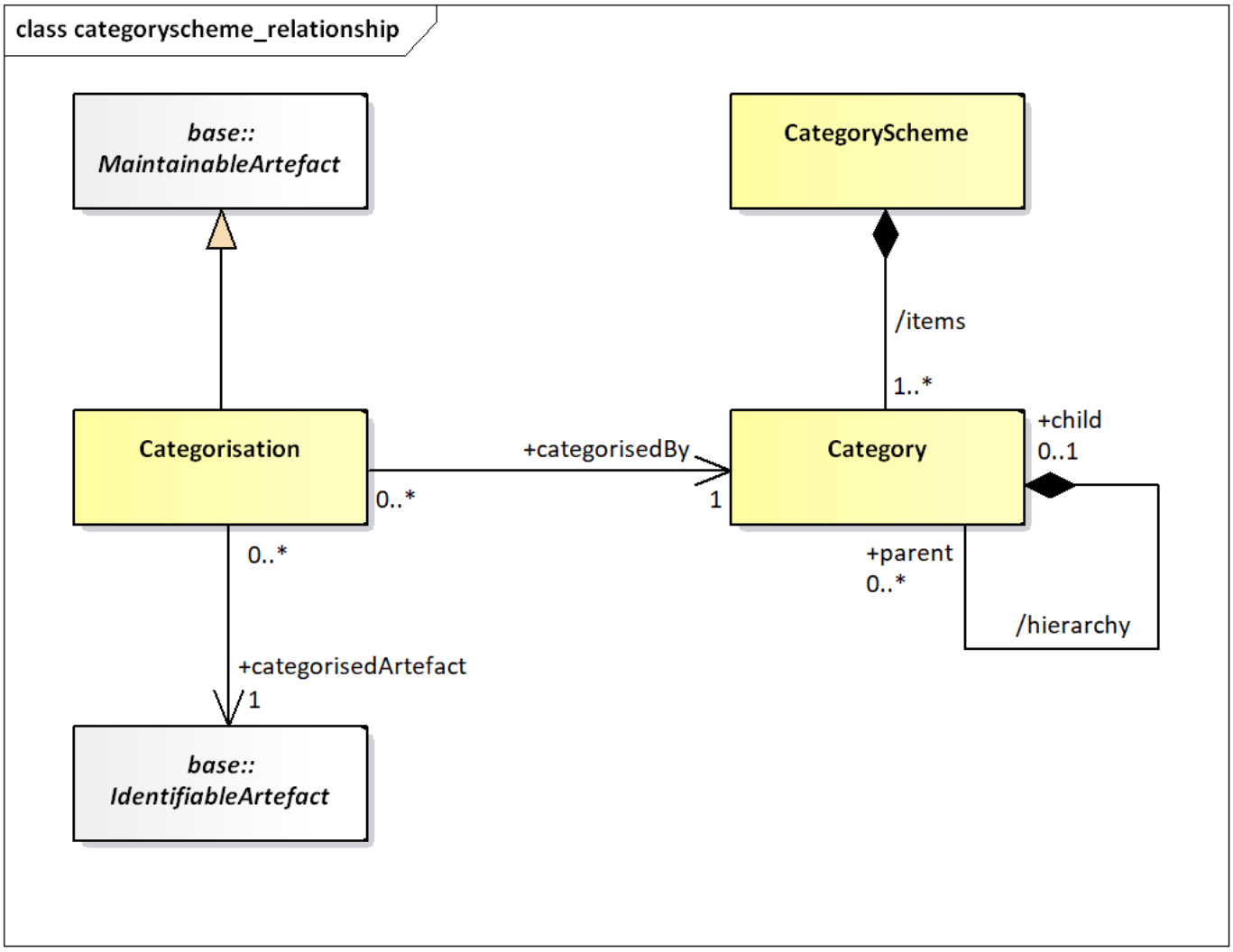


Figure 24. Relationship Class diagram of the `Category` Scheme

The `CategoryScheme` can have one or more `Category` s. The `Category` is Identifiable and has identity information. A `Category` can have zero or more child `Category` s, thus supporting a hierarchy of `Category` s. Any `IdentifiableArtefact` can be `+categorisedBy` a `Category` . This is achieved by means of a Categorisation. Each Categorisation can associate one `IdentifiableArtefact` with one `Category` . Multiple Categorisations can be used to build a set of `IdentifiableArtefacts` that are `+categorisedBy` the same `Category` . Note that there is no navigation (i.e. no embedded reference) to the Categorisation from the `Category` . From an implementation perspective this is necessary as Categorisation has no affect on the versioning of either the `Category` or the `IdentifiableArtefact` .

Copyright © SDMx TWG 2020

DEFINITIONS

Class	Feature	Description
`CategoryScheme`	Inherits from `ItemScheme`	The descriptive information for an arrangement or division of categories into groups based on characteristics, which the objects have in common.
	/items	Associates the categories.
`Category`	Inherits from `Item`	An item at any level within a classification, typically tabulation categories, sections, subsections, divisions, subdivisions, groups, subgroups, classes and subclasses.
	/hierarchy	Associates the parent and the child `Category`.
`Categorisation`	Inherits from `MaintainableArtefact`	Associates an identifiable artefact with a `Category`.
	`categorisedArtefact`	Associates the identifiable artefact.
	`categorisedBy`	Associates the `Category`.

Copyright © SDMx TWG 2020

3.5.7 Organisation Scheme

Class Diagram

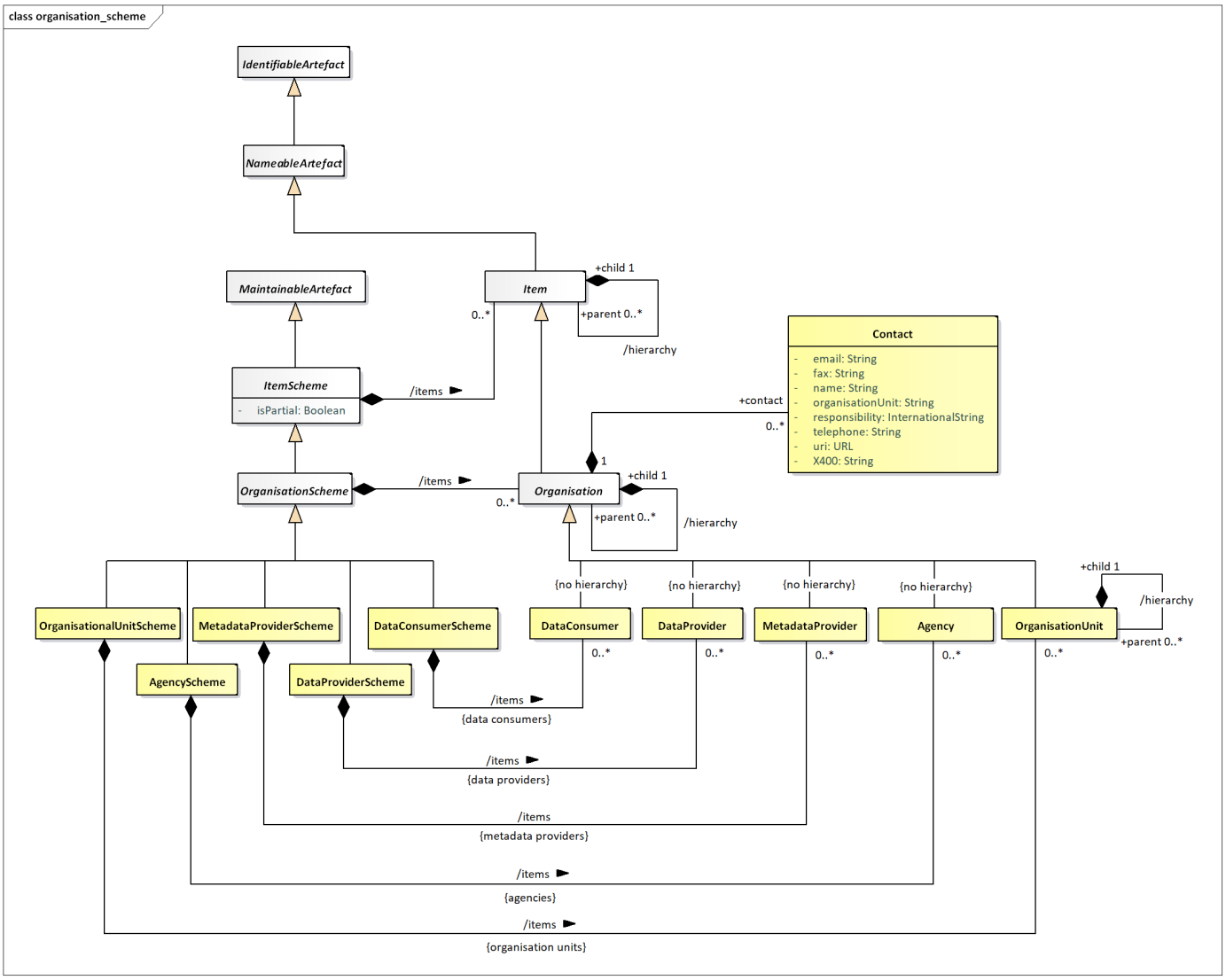


Figure 25. The Organisation Scheme class diagram

Explanation of the Diagram

NARRATIVE

The `OrganisationScheme` is abstract. It contains `Organisation` which is also abstract. The `Organisation` can have child `Organisation`.

The `OrganisationScheme` can be one of five types:

1. `AgencyScheme` – contains `Agency` which is restricted to a flat list of agencies (i.e., there is no hierarchy). Note that the SDMX system of (Maintenance) `Agency` can be hierarchic and this is explained in more detail in the SDMX Standards Section 6 “Technical Notes”.
2. `DataProviderScheme` – contains `DataProvider` which is restricted to a flat list of agencies (i.e., there is no hierarchy).
3. `MetadataProviderScheme` – contains `MetadataProvider` which is restricted to a flat list of agencies (i.e., there is no hierarchy).
4. `DataConsumerScheme` – contains `DataConsumer` which is restricted to a flat list of agencies (i.e., there is no hierarchy).
5. `OrganisationUnitScheme` – contains `OrganisationUnit` which does inherit the /hierarchy association from Organisation.

Copyright © SDMx TWG 2020

Reference metadata can be attached to the `Organisation` by means of the metadata attachment mechanism. This mechanism is explained in the Reference Metadata section of this document (see section 7). This means that the model does not specify the specific reference metadata that can be attached to a `DataProvider`, `MetadataProvider`, `DataConsumer`, `OrganisationUnit` or `Agency`, except for limited Contact information.

A partial `OrganisationScheme` (where `isPartial` is set to `true`) is identical to an `OrganisationScheme` and contains the `Organisation` and associated names and descriptions, just as in a normal `OrganisationScheme`. However, its content is a subset of the full `OrganisationScheme`. The way this works is described in section Section The Item Scheme Pattern, paragraph “Explanation of the Diagram” - “Narrative”.

Copyright © SDMx TWG 2020

DEFINITIONS

Copyright © SDMx TWG 2020

Class	Feature	De
`OrganisationScheme`	Abstract Class. Inherits from `ItemScheme`. Subclasses are: `AgencyScheme`, `DataProviderScheme`, `MetadataProviderScheme`, `DataConsumerScheme`, `OrganisationUnitScheme`	A co Or
	`/items`	As th Or in
`Organisation`	Abstract Class. Inherits from `Item`. Subclasses are: `Agency`, `DataProvider`, `MetadataProvider`, `DataConsumer`, `OrganisationUnit`	An or a fra au wi pe pe ar to so
	`+contact`	As th inf
	`/hierarchy`	As ch Or
`Contact`		An a inc or (o or pa or pe wh inf ite ma ob or ca or sp co
	`name`	Th de th pe lin ex
	`organisationUnit`	Th de th or

Copyright © SDMx TWG 2020

Class	Feature	De
		str lin ex wi co wo
	`responsibility`	Th th pe res or rol thi th
	`telephone`	Th nu co
	`fax`	Th nu co
	`email`	Th ma th
	`X400`	Th ad co
	`uri`	Th ad co
`AgencyScheme`		A co Ma Ag
	`/items`	As th Ma Ag sc
`DataProviderScheme`		A co Da
	`/items`	As th Pr th
`MetadataProviderScheme`		A co Me Pr

Copyright © SDMx TWG 2020

Class	Feature	De
	`/items`	As th Pr th
`DataConsumerScheme`		A co Da Co
	`/items`	As th Co th
`OrganisationUnitScheme`		A co Or Un
	`/items`	As th Or Un sc
`Agency`	Inherits from `Organisation`	Re ag ma art as cla gl str me as Me St De Co `Co`
`DataProvider`	Inherits from `Organisation`	An or th da
`MetadataProvider`	Inherits from `Organisation`	An or th ref me
`DataConsumer`	Inherits from `Organisation`	An or us in fu pr

Copyright © SDMx TWG 2020

Class	Feature	De
`OrganisationUnit`	Inherits from `Organisation`	A in or st
	`/hierarchy`	As ch Or Un

Copyright © SDMx TWG 2020

3.5.8 Reporting Taxonomy

Class Diagram

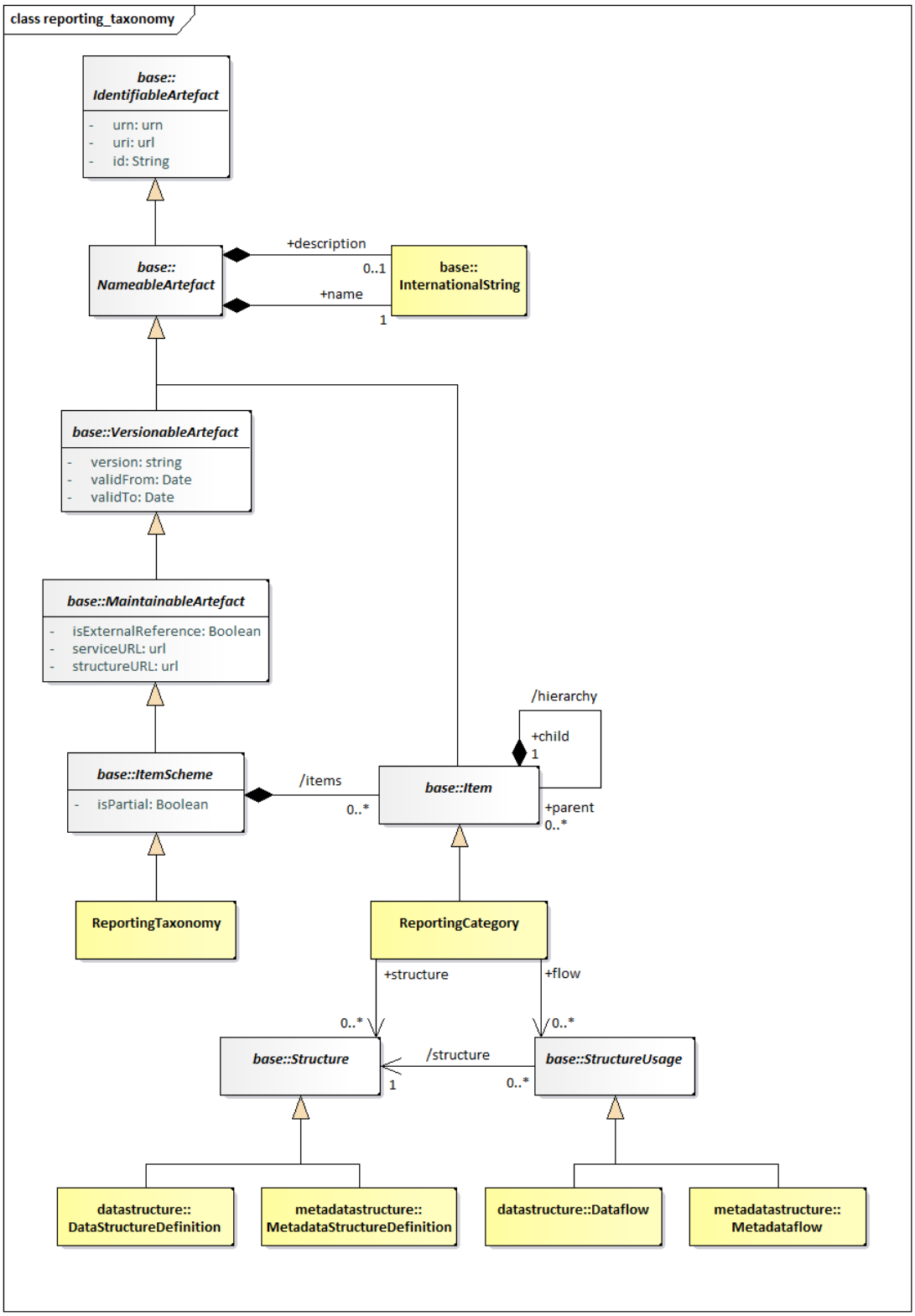


Figure 26. Class diagram of the Reporting Taxonomy

Copyright © SDMx TWG 2020

Explanation of the Diagram

NARRATIVE

In some data reporting environments, and in particular those in primary reporting, a report may comprise a variety of heterogeneous data, each described by a different `Structure`. Equally, a specific disseminated or published report may also comprise a variety of heterogeneous data. The definition of the set of linked sub reports is supported by the `ReportingTaxonomy`.

The `ReportingTaxonomy` is a specialised form of `ItemScheme`. Each `ReportingCategory` of the `ReportingTaxonomy` can link to one or more `StructureUsage` which itself can be one of `Dataflow`, or `Metadataflow`, and one or more `Structure`, which itself can be one of `DataStructureDefinition` or `MetadataStructureDefinition`. It is expected that within a specific `ReportingTaxonomy` each `Category` that is linked in this way will be linked to the same class (e.g. all `Category` in the scheme will link to a Dataflow). Note that a `ReportingCategory` can have child `ReportingCategory` and in this way it is possible to define a hierarchical `ReportingTaxonomy`. It is possible in this taxonomy that some `ReportingCategory` are defined just to give a reporting structure. For instance:

Section 1

1. linked to `Dataflow_1`
2. linked to `Dataflow_2`

Section 2

1. linked to `Dataflow_3`
2. linked to `Dataflow_4`

Here, the nodes of Section 1 and Section 2 would not be linked to Dataflow but the other would be linked to a Dataflow (and hence the `DataStructureDefinition`).

A partial `ReportingTaxonomy` (where `isPartial` is set to `true`) is identical to a `ReportingTaxonomy` and contains the `ReportingCategory` and associated names and descriptions, just as in a normal `ReportingTaxonomy`. However, its content is a sub set of the full `ReportingTaxonomy` The way this works is described in Section The Item Scheme Pattern, paragraph “Explanation of the Diagram” - “Narrative”.

DEFINITIONS

Class	Feature	Description
`ReportingTaxonomy`	Inherits from `ItemScheme`	A scheme which defines the composition structure of a data report where each component can be described by an independent `Dataflow` or `Metadataflow`.
	`/items`	Associates the Reporting `Category`.
`ReportingCategory`	Inherits from `Item`	A component that gives structure to the report and links to data and metadata.
	`/hierarchy`	Associates child Reporting `Category`.
	`+flow`	Association to the data and metadata flows that link to metadata about the provisioning and related data and metadata sets, and the structures that define them.
	`+structure`	Association to the `DataStructureDefinition` and `MetadataStructureDefinition` which define the structural metadata describing the data and metadata that are contained at this part of the report.

1. The `Codelist` that extends 0..* `Codelist`s is the ‘extending’ `Codelist`, while the `Codelist`(s) that are inherited is/are the ‘extended’ `Codelist`(s). ↩

Copyright © SDMx TWG 2020

3.6 Data Structure Definition and Dataset

3.6.1 Introduction

The `DataStructureDefiniton` is the class name for a structure definition for data. Some organisations know this type of definition as a “Key Family” and so the two names are synonymous. The term Data Structure Definition (also referred to as DSD) is used in this specification.

Many of the constructs in this layer of the model inherit from the SDMX Base Layer. Therefore, it is necessary to study both the inheritance and the relationship diagrams to understand the functionality of individual packages. In simple sub models these are shown in the same diagram but are omitted from the more complex sub models for the sake of clarity. In these cases, the inheritance diagram below shows the full inheritance tree for the classes concerned with data structure definitions.

There are very few additional classes in this sub model other than those shown in the inheritance diagram below. In other words, the SDMX Base gives most of the structure of this sub model both in terms of associations and in terms of attributes. The relationship diagrams shown in this section show clearly when these associations are inherited from the SDMX Base (see the Appendix A Short Guide to UML in the SDMX Information Model to see the diagrammatic notation used to depict this).

The actual SDMX Base construct from which the concrete classes inherit depends upon the requirements of the class for:

- Annotation – `AnnotableArtefact`
- Identification – `IdentifiableArtefact`
- Naming – `NameableArtefact`
- Versioning – `VersionableArtefact`
- Maintenance – `MaintainableArtefact`

Copyright © SDMx TWG 2020

3.6.2 Inheritance View

Class Diagram

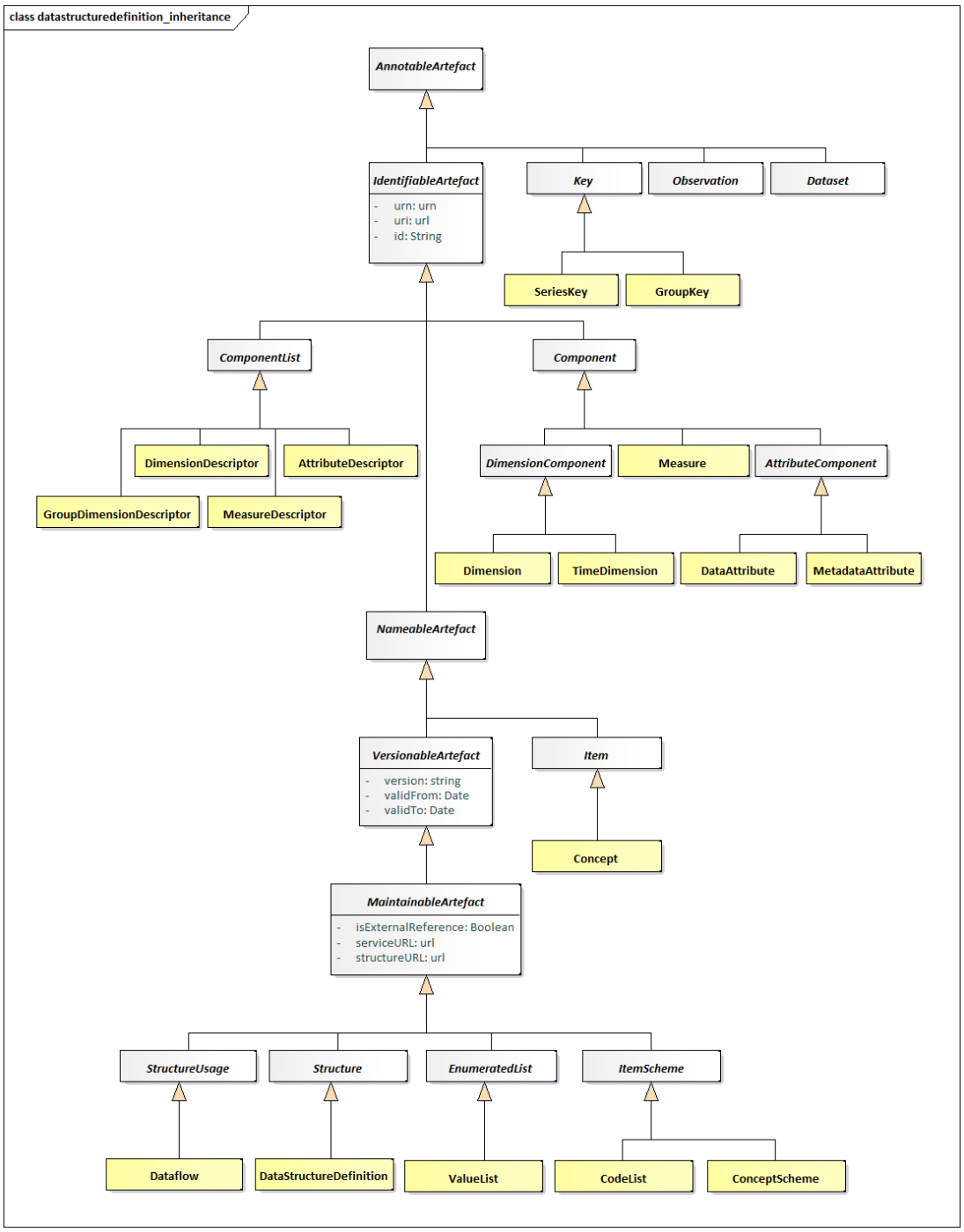


Figure 27. Class inheritance in the Data Structure Definition and Data Set Packages

Copyright © SDMx TWG 2020

Explanation of the Diagram

NARRATIVE

Those classes in the SDMX metamodel which require annotations inherit from `AnnotableArtefact`. These are:

- `IdentifiableArtefact`
- `DataSet`
- `Key` (and therefore `SeriesKey` and `GroupKey`)
- `Observation`

Those classes in the SDMX metamodel which require annotations and global identity are derived from `IdentifiableArtefact`. These are:

- `NameableArtefact`
- `ComponentList`
- `Component`

Those classes in the SDMX metamodel which require annotations, global identity, multilingual name and multilingual description are derived from `NameableArtefact`. These are:

- `VersionableArtefact`
- `Item`

The classes in the SDMX metamodel which require annotations, global identity, multilingual name and multilingual description, and versioning are derived from `VersionableArtefact`. These are:

- `MaintainableArtefact`

Abstract classes which represent information that is maintained by Maintenance Agencies all inherit from `MaintainableArtefact`, they also inherit all the features of a `VersionableArtefact`, and are:

- `StructureUsage`
- `Structure`
- `ItemScheme`

All the above classes are abstract. The key to understanding the class diagrams presented in this section are the concrete classes that inherit from these abstract classes.

Those concrete classes in the SDMX Data Structure Definition and Dataset packages of the metamodel which require to be maintained by Agencies all inherit (via other abstract classes) from `MaintainableArtefact`, these are:

- `Dataflow`
- `DataStructureDefinition`

The component structures that are lists of lists, inherit directly from `Structure`. A `Structure` contains several lists of components. The concrete class that inherits from `Structure` is:

- `DataStructureDefinition`

A `DataStructureDefinition` contains a list of dimensions, a list of measures and a list of attributes.

The concrete classes which inherit from `ComponentList` and are subcomponents of the `DataStructureDefinition` are:

- `DimensionDescriptor` – content is `Dimension` and `TimeDimension`
- `DimensionGroupDescriptor` – content is an association to `Dimension`, `TimeDimension`
- `MeasureDescriptor` – content is `Measure`
- `AttributeDescriptor` – content is `DataAttribute` and an association to
- `MetadataAttribute`

Copyright © SDMx TWG 2020

The classes that inherit from `Component` are:

- `Measure`
- `DimensionComponent` and thereby its sub classes of `Dimension` and `TimeDimension`
- `Attribute` and thereby its sub classes of `DataAttribute` and `MetadataAttribute`

The concrete classes identified above are the majority of the classes required to define the metamodel for the `DataStructureDefinition`. The diagrams and explanations in the rest of this section show how these concrete classes are related in order to support the functionality required.

3.6.3 Data Structure Definition – Relationship View

Class Diagram

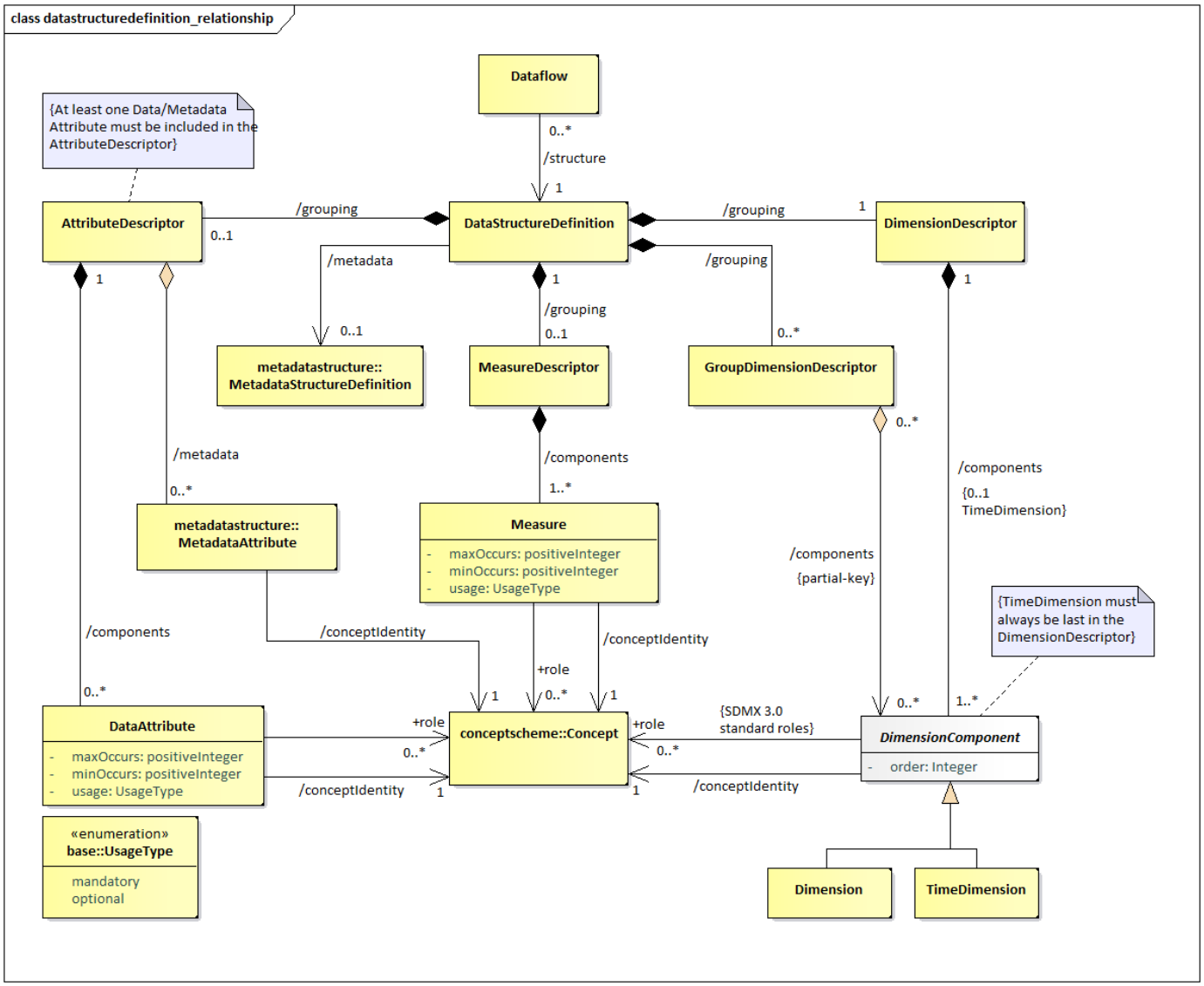


Figure 28. Relationship class diagram of the Data Structure Definition excluding representation

Copyright © SDMx TWG 2020

Explanation of the Diagrams

NARRATIVE

A DataStructureDefinition defines the Dimensions, TimeDimension, DataAttributes, and Measures, and associated Representations, that comprise the valid structure of data and related attributes that are contained in a DataSet, which is defined by a Dataflow. In addition, a DataStructureDefinition may be related to one MetadataStructureDefinition, in order to use the latter's MetadataAttributes, by relating them to other Components within the DSD, as explained below.

The Dataflow may also have additional metadata attached that define qualitative information and Constraints on the use of the DataStructureDefinition such as the subset of Codes used in a Dimension (this is covered later in this document – see sections Constraints and Data Provisioning). Each Dataflow has a maximum of one DataStructureDefinition specified which defines the structure of any DataSets to be reported/disseminated.

There are two types of dimensions each having a common association to Concept:

- Dimension
- TimeDimension

Note that DimensionComponent can be any or all its sub classes i.e., Dimension, TimeDimension.

The DimensionComponent, DataAttribute, MetadataAttribute and Measure link to the Concept that defines its name and semantic (/conceptIdentity association to Concept). The DataAttribute, Dimension (but not TimeDimension) and Measure can optionally have a +conceptRole association with a Concept that identifies its role in the DataStructureDefinition, or one of the standard pre-defined roles, i.e., those published in "GUIDELINES FOR SDMX CONCEPT ROLES" by the SDMX SWG. The use of these roles is to enable applications to process the data in a meaningful way (e.g., relating a dimension value to a mapping vector). It is expected, beyond the standard roles published by the SWG, that communities (such as the official statistics community) will harmonise such roles within their community so that data can be exchanged and shared in a meaningful way within that community.

The valid values for a DimensionComponent, Measure, DataAttribute or MetadataAttribute, when used in this DataStructureDefinition, are defined by the Representation. This Representation is taken from the Concept definition (coreRepresentation) unless it is overridden in this DataStructureDefinition (localRepresentation) – see Figure 28. Note also that TimeDimension is constrained to specific FacetValueTypes. Moreover, the Representations of MetadataAttributes are specified in the corresponding MetadataStructureDefinition, linked by the DataStructureDefinition.

There will always be a DimensionDescriptor grouping that identifies all of the Dimension comprising the full key. Together the Dimensions specify the key of an Observation.

The DimensionComponent can optionally be grouped by multiple GroupDimensionDescriptors each of which identifies the group of Dimensions that can form a partial key. The GroupDimensionDescriptor must be identified (GroupDimensionDescriptor.id) and this is used in the GroupKey of the DataSet to declare which DataAttributes or MetadataAttributes are reported at this group level in the DataSet.

There can be a maximum of one TimeDimension specified in the DimensionDescriptor. The TimeDimension is used to specify the Concept used to convey the time period of the observation in a data set. The TimeDimension must contain a valid representation of time and cannot be coded.

There can be one or more Measures under the MeasureDescriptor. Measures represent the observable phenomena. Each Measure may have a valid representation, a maxOccurs attribute limiting the maximum number of values per Measure (which may be set to 'unbounded' for unlimited occurrences), as well as a minOccurs attribute, indicating the minimum required number of values, when the Measure is reported. If minOccurs or maxOccurs are omitted (they both default to '1'), the Measure is considered to take a single value; otherwise, it is an array. Moreover, the usage attribute indicates whether a Measure must be reported or not, by the corresponding values: mandatory or optional.

The AttributeDescriptor may contain one or more Attributes, i.e., at least one DataAttribute definition or one MetadataAttribute reference.

The DataAttribute defines a characteristic of data that are collected or disseminated and is grouped in the DataStructureDefinition by a single AttributeDescriptor. The DataAttribute can be indicated if it must be reported or not, by the corresponding value of the usage attribute: i.e., mandatory or optional. The property minOccurs specifies the minimum number of array values to be included when the DataAttribute is reported. Moreover, a maxOccurs attribute indicates whether the

Copyright © SDMx TWG 2020

`DataAttribute` may need to report more than one values, i.e., an array of values. The `DataAttribute` may play a specific role in the structure and this is specified by the +role association to the `Concept` that identifies its role.

The `MetadataAttribute` defines reference metadata that may be collected or disseminated and is grouped together with `DataAttribute` under the `AttributeDescriptor`.

A `DataAttribute` or a `MetadataAttribute` (i.e., an `AttributeComponent`) is specified as being `+relatedTo` an `AttributeRelationship`, which defines the constructs to which the `AttributeComponent` is to be reported within a `DataSet`. An `AttributeComponent` can be specified as being related to one of the following artefacts:

- All data within the dataset (`DataflowRelationship`) – this is equivalent to attaching an Attribute to all data within the `Dataflow`.
- `Dimension` or set of `Dimension`s (`DimensionRelationship`)
- Set of `Dimension`s specified by a `GroupKey` (`GroupRelationship` – this is retained for compatibility reasons – or `+groupKey` of the `DimensionRelationship`)
- Observation (`ObservationRelationship`)
- In addition to the positioning of an `AttributeComponent` within a `DataSet`, another relationship indicates the `Measure`(s) for which the `AttributeComponent` is reported. Regardless of the position of the `AttributeComponent` within the `DataSet`, the `AttributeComponent` may concern one, more than one, or all `Measure`s included in the DSD. This is expressed using the `MeasureRelationship` class, which relates a `DataAttribute` to one or more `Measure`s. Lack of the `MeasureRelationship` defaults to a relationship to all `Measure`s.

Copyright © SDMx TWG 2020

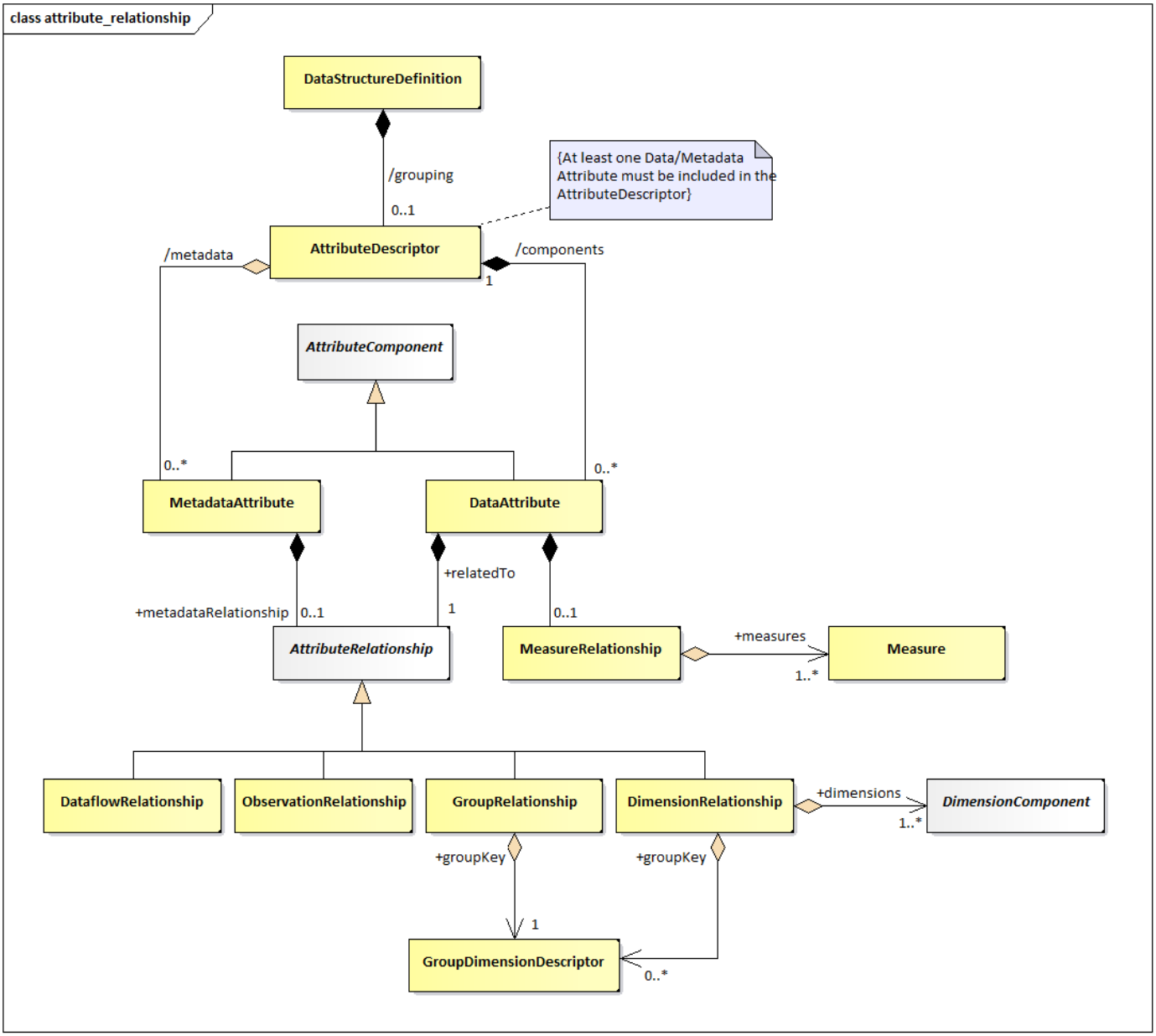


Figure 29. Attribute Attachment Defined in the Data Structure Definition

Copyright © SDMx TWG 2020

The following table details the possible relationships a `DataAttribute` may specify. Note that these relationships are mutually exclusive, and therefore only one of the following is possible.

Relationship	Meaning	Location in Data Set at which the Attribute is reported
`DataflowRelationship`	The value of the attribute is fixed for all data contained in the dataset. The attribute value applies to all data defined by the underlying `Dataflow`.	The attribute is reported at the dataset level.
`Dimension` (1..n)	The value of the attribute will vary with the value(s) of the referenced `Dimension`(s). In this case, group(s) to which the attribute should be attached may optionally be specified.	The attribute is reported at the lowest level of the `Dimension` to which the attribute is related, otherwise at the level of the group if attachment group(s) is specified.
`Group`	The value of the attribute varies with combination of values for all of the `Dimension`s contained in the group. This is added as a convenience to listing all `Dimension`s and the attachment group, but should only be used when the attribute value varies based on **all** group `Dimension` values.	The attribute is reported at the level of group.
`Observation`	The value of the attribute varies with the observed value.	The attribute is reported at the level of observation.

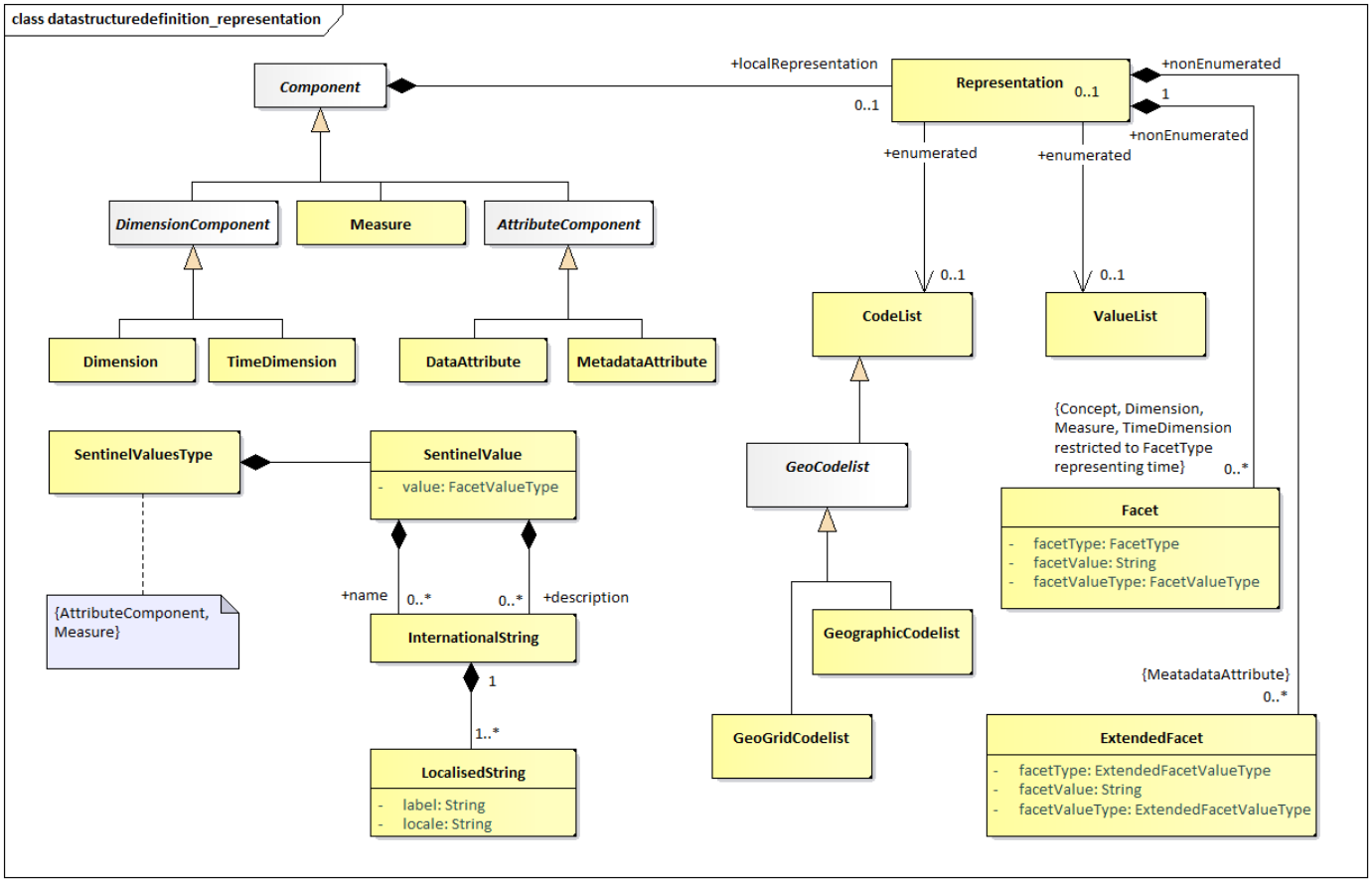


Figure 30. Representation of DSD Components

Each of `Dimension`, `TimeDimension`, `Measure`, `DataAttribute` and `MetadataAttribute` can have a `Representation` specified (using the `localRepresentation` association). If this is not specified in the `DataStructureDefinition` then the representation specified for

Copyright © SDMx TWG 2020

`Concept` (`coreRepresentation`) is used. `Measure`, and `DataAttribute` may be also represented by multilingual text (as seen in the `DataSet` diagram further down). An exception is the `MetadataAttribute`, where its `Representation` is specified in the `MetadataStructureDefinition`.

A `DataStructureDefinition` can be extended to form a derived `DataStructureDefinition`. This is supported in the `StructureMap`.

Copyright © SDMx TWG 2020

DEFINITIONS

Copyright © SDMx TWG 2020

Class	Feature	Description
`StructureUsage`		See “SDMX Base”.
`Dataflow`	Inherits from `StructureUsage`	Abstract concept (the structure without any data) of a flow of data that providers will provide for different reference periods.
	`/structure`	Associates a `Dataflow` to the `DataStructureDefinition`.
`DataStructureDefinition`		A collection of metadata concepts, their structure and usage when used to collect or disseminate data.
	`/grouping`	An association to a set of metadata concepts that have an identified structural role in a `DataStructureDefinition`.
`GroupDimensionDescriptor`	Inherits from `ComponentList`	A set of metadata concepts that define a partial key derived from the `DimensionDescriptor` in a `DataStructureDefinition`.
	`/components`	An association to the `Dimension` components that comprise the group.
`DimensionDescriptor`	Inherits from `ComponentList`	An ordered set of metadata concepts that, combined, classify a statistical series, and whose values, when combined (the key) in an instance such as a data set, uniquely identify a specific observation.
	`/components`	An association to the `Dimension` and `TimeDimension` comprising the key descriptor.
`AttributeDescriptor`	Inherits from `ComponentList`	A set of metadata concepts that define the attributes of a `DataStructureDefinition`.
	`/components`	An association to a `DataAttribute` component.
`MeasureDescriptor`	Inherits from `ComponentList`	A metadata concept that defines the `Measure`s of a `DataStructureDefinition`.
	`/components`	An association to a `Measure` component.
`DimensionComponent`		An abstract class representing any component

Copyright © SDMx TWG 2020

Class	Feature	Description
	Inherits from `Component` Sub class: `Dimension`, `TimeDimension`	that can be used for identifying observations.
	`order`	Specifies the order of the `DimensionComponent`s within the DSD. The property is used to indicate the position of the `DimensionComponent` and determines the key for identifying observations or series. The `TimeDimension`, when specified, must be the last within the `DimensionDescriptor`.
`Dimension`	Inherits from `DimensionComponent`	A metadata concept used (often together with other metadata concepts) to classify a statistical series, e.g., a statistical concept indicating a certain economic activity or a geographical reference area.
	`/role`	Association to the `Concept` that specifies the role that the `Dimension` plays in the `DataStructureDefinition`.
	`/conceptIdentity`	An association to the metadata concept which defines the semantic of the `Dimension`.
`TimeDimension`	Inherits from `DimensionComponent`	A metadata concept that identifies the component in the key structure that has the role of “time”.
`DataAttribute`	Inherits from `Component`	A characteristic of an object or entity.
	`/role`	Association to the `Concept` that specifies the role that the `DataAttribute` plays in the `DataStructureDefinition`.
	`minOccurs`	Defines the minimum required occurrences for the attribute. When equal to zero, the attribute is conditional.
	`maxOccurs`	Defines the maximum allowed occurrences for the attribute.
	`usage`	Defines whether a `DataAttribute` must be reported or not.

Copyright © SDMx TWG 2020

Class	Feature	Description
	`+relatedTo`	Association to an `AttributeRelationship`.
	`/conceptIdentity`	An association to the `Concept` which defines the semantic of the component.
`Measure`	Inherits from `Component`	The metadata concept that is the phenomenon to be measured in a data set. In a data set the instance of the measure is often called the observation.
	`/conceptIdentity`	An association to the `Concept` which carries the values of the measures.
	`minOccurs`	Defines the minimum required occurrences for the `Measure`. When equal to zero, the `Measure` is conditional.
	`maxOccurs`	Defines the maximum allowed occurrences for the `Measure`.
	`usage`	Defines whether a `Measure` must be reported or not.
`AttributeRelationship`	Abstract class Sub classes: `ObservationRelationship`, `GroupRelationship`, `DimensionRelationship`	Specifies the type of artefact to which a `DataAttribute` can be attached in a data set.
`ObservationRelationship`		The `DataAttribute` is related to the observations of the data set.
`GroupRelationship`		The `DataAttribute` is related to a `GroupDimensionDescriptor` construct.
	`+groupKey`	An association to the `GroupDimensionDescriptor`.
`DimensionRelationship`		The `DataAttribute` is related to a set of `Dimension`s.
	`+dimensions`	Association to the set of `Dimension`s to which the `DataAttribute` is related.
	`+groupKey`	Association to the `GroupDimensionDescriptor` which specifies the set of `Dimension`s to which the `DataAttribute` is attached.
`MeasureRelationship`		

Copyright © SDMx TWG 2020

Class	Feature	Description
		The `Measure`s that a `DataAttribute` is reported for.
	`+measures`	Association to the set of `Measure`s to which a `DataAttribute` is related.
`SentinelValuesType`		This facet indicates that an attribute or a `Measure` has sentinel values with special meaning within their data type. This is realised by providing such values within the `TextFormat`, in addition to any `textType` or other facet.
`SentinelValue`		A value that has a special meaning within the text format representation of the component.
	`+name`	An association of a sentinel value to a multilingual name.
	`+description`	An association of a sentinel value to a multilingual description.

The explanation of the classes, attributes, and associations comprising the `Representation` is described in the section on the SDMX Base.

3.6.4 Data Set – Relationship View

Context

A data set comprises the collection of data values and associated metadata that are collected or disseminated according to a known `DataStructureDefinition`.

Copyright © SDMx TWG 2020

Class Diagram

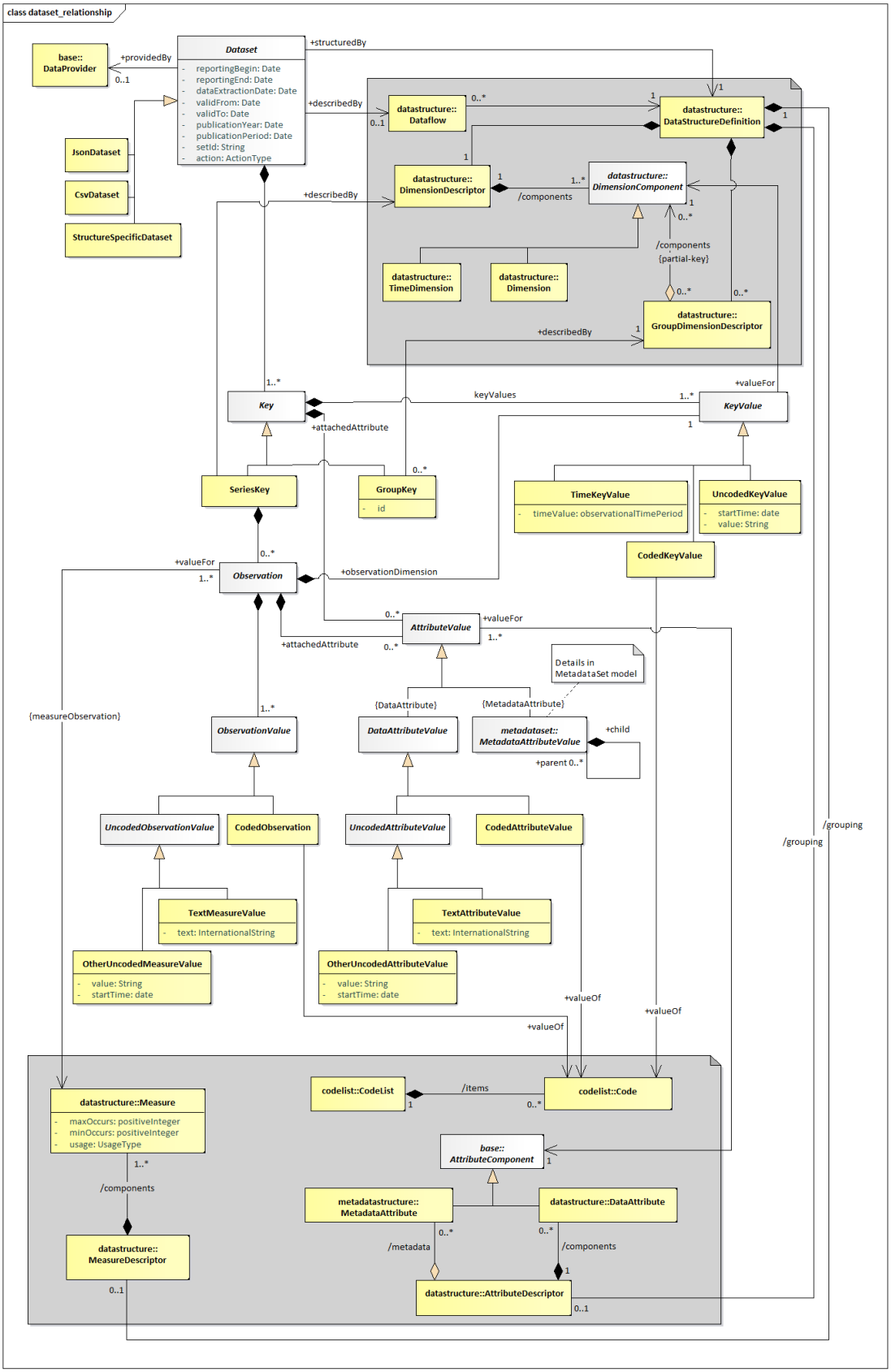


Figure 31. Class Diagram of the Data Set

Copyright © SDMx TWG 2020

Explanation of the Diagram

NARRATIVE – DATA SET

Note that the `DataSet` must conform to the `DataStructureDefinition` associated to the `Dataflow` for which this `DataSet` is an "instance of data". Whilst the model shows the association to the classes of the `DataStructureDefinition`, this is for conceptual purposes to show the link to the `DataStructureDefinition`. In the actual `DataSet` as exchanged there must, of course, be a reference to the `DataStructureDefinition` and optionally a `Dataflow` or a `ProvisionAgreement`, but the `DataStructureDefinition` is not necessarily exchanged with the data. Therefore, the `DataStructureDefinition` classes are shown in the grey areas, as these are not a part of the `DataSet` when the `DataSet` is exchanged. However, the structural metadata in the `DataStructureDefinition` can be used by an application to validate the contents of the `DataSet` in terms of the valid content of a `KeyValue` as defined by the `Representation` in the `DataStructureDefinition`.

An organisation playing the role of `DataProvider` can be responsible for one or more `DataSet`.

A `DataSet` is formatted as a `DataStructureDefinition` specific data set (`StructureSpecificDataSet`). The structured data set is structured according to one specific `DataStructureDefinition`; hence the latter is required for validation at the syntax level.

A `DataSet` is a collection of a set of `Observation`s that share the same dimensionality, which is specified by a set of unique components (`Dimension`, `TimeDimension`) defined in the `DimensionDescriptor` of the `DataStructureDefinition`, together with associated `AttributeValue`s that define specific characteristics about the artefact to which it is attached – `Observation`s, set of `Dimension`s. It can be structured in terms of a `SeriesKey` to which `Observation`s are reported.

The `Observation` can be the value(s) of the variable(s) being measured for the `Concept` associated to the `Measure`(s) in the `MeasureDescriptor` of the `DataStructureDefinition`. Each `Observation` associates one or more `ObservationValue`s with a `KeyValue` (`+observationDimension`) which is the value for the "`Dimension` at the Observation Level". Any `Dimension` can be specified as being the "`Dimension` at the Observation Level", and this specification is made at the level of the `DataSet` (i.e., it must be the same `Dimension` for the entire `DataSet`).

The `KeyValue` is a value for one of `TimeDimension` or `Dimension` specified in the `DataStructureDefinition`. If it is a `Dimension`, it can be coded (`CodedKeyValue`) or uncoded (`UncodedKeyValue`). If it is the `TimeDimension` then it is a `TimeKeyValue`. The actual value that the `Coded` Dimension `Value` can take must be one of the Codes in the Codelist specified as the `Representation` of the `Dimension` in the `DataStructureDefinition`.

An `ObservationValue` can be coded – this is the `CodedObservation` – or it can be uncoded – this is the `UncodedObservation`. In the case of uncoded observations, the values may be multilingual – expressed via the `Text` Measure `Value` – or not (`OtherUncoded` Measure `Value`).

The `GroupKey` is a subunit of the `Key` that has the same dimensionality as the `SeriesKey` but defines a subset of the `KeyValues` of the `SeriesKey`. Its sub dimension structure is defined in the `Group` Dimension `Descriptor` of the `DataStructureDefinition` identified by the same id as the `GroupKey`. The id identifies a "type" of group and the purpose of the `GroupKey` is to report one or more `AttributeValue` that are contained at this group level. The `GroupKey` is present when the `Group` Dimension `Descriptor` is related to the `GroupRelationship` in the `DataStructureDefinition`. There can be many types of groups in a `DataSet`. If the Group is related to the `DimensionRelationship` in the `DataStructureDefinition` then the `AttributeValue` will be reported with the appropriate dimension in the `SeriesKey` or Observation.

In this way each of `SeriesKey`, `GroupKey`, and Observation can have zero or more `AttributeValue`s that define some metadata about the object to which it is associated. The `AttributeValue` may be either a `DataAttributeValue` or a `MetadataAttributeValue`, representing values of `DataAttributes` defined in the DSD or `MetadataAttributes` of the linked MSD, respectively. The allowable `Concept`s and the objects to which these metadata can be associated (attached) are defined in the `DataStructureDefinition` and the linked `MetadataStructureDefinition`.

The `AttributeValue` links to the object type (`SeriesKey`, `GroupKey`, Observation) to which it is associated.

Copyright © SDMx TWG 2020

DEFINITIONS

Copyright © SDMx TWG 2020

Class	Feature	Description
`DataSet`	Abstract class. Subclasses: `StructureSpecificDataSet`	An organised collection of data.
	`reportingBegin`	A specific time period in a known system of time periods that identifies the start period of a report.
	`reportingEnd`	A specific time period in a known system of time periods that identifies the end period of a report.
	`dataExtractionDate`	A specific time period that identifies the date and time that the data are extracted from a data source.
	`validFrom`	Indicates the inclusive start time indicating the validity of the information in the data set.
	`validTo`	Indicates the inclusive end time indicating the validity of the information in the data set.
	`publicationYear`	Specifies the year of publication of the data or metadata in terms of whatever provisioning agreements might be in force.
	`publicationPeriod`	Specifies the period of publication of the data or metadata in terms of whatever provisioning agreements might be in force.
	`setId`	Provides an identification of the data set.
	`action`	Defines the action to be taken by the recipient system (`information` , `append` , `replace` , `delete`).
	`describedBy`	Associates a `Dataflow` and thereby a Data Structure Definition to the data set.
	`structuredBy`	Associates the Data Structure Definition that defines the structure of the data set. Note that the Data Structure Definition is the same as that associated (non-mandatory) to the `Dataflow` .
	`publishedBy`	Associates the Data Provider that reports/publishes the data.
`StructureSpecificDataSet`		An XML specific data format structure that contains data corresponding to one specific Data Structure Definition.
`Key`	Abstract class. Subclasses: `SeriesKey` , `GroupKey`	Comprises the cross product of values of dimensions that identify uniquely an Observation.
	`keyValues`	

Copyright © SDMx TWG 2020

Class	Feature	Description
		Association to the individual Key Values that comprise the Key.
	`attachedAttribute`	Association to the Attribute Values relating to the Series Key or Group Key.
`KeyValue`	Abstract class. Subclasses: `TimeKeyValue`, `CodedKeyValue`, `UncodedKeyValue`	The value of a component of a key such as the value of the instance of a `Dimension` in a `DimensionDescriptor` of a Data Structure Definition.
	`valueFor`	Association to the key component in the Data Structure Definition for which this Key Value is a valid representation. Note that this is a conceptual association as the key component is identified explicitly in the data set.
`TimeKeyValue`	Inherits from `KeyValue`	The value of the Time `Dimension` component of the key.
`CodedKeyValue`	Inherits from `KeyValue`	The value of a coded component of the key. The value is the Code to which this class is associated.
	`valueOf`	Association to the Code. Note that this is a conceptual association showing that the Code must exist in the Code list associated with the `Dimension` in the Data Structure Definition. In the actual Data Set the value of the Code is placed in the Key Value.
`UncodedKeyValue`	Inherits from `KeyValue`	The value of an uncoded component of the key.
	`value`	The value of the key component.
	`startTime`	This attribute is only used if the `textFormat` of the attribute is of the Timespan type in the Data Structure Definition (in which case the value field takes a duration).
`GroupKey`	Inherits from `Key`	A set of Key Values that comprise a partial key, of the same dimensionality as the Time Series Key for the purpose of attaching Data Attributes.
	`describedBy`	Associates the Group `DimensionDescriptor` defined in the Data Structure Definition.
`SeriesKey`	Inherits from `Key`	Comprises the cross product of values of all the Key Values that, together with the Key Value of the observation `Dimension`, identify uniquely an Observation.

Copyright © SDMx TWG 2020

Class	Feature	Description
	`describedBy`	Associates the `DimensionDescriptor` defined in the Data Structure Definition.
`Observation`		The value(s) of the observed phenomenon in the context of the Key Values comprising the key.
	`valueFor`	Associates the `Measure`(s) defined in the Data Structure Definition. The source multiplicity (1..*) indicates that more than one value may be provided for a `Measure`, if the latter allows it.
	`attachedAttribute`	Association to the Attribute Values relating to the Observation.
	`observationDimension`	Association to the Key Value that holds the value of the “Dimension at the Observation Level”.
`ObservationValue`	Abstract class. Subclasses: `UncodedObservationValue`, `CodedObservation`	
`UncodedObservationValue`	Abstract class. Inherits from `ObservationValue`. Subclasses: `OtherUncodedMeasureValue`, `TextMeasureValue`	
`OtherUncodedMeasureValue`	Inherits from `UncodedObservationValue`	An observation that has a text value.
	`value`	The value of the Uncoded Observation.
	`startTime`	This attribute is only used if the `textFormat` of the `Measure` is of the Timespan type in the Data Structure Definition (in which case the value field takes a duration).
`TextMeasureValue`	Inherits from `UncodedObservationValue`	An observation that has a localised text value.
	`text`	The localised text values.
`CodedObservation`	Inherits from `ObservationValue`	An Observation that takes its value from a code in a Code list.
	`valueOf`	Association to the Code that is the value of the Observation. Note that this is a conceptual association showing that the Code must exist in the Codelist(s) associated with the `Measure`(s) in the Data Structure Definition. In the actual Data Set the value of the Code is placed in the Observation.
`AttributeValue`	Abstract class. Subclasses: `DataAttributeValue`, `MetadataAttributeValue`	Represents the value for any Attribute reported in the Dataset, i.e., Data or Metadata Attribute.
`DataAttributeValue`	Abstract class. Inherits from `AttributeValue`. Subclasses: `UncodedAttributeValue`, `CodedAttributeValue`	The value of a Data Attribute, such as the instance of a Coded Attribute or of

Copyright © SDMx TWG 2020

Class	Feature	Description
		an Uncoded Attribute in a structure such as a Data Structure Definition.
	`valueFor`	Association to the Data Attribute defined in the Data Structure Definition. Note that this is conceptual association as the `Concept` is identified explicitly in the data set. The source multiplicity (1..*) indicates the possibility to provide more than one value for a Data Attribute, if the latter allows it.
`MetadataAttributeValue`	(explained further in section “Metadata Set”)	The value of a Metadata Attribute, as specified in the Metadata Structure Definition, which is linked in the Data Structure Definition.
`UncodedAttributeValue`	Abstract class. Inherits from `AttributeValue`. Subclasses: `OtherUncodedAttributeValue`, `TextAttributeValue`	
`OtherUncodedAttributeValue`	Inherits from `UncodedAttributeValue`	An attribute value that has a text value.
	`value`	The value of the Uncoded attribute.
	`startTime`	This attribute is only used if the `textFormat` of the attribute is of the Timespan type in the Data Structure Definition (in which case the value field takes a duration).
`TextAttributeValue`	Inherits from `UncodedAttributeValue`	An attribute that has a localised text value.
	`text`	The localised text values.
`CodedAttributeValue`	Inherits from `AttributeValue`	An attribute that takes its value from a Code in Code list.
	`valueOf`	Association to the Code that is the value of the Attribute Value. Note that this is a conceptual association showing that the Code must exist in the Code list associated with the Data Attribute in the Data Structure Definition. In the actual Data Set the value of the Code is placed in the Attribute Value.

Copyright © SDMx TWG 2020

3.7 Cube

3.7.1 Context

Some statistical systems create views of data based on a “cube” structure. In essence, a cube is an n-dimensional object where the value of each dimension can be derived from a hierarchical code list. The utility of such cube systems is that it is possible to “roll up” or “drill down” each of the hierarchy levels for each of the dimensions to specify the level of granularity required to give a “view” of the data – some dimensions may be rolled up, others may be drilled down. Such systems give a dynamic view of the data, with aggregated values for rolled up dimension positions. For example, the individual countries may be rolled up into an economic region such as the EU, or a geographical region such as Europe, whilst another dimension, such as “type of road” may be drilled down to its lower level. The resulting measure (such as “number of accidents”) would then be an aggregation of the value for each individual country for the specific type of road.

Such cube systems rely, not on simple code lists, but on hierarchical code sets (see Section Hierarchy).

3.7.2 Support for the Cube in the Information Model

Data reported using a Data Structure Definition structure (where each dimension value, if coded, is taken from a flat code list) can be described by a cube definition and can be processed by cube aware systems. The SDMX-IM supports the definition of such cubes in the following way:

- The `Hierarchy` defines the (often complex) hierarchies of codes.
- If required:
  - The `StructureMap` can group `DataStructureDefinition` that describe the cube
  - The `HierarchyAssociation` can provide a mechanism to apply a `Hierarchy` to the `Code`s in the `Codelist`s used by the `DataStructureDefinition`, providing also the context of which the hierarchy applies (e.g., a `Dataflow`).

Copyright © SDMx TWG 2020

3.8 Metadata Structure Definition and Metadata Set

3.8.1 Context

Besides the possibility to extend the components of Data Structure Definitions by metadata attributes defined in Metadata Structure Definitions, the SDMX metamodel allows metadata to describe any identifiable artefact. These metadata can be:

1. Exchanged without the need to embed it within the object that it is describing.
2. Stored separately from the object that it describes, yet be linked to it (for example, an organisation has a metadata repository which supports the dissemination of metadata resulting from metadata requests generated by systems or services that have access to the object for which the metadata pertains. This is common in web dissemination where additional metadata is available for viewing (and eventually downloading) by clicking on an “information” icon next to the object to which the metadata is attached).
3. Versioned and maintained like structural metadata, but from Metadata Providers than Agencies.
4. Reported according to a defined structure.

In order to achieve this, the following structures are modelled:

- The Metadata Structure Definition which comprises the metadata attributes that can be attached to the various object types (these attributes can be structured in a hierarchy), together with any constraints that may apply (e.g., association to a code list that contains valid values for the attribute when reported in a metadata set),
- The `Metadataflow` and/or Metadata Provision Agreement, which contains the objects to which the metadata are to be associated (attached),
- The Metadata Set, which contains reported metadata.

3.8.2 Inheritance

Introduction

As with the Data Structure Definition Structure, many of the constructs in this layer of the model inherit from the SDMX Base layer. Therefore, it is necessary to study both the inheritance and the relationship diagrams to understand the functionality of individual packages. The diagram below shows the full inheritance tree for the classes concerned with the `MetadataStructureDefinition`, the `MetadataProvisionAgreement`, the `Metadataflow` and the `MetadataSet`.

There are very few additional classes in the `MetadataStructureDefinition` package that do not themselves inherit from classes in the SDMX Base. In other words, the SDMX Base gives most of the structure of this sub model both in terms of associations and in terms of attributes. The relationship diagrams shown in this section show clearly when these associations are inherited from the SDMX Base (see the Appendix A Short Guide to UML in the SDMX Information Model to see the diagrammatic notation used to depict this).

Copyright © SDMx TWG 2020

Class Diagram - Inheritance

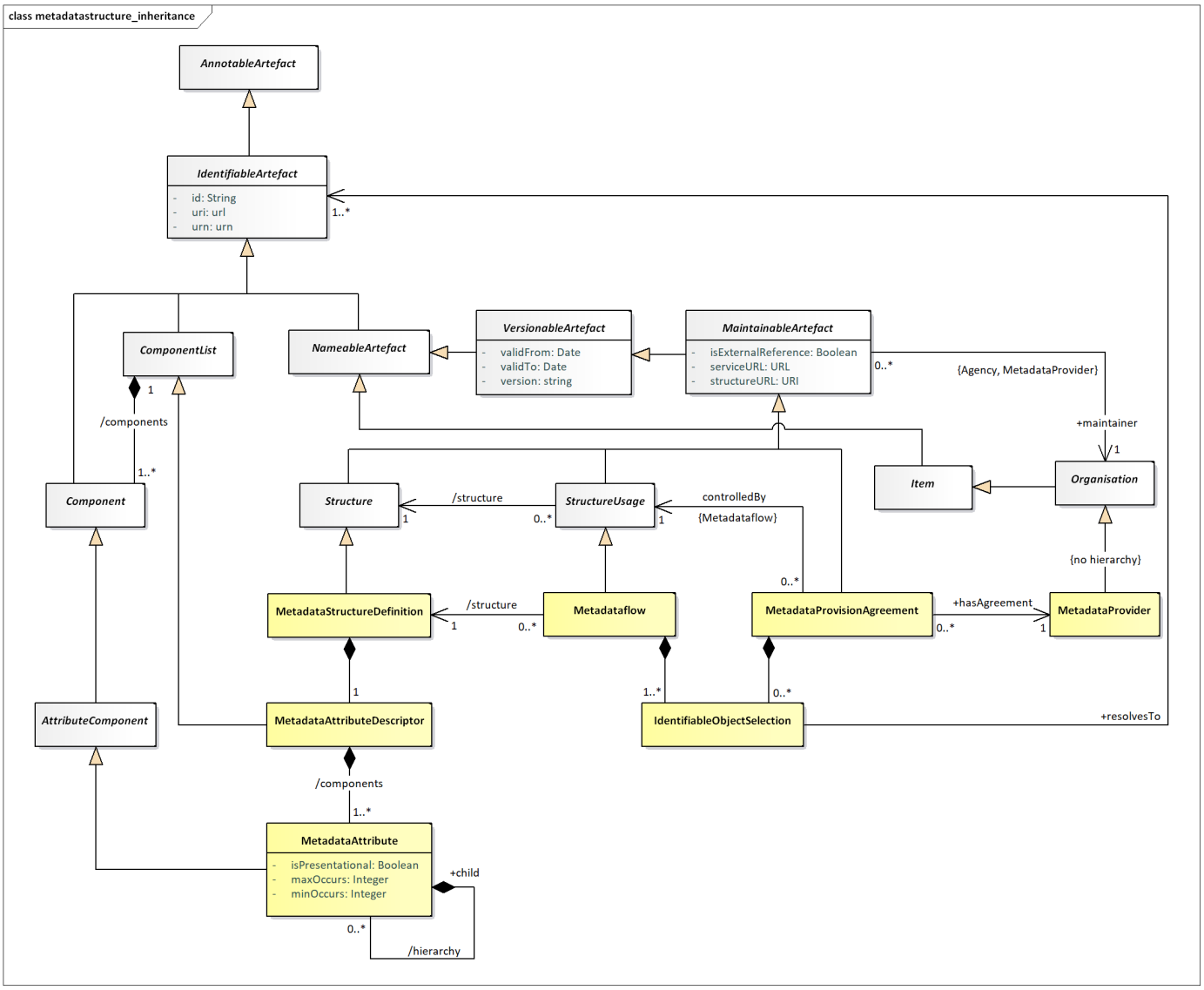


Figure 32. Inheritance class diagram of the Metadata Structure Definition

Explanation of the Diagram

NARRATIVE

It is important to the understanding of the relationship class diagrams presented in this section to identify the concrete classes that inherit from the abstract classes.

The concrete classes in this part of the SDMX metamodel, which require to be maintained by Maintenance Agencies, all inherit from `MaintainableArtefact`. These are:

- `StructureUsage` (concrete class is `Metadataflow`)
- `Structure` (concrete class is `MetadataStructureDefinition`)
- `MetadataProvisionAgreement`

These classes also inherit the identity and versioning facets of `IdentifiableArtefact`, `NameableArtefact` and `VersionableArtefact`.

Copyright © SDMx TWG 2020

A `Structure` may contain several lists of components. In this case the `MetadataStructureDefinition` acts as a list and contains `Component`s, i.e., `MetadataAttributes`.

3.8.3 Metadata Structure Definition

Introduction

The diagrams and explanations in the rest of this section show how these concrete classes are related in order to support the required functionality.

Structures Already Described

The `MetadataStructureDefinition` only contains `MetadataAttributes`, since target objects are contained in `Metadataflow` and `MetadataProvisionAgreement`, since SDMX 3.0.

Class Diagram – Relationship

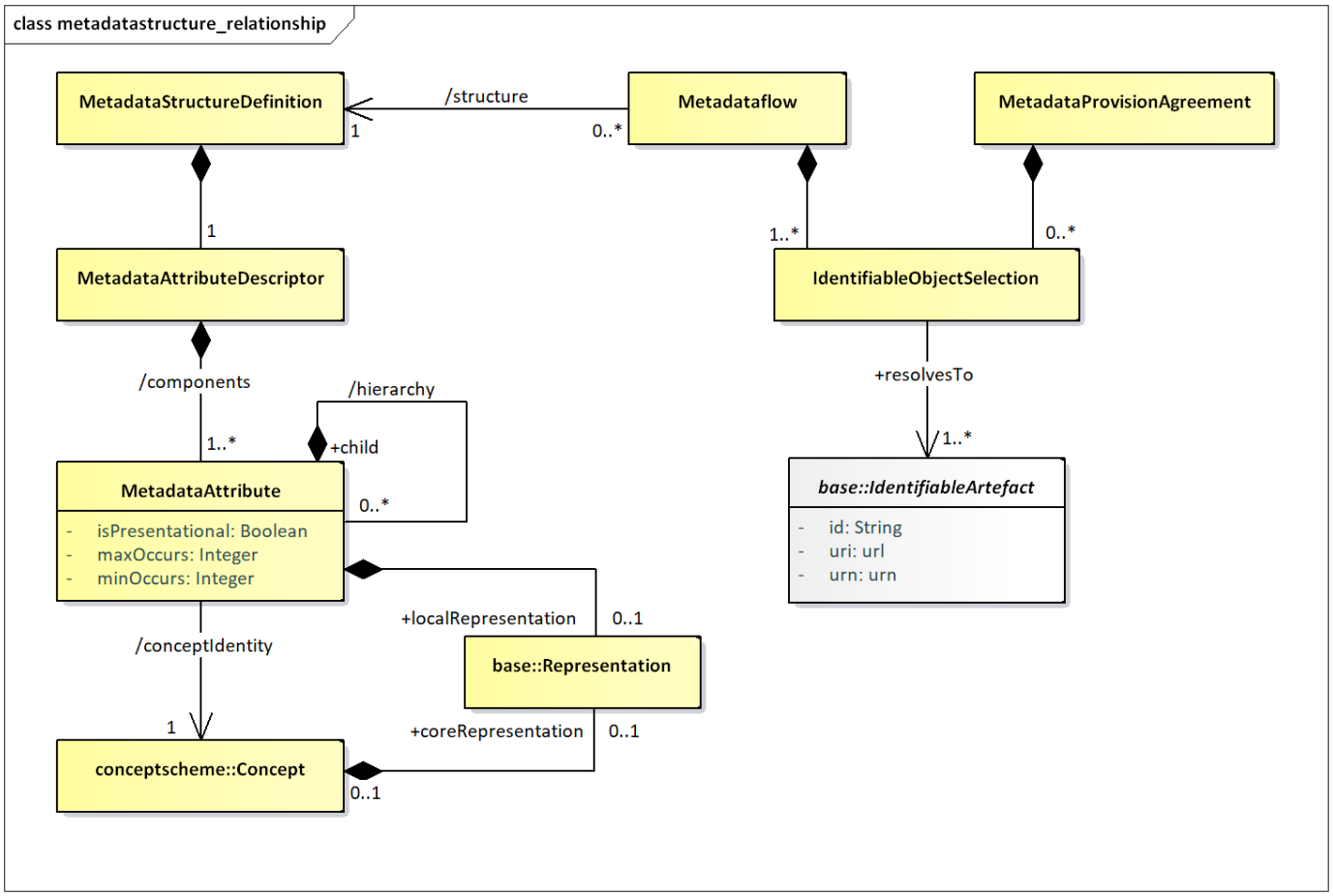


Figure 33. Relationship class diagram of the Metadata Structure Definition

Explanation of the Diagram

NARRATIVE

In brief, a `MetadataStructureDefinition` (MSD) defines the `MetadataAttributes`, within an `MetadataAttributeDescriptor`, that can be associated with the objects identified in the `Metadataflow`s and `MetadataProvisionAgreements` that refer to the MSD. The hierarchy of the `MetadataAttributes` is specified within the `MetadataAttributeDescriptor`.

Copyright © SDMx TWG 2020

The `MetadataAttributeDescriptor` comprises a set of `MetadataAttributes` – these can be defined as a hierarchy. Each `MetadataAttribute` identifies a Concept that is reported or disseminated in a `MetadataSet` (`/conceptIdentity`) that uses this `MetadataStructureDefinition` . Different `MetadataAttributes` in the same `MetadataAttributeDescriptor` can use Concepts from different `ConceptSchemes` . Note that a `MetadataAttribute` does not link to a Concept that defines its role in this `MetadataStructureDefinition` (i.e., the `MetadataAttribute` does not play a role).

The `MetadataAttribute` can be specified as having multiple occurrences and/or specified as being mandatory (`minOccurs` =1 or more) or optional (`minOccurs` =0). A hierarchical `MetadataStructureDefinition` can be defined by specifying a hierarchy for a `MetadataAttribute` .

It can be seen from this, that the specification of the objects to which a `MetadataAttribute` can be attached is indirect: the `MetadataAttributes` are defined in a `MetadataStructureDefinition` , but they are attached to one or more `IdentifiableArtefacts` as defined in the `Metadataflow` s or `MetadataProvisionAgreements` . This gives a flexible mechanism by which the actual objects need not be defined in concrete terms in the model but are defined dynamically by the `IdentifiableObjectSelection` . In this way, the `MetadataStructureDefinition` can be used to define any set of `MetadataAttributes` regardless of the objects to which they can be attached.

Each `MetadataAttribute` can have a `Representation` specified (using the `/localRepresentation` association). If this is not specified in the `MetadataStructureDefinition` then the `Representation` is taken from that defined for the Concept (the `coreRepresentation` association).

The definition of the various types of `Representation` can be found in the specification of the Base constructs. Note that if the `Representation` is non-enumerated then the association is to the `ExtendedFacet` (which allows for XHTML as a `FacetValueType`). If the `Representation` is enumerated, then is must use a `Codelist` .

The `Metadataflow` is linked to a `MetadataStructureDefinition` . The `Metadataflow` , in addition to the attributes inherited from the Base classes, it also has a list of `IdentifiableObjectSelection` constructs, which resolve into the `IdentifiableArtefacts` that the Metadatasets will refer to. The `IdentifiableObjectSelection` acts like a reference, but it may also include wildcarding part of the reference terms.

The `MetadataProvisionAgreement` is linked to a `Metadataflow` . The former, like the `Metadataflow` , may have `IdentifiableObjectSelection` constructs to be used for specifying the proper targets for reference metadata.

Copyright © SDMx TWG 2020

DEFINITIONS

Class	Feature	Description
`StructureUsage`		See “SDMX Base”.
`Metadataflow`	Inherits from: `StructureUsage`	Abstract concept (the structure without any metadata) of a flow of metadata that providers will provide for different reference periods. Specifies possible targets for metadata, via the `IdentifiableObjectSelection`.
	`/structure`	Associates a `MetadataStructureDefinition`.
`MetadataProvisionAgreement`		Links the `MetadataProvider` to the relevant `StructureUsage` (that is, `Metadataflow`) for which the provider supplies metadata. The agreement may constrain the scope of the metadata that can be provided, by means of a `Constraint`. Specifies possible targets for metadata, via the `IdentifiableObjectSelection`.
`MetadataProvider`		See Organisation Scheme.
`IdentifiableObjectSelection`		A list or wildcarded expression resolving into identifiable objects that metadata will refer to.
`MetadataStructureDefinition`	Inherits from: `MaintainableArtefact`	A collection of metadata concepts and their structure when used to collect or disseminate reference metadata.
`MetadataAttributeDescriptor`	Inherits from: `ComponentList`	Defines a set of concepts that comprises the metadata attributes to be reported.
	`/components`	An association to the metadata attributes relevant to the `MetadataAttributeDescriptor`.
`MetadataAttribute`		Identifies a concept for which a value may be reported in a `MetadataSet`.
	`/hierarchy`	Association to one or more child metadata attribute.
	`/conceptIdentity`	An association to the concept which defines the semantic of the attribute.
	`isPresentational`	Indication that the metadata attribute is present for structural purposes (it has child attributes) and that no value for this attribute is expected to be reported in a `MetadataSet`.
	`minOccurs`, `maxOccurs`	Specifies how many occurrences of the metadata attribute may be reported at this point in the metadata set.
	`/localRepresentation`	Associates a `Representation` that overrides any core representation specified for the concept itself.
`Representation`		The representation of the metadata attribute.

Copyright © SDMx TWG 2020

3.8.4 Metadata Set

Class Diagram

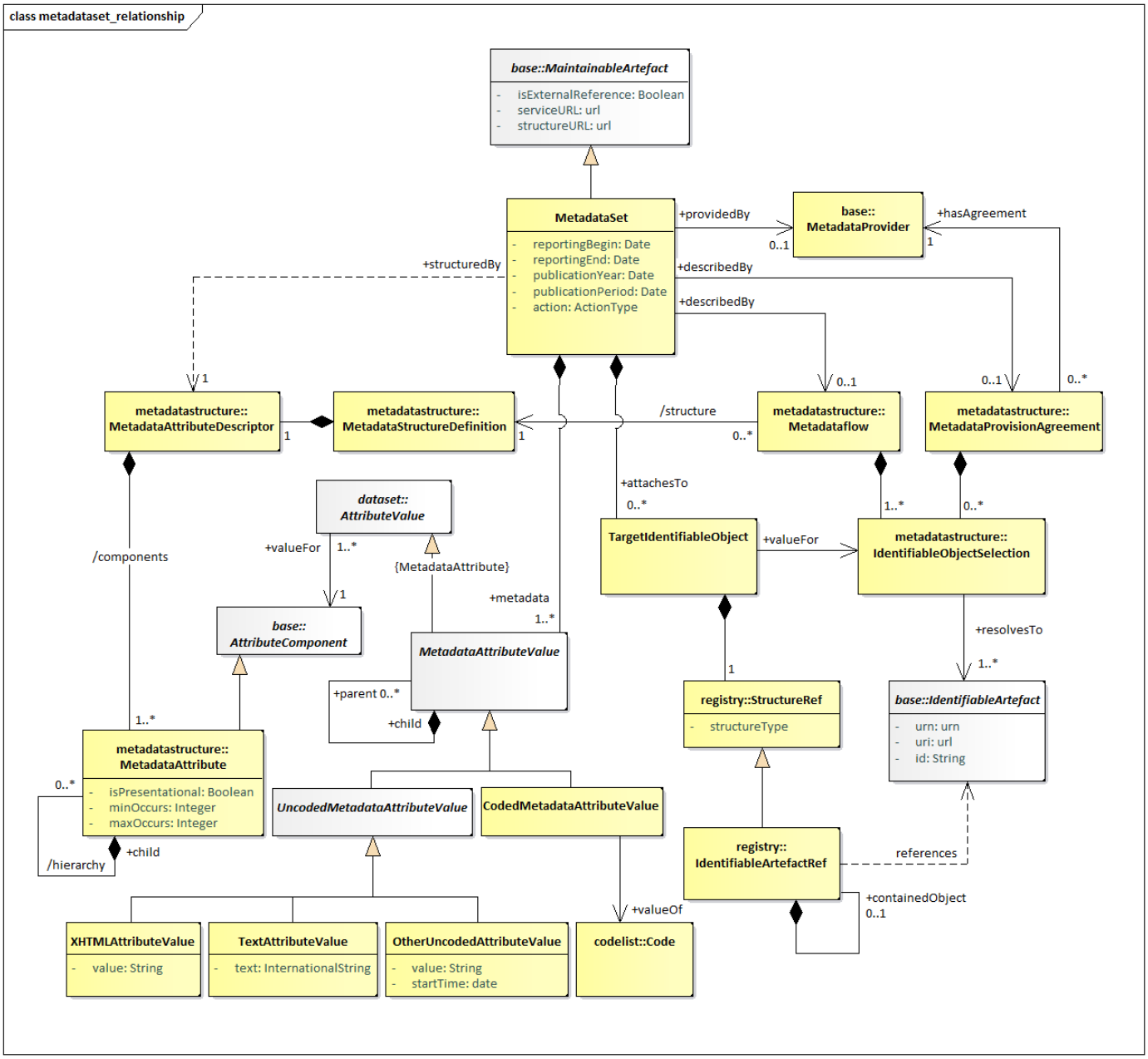


Figure 34. Relationship Class Diagram of the Metadata Set

Explanation of the Diagram

NARRATIVE

Note that the `MetadataSet` must conform to the `MetadataStructureDefinition` associated to the `Metadataflow` or `MetadataProvisionAgreement` for which this `MetadataSet` is an “instance of metadata”. Whilst the model shows the association to the classes of the `MetadataStructureDefinition`, this is for conceptual purposes to show the link to the `MetadataStructureDefinition`. In the actual `MetadataSet`, as exchanged, there must, of course, be a reference to the `MetadataStructureDefinition` and optionally a `Metadataflow` or a `MetadataProvisionAgreement`, but the `MetadataStructureDefinition` is not necessarily exchanged with the metadata. Note that the `MetadataStructureDefinition` classes are shown also but are not a part of the `MetadataSet` itself.

Copyright © SDMx TWG 2020

A `MetadataProvider` is maintaining one or more `MetadataSets`, as the latter is a `MaintainableArtefact`.

A `MetadataSet` comprises a set of `MetadataAttributeValue`s and a set of `TargetIdentifiableObjects`, which must be part of those specified in the relevant `Metadataflow` or `MetadataProvisionAgreement`.

The `MetadataStructureDefinition` specifies which `MetadataAttributes` are expected as `MetadataAttributeValue`s. The `TargetIdentifiableObjects` point to the `IdentifiableArtefact`s for which the `MetadataAttributeValue`s are reported.

A simple text value for the `MetadataAttributeValue` uses the `UncodedMetadataAttributeValue` sub class of `MetadataAttributeValue` whilst a coded value uses the `CodedMetadataAttributeValue` sub class.

The `UncodedMetadataAttributeValue` can be one of:

- `XHTMLAttributeValue` – the content is XHTML,
- `TextAttributeValue` – the content is textual and may contain the text in multiple languages,
- `OtherUncodedAttributeValue` – the content is a string value that must conform to the `Representation` specified for the `MetadataAttribute` in the `MetadataStructureDefinition`.

The `CodedMetadataAttributeValue` contains a value for a Code specified as the `Representation` for a `MetadataAttribute` in the `MetadataStructureDefinition`.

Copyright © SDMx TWG 2020

DEFINITIONS

Copyright © SDMx TWG 2020

Class	Feature	Description
`MetadataSet`		Any organised collection of
	`reportingBegin`	A specific time period in a l system of time periods tha the start period of a report
	`reportingEnd`	A specific time period in a l system of time periods tha the end period of a report.
	`publicationYear`	Specifies the year of public the data or metadata in ter whatever provisioning agre might be in force.
	`publicationPeriod`	Specifies the period of pub the data or metadata in ter whatever provisioning agre might be in force.
	`action`	Defines the action to be tak recipient system (`informat` `append` , `replace` , `delete`).
	`+describedBy`	Associates a `Metadataflow` Metadata Provision Agreem `MetadataSet` .
	`+structuredBy`	Associates the `MetadataAttributeDescripto` `MetadataStructureDefinitio` defines the structure of the `MetadataSet` . Note that this dependency explains that t `MetadataSet` is structured a the `MetadataStructureDefin` the linked (by the `+describ` `Metadataflow` or the Metada Provision Agreement.
	`+publishedBy`	Associates the Data Provid reports or publishes the me
	`+attachesTo`	Associates the target ident objects to which metadata attached.
	`+metadata`	Associates the Metadata At values which are to be asso with the object or objects i the Target Identifiable Obje
`TargetIdentifiableObject`		Specifies the identification identifiable object.
	`+valueFor`	Associates the Target Ident Object being a part of the I Object Selection specified i Dataflow or Metadata Provi Agreement.

Copyright © SDMx TWG 2020

Class	Feature	Description
`StructureRef`		Contains the identification identifiable object.
	`structureType`	The object type of the targ
`IdentifiableArtefactRef`		Identification of the target
	`+containedObject`	Association to a contained hierarchy of identifiable ob as a Transition in a Process
`MetadataAttributeValue`	*Abstract class* Sub classes are: *`UncodedMetadataAttributeValue`*, `CodedMetadataAttributeValue`	The value for a Metadata A
	`+valueFor` (inherited from the *`AttributeValue`*)	Association to the Metadat in the `MetadataStructureDef` that identifies the Concept allowed `Representation` for Metadata Attribute value. N this is a conceptual associa showing the link to the MSI construct. The syntax for th Metadata Attribute value w some form, the id of the Me Attribute.
	`+child`	Association to a child Meta Attribute value consistent w hierarchy defined in the MS Metadata Attribute for whic is a Metadata Attribute valu
`UncodedMetadataAttributeValue`	Inherits from *`MetadataAttributeValue`* Sub class: `XHTMLAttributeValue`, `TextAttributeValue`, `OtherUncodedAttributeValue`	The content of a Metadata value where this is textual.
`XHTMLAttributeValue`		This contains XHTML.
	`value`	The string value of the XHT
`TextAttributeValue`		This value of a Metadata At value where the content is readable text.
	`text`	The string value is text. Th present in multiple languag
`OtherUncodedAttributeValue`		The value of a Metadata At value where the content is human-readable text.
	`value`	A text string that is consist format to that defined in th `Representation` of the Meta Attribute for which this is a Attribute value.
	`startTime`	This attribute is only used i `textFormat` of the Metadata is of the Timespan type in t `MetadataStructureDefinitio` case the value field takes a

Copyright © SDMx TWG 2020

Class	Feature	Description
`CodedMetadataAttributeValue`	Inherits from *`MetadataAttributeValue`*	The content of a Metadata value that is taken from a C Code list.
	`value`	The Code value of the Meta Attribute value.
	`+value`	Association to a Code in th specified in the `Representa` Metadata Attribute for whic Metadata Attribute value is Note that this shows the co link to the Item that is the reality, the value itself will contained in the Coded Me Attribute Value.

Copyright © SDMx TWG 2020

3.9 Hierarchy

3.9.1 Scope

The `Codelist` described in the section on structural definitions supports a simple hierarchy of `Code`s and restricts any child `Code` to having just one parent `Code`. Whilst this structure is useful for supporting the needs of the `DataStructureDefinition` and the `MetadataStructureDefinition`, it may not be sufficient for supporting the more complex associations between codes that are often found in coding schemes such as a classification scheme. Often, the `Codelist` used in a `DataStructureDefinition` is derived from a more complex coding scheme. Access to such a coding scheme can aid applications, such as OLAP applications or data visualisation systems, to give more views of the data than would be possible with the simple `Codelist` used in the `DataStructureDefinition`. A `Hierarchy` may be linked to an `IndentifiableArtefact`, in order to assist

Note that a `Hierarchy` is not necessarily a balanced tree. A balanced tree is where levels are pre-defined and fixed, (i.e. a level always has the same set of codes, and any code has a fixed parent and child relationship to other codes). A statistical classification is an example of a balanced tree, and the support for a balanced hierarchy is a subset, and special case, of hierarchies.

The principal features of the `Hierarchy` are:

1. A child code can have more than one parent.
2. There can be more than one code that has no parent (i.e. more than one “root node”).
3. The levels in a hierarchy can be explicitly defined or they can be implicit: i.e. they exist only as parent/child relationships in the coding structure.
4. Hierarchies may be associated to the structures they refer to, via the `HierarchyAssociation`.

Copyright © SDMx TWG 2020

3.9.2 Inheritance

Class Diagram

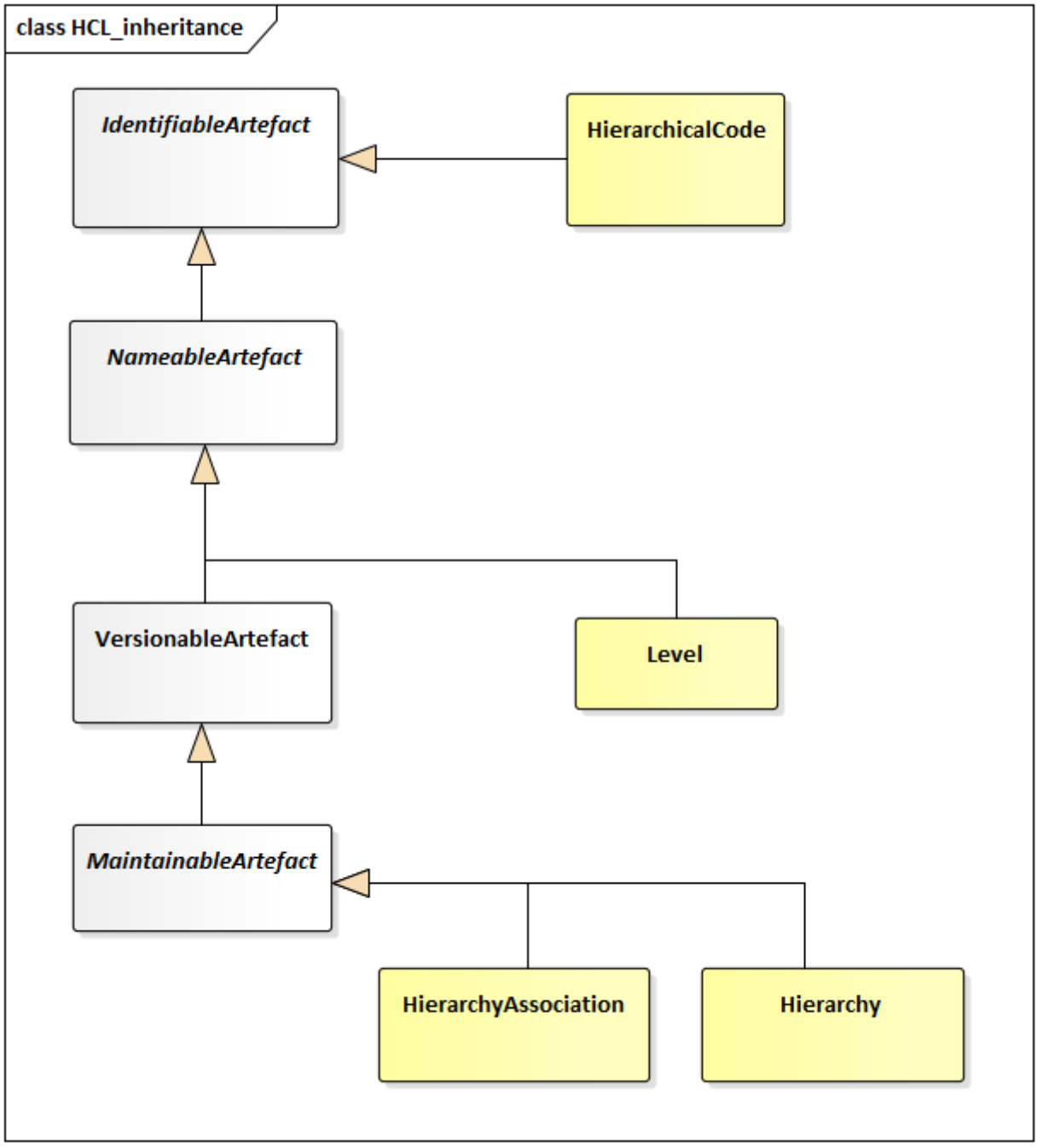


Figure 35. Inheritance class diagram for the `Hierarchy`

Explanation of the Diagram

NARRATIVE

The `Hierarchy` and `HierarchyAssociation` inherit from `MaintainableArtefact` and thus have identification, naming, versioning and a maintenance agency. The `Level` is a `NameableArtefact` and therefore has an Id, multi-lingual name and multi-lingual description. A `HierachicalCode` is an `IdentifiableArtefact`.

Copyright © SDMx TWG 2020

It is important to understand that the `Code`s participating in a `Hierarchy` are not themselves contained in the list – they are referenced from the list and are maintained in one or more `Codelist`s. This is explained in the narrative of the relationship class diagram below.

DEFINITIONS

The definitions of the various classes, attributes, and associations are shown in the relationship section below.

3.9.3 Relationship

Class Diagram

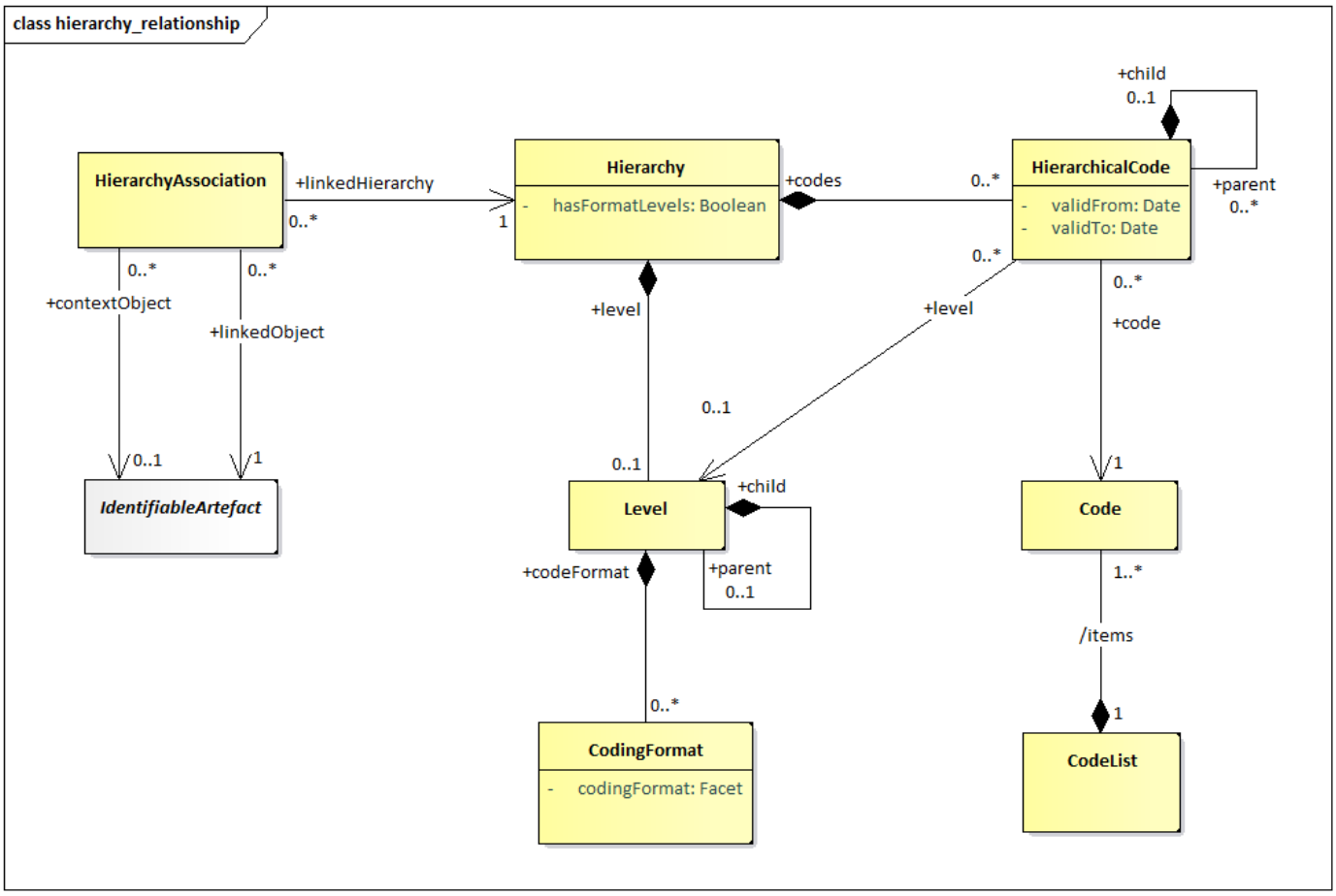


Figure 36. Relationship class diagram of the `Hierarchy`

Explanation of the Diagram

NARRATIVE

The basic principles of the `Hierarchy` are:

1. The `Hierarchy` is a specification of the structure of the `Code`s.
2. The `Code`s in the `Hierarchy` are not themselves a part of the artefact, rather they are references to `Code`s in one or more external `Codelist`s.
3. The hierarchy of `Code`s is specified in `HierarchicalCode`. This references the `Code` and its immediate child `HierarchicalCodes`.

A `Hierarchy` can have formal levels (`hasFormalLevels="true"`). However, even if `hasFormalLevels="false"` the `Hierarchy` can still have one or more Levels associated in order to document information about the `HierarchicalCodes`.

Copyright © SDMx TWG 2020

If `hasFormalLevels="false"` the `Hierarchy` is “value based” comprising a hierarchy of codes with no formal Levels. If `hasFormalLevels="true"` then the hierarchy is “level based” where each Level is a formal Level in the `Hierarchy`, such as those present in statistical classifications. In a “level based” hierarchy each `HierarchicalCode` is linked to the Level in which it resides. It is expected that all `HierarchicalCodes` at the same hierarchic level defined by the `+parent`/`+child` association will be linked to the same Level. Note that the `+level` association need only be specified if the `HierarchicalCode` is at a different hierarchical level (implied by the `HierarchicalCode` parent/child association) than the actual Level in the level hierarchy (implied by the Level parent/child association).

[Note that organisations wishing to be compliant with accepted models for statistical classifications should ensure that the Id is the number associated with the `Level`, where `Level`s are numbered consecutively starting with level 1 at the highest Level].

The `Level` may have `CodingFormat` information defined (e.g. coding type at that level).

A `HierarchyAssociation` links an `IdentifiableArtefact` (`+linkedObject`), that needs a `Hierarchy`, with the latter (`+linkedHierarchy`). The association is performed in a certain context (`+contextObject`), e.g. a Dimension in the context of a Dataflow.

Copyright © SDMx TWG 2020

DEFINITIONS

Class	Feature	Description
`Hierarchy`	Inherits from: `MaintainableArtefact`	A classification structure arranged in levels of detail from the broadest to the most detailed level.
	`hasFormalLevels`	If `true`, this indicates a hierarchy where the structure is arranged in levels of detail from the broadest to the most detailed level. If `false`, this indicates a hierarchy structure where the items in the hierarchy have no formal level structure.
	`+codes`	Association to the top-level hierarchical `Code`s in the `Hierarchy`.
	`+level`	Association to the top `Level` in the `Hierarchy`.
`Level`	Inherits from `NameableArtefact`	In a “level based” hierarchy this describes a group of `Code`s which are characterised by homogeneous coding, and where the parent of each `Code` in the group is at the same higher level of the `Hierarchy`. In a “value based” hierarchy this describes information about the hierarchical `Code`s at the specified nesting level.
	`+codeFormat`	Association to the `CodingFormat`.
	`+child`	Association to a child `Level` of `Level`.
`CodingFormat`		Specifies format information for the codes at this level in the hierarchy such as whether the codes at the level are alphabetic, numeric or alphanumeric and the code length.
`HierarchicalCode`		A hierarchic structure of code references.
	`validFrom`	Date from which the construct is valid.
	`validTo`	Date from which the construct is superseded.
	`+code`	Association to the `Code` that is used at the specific point in the hierarchy.
	`+child`	Association to a child `Code` in the hierarchy.
	`+level`	Association to a `Level` where levels have been defined for the `Hierarchy`.
`Code`		The `Code` to be used at this point in the hierarchy.
	`/items`	Association to the `Codelist` containing the `Code`.
`Codelist`		The `Codelist` containing the `Code`.
`HierarchyAssociation`	Inherits from: `MaintainableArtefact`	An association between an `IdentifiableArtefact` and a `Hierarchy`, within a specific context.
	`+contextObject`	The context within which the association is performed.
	`+linkedObject`	Associates the `IdentifiableArtefact` that needs the `Hierarchy`.
	`+linkedHierarchy`	Associates the `Hierarchy`.

Copyright © SDMx TWG 2020

3.10 Structure Map

3.10.1 Scope

A `StructureMap` allows mapping between Data Structures or Dataflows. It ultimately maps one `DataStructureDefinition` to another (source to target) although it can do this via the Dataflow or directly against the `DataStructureDefinition`.

The `StructureMap` defines how the *structure* of a source `DataStructureDefinition` relates to the *structure* of the target `DataStructureDefinition`. The term *structure* in this instance refers to the Dimensions and Attributes (collectively called Components). An example relationship is source `REF_AREA` Dimension maps to target `COUNTRY` Dimension. When converting data, systems should interpret this, as 'data reported against `REF_AREA` in the source dataset, should be converted to data against `COUNTRY` in the target dataset'. `StructureMaps` can make use of the `RepresentationMap` to describe how the reported value map, if there is a mapping to be done on the value, for example source `REF_AREA.US` may map to `COUNTRY.USA`. In the case of mapping Dates, the `EpochMap` or `DatePatternMap` is used and maintained in the `StructureMap` that uses it.

Class Diagram – Relationship

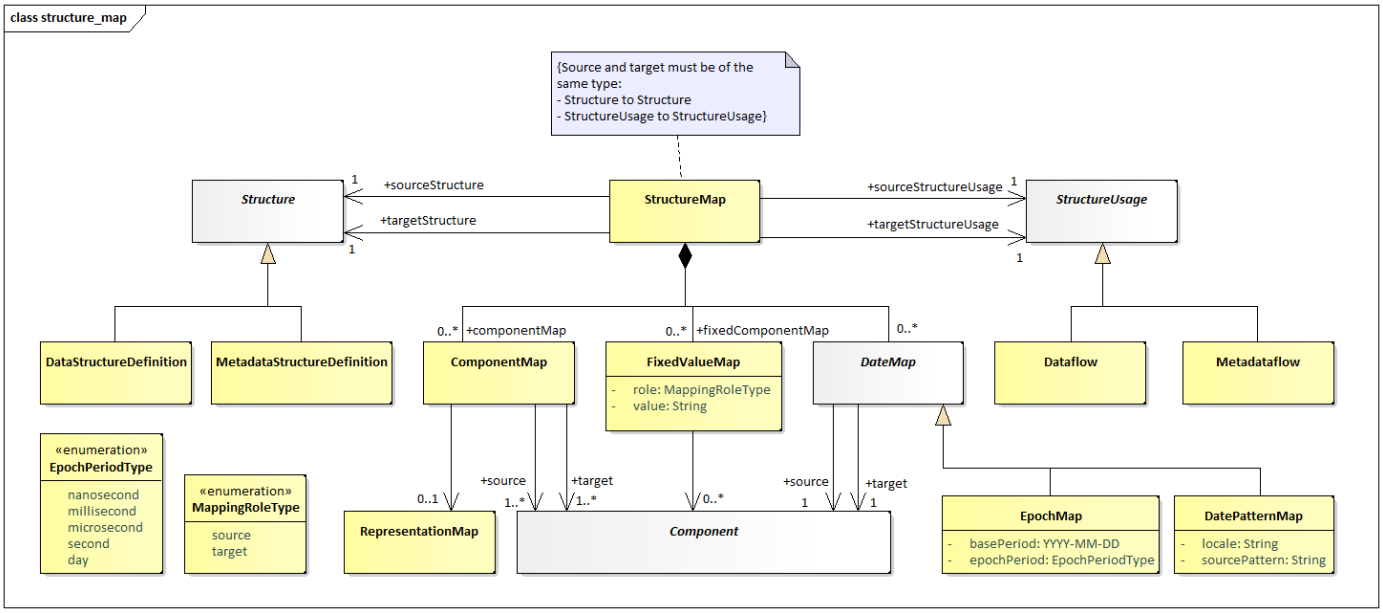


Figure 37. Relationship Class diagram of the Structure Map

Explanation of the Diagram

NARRATIVE

The `StructureMap` is a `MaintainableArtefact`. The `StructureMap` can either map a source and target `DataStructureDefinition` or a source and target Dataflow, it cannot mix source and target types. The `StructureMap` contains zero or more `ComponentMap`s. Each `ComponentMap` maps one or more `Component`s from the source `DataStructureDefinition` to one or more `Component`s in the target `DataStructureDefinition`[1]. In addition, the `StructureMap` contains zero or more `FixedValueMap`s. In this case, one or more `Component`s, from the source or target `DataStructureDefinition`, map to a fixed value.

The rules pertaining to how reported values map, are maintained in either a `RepresentationMap`, `EpochMap`, or `DatePatternMap`. A `ComponentMap` can only reference one of these mapping types to define how the reported values relate from source Dataset to the target Dataset. If a `ComponentMap` has more than 1 source or target, a `RepresentationMap` must be used to describe how the values map, as it is the only map which can define multiple source and target values in combination.

If the `ComponentMap` does not reference any map type to describe how the values map in a Dataset, then the values from the source Dataset are copied to the target Dataset verbatim, with no mapping rules being applied.

Copyright © SDMx TWG 2020

A `RepresentationMap` is a separate Maintainable structure. `EpochMap` and `DatePatternMap` are maintained in the same `StructureMap` and are referenced locally from the `ComponentMap`. `EpochMap` and `DatePatternMap` are maintained outside of the `ComponentMap` and can therefore be reused by multiple `ComponentMap`s.

Class Diagram – Epoch Mapping and Date Pattern Mapping

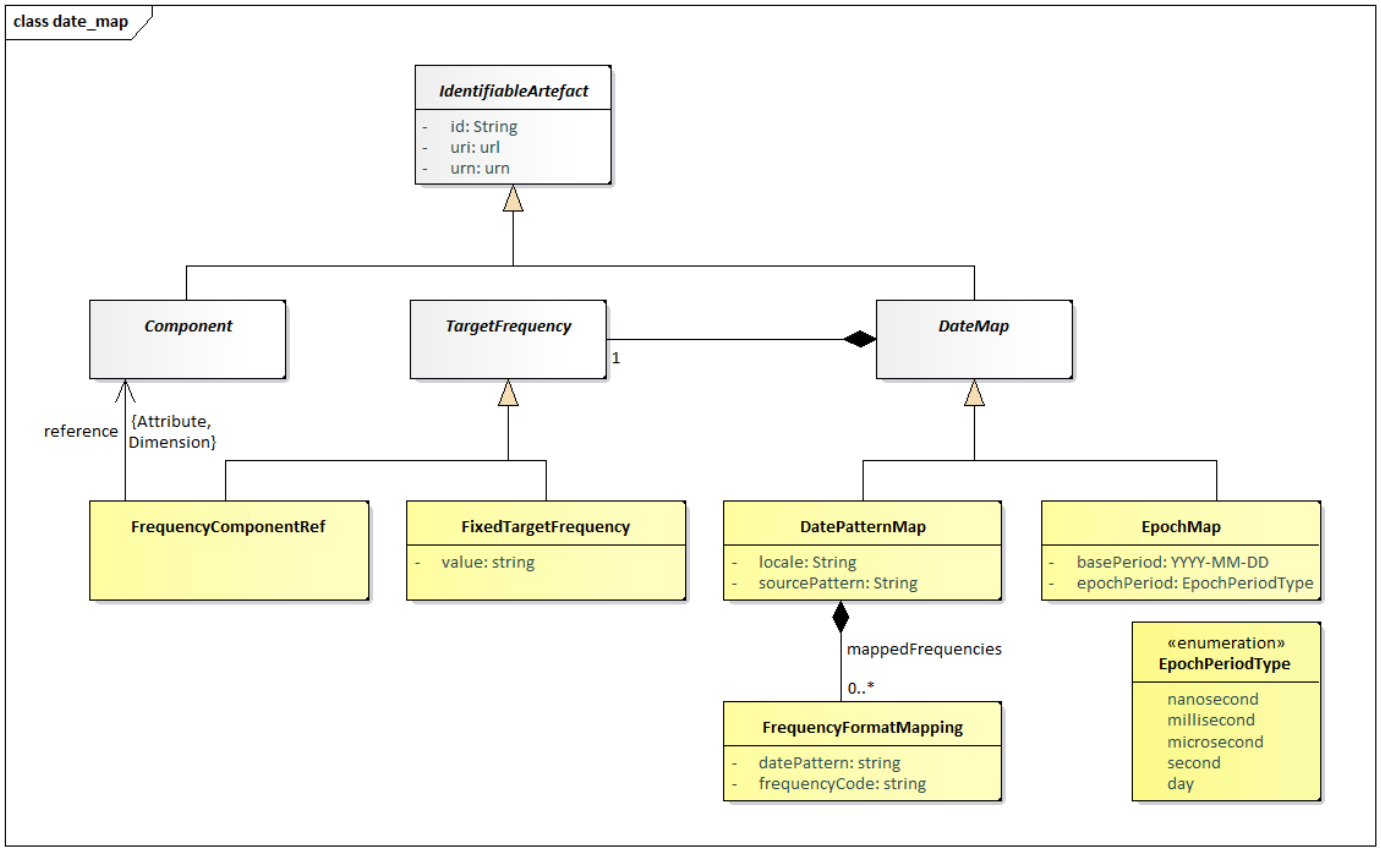


Figure 38. Relationship Class diagram of the `EpochMap` and `DatePatternMap`

Explanation of the Diagram

NARRATIVE

The `EpochMap` and `DatePatternMap` are both `IdentifiableArtefact`. An `EpochMap` and `DatePatternMap` both provide the ability to map source to target date formats. The `EpochMap` describes the source date as the number of epochs since a point in time, where the duration of each epoch is defined, e.g., number of milliseconds since 1970. The `DatePatternMap` describes the source date as a pattern for example `MM-YYYY`, accompanied by the appropriate locale.

Both mappings describe the target date as a frequency Identifier. The frequency identifier is given either a fixed value, e.g., `A` or a reference to a Dimension or Attribute in the target `DataStructureDefinition` of the `StructureMap`, e.g. `FREQ`. In the latter case, the frequency id is derived at run time when the output series and observations are generated. Dates mapped using the frequency lookup can therefore be mapped using different frequencies depending on the series or observation being converted.

If the Frequency Identifier aligns with standard SDMX frequencies the output date format can be derived using standard SDMX date formatting (e.g., `A=YYYY`, `Q=YYYY-Qn`). If the SDMX standard formatting is not desired or if the frequency Id is not a standard SDMX frequency Code, the `FrequencyFormatMapping` can be used to describe the relationship between the frequency Id and the output date format, e.g., `A01=YYYY`.

Copyright © SDMx TWG 2020

DEFINITIONS

Copyright © SDMx TWG 2020

Class	Feature	Description
`StructureMap`	Inherits from	`MaintainableArtefact`. Links a source and target structure where there is a semantic equivalence between the source and the target structures.
	`+sourceStructure`	Association to the source Data Structure.
	`+targetStructure`	Association to the target Data Structure.
	`+sourceStructureUsage`	Association to the source Dataflow.
	`+targetStructureUsage`	Association to the target Dataflow.
`ComponentMap`	Inherits from `AnnotableArtefact`.	Links source and target Component(s) where there is a semantic equivalence between the source and the target Components.
	`+source`	Association to zero or more source Components.
	`+target`	Association to zero or more the target Components.
	`mappingRules`	Reference to either a `RepresentationMap`, an `EpochMap` or a `DatePatternMap`.
`FixedValueMap`	Inherits from `AnnotableArtefact`.	Links a Component (source or target) to a fixed value.
	`value`	The value that a Component will be fixed in a fixed component map.
`DateMap`	Inherits from `IdentifiableArtefact`.	
	`freqDimension`	The Dimension or Attribute of the target Data Structure Definition which will hold the frequency information for date conversion. Mutually exclusive with `targetFrequencyId`.
	`yearStart`	The date of the start of the year, enabling mapping from high frequency to lower frequency formats.
	`resolvePeriod`	Which point in time to resolve to when mapping from low frequency to high frequency periods.
	`mappedFrequencies`	A reference to a map of frequency id to date pattern for output.
`EpochMap`	Inherits from `DateMap`.	
	`basePeriod`	Epoch zero starts on this period.
	`targetFrequencyId`	The frequency to convert the input date into. Mutually exclusive with `freqDimension`.
	`epochPeriod`	Describes the period of time that each epoch represents.
`DatePatternMap`	Inherits from `DateMap`.	Describes a source date based on a string pattern, and how it maps to the target date.
	`locale`	The locale on which the input will be parsed according to the pattern.
`DateMapping`		
	`sourcePattern`	Describes the source date using conventions for describing years, months, days, etc.
	`targetFrequencyId`	

Copyright © SDMx TWG 2020

Class	Feature	Description
		The frequency to convert the input date into. Mutually exclusive with `freqDimension`.
`FrequencyFormatMapping`	Inherits from `IdentifiableArtefact`.	Describes the relationship between a frequency Id to what the output date is formatted.
	`frequencyId`	The string used to describe the frequency.
	`datePattern`	The output date pattern for that frequency.

1. Source and target Data Structure Definition are either directly linked from the `StructureMap` or indirectly via the linked source and target Dataflow ↩

Copyright © SDMx TWG 2020

3.11 RepresentationMap

3.11.1 Scope

A `RepresentationMap` describes a mapping between source value(s) and target value(s) where the values are restricted to those in a `Codelist`, `ValueList` or be of a certain data type, e.g., `Integer`.

The `RepresentationMap` maps information from one or more sources, where the values for each source are used in combination to derive the output value for one or more targets. Each source value may match a substring of the original data (using `startIndex` and/or `endIndex`) or define a pattern matching rule described by a regular expression. The target value is provided as an absolute string, although it can make use of regular expression groups to carry across values from the source string to the target string without having to explicitly state the value to carry. An example is a regular expression which states 'match a value starting with `AB` followed by anything, where the anything is marked a capture group', the target can state 'take the anything value and postfix it with `AB`' thus enabling the mapping of `ABX` to `XAB` and `ABY` to `YAB`.

The absence of an output for an input is interpreted as 'no output value for the given source value(s)'.

Copyright © SDMx TWG 2020

Class Diagram – Relationship

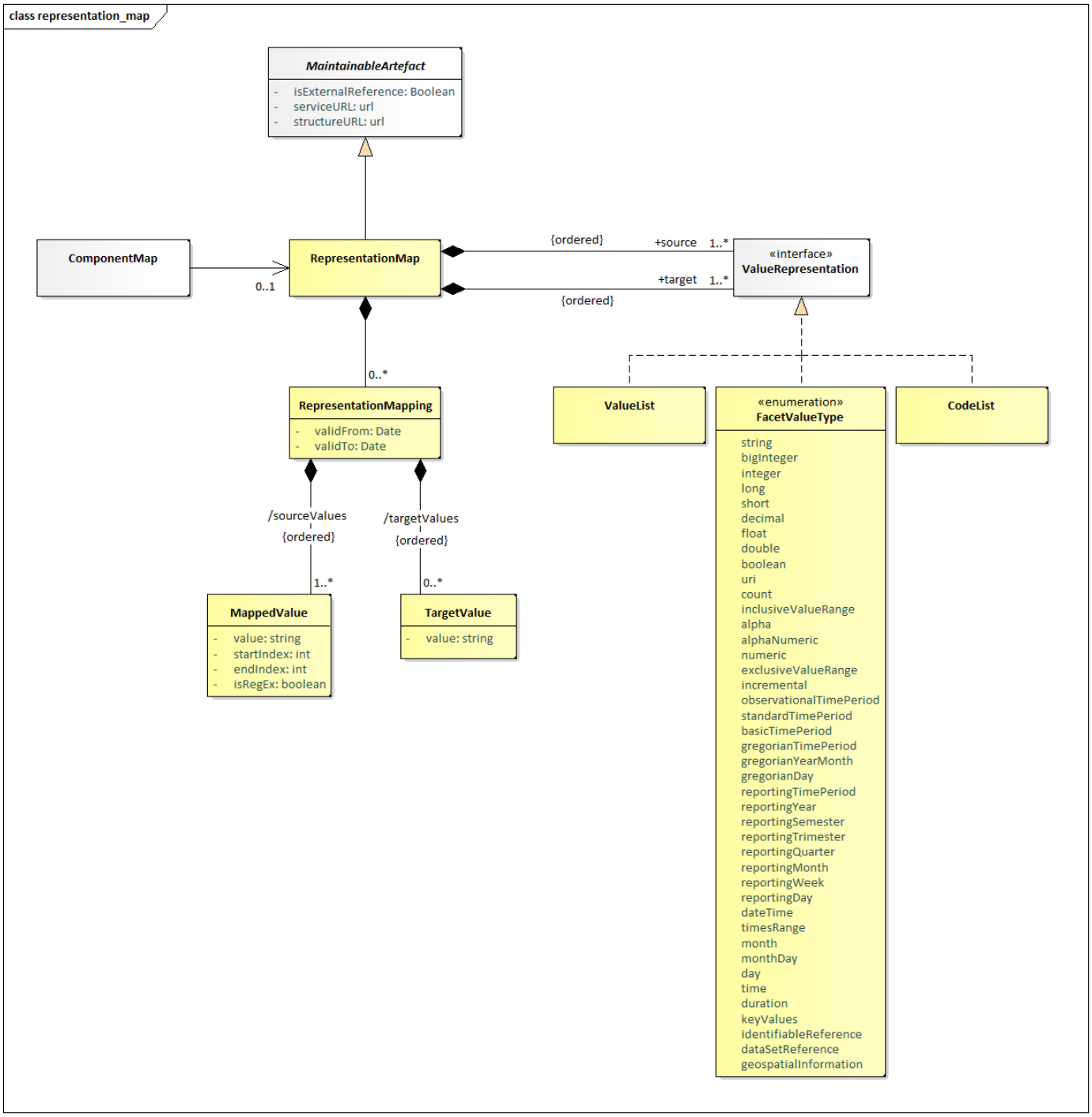


Figure 39. Representation Map

Explanation of the Diagram

NARRATIVE

The `RepresentationMap` is a `MaintainableArtefact`. It maps one or more source values to one or more target values, where values that are being mapped are defined by the `ValueRepresentation`. A `ValueRepresentation` is an abstract container which is either a Codelist, `ValueList` or a `FacetValueType`. Source and target values are in a list where the list order is important as the `RepresentationMapping` `sourceValues` and `targetValues` must match the order. It is permissible to mix types for both source and

Copyright © SDMx TWG 2020

target values, allowing for example a Codelist to map to an Integer (which is a `FacetValueType`). The list of source or targets can also be mixed, for example a Codelist in conjunction with a `FacetValueType` and `ValueList` and can be defined as the source of a mapping, thus allowing rules such as `When CL_AREA=UK AND AGE=26 CURRENCY=$`.

DEFINITIONS

Class	Feature	Description
`RepresentationMap`	Inherits from `MaintainableArtefact`	Links source and target representations, whose values may conform to a linked Codelist, `ValueList` or enumerated type such as `Integer`.
	`source`	Association to one or more `Codelist`, `ValueList`, or `FacetValue` — mixed types are permissible
	`target`	Association to one or more `Codelist`, `ValueList`, or `FacetValue` — mixed types are permissible
`RepresentationMapping`	Inherits from `AnnotableArtefact`	Describes how the source value(s) map to the target value(s)
	`validFrom`	Optional period describing when the mapping is applicable
	`validTo`	Optional period describing when the mapping is no longer applicable
	`sourceValues`	Input value for source in the `RepresentationMap`
	`targetValues`	Output value for each mapped target in the `RepresentationMap`
`MappedValue`		Describes an input value that is part of the `sourceValues` in a `RepresentationMapping`
	`value`	The value to compare the source data with
	`isRegEx`	If true, the `value` field should be treated as a regular expression when comparing with the source data
	`startIndex`	If provided, a substring of the source data should be taken, starting from this index (starting at zero) before comparing with the `value` field for matching
	`endIndex`	If provided, a substring of the source data should be taken, ending at this index (starting at zero) before comparing with the `value` field for matching
`TargetValue`		Describes the target value that is part of the `targetValues` of a `RepresentationMapping`
	`value`	Represents a value for the `targetValues` of a `RepresentationMapping`

Copyright © SDMx TWG 2020

3.12 ItemSchemeMap

3.12.1 Scope

An `ItemSchemeMap` is an abstract container to describe mapping rules between any item scheme, with the exception of `Codelists` and `ValueLists` which are mapped using the `RepresentationMap`. A single source `ItemScheme` is mapped to a single target `ItemScheme`. The `ItemSchemeMap` then contains the rules for how the values from the source `ItemScheme` map to the values in the target `ItemScheme`. Each source value may match a substring of the original data (using `startIndex` and/or `endIndex`) or define a pattern matching rule described by a regular expression. The target value is provided as an absolute string, although it can make use of regular expression groups to carry across values from the source string to the target string without having to explicitly state the value to carry. An example is a regular expression which states 'match a value starting with `AB` followed by anything, where the `anything` is marked a capture group', the target can state 'take the `anything` value and postfix it with `AB`' thus enabling the mapping of `ABX` to `XAB` and `ABY` to `YAB`.

The absence of an output for an input is interpreted as 'no output value for the given source value(s)'.

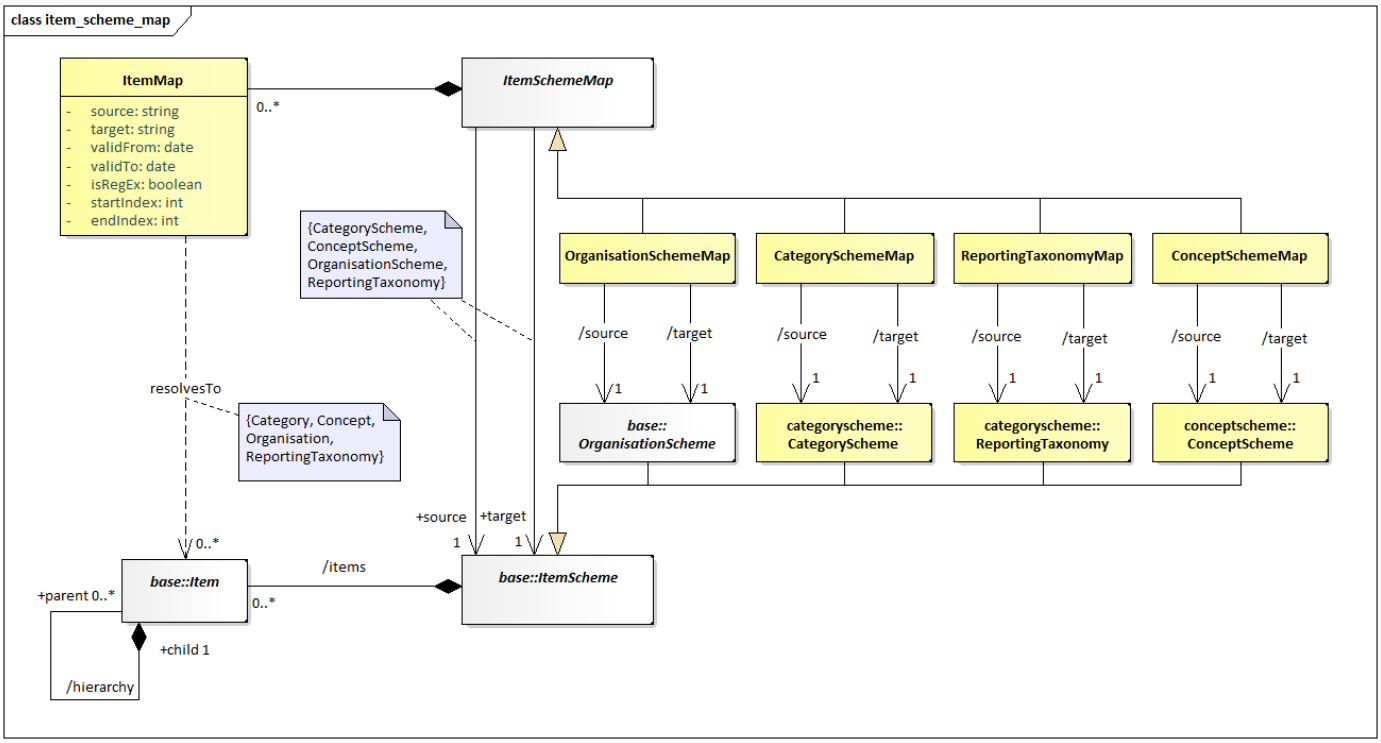


Figure 40. Item Scheme Map

Explanation of the Diagram

NARRATIVE

An `ItemSchemeMap` is an abstract type which inherits from Maintainable. It is subclassed by the 4 concrete classes:

- `OrganisationSchemeMap`
- `ConceptSchemeMap`
- `CategorySchemeMap`
- `ReportingTaxonomyMap`

An `OrganisationSchemeMap` maps a source `AgencyScheme`, `DataProviderScheme`, `DataConsumerScheme` or `OrganisationUnitScheme` to a target `AgencyScheme`, `DataProviderScheme`, `DataConsumerScheme` or `OrganisationUnitScheme`. It is permissible to mix source and target

Copyright © SDMx TWG 2020

types to define an equivalence between Organisations of different roles. The mapped items refer to the Organisations in the source/target schemes.

A `ConceptSchemeMap` maps a source `ConceptScheme` to a target `ConceptScheme`. The mapped items refer to the `Concept`s in the source/target schemes.

A `CategorySchemeMap` maps a source `CategoryScheme` to a target `CategoryScheme`. The mapped Items refer to the `Categories` in the source/target schemes.

A `ReportingTaxonomyMap` maps a source `ReportingTaxonomy` to a target `ReportingTaxonomy`. The mapped Items refer to the `ReportingCategory` in the source/target schemes.

DEFINITIONS

Class	Feature	Description
`ItemSchemeMap`	Inherits from `MaintainableArtefact`	Links source and target `ItemSchemes`
	`source`	Association to a source `ItemScheme`
	`target`	Association to a target `ItemScheme`
`ItemMap`	Inherits from `AnnotableArtefact`	Describes how the source value maps to the target value
	`validFrom`	Optional period describing when the mapping is applicable
	`validTo`	Optional period describing when the mapping is no longer applicable
	`sourceValue`	Input value for source
	`targetValue`	Output value for each mapped target
	`isRegEx`	If true, the `sourceValue` field should be treated as a regular expression when comparing with the source data
	`startIndex`	If provided, a substring of the source data should be taken, starting from this index (starting at zero) before comparing with the `value` field for matching
	`endIndex`	If provided, a substring of the source data should be taken, ending at this index (starting at zero) before comparing with the `value` field for matching
`OrganisationSchemeMap`	Inherits from `ItemSchemeMap`	Concrete `Maintainable` subtype of `ItemSchemeMap`
`ConceptSchemeMap`	Inherits from `ItemSchemeMap`	Concrete `Maintainable` subtype of `ItemSchemeMap`
`CategorySchemeMap`	Inherits from `ItemSchemeMap`	Concrete `Maintainable` subtype of `ItemSchemeMap`
`ReportingTaxonomyMap`	Inherits from `ItemSchemeMap`	Concrete `Maintainable` subtype of `ItemSchemeMap`

Copyright © SDMx TWG 2020

3.13 Constraints

3.13.1 Scope

The scope of this section is to describe the support in the metamodel for specifying both the access to and the content of a data source. The information may be stored in a resource such as a registry for use by applications wishing to locate data and metadata which are available via the Internet. The `Constraint` is also used to specify a subset of a `Codelist` which may be used as a partial `Codelist`, relevant in the context of the artefact to which the `Constraint` is attached e.g., `DataStructureDefinition`, `Dataflow`, `ProvisionAgreement`, `MetadataStructureDefinition`, `Metadataflow`, `MetadataProvisionAgreement`.

Note that in this metamodel the term data source refers to both data and metadata sources, and data provider refers to both data and metadata providers.

A data source may be a simple file of data or metadata (in SDMX-ML, JSON or other format), or a database or metadata repository. A data source may contain data for many data or metadata flows (called `Dataflow`, and `Metadataflow` in the model), and the mechanisms described in this section allow an organisation to specify precisely the scope of the content of the data source where this data source is registered (`SimpleDataSource`, `QueryDataSource`).

The `Dataflow` and `Metadataflow`, themselves may be specified as containing only a subset of all the possible keys that could be derived from a `DataStructureDefinition` or `MetadataStructureDefinition`. Respectively, further subsets may be defined within a `ProvisionAgreement` and `MetadataProvisionAgreement`.

These specifications are called `Constraint` in this model.

3.13.2 Inheritance

Class Diagram of Constrainable Artefacts - Inheritance

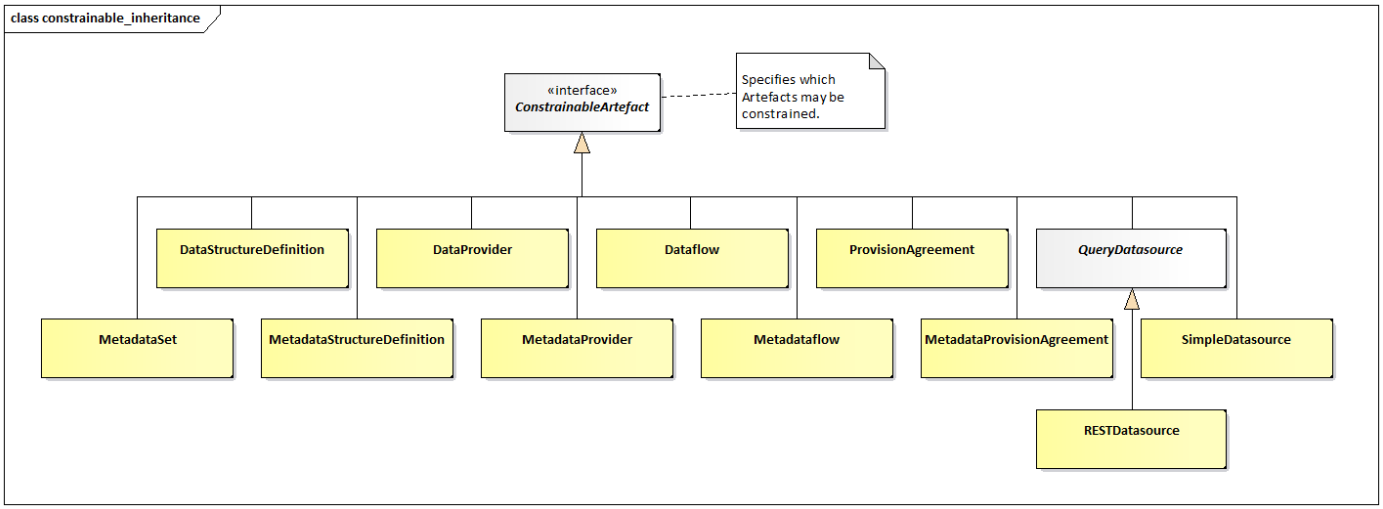


Figure 41. Inheritance class diagram of constrainable and provisioning artefacts

Copyright © SDMx TWG 2020

Explanation of the Diagram

NARRATIVE

Any artefact that inherits from the `ConstrainableArtefact` interface can have constraints defined. The artefacts that can have constraint metadata attached are:

- `Dataflow`
- `ProvisionAgreement`
- `DataProvider` – this is restricted to release calendar
- `DataStructureDefinition`
- `Metadataflow`
- `MetaDataProvider` – this is restricted to release calendar
- `MetadataProvisionAgreement`
- `MetadataSetMetadataStructureDefinition`
- `SimpleDataSource` – this is a registered data source where the registration references the actual Data Set or Metadata Set

`QueryDataSource`

Note that, because the `Constraint` can specify a subset of the component values implied by a specific `Structure` (such as a specific `DataStructureDefinition` or specific `MetadataStructureDefinition`), the `ConstrainableArtefact`s must be associated with a specific `Structure`. Therefore, whilst the `Constraint` itself may not be linked directly to a `DataStructureDefinition` or `MetadataStructureDefinition`, the artefact that it is constraining will be linked to a `DataStructureDefinition` or `MetadataStructureDefinition`. As a `DataProvider` or a `MetadataProvider` does not link to any one specific DSD or MSD the type of information that can be contained in a Constraint linked to a `DataProvider` / `MetadataProvider` is restricted to `ReleaseCalendar`.

3.13.3 Constraints

Relationship Class Diagram – high level view

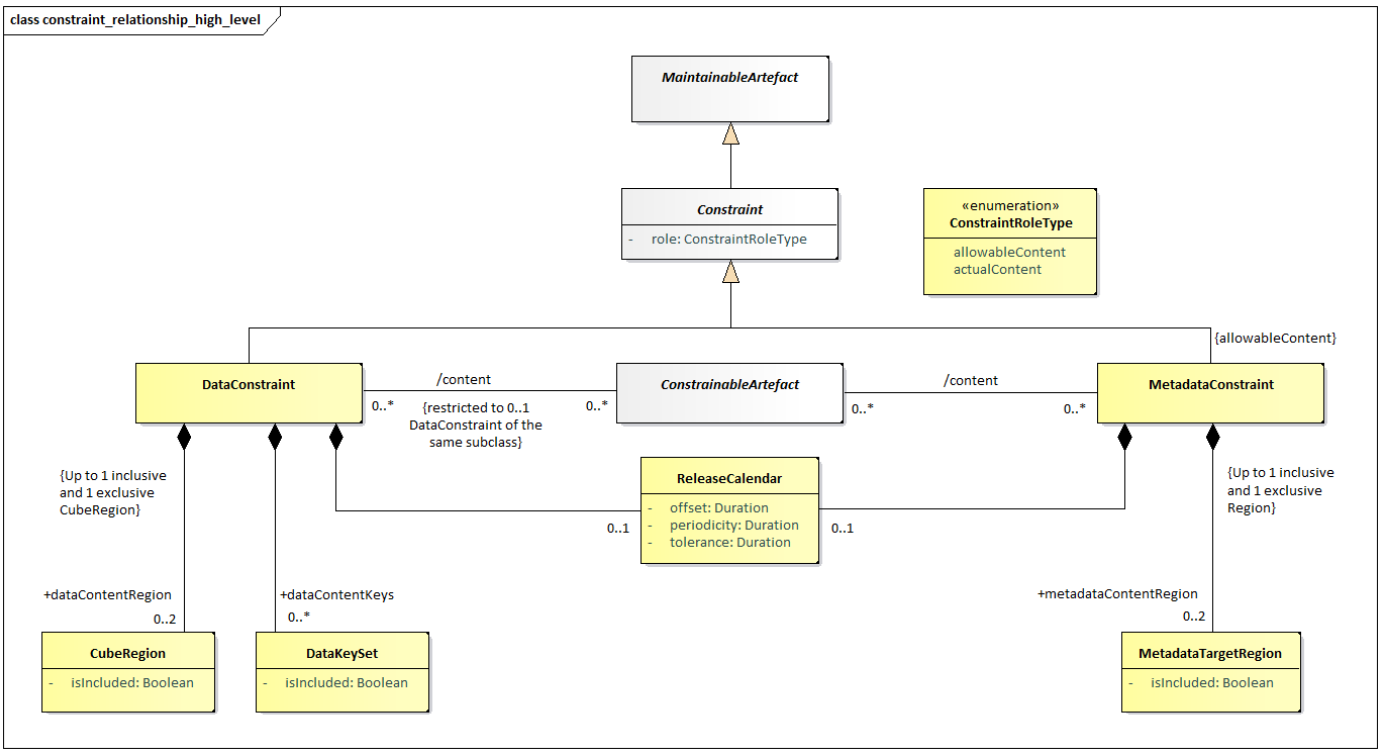


Figure 42. Relationship class diagram showing constraint metadata

Copyright © SDMx TWG 2020

Explanation of the Diagram

NARRATIVE

The constraint mechanism allows specific constraints to be attached to a `ConstrainableArtefact`. With the exception of `ReleaseCalendar` these constraints specify a subset of the total set of values or keys that may be present in any of the `ConstrainableArtefacts`.

For instance, a `DataStructureDefinition` specifies, for each Dimension, the list of allowable code values. However, a specific `Dataflow` that uses the `DataStructureDefinition` may contain only a subset of the possible range of keys that is theoretically possible from the `DataStructureDefinition` definition (the total range of possibilities is sometimes called the Cartesian product of the dimension values). In addition to this, a `DataProvider` that is capable of supplying data according to the `Dataflow` has a `ProvisionAgreement`, and the `DataProvider` may also wish to supply constraint information which may further constrain the range of possibilities in order to describe the data that the provider can supply. It may also be useful to describe the content of a data source in terms of the `KeySets` or `CubeRegions` contained within it.

A `ConstrainableArtefact` can have two types of `Constraint`s:

1. `DataConstraint` – is used as a mechanism to specify, either the available set of keys (`DataKeySet`), or set of component values (`CubeRegion`) in a `DataSource` such as a `Simpledatasource` or a database (`QueryDatasource`), or the allowable keys that can be constructed from a `DataStructureDefinition`. Multiple such `DataConstraints` may be present for a `ConstrainableArtefact`. For instance, there may be a `DataConstraint` that specifies the values allowed for the `ConstrainableArtefact` (role is `allowableContent`) which can be used for validation or for constructing a partial code list for one Dimension, while another provides the validation for another Dimension within the same DSD.
2. `MetadataConstraint` – is used as a mechanism to specify a set of component values (`MetadatTargetRegion`) in a `DataSource` such as a `MetadataSet` or a database (`QueryDatasource`). Multiple such `MetadataConstraints` may be present for a `ConstrainableArtefact`. For instance, there may be a `MetadataConstraint` that specifies the values allowed for the `ConstrainableArtefact` (role is `allowableContent`) which can be used for validation or for constructing a partial code list, whilst another `MetadataConstraint` can specify the actual content of a metadata source (role is `actualContent`).

In addition to `DataKeySet` and/or `CubeRegion` / `MetadataTargetRegion` a Constraint can have a `ReleaseCalendar` specifying when data or metadata are released for publication or reporting.

Note also that another possible type of a `DataConstraint` is available; that is a `DataConstraint` with the role of `actualContent` where it describes the data that an SDMX Web Service contains. This type of `DataConstraint` is not maintained in a Registry and is always a response to the data availability SDMX REST API. Thus, its identification is auto-generated by the service responding to a data availability request.

Copyright © SDMx TWG 2020

Relationship Class Diagram – Detail

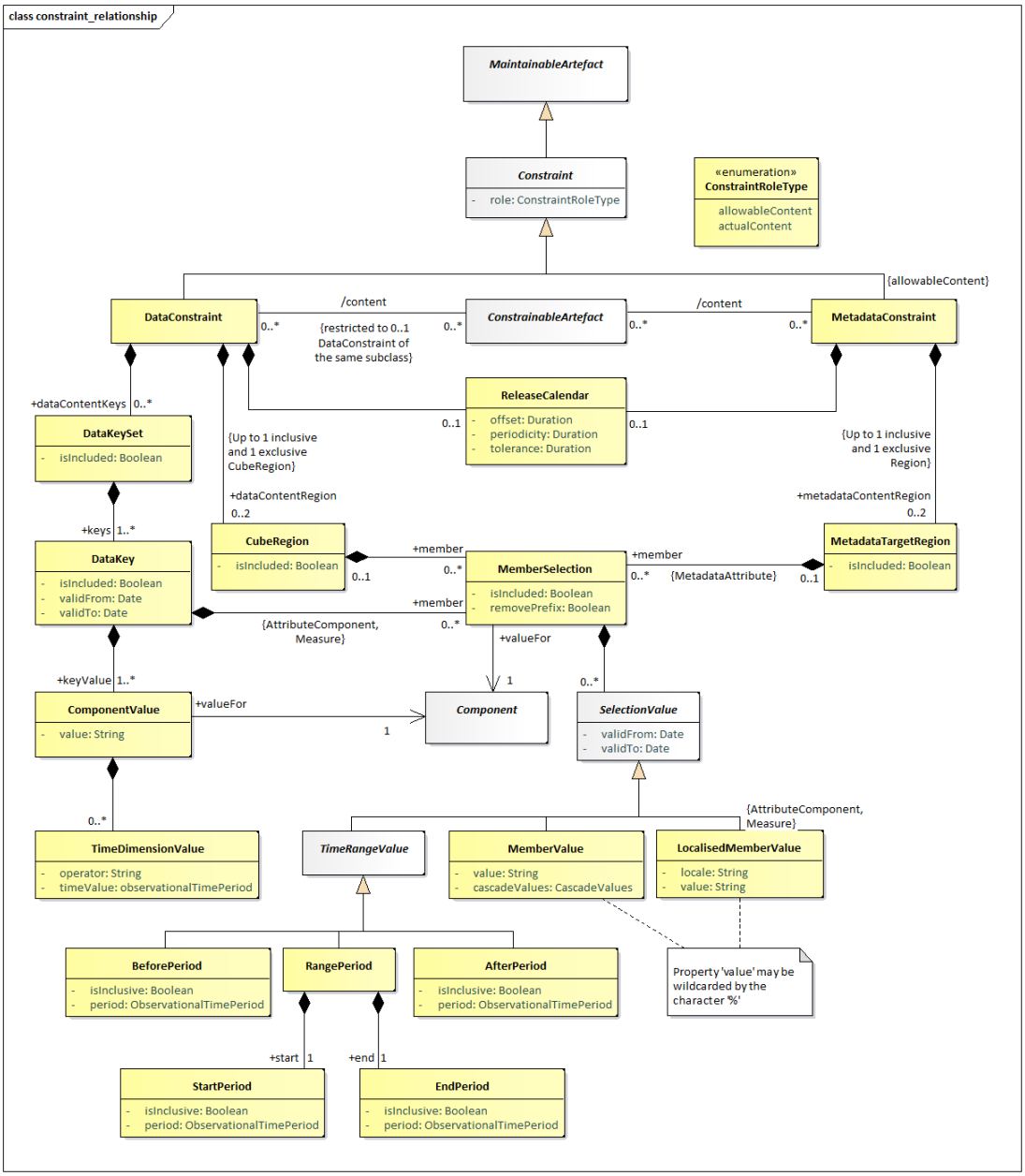


Figure 43. Constraints – Key Set, Cube Region and Metadata Target Region

EXPLANATION OF THE DIAGRAM

A `Constraint` is a `MaintainableArtefact`.

A `DataConstraint` has a choice of two ways of specifying value subsets:

1. As a set of keys that can be present in the `DataSet` (`DataKeySet`). Each `DataKey` specifies a number of `ComponentValues` each of which reference a `Component` (e.g., Dimension, `DataAttribute`). Each `ComponentValue` is a value that may be present for a `Component` of a structure when contained in a `DataSet`. In addition, each `DataKeySet` may also include `MemberSelections` for `AttributeComponents` or Measures.
2. As a set of `CubeRegions` each of which defines a “slice” of the total structure (`MemberSelection`) in terms of one or more `MemberValues` that may be present for a `Component` of a structure when contained in a `DataSet`.

Copyright © SDMx TWG 2020

The difference between (1) and (2) above is that in (1) a complete key is defined whereas in (2) above the “slice” defines a list of possible values for each of the `Component`s but does not specify specific key combinations. In addition, in (1) the association between `Component` and `DataKeyValue` is constrained to the components that comprise the key, whereas in (2) it can contain other component types (such as `AttributeComponents` or Measures). By adding `MemberSelections` to the `DataKeySets` of (1), `AttributeComponents` and Measures are constrained for the related `DataKeys`.

A `MetadataConstraint` has only one way of specifying value subsets:

1. As a set of `MetadataTargetRegions` each of which defines a “slice” of the total structure (`MemberSelection`) in terms of one or more `MemberValues` that may be present for a `Component` of a structure when contained in a `MetadataSet`.

In both `CubeRegion` and `MetadataTargetRegion`, the value in `ComponentValue`.value and `MemberValue`.value must be consistent with the `Representation` declared for the `Component` in the `DataStructureDefinition` (Dimension or `DataAttribute`) or `MetadataStructureDefinition` (`MetadataAttribute`). Note that in all cases the “operator” on the value is deemed to be “equals”, unless the wildcard character is used `'%'`. In the latter case the “operation” is a partial matching, where the percentage character (`'%'`) may match zero or more characters. Furthermore, it is possible in a `MemberValue` to specify that child values (e.g., child codes) are included in the Constraint by means of the `cascadeValues` attribute. The latter may take the following values:

- `"true"`: all children are included,
- `"false"` (default), or
- `"excludeRoot"`, where all children are included, and the root Code is excluded (i.e. the referenced Code).

It is possible to define for the `DataKeySet`, `DataKey`, `CubeRegion`, `MetadataTargetRegion` and `MemberSelection` whether the set is included (`isIncluded = "true"`, default) or excluded (`isIncluded = "false"`) from the Constraint definition. This attribute is useful if, for example, only a small sub-set of the possible values are not included in the set, then this smaller sub-set can be defined and excluded from the constraint. Note that if the child construct is “included” and the parent construct is “excluded” then the child construct is included in the list of constructs that are “excluded”.

In any `MemberSelection` that the corresponding `Component` was using Codelist with extensions, it is possible to remove the prefix that has been used, in order to refer to the original Codes. This is achieved via property `removePrefix`, which defaults to `“false”`.

In `DataKeys` and `MemberValues` it is possible, via the `validFrom` and `validTo` properties, to set a validity period for which the selected key or value is constrained.

Copyright © SDMx TWG 2020

DEFINITIONS

Copyright © SDMx TWG 2020

Class	Feature	Description
`ConstrainableArtefact`	Abstract Class Sub classes are: `Dataflow` `DataProvider` `DataStructureDefinition` `Metadataflow` `MetadataProvisionAgreement` `MetadataSet` `MetadataStructureDefinition` `ProvisionAgreement` `QueryDatasource` `SimpleDatasource`	An artefact that can have Constraints specified.
	`content`	Associates the metadata that constrains the content to be found in a data or metadata source linked to the Constrainable Artefact.
`Constraint`	Inherits from `MaintainableArtefact` Abstract class Sub classes are: `DataConstraint` `MetadataConstraint`	Specifies a subset of the definition of the allowable or actual content of a data or metadata source that can be derived from the Structure that defines code lists and other valid content.
	`+dataContentKeys`	Association to a subset of Data Key Sets (i.e., value combinations) that can be derived from the definition of the structure to which the Constrainable Artefact is linked.
	`+dataContentRegion`	Association to a subset of component values that can be derived from the Data Structure Definition to which the Constrainable Artefact is linked.
	`+metadataContentRegion`	Association to a subset of component values that can be derived from the Metadata Structure Definition to which the Constrainable Artefact is linked.
	role	Association to the role that the Constraint plays
`DataConstraint`	Inherits from `Constraint`	Defines a Constraint in terms of the content that can be found in data sources linked to the Constrainable Artefact to which this constraint is associated.
`ConstraintRoleType`		Specifies the way the type of content of a Constraint in terms of its purpose.
	`allowableContent`	The Constraint contains a specification of the valid subset of the Component values or keys.
	`actualContent`	The Constraint contains a specification of the actual content of a data or metadata source in terms of the Component values or keys in the source.
`MetadataConstraint`	Inherits from `Constraint`	Defines a Constraint in terms of the content that can be found in metadata sources linked to the Constrainable Artefact to which this constraint is associated.
`DataKeySet`		A set of data keys.
	`isIncluded`	Indicates whether the Data Key Set is included in the constraint definition or excluded from the constraint definition.
	`+keys`	Association to the Data Keys in the set.
	`+member`	Association to the selection of a value subset for Attributes and Measures.

Copyright © SDMx TWG 2020

Class	Feature	Description
`DataKey`		The values of a key in a data set.
	`isIncluded`	Indicates whether the Data Key is included in the constraint definition or excluded from the constraint definition.
	`+keyValue`	Associates the Component Values that comprise the key.
	`validFrom`	Date from which the Data Key is valid.
	`validTo`	Date from which the Data Key is superseded.
`ComponentValue`		The identification and value of a Component of the key (e.g., Dimension)
	`value`	The value of Component
	`+valueFor`	Association to the Component (e.g., Dimension) in the Structure to which the Constrainable Artefact is linked.
`TimeDimensionValue`		The value of the Time Dimension component.
	`timeValue`	The value of the time period.
	`operator`	Indicates whether the specified value represents and exact time or time period, or whether the value should be handled as a range. A value of `greaterThan` or `greaterThanOrEqual` indicates that the value is the beginning of a range (exclusive or inclusive, respectively). A value of `lessThan` or `lessThanOrEqual` indicates that the value is the end or a range (exclusive or inclusive, respectively). In the absence of the opposite bound being specified for the range, this bound is to be treated as infinite (e.g., any time period after the beginning of the provided time period for `greaterThanOrEqual`)
`CubeRegion`		A set of Components and their values that defines a subset or “slice” of the total range of possible content of a data structure to which the Constrainable Artefact is linked.
	`isIncluded`	Indicates whether the Cube Region is included in the constraint definition or excluded from the constraint definition.
	`+member`	Associates the set of Components that define the subset of values.
`MetadataTargetRegion`		A set of Components and their values that defines a subset or “slice” of the total range of possible content of a metadata structure to which the Constrainable Artefact is linked.
	`isIncluded`	Indicates whether the Metadata Target Region is included in the constraint definition or excluded from the constraint definition.
	`+member`	Associates the set of Components that define the subset of values.
`MemberSelection`		

Copyright © SDMx TWG 2020

Class	Feature	Description
		A set of permissible values for one component of the axis.
	`isIncluded`	Indicates whether the Member Selection is included in the constraint definition or excluded from the constraint definition.
	`removePrefix`	Indicates whether the Codes should keep or not the prefix, as defined in the extension of Codelist.
	`+valuesFor`	Association to the Component in the Structure to which the Constrainable Artefact is linked, which defines the valid Representation for the Member Values.
`SelectionValue`	Abstract class. Sub classes are: `MemberValue` `TimeRangeValue` `LocalisedMemberValue`	A collection of values for the Member Selections that, combined with other Member Selections, comprise the value content of the Cube Region.
	`validFrom`	Date from which the Selection Value is valid.
	`validTo`	Date from which the Selection Value is superseded.
`MemberValue`	Inherits from `SelectionValue`	A single value of the set of values for the Member Selection.
	`value`	A value of the member.
	`cascadeValues`	Indicates that the child nodes of the member are included in the Member Selection (e.g., child codes)
`LocalisedMemberValue`	Inherits from `SelectionValue`	A single localised value of the set of values for a Member Selection.
	`value`	A value of the member.
	`locale`	The locale that the values must adhere to in the dataset.
`TimeRangeValue`	Inherits from `SelectionValue` Abstract Class Concrete Classes: `BeforePeriod` `AfterPeriod` `RangePeriod`	A time value or values that specifies the date or dates for which the constrained selection is valid.
`BeforePeriod`	Inherits from `TimeRangeValue`	The period before which the constrained selection is valid.
	`isInclusive`	Indication of whether the date is inclusive in the period.
	`period`	The time period which acts as the latest possible reported period
`AfterPeriod`	Inherits from `TimeRangeValue`	The period after which the constrained selection is valid.
	`isInclusive`	Indication of whether the date is inclusive in the period.
	`period`	The time period which acts as the earliest possible reported period
`RangePeriod`		The start and end periods in a date range.
	`+start`	Association to the Start Period.

Copyright © SDMx TWG 2020

Class	Feature	Description
	`+end`	Association to the End Period.
`StartPeriod`	Inherits from `TimeRangeValue`	The period from which the constrained selection is valid.
	`isInclusive`	Indication of whether the date is inclusive in the period.
	`period`	The time period which acts as the start of the range
`EndPeriod`	Inherits from `TimeRangeValue`	The period to which the constrained selection is valid.
	`isInclusive`	Indication of whether the date is inclusive in the period.
	`period`	The time period which acts as the end of the range
`ReleaseCalendar`		The schedule of publication or reporting of the data or metadata
	`periodicity`	The time period between the releases of the data or metadata
	`offset`	Interval between January 1st and the first release of the data
	`tolerance`	Period after which the data or metadata may be deemed late.

Copyright © SDMx TWG 2020

3.14 Data Provisioning

3.14.1 Class Diagram

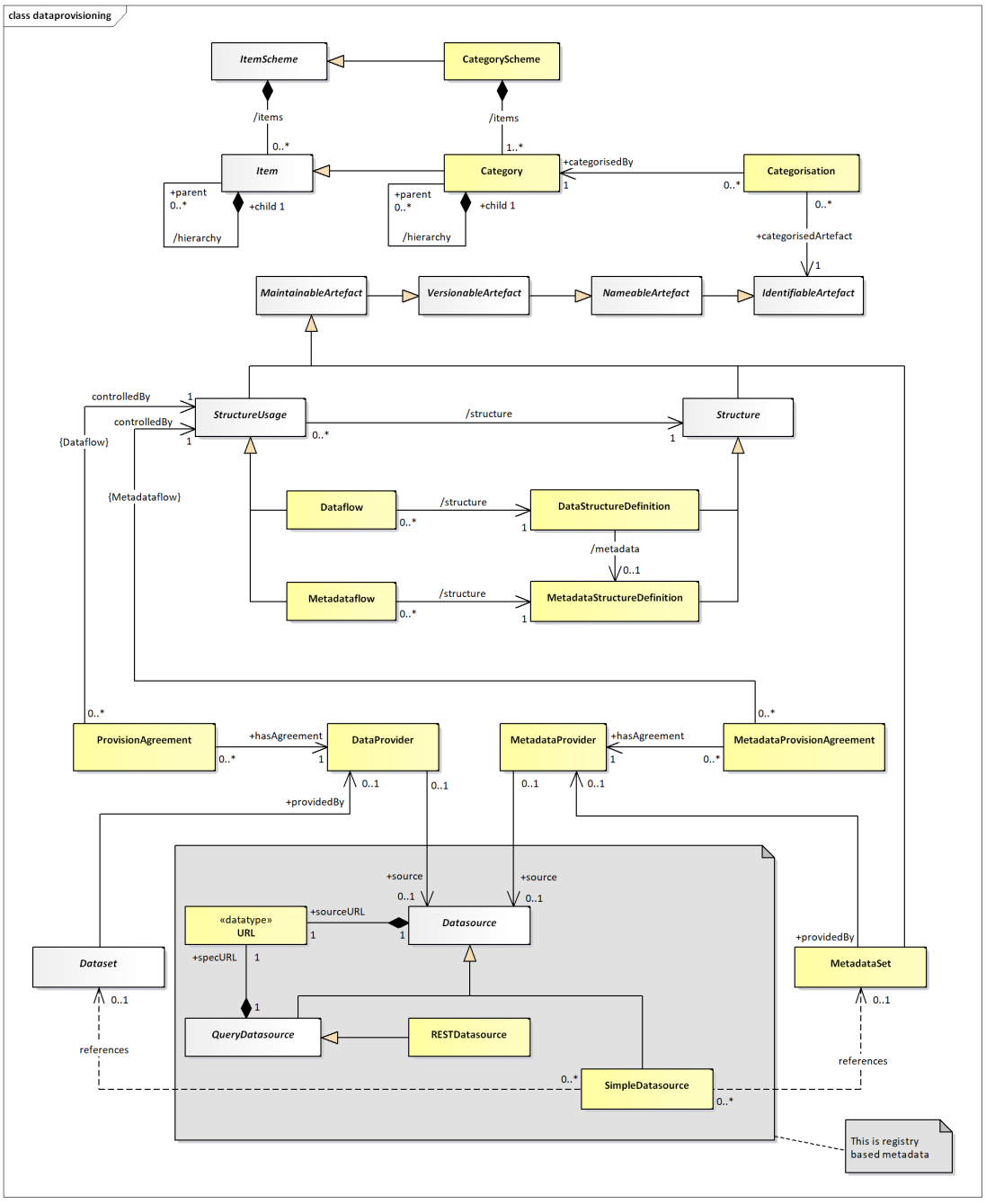


Figure 44. Relationship and inheritance class diagram of data/ metadata provisioning

Copyright © SDMx TWG 2020

3.14.2 Explanation of the Diagram

Narrative

This sub model links many artefacts in the SDMX-IM and is pivotal to an SDMX metadata registry, as all of the artefacts in this sub model must be accessible to an application that is responsible for data and metadata registration or for an application that requires access to the data or metadata.

Whilst a registry contains all of the metadata depicted on the diagram above, the classes in the grey shaded area are specific to a registry-based scenario where data sources (either physical data and metadata sets or databases and metadata repositories) are registered. More details on how these classes are used in a registry scenario can be found in the SDMX Registry Interface document. (Section Registry Specification).

A `ProvisionAgreement` / `MetadataProvisionAgreement` links the artefact that defines how data / metadata are structured and classified (`StructureUsage`) to the `DataProvider` / `MetadataProvider` . By means of a data or metadata registration, it references the `Datasource` (this can be data or metadata), whether this be an SDMX conformant file on a website (`SimpleDatasource`) or a database service capable of supporting an SDMX query and responding with an SDMX conformant document (`QueryDatasource`).

The `StructureUsage` , which has concrete classes of `Dataflow` and `Metadataflow` identifies the corresponding `DataStructureDefinition` or `MetadataStructureDefinition` , and, via Categorisation, can link to one or more `Category` (s) in a `CategoryScheme` such as a subject matter domain scheme, by which the `StructureUsage` can be classified. This can assist in drilling down from subject matter domains to find the data or metadata that may be relevant.

The `SimpleDatasource` links to the actual `DataSet` or `MetadataSet` on a website (this is shown on the diagram as a dependency called “references”). The `sourceURL` is obtained during the registration process of the `DataSet` or the `MetadataSet` . Additional information about the content of the `SimpleDatasource` is stored in the registry in terms of a `Constraint` (see the Section Constraints) for the Registration.

The `QueryDatasource` is an abstract class that represents a data source, which can understand an SDMX RESTful query (RESTDatasource) and respond appropriately. Each of these different `Datasource` s inherit the `dataURL` from `Datasource` , and the `QueryDatasource` has an additional URL, the `specURL` , to locate the specification of the service (i.e., the open API specification for RESTDatasource), which describes how to access it. All other supported protocols are assumed to use the `SimpleDatasource` URL.

The diagram below shows in schematic way the essential navigation through the SDMX structural artefacts that eventually link to a data or metadata registration[1].

Copyright © SDMx TWG 2020

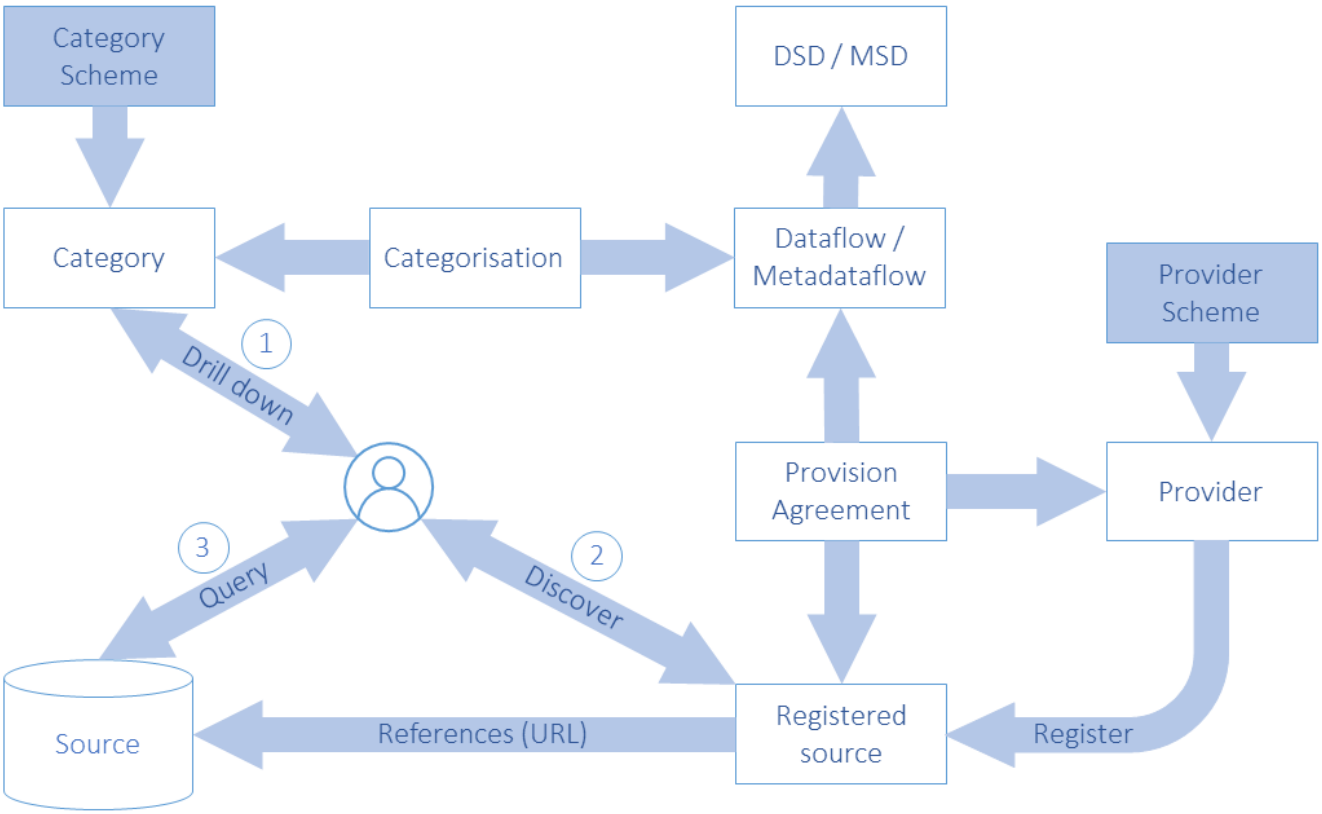


Figure 45. Schematic of the linking of structural metadata to data and metadata registration

Copyright © SDMx TWG 2020

Definitions

Copyright © SDMx TWG 2020

Class	Feature	Description
`StructureUsage`	Abstract class: Sub classes are: `Dataflow`, `Metadataflow`	This is described in the Base.
	`controlledBy`	Association to the `ProvisionAgreement`s that comprise the metadata related to the provision of data.
`DataProvider`		See `OrganisationScheme`.
	`hasAgreement`	Association to the `ProvisionAgreement`s for which the provider supplies data or metadata.
	`+source`	Association to a data source, which can process a data query.
`MetadataProvider`		See `OrganisationScheme`.
	`hasAgreement`	Association to the `MetadataProvisionAgreement`s for which the provider supplies data or metadata.
	`+source`	Association to a metadata source, which can process a metadata query.
`ProvisionAgreement`		Links the `DataProvider` to the relevant `StructureUsage` (i.e., the `Dataflow`) for which the provider supplies data. The agreement may constrain the scope of the data that can be provided, by means of a `DataConstraint`.
	`+source`	Association to a data source, which can process a data query.
`MetadataProvisionAgreement`		Links the `MetadataProvider` to the relevant `StructureUsage` (i.e., the `Metadataflow`) for which the provider supplies metadata. The agreement may constrain the scope of the metadata that can be provided, by means of a `MetadataConstraint`.
	`+source`	Association to reference metadata source, which can process a metadata query.
`Datasource`	Abstract class. Sub classes are: `SimpleDatasource`, `QueryDatasource`	Identification of the location or service from where data or reference metadata can be obtained.
	`+sourceURL`	The URL of the data or reference metadata source (a file or a web service).
`SimpleDatasource`		An SDMX dataset or metadataset accessible as a file at a URL.
`QueryDatasource`	Abstract class. Inherits from: `Datasource`. Sub classes are: `RESTDatasource`	A data or reference metadata source, which can process a data or metadata query.
`RESTDatasource`		A data or reference metadata source that is accessible via a RESTful web services interface.
	`+specificationURL`	Association to the URL for the specification of the web service.

Copyright © SDMx TWG 2020

Class	Feature	Description
`Registration`		This is not detailed here but is shown as the link between the SDMX-IM and the `RegistryServiceAPI`. It denotes a data or metadata registration document.

1. Provider Scheme, Provider, Provision Agreement and Registered source refer both to data and reference metadata. ↩

Copyright © SDMx TWG 2020

3.15 Process

3.15.1 Introduction

In any system that processes data and reference metadata the system itself is a series of processes and in each of these processes the data or reference metadata may undergo a series of transitions. This is particularly true of its path from raw data to published data and reference metadata. The process model presented here is a generic model that can capture key information about these stages in both a textual way and also in a more formalised way by linking to specific identifiable objects, and by identifying software components that are used.

3.15.2 Model – Inheritance and Relationship view

Class Diagram

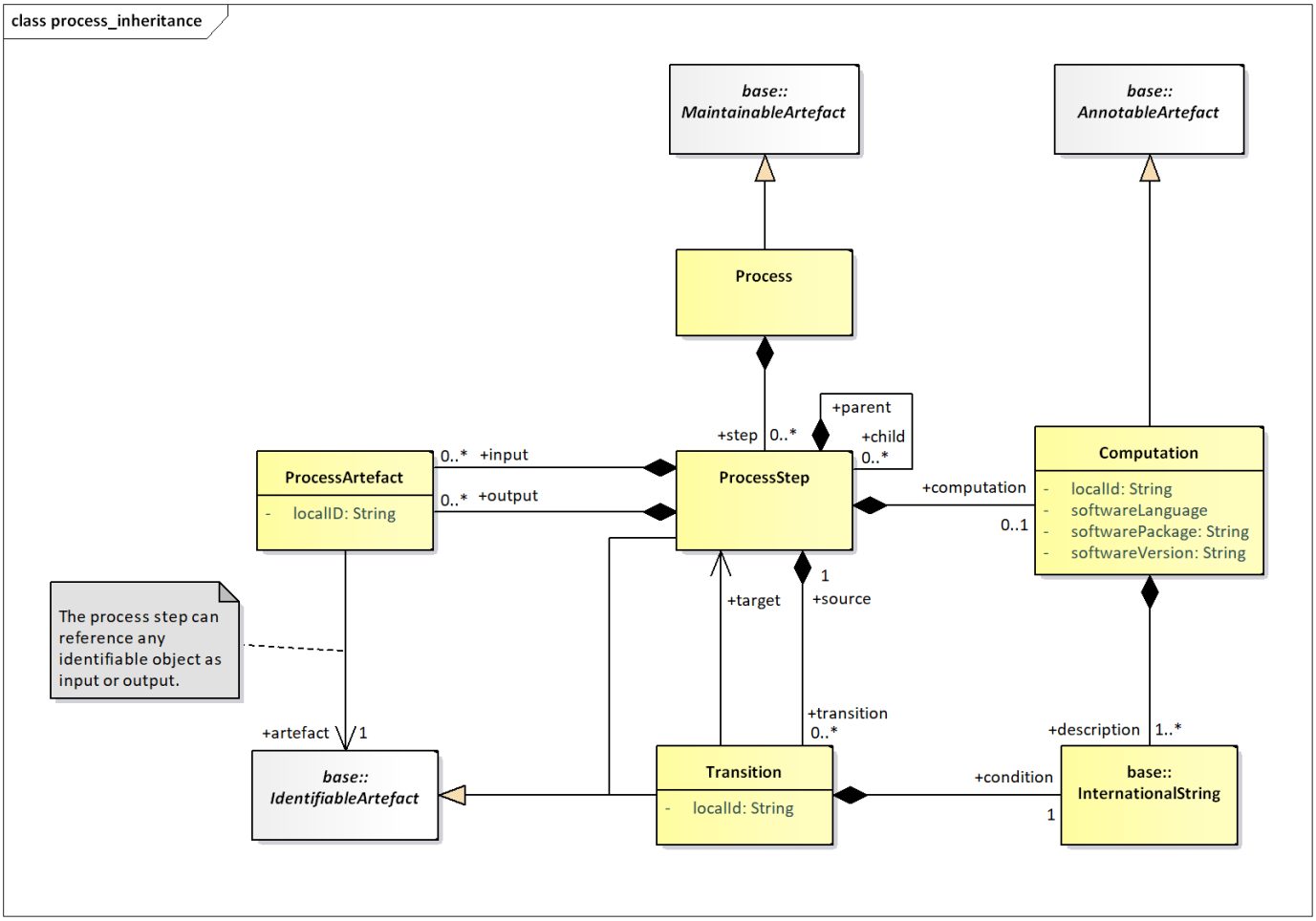


Figure 46. Inheritance and Relationship class diagram of Process and Transitions

Explanation of the Diagram

NARRATIVE

The Process is a set of hierarchical `ProcessSteps`. Each `ProcessStep` can take zero or more `IdentifiableArtefact`s as input and output. Each of the associations to the input and output `IdentifiableArtefact`s (`ProcessArtefact`) can be assigned a `localID`.

Copyright © SDMx TWG 2020

The computation performed by a `ProcessStep` is optionally described by a Computation, which can identify the software used by the `ProcessStep` and can also be described in textual form (`+description`) in multiple language variants. The Transition describes the execution of `ProcessSteps` from `+source` `ProcessStep` to `+target` `ProcessStep` based on the outcome of a `+condition` that can be described in multiple language variants.

Copyright © SDMx TWG 2020

DEFINITIONS

Class	Feature	Description
`Process`	Inherits from `Maintainable`	A scheme which defines or documents the operations performed on data or metadata in order to validate data or metadata to derive new information according to a given set of rules.
	`+step`	Associates the process steps.
`ProcessStep`	Inherits from `IdentifiableArtefact`	A specific operation, performed on data or metadata in order to validate or to derive new information according to a given set of rules.
	`+input`	Association to the process artefact that identifies the objects which are input to the process step.
	`+output`	Association to the process artefact that identifies the objects which are output from the process step.
	`+child`	Association to child processes that combine to form a part of this process.
	`+computation`	Association to one or more computations.
	`+transition`	Association to one or more transitions.
`Computation`		Describes in textual form the computations involved in the process.
	`localId`	Distinguishes between computations in the same process.
	`softwarePackage` / `softwareLanguage` / `softwareVersion`	Information about the software that is used to perform the computation.
	`+description`	Text describing or giving additional information about the computation. This can be in multiple language variants.
`Transition`	Inherits from `IdentifiableArtefact`	An expression in a textual or formalised way of the transformation of data between two specific operations (processes) performed on the data.
	`+target`	Associates the process step that is the target of the transition.
	`+condition`	Associates a textual description of the transition.
`ProcessArtefact`		Identification of an object that is an input to or an output from a process step.
	`+artefact`	Association to an identifiable artefact that is the input to or the output from the process step.

Copyright © SDMx TWG 2020

3.16 Validation and Transformation Language

3.16.1 Introduction

This SDMX model package supports the definition of Transformations, which are algorithms to calculate new data starting from already existing ones, written using the Validation and Transformation Language (VTL)[1].

The purpose of this model package is to enable the:

- definition of validation and transformation algorithms by means of VTL, in order to specify how to calculate new SDMX data from existing ones;
- exchange of the definition of VTL algorithms, also together the definition of the data structures of the involved data (for example, exchange the data structures of a reporting framework together with the validation rules to be applied, exchange the input and output data structures of a calculation task together with the VTL transformations describing the calculation algorithms);
- execution of VTL algorithms, either interpreting the VTL transformations or translating them in whatever other computer language is deemed as appropriate;

This model package does not explain the VTL language or any of the content published in the VTL guides. Rather, this is an illustration of the SDMX classes and attributes that allow defining VTL transformations applied to SDMX artefacts.

The SDMX model represented below is consistent with the VTL 2.0 specification. However, the former uses the SDMX terminology and is a model at technical level (from which the SDMX implementation artefacts for defining VTL transformations are built), whereas the latter uses the VTL terminology and is at conceptual level. The guidelines for mapping these terminologies and using the VTL in the SDMX context can be found in a dedicated chapter Validation and Transformation Language of the Section SDMX Technical Notes, often referenced below.

Copyright © SDMx TWG 2020

3.16.2 Model - Inheritance view

Class Diagram

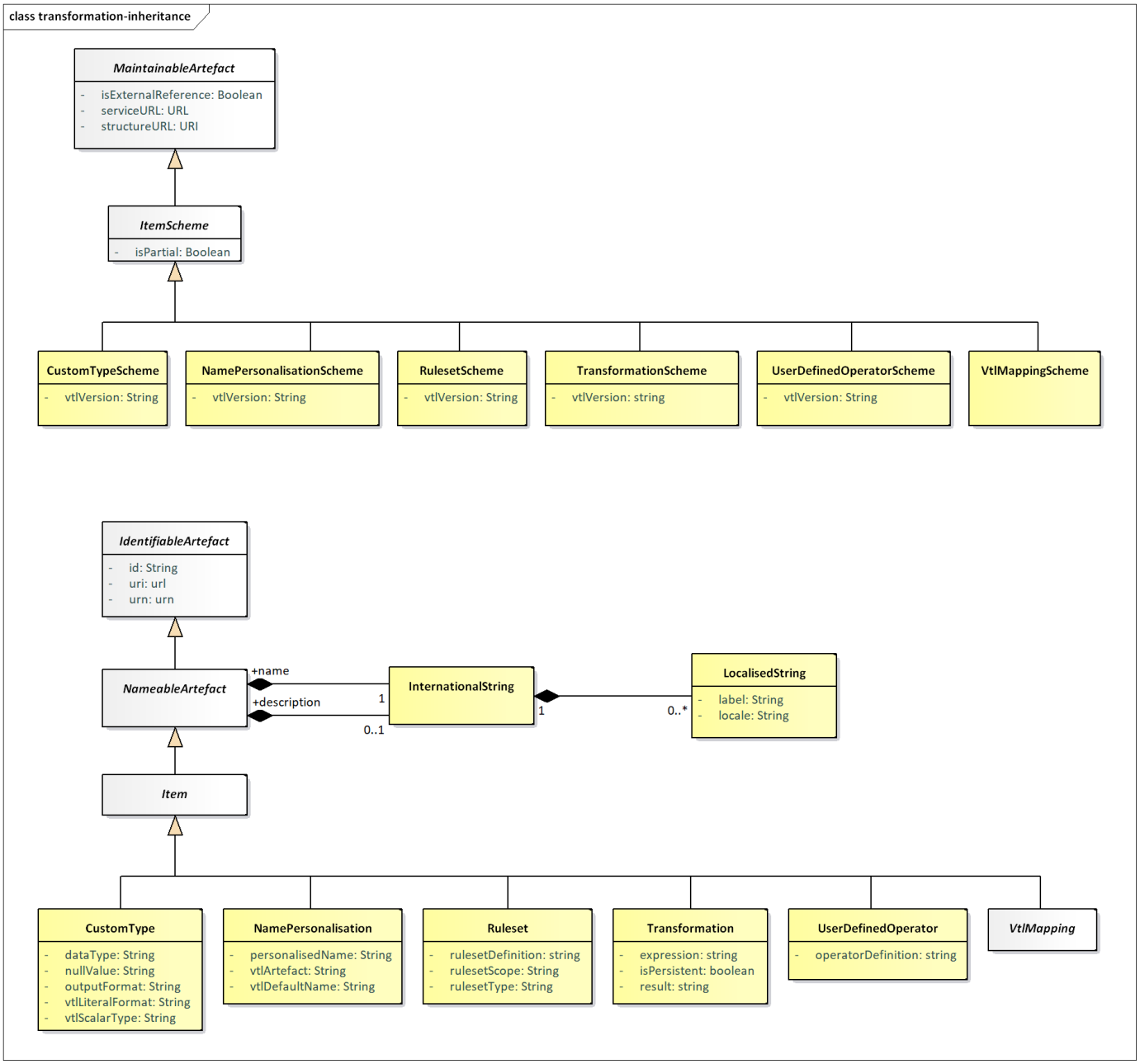


Figure 47. Class inheritance diagram in the Transformations and Expressions Package

Explanation of the Diagram

NARRATIVE

The model artefacts `TransformationScheme`, `RulesetScheme`, `UserDefinedOperatorScheme`, `NamePersonalisationScheme`, `CustomTypeScheme`, and `VtlMappingScheme` inherit from `ItemScheme`

Copyright © SDMx TWG 2020

These schemes inherit from the `ItemScheme` and therefore have the following attributes:

- `id`
- `uri`
- `urn`
- `version`
- `validFrom`
- `validTo`
- `isExternalReference`
- `registryURL`
- `structureURL`
- `repositoryURL`
- `isPartial`

The model artefacts `Transformation`, `Ruleset`, `UserDefinedOperator`, `NamePersonalisation`, `VtlMapping`, `CustomType` inherit the attributes and associations of `Item` which itself inherits from `NameableArtefact`. They have the following attributes:

- `id`
- `uri`
- `urn`

The multi-lingual name and description are provided by the relationship to `InternationalString` from `NameableArtefact`.

Copyright © SDMx TWG 2020

3.16.3 Model - Relationship View

Class Diagram

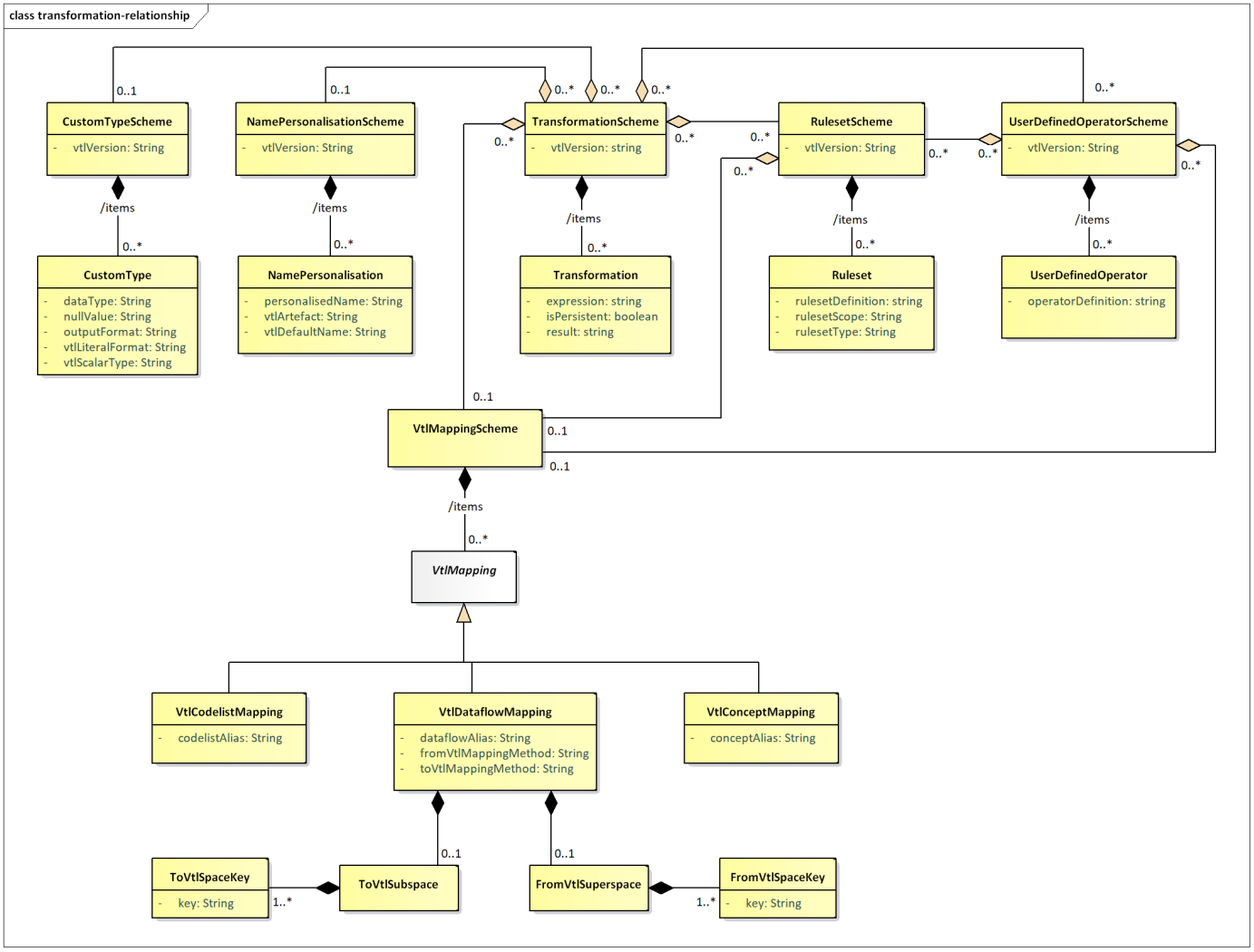


Figure 48. Relationship diagram in the Transformations and Expressions Package

Explanation of the Diagram

NARRATIVE - OVERVIEW

Transformation Scheme

A `TransformationScheme` is a set of `Transformation`s aimed at obtaining some meaningful results for the user (e.g. the validation of one or more Data Sets). This set of `Transformation`s is meant to be executed together (in the same run) and may contain any number of `Transformation`s in order to produce any number of results. Therefore, a `TransformationScheme` can be considered as a VTL program.

The `TransformationScheme` must include the attribute `vtlVersion` expressed as a string (e.g. “2.0”), as the version of the VTL determines which syntax is used in defining the transformations of the scheme.

A `Transformation` consists of a statement which assigns the outcome of the evaluation of a VTL expression to a result (an artefact of the VTL Information Model, which in the SDMX context can be a persistent or non-persistent Dataflow[2]).

Copyright © SDMx TWG 2020

For example, assume that D1, D2 and D3 are SDMX Dataflows (called Data Sets in VTL) containing information on some goods, specifically: D3 the current stocks, D1 the stocks of the previous date, D2 the flows in the last period. A possible VTL `Transformation` aimed at checking the consistency between flows and stocks is the following:

```
Dr := If ( (D1 + D2) = D3, then "true", else "false")
```

In this `Transformation`:

- `Dr` is the result (a new dataflow)
- `:=` is an assignment operator
- `If((D1+D2)=D3, then "true", else "false")` is the expression
- `D1, D2, D3` are the operands
- `If, ( ), +, =` are VTL operators

The `Transformation` model artefact contains three attributes:

1. `result`

   The left-hand side of a VTL statement, which specifies the Artefact to which the outcome of the expression is assigned. An artefact cannot be result of more than one `Transformation`.

2. `isPersistent`

   An assignment operator, which specifies also the persistency of the left-hand side. The assignment operators are two, namely `:=` for non-persistent assignment (the result is non-persistent) and `<-` for persistent assignment (the result is persistent).

3. `expression`

   The right-hand side of a VTL statement, which is the expression to be evaluated. An expression consists in the invocation of VTL operators in a certain order. When an operator is invoked, for each input parameter, an actual argument is passed to the operator, which returns an actual argument for the output parameter. An expression is simply a text string written according the VTL grammar.

Because an Artefact can be the result of just one `Transformation` and a Transformation belongs to just one `TransformationScheme`, it follows also that a derived Artefact (e.g., a new Dataflow) is produced in just one `TransformationScheme`.

The result of a `Transformation` can be input of other `Transformation`s. The VTL assumes that non-persistent results are maintained only within the same `TransformationScheme` in which they are produced. Therefore, a non-persistent result of a `Transformation` can be the operand of other Transformations of the same `TransformationScheme`, whereas a persistent result can be operand of transformations of any `TransformationScheme`[3].

The `TransformationScheme` has an association to zero of more `RulesetScheme`, zero or more `UserDefinedOperatorScheme`, zero or one `NamePersonalisationScheme`, zero or one `VtlMappingScheme`, and zero or one `CustomTypeScheme`.

The `RulesetScheme`, `UserDefinedOperatorScheme`, `NamePersonalisationScheme` and `CustomTypeScheme` have the attribute `vtlVersion`. Thus, a `TransformationScheme` using a specific version of VTL can be linked to such schemes only if they are consistent with the same VTL version.

The `VtlMappingScheme` associated to a `TransformationScheme` must contain the mappings between the references to the SDMX artefacts from the `TransformationScheme` and the structured identifiers of these SDMX artefacts.

Ruleset Scheme

Some VTL Operators can invoke rulesets, i.e., sets of previously defined rules to be applied by the Operator. Once defined, a Ruleset is persistent and can be invoked as many times as needed. The knowledge of the rulesets' definitions (if any) is essential for understanding the actual behaviour of the `Transformation` that use them: this is achieved through the `RulesetScheme` model artefact. The `RulesetScheme` is the container for one or more Ruleset.

Copyright © SDMx TWG 2020

The Ruleset model artefact contains the following attributes:

1. `rulesetType` – the type of the ruleset according to VTL (VTL 2.0 allows two types: “datapoint” and “hierarchical” ruleset);
2. `rulesetScope` – the VTL artefact on which the ruleset is defined; VTL 2.0 allows rulesets defined on Value Domains, which correspond to SDMX Codelists and rulesets defined on Variables, which correspond to SDMX Concepts for which a definite Representation is assumed;
3. `rulesetDefinition` – the VTL statement that defines the ruleset according to the syntax of the VTL definition language.

The `RulesetScheme` can have an association with zero or more `VtlMappingScheme`. These mappings define the correspondence between the references to the SDMX artefacts contained in the `rulesetDefinition` and the structured identifiers of these SDMX artefacts.

The rulesets defined on Value Domains reference `Codelists`. The rulesets defined on Variables reference `Concepts` (for which a definite `Representation` is assumed). In conclusion, in the VTL rulesets there can exist mappings for: `Codelists` and `Concepts`.

User Defined Operator Scheme

The `UserDefinedOperatorScheme` is a container for zero of more `UserDefinedOperator`. The `UserDefinedOperator` is defined using VTL standard operators. This is essential for understanding the actual behaviour of the `Transformation`s that invoke them.

The attribute `operatorDefinition` contains the VTL statement that defines the operator according to the syntax of the VTL definition language.

Although the VTL user defined operators are conceived to be defined on generic operands, so that the specific artefacts to be manipulated are passed as parameters at the invocation, it is also possible that they reference specific SDMX artefacts like `Dataflows` and `Codelists`. Therefore, the `UserDefinedOperatorScheme` can link to zero or one `VtlMappingScheme`, which must contain the mappings between the VTL references and the structured URN of the corresponding SDMX artefacts (see also the VTL mapping section below).

The definition of a `UserDefinedOperator` can also make use of VTL rulesets; therefore, the `UserDefinedOperatorScheme` can link to zero, one or more `RulesetScheme`, **which** must contain the definition of these Rulesets (see also the Ruleset Scheme Section above).

Name Personalisation Scheme

In some operations, the VTL assigns by default some standard names to some measures and/or attributes of the data structure of the result[4]. The VTL allows also to personalise the names to be assigned. The knowledge of the personalised names (if any) is essential for understanding the actual behaviour of the `Transformation`: this is achieved through the `NamePersonalisationScheme`. A `NamePersonalisation` specifies a personalised name that will be assigned in place of a VTL default name. The `NamePersonalisationScheme` is a container for zero or more `NamePersonalisation`.

VTL Mapping

The mappings between SDMX and VTL can be relevant to the names of the artefacts and to the methods for converting the data structures from SDMX to VTL and vice-versa. These features are achieved through the `VtlMappingScheme`, which is a container for zero or more `VtlMapping`.

The VTL assumes that the operands are directly referenced through their actual names (unique identifiers). In the VTL transformations, rulesets, user defined operators, the SDMX artefacts are referenced through VTL aliases. The alias can be the complete URN of the artefact, an abbreviated URN, or another user-defined name, as described in the Section 6 of the SDMX Standards.[5]

The VTLmapping defines the correspondence between the VTL alias and the structured identifier of the SDMX artefact, for each referenced SDMX artefact. This correspondence is needed for the following kinds of SDMX artefacts: `Dataflows`, `Codelists` and `Concepts`. Therefore, there are the following corresponding mapping subclasses: `VtlDataflowMapping`, `VtlCodelistMapping` and `VtlConceptMapping`.

As for the `Dataflows`, it is also possible to specify the method to convert the Data Structure of the Dataflow. This kind of conversion can happen in two directions, from SDMX to VTL when a SDMX Dataflow is accessed by a VTL Transformation (`toVtlMappingMethod`), or from VTL to SDMX when a SDMX derived Dataflow is calculated through VTL (`fromVtlMappingMethod`).[6]

Copyright © SDMx TWG 2020

The default mapping method from SDMX to VTL is called “Basic”. Three alternative mapping methods are possible, called “Pivot”, “Basic-A2M”, “Pivot-A2M” (“A2M” stands for “Attributes to Measures”, i.e. the SDMX `DataAttributes` become VTL measures).

The default mapping method from VTL to SDMX is also called “Basic”, and the two alternative mapping methods are called “Unpivot” and “M2A” (“M2A” stands for “Measures to Attributes”, i.e. some VTL measures become SDMX `DataAttributes` according to what is declared in the DSD).

In both the mapping directions, no specification is needed if the default mapping method (Basic) is used. When an alternative mapping method is applied for some Dataflow, this must be specified in `toVtlMappingMethod` or `fromVtlMappingMethod`.

`ToVtlSubspace`, `ToVtlSpaceKey`, `FromVtlSuperspace`, `FromVtlSpaceKey`

Although in general one SDMX Dataflow is mapped to one VTL dataset and vice-versa, it is also allowed to map distinct parts of a single SDMX Dataflow to distinct VTL data sets according to the rules and conventions described in the Section 6 of the SDMX Standards.[7]

In the direction from SDMX to VTL, this is achieved by fixing the values of some predefined Dimensions of the SDMX Data Structure: all the observations having such combination of values are mapped to one corresponding VTL dataset (the Dimensions having fixed values are not maintained in the Data Structure of the resulting VTL dataset). The `ToVtlSubspace` and `ToVtlSpaceKey` classes allow to define these Dimensions. When one SDMX Dataflow is mapped to just one VTL dataset these classes are not used.

Analogously, in the direction from VTL to SDMX, it is possible to map more calculated VTL datasets to distinct parts of a single SDMX Dataflow, as long as these VTL datasets have the same Data Structure. This can be done by providing, for each VTL dataset, distinct values for some additional SDMX Dimensions that are not part of the VTL data structure. The `FromVtlSuperspace` and `FromVtlSpaceKey` classes allow to define these dimensions. When one VTL dataset is mapped to just one SDMX Dataflow these classes are not used.

Custom Type Scheme

As already said, a `Transformation` consists of a statement which assigns the outcome of the evaluation of a VTL expression to a result, i.e. an artefact of the VTL Information Model. which in the SDMX context can be a persistent or non-persistent `Dataflow`[8]. Therefore, the VTL data type of the outcome of the VTL expression has to be converted into the SDMX data type of the resulting `Dataflow`. A default conversion table from VTL to SDMX data types is assumed[9]. The `CustomTypeScheme` allows to specify custom conversions that override the default conversion table. The `CustomTypeScheme` is a container for zero or more `CustomType`. A `CustomType` specifies the custom conversion from a VTL scalar type that will override the default conversion. The overriding SDMX data type is specified by means of the `dataType` and `outputFormat` attributes (the SDMX data type assumes the role of external representation in respect to VTL[10]).

Moreover, the `CustomType` allows to customize the default format of VTL literals and the (possible) SDMX value to be produced when a VTL measure or attribute is NULL.

VTL expression can contain literals, i.e. specific values of a certain VTL data type written according to a certain format. For example, consider the following `Transformation` that extracts from the dataflow D1 the observations for which the “reference_date” belongs to the years 2018 and 2019:

```
Dr := D1 [ filter between (reference_date, 2018-01-01, 2019-12-31)]
```

In this expression, the two values `2018-01-01` and `2019-12-31` are literals of the VTL “date” scalar type expressed in the format `YYYY-MM-DD`.

The VTL literals are assumed to be written in the same SDMX format specified in the default conversion table mentioned above, for the conversion from VTL to SDMX data types. If a different format is used for a certain VTL scalar type, it must be specified in the `vtlLiteralFormat` attribute of the `CustomType`.

Regarding the management of NULLs, in the conversions between SDMX and VTL, by default a missing value in SDMX in converted in VTL NULL and vice-versa, for any VTL scalar type. If a different value is needed, after the conversion from SDMX to VTL, proper VTL operators can be used for obtaining it. In the conversion from VTL to SDMX the desired value can be declared in the `nullValue` attribute (separately for each VTL basic scalar type).

Copyright © SDMx TWG 2020

DEFINITIONS

Copyright © SDMx TWG 2020

Class	Feature	Description
Transformation Scheme	Inherits from `ItemScheme`	Contains the definitions of transformations meant to produce some derived data and be executed together
	`vtlVersion`	The version of the VTL language used for defining transformations
Transformation	Inherits from `Item`	A VTL statement which assigns the outcome of an expression to a result.
	result	The left-hand side of the VTL statement, which identifies the result artefact.
	`isPersistent`	A boolean that indicates whether the result is permanently stored or not, depending on the VTL assignment operator.
	expression	The right-hand side of the VTL statement that is the expression to be evaluated, which includes the references to the operands of the Transformation.
`RulesetScheme`	Inherits from `ItemScheme`	Container of rulesets.
	`vtlVersion`	The version of the VTL language used for defining the rulesets
Ruleset	Inherits from `Item`	A persistent set of rules which can be invoked by means of appropriate VTL operators.
	`rulesetDefinition`	A VTL statement for the definition of a ruleset (according to the syntax of the VTL definition language)
	`rulesetType`	The VTL type of the ruleset (e.g., in VTL 2.0, datapoint or hierarchical)
	`rulesetScope`	The model artefact on which the ruleset is defined (e.g., in VTL 2.0, valuedomain or variable)
`UserDefinedOperatorScheme`	Inherits from `ItemScheme`	Container of user defined operators

Copyright © SDMx TWG 2020

Class	Feature	Description
	`vtlVersion`	The version of the VTL language used for defining the user defined operators
`UserDefinedOperator`	Inherits from `Item`	Custom VTL operator (not existing in the standard library) that extends the VTL standard library for specific purposes.
	`operatorDefinition`	A VTL statement for the definition of a new operator: it specifies the operator name, its parameters and their data types, the VTL expression that defines its behaviour.
`NamePersonalisationScheme`	Inherits from `ItemScheme`	Container of name personalisations.
	`vtlVersion`	The VTL version which the VTL default names to be personalised belong to.
`NamePersonalisation`	Inherits from `Item`	Definition of personalised name to be used in place of a VTL default name.
	`vtlArtefact`	VTL model artefact to which the VTL default name to be personalised refers, e.g. variable, value domain.
	`vtlDefaultName`	The VTL default name to be personalised.
	`personalisedName`	The personalised name to be used in place of the VTL default name.
`VtlMappingScheme`	Inherits from `ItemScheme`	Container of VTL mappings.
`VtlMapping`	Inherits from `Item` Sub classes are: `VtlDataflowMapping`, `VtlCodelistMapping`, `VtlConceptMapping`	Single mapping between the reference to a SDMX artefact made from VTL transformations, rulesets, user defined operators and the corresponding SDMX structure identifier.
`VtlDataflowMapping`	Inherits from `VtlMapping`	Single mapping between the reference to a SDMX dataflow and the corresponding SDMX structure identifier
	`dataflowAlias`	Alias used in VTL to reference a SDMX

Copyright © SDMx TWG 2020

Class	Feature	Description
		dataflow (it can be the URN, the abbreviated URN or a user defined alias). The alias must be univocal: different SDMX artefacts cannot have the same VTL alias.
	`toVtlMappingMethod`	Custom specification of the mapping method from SDMX to VTL data structures for the dataflow (overriding the default “basic” method).
	`fromVtlMappingMethod`	Custom specification of the mapping method from VTL to SDMX data structures for the dataflow (overriding the default “basic” method).
`VtlCodelistMapping`	Inherits from `VtlMapping`	Single mapping between the VTL reference to a SDMX codelist and the SDMX structure identifier of the codelist.
	`codelistAlias`	Name used in VTL to reference a SDMX codelist. The name/alias must be univocal: different SDMX artefacts cannot have the same VTL alias.
`VtlConceptMapping`	Inherits from `VtlMapping`	Single mapping between the VTL reference to a SDMX concept and the SDMX structure identifier of the concept.
	`conceptAlias`	Name used in VTL to reference a SDMX concept. The name/alias must be univocal: different SDMX artefacts cannot have the same VTL alias.
`ToVtlSubspace`		Subspace of the dimensions of the SDMX dataflow used to identify the parts of the dataflow to be mapped to distinct VTL datasets
`ToVtlSpaceKey`		A dimension of the SDMX dataflow that contributes to identify the parts of the dataflow to be mapped to distinct VTL datasets.

Copyright © SDMx TWG 2020

Class	Feature	Description
	Key	The identity of the dimension in the data structure definition of the dataflow that contributes to identify the parts of the dataflow to be mapped to distinct VTL datasets
`FromVtlSuperspace`		Superspace is composed of the dimensions to be added to the data structure of the VTL result dataset in order to obtain the data structure of the derived SDMX dataflow (in case the latter is a superset of distinct VTL datasets calculated independently).
`FromVtlSpaceKey`		A SDMX dimension to be added to the data structure of the VTL result dataset in order to obtain the data structure of the derived SDMX dataflow
	Key	The identity of the dimension to be added to the data structure of the VTL result dataset in order to obtain the data structure of the derived SDMX dataflow.
`CustomTypeScheme`	Inherits from `ItemScheme`	Container of custom specifications for VTL basic scalar types.
	`vtlVersion`	The VTL version, which the VTL scalar types belong to.
`CustomType`	Inherits from `Item`	Custom specification for a VTL basic scalar type.
	`vtlScalarType`	VTL scalar type for which the custom specifications are given.
	`outputFormat`	Custom specification of the VTL formatting mask needed to obtain to the desired representation, i.e. the desired SDMX format (e.g. YYYY-MM-DD, see also the VTL formatting mask in the VTL Reference Manual and the SDMX Technical Notes). If not

Copyright © SDMx TWG 2020

Class	Feature	Description
		specified, the “Default output format” of the default conversion table from VTL to SDMX is used.[11]
	`datatype`	Custom specification of the external (SDMX) data type in which the VTL data type must be converted (e.g. the `GregorianDay`). If not specified, the “Default SDMX data type” of the default conversion table from VTL to SDMX is used.[12]
	`nullValue`	Custom specification of the SDMX value to be produced for the VTL NULL values, with reference to the `vtlScalarType` specified above. If no value is specified, no value is produced.
	`vtlLiteralFormat`	Custom specification of the format of the VTL literals belonging to the `vtlScalarType` used in the VTL program (e.g. YYYY-MM-DD)[13]. If not specified, the “Default output format” of the default conversion table from VTL to SDMX is assumed.[14]

1. The Validation and Transformation Language is a standard language designed and published under the SDMX initiative. VTL is described in the VTL User and Reference Guides available on the SDMX website. ↩
2. Or a part of a Dataflow, see also the Section “Mapping dataflow subsets to distinct VTL data sets” of the SDMX Technical Notes. ↩
3. Provided that the VTL consistency rules are accomplished (see the “Generic Model for Transformations” in the VTL User Manual and its sub-section “Transformation Consistency”) ↩
4. For example, the `check` operator produces some new components in the result called by default `bool_var`, `errorcode`, `errorlevel`, `imbalance`. These names can be personalised if needed. ↩
5. See Section “References to SDMX artefacts from VTL statements” in the SDMX Technical Notes. ↩
6. For a more thorough description of these conversions, see the Section “Mapping between SDMX and VTL” in the SDMX Technical Notes. ↩
7. See the Section “Mapping dataflow subsets to distinct VTL data sets” in the SDMX Technical Notes. ↩
8. Or a part of a Dataflow, as described in the previous paragraph. ↩
9. The default conversion table from VTL to SDMX is described in the Section “Mapping VTL basic scalar types to SDMX data types” in the SDMX Technical Notes. ↩
10. About VTL internal and external representations, see also the VTL User Manual, section “Basic scalar types”, p.53. ↩
11. See “Mapping VTL basic scalar types to SDMX data types” in the SDMX Technical Notes. ↩
12. See “Mapping VTL basic scalar types to SDMX data types” in the SDMX Technical Notes. ↩
13. See also the VTL formatting mask in the VTL Reference Manual and the SDMX Technical Notes. ↩
14. See “Mapping VTL basic scalar types to SDMX data types” in the SDMX Technical Notes. ↩

Copyright © SDMx TWG 2020

3.17 Appendix 1: A Short Guide to UML in the SDMX Information Model

3.17.1 Scope

The scope of this document is to give a brief overview of the diagram notation used in UML. The examples used in this document have been taken from the SDMX UML model.

3.17.2 Use Cases

In order to develop the data models it is necessary to understand the functions that require to be supported. These are defined in a use case model. The use case model comprises actors and use cases and these are defined below.

The **actor** can be defined as follows:

> "An actor defines a coherent set of roles that users of the system can play when interacting with it. An actor instance can be played by either an individual or an external system"

The actor is depicted as a stick man as shown below.



Figure 49. Actor

The **use case** can be defined as follows:

> "A use case defines a set of use-case instances, where each instance is a sequence of actions a system performs that yields an observable result of value to a particular actor"



Figure 50. Use case



Figure 51. Actor and use case

Copyright © SDMx TWG 2020

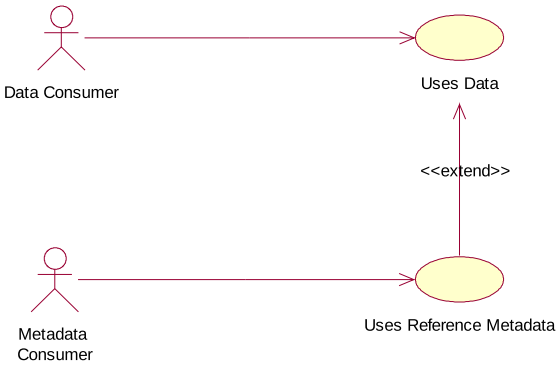


Figure 52. Extend use cases

An extend use case is where a use case may be optionally extended by a use case that is independent of the using use case. The arrow in the association points to he owning use case of the extension. In the example above the Uses Data use case is optionally extended by the Uses Metadata use case.

3.17.3 Classes and Attributes

General

A class is something of interest to the user. The equivalent name in an entity-relationship model (E-R model) is the entity and the attribute. In fact, if the UML is used purely as a means of modelling data, then there is little difference between a class and an entity.

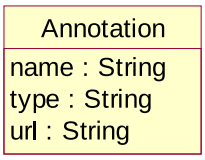


Figure 53. Class and its attributes

Figure 53 shows that a class is represented by a rectangle split into three compartments. The top compartment is for the class name, the second is for attributes and the last is for operations. Only the first compartment is mandatory. The name of the class is `Annotation`, and it belongs to the package SDMX-Base. It is common to group related artefacts (classes, use-cases, etc.) together in packages. `.Annotation` has three “String” attributes – `name`, `type`, and `url`. The full identity of the attribute includes its class e.g. the name attribute is `Annotation.name`.

Note that by convention the class names use `UpperCamelCase` – the words are concatenated and the first letter of each word is capitalized. An attribute uses `lowerCamelCase` - the first letter of the first (or only) word is not capitalized, the remaining words have capitalized first letters.

Abstract Class

An abstract class is drawn because it is a useful way of grouping classes, and avoids drawing a complex diagram with lots of association lines, but where it is not foreseen that the class serves any other purpose (i.e. it is always implemented as one of its sub classes). In the diagram in this document an abstract class is depicted with its name in italics, and coloured white.



Figure 54. Abstract and concrete classes

Copyright © SDMx TWG 2020

3.17.4 Associations

General

In an E-R model these are known as relationships. A UML model can give more meaning to the associations than can be given in an E-R relationship. Furthermore, the UML notation is fixed (i.e. there is no variation in the way associations are drawn). In an E-R diagram, there are many diagramming techniques, and it is the relationship in an E-R diagram that has many forms, depending on the particular E-R notation used.

Simple Association

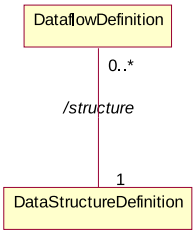


Figure 55. A simple association

Here the `DataflowDefinition` class has an association with the DataStructureDefinition class. The diagram shows that a `DataflowDefinition` can have an association with only one DataStructureDefinition (1) and that a `DataStructureDefinition` can be linked to many `DataflowDefinitions` (0..*). The association is sometimes named to give more semantics.

In UML it is possible to specify a variety of “multiplicity” rules. The most common ones are:

- Zero or one (0..1)
- Zero or many (0..*)
- One or many (1..*)
- Many (*)
- Unspecified (blank)

Aggregation

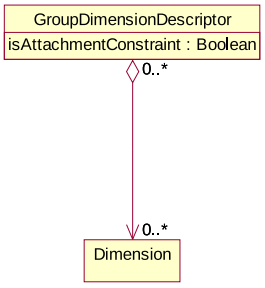


Figure 56. A simple aggregate association

Copyright © SDMx TWG 2020

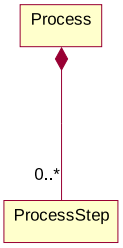


Figure 57. A composition aggregate association

An association with an aggregation relationship indicates that one class is a subordinate class (or a part) of another class. In an aggregation relationship. There are two types of aggregation, a simple aggregation where the child class instance can outlive its parent class, and a composition aggregation where the child class's instance lifecycle is dependent on the parent class's instance lifecycle. In the simple aggregation it is usual, in the SDMX Information model, for this association to also be a reference to the associated class.

Association Names and Association-end (role) Names

It can be useful to name associations as this gives some more semantic meaning to the model i.e. the purpose of the association. It is possible for two classes to be joined by two (or more) associations, and in this case it is extremely useful to name the purpose of the association. Figure 58 shows a simple aggregation between `CategoryScheme` and `Category` called `/items` (this means it is derived from the association between the super classes – in this case between the `ItemScheme` and the `Item`, and another between Category called `/hierarchy`.

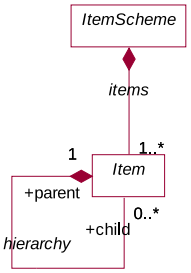


Figure 58. Association names and end names

Furthermore, it is possible to give role names to the association-ends to give more semantic meaning – such as parent and child in a tree structure association. The role is shown with `+` preceding the role name (e.g. in the diagram above the semantic of the association is that a `Item` can have zero or one parent `Items` and zero or many child `Item`).

In this model the preference has been to use role names for associations between concrete classes and association names for associations between abstract classes. The reason for using an association name is often useful to show a physical association between two sub classes that inherit the actual association between the super class from which they inherit. This is possible to show in the UML with association names, but not with role names. This is covered later in "Derived Association".

Note that in general the role name is given at just one end of the association.

Navigability

Associations are, in general, navigable in both directions. For a conceptual data model it is not necessary to give any more semantic than this.

However, UML allows a notation to express navigability in one direction only. In this model this "navigability" feature has been used to represent referencing. In other words, the class at the navigable end of the association is referenced from the class at the non-navigable end. This is aligned, in general, with the way this is implemented in the XML schemas.

Copyright © SDMx TWG 2020



Figure 59. One way association

Here it is possible to navigate from `A` to `B`, but there is no implementation support for navigation from `B` to `A` using this association.

Inheritance

Sometimes it is useful to group common attributes and associations together in a super class. This is useful if many classes share the same associations with other classes, and have many (but not necessarily all) attributes in common. Inheritance is shown as a triangle at the super class.

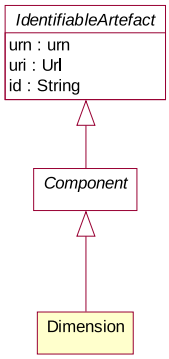


Figure 60. Inheritance

Here the `Dimension` is derived from `Component` which itself is derived from `IdentifiableArtefact`. Both `Component` and `IdentifiableArtefact` are abstract superclasses. The `Dimension` inherits the attributes and associations of all of the the super classes in the inheritance tree. Note that a super class can be a concrete class (i.e. it exists in its own right as well as in the context of one of its sub classes), or an abstract class.

Derived association

It is often useful in a relationship diagram to show associations between sub classes that are derived from the associations of the super classes from which the sub classes inherit. A derived association is shown by `/` preceding the association name e.g. `/name`.

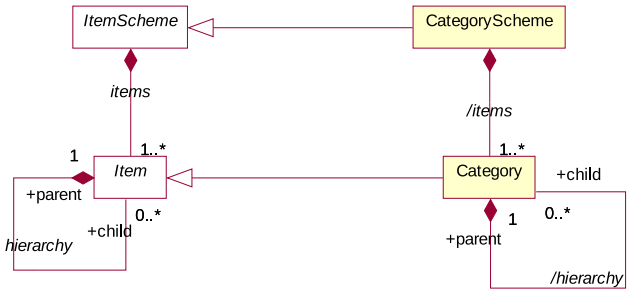


Figure 61. Derived associations

Copyright © SDMx TWG 2020

4. Registry Specification and Logical Interfaces

4.1 Registry Specification

4.1.1 Introduction

The business vision for SDMX envisages the promotion of a “data sharing” model to facilitate low-cost, high-quality statistical data and metadata exchange. Data sharing reduces the reporting burden of organisations by allowing them to publish data once and let their counterparties “pull” data and related metadata as required. The scenario is based on:

- the availability of an abstract information model capable of supporting time series and cross-sectional data, structural metadata, and reference metadata (SDMX-IM)
- standardised XML and JSON schemas for the SDMX-ML and SDMX-JSON formats derived from the model (XSD, JSON)
- the use of web-services technology (XML, JSON, Open API)

Such an architecture needs to be well organised, and the SDMX Registry/Repository (SDMX-RR) is tasked with providing structure, organisation, and maintenance and query interfaces for most of the SDMX components required to support the data sharing vision.

However, it is important to emphasise that the SDMX-RR provides support for the submission and retrieval of all SDMX structural metadata and provisioning metadata. Therefore, the Registry not only supports the data-sharing scenario, but this metadata is also vital in order to provide support for data and metadata reporting/collection, and dissemination scenarios.

Standard formats for the exchange of aggregated statistical data and metadata as prescribed in SDMX v3.0 are envisaged to bring benefits to the statistical community because data reporting and dissemination processes can be made more efficient.

As organisations migrate to SDMX enabled systems, many XML, JSON (and conventional) artefacts will be produced (e.g., Data Structure, Metadata Structure, Code List and Concept definitions – often collectively called structural metadata – XML schemas generated from data structure definitions, XSLT stylesheets for transformation and display of data and metadata, terminology references, etc.). The SDMX model supports interoperability, and it is important to be able to discover and share these artefacts between parties in a controlled and organized way.

This is the role of the registry.

With the fundamental SDMX standards in place, a set of architectural standards are needed to address some of the processes involved in statistical data and metadata exchange, with an emphasis on maintenance, retrieval and sharing of the structural metadata. In addition, the architectural standards support the registration and discovery of data and referential metadata.

These architectural standards address the ‘how’, rather than the ‘what’, and are aimed at enabling existing SDMX standards to achieve their mission. The architectural standards address registry services, which initially comprise:

- structural metadata repository
- data and metadata registration
- query

The registry services outlined in this specification are designed to help the SDMX community manage the proliferation of SDMX assets and to support data sharing for reporting and dissemination.

Copyright © SDMx TWG 2020

4.1.2 Scope and Normative Status

The scope of this document is to specify the logical interfaces for the SDMX registry in terms of the functions required and the data that may be present in the function call, and the behaviour expected of the registry.

In this document, functions and behaviours of the Registry Interfaces are described in four ways:

- in text
- with tables
- with UML diagrams excerpted from the SDMX Information Model (SDMX-IM)
- with UML diagrams that are not a part of the SDMX-IM but are included here for clarity and to aid implementations (these diagrams are clearly marked as “Logical Class Diagram …”)

Whilst the introductory section contains some information on the role of the registry, it is assumed that the reader is familiar with the uses of a registry in providing shared metadata across a community of counterparties.

Note that chapters 5 and 6 below contain normative rules regarding the Registry Interface and the identification of registry objects. Further, the minimum standard for access to the registry is via a REST interface (HTTP or HTTPS), as described in the appropriate sections. The notification mechanism must support e-mail and HTTP/HTTPS protocols as described. Normative registry interfaces are specified in the SDMX-ML specification (Section 3 of the SDMX Standard). All other sections of this document are informative.

Note that although the term “authorised user” is used in this document, the SDMX standards do not define an access control mechanism. Such a mechanism, if required, must be chosen and implemented by the registry software provider.

Copyright © SDMx TWG 2020

4.1.3 Scope of the SDMX Registry/Repository

Objective

The objective of the SDMX registry/repository is, in broad terms, to allow organisations to publish statistical data and reference metadata in known formats such that interested third parties can discover these data and interpret them accurately and correctly. The mechanism for doing this is twofold:

1. To maintain and publish structural metadata that describes the structure and valid content of data and reference metadata sources such as databases, metadata repositories, data sets, metadata sets. This structural metadata enables software applications to understand and to interpret the data and reference metadata in these sources.
2. To enable applications, organisations, and individuals to share and to discover data and reference metadata. This facilitates data and reference metadata dissemination by implementing the data sharing vision of SDMX.

Structural Metadata

Setting up structural metadata and the exchange context (referred to as “data provisioning”) involves the following steps for maintenance agencies:

- agreeing and creating a specification of the structure of the data (called a Data Structure Definition or DSD in this document but also known as “key family”), which defines the dimensions, measures and attributes of a dataset and their valid value set;
- if required, defining a subset or view of a DSD which allows some restriction of content called a “dataflow definition”;
- agreeing and creating a specification of the structure of reference metadata (Metadata Structure Definition) which defines the metadata attributes and their presentational arrangement in a Metadataset or as part of a Dataset, and their valid values and content;
- if required, defining a subset or view of an MSD which allows some restriction of content called a “metadataflow”;
- defining which subject matter domains (specified as a Category Scheme) are related to the Dataflow and Metadataflow to enable browsing;
- defining one or more lists of Data and Metadata Providers;
- defining which Data/Metadata Providers have agreed to publish a given Dataflow/Metadataflow – this is called a Provision Agreement or Metadata Provision Agreement, respectively.

Copyright © SDMx TWG 2020

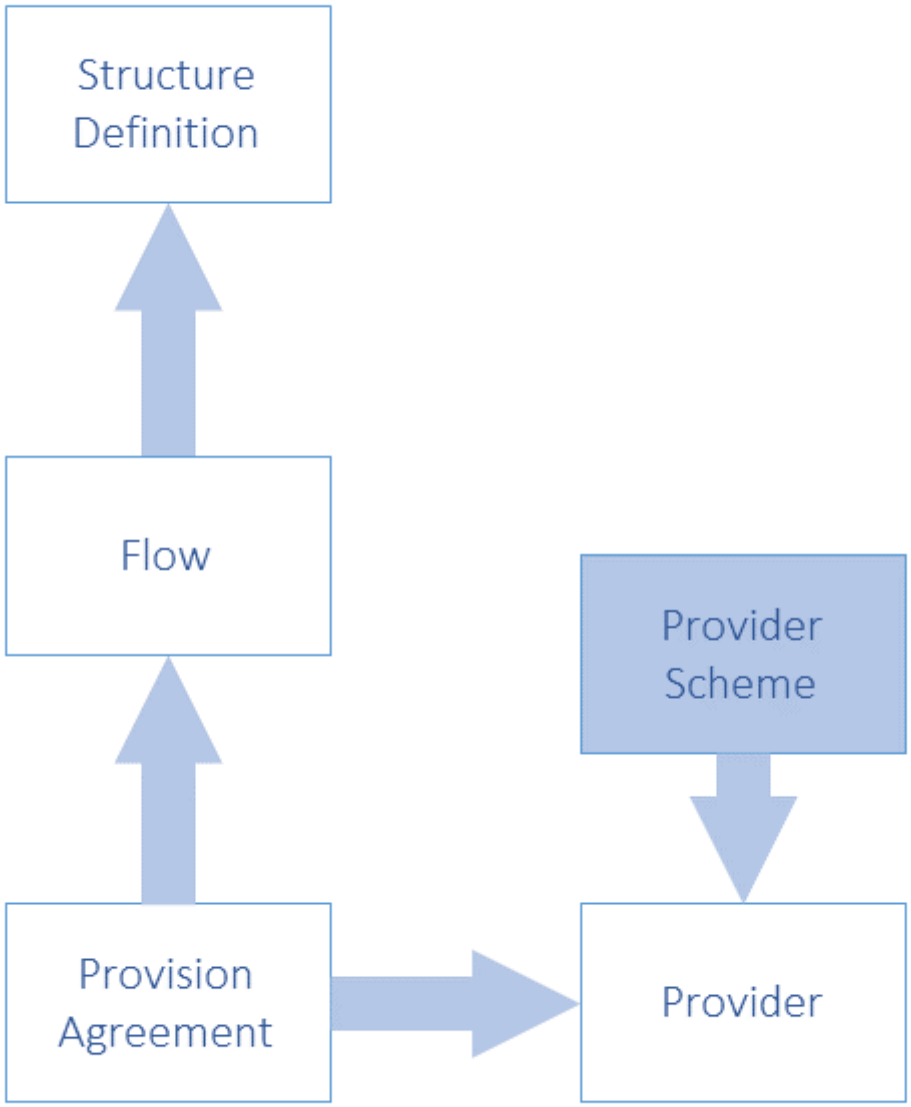


Figure 1. Schematic of the Basic Structural Artefacts in the SDMX-IM

Note that in Figure 1 (but also most of the relevant subsequent figures) terms that include both data and metadata have been used. For example:

- Structure Definition: refers to Data Structure Definition (DSD) and Metadata Structure Definition (MSD)
- Flow: refers to Dataflow and Metadataflow
- Provision Agreement: refers to Provision Agreement (for data) and Metadata Provision Agreement
- Provider Scheme: refers to Data Provider Scheme and Metadata Provider Scheme
- Provider: refers to Data Provider and Metadata Provider

In that context, the term “Metadata” refers to reference metadata.

Copyright © SDMx TWG 2020

Registration

Publishing the data and reference metadata involves the following steps for a Data/Metadata Provider:

- making the reference metadata and data available in SDMX-ML/JSON conformant data files or databases (which respond to an SDMX query with data). The data and reference metadata files or databases must be web accessible, and must conform to an agreed Dataflow or Metadataflow (Data Structure Definition or Metadata Structure Definition subset);
- registering the existence of published reference metadata and data files or databases with one or more SDMX registries.

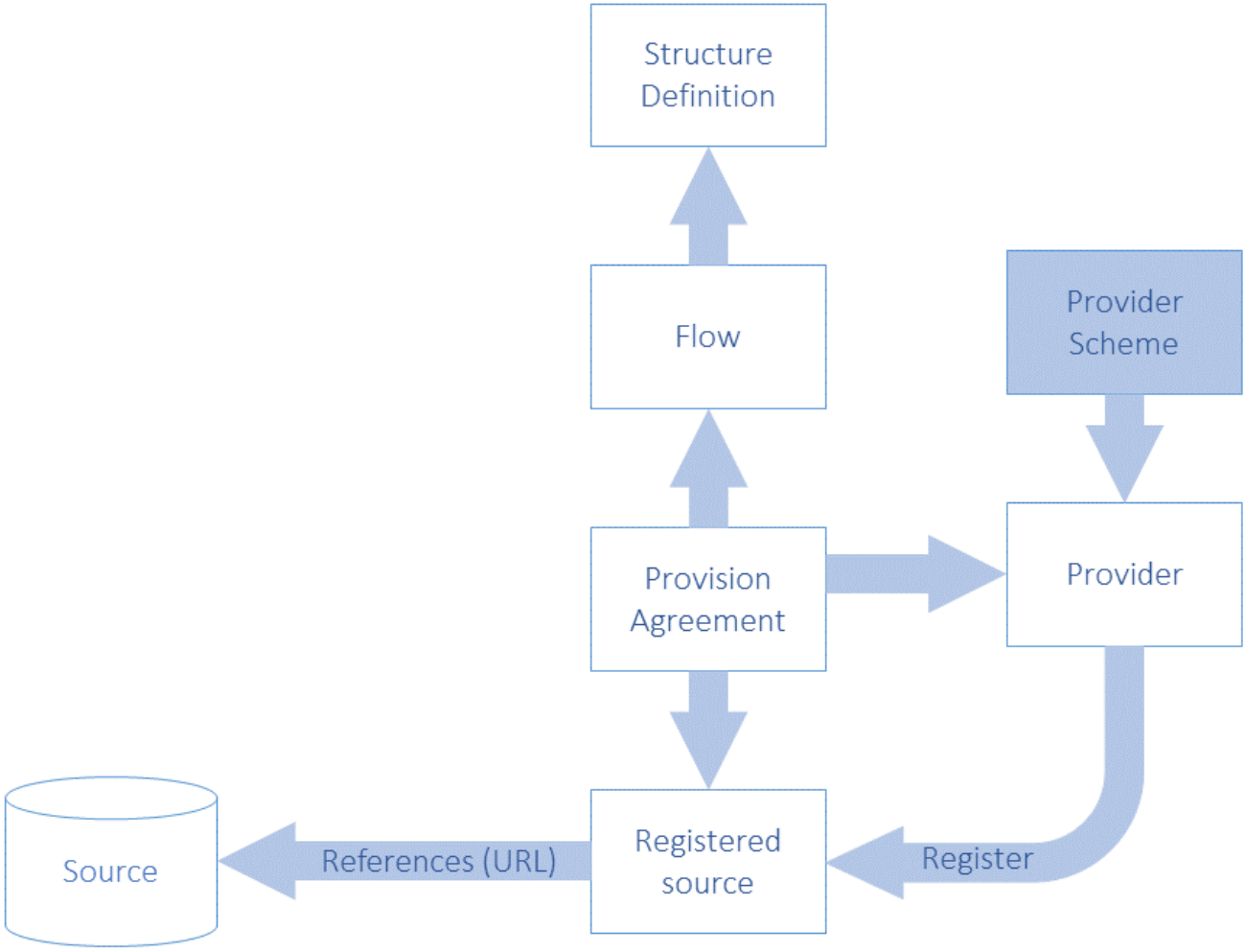


Figure 2. Schematic of Registered Data and Metadata Sources in the SDMX-IM

Notification

Notifying interested parties of newly published or re-published data, reference metadata or changes in structural metadata involves:

- registry support of a subscription-based notification service which sends an email or notifies an HTTP address announcing all published data that meets the criteria contained in the subscription request.

Copyright © SDMx TWG 2020

Discovery

Discovering published data and reference metadata involves interaction with the registry to fulfil the following logical steps that would be carried out by a user interacting with a service that itself interacts with the registry and an SDMX-enabled data or reference metadata resource:

- optionally browsing a subject matter domain category scheme to find Dataflows (and hence Data Structure Definitions) and Metadataflows which structure the type of data and/or reference metadata being sought;
- build a query, in terms of the selected Data Structure Definition or Metadata Structure Definition, which specifies what data are required and submitting this to a service that can query an SDMX registry which will return a list of (URLs of) data and reference metadata files and databases which satisfy the query;
- processing the query result set and retrieving data and/or reference metadata from the supplied URLs.

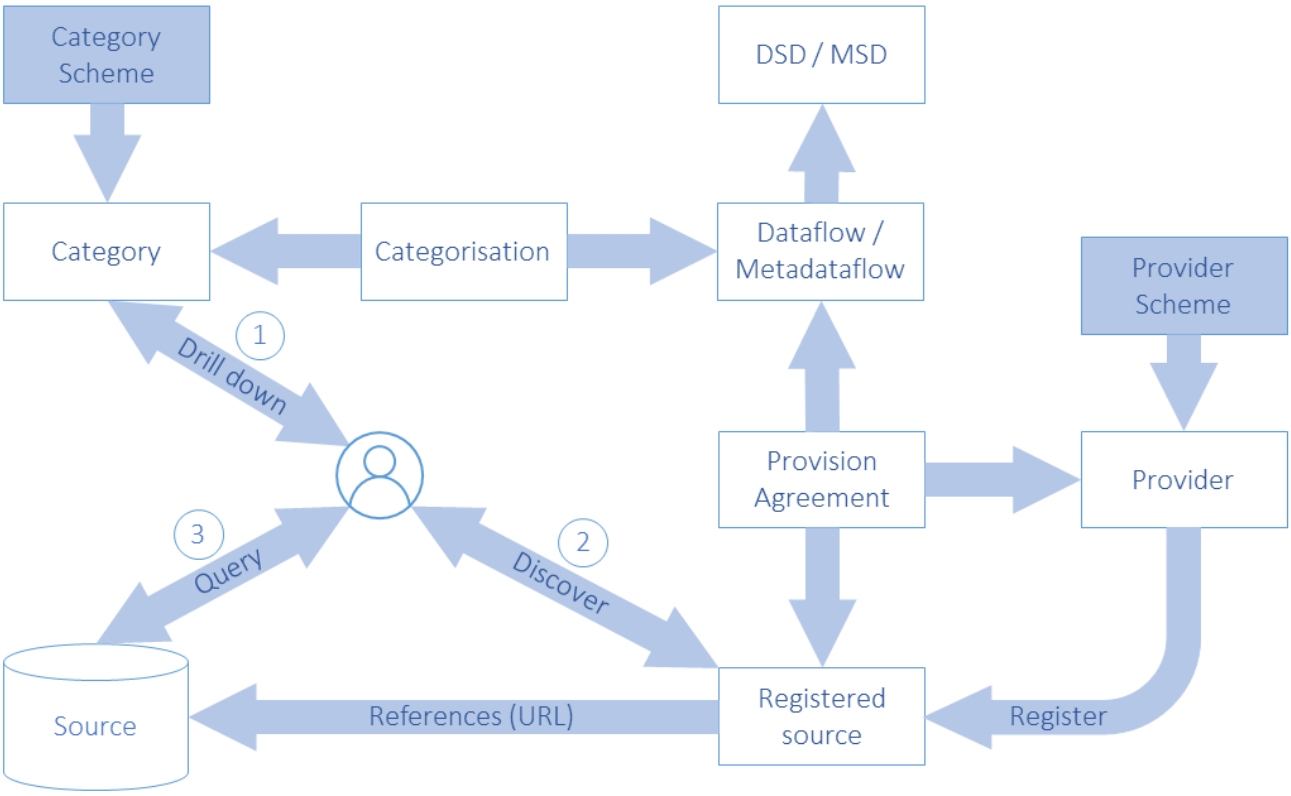


Figure 3. Schematic of Data and Metadata Discovery and Query in the SDMX-IM

Copyright © SDMx TWG 2020

4.1.4 SDMX Registry/Repository Architecture

Architectural Schematic

The architecture of the SDMX registry/repository is derived from the objectives stated above. It is a layered architecture that is founded by a structural metadata repository which supports a provisioning metadata repository which supports the registry services. These are all supported by the SDMX-ML schemas. Applications can be built on top of these services which support the reporting, storage, retrieval, and dissemination aspects of the statistical lifecycle as well as the maintenance of the structural metadata required to drive these applications.

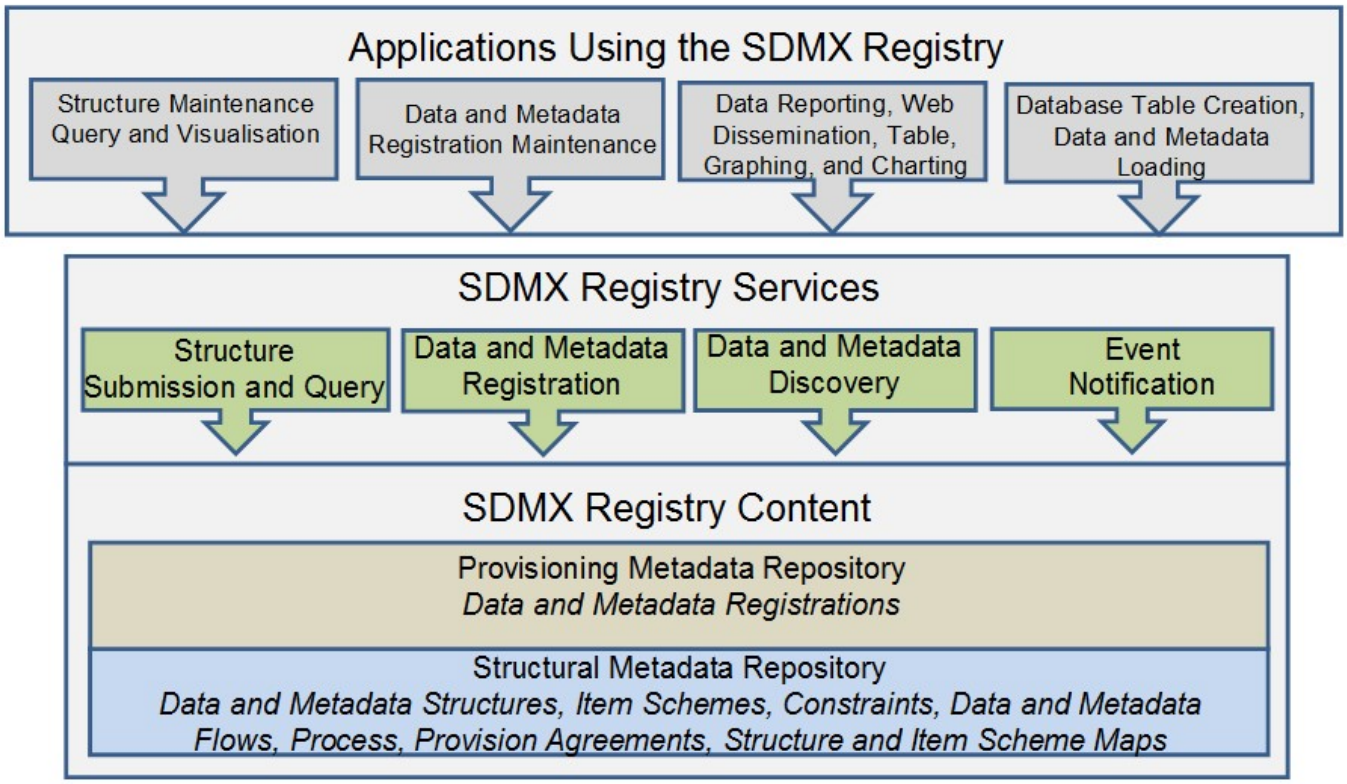


Figure 4. Schematic of the Registry Content and Services

Structural Metadata Repository

The basic layer is that of a structural metadata service which supports the lifecycle of SDMX structural metadata artefacts such as Maintenance Agencies, Data Structure Definitions, Metadata Structure Definitions, Provision Agreements, Processes etc. This layer is supported by the Structure Submission and Query Service.

Note that the SDMX REST API supports all of the SDMX structural artefacts. The only structural artefacts that are not yet supported are:

- Registration of data and metadata sources
- Subscription and Notification

As of the initial version of SDMX 3.0 no messages are defined to support these artefacts; hence, users may need to use SDMX 2.1 Registry Interface messages, instead.

Copyright © SDMx TWG 2020

Provisioning Metadata Repository

The function of this repository is to support the definition of the structural metadata that describes the various types of data-store which model SDMX-conformant databases or files, and to link to these data sources. These links can be specified for a data/metadata provider, for a specific data or metadata flow. In the SDMX model this is called the Provision or Metadata Provision Agreement.

This layer is supported by the Data and Metadata Registration Service.

Copyright © SDMx TWG 2020

4.1.5 Registry Interfaces and Services

Registry Interfaces

The Registry Interfaces are:

- Notify Registry Event
- Submit Subscription Request
- Submit Subscription Response
- Submit Registration Request
- Submit Registration Response
- Query Registration Request
- Query Registration Response
- Query Subscription Request
- Query Subscription Response

The registry interfaces are invoked in one of two ways:

1. The interface is the name of the root node of the SDMX-ML document
2. The interface is invoked as a child element of the RegistryInterface message where the RegistryInterface is the root node of the SDMX-ML document.

In addition to these interfaces the registry must support a mechanism for submitting and querying for structural metadata. This is detailed in Sections Structure Submission Service and Structure Query Service.

All these interactions with the Registry – with the exception of NotifyRegistryEvent – are designed in pairs. The first document, the one which invokes the SDMX-RR interface, is a “Request” document. The message returned by the interface is a “Response” document.

It should be noted that all interactions are assumed to be synchronous, with the exception of Notify Registry Event. This document is sent by the SDMX-RR to all subscribers whenever an even occurs to which any users have subscribed. Thus, it does not conform to the request-response pattern, because it is inherently asynchronous.

Registry Services

INTRODUCTION

The services described in this section do not imply that each is implemented as a discrete web service.

STRUCTURE SUBMISSION SERVICE

The registry must support a mechanism for submitting structural metadata. This mechanism can be the SDMX REST interface for structural metadata (this is defined in the corresponding GitHub project and the dedicated Section on the SDMX REST API). In order for the architecture to be scalable, the finest-grained piece of structural metadata that can be processed by the SDMX-RR is a MaintainableArtefact, with the exception of Item Schemes, where changes at an Item level is also possible (see next section on the SDMX Information Model).

STRUCTURE QUERY SERVICE

The registry must support a mechanism for querying for structural metadata. This mechanism can be the SDMX REST interface for structural metadata (this is defined in the corresponding GitHub project and the dedicated Section on the SDMX REST API). The registry response to this query mechanism is the SDMX Structure message, which has as its root node:

- `Structure`

Copyright © SDMx TWG 2020

The SDMX structural artefacts that may be queried are:

- data flows and metadata flows
- data structure definitions and metadata structure definitions
- code lists
- value lists
- concept schemes
- reporting taxonomies
- provision agreements and metadata provision agreements
- structure maps
- representation map
- organisation scheme map
- concept scheme map
- category scheme map
- reporting taxonomy map
- processes
- hierarchies
- constraints
- category schemes
- categorisations and categorised objects (examples are categorised data flows and metadata flows, data structure definitions, metadata structure definitions, provision agreements registered data sources and metadata sources)
- organisation schemes (agency scheme, data provider scheme, data consumer scheme, organisation unit scheme)

Due to the VTL implementation the other structural metadata artefacts that may be queried are:

- Transformation schemes
- Custom type schemes
- Name personalisation schemes
- VTL mapping schemes
- Ruleset schemes
- User defined operator schemes

DATA AND REFERENCE METADATA REGISTRATION SERVICE

This service must implement the following Registry Interfaces:

- `SubmitRegistrationRequest`
- `SubmitRegistrationResponse`
- `QueryRegistrationRequest`
- `QueryRegistrationResponse`

The Data and Metadata Registration Service allows SDMX conformant files and web-accessible databases containing published data and reference metadata to be registered in the SDMX Registry. The registration process MAY validate the content of the datasets or metadata-sets, and MAY extract a concise representation of the contents in terms of concept values (e.g., values of the data attribute, dimension, metadata attribute), or entire keys, and storing this as a record in the registry to enable discovery of the original dataset or metadata-set. These are called Constraints in the SDMX-IM.

Copyright © SDMx TWG 2020

The Data and Metadata Registration Service MAY validate the following, subject to the access control mechanism implemented in the Registry:

- that the data/metadata provider is allowed to register the dataset or metadataset;
- that the content of the dataset or metadataset meets the validation constraints. This is dependent upon such constraints being defined in the structural repository and which reference the relevant Dataflow, Metadataflow, Data Provider, Metadata Provider, Data Structure Definition, Metadata Structure Definition, Provision Agreement, Metadata Provision Agreement;
- that a queryable data source exists – this would necessitate the registration service querying the service to determine its existence;
- that a simple data source exists (i.e., a file accessible at a URL);
- that the correct Data Structure Definition or Metadata Structure Definition is used by the registered data;
- that the components (Dimensions, Attributes, Measures, Metadata Attributes, etc.) are consistent with the Data Structure Definition or Metadata Structure Definition;
- that the valid representations of the concepts to which these components correspond conform to the definition in the Data Structure Definition or Metadata Structure Definition.

The Registration has an action attribute which takes one of the following values:

Action Attribute Value	**Behaviour**
`Append`	Add this registration to the registry
`Replace`	Replace the existing registration with this registration identified by the `id` in the registration of the submit registration request
`Delete`	Delete the existing registration identified by the `id` in the registration of the submit registration request

The Registration has three Boolean attributes which may be present to determine how an SDMX compliant dataset or metadataset indexing application must index the datasets or metadatasets upon registration. The indexing application behaviour is as follows:

Boolean Attribute	**Behaviour if Value is `true`**
`indexTimeSeries`	A compliant indexing application must index all the time series keys (for a Dataset registration) or metadata target values (for a Metadataset registration).
`indexDataSet`	A compliant indexing application must index the range of actual (present) values for each dimension of the Dataset (for a Dataset registration) or the range of actual (present) values for each Metadata Attribute which takes an enumerated value.
Note that for data this requires much less storage than full key indexing, but this method cannot guarantee that a specific combination of Dimension values (the Key) is actually present in the Dataset.	
`indexReportingPeriod`	A compliant indexing application must index the time period range(s) for which data are present in the Dataset. The validity period of the Metadatasets may also be indexed.

DATA AND REFERENCE METADATA DISCOVERY

The Data and Metadata Discovery Service implements the following Registry Interfaces:

- `QueryRegistrationRequest`
- `QueryRegistrationResponse`

Copyright © SDMx TWG 2020

SUBSCRIPTION AND NOTIFICATION

The Subscription and Notification Service implements the following Registry Interfaces:

- `SubmitSubscriptionRequest`
- `SubmitSubscriptionResponse`
- `NotifyRegistryEvent`

The data sharing paradigm relies upon the consumers of data and metadata being able to pull information from data providers' dissemination systems. For this to work efficiently, a data consumer needs to know when to pull data, i.e., when something has changed in the registry (e.g., a dataset has been updated and re-registered). Additionally, SDMX systems may also want to know if a new Data Structure Definition, Code List or Metadata Structure Definition has been added. The Subscription and Notification Service comprises two parts: subscription management, and notification.

Subscription management involves a user submitting a subscription request which contains:

- a query or constraint expression in terms of a filter which defines the events for which the user is interested (e.g., new data for a specific dataflow, or for a domain category, or changes to a Data Structure Definition).
- a list of URIs or endpoints to which an XML notification message can be sent. Supported endpoint types will be email (mailto:) and HTTP POST (a normal http:// address);
- request for a list of submitted subscriptions;
- deletion of a subscription;

Notification requires that the structural metadata repository and the provisioning metadata repository monitor any event which is of interest to a user (the object of a subscription request query), and to issue an SDMX notification document to the endpoints specified in the relevant subscriptions.

REGISTRY BEHAVIOUR

The following table defines the behaviour of the SDMX Registry for the various Registry Interface messages. It should be noted, though, that as of SDMX 3.0, an extended versioning scheme newly including semantic versioning is foreseen for all Maintainable Artefacts. Moreover, while the old versioning scheme is allowed, given there is no more a "final" flag, there is no way guaranteeing the consistency across version of a Maintainable, unless semantic versioning is used.

Given the above, the behaviour described in the following table concerns either draft Artefacts using semantic versioning or any Artefacts using the old versioning scheme. Nevertheless, in the case of semantic versioning the registry must respect the versioning rules when performing the actions below. For example, it is not possible to replace a non-draft Artefact that follows semantic versioning, unless a newer version is introduced according to the semantic versioning rules. Furthermore, even when draft Artefacts are submitted, the registry has to verify semantic versioning is respected against the previous non-draft versions. It is worth noting that the rules for semantic versioning and replacing or maintaining semantically versioned Artefacts applies to externally shared Artefacts. This means that any system may internally perform any change within a version of an Artefact, until

Copyright © SDMx TWG 2020

the latter is shared outside of that system or becomes public. Then (as also explained in the SDMX Standards Section Technical Notes) the Artefacts must adhere to the Semantic Versioning rules.

Interface	Behaviour
All	1. If the action is set to `replace` (or a maintainable artefact is PUT or POSTed) then the entire contents of the existing maintainable object in the registry **must** be replaced by the object submitted. 2. Cross referenced structures **must** exist in either the submitted document (in `Structures` or `StructureLocation`) or in the registry to which the request is submitted. 3. If the action is set to `delete` (or a maintainable artefact is DELETEd) then the registry **must** verify that the object can be deleted. In order to qualify for deletion, the object must: a. Be a draft version. b. Not be explicitly[1] referenced from any other object in the registry. 4. The semantic versioning rules in the SDMX documentation **must** be obeyed.
Structure submission	Structures are submitted at the level of the Maintainable Artefact and the behaviour in “All” above is therefore at the level of the Maintainable Artefact.
`SubmitRegistrationRequest`	If the datasource is a file (simple datasource) then the file *may* be retrieved and indexed according to the Boolean attributes set in the registration. For a queryable datasource the registry *may* validate that the source exists and can accept an SDMX data query.

1. With semantic versioning, it is allowed to reference a range of artefacts, e.g., a DSD referencing a Codelist with version 1.2.3+ means all patch versions greater than 1.2.3. This means that deleting 1.2.4-draft does not break integrity of the aforementioned DSD. ↩

Copyright © SDMx TWG 2020

4.2 Logical Interfaces

4.2.1 Identification of SDMX Objects

Identification, Versioning, and Maintenance

All major classes of the SDMX Information model inherit from one of:

- `IdentifiableArtefact` – this gives an object the ability to be uniquely identified (see following section on identification), to have a user-defined URI, and to have multi-lingual annotations.
- `NameableArtefact` – this has all of the features of `IdentifiableArtefact` plus the ability to have a multi-lingual name and description.
- `VersionableArtefact` – this has all of the above features plus a version number, according to the SDMX versioning rules in SDMX Standards Section 6 “Technical Notes”, paragraph “4.3 Versioning”, and a validity period.
- `MaintainableArtefact` – this has all of the above features, plus registry and structure URIs, and an association to the maintenance organisation of the object.

Copyright © SDMx TWG 2020

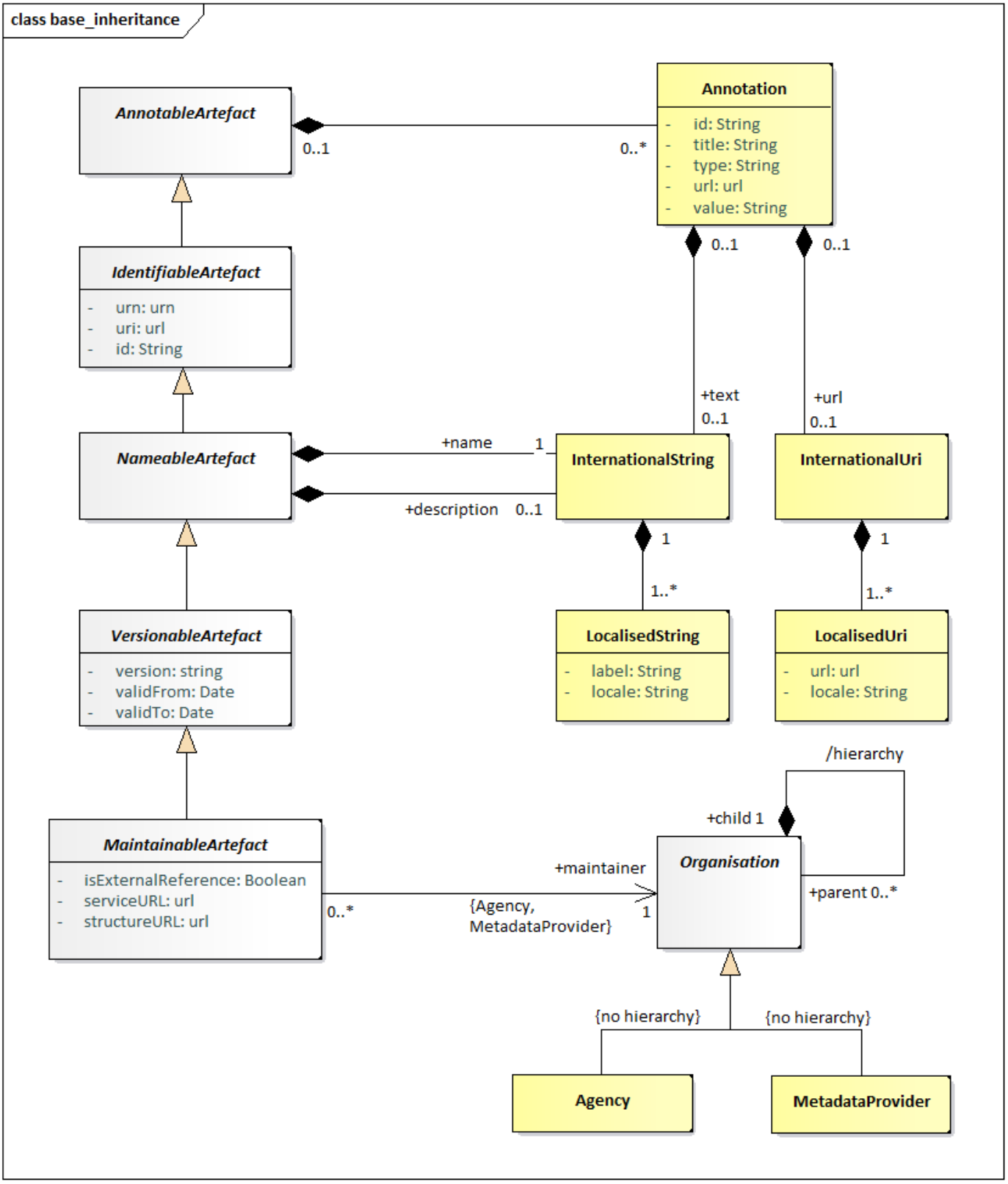


Figure 5. Class diagram of fundamental artefacts in the SDMX-IM

Copyright © SDMx TWG 2020

The table below shows the identification and related data attributes to be stored in a registry for objects that are one of:

- `Annotable`
- `Identifiable`
- `Nameable`
- `Versionable`
- `Maintainable`

Copyright © SDMx TWG 2020

Object Type	Data Attributes	Status	Data type	Notes
`Annotable`	`annotationTitle`	C	string	
	`AnnotationType`	C	string	
	`AnnotationURN`	C	string	
	`AnnotationText` (in the form of `InternationalString`)	C		This can have language-specific variants
`Identifiable`	All content as for `Annotable` plus			
	`id`	M	string	
	`uri`	C	string	
	`urn`	C	string	Although the `urn` is computable and may not be submitted or stored physically, the registry must return the `urn` for each object, and must be able to service a query on an object referenced solely by its `urn`.
`Nameable`	All content as for `Identifiable` plus			
	`Name` in the form of `InternationalString`	M	string	Can have language-specific variants
	`Description` in the form of `InternationalString`	C	string	Can have language-specific variants
`Versionable`	All content as for `Identifiable` plus			
	`version`	M	string	This is the version number according to SDMX versioning rules
	`validFrom`	C	date/time	
	`validTo`	C	date/time	
`Maintainable`	All content as for `Versionable` plus			
	`isExternalReference`	C	boolean	Value of `true` indicates that the actual resource is held outside of this registry. The actual reference is given in the registry URI or the `structureURL`, each of which must return a valid SDMX-ML file.
	`serviceURL`	C	string	The URL of the service that can be queried for this resource

Copyright © SDMx TWG 2020

Object Type	Data Attributes	Status	Data type	Notes
	`structureURL`	C	string	The URL of the resource
	(Maintenance) `organisationId`	M	string	The object must be linked to a maintenance organisation, i.e., agency or metadata provider

Table 1. Table: Common Attributes of Object Types Common Attributes of Object Types

Unique identification of SDMX objects

AGENCIES AND METADATA PROVIDERS

The Maintenance Agency in SDMX is maintained in an Agency Scheme which itself is a sub class of Organisation Scheme – this is shown in the class diagram below.

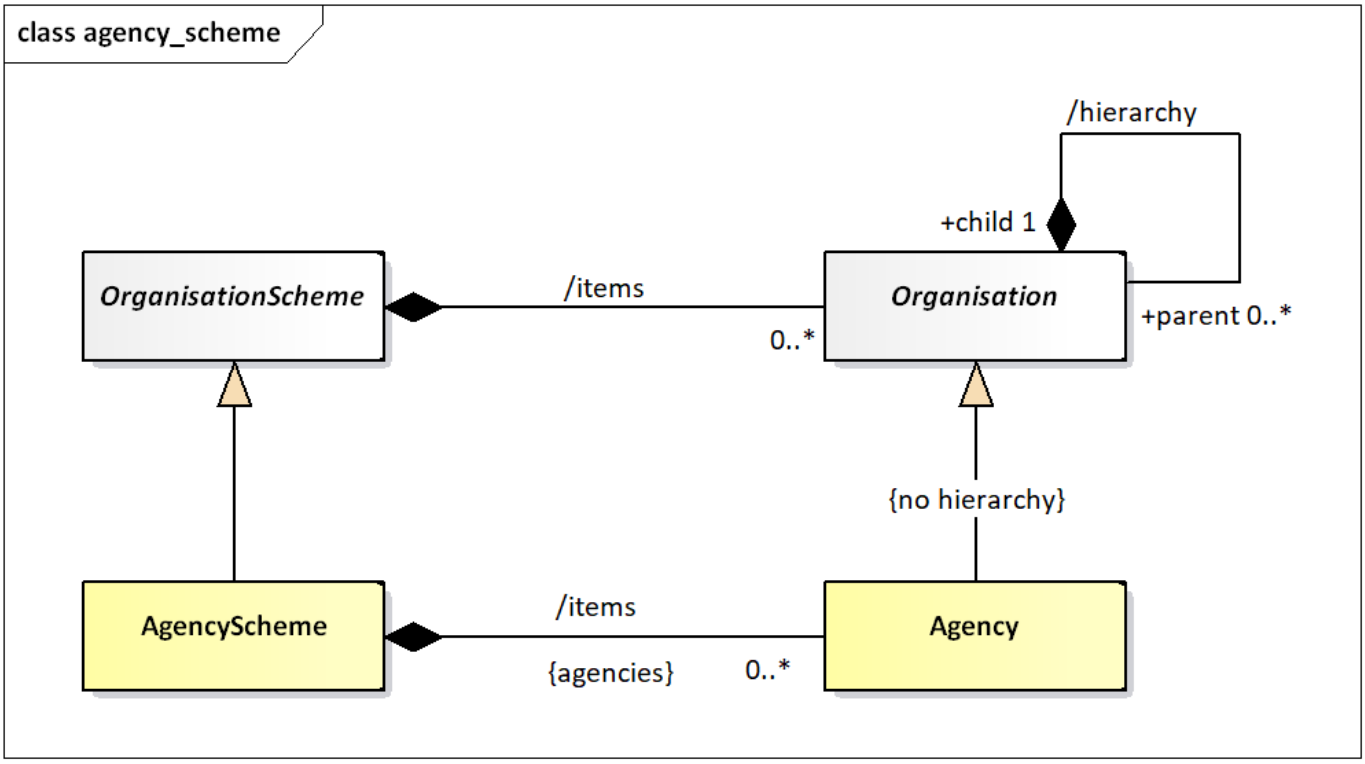


Figure 6. Agency Scheme Model

The Agency in SDMX is extremely important. The Agency Id system used in SDMX is an n-level structure. The top level of this structure is maintained by SDMX. Any Agency in this top level can declare sub agencies and any sub agency can also declare sub agencies. The Agency Scheme has a fixed id and version (version ‘1.0’) and is never declared explicitly in the SDMX object identification mechanism.

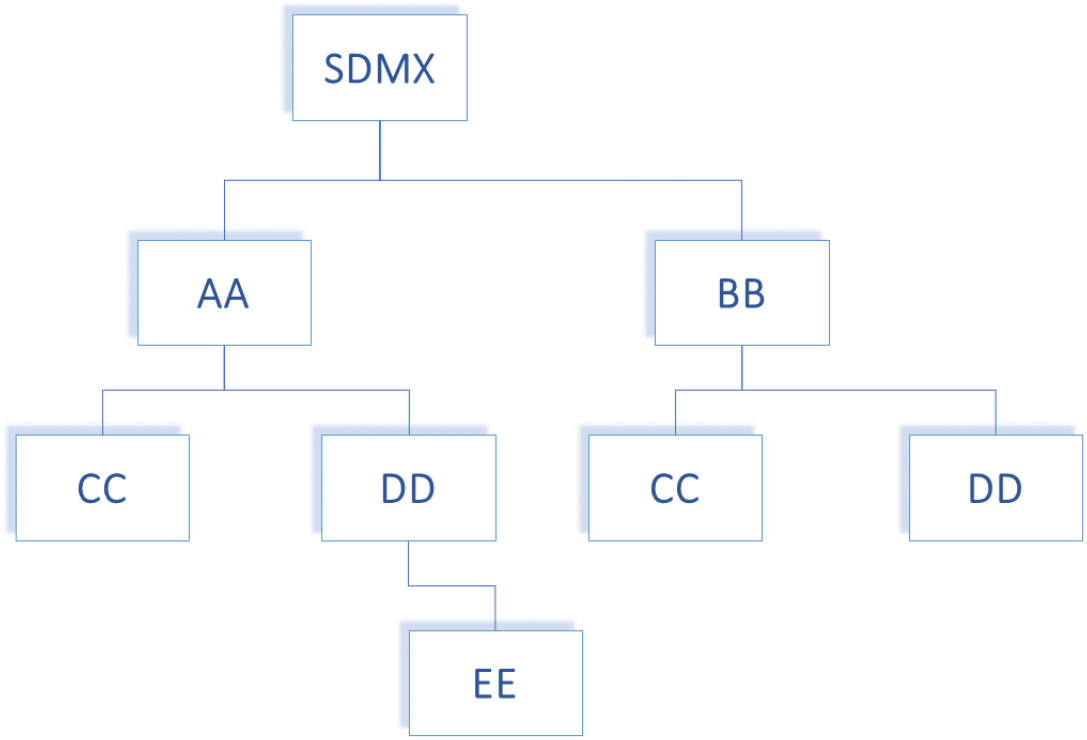
Copyright © SDMx TWG 2020

In order to achieve this SDMX adopts the following rules:

- Agencies are maintained in an Agency Scheme (which is a sub class of Organisation Scheme).
- The agency of the Agency Scheme must also be declared in a (different) Agency Scheme.
- The “top-level” agency is SDMX and maintains the “top-level” Agency Scheme.
- Agencies registered in the top-level scheme can themselves maintain a single Agency Scheme. Agencies in these second-tier schemes can themselves maintain a single Agency Scheme and so on.
- The `AgencyScheme` has a fixed version, i.e., ‘1.0’, hence it is an exception from the Semantic Versioning that other Artefacts follow.
- There can be only one `AgencyScheme` maintained by any one Agency. It has a fixed id of AGENCIES.
- The /hierarchy of Organisation is not inherited by Maintenance Agency – thus each Agency Scheme is a flat list of Maintenance Agencies.
- The format of the agency identifier is `agencyID.agencyID` etc. The top-level agency in this identification mechanism is the agency registered in the SDMX agency scheme. In other words, SDMX is not a part of the hierarchical ID structure for agencies. However, SDMX is, itself, a maintenance agency and is contained in the top-level Agency Scheme.

This supports a hierarchical structure of `agencyID`.

An example is shown below.



The following organizations maintain an Agency Scheme.

- `SDMX` – contains Agencies `AA`, `BB`
- `AA` – contains Agencies `CC`, `DD`
- `BB` – contains Agencies `CC`, `DD`
- `DD` – Contains Agency `EE`

Each agency is identified by its full hierarchy excluding SDMX.

e.g., the id of `EE` as an `agencyID` is `AA.DD.EE`

Copyright © SDMx TWG 2020

An example of this is shown in the XML snippet below:

```
<str:Codelists>
    <str:Codelist id="CL_FREQ" `agencyID`="SDMX" version="1.0.0">
        <com:Name xml:lang="en">Standard frequency Codelist</com:Name>
    </str:Codelist>
    <str:Codelist id="CL_FREQ" `agencyID`="AA" version="1.0.0">
        <com:Name xml:lang="en">Codelist maintained by agency AA</com:Name>
    </str:Codelist>
    <str:Codelist id="CL_FREQ" `agencyID`="AA.CC" version="1.0.0">
        <com:Name xml:lang="en">Codelist maintained by the AA unit CC</com:Name>
    </str:Codelist>
    <str:Codelist id="CL_FREQ" `agencyID`="BB.CC" version="1.0.0">
        <com:Name xml:lang="en">Codelist maintained by the BB unit CC</com:Name>
    </str:Codelist>
```

Figure 7. Example Showing Use of Agency Identifiers

Each of these maintenance agencies has an identical Code list with the Id `CL_BOP`. However, each is uniquely identified by means of the hierarchic agency structure.

Following the same principles, the Metadata Provider is the maintenance organisation for a special subset of Maintainable Artefacts, i.e., the Metadatasets; the latter are the containers of reference metadata combined with a target that those metadata refer to.

UNIVERSAL RESOURCE NAME (URN)

Introduction

To provide interoperability between SDMX Registry/Repositories in a distributed network environment, it is important to have a scheme for uniquely identifying (and thus accessing) all first-class (Identifiable) SDMX-IM objects. Most of these unique identifiers are composite (containing maintenance agency, or parent object identifiers), and there is a need to be able to construct a unique reference as a single string. This is achieved by having a globally unique identifier called a universal resource name (URN) which is generated from the actual identification components in the SDMX-RR APIs. In other words, the URN for any Identifiable Artefact is constructed from its component identifiers (agency, id, version etc.).

URN Structure

Case Rules for URN

For the URN, all parts of the string are case sensitive. The generic structure of the URN is as follows:

```
SDMXprefix.SDMX-IM-package-name.class-name=agencyid:maintainedobject-id(maintainedobject-version).*containerobject-id.object-id
```

* this can repeat and may not be present (see explanation below)

Note that in the SDMX Information Model there are no concrete Versionable Artefacts that are not a Maintainable Artefact. For this reason, the only version information that is allowed is for the maintainable object.

The Maintenance agency identifier is separated from the maintainable artefact identifier by a colon ‘:’. All other identifiers in the SDMX URN syntax are separated by a period ‘.’. The version information is encapsulated in parentheses ‘()’ and adheres to the SDMX versioning rules, as explained in SDMX Standards Section 6 “Technical Notes”, paragraph “4.3 Versioning.

Explanation of the generic structure

In the explanation below the actual object that is the target of the URN is called the **actual object**.

SDMXPrefix: `urn:sdmx:org`

SDMX-IM-package-name: `sdmx.infomodel.package=`

Copyright © SDMx TWG 2020

The packages are:

- `base`
- `codelist`
- `conceptscheme`
- `datastructure`
- `categoryscheme`
- `registry`
- `metadatastructure`
- `process`
- `structuremapping`
- `transformation`

maintainable-object-id is the identifier of the maintainable object. This will always be present as all identifiable objects are either a maintainable object or contained in a maintainable object.

maintainable-object-version is the version, according to the SDMX versioning rules, of the maintainable object and is enclosed in parentheses '()', which are always present.

container-object-id is the identifier of an intermediary object that contains the actual object which the URN is identifying. It is not mandatory as many actual objects do not have an intermediary container object. For instance, a Code is in a maintained object (`Codelist`) and has no intermediary container object, whereas a `MetadataAttribute` has an intermediary container object (`MetadataAttributeDescriptor`) and may have an intermediary container object, which is its parent `MetadataAttribute` . For this reason, the container object id may repeat, with each repetition identifying the object at the next-lower level in its hierarchy. Note that if there is only a single containing object in the model then it is NOT included in the URN structure. This applies to `AttributeDescriptor` , `DimensionDescriptor` , and `MeasureDescriptor` where there can be only one such object and this object has a fixed id. Therefore, whilst each of these has a URN, the id of the `AttributeDescriptor` , `DimensionDescriptor` , and `MeasureDescriptor` is not included when the actual object is a `DataAttribute` or a Dimension/ `TimeDimension` , or a Measure.

Note that although a `Code` can have a parent `Code` and a `Concept` can have a parent `Concept` these are maintained in a flat structure and therefore do not have a `container-object-id` .

For example, the sequence is `agency:DSDid(version).DimensionId` and not `agency:DSDid(version).DimensionDescriptorId.DimensionId` .

`object-id` is the identifier of the actual object unless the actual object is a `Maintainable` object. If present it is always the last id and is not followed by any other character.

Copyright © SDMx TWG 2020

Generic Examples of the URN Structure

- Actual object is a maintainable

```
SDMXPrefix.SDMX-IM-package-name.classname=agencyid:maintained-object-id(version)
```

- Actual object is contained in a maintained object with no intermediate containing object

```
SDMXPrefix.SDMX-IM-package-name.classname=agencyid:maintained-object-id(version).object-id
```

- Actual object is contained in a maintained object with an intermediate containing object

```
SDMXPrefix.SDMX-IM-package-name.classname=agencyid:maintained-object-id(version).contained-object-id.object-id
```

- Actual object is contained in a maintained object with no intermediate containing object but the object type itself is hierarchical

  In this case the object id may not be unique in itself but only within the context of the hierarchy. In the general syntax of the URN all intermediary objects in the structure (with the exception, of course, of the maintained object) are shown as a contained object. An example here would be a Category in a `CategoryScheme`. The Category is hierarchical, and all intermediate Categories are shown as a contained object. The example below shows the generic structure for `CategoryScheme` / Category/ Category.

  ```
  SDMXPrefix.SDMX-IM-package-name.classname=agencyid:maintained-object-id(version).contained-object-id.object-id
  ```

  Actual object is contained in a maintained object with an intermediate containing object and the object type itself is hierarchical

  In this case the generic syntax is the same as for the example above as the parent object is regarded as a containing object, even if it is of the same type. An example here is a `MetadataAttribute` where the contained objects are `MetadataAttributeDescriptor` (first contained object id) and `MetadataAttribute` (subsequent contained object ids). The example below shows the generic structure for MSD/ `MetadataAttributeDescriptor` / `MetadataAttribute` / `MetadataAttribute`

  ```
  SDMXPrefix.SDMX-IM-package-name.classname=agencyid:maintained-object-id(version).contained-object-id.contained-object-id
  contained-object-id.object-id
  ```

Concrete Examples of the URN Structure

The Data Structure Definition `CRED_EXT_DEBT` of legacy version 2.1 maintained by the top-level Agency `TFFS` would have the URN:

```
urn:sdmx:org.sdmx.infomodel.datastructure.DataStucture=TFFS:CRED_EXT_DEBT(2.1)
```

The URN for a code for Argentina maintained by ISO in the code list `CL_3166A2` of semantic version 1.0.0 would be:

```
urn:sdmx:org.sdmx.infomodel.codelist.Code=ISO:CL_3166A2(1.0.0).AR
```

The URN for a category (id of 1) which has parent category (id of 2) maintained by `SDMX` in the category scheme `SUBJECT_MATTER_DOMAINS` of the semantic extended version 1.0.0-draft would be:

```
urn:sdmx:org.sdmx.infomodel.categoryscheme.Category=SDMX:SUBJECT_MATTER_DOMAINS(1.0.0-draft).1.2
```

The URN for a Metadata Attribute maintained by SDMX in the MSD `CONTACT_METADATA` of semantic version 1.0.0 where the hierarchy of the Metadata Attribute is `CONTACT_DETAILS` / `CONTACT_NAME` would be:

```
urn:sdmx:org.sdmx.infomodel.metadatastructure.MetadataAttribute=SDMX:CONTACT_METADATA(1.0.0).CONTACT_DETAILS.CONTACT_NAME
```

The `TFFS` defines `ABC` as a sub-Agency of `TFFS` then the URN of a Dataflow maintained by `ABC` and identified as `EXTERNAL_DEBT` of semantic version 1.0.0 would be:

```
urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=TFFS.ABC:EXTERNAL_DEBT(1.0.0)
```


Copyright © SDMx TWG 2020

The SDMX-RR MUST support this globally unique identification scheme. The SDMX-RR MUST be able to create the URN from the individual identification attributes submitted and to transform the URN to these identification attributes. The identification attributes are:

- **Identifiable and Nameable Artefacts**: `id` (in some cases this id may be hierarchic)
- **Maintainable Artefacts**: `id`, `version`, `agencyId`

The SDMX-RR MUST be able to resolve the unique identifier of an SDMX artefact and to produce an SDMX-ML rendering of that artefact if it is located in the Registry.

Copyright © SDMx TWG 2020

TABLE OF SDMX-IM PACKAGES AND CLASSES

The table below lists all of the packages in the SDMX-IM together with the concrete classes that are in these packages and whose objects have a URN.

Copyright © SDMx TWG 2020

Package	URN class name (model class name where this is different)
base	Agency
	AgencyScheme
	DataConsumer
	DataConsumerScheme
	DataProvider
	DataProviderScheme
	MetadataProvider
	MetadataProviderScheme
	OrganisationUnit
	OrganisationUnitScheme
datastructure	AttributeDescriptor
	DataAttribute
	Dataflow
	DataStructure (DataStructureDefinition)
	Dimension
	DimensionDescriptor
	GroupDimensionDescriptor
	Measure
	MeasureDescriptor
	TimeDimension
metadatastructure	MetadataAttribute
	MetadataAttributeDescriptor
	MetadataStructure (MetadataStructureDefinition)
	Metadataflow
	MetadataSet
process	Process
	ProcessStep
	Transition
registry	DataConstraint
	MetadataConstraint
	MetadataProvisionAgreement
	ProvisionAgreement
	Subscription

Copyright © SDMx TWG 2020

Package	URN class name (model class name where this is different)
structuremapping	CategorySchemeMap
	ConceptSchemeMap
	OrganisationSchemeMap
	ReportingTaxonomyMap
	RepresentationMap
	StructureMap
codelist	Code
	Codelist
	HierarchicalCode
	Hierarchy
	HierarchyAssociation
	Level
	ValueList
categoryscheme	Categorisation
	Category
	CategoryScheme
	ReportingCategory
	ReportingTaxonomy
conceptscheme	Concept
	ConceptScheme
transformation	CustomType
	CustomTypeScheme
	NamePersonalisation
	NamePersonalisationScheme
	Ruleset
	RulesetScheme
	Transformation
	TransformationScheme
	UserDefinedOperator
	UserDefinedOperatorScheme
	VtlCodelistMapping
	VtlConceptMapping

Copyright © SDMx TWG 2020

Package	URN class name (model class name where this is different)
	`VtlDataflowMapping`
	`VtlMappingScheme`

Table 2. SDMX-IM Packages and Contained Classes

URN IDENTIFICATION COMPONENTS OF SDMX OBJECTS

The table below describes the identification components for all SDMX object types that have identification. Note the actual attributes are all ‘id’ but have been prefixed by their class name or multiple class names to show navigation, e.g., `conceptSchemeAgencyId` is really the ‘Id’ attribute of the Agency class that is associated to the `ConceptScheme`.

Note that for brevity the URN examples omit the prefix. All URNs have the prefix:

Copyright © SDMx TWG 2020

```
urn:sdmx.org.sdmx.infomodel.{package}.{classname}=
```


Copyright © SDMx TWG 2020

Classname	Ending URN pattern
`Agency` [1]	`agencySchemeAgencyId:AGENCIES(1.0).agencyId`
`AgencyScheme`	`agencySchemeAgencyId:AGENCIES(1.0)`
`Categorisation`	`categorisationAgencyId:categorisationId(version)`
`Category`	`categorySchemeAgencyId:categorySchemeId(version).categoryId.categoryId.category`
`CategoryScheme`	`categorySchemeAgencyId:categorySchemeId(version)`
`CategorySchemeMap`	`catSchemeMapAgencyId:catSchemeMapId(version)`
`Code`	`codeListAgencyId:codelistId(version).codeId`
`Codelist`	`codeListAgencyId:codeListId(version)`
`ComponentMap`	`structureMapAgencyId:structureMap(version).componentMapId`
`Concept`	`conceptSchemeAgencyId:conceptSchemeId(version).conceptId`
`ConceptScheme`	`conceptSchemeAgencyId:conceptSchemeId(version)`
`ConceptSchemeMap`	`conceptSchemeMapAgencyId:conceptSchemeMapId(version)`
`CustomType`	`customTypeSchemeAgencyId:customTypeSchemeId(version).customTypeId`
`CustomTypeScheme`	`customTypeSchemeAgencyId:customTypeSchemeId(version)`
`DataAttribute`	`dataStructureDefinitionAgencyId:dataStructureDefinitionId(version).dataAttribut`
`DataConstraint`	`dataConstraintAgencyId:dataConstraintId(version)`
`DataConsumer`	`dataConsumerSchemeAgencyId:DATA_CONSUMERS(1.0).dataConsumerId`
`DataConsumerScheme`	`dataConsumerSchemeAgencyId:DATA_CONSUMERS(1.0)`
`Dataflow`	`dataflowAgencyId:dataflowId(version)`
`DataProvider`	`dataProviderSchemeAgencyId:DATA_PROVIDERS(1.0).dataProviderId`
`DataProviderScheme`	`dataProviderSchemeAgencyId:DATA_PROVIDERS(1.0)`
`DataStructure`	`dataStructureDefinitionAgencyId:dataStructureDefinitionId(version)`
`Dimension`	`dataStructureDefinitionAgencyId:dataStructureDefinitionId(version).dimensionId`
`DimensionDescriptor`, `MeasureDescriptor`, `AttributeDescriptor`	`dataStructureDefinitionAgencyId:dataStructureDefinitionId(version).componentLis` where `componentListId` is the name of the class (only one occurrence of each in t Structure Definition)
`GroupDimensionDescriptor`	`dataStructureDefinitionAgencyId:dataStructureDefinitionId(version).groupDimensi`
`HierarchicalCode`	`hierarchyAgencyId:hierarchyId(version).hierarchicalCode.hierarchicalCode`
`Hierarchy`	`hierarchyAgencyId:hierarchyId(version)`
`HierarchyAssociation`	`hierarchyAssociationAgencyId:hierarchyAssociationId(version)`
`Level`	`hierarchyAgencyId:hierarchyId(version).level`
`Measure`	`dataStructureDefinitionAgencyId:dataStructureDefinitionId(version).measureId`
`MetadataAttribute`	`msdAgencyId:msdId(version).metadataAttributeId.metadataAttributeId`
`MetadataAttributeDescriptor`	`msdAgencyId:msdId(version).metadataAttributeDescriptorId`
`MetadataConstraint`	`metadataConstraintAgencyId:metadataConstraintId(version)`

Copyright © SDMx TWG 2020

Classname	Ending URN pattern
`Metadataflow`	`metadataflowAgencyId:metadataflowId(version)`
`MetadataProvider`	`metadataProviderSchemeAgencyId:METADATA_PROVIDERS(1.0).metadataProviderId`
`MetadataProviderScheme`	`metadataProviderSchemeAgencyId:METADATA_PROVIDERS(1.0)`
`MetadataProvisionAgreement`	`metadataProvisionAgreementAgencyId:metadataProvisionAgreementId(version)`
`MetadataSet`	`metadataProviderId:metadataSetId(version)`
`MetadataStructure`	`msdAgencyId:msdId(version)`
`NamePersonalisation`	`namePersonalisationSchemeAgencyId:namePersonalisationSchemeId(version).namePers`
`NamePersonalisationScheme`	`namePersonalisationSchemeAgencyId:namePersonalisationSchemeId(version)`
`OrganisationSchemeMap`	`orgSchemeMapAgencyId:orgSchemeMapId(version)`
`OrganisationUnit`	`organisationUnitSchemeAgencyId:organisationUnitSchemeId(version).organisationUn`
`OrganisationUnitScheme`	`organisationUnitSchemeAgencyId:organisationUnitSchemeId(version)`
`Process`	`processAgencyId:processId(version)`
`ProcessStep`	`processAgencyId:processId(version).processStepId.processStepId`
`ProvisionAgreement`	`provisionAgreementAgencyId:provisionAgreementId(version)`
`ReportingCategory`	`reportingTaxonomyAgencyId:reportingTaxonomyId(version).reportingCategoryId.repo`
`ReportingTaxonomy`	`reportingTaxonomyAgencyId:reportingTaxonomyId(version)`
`ReportingTaxonomyMap`	`repTaxonomyAgencyId:repTaxonomyId(version)`
`RepresentationMap`	`repMapAgencyId:repMapId(version)`
`Ruleset`	`rulesetSchemeAgencyId:rulesetSchemeId(version).rulesetId`
`RulesetScheme`	`rulesetSchemeAgencyId:rulesetSchemeId(version)`
`StructureMap`	`structureMapAgencyId:structureMap(version)`
`Subscription`	The Subscription is not itself an Identifiable Artefact and therefore it does not fol for URN structure. The name of the URN is `registryURN`. There is no pre-determined format.
`TimeDimension`	`dataStructureDefinitionAgencyId:dataStructureDefinitionId(version).timeDimensio`
`Transformation`	`transformationSchemeAgencyId:transformationSchemeId(version).transformationId`
`TransformationScheme`	`transformationSchemeAgencyId:transformationSchemeId(version)`
`Transition`	`processAgencyId:processId(version).processStepId.transitionId`
`UserDefinedOperator`	`userDefinedOperatorSchemeAgencyId:userDefinedOperatorSchemeId(version).userDefi`
`UserDefinedOperatorScheme`	`userDefinedOperatorSchemeAgencyId:userDefinedOperatorSchemeId(version)`
`ValueList`	`valuelistAgencyId:valuelistId(version)`

Copyright © SDMx TWG 2020

Classname	Ending URN pattern
`VtlCodelistMapping`	`vtlMappingSchemeAgencyId:vtlMappingSchemeId(version).vtlCodelistMappingId`
`VtlConceptMapping`	`vtlMappingSchemeAgencyId:vtlMappingSchemeId(version).vtlConceptMappingId`
`VtlDataflowMapping`	`vtlMappingSchemeAgencyId:vtlMappingSchemeId(version).vtlDataflowMappingId`
`VtlMappingScheme`	`vtlMappingSchemeAgencyId:vtlMappingSchemeId(version)`

Table 3. Table of identification components for SDMX Identifiable Artefacts

---

1. The identification of an `Agency` in the URN structure for the maintainable object is by means of the `agencyId`. The `AgencyScheme` is not identified as SDMX has a mechanism for identifying an Agency uniquely by its Id. Note that this Id may be hierarchical. For example, a sub-agency of IMF is referred like this: `IMF.SubAgency1` ↩

Copyright © SDMx TWG 2020

4.2.2 Implementation Notes

Structural Definition Metadata

INTRODUCTION

The SDMX Registry must have the ability to support agencies in their role of defining and disseminating structural metadata artefacts. These artefacts include data structure definitions, code lists, concepts etc. and are fully defined in the SDMX-IM. An authenticated agency may submit valid structural metadata definitions which must be stored in the registry. Note that the term “structural metadata” refers as a general term to all structural components (Data Structure Definitions, Metadata Structure Definitions, Code Lists, Concept Schemes, etc.)

At a minimum, structural metadata definitions may be submitted to and queried from the registry via an HTTP/HTTPS POST in the form of one of the SDMX-ML messages for structural metadata and the SDMX RESTful API for structure queries. The message may contain all structural metadata items for the whole registry, structural metadata items for one maintenance agency, or individual structural metadata items.

Structural metadata items

- may only be modified by the maintenance agency which created them;
- may only be deleted by the agency which created them;
- may not be deleted if they are referenced from other constructs in the Registry.

The level of granularity for the maintenance of SDMX Structural Metadata objects in the registry is the Maintainable Artefact. Especially for Item Schemes, though, partial maintenance may be performed, i.e., at the level of the Item, by submitting an Item Scheme with the `isPartial` flag set and a reduced set of Items.

Copyright © SDMx TWG 2020

The following table lists the Maintainable Artefacts.

Copyright © SDMx TWG 2020

Abstract Class	Concrete Class	Content
Item Scheme	Codelist	Code
	Concept Scheme	Concept
	Category Scheme	Category
	Organisation Unit Scheme	Organisation Unit
	Agency Scheme	Agency
	Data Provider Scheme	Data Provider
	Metadata Provider Scheme	Metadata Provider
	Data Consumer Scheme	Data Consumer
	Reporting Taxonomy	Reporting Category
	Transformation Scheme	Transformation
	Custom Type Scheme	Custom Type
	Name Personalisation Scheme	Name Personalisation
	Vtl Mapping Scheme	Vtl Codelist Mapping, Vtl Concept Mapping
	Ruleset Scheme	Ruleset
	User Defined Operator Scheme	User Defined Operator
Enumerated List	Value List	Value Item
Structure	Data Structure Definition	Dimension Descriptor, Group Dimension Descriptor, Dimension, Time Dimension, Attribute Descriptor, Data Attribute, Measure Descriptor, Measure
	Metadata Structure Definition	Metadata Attribute Descriptor, Metadata Attribute
Structure Usage	Dataflow	
	Metadataflow	
None	Process	Process Step
None	Structure Map	Component Map, Epoch Map, Date Pattern Map
None	Representation Map	Representation Mapping
Item Scheme Map	Organisation Scheme Map	Item Map
	Concept Scheme Map	Item Map
	Category Scheme Map	Item Map
	Reporting Taxonomy Map	Item Map
None	Provision Agreement	

Copyright © SDMx TWG 2020

Abstract Class	Concrete Class	Content
None	Metadata Provision Agreement	
None	Hierarchy	Hierarchical Code
None	Hierarchy Association	
None	Categorisation	

Table 4. Table of Maintainable Artefacts for Structural Definition Metadata

ITEM SCHEME, STRUCTURE

The artefacts included in the structural definitions are:

- All types of Item Scheme (Codelist, Concept Scheme, Category Scheme, Organisation Scheme, Agency Scheme, Data Provider Scheme, Metadata Provider Scheme, Data Consumer Scheme, Organisation Unit Scheme, Transformation Scheme, Name Personalisation Scheme, Custom Type Scheme, Vtl Mapping Scheme, Ruleset Scheme, User Defined Operator Scheme)
- All types of Enumerated List (`ValueList`)[1]
- All types of Structure (Data Structure Definition, Metadata Structure Definition)
- All types of Structure Usage (Dataflow, Metadataflow)

STRUCTURE USAGE

Structure Usage: Basic Concepts

The Structure Usage defines, in its concrete classes of Dataflow and Metadataflow, which flows of data and metadata use which specific Structure, and importantly for the support of data and metadata discovery, the Structure Usage can be linked to one or more Category in one or more Category Scheme using the Categorisation mechanism. This gives the ability for an application to discover data and metadata by “drilling down” the Category Schemes.

Copyright © SDMx TWG 2020

Structure Usage Schematic

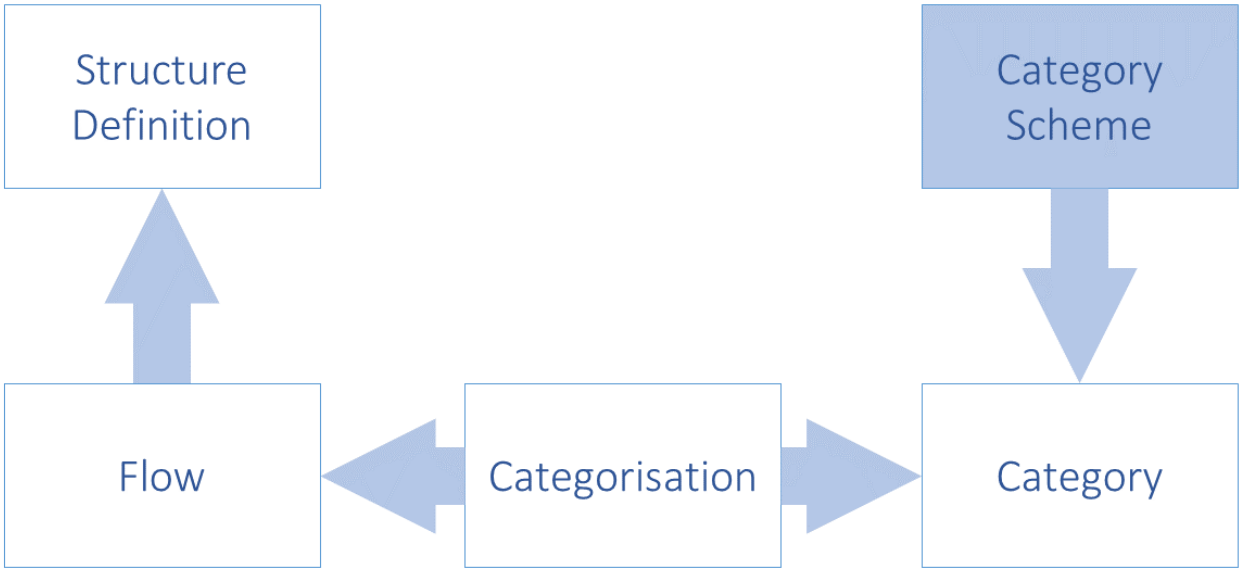


Figure 9. Schematic of Linking the Data and Metadata Flows to Categories and Structure Definitions

Copyright © SDMx TWG 2020

Structure Usage Model

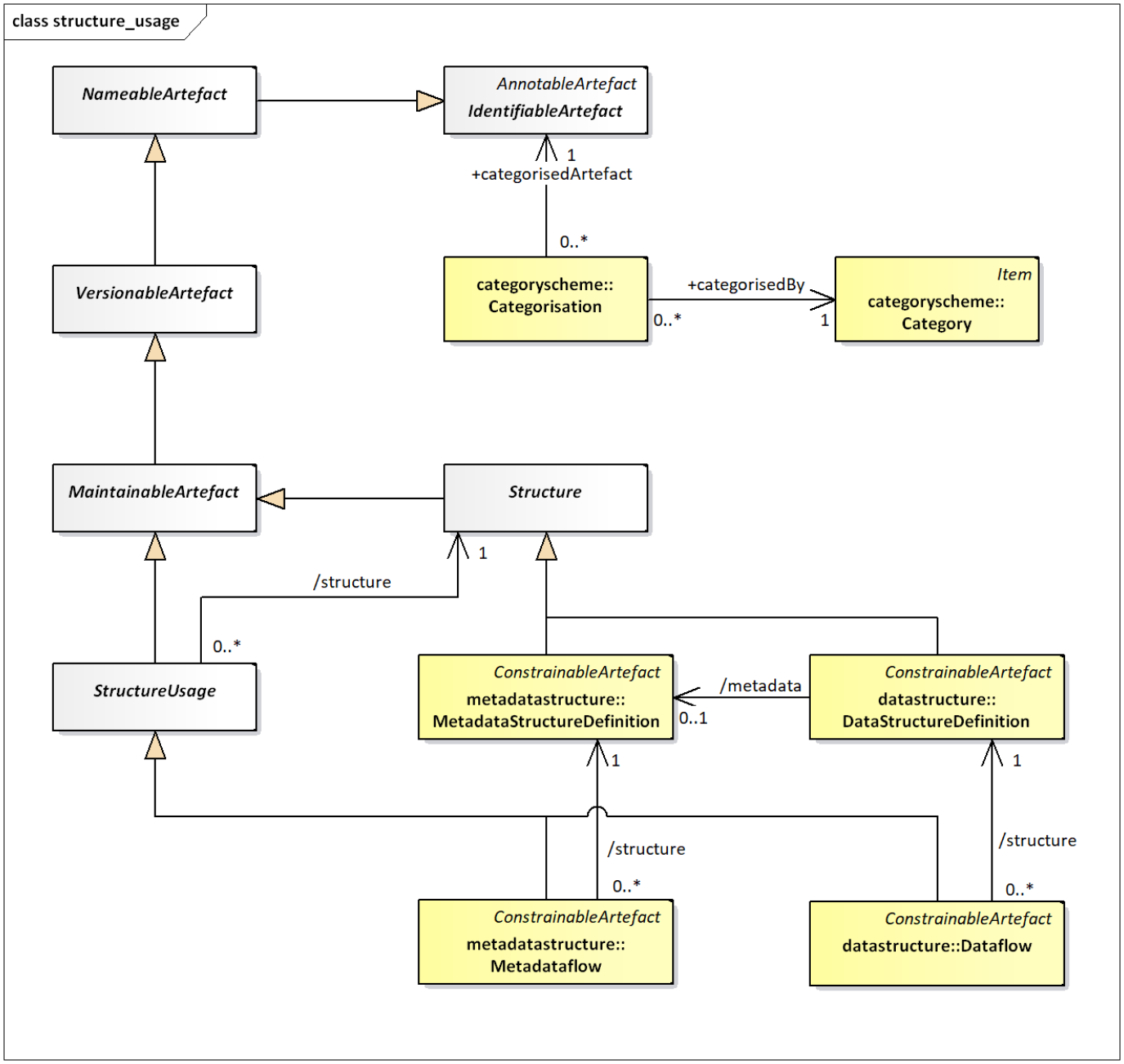


Figure 10. SDMX-IM of links from Structure Usage to Category

In addition to the maintenance of the Dataflow and the Metadataflow, the following links must be maintained in the registry:

- Dataflow to Data Structure Definition
- Metadataflow to Metadata Structure Definition

The following links may be created by means of a Categorisation

- Categorisation to Dataflow and Category
- Categorisation to Metadataflow and Category

Copyright © SDMx TWG 2020

Data and Metadata Provisioning

PROVISIONING AGREEMENT: BASIC CONCEPTS

Data/Metadata provisioning defines a framework in which the provision of different types of statistical data and metadata by various data/metadata providers can be specified and controlled. This framework is the basis on which the existence of data can be made known to the SDMX-enabled community and hence the basis on which data can subsequently be discovered. Such a framework can be used to regulate the data content to facilitate the building of intelligent applications. It can also be used to facilitate the processing implied by service level agreements, or other provisioning agreements in those scenarios that are based on legal directives. Additionally, quality and timeliness metadata can be supported by this framework which makes it practical to implement information supply chain monitoring.

Note that the term “data provisioning” here includes both the provisioning of data and metadata.

Although the Provision Agreement directly supports the data-sharing “pull” model, it is also useful in “push” exchanges (bilateral and gateway scenarios), or in a dissemination environment. It should be noted, too, that in any exchange scenario, the registry functions as a repository of structural metadata.

PROVISIONING AGREEMENT MODEL – PULL USE CASE

An organisation which publishes statistical data or reference metadata and wishes to make it available to an SDMX enabled community is called a Data Provider. In terms of the SDMX Information Model, the Data Provider is maintained in a Data Provider Scheme.

Copyright © SDMx TWG 2020

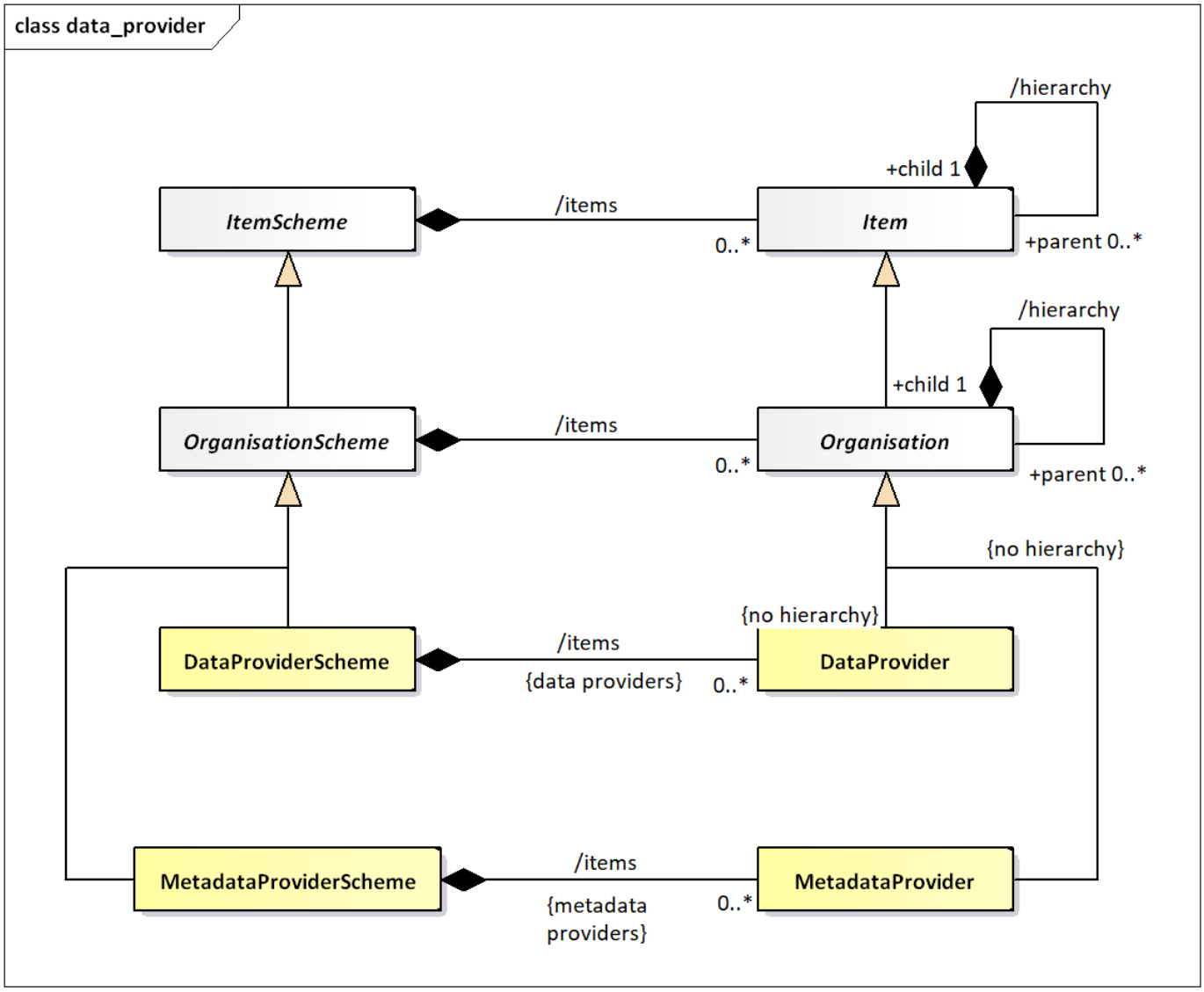


Figure 11. SDMX-IM of the Data Provider

Note that the Data Provider does not inherit the hierarchy association. The diagram below shows a logical schematic of the data model classes required to maintain provision agreements.

Copyright © SDMx TWG 2020

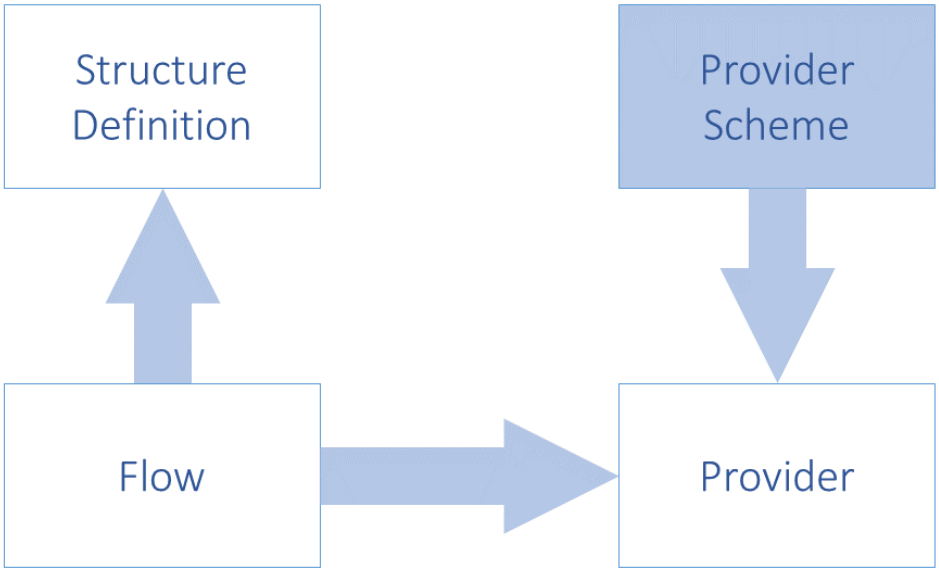


Figure 12. Schematic of the Provision Agreement

The diagram below is a logical representation of the data required in order to maintain Provision Agreements.

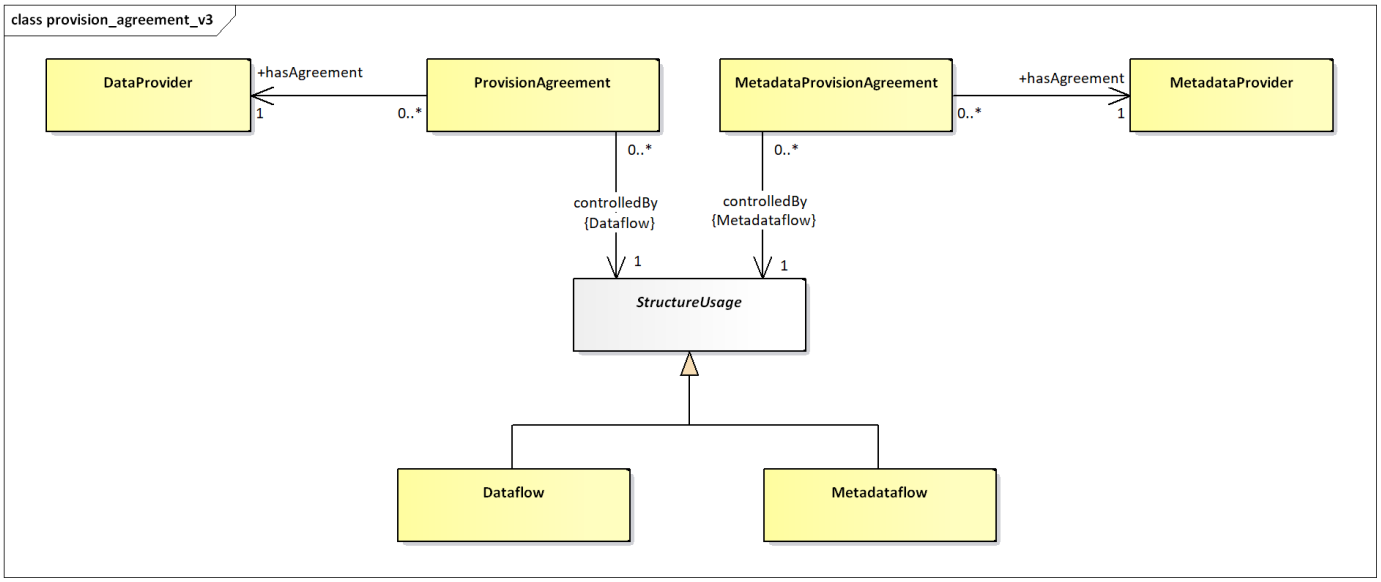


Figure 13. Logical class diagram of the information contained in the Provision Agreement

A Provision Agreement is structural metadata. Each Provision Agreement must reference a Data Provider or Metadata Provider and a Dataflow or Metadataflow Definition. The Data/Metadata Provider and the Dataflow/Metadataflow must exist already in order to set up a Metadata Provision or Provision Agreement.

Copyright © SDMx TWG 2020

Data and Metadata Constraints

DATA AND METADATA CONSTRAINTS: BASIC CONCEPTS

Constraints are, effectively, lists of the valid or actual content of data and metadata. Constraints can be used to specify a subset of the theoretical content of data set or metadata set which can be derived from the specification of the DSD or MSD. A Constraint can comprise a list of keys or a list of content (usually code values) of a specific component such as a dimension or attribute.

Constraints comprise the specification of subsets of key or attribute values that are contained in a data source, or is to be provided for a Dataflow or Metadataflow, or directly attached to a Data Structure Definition or Metadata Structure Definition. This is important metadata because, for example, the full range of possibilities which is implied by the Data Structure Definition (e.g., the complete set of valid keys is the Cartesian product of all the values in the code lists for each of the Dimensions) is often more than is actually present in any specific data source, or more than is intended to be supplied according to a specific Dataflow.

Often a Data Provider will not be able to provide data for all key combinations, either because the combination itself is not meaningful, or simply because the provider does not have the data for that combination. In this case the Data Provider could constrain the data source (at the level of the Provision Agreement or the Data Provider) by supplying metadata that defines the key combinations or cube regions that are available. This is done by means of a Constraint. The Constraint is also used to define a code list subset which is used to populate a partial code list.

Furthermore, it is often useful to define subsets or views of the Data Structure Definition which restrict values in some code lists, especially where many such subsets restrict the same Data Structure Definition. Such a view is called a Dataflow, and there can be one or more defined for any Data Structure Definition.

Whenever data is published or made available by a Data Provider, it must conform to a Dataflow (and hence to a Data Structure Definition). The Dataflow is thus a means of enabling content based processing.

In addition, Constraints can be extremely useful in a data visualisation system, such as dissemination of statistics on a website. In such a system a Cube Region can be used to specify the Dimension codes that actually exist in a data source (these can be used to build relevant selection tables), and the Key Set can be used to specify the keys that exist in a data source (these can be used to guide the user to select only those Dimension code values that will return data based on the Dimension values already selected).

Copyright © SDMx TWG 2020

DATA AND METADATA CONSTRAINTS: SCHEMATIC

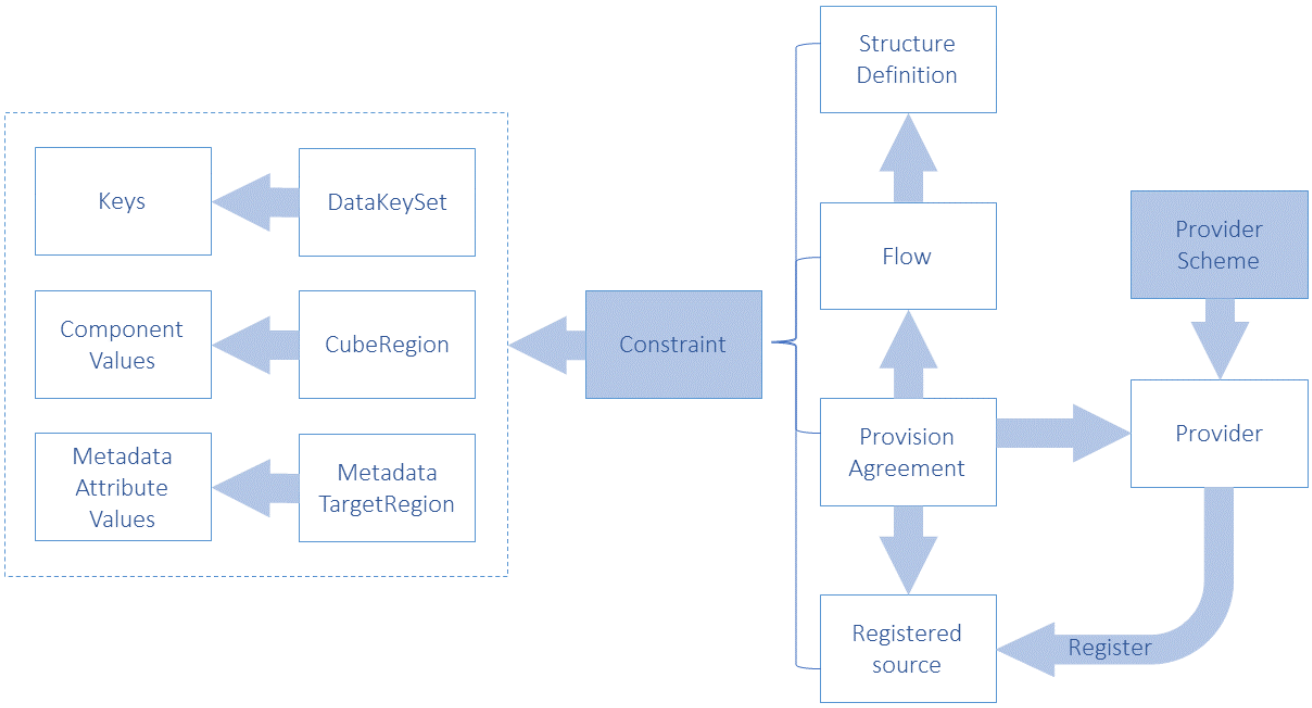


Figure 14. Schematic of the Constraint and the Artefacts that can be constrained

Copyright © SDMx TWG 2020

DATA AND METADATA CONSTRAINTS: MODEL

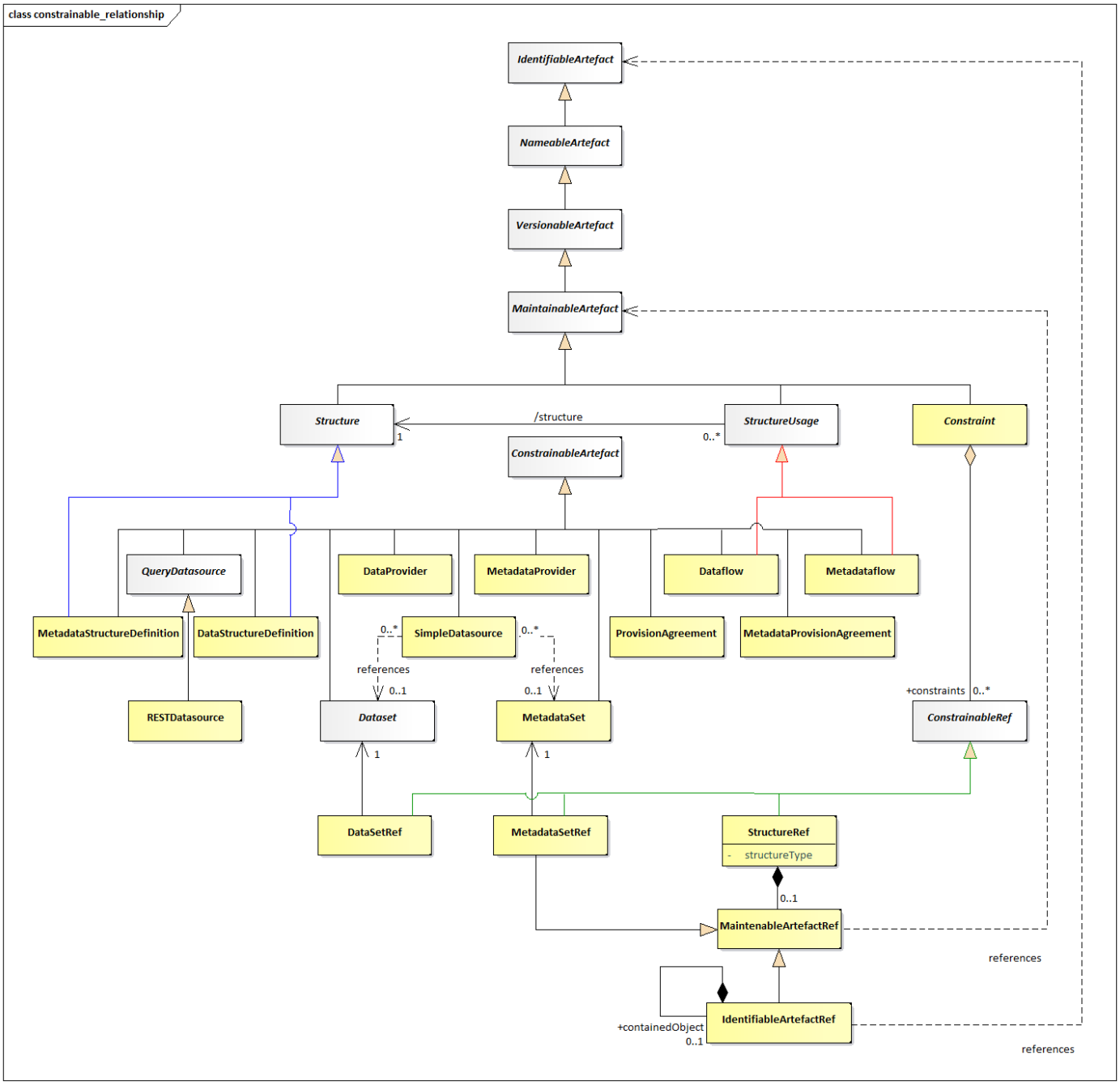


Figure 15. Logical class diagram showing inheritance between and reference to constrainable artefacts

Logical class diagram showing inheritance between and reference to constrainable artefacts

The class diagram above shows that Data Provider, Metadata Provider, Dataflow, Metadataflow, Provision Agreement, Metadata Provision Agreement, Data Structure Definition, Metadata Structure Definition, Simple Datasource and REST Datasource (via the abstract Query Datasource) are all concrete sub-classes of Constrainable Artefact and can therefore have Constraints specified. Note that the actual Constraint as submitted is associated to the reference classes which inherit from `ConstrainableRef`: these are used to refer to the classes to which the Constraint applies.

The content of the Constraint can be found in the SDMX Information Model document.

Copyright © SDMx TWG 2020

Data and Metadata Registration

BASIC CONCEPTS

A Data Provider has published a new dataset conforming to an existing Dataflow (and hence Data Structure Definition). This is implemented as either a web-accessible SDMX-ML file, or in a database which has a web-services interface capable of responding to an SDMX RESTful query with an SDMX-ML data stream.

The Data Provider wishes to make this new data available to one or more data collectors in a “pull” scenario, or to make the data available to data consumers. To do this, the Data Provider registers the new dataset with one or more SDMX conformant registries that have been configured with structural and provisioning metadata. In other words, the registry “knows” the Data Provider and “knows” what data flows the data provider has agreed to make available.

The same mechanism can be used to report or make available a metadata set.

SDMX-RR supports dataset and metadata set registration via the Registration Request, which can be created by the Data/Metadata Provider (giving the Data Provider maximum control). The registry responds to the registration request with a registration response which indicates if the registration was successful. In the event of an error, the error messages are returned as a registry exception within the response.

THE REGISTRATION REQUEST

Registration Request Schematic

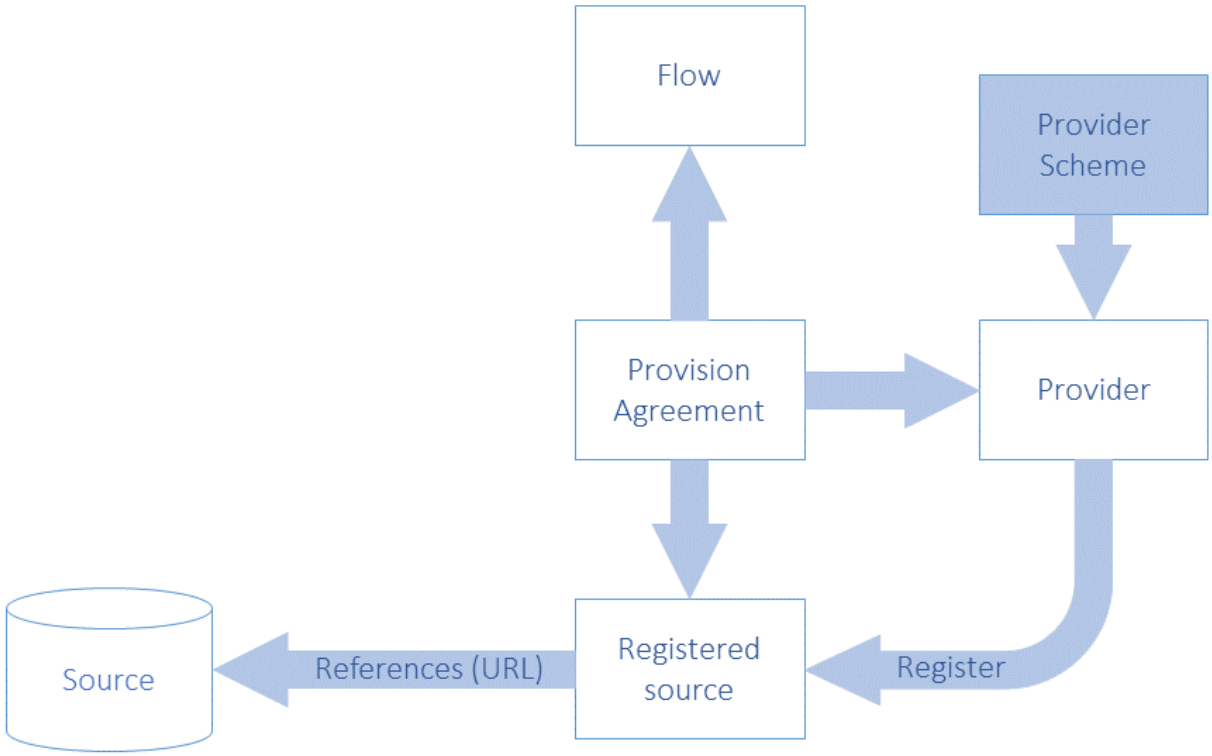


Figure 16. Schematic of the Objects Concerned with registration

Copyright © SDMx TWG 2020

Registration Request Model

The following UML diagram shows the composition of the registration request. Each request is made up of one or more Registrations, one per dataset or metadata set to be registered. The Registration can optionally have information, which has been extracted from the Registration:

- `validFrom`
- `validTo`
- `lastUpdated`

The last updated date is useful during the discovery process to make sure the client knows which data is freshest.

The Registration has an action attribute which takes one of the following values:

Action Attribute Value	Behaviour
Append	Add this Registration to the registry
Replace	Replace the existing Registration with identified by the id in the, Registration of the `SubmitRegistrationRequest`
Delete	Delete the existing Registration identified by the id in the, Registration of the `SubmitRegistrationRequest`

Copyright © SDMx TWG 2020

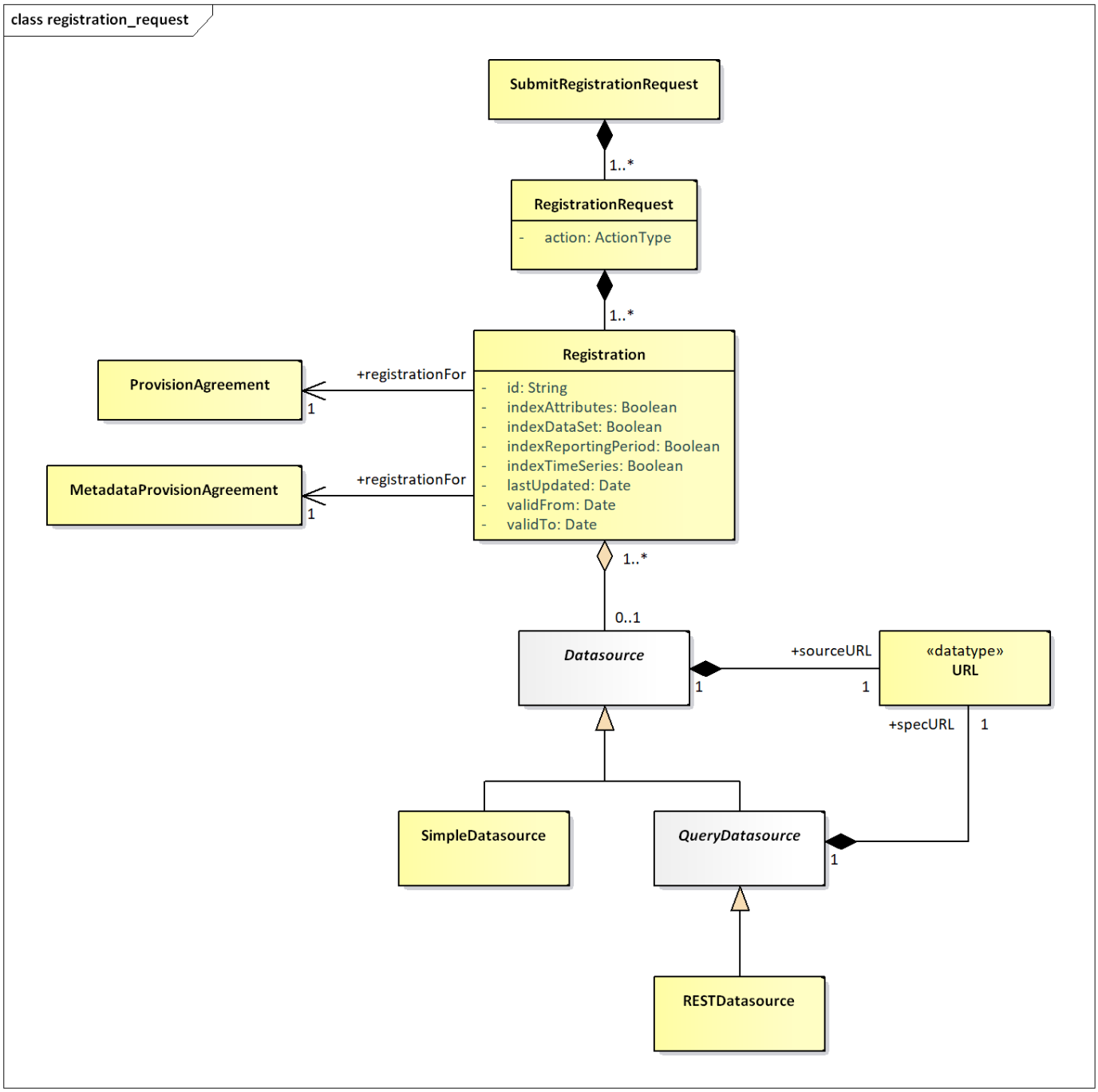


Figure 17. Logical Class Diagram of Registration of Data and Metadata

The `QueryDatasource` is an abstract class that represents a data source, which can understand an API query (i.e., a RESTful query – `RESTDatasource`) and respond appropriately. Each data source inherits the `dataURL` from `Datasource`, and the `QueryDatasource` has an additional URL to locate the specification of the service (`specURL`) to describe how to access it. All other supported protocols are assumed to use the `SimpleDatasource` URL.

A `SimpleDatasource` is used to reference a physical SDMX-ML file that is available at a URL.

The `RegistrationRequest` has an action attribute which defines whether this is a new (append) or updated (replace) `Registration`, or that the `Registration` is to be deleted (delete). The id is only provided for the replace and delete actions, as the Registry will allocate the unique id of the (new) `Registration`.

Copyright © SDMx TWG 2020

The `Registration` includes attributes that state how a `SimpleDatasource` is to be indexed when registered. The Registry registration process must act as follows:

Information in the data or metadata set is extracted and placed in one or more `Constraint`s (see the `Constraint` model in the SDMX Information Model Section). The information to be extracted is indicated by the Boolean values set on the `ProvisionAgreement` or `MetadataProvisionAgreement` as shown in the table below.

Indexing Required	Registration Process Activity
`indexTimeSeries`	Extract all the series keys and create a `KeySet`(s) Constraint.
`indexDataSet`	Extract all the codes and other content of the Key value of the Series Key in a Data Set and create one or more Cube Regions containing Member Selections of Dimension Components of the Constraints model in the SDMX-IM, and the associated Selection Value.
`indexReportingPeriod`	This applies only to a registered dataset. Extract the Reporting Begin and Reporting End from the Header of the Message containing the data set, and create a Reference Period constraint.
`indexAttributes`	**Data Set:** Extract the content of the Attribute Values in a Data Set and create one or more Cube Regions containing Member Selections of Data Attribute Components of the Constraints model in the SDMX-IM, and the associated Selection Value. **Metadata Set:** Indicate the presence of a Reported Attribute by creating one or more Cube Regions containing Member Selections of Metadata Attribute Components of the Constraints model in the SDMX-IM. Note that the content is not stored in the Selection Value.

Constraints that specify the contents of a `QueryDatasource` are submitted to the Registry via the structure submission service (i.e., the RESTful API).

The `Registration` must reference the `ProvisionAgreement` or `MetadataProvisionAgreement` to which it relates.

REGISTRATION RESPONSE

After a registration request has been submitted to the registry, a response is returned to the submitter indicating success or failure. Given that a registration request can hold many Registrations, then there must be a registration status for each `Registration`. The `SubmitRegistration` class has a status field, which is either set to “Success”, “Warning” or “Failure”.

If the registration has succeeded, a `Registration` will be returned – this holds the Registry-allocated Id of the newly registered `Datasource` plus a `Datasource` holding the URL to access the dataset, metadataset, or query service.

The `RegistrationResponse` returns set of registration status (one for each registration submitted) in terms of a `StatusMessage` (this is common to all Registry responses) that indicates success or failure. In the event of registration failure, a set of `MessageText` are returned, giving the error messages that occurred during registration. It is entirely possible when registering a batch of datasets, that the response will contain some successful and some failed statuses. The logical model for the `RegistrationResponse` is shown below:

Copyright © SDMx TWG 2020

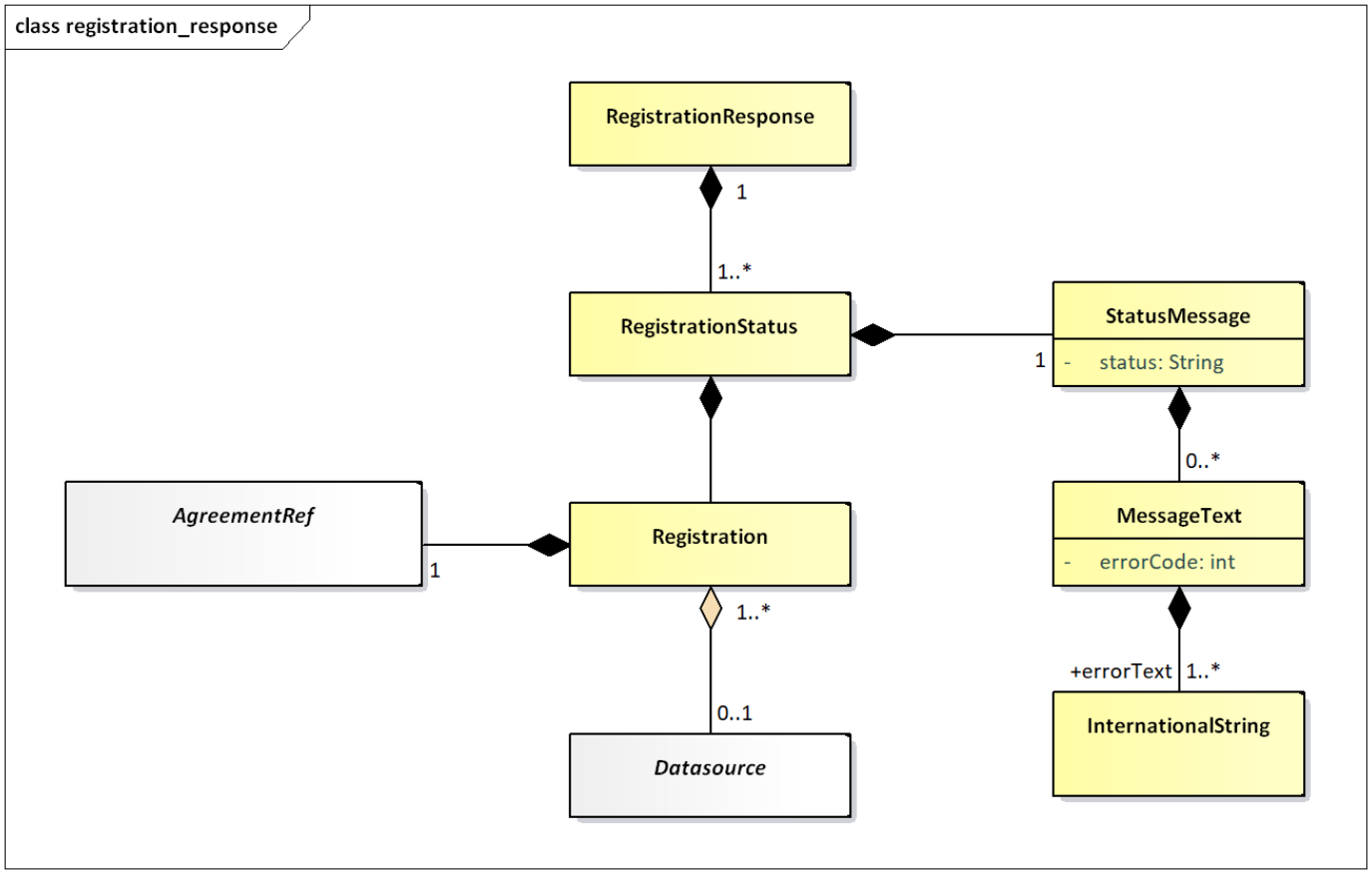


Figure 18. Logical class diagram showing the registration response

Subscription and Notification Service

The contents of the SDMX Registry/Repository will change regularly: new code lists and key families will be published and new datasets and metadata-sets will be registered. To obviate the need for users to repeatedly query the registry to see when new information is available, a mechanism is provided to allow users to be notified when these events happen.

A user can submit a subscription in the registry that defines which events are of interest, and either an email and/or an HTTP address to which a notification of qualifying events will be delivered. The subscription will be identified in the registry by a URN, which is returned to the user when the subscription is created. If the user wants to delete the subscription at a later point, the subscription URN is used as identification. Subscriptions have a validity period expressed as a date range (`startDate` , `endDate`) and the registry may delete any expired subscriptions, and will notify the subscriber on expiry.

When a registry/repository artefact is modified, any subscriptions which are observing the object are activated, and either an email or HTTP POST is instigated to report details of the changes to the user specified in the subscription. This is called a “notification”.

Copyright © SDMx TWG 2020

SUBSCRIPTION LOGICAL CLASS DIAGRAM

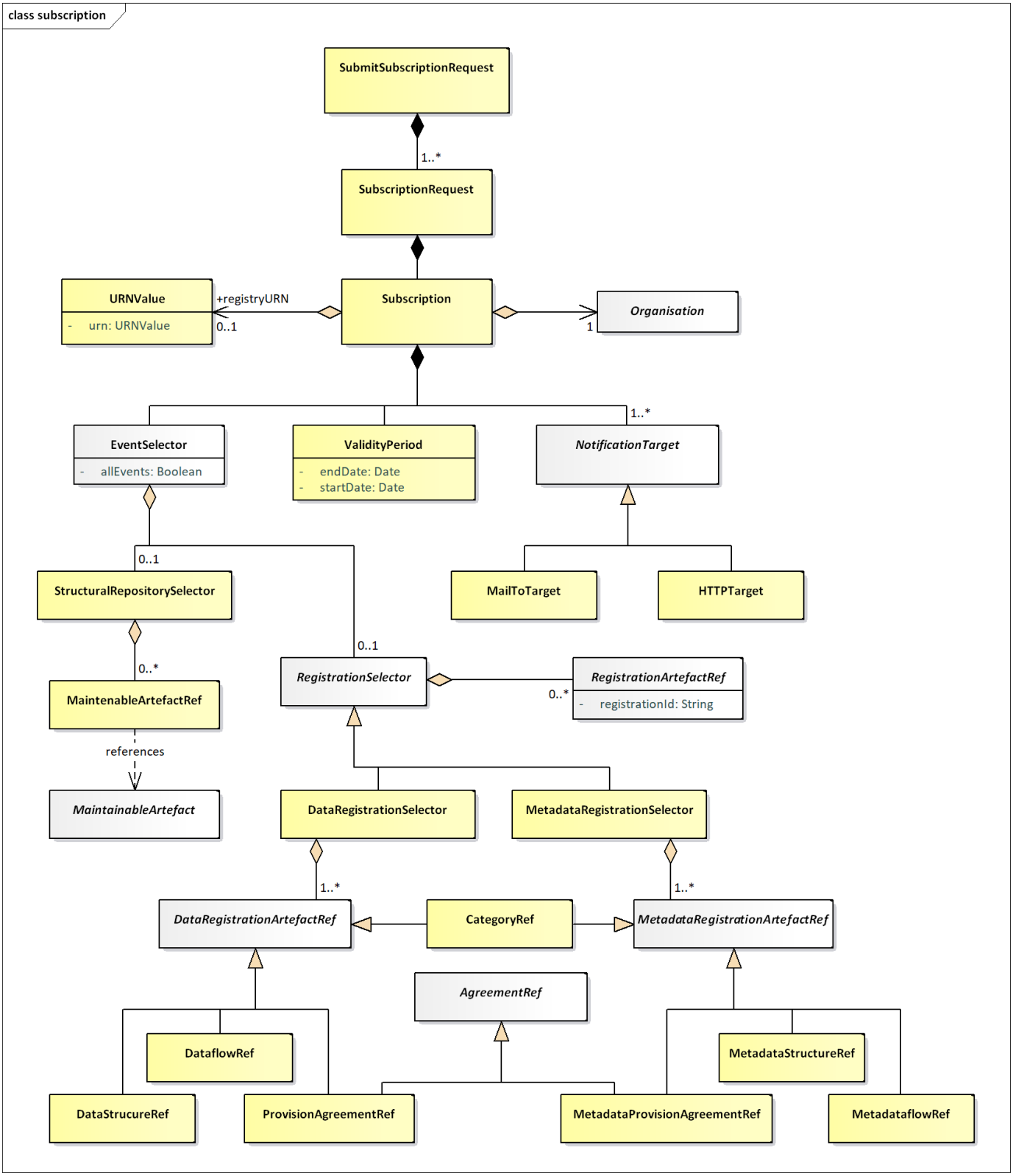


Figure 19. Logical Class Diagram of the Subscription

Copyright © SDMx TWG 2020

SUBSCRIPTION INFORMATION

Regardless of the type of registry/repository events being observed, a subscription always contains:

1. A set of URIs describing the end-points to which notifications must be sent if the subscription is activated. The URIs can be either mailto: or http: protocol. In the former case an email notification is sent; in the latter an HTTP POST notification is sent.
2. A user-defined identifier, which is returned in the response to the subscription request. This helps with asynchronous processing and is NOT stored in the Registry.
3. A validity period which defines both when the subscription becomes active and expires. The subscriber may be sent a notification on expiration of the subscription.
4. A selector which specifies which type of events are of interest. The set of event types is:

Event Type	Comment
`STRUCTURAL_REPOSITORY_EVENTS`	Life-cycle changes to Maintainable Artefacts in the structural, metadata repository.
`DATA_REGISTRATION_EVENTS`	Whenever a published dataset is registered. This can be either a, SDMXML data file or an SDMX conformant database.
`METADATA_REGISTRATION_EVENTS`	Whenever a published metadataset is registered. This can be either a, SDMXML reference metadata file or an SDMX conformant database.
`ALL_EVENTS`	All events of the specified `EventType`

WILDCARD FACILITY

Subscription notification supports wildcarded identifier components URNs, which are identifiers which have some or all of their component parts replaced by the wildcard character `*`. Identifier components comprise:

- `agencyID`
- `id`
- `version`

Examples of wildcarded identifier components for an identified object type of `Codelist` are shown below:

```
AgencyID = *
Id = *
Version = *
```

This subscribes to all `Codelist`s of all versions for all agencies.

```
AgencyID = AGENCY1
Id = CODELIST1
Version = *
```

This subscribes to all versions of `Codelist` `CODELIST1` maintained by the agency `AGENCY1`.

```
AgencyID = AGENCY1
Id = *
Version = *
```

This subscribes to all versions of all `Codelist` objects maintained by the agency `AGENCY1`.

```
AgencyID = *
Id = CODELIST1
Version = *
```

This subscribes to all versions of `Codelist` `CODELIST1` maintained by any agency.

Note that if the subscription is to the latest stable version then this can be achieved by the `+` character, i.e.:

```
Version = +
```

A subscription to the latest version (whether stable, draft or non-versioned) can be achieved by the `~` character, i.e.:

Copyright © SDMx TWG 2020

```
Version = ~
```

A subscription to the latest stable version within major version 2 starting with version 2.3.1 can be achieved by adding the + character after the minor version number, i.e.:

```
Version = 2.3+.1
```

The complete SDMX versioning syntax can be found in the SDMX Standards Section “Technical Notes”, paragraph “Versioning”.

STRUCTURAL REPOSITORY EVENTS

Whenever a maintainable artefact (data structure definition, concept scheme, codelist, metadata structure definition, category scheme, etc.) is added to, deleted from, or modified in the structural metadata repository, a structural metadata event is triggered. Subscriptions may be set up to monitor all such events, or focus on specific artefacts such as a Data Structure Definition.

REGISTRATION EVENTS

Whenever a dataset or metadata-set is registered a registration event is created. A subscription may be observing all data or metadata registrations, or it may focus on specific registrations as shown in the table below:

Selector	Comment
`DataProvider` & `MetadataProvider`	Any datasets or metadata sets registered by the specified data or, metadata provider will activate the notification.
`ProvisionAgreement` & `MetadataProvisionAgreement`	Any datasets or metadata sets registered for the agreement will, activate the notification.
Dataflow & Metadataflow	Any datasets or metadata sets registered for the specified dataflow, (or metadataflow) will activate the notification.
`DataStructureDefinition` & `MetadataStructureDefinition`	Any datasets or metadata sets registered for those dataflows (or, metadataflows) that are based on the specified Data Structure Definition, will activate the notification
Category	Any datasets or metadata sets registered for those dataflows,, metadataflows, provision agreements that are categorised by the, category.

The event will also capture the semantic of the registration: deletion or replacement of an existing registration or a new registration.

Copyright © SDMx TWG 2020

Notification

LOGICAL CLASS DIAGRAM

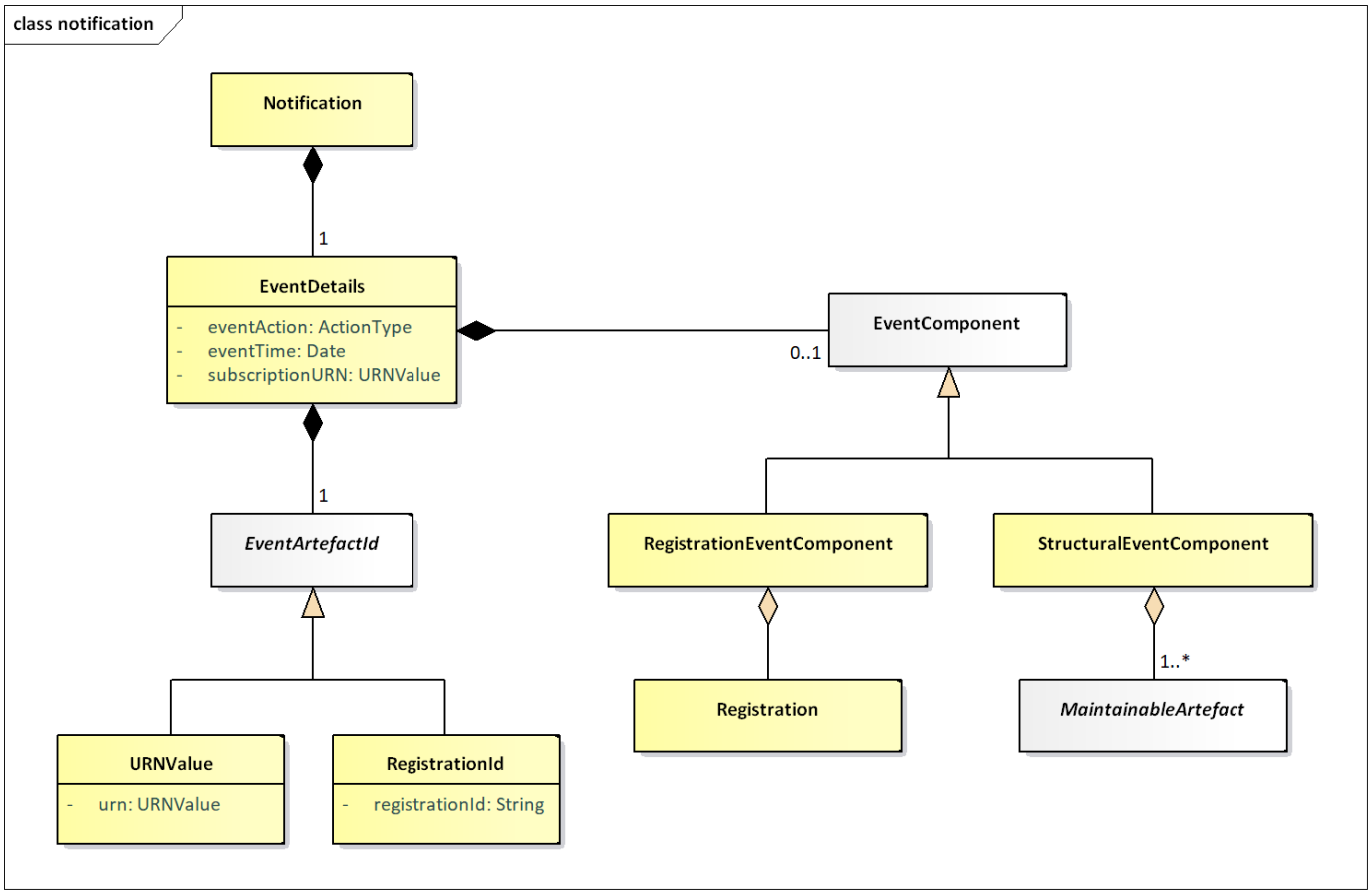


Figure 20. Logical Class Diagram of the Notification

A notification is an XML document that is sent to a user via email or http POST whenever a subscription is activated. It is an asynchronous one-way message.

Regardless of the registry component that caused the event to be triggered, the following common information is in the message:

- Date and time that the event occurred
- The URN of the artefact that caused the event
- The URN of the Subscription that produced the notification
- Event Action: Add, Replace, or Delete.

Additionally, supplementary information may be contained in the notification as detailed below.

STRUCTURAL EVENT COMPONENT

The notification will contain the `MaintainableArtefact` that triggered the event in a form similar to the SDMX-ML structural message (using elements from that namespace).

REGISTRATION EVENT COMPONENT

The notification will contain the Registration.

---

1. Note that Codelist is also an `EnumeratedList`. ↩

Copyright © SDMx TWG 2020

5. Technical Notes

5.1 Purpose and Structure

5.1.1 Purpose

The intention of Section 6 is to document certain aspects of SDMX that are important to understand and will aid implementation decisions. The explanations here supplement the information documented in the SDMX XML/JSON schemas and the Information Model.

5.1.2 Structure

This document is organized into the following major parts:

- A guide to the SDMX Information Model relating to Data Structure Definitions and Data Sets, statement of differences in functionality supported by the different formats and syntaxes for Data Structure Definitions and Data Sets, and best practices for use of SDMX formats, including the representation for time period.
- A guide to the SDMX Information Model relating to Metadata Structure Definitions, and Metadata Sets.
- Other structural artefacts of interest: Agencies, Concept Role, Constraint, Codelist.

Copyright © SDMx TWG 2020

5.2 General Notes on This Document

As of version SDMX 2.1, the term “Key family” has been replaced by Data Structure Definition (also known and referred to as DSD) both in the XML schemas and the Information Model. The term “Key family” is not familiar to many people and its name was taken from the model of SDMX-EDI (previously known as GESMES/TS). The more familiar name “Data Structure Definition” which was used in many documents is now also the technical artefact in the SDMX-ML and Information Model technical specifications. The SDMX-EDI specification, that was using the term “Key family”, is deprecated in this version of the specification.

There has been much work within the SDMX community on the creation of user guides, tutorials, and other aids to implementation and understanding of the standard. This document is not intended to duplicate the function of these documents, but instead represents a short set of technical notes not generally covered elsewhere.

Copyright © SDMx TWG 2020

5.3 Guide for SDMX Format Standards

5.3.1 Introduction

This guide exists to provide information to implementers of the SDMX format standards – SDMX-ML, SDMX-JSON and SDMX-CSV – that are concerned with data, i.e., Data Structure Definitions and Data Sets. This section is intended to provide information that will help users of SDMX understand and implement the standards. It is not normative, and it does not provide any rules for the use of the standards, such as those found in SDMX-ML: Schema and Documentation.

5.3.2 SDMX Information Model for Format Implementers

Introduction

The purpose of this sub-section is to provide an introduction to the SDMX-IM relating to Data Structure Definitions and Data Sets for those whose primary interest is in the use of the XML, JSON or CSV formats. For those wishing to have a deeper understanding of the Information Model, the full SDMX-IM document, and other sections in this guide provide a more in-depth view, along with UML diagrams and supporting explanation. For those who are unfamiliar with DSDs, an appendix to the SDMX-IM provides a tutorial which may serve as a useful introduction.

The SDMX-IM is used to describe the basic data and metadata structures used in all of the SDMX data formats. The Information Model concerns itself with statistical data and its structural metadata, and that is what is described here. Both structural metadata and data have some additional metadata in common, related to their management and administration. These aspects of the data model are not addressed in this section and covered elsewhere in this guide or in the full SDMX-IM document.

Note that in the descriptions below, text in courier and italics are the names used in the information model (e.g., `DataSet`).

5.3.3 SDMX Formats: Expressive Capabilities and Function

SDMX offers several equivalent formats for describing data and structural metadata, optimized for use in different applications. Although all of these formats are derived directly from the SDMX-IM, and are thus equivalent, the syntaxes used to express the model place some restrictions on their use. Also, different optimizations provide different capabilities. This section describes these differences and provides some rules for applications which may need to support more than one SDMX format or syntax. This section is constrained to the Data Structure Definition and the `DataSet`.

Format Optimizations and Differences

The following section provides a brief overview of the differences between the various SDMX formats.

Version 2.0 was characterised by 4 data messages, each with a distinct format: Generic, Compact, Cross-Sectional and Utility. Because of the design, data in some formats could not always be related to another format. In version 2.1, this issue has been addressed by merging some formats and eliminating others. As a result, in SDMX 2.1 there were just two types of data formats: `GenericData` and `StructureSpecificData` (i.e., specific to one Data Structure Definition). As of SDMX 3.0, based also on the real-life usage of 2.1 XML formats but also the new formats introduced (JSON and CSV), only one XML format remains, i.e., `StructureSpecificData`. Furthermore, the time specific sub-formats have also been deprecated due to the lack of usage.

SDMX-JSON and SDMX-CSV feature also only one flavour, each. It should be noted, though, that both XML and JSON messages allow for series oriented as well as flat representations.

STRUCTURE DEFINITION

- The SDMX-ML Structure Message is currently the main way of modelling a DSD. The SDMX-JSON version follows the same principles, while the SDMX-CSV does not support structures, yet.
- The SDMX-ML Structure Message allows for the structures on which a Data Structure Definition depends – that is, codelists and concepts – to be either included in the message or to be referenced by the message containing the data structure definition. XML syntax is designed to leverage URIs and other Internet-based referencing mechanisms, and these are used in the SDMX-ML message. This option is also available in SDMX-JSON. The latter, though, further supports conveying data with some structural metadata within a single message.

Copyright © SDMx TWG 2020

- All structures can be inserted, replaced or deleted, unless structural dependencies are not respected.

VALIDATION

- The SDMX-ML structure specific messages will allow validation of XML syntax and data typing to be performed with a generic XML parser and enforce agreement between the structural definition and the data to a moderate degree with the same tool.
- Similarly, the SDMX-JSON message can be validated using JSON Schema and hence may also be generically parsed and validated.
- The SDMX-CSV format cannot be validated by generic tools.

UPDATE AND DELETE MESSAGES

- All data messages allow for both append/replace/delete messages.
- These messages allow also transmitting only data or only documentation (i.e., Attribute values without Observation values).

CHARACTER ENCODINGS

All formats use the UTF-8 encoding. The SDMX-CSV may use a different encoding if this is reported properly in the mime type of a web service response.

DATA TYPING

The XML syntax and JSON syntax have similar data-typing mechanisms. Hence, there is no need for conventions in order to allow transition from one format to another, like those required for EDIFACT in SDMX 2.1. On the other hand, JSON schema has a simpler set of data types (as explained in Section Information Model, paragraph “Representation Constructs”) but complements its data types with a fixed set of formats or regular expressions. In addition, the JSON schema has also types that are not natively supported in XML schema and need to be implemented as complex types in the latter. The section below provides examples of those cases that are not natively supported by either the XML or JSON data types. More details on the data mapping between XML and JSON schemas are also explained in section “Data Types”.

5.3.4 SDMX Best Practices

Reporting and Dissemination Guidelines

CENTRAL INSTITUTIONS AND THEIR ROLE IN STATISTICAL DATA EXCHANGES

Central institutions are the organisations to which other partner institutions “report” statistics. These statistics are used by central institutions either to compile aggregates and/or they are put together and made available in a uniform manner (e.g., on-line or on a CD-ROM or through file transfers). Therefore, central institutions receive data from other institutions and, usually, they also “disseminate” data to individual and/or institutions for end-use. Within a country, a NSI or a national central bank (NCB) plays, of course, a central institution role as it collects data from other entities and it disseminates statistical information to end users. In SDMX the role of central institution is very important: every statistical message is based on underlying structural definitions (statistical concepts, code lists, DSDs) which have been devised by a particular agency, usually a central institution. Such an institution plays the role of the reference “structural definitions maintenance agency” for the corresponding messages which are exchanged. Of course, two institutions could exchange data using/referring to structural information devised by a third institution.

Central institutions can play a double role:

- collecting and further disseminating statistics;
- devising structural definitions for use in data exchanges.

DEFINING DATA STRUCTURE DEFINITIONS (DSDS)

The following guidelines are suggested for building a DSD. However, it is expected that these guidelines will be considered by central institutions when devising new DSDs.

Dimensions, Attributes and Codelists

- **Avoid dimensions that are not appropriate for all the series in the data structure definition**. If some dimensions are not applicable (this is evident from the need to have a code in a code list which is marked as “not applicable”, “not relevant” or “total”) for some series then consider moving these series to a new data structure definition in which these dimensions are

Copyright © SDMx TWG 2020

dropped from the key structure. This is a judgement call as it is sometimes difficult to achieve this without increasing considerably the number of DSDs.

- **Devise DSDs with a small number of Dimensions for public viewing of data**. A DSD with the number dimensions in excess 6 or 7 is often difficult for non-specialist users to understand. In these cases, it is better to have a larger number of DSDs with smaller “cubes” of data, or to eliminate dimensions and aggregate the data at a higher level. Dissemination of data on the web is a growing use case for the SDMX standards: the differentiation of observations by dimensionality, which are necessary for statisticians and economists, are often obscure to public consumers who may not always understand the semantic of the differentiation.
- **Avoid composite dimensions**. Each dimension should correspond to a single characteristic of the data, not to a combination of characteristics.
- Consider the inclusion of the following attributes. Once the key structure of a data structure definition has been decided, then the set of (preferably mandatory) attributes of this data structure definition has to be defined. In general, some statistical concepts are deemed necessary across all Data Structure Definitions to qualify the contained information. Examples of these are:
    - A descriptive title for the series (this is most useful for dissemination of data for viewing e.g., on the web).
    - Collection (e.g., end of period, averaged or summed over period).
    - Unit (e.g., currency of denomination).
    - Unit multiplier (e.g., expressed in millions).
    - Availability (which institutions can a series become available to).
    - Decimals (i.e., number of decimal digits used in numerical observations).
    - Observation Status (e.g., estimate, provisional, normal).

Moreover, additional attributes may be considered as mandatory when a specific data structure definition is defined.

- **Avoid creating a new code list where one already exists**. It is highly recommended that structural definitions and code lists be consistent with internationally agreed standard methodologies, wherever they exist, e.g., System of National Accounts 1993; Balance of Payments Manual, Fifth Edition; Monetary and Financial Statistics Manual; Government Finance Statistics Manual, etc. When setting-up a new data exchange, the following order of priority is suggested when considering the use of code lists:
    - international standard code lists;
    - international code lists supplemented by other international and/or regional institutions;
    - standardised lists used already by international institutions;
    - new code lists agreed between two international or regional institutions;
    - new code lists which extend existing code lists, by adding only missing codes;
    - new specific code lists.

The same code list can be used for several statistical concepts, within a data structure definition or across DSDs. Note that SDMX has recognised that these classifications are often quite large and the usage of codes in any one DSD is only a small extract of the full code list. In this version of the standard, it is possible to exchange and disseminate a **partial code list** which is extracted from the full code list and which supports the dimension values valid for a particular DSD.

Data Structure Definition Structure

- The following items have to be specified by a structural definitions maintenance agency when defining a new data structure definition:
- Data structure definition (DSD) identification:
    - DSD identifier
    - DSD name
- A list of metadata concepts assigned as dimensions of the data structure definition. For each:
    - (statistical) concept identifier
    - code list identifier (id, version, maintenance agency) if the representation is coded

Copyright © SDMx TWG 2020

- A list of (statistical) concepts assigned as attributes for the data structure definition. For each:
  - (statistical) concept identifier
  - code list identifier if the concept is coded
  - usage: mandatory, optional
  - relationship to dimensions and measures
  - maximum text length for the uncoded concepts
  - maximum code length for the coded concepts
- A list of the code lists used in the data structure definition. For each:
  - code list identifier
  - code list name
  - code values and descriptions
- Definition of Dataflow. Two (or more) partners performing data exchanges in a certain context need to agree on:
  - the list of dataset identifiers they will be using;
  - for each Dataflow:
    - its content (e.g., by Constraints) and description
    - the relevant DSD that defines the structure of the data reported or disseminated according the Dataflow

EXCHANGING ATTRIBUTES

Attributes on series and group levels

Static properties.

- Upon creation of a series the sender has to provide to the receiver values for all mandatory attributes. In case they are available, values for conditional attributes should also be provided. Whereas initially this information may be provided by means other than SDMX-ML/JSON/CSV messages (e.g., paper, telephone) it is expected that partner institutions will be in a position to provide this information in the available formats over time.
- A centre may agree with its data exchange partners special procedures for authorising the setting of attributes' initial values.

Communication of changes to the centre.

- Following the creation of a series, the attribute values do not have to be reported again by senders, as long as they do not change.
- Whenever changes in attribute values for a series (or group) occur, the reporting institutions should report either all attribute values again (this is the recommended option) or only the attribute values which have changed. This applies both to the mandatory and the conditional attributes. For example, if a previously reported value for a conditional attribute is no longer valid, this has to be reported to the centre.
- A centre may agree with its data exchange partners special procedures for authorising modifications in the attribute values.
- Communication of observation level attributes "observation status", "observation confidentiality", "observation pre-break" is recommended.
- Whenever an observation is exchanged, the corresponding observation status is recommended to also be exchanged attached to the observation, regardless of whether it has changed or not since the previous data exchange.
- If the "observation status" changes and the observation remains unchanged, both components would have to be reported (unless the observation is deleted).

  For Data Structure Definitions having also the observation level attributes "observation confidentiality" and "observation pre-break" defined, this rule applies to these attributes as well: if an institution receives from another institution an observation with an observation status attribute only attached, this means that the associated observation confidentiality and pre-break observation attributes either never existed or from now they do not have a value for this observation.

Copyright © SDMx TWG 2020

Best Practices for Batch Data Exchange

INTRODUCTION

Batch data exchange is the exchange and maintenance of entire databases between counterparties. It is an activity that often employs SDMX-CSV format, and might also use the SDMX-ML dataset. The following points apply equally to both formats.

POSITIONING OF THE DIMENSION “FREQUENCY”

In SDMX 3.0, the “frequency” dimension is not special in the data structure definition. Many central institutions devising structural definitions have decided to assign to this dimension the first position in the key structure. Nevertheless, a standard role (i.e., that of ‘Frequency’) may facilitate the easy identification of this dimension. This is necessary to frequency’s crucial role in several database systems and in attaching attributes at the “sibling” group level.

IDENTIFICATION OF DATA STRUCTURE DEFINITIONS (DSDS)

In order to facilitate the easy and immediate recognition of the structural definition maintenance agency that defined a data structure definition, some central institutions devising structural definitions use the first characters of the data structure definition identifiers to identify their institution: e.g., BIS_EER, EUROSTAT_BOP_01, ECB_BOP1, etc. Nevertheless, using the `AgencyId` may disambiguate any Artefact in a more efficient and machine readable way.

IDENTIFICATION OF THE DATAFLOWS

In order to facilitate the easy and immediate recognition of the institution administrating a Dataflow, some central institutions prefer to use the first characters of the Dataflow identifiers to identify their institution: e.g. `BIS_EER`, `ECB_BOP1`, `ECB_BOP1`, etc. Nevertheless, using the `AgencyId` may disambiguate any Artefact in a more efficient and machine readable way.

The statistical information in SDMX is broken down into two fundamental parts – structural metadata (comprising the `DataStructureDefinition`, and associated `Concepts` and `Codelists`) – see Framework for Standards – and observational data (the `DataSet`). This is an important distinction, with specific terminology associated with each part. Data, which is typically a set of numeric observations at specific points in time, is organised into data sets (`DataSet`). These data sets are structured according to a specific `DataStructureDefinition` and are described in the `Dataflow` (via `Constraints`). The `DataStructureDefinition` describes the metadata that allows an understanding of what is expressed in the `DataSet`, whilst the `Dataflow` provides the identifier and other important information (such as the periodicity of reporting) that is common to all its Components.

Note that the role of the `Dataflow` and `DataSet` is very specific in the model, and the terminology used may not be the same as used in all organisations, and specifically the term `DataSet` is used differently in SDMX than in GESMES/TS. Essentially the GESMES/TS term “Data Set” is, in SDMX, the “Dataflow” whilst the term “Data Set” in SDMX is used to describe the “container” for an instance of the data.

SPECIAL ISSUES

“Frequency” related issues

- **Special frequencies.** The issue of data collected at special (regular or irregular) intervals at a lower than daily frequency (e.g., 24 or 36 or 48 observations per year, on irregular days during the year) is not extensively discussed here. However, for data exchange purposes:
  - such data can be mapped into a series with daily frequency; this daily series will only hold observations for those days on which the measured event takes place;
  - if the collection intervals are regular, additional values to the existing frequency code list(s) could be added in the future.
- **Tick data.** The issue of data collected at irregular intervals at a higher than daily frequency (e.g., tick-by-tick data) is not discussed here either.

Copyright © SDMx TWG 2020

5.4 General Notes for Implementers

This section discusses a number of topics other than the exchange of data sets in SDMX formats. Supported only in SDMX-ML (and some in SDMX-JSON), these topics include the use of the reference metadata mechanism in SDMX, the use of Structure Sets and Reporting Taxonomies, the use of Processes, a discussion of time and data-typing, and the conventional mechanisms within the SDMX-ML Structure message regarding versioning and referencing.

5.4.1 Representations

This section does not go into great detail on these topics but provides a useful overview of these features to assist implementors in further use of the parts of the specification which are relevant to them.

There are several different representations in SDMX-ML, taken from XML Schemas and common programming languages. The table below describes the various representations, which are found in SDMX-ML, and their equivalents.

SDMX-ML Data Type	XML Schema Data Type	.NET Framework Type	Java Data Type
`String`	`xsd:string`	`System.String`	`java.lang.String`
`Big Integer`	`xsd:integer`	`System.Decimal`	`java.math.BigInteger`
`Integer`	`xsd:int`	`System.Int32`	`int`
`Long`	`xsd:long`	`System.Int64`	`long`
`Short`	`xsd:short`	`System.Int16`	`short`
`Decimal`	`xsd:decimal`	`System.Decimal`	`java.math.BigDecimal`
`Float`	`xsd:float`	`System.Single`	`float`
`Double`	`xsd:double`	`System.Double`	`double`
`Boolean`	`xsd:boolean`	`System.Boolean`	`boolean`
`URI`	`xsd:anyURI`	`System.Uri`	`Java.net.URI` or `java.lang.String`
`DateTime`	`xsd:dateTime`	`System.DateTime`	`javax.xml.datatype.XMLGregorianCalendar`
`Time`	`xsd:time`	`System.DateTime`	`javax.xml.datatype.XMLGregorianCalendar`
`GregorianYear`	`xsd:gYear`	`System.DateTime`	`javax.xml.datatype.XMLGregorianCalendar`
`GregorianMonth`	`xsd:gYearMonth`	`System.DateTime`	`javax.xml.datatype.XMLGregorianCalendar`
`GregorianDay`	`xsd:date`	`System.DateTime`	`javax.xml.datatype.XMLGregorianCalendar`
`Day`, `MonthDay`, `Month`	`xsd:g*`	`System.DateTime`	`javax.xml.datatype.XMLGregorianCalendar`
`Duration`	`xsd:duration`	`System.TimeSpan`	`javax.xml.datatype.Duration`

Copyright © SDMx TWG 2020

There are also a number of SDMX-ML data types which do not have these direct correspondences, often because they are composite representations or restrictions of a broader data type. For most of these, there are simple types which can be referenced from the SDMX schemas, for others a derived simple type will be necessary:

- `AlphaNumeric` (`common:AlphaNumericType`, string which only allows `A-z` and `0-9`)
- `Alpha` (`common:AlphaType`, string which only allows `A-z`)
- `Numeric` (`common:NumericType`, string which only allows 0-9, but is not numeric so that is can having leading zeros)
- `Count` (`xs:integer`, a sequence with an interval of "1")
- `InclusiveValueRange` (`xs:decimal` with the `minValue` and `maxValue` facets supplying the bounds)
- `ExclusiveValueRange` (`xs:decimal` with the `minValue` and `maxValue` facets supplying the bounds)
- `Incremental` (`xs:decimal` with a specified `interval`; the interval is typically enforced outside of the XML validation)
- `TimeRange` (`common:TimeRangeType`, `startDateTime` + `Duration`)
- `ObservationalTimePeriod` (`common:ObservationalTimePeriodType`, a union of `StandardTimePeriod` and `TimeRange`).
- `StandardTimePeriod` (`common:StandardTimePeriodType`, a union of `BasicTimePeriod` and `ReportingTimePeriod`).
- `BasicTimePeriod` (`common:BasicTimePeriodType`, a union of `GregorianTimePeriod` and `DateTime`)
- `GregorianTimePeriod` (`common:GregorianTimePeriodType`, a union of `GregorianYear`, `GregorianMonth`, and `GregorianDay`)
- `ReportingTimePeriod` (`common:ReportingTimePeriodType`, a union of `ReportingYear`, `ReportingSemester`, `ReportingTrimester`, `ReportingQuarter`, `ReportingMonth`, `ReportingWeek`, and `ReportingDay`).
- `ReportingYear` (`common:ReportingYearType`)
- `ReportingSemester` (`common:ReportingSemesterType`)
- `ReportingTrimester` (`common:ReportingTrimesterType`)
- `ReportingQuarter` (`common:ReportingQuarterType`)
- `ReportingMonth` (`common:ReportingMonthType`)
- `ReportingWeek` (`common:ReportingWeekType`)
- `ReportingDay` (`common:ReportingDayType`)
- `XHTML` (`common:StructuredText`, allows for multi-lingual text content that has `XHTML` markup)
- `KeyValues` (`common:DataKeyType`)
- `IdentifiableReference` (types for each IdentifiableObject)
- `GeospatialInformation` (a geo feature set, according to the pattern in section 7.2)

Data types also have a set of facets:

- `isSequence = true | false` (indicates a sequentially increasing value)
- `minLength = positive integer` (# of characters/digits)
- `maxLength = positive integer` (# of characters/digits)
- `startValue = decimal` (for numeric sequence)
- `endValue = decimal` (for numeric sequence)
- `interval = decimal` (for numeric sequence)
- `timeInterval = duration`
- `startTime = BasicTimePeriod` (for time range)
- `endTime = BasicTimePeriod` (for time range)
- `minValue = decimal` (for numeric range)
- `maxValue = decimal` (for numeric range)
- `decimal = Integer` (# of digits to right of decimal point)
- `pattern =` (a regular expression, as per W3C XML Schema)
- `isMultiLingual = boolean` (for specifying text can occur in more than one language)

Copyright © SDMx TWG 2020

Note that code lists may also have textual representations assigned to them, in addition to their enumeration of codes.

Data Types

XML and JSON schemas support a variety of data types that, although rich, are not mapped one-to-one in all cases. This section provides an explanation of the mapping performed in SDMX 3.0, between such cases.

For identifiers, text fields and Codes there are no restriction from either side, since a generic type (e.g., that of string) accompanied by the proper regular expression works equally well for both XML and JSON.

For example, for the id type, this is the XML schema definition:

```
<xs:simpleType name="IDType">
  <xs:restriction base="NestedIDType">
    <xs:pattern value="[A-Za-z0-9_@$\-]+"/>
  </xs:restriction>
</xs:simpleType>
```

Where the `NestedIDType` is also a restriction of `string`.

The above looks like this, in JSON schema:

```
"idType": {
    "type": "string",
    "pattern": "^\[A-Za-z0-9\_@$-\]+$"
}
```

There are also cases, though, that data types cannot be mapped like above. One such case is the array data type, which was introduced in SDMX 3.0 as a new representation. In JSON schema an array is already natively foreseen, while in the XML schema, this has to be defined as a complex type, with an SDMX specific definition (i.e., specific element/attribute names for SDMX). Beyond that, the minimum and/or maximum number of items within an array is possible in both cases.

Further to the above, the mapping between the non-native data types is presented in the table below:

SDMX Facet	XML Schema	JSON schema "pattern"[1] for "string" type
`GregorianYear`	`xsd:gYear`	`"^-?([1-9][0-9]{3,}|0[0-9]{3})(Z|(\+|-)((0[0-9]|1[0-3]):[0-5][0-9]|14:00))?$"`
`GregorianMonth`	`xsd:gYearMonth`	`"^-?([1-9][0-9]{3,}|0[0-9]{3})-(0[1-9]|1[0-2])(Z|(\+|-)((0[0-9]|1[0-3]):[0-5][0-9]|14:00))?$"`
`GregorianDay`	`xsd:date`	`"^-?([1-9][0-9]{3,}|0[0-9]{3})-(0[1-9]|1[0-2])-(0[1-9]|[12][0-9]|3[01])(Z|(\+|-)((0[0-9]|1[0-3]):[0-5][0-9]|14:00))?$"`
`Day`	`xsd:gDay`	`"^---(0[1-9]|[12][0-9]|3[01])(Z|(\+|-)((0[0-9]|1[0-3]):[0-5][0-9]|14:00))?$"`
`MonthDay`	`xsd:gMonthDay`	`"^--(0[1-9]|1[0-2])-(0[1-9]|[12][0-9]|3[01])(Z|(\+|-)((0[0-9]|1[0-3]):[0-5][0-9]|14:00))?$"`
`Month`	`xsd:Month`	`"^--(0[1-9]|1[0-2])(Z|(\+|-)((0[0-9]|1[0-3]):[0-5][0-9]|14:00))?$"`
`Duration`	`xsd:duration`	`"^-?P[0-9]+Y?([0-9]+M)?([0-9]+D)?(T([0-9]+H)?([0-9]+M)?([0-9]+(\.[0-9]+)?S)?)?$"`

5.4.2 Time and Time Format

This section does not go into great detail on these topics but provides a useful overview of these features to assist implementors in further use of the parts of the specification which are relevant to them.

Introduction

First, it is important to recognize that most observation times are a period. SDMX specifies precisely how Time is handled.

Copyright © SDMx TWG 2020

The representation of time is broken into a hierarchical collection of representations. A data structure definition can use of any of the representations in the hierarchy as the representation of time. This allows for the time dimension of a particular data structure definition allow for only a subset of the default representation.

The hierarchy of time formats is as follows (**bold** indicates a category which is made up of multiple formats, *italic* indicates a distinct format):

- **Observational Time Period**
  - **Standard Time Period**
    - **Basic Time Period**
      - **Gregorian Time Period**
      - *Date Time*
    - **Reporting Time Period**
  - *Time Range*

The details of these time period categories and of the distinct formats which make them up are detailed in the sections to follow.

Observational Time Period

This is the superset of all time representations in SDMX. This allows for time to be expressed as any of the allowable formats.

Standard Time Period

This is the superset of any predefined time period or a distinct point in time. A time period consists of a distinct start and end point. If the start and end of a period are expressed as date instead of a complete date time, then it is implied that the start of the period is the beginning of the start day (i.e. 00:00:00) and the end of the period is the end of the end day (i.e. 23:59:59).

Gregorian Time Period

A Gregorian time period is always represented by a Gregorian year, year-month, or day. These are all based on ISO 8601 dates. The representation in SDMX-ML messages and the period covered by each of the Gregorian time periods are as follows:

- **Gregorian Year:**
  - Representation: `xs:gYear (YYYY)`
  - Period: the start of January 1 to the end of December 31
- **Gregorian Year Month**:
  - Representation: `xs:gYearMonth (YYYY-MM)`
  - Period: the start of the first day of the month to end of the last day of the month
- **Gregorian Day**:
  - Representation: `xs:date (YYYY-MM-DD)`
  - Period: the start of the day (00:00:00) to the end of the day (23:59:59)

Date Time

This is used to unambiguously state that a date-time represents an observation at a single point in time. Therefore, if one wants to use SDMX for data which is measured at a distinct point in time rather than being reported over a period, the date-time representation can be used.

Representation: `xs:dateTime YYYY-MM-DDThh:mm:ss` [2]

Copyright © SDMx TWG 2020

Standard Reporting Period

Standard reporting periods are periods of time in relation to a reporting year. Each of these standard reporting periods has a duration (based on the ISO 8601 definition) associated with it. The general format of a reporting period is as follows:

`[REPORTING_YEAR]-[PERIOD_INDICATOR][PERIOD_VALUE]`

Where:

- `REPORTING_YEAR` represents the reporting year as four digits (`YYYY`)
- `PERIOD_INDICATOR` identifies the type of period which determines the duration of the period
- `PERIOD_VALUE` indicates the actual period within the year

Copyright © SDMx TWG 2020

The following section details each of the standard reporting periods defined in SDMX:

Copyright © SDMx TWG 2020

- **Reporting Year**:
  - Period Indicator: `A`
  - Period Duration: `P1Y` (one year)
  - Limit per year: 1
  - Representation: `common:ReportingYearType` (`YYYY-A1` , e.g. `2000-A1`)
- **Reporting Semester:**
  - Period Indicator: `S`
  - Period Duration: `P6M` (six months)
  - Limit per year: 2
  - Representation: `common:ReportingSemesterType` (`YYYY-Ss` , e.g. `2000-S2`)
- **Reporting Trimester:**
  - Period Indicator: `T`
  - Period Duration: `P4M` (four months)
  - Limit per year: 3
  - Representation: `common:ReportingTrimesterType` (`YYYY-Tt` , e.g. `2000-T3`)
- **Reporting Quarter:**
  - Period Indicator: `Q`
  - Period Duration: `P3M` (three months)
  - Limit per year: 4
  - Representation: `common:ReportingQuarterType` (`YYYY-Qq` , e.g. `2000-Q4`)
- **Reporting Month**:
  - Period Indicator: `M`
  - Period Duration: `P1M` (one month)
  - Limit per year: 1
  - Representation: `common:ReportingMonthType` (`YYYY-Mmm` , e.g. `2000-M12`)
  - Notes: The reporting month is always represented as two digits, therefore 1-9 are 0 padded (e.g. 01). This allows the values to be sorted chronologically using textual sorting methods.
- **Reporting Week**:
  - Period Indicator: `W`
  - Period Duration: `P7D` (seven days)
  - Limit per year: 53
  - Representation: `common:ReportingWeekType` (`YYYY-Www` , e.g. `2000-W53`)
  - Notes: There are either 52 or 53 weeks in a reporting year. This is based on the ISO 8601 definition of a week (Monday - Saturday), where the first week of a reporting year is defined as the week with the first Thursday on or after the reporting year start day.[3] The reporting week is always represented as two digits, therefore 1-9 are 0 padded (e.g. 01). This allows the values to be sorted chronologically using textual sorting methods.
- **Reporting Day**:
  - Period Indicator: `D`
  - Period Duration: `P1D` (one day)
  - Limit per year: 366
  - Representation: `common:ReportingDayType` (`YYYY-Dddd` , e.g. `2000-D366`)
  - Notes: There are either 365 or 366 days in a reporting year, depending on whether the reporting year includes leap day (February 29). The reporting day is always represented as three digits, therefore 1-99 are 0 padded (e.g. 001). This allows the values to be sorted chronologically using textual sorting methods.

Copyright © SDMx TWG 2020

The meaning of a reporting year is always based on the start day of the year and requires that the reporting year is expressed as the year at the start of the period. This start day is always the same for a reporting year, and is expressed as a day and a month (e.g. July 1). Therefore, the reporting year 2000 with a start day of July 1 begins on July 1, 2000.

A specialized attribute (reporting year start day) exists for the purpose of communicating the reporting year start day. This attribute has a fixed identifier (`REPORTING_YEAR_START_DAY`) and a fixed representation (`xs:gMonthDay`) so that it can always be easily identified and processed in a data message. Although this attribute exists in specialized sub-class, it functions the same as any other attribute outside of its identification and representation. It must takes its identity from a concept and state its relationship with other components of the data structure definition. The ability to state this relationship allows this reporting year start day attribute to exist at the appropriate levels of a data message. In the absence of this attribute, the reporting year start date is assumed to be January 1; therefore if the reporting year coincides with the calendar year, this Attribute is not necessary.

Since the duration and the reporting year start day are known for any reporting period, it is possible to relate any reporting period to a distinct calendar period. The actual Gregorian calendar period covered by the reporting period can be computed as follows (based on the standard format of `[REPROTING_YEAR]-[PERIOD_INDICATOR][PERIOD_VALUE]` and the reporting year start day as `[REPORTING_YEAR_START_DAY]`):

1. Determine `[REPORTING_YEAR_BASE]` :

   Combine `[REPORTING_YEAR]` of the reporting period value (`YYYY`) with `[REPORTING_YEAR_START_DAY]` (MM-DD) to get a date (`YYYY-MM-DD`). This is the `[REPORTING_YEAR_START_DATE]`

2. If the `[PERIOD_INDICATOR]` is W:

   a. If `[REPORTING_YEAR_START_DATE]` is a Friday, Saturday, or Sunday:

      Add[4] (`P3D` , `P2D` , or `P1D` respectively) to the `[REPORTING_YEAR_START_DATE]` . The result is the `[REPORTING_YEAR_BASE]` .

   b. **If `[REPORTING_YEAR_START_DATE]` is a Monday, Tuesday, Wednesday, or Thursday:

   ```
   Add[^4] (`P0D`, `-P1D`, `-P2D`, or `-P3D` respectively) to the
   `[REPORTING_YEAR_START_DATE]`. The result is the
   `[REPORTING_YEAR_BASE]`.
   ```

3. Else:

   The `[REPORTING_YEAR_START_DATE]` is the `[REPORTING_YEAR_BASE]` .

4. Determine `[PERIOD_DURATION]` :

   a. If the `[PERIOD_INDICATOR]` is A, the `[PERIOD_DURATION]` is P1Y.

   b. If the `[PERIOD_INDICATOR]` is S, the `[PERIOD_DURATION]` is P6M.

   c. If the `[PERIOD_INDICATOR]` is T, the `[PERIOD_DURATION]` is P4M.

   d. If the `[PERIOD_INDICATOR]` is Q, the `[PERIOD_DURATION]` is P3M.

   e. If the `[PERIOD_INDICATOR]` is M, the `[PERIOD_DURATION]` is P1M.

   f. If the `[PERIOD_INDICATOR]` is W, the `[PERIOD_DURATION]` is P7D.

   g. If the `[PERIOD_INDICATOR]` is D, the `[PERIOD_DURATION]` is P1D.

5. Determine `[PERIOD_START]` :

   Subtract one from the `[PERIOD_VALUE]` and multiply this by the `[PERIOD_DURATION]` . Add[4] this to the `[REPORTING_YEAR_BASE]` . The result is the `[PERIOD_START]` .

6. Determine the `[PERIOD_END]` :

   Multiply the `[PERIOD_VALUE]` by the `[PERIOD_DURATION]` . Add[4] this to the `[REPORTING_YEAR_BASE]` add[4] -P1D. The result is the `[PERIOD_END]` .

For all of these ranges, the bounds include the beginning of the `[PERIOD_START]` (i.e. 00:00:00) and the end of the `[PERIOD_END]` (i.e. 23:59:59).

Copyright © SDMx TWG 2020

EXAMPLES:

2010-Q2, REPORTING_YEAR_START_DAY = –07-01 (July 1)

1. `[REPORTING_YEAR_START_DATE] = 2010-07-01`

   case b) `[REPORTING_YEAR_BASE] = 2010-07-01`
2. `[PERIOD_DURATION] = P3M`
3. `(2-1) * P3M = P3M`

   `2010-07-01 + P3M = 2010-10-01`

   `[PERIOD_START] = 2010-10-01`
4. `2 * P3M = P6M`

   `2010-07-01 + P6M = 2010-13-01 = 2011-01-01`

   `2011-01-01 + -P1D = 2010-12-31`

   `[PERIOD_END] = 2010-12-31`

   The actual calendar range covered by `2010-Q2` (assuming the reporting year begins July 1) is `2010-10-01T00:00:00` / `2010-12-31T23:59:59`

2011-W36, REPORTING_YEAR_START_DAY = –07-01 (July 1)

1. `[REPORTING_YEAR_START_DATE] = 2010-07-01`

   case a) `2011-07-01 = Friday`

   `2011-07-01 + P3D = 2011-07-04`

   `[REPORTING_YEAR_BASE] = 2011-07-04`
2. `[PERIOD_DURATION] = P7D`
3. `(36-1) \* P7D = P245D`

   `2011-07-04 + P245D = 2012-03-05`

   `[PERIOD_START] = 2012-03-05`
4. `36 * P7D = P252D`

   `2011-07-04 + P252D =2012-03-12`

   `2012-03-12 + -P1D = 2012-03-11`

   `[PERIOD_END] = 2012-03-11`

   The actual calendar range covered by `2011-W36` (assuming the reporting year begins July 1) is `2012-03-05T00:00:00` / `2012-03-11T23:59:59`

Distinct Range

In the case that the reporting period does not fit into one of the prescribe periods above, a distinct time range can be used. The value of these ranges is based on the ISO 8601 time interval format of start/duration. Start can be expressed as either an ISO 8601 date or a date-time, and duration is expressed as an ISO 8601 duration. However, the duration can only be positive.

Copyright © SDMx TWG 2020

Time Format

In version 2.0 of SDMX there is a recommendation to use the time format attribute to gives additional information on the way time is represented in the message. Following an appraisal of its usefulness this is no longer required. However, it is still possible, if required , to include the time format attribute in SDMX-ML.

Code	Format
OTP	Observational Time Period: Superset of all SDMX time formats(Gregorian Time Period, Reporting Time Period, and Time Range)
STP	Standard Time Period: Superset of Gregorian and Reporting TimePeriods
GTP	Superset of all Gregorian Time Periods and date-time
RTP	Superset of all Reporting Time Periods
TR	Time Range: Start time and duration(`YYYY-MM-DD(Thh:mm:ss)?/<duration>`)
GY	Gregorian Year (`YYYY`)
GTM	Gregorian Year Month (`YYYY-MM`)
GD	Gregorian Day (`YYYY-MM-DD`)
DT	Distinct Point: date-time (`YYYY-MM-DDThh:mm:ss`)
RY	Reporting Year (`YYYY-A1`)
RS	Reporting Semester (`YYYY-Ss`)
RT	Reporting Trimester (`YYYY-Tt`)
RQ	Reporting Quarter (`YYYY-Qq`)
RM	Reporting Month (`YYYY-Mmm`)
RW	Reporting Week (`YYYY-Www`)
RD	Reporting Day (`YYYY-Dddd`)

Time Zones

In alignment with ISO 8601, SDMX allows the specification of a time zone on all time periods and on the reporting year start day. If a time zone is provided on a reporting year start day, then the same time zone (or none) should be reported for each reporting

Copyright © SDMx TWG 2020

time period. If the reporting year start day and the reporting period time zone differ, the time zone of the reporting period will take precedence. Examples of each format with time zones are as follows (time zone indicated in bold):

- Time Range (start date): 2006-06-05**-05:00**/P5D
- Time Range (start date-time): 2006-06-05T00:00:00**-05:00**/P5D
- Gregorian Year: 2006**-05:00**
- Gregorian Month: 2006-06**-05:00**
- Gregorian Day: 2006-06-05**-05:00**
- Distinct Point: 2006-06-05T00:00:00**-05:00**
- Reporting Year: 2006-A1**-05:00**
- Reporting Semester: 2006-S2**-05:00**
- Reporting Trimester: 2006-T2**-05:00**
- Reporting Quarter: 2006-Q3**-05:00**
- Reporting Month: 2006-M06**-05:00**
- Reporting Week: 2006-W23**-05:00**
- Reporting Day: 2006-D156**-05:00**
- Reporting Year Start Day: –07-01**-05:00**

According to ISO 8601, a date without a time-zone is considered “local time”. SDMX assumes that local time is that of the sender of the message. In this version of SDMX, an optional field is added to the sender definition in the header for specifying a time zone. This field has a default value of ‘Z’ (UTC). This determination of local time applies for all dates in a message.

Representing Time Spans Elsewhere

It has been possible since SDMX 2.0 for a Component to specify a representation of a time span. Depending on the format of the data message, this resulted in either an element with 2 XML attributes for holding the start time and the duration or two separate XML attributes based on the underlying Component identifier. For example, if `REF_PERIOD` were given a representation of time span, then in the Compact data format, it would be represented by two XML attributes; `REF_PERIODStartTime` (holding the start) and `REF_PERIOD` (holding the duration). If a new simple type is introduced in the SDMX schemas that can hold ISO 8601 time intervals, then this will no longer be necessary. What was represented as this:

```
<Series REF_PERIODStartTime="2000-01-01T00:00:00" REF_PERIOD="P2M">
```

can now be represented with this:

```
<Series REF_PERIOD="2000-01-01T00:00:00/P2M">
```

Notes on Formats

There is no ambiguity in these formats so that for any given value of time, the category of the period (and thus the intended time period range) is always clear. It should also be noted that by utilizing the ISO 8601 format, and a format loosely based on it for the report periods, the values of time can easily be sorted chronologically without additional parsing.

Effect on Time Ranges

All SDMX-ML data messages are capable of functioning in a manner similar to SDMX-EDI if the Dimension at the observation level is time: the time period for the first observation can be stated and the rest of the observations can omit the time value as it can be derived from the start time and the frequency. Since the frequency can be determined based on the actual format of the time value for everything but distinct points in time and time ranges, this makes is even simpler to process as the interval between time ranges is known directly from the time value.

Copyright © SDMx TWG 2020

Time in Query Messages

When querying for time values, the value of a time parameter can be provided as any of the Observational Time Period formats and must be paired with an operator. This section will detail how systems processing query messages should interpret these parameters.

Fundamental to processing a time value parameter in a query message is understanding that all time periods should be handled as a distinct range of time. Since the time parameter in the query is paired with an operator, this also effectively represents a distinct range of time. Therefore, a system processing the query must simply match the data where the time period for requested parameter is encompassed by the time period resulting from value of the query parameter. The following table details how the operators should be interpreted for any time period provided as a parameter.

Operator	Rule
Greater Than	Any data after the last moment of the period
Less Than	Any data before the first moment of the period
Greater Than or Equal To	Any data on or after the first moment of the period
Less Than or Equal To	Any data on or before the last moment of the period
Equal To	Any data which falls on or after the first moment of the period and before or on the last moment of the period

Reporting Time Periods as query parameters are handled like this: any data within the bounds of the reporting period for the year is matched, regardless of the actual start day of the reporting year. In addition, data reported against a normal calendar period is matched if it falls within the bounds of the time parameter based on a reporting year start day of January 1. When determining whether another reporting period falls within the bounds of a report period query parameter, one will have to take into account the actual time period to compare weeks and days to higher order report periods. This will be demonstrated in the examples to follow.

Gregorian Period

Query Parameter: Greater than 2010

Literal Interpretation: Any data where the start period occurs after 2010-12-31T23:59:59.

Example Matches:

- 2011 or later
- 2011-01 or later
- 2011-01-01 or later
- 2011-01-01/P[Any Duration] or any later start date
- 2011-[Any reporting period] (any reporting year start day)
- 2010-S2 (reporting year start day –07-01 or later)
- 2010-T3 (reporting year start day –07-01 or later)
- 2010-Q3 or later (reporting year start day –07-01 or later)
- 2010-M07 or later (reporting year start day –07-01 or later)
- 2010-W28 or later (reporting year start day –07-01 or later)
- 2010-D185 or later (reporting year start day –07-01 or later)

Copyright © SDMx TWG 2020

Reporting Period

Query Parameter: Greater than or equal to 2010-Q3

Literal Interpretation: Any data with a reporting period where the start period is on or after the start period of 2010-Q3 for the same reporting year start day, or and data where the start period is on or after 2010-07-01.

Example Matches:

- 2011 or later
- 2010-07 or later
- 2010-07-01 or later
- 2010-07-01/P[Any Duration] or any later start date
- 2011-[Any reporting period] (any reporting year start day)
- 2010-S2 (any reporting year start day)
- 2010-T3 (any reporting year start day)
- 2010-Q3 or later (any reporting year start day)
- 2010-M07 or later (any reporting year start day)
- 2010-W27 or later [reporting year start day –01-01][5]
- 2010-D182 or later (reporting year start day –01-01)
- 2010-W28 or later [reporting year start day –07-01][6]
- 2010-D185 or later (reporting year start day –07-01)

5.4.3 Versioning

Versioning operates at the level of versionable and maintainable objects in the SDMX information model. Within the SDMX Structure and MetadataSet messages, there is a well-defined pattern for artefact versioning and referencing. The artefact identifiers are qualified by their version numbers – that is, an object with an Agency of “A”, and ID of “X” and a version of “1.0.0” is a different object than one with an Agency of “A”, an ID of “X”, and a version of “1.1.0”.

As of SDMX 3.0, the versioning rules are extended to allow for truly versioned artefacts through the implementation of the rules of the well-known practice called Semantic Versioning, in addition to the legacy non-restrictive versioning scheme. In addition, the `"isFinal"` property is removed from `MaintainableArtefact`. According to the legacy versioning, any artefact defined without a version is equivalent to following the legacy versioning, thus having version ‘1.0’.

Non-versioned artefacts

Indeed, some use cases do not need or are incompatible with versioning for some or all their structural artefacts, such as the Agency, Data Providers, Metadata Providers and Data Consumer Schemes. These artefacts follow the legacy versioning, with a fixed version set to ‘1.0’.

Many existing organisation’s data management systems work with version-less structures and apply ad-hoc structural metadata governance processes. The new non-versioned artefacts will allow supporting those numerous situations, where organisations do not manage version numbers.

Semantically versioned artefacts

Since the purpose of SDMX versioning is to allow communicating the structural artefact changes to data exchange partners and connected systems, SDMX 3.0 offers Semantic Versioning (aka SemVer) with a clear and unambiguous syntax to all semantically versioned SDMX 3.0 structural artefacts. Semantic versioning will thus better respond to situations where the SDMX standard itself is the only structural contract between data providers and data consumers and where changes in structures can only be communicated through the version number increases.

Copyright © SDMx TWG 2020

The semantic version number consists of four parts: `MAJOR`, `MINOR`, `PATCH` and `EXTENSION`, the first three parts being separated by a dot (`.`), the last two parts being separated by a hyphen (`-`): `MAJOR.MINOR.PATCH-EXTENSION`. All versions are ordered.

The detailed rules for semantic versioning are listed in chapter 14 in the annex for “Semantic Versioning”. In short, they define:

Given a version number `MAJOR.MINOR.PATCH` (without `EXTENSION`), when making changes to that semantically versioned SDMX artefact, then one must increment the:

1. `MAJOR` version when backwards incompatible artefact changes are made,
2. `MINOR` version when artefact elements are added in a backwards compatible manner, or
3. `PATCH` version when backwards compatible artefact property changes are made.

When incrementing a version part, the right-hand side parts are 0-ed (reset to ‘0’).

Extensions can be added, changed or dropped.

Given an extended version number `MAJOR.MINOR.PATCH-EXTENSION`, when making changes to that versioned artefact, then one is not required to increment the version if those changes are within the allowed scope of the version increment from the previous version (if that existed); otherwise, the above version increment rules apply. `EXTENSION`s can be used e.g., for drafting or a pre-release.

Semantically versioned SDMX artefacts will thus be safe to use. Specific version patterns allow them to become either immutable, i.e., the maintainer commits to never change their content, or changeable only within a well-defined scope. If any further change is required, a new version must be created first. Furthermore, the impact of the further change is communicated using a clear version increment. The built-in version extension facility allows for eased drafting of new SDMX artefact versions.

The production versions of identifiable artefacts are assumed stable, i.e., they do not have an `EXTENSION`. This is because once in production, an artefact cannot change in any way, or it must change the version. For cases where an artefact is not static, like during the drafting, the version must indicate this by including an `EXTENSION`. Draft artefacts should not be used outside of a specific system designed to accommodate them. For most purposes, all artefacts should become stable before being used in production.

Legacy-versioned artefacts

Organisations wishing to keep a maximum of backwards compatibility with existing implementations can continue using the previous 2-digit convention for version numbers (`MAJOR.MINOR`) as in the past, such as ‘2.3’, but without the ‘isFinal’ property. The new SDMX 3.0 standard does not add any strict rules or guarantees about changes in those artefacts, since the legacy versioning rules were rather loose and non-binding, including the meaning of the ‘isFinal’ property, and their implementations were varying.

In order to make artefacts immutable or changes truly predictable, a move to the new semantic versioning syntax is required.

Dependency management and references

New flexible dependency specifications with wildcarding allow for easier data model maintenance and enhancements for semantically versioned SDMX artefacts. This allows implementing a smart referencing mechanism, whereby an artefact may reference:

- a fixed version of another artefact
- the **latest available** version of another artefact
- the **latest backward compatible** version of another artefact, or the **latest backward and forward compatible** version of another artefact.

References not representing a strict artefact dependency, such as the target artefacts defined in a `MetadataProvisionAgreement` allow for linking to **all currently available** versions of another artefact. Another illustrative case for such loose referencing is that of Constraints and flows. A Constraint may reference many Dataflows or Metadataflows, the addition of more references to flow objects does not version the Constraint. This is because the Constraints are not properties of the flows – they merely make references to them.

Copyright © SDMx TWG 2020

Semantically versioned artefacts must only reference other semantically versioned artefacts, which may include extended versions. Non-versioned and legacy-versioned artefacts can reference any other non-versioned or versioned (whether semantic or legacy) artefacts. The scope of wildcards in references adapts correspondingly.

The mechanism named “early binding” refers to a dependency on a stable versioned artefact – everything with a stable versioned identity is a known quantity and will not change. The “late binding” mechanism is based on a wildcarded reference, and it resolves that reference and determines the currently related artefact at runtime.

One area which is much impacted by this versioning scheme is the ability to reference external objects. With the many dependencies within the various structural objects in SDMX, it is useful to have a scheme for external referencing. This is done at the level of maintainable objects (DSDs, Codelists, Concept Schemes, etc.) In an SDMX Structure Message, whenever an `"isExternalReference"` attribute is set to true, then the application must resolve the address provided in the associated `"uri"` attribute and use the SDMX Structure Message stored at that location for the full definition of the object in question. Alternately, if a registry `"urn"` attribute has been provided, the registry can be used to supply the full details of the object.

The detailed rules for dependency management and references are listed in chapter 14 in the annex for “Semantic Versioning”.

In order to allow resolving the described new forms of dependencies, the SDMX 3.0 Rest API supports retrievals legacy-versioned, wildcarded and extended artefact versions:

- Artefact queries for a **specific** version (X.Y, X.Y.Z or X.Y.Z-EXT).
- Artefact queries for **latest available** semantic versions within the wildcard scope (X+.Y.Z, X.Y+.Z or X.Y.Z+).
- Queries for **non-versioned** artefacts.
- Artefact queries for **all available** semantic versions within the wildcard scope (*, X.* or X.Y.*), where only the first form is required for resolving wildcarded loose references.

The combination of wildcarded queries with a specific version extension is not permitted.

Full details can be found in the SDMX RESTful web services specification.

5.4.4 Structural Metadata Querying Best Practices

When querying for structural metadata, the ability to state how references should be resolved is quite powerful. However, this mechanism is not always necessary and can create an undue burden on the systems processing the queries if it is not used properly.

Any structural metadata object which contains a reference to an object can be queried based on that reference. For example, a categorisation references both a category and the object is it categorising. As this is the case, one can query for categorisations which categorise a particular object or which categorise against a particular category or category scheme. This mechanism should be used when the referenced object is known.

When the referenced object is not known, then the reference resolution mechanism could be used. For example, suppose one wanted to find all category schemes and the related categorisations for a given maintenance agency. In this case, one could query for the category scheme by the maintenance agency and specify that parent and sibling references should be resolved. This would result in the categorisations which reference the categories in the matched schemes to be returned, as well as the object which they categorise.

---

1. Regular expressions, as specified in W3C XML Schema Definition. ↩
2. The seconds can be reported fractionally ↩
3. ISO 8601 defines alternative definitions for the first week, all of which produce equivalent results. Any of these definitions could be substituted so long as they are in relation to the reporting year start day. ↩
4. The rules for adding durations to a date time are described in the W3C XML Schema specification. See w3.org for further details. ↩↩ ↩↩
5. 2010-Q3 (with a reporting year start day of –01-01) starts on 2010-07-01. This is day 4 of week 26, therefore the first week matched is week 27. ↩
6. 2010-Q3 (with a reporting year start day of –07-01) starts on 2011-01-01. This is day 6 of week 27, therefore the first week matched is week 28. ↩

Copyright © SDMx TWG 2020

5.5 Reference Metadata

5.5.1 Scope of the Metadata Structure Definition (MSD)

The scope of the MSD is reduced in SDMX 3.0, by means of simplifying its structure, but also in the way referenced Artefacts are targeted. In fact, the MSD is restricted to play the role of a single container, without targeting any specific Artefact. The possible targets of Metadata Set are specified in the Metadataflows or Metadata Provision Agreements relating to that MSD. To achieve that, the structure of the Metadataflow has changed in this version of the standard. Moreover, the Metadata Provision Agreement Artefact is introduced to include this feature.

Two more changes, introduced in this version, are the following:

- The Metadata Set becomes a Maintainable Artefact but maintained by a Metadata Provider (another new Artefact in this version).
- Metadata Attributes may also be used in Data Structure Definitions, as long as the latter reference the Metadata Structure Definition that specify those Metadata Attributes.

5.5.2 Identification of the Object(s) to which the Metadata is to be attached

The following example shows the structure and naming of the MSD and related components for creating reference metadata.

The schematic structure of an MSD is shown below.

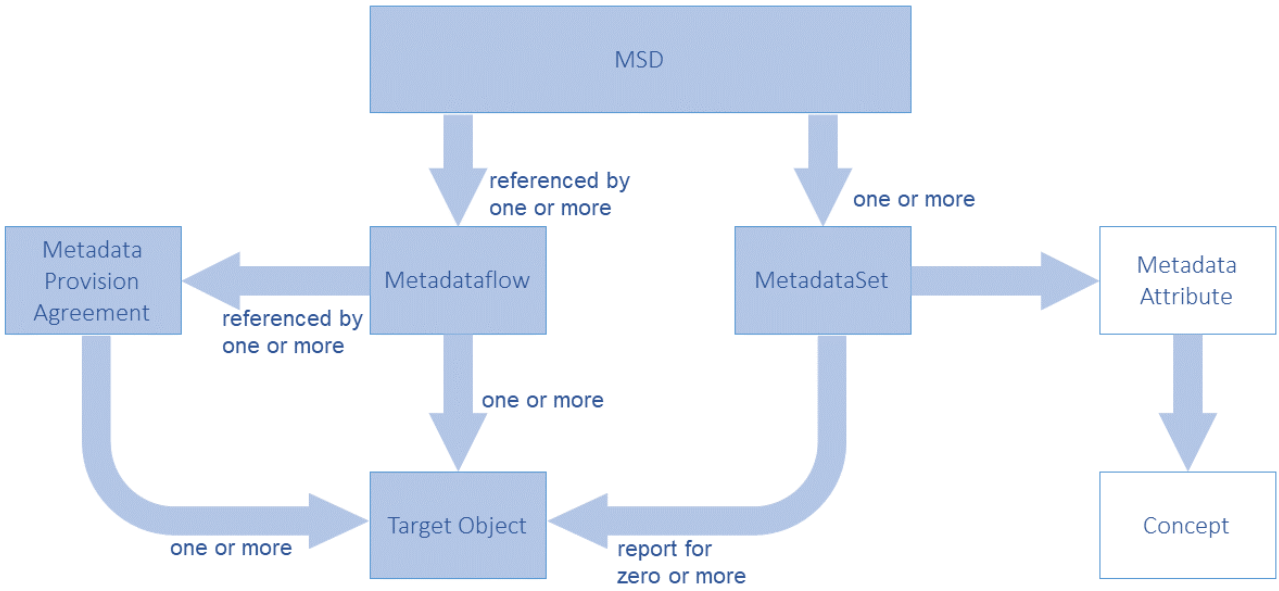


Figure 1. Schematic of the Metadata Structure Definition

The MSD contains one Metadata Attribute Descriptor comprising the Metadata Attributes that identify the Concepts for which metadata may be reported in the Metadata Set. The Metadataflow and Metadata Provision Agreement comprise the specification of the objects to which metadata can be reported in a Metadata Set (Metadata Target(s)).

The high-level view of the MSD, as well as the way the Metadataflow and Metadata Provision Agreement specify the Targets:

Copyright © SDMx TWG 2020

```
<str:MetadataStructure agencyID="SDMX" id="MSD" version="1.0.0-draft">
  <com:Name>MSD 3.0 sample</com:Name>
  <str:MetadataAttributeDescriptor id="MetadataAttributeDescriptor">
    ...
  </str: MetadataAttributeDescriptor>
</str:MetadataStructure>
```

Figure 2. The high-level view of the MSD containing one Metadata Attribute Descriptor

```
<str:Metadataflow agencyID="OECD" id="GENERAL_METADATA" version="1.0.0-draft">
  <com:Name xml:lang="en">Metadataflow 3.0 sample</com:Name>
  <str:Structure>urn:sdmx:org.sdmx.infomodel.metadatastructure.
    MetadataStructure=OECD:MSD(1.0.0-draft)</str:Structure>
  <!-- Attach to any Dataflows maintained by the OECD -->
  <str:Targets>urn:sdmx:org.sdmx.infomodel.datastructure.
    Dataflow=OECD:*(*)</str:Targets>
</str:Metadataflow>
```

Figure 3. Wildcarded Target Objects as specified in a Metadataflow

```
<str:MetadataProvisionAgreement agencyID="OECD" id="ABS_INDICATORS" version="1.0.0-draft">
  <com:Name xml:lang="en">Metadata Provision Agreement 3.0 sample</com:Name>
  <str:StructureUsage>urn:sdmx:org.sdmx.infomodel.metadatastructure.
    Metadataflow=OECD:GENERAL_METADATA(1.0.0-draft)</str:StructureUsage>
  <str:MetadataProvider>urn:sdmx:org.sdmx.infomodel.base.
    MetadataProvider=OECD:METADATA_PROVIDERS(1.0).ABS</str:MetadataProvider>
  <!-- Attach to specific Dataflows maintained by the OECD -->
  <str:Target>urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=
    OECD:GDP(*)</str:Target>
  <str:Target>urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=
    OECD:EXR(*)</str:Target>
  <str:Target>urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=
    OECD:ABC(*)</str:Target>
</str:MetadataProvisionAgreement>
```

Figure 4. Specific Target Objects as specified in a Metadata Provision Agreement

Note that the SDMX-ML schemas have specific XML elements for each identifiable object type because identifying, for instance, a Maintainable Object has different properties from an Identifiable Object which must also include the agencyId, version, and id of the Maintainable Object in which it resides.

5.5.3 Metadata Structure Definition

An example is shown below.

```
<str:MetadataStructure agencyID="SDMX" id="MSD" version="1.0.0-draft">
  <com:Name>MSD 3.0 sample</com:Name>
  <str:MetadataAttributeDescriptor id="MetadataAttributeDescriptor">
    <str:MetadataAttribute id="CONTACT" isPresentational="true">
      <str:ConceptIdentity>urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=SDMX:CONCEPTS(1.0.0).CONTACT</str:ConceptIdentity>
      <str:MetadataAttribute id="CONTACT_NAME" minOccurs="1" maxOccurs="1">
        <str:ConceptIdentity>urn:sdmx:org.sdmx.infomodel.conceptscheme.
          Concept=SDMX:CONCEPTS(1.0.0).CONTACT_NAME</str:ConceptIdentity>
        <str:LocalRepresentation>
          <str:TextFormat textType="String"/>
        </str:LocalRepresentation>
      </str:MetadataAttribute>
      <str:MetadataAttribute id="ADDRESS" minOccurs="1" maxOccurs="3" isPresentational="true">
        <str:ConceptIdentity>urn:sdmx:org.sdmx.infomodel.conceptscheme.
          Concept=SDMX:CONCEPTS(1.0.0).ADDRESS</str:ConceptIdentity>
        <str:MetadataAttribute id="HOUSE_NUMBER" minOccurs="1" maxOccurs="1">
          <str:ConceptIdentity>urn:sdmx:org.sdmx.infomodel.conceptscheme.
            Concept=SDMX:CONCEPTS(1.0.0).HOUSE_NUMBER</str:ConceptIdentity>
          <str:LocalRepresentation>
            <str:TextFormat textType="Integer"/>
          </str:LocalRepresentation>
        </str:MetadataAttribute>
      </str:MetadataAttribute>
    </str:MetadataAttribute>
  </str:MetadataAttributeDescriptor>
</str:MetadataStructure>
```


Copyright © SDMx TWG 2020

Figure 5. Example MSD showing specification of some Metadata Attributes

This example shows the following hierarchy of Metadata Attributes:

- Contact – this is presentational; no metadata is expected to be reported at this level
  - Contact Name
  - Address – this is also presentational; up to 3 addresses are allowed
    - House Number

5.5.4 Metadata Set

An example of reporting metadata according to the MSD described above, is shown below.

```
<msg:MetadataSet id="ALB" metadataProviderID="OECD" version="1.0.0">
 <str:MetadataProvision>urn:sdmx:org.sdmx.infomodel.registry.MetadataProvisionAgreement=OECD:ABS_INDICATORS(1.0.0-draft)</str:MetadataProvision>
 <str:Target>urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=OECD:GDP(1.0.0)</str:Target>
 <md:AttributeSet>
  <md:ReportedAttribute id="CONTACT">
   <md:AttributeSet>
    <md:ReportedAttribute id="CONTACT_NAME">John Doe
    </md:ReportedAttribute>
    <md:ReportedAttribute id="ADDRESS">
     <md:AttributeSet>
      <md:ReportedAttribute id="STREET_NAME">
       <com:Text xml:lang="en">5th Avenue</com:Text>
      </md:ReportedAttribute>
      <md:ReportedAttribute id="HOUSE_NUMBER">12
      </md:ReportedAttribute>
     </md:AttributeSet>
    </md:ReportedAttribute>
    <md:ReportedAttribute id="HTML_ATTR">
     <com:StructuredText xml:lang="en">
      <div xmlns="http://www.w3.org/1999/xhtml">
       <p>Lorem Ipsum</p>
      </div>
     </com:StructuredText>
    </md:ReportedAttribute>
   </md:AttributeSet>
  </md:ReportedAttribute>
 </md:AttributeSet>
</msg:MetadataSet>
```

Figure 6. Example Metadata Set

This example shows:

1. The reference to the Metadata Provision Agreement and Metadata Target
2. The reported metadata attributes (AttributeSet)

5.5.5 Reference Metadata in Data Structure Definition and Dataset

An important change of SDMX 3.0 is the ability to reference an MSD within a DSD, in order to report any Metadata Attributes defined in the former to Datasets of the latter. This is achieved by the following:

- In a DSD, the user may add a reference to one MSD.
- In the Attribute Descriptor of the DSD, the user may include any Metadata Attributes defined in the linked MSD.
  - For each link to a Metadata Attribute, an Attribute Relationship may be specified (similarly to that for Data Attributes).
- In any Dataset complying with this DSD, Metadata Attributes may be reported according to the specified Attribute Relationship.
  - The hierarchy of the Metadata Attributes defined in the MSD must be respected and they are reported in the same way as in a Metadataset, under the level they are related within the DSD, via their Attribute Relationship.
- In Data Constraints, the user is allowed to restrict values for Metadata Attributes, in the same way as Data Attributes (more on this in section “10 Constraints”).

Copyright © SDMx TWG 2020

5.6 Codelist

As of SDMX 3.0, Codelists have gained new features like geospatial properties, inheritance and extension. Moreover, hierarchies (used to build complex hierarchies of one or more Codelists) are now linked to other Artefacts in order to facilitate the formers' usage in dissemination or other scenarios. For all geospatial related features, as well as the new Geographical Codelist, please refer to section 7.

5.6.1 Codelist extension and discriminated unions

A `Codelist` can extend one or more Codelists. `Codelist` extensions are defined as a list of references to parent Codelists. The order of the references is important when it comes to conflict resolution on Code Ids. When two Codelists have the same Code Id, the `Codelist` referenced later takes priority. In the example below, the code `'A'`, exists in both `CL_INDICATOR` and `CL_SERIES`. The `Codelist` `CL_INDICATOR_EX` will contain the code 'A' from `CL_SERIES` as this was the second `Codelist` to be referenced in the sequence of references.

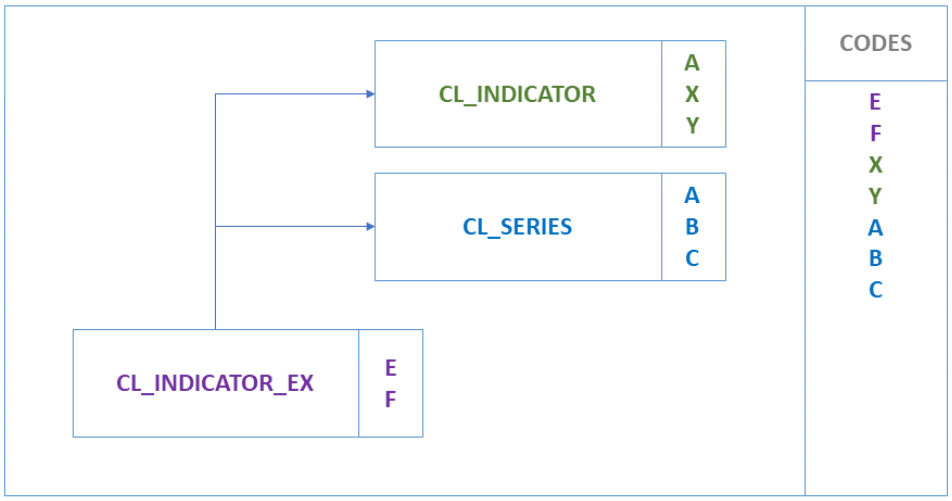


Figure 7. Codelist extension

As the extended `Codelist`, `CL_INDICATOR_EX` in this example, may also define its own Codes, these take the ultimate priority over any referenced Codelists. If `CL_INDICATOR_EX` defines Code `'A'`, then this will be used instead of Code `'A'` from `CL_INDICATOR` and `CL_SERIES`, as shown below:

Copyright © SDMx TWG 2020

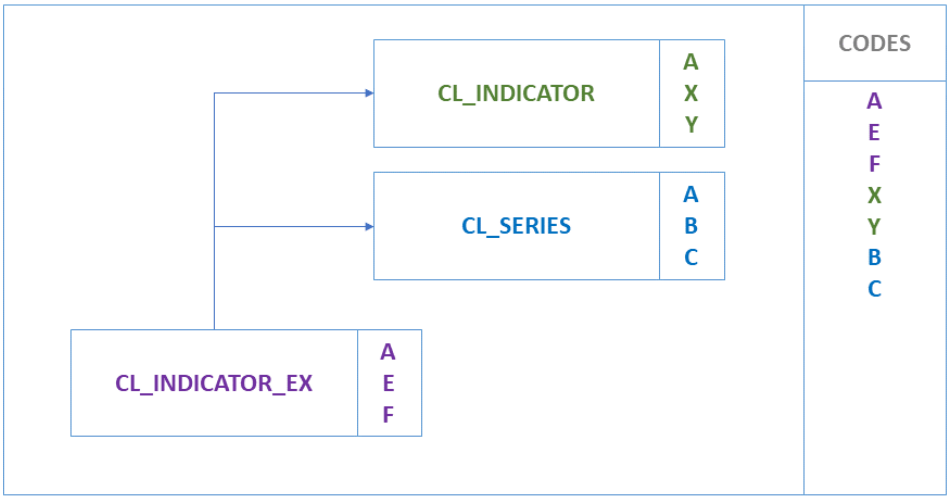


Figure 8. Codelist extension with new Codes

Prefixing Code Ids

A reference to a `Codelist` may contain a prefix. If a prefix is provided, this prefix will be applied to all the codes in the `Codelist` before they are imported into the extended `Codelist`. Following the above example if the `CL_SERIES` reference includes a prefix of `'SER_'` then the resulting `Codelist` would contain 7 codes, `A`, `E`, `F`, `X`, `Y`, `SER_A`, `SER_B`, `SER_C`.

Copyright © SDMx TWG 2020

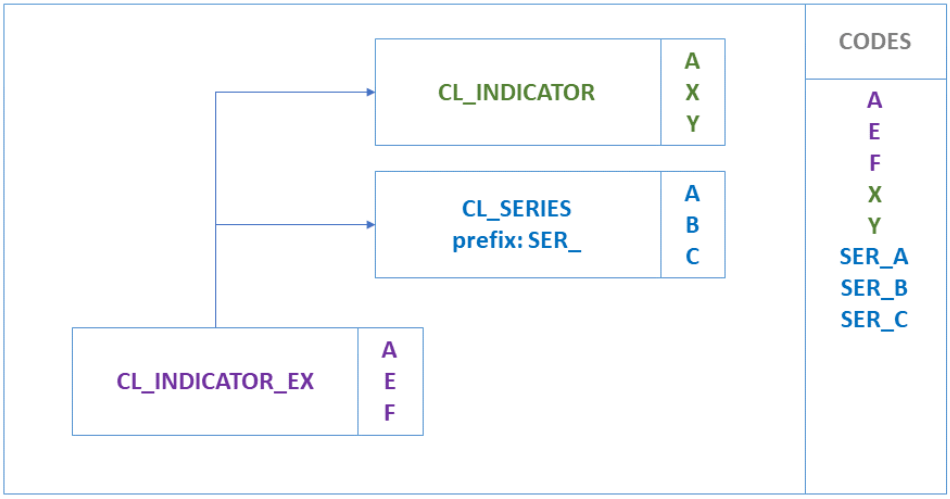


Figure 9. Extended Codelist with prefix

Including / Excluding Specific Codes

The default behaviour of extending another `Codelist` is to import all Codes. However, an explicit list of Code Ids may be provided for explicit inclusion or exclusion. This list of Ids may contain wildcards using the same notation as `Constraints` (`%`). Cascading values is also supported using the same syntax as the `Constraints`. The list of Ids is either a list of excluded items, or included items, exclusion and inclusion is not supported against a single `Codelist`.

Copyright © SDMx TWG 2020

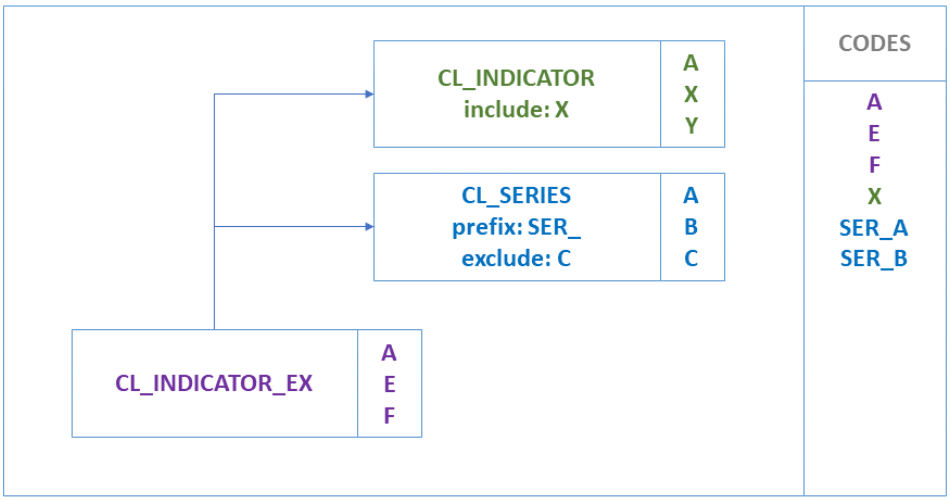


Figure 10. Extended Codelist with include/exclude terms

Parent Ids

Parent `Id`s are preserved in the extended `Codelist` if they can be. If a `Code` is inherited but its parent `Code` is excluded, then the `Code`'s parent `Id` will be removed. This rule is also true if the parent `Code` is excluded because it is overwritten by another `Code` with the same `Id` from another `Codelist`. This ensures the parent `Id`s do not inadvertently link to `Code`s originating from different `Codelist`s, and also prevents circular references from occurring.

Copyright © SDMx TWG 2020

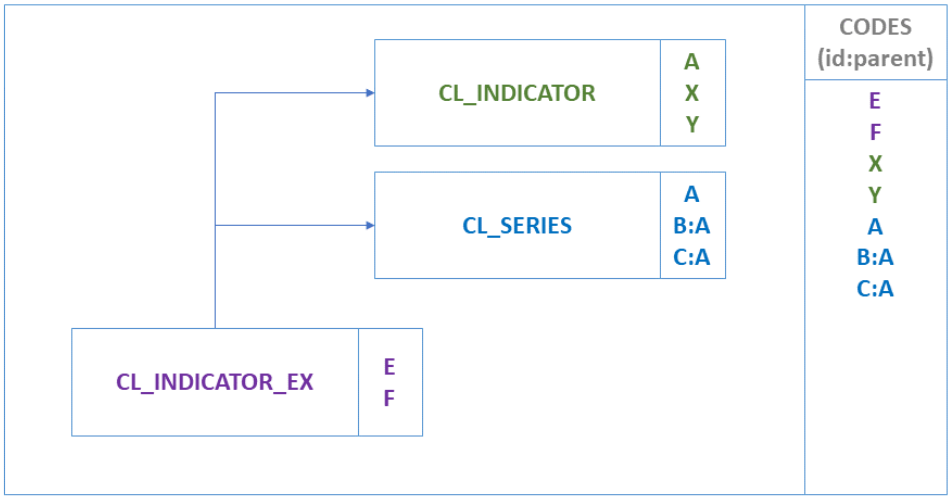


Figure 11. Parent Code included

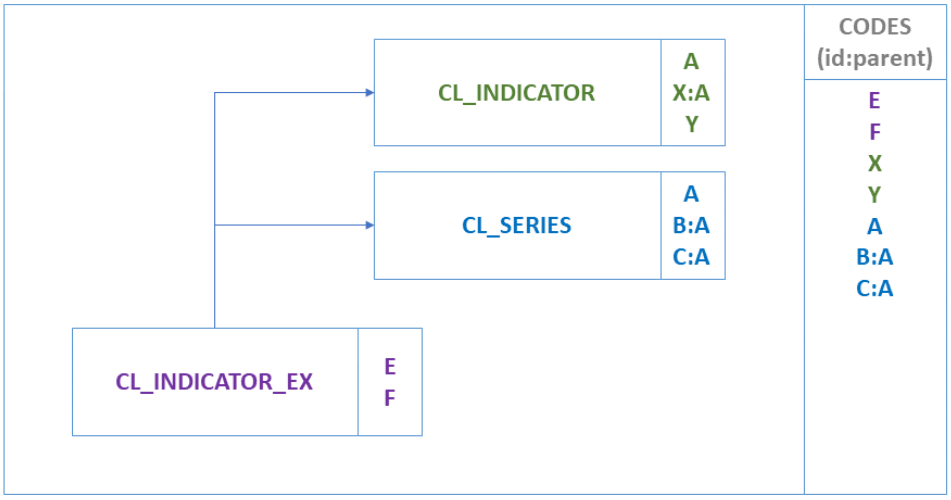


Figure 12. Parent Code from different extended Codelist

Copyright © SDMx TWG 2020

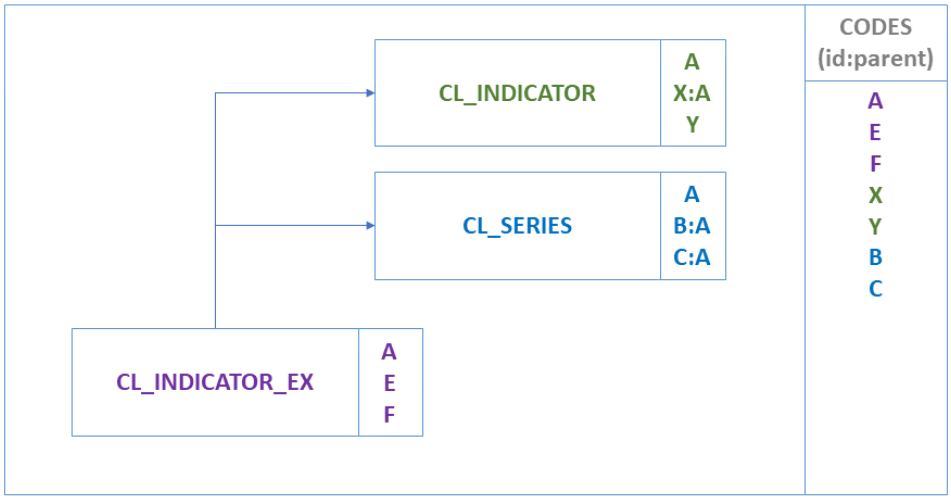


Figure 13. Parent Code overridden by local Code

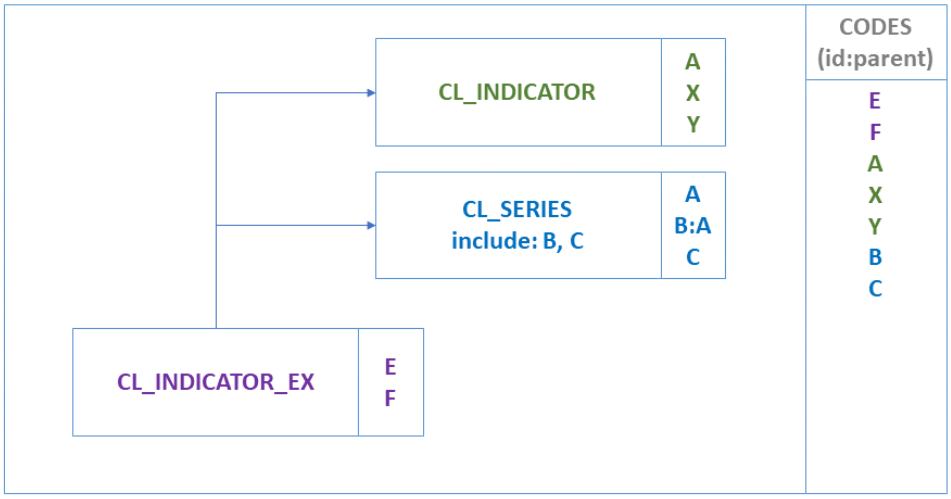


Figure 14. Parent Code not included

Copyright © SDMx TWG 2020

Discriminated Unions

A common use case solved in SDMX 3.0 is that of discriminated unions, i.e., dealing with classification or breakdown “variants” which are all valid but mutually exclusive. For example, there are many versions of the international classification for economic activities `"ISIC"`. In SDMX, classifications are enumerated concepts, normally modelled as dimensions corresponding to breakdowns. Each enumerated concept is associated to one and only one code list.

To support this use case, the following have to be considered:

- **Independent Codelists per variant**: Having each variant in a separate `Codelist` facilitates the maintenance and allows keeping the original codes, even if different versions of the classification have the same code for different concepts. For example, in ISIC Rev. 4 the code `"A"` represents `"Agriculture, forestry and fishing"`, while in ISIC 3.1 `"A"` means `"Agriculture, hunting and forestry"`.
- **Prefixing Code Ids**: When extending Codelists, the reference to an extension `Codelist` may contain a prefix. If a prefix is provided, this prefix will be applied to all the codes in the `Codelist` before they are imported into the extended `Codelist`. In this case, the reference to `CL_ISIC4` includes a prefix of `"ISIC4_"` and the reference to `ISIC3` includes `"ISIC3_"`, so the resulting `Codelist` will have no conflict for the `"A"` items which will become `"ISIC3_A"` and `"ISIC4_A"`.
- **Including / Excluding Specific Codes**: As explained above, there will be independent DFs/PAs with specific `Constraint` attached, in order to keep the proper items according to the variant in use by each data provider.

For example, assuming:

- DSD `DSD_EXDU` contains a Dimension: `ACTIVITY` enumerated by `CL_ACTIVITY`.
- `CL_ACTIVITY` has no items and is extended by:
- `CL_ISIC4`, `prefix="ISIC4_"`
- `CL_ISIC3`, `prefix="ISIC3_"`
- `CL_NACE2`, `prefix="NACE2_"`
- `CL_AGGR`, `prefix="AGGR_"`
- Dataflow `DF1`, with a `DataConstraint` `CC_NACE2`, `CubeRegion` for `ACTIVITY` and `Value="NACE2_%"`
- Dataflow `DF2`, with a `DataConstraint` `CC_ISIC3`, `CubeRegion` for `ACTIVITY` and `Value="ISIC3_%"`
- Dataflow `DF3`, with a `DataConstraint` `CC_ISIC4`, `CubeRegion` for `ACTIVITY` and `Value="ISIC4_%", Value="AGGR_TOTAL", Value="AGGR_Z"`

The discriminated unions are achieved, by requesting any of the above `Dataflow`s with `references="all"` and `detail="referencepartial"`: returns `CL_ACTIVITY` with the corresponding extensions resolved and the `DataConstraint`, referencing the `Dataflow`, applied. Thus, the `CL_ACTIVITY` will only include Codes prefixed according to the Dataflow, i.e.:

- Prefix `"NACE2_%"` for `DF1`;
- Prefix `"ISIC3_%"` for `DF2`;
- Prefix `"ISIC4_%"` for `DF3`; note that Codes `"AGGR_TOTAL"` and `"AGGR_Z"` are also included in this case.

5.6.2 Linking Hierarchies

To facilitate the usage of `Hierarchy` within other SDMX Artefacts, the `HierarchyAssociation` is defined to link any `Hierarchy` with any `IdentifiableArtefact` within a specific context.

The `HierarchyAssociation` is a simple Artefact operating like a Categorisation. The former specifies three references:

- The link to a `Hierarchy`;
- The link to the `IdentifiableArtefact` that the `Hierarchy` is linked (e.g., a Dimension);
- The link to the context that the linking is taking place (e.g., a DSD).

Copyright © SDMx TWG 2020

As an example, let’s assume:

- A DSD with a `COUNTRY` Dimension that uses `Codelist` `CL_AREA` as representation.
- A `Hierarchy` (e.g., `EU_COUNTRIES`) that builds a hierarchy for the `CL_AREA` `Codelist`.
- In order to use this Hierarchy for data of a Dataflow (e.g., `EU_INDICATORS`), we need to build the following `HierarchyAssociation`:
- Links to the Hierarchy `EU_COUNTRIES` (what is associated?)
- Links to the Dimension `COUNTRY` (where is it associated?)
- Links to the context: Dataflow `EU_INDICATORS` (when is it associated?)
- The above are also shown in the schematic below:

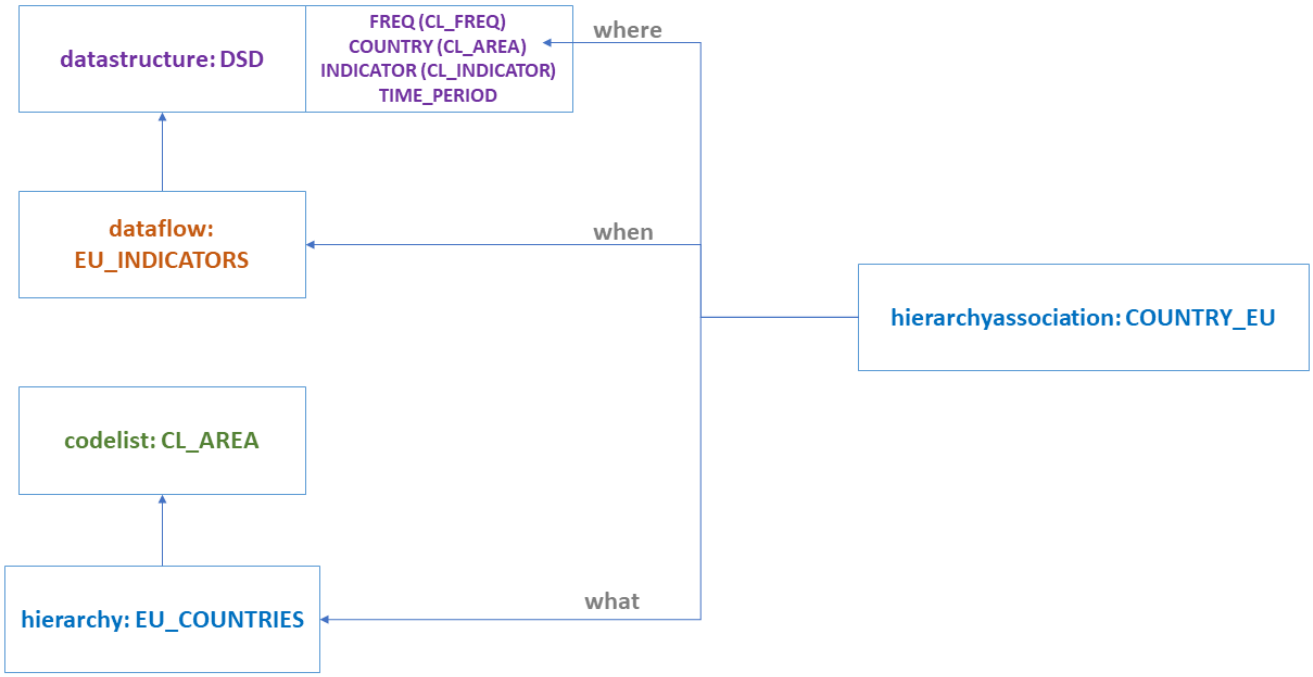


Figure 15. Hierarchy Association

Copyright © SDMx TWG 2020

5.7 Geospatial information support

SDMX recognizes that statistics refers to units or facts sited in places or areas that may be referenced to geodesic coordinates. This section presents the technical specifications to “geo-reference” those objects and facts in SDMX, by establishing ways to make relations to geographic features over the Earth using a defined coordinates system.

SDMX can support three different ways for referencing geospatial data:

1. **Indirect Reference to Geospatial Information.** Including a link to an external file containing the geospatial information. This is the only backwards compatible approach. Since this representation of geospatial information is not included inside the data message, the main use case would be connecting dissemination systems for making use of external tools, like GIS software.
2. **Geographic Coordinates.** Including the coordinates of a specific geospatial feature as a set of coordinates. This is suitable for any statistical information that needs to be georeferenced especially for the exchange of microdata.
3. **A Geographic Codelist.** Includes a type of Codelist, listing predefined geographies that are represented by geospatial information. These geographies could be administrative (including administrative boundaries or enumeration areas), lines, points, or gridded geographies. Regardless, the geospatial information used to represent the geography would contain both dimensions and/or attributes; therefore, representing an advantage for the data modellers as it provides a clear way to identify those dimensions developing a “Geospatial” role.

5.7.1 Indirect Reference to Geospatial Information

This option provides a way to include external references to geospatial information through a file containing it. The external content may be geographical or thematic maps with different levels of precision. All the processing of geospatial data is made through external applications that can interpret the information in different formats.

The reference to the external files containing geospatial information is made using some recommended SDMX Attributes, with the following content:

- `GEO_INFO_TEXT`. A description of the kind of information being referenced.
- `GEO_INFO_URL`. A URL which points to the resource containing the referred geospatial information. The resource might be a file with static geodesic information or a web service providing dynamic construction of geometries.
- `GEO_INFO_TYPE`. Coded information describing a standard format of the file that contains the geospatial information. The format types are taken from the list of Format descriptions for Geospatial Data managed by the US Library of the Congress. Allowed types in SDMX are listed in the **Geographical Formats** code list (`CL_GEO_FORMATS`). Examples of the codes contained in the document are:

Code	Description
GML	Geography Markup Language
GeoTIFF	GeoTIFF
KML_2_2	KML Version 2.2
GEOJSON_1_1	GeoJSON Version 1.1

Depending on the intended use, these attributes may be attached at the dataflow level, the series level or the observation level.

The indirect reference to geospatial information in SDMX is recommended to be used for dissemination purposes, where the statistical information is complemented by geographical representations of places or regions.

5.7.2 Geographic Coordinates

This option to represent geospatial information in SDMX provides an efficient way for including geographic information with different levels of granularity, due to its flexibility. Geospatial information is represented using the `GeospatialInformation` type, as defined in the data types of the SDMX Information Model. A `"GEO_FEATURE_SET"` role should be assigned to any Component of this type.

Copyright © SDMx TWG 2020

The `GeospatialInformation` data type can be assigned to a `Dimension`, `DataAttribute`, `MetadataAttribute` or a `Measure` with the `"GEO_FEATURE_SET"` role assigned; it can be included in a dataset or metadataset.

Any Component used for representing a Geographical Feature Set, i.e., used to describe geographical characteristics, must have a `“GEO_FEATURE_SET”` role. Its Representation would be of `textType="GeospatialInformation"`. The `GeospatialInformation` type is not intended to replace standard geospatial information formats, but instead provide a simple way to describe precise geographical characteristics in SDMX data sets agnostic of any particular geospatial software product or use case.

The `GeospatialInformation` type should be used to describe geographical features like points (e.g., locations of places, landmarks, buildings, etc.), lines (e.g., rivers, roads, streets, etc.), or areas (e.g., geographical regions, countries, islands, land lots, etc.). A mix of different features is possible too, e.g., combining polygons and geographical points to describe a country and the location of its capital.

The components that conform to the structure of the `GeospatialInformation` type are:

- `X_COORDINATE`: The horizontal (longitude) value of a pair of coordinates expressed in the Coordinate Reference System (CRS), mandatory.
- `Y_COORDINATE`: The vertical value (latitude) of a pair of coordinates expressed in the CRS units, mandatory.
- `ALT`: The height (altitude) from the reference surface is expressed in meters, optional.
- `CRS`: The code of the Coordinate Reference System is used to reference the coordinates in the flow, optional.

  The code of the CRS will be as it appears in the EPSG Geodetic Parameter Registry maintained by the International Association of Oil and Gas Producers. If this element is omitted, by default, the CRS will be the World Geodetic System 1984 (WGS 84, EPSG: 4326).
- `PRECISION`: Precision of the coordinates, expressing the possible deviation in meters from the exact geodesic point, optional.

  This component is only allowed if the CRS is specified too. If omitted, it will be interpreted as limited it to the expected measurement accuracy (e. g. a standard GPS has an accuracy of ~ 10m).
- `GEO_DESCRIPTION`: Text for additional information about the place, geographical feature, or set of geographical features, optional.

Geographical features (`GEO_FEATURES`) are collections of geographical features intended to be used to represent geographical areas like countries, regions, etc., or objects, like water bodies (e. g. rivers, lakes, oceans, etc.), roads (streets, highways, etc.), hospitals, schools, and the like. They are represented in the following way:

```
(GEO_FEATURE, GEO_FEATURE): GEO_DESCRIPTION
```

- `GEO_FEATURE` represents a set of points defining a feature following the ISO/IEC 13249-3:2016 standard to conform Well-known Text (WKT) for the representation of geometries in a format defined in the following way:

  ```
  GEOMETRY_TYPE (GEOMETRY_REP)
  ```
- `GEOMETRY_TYPE`: A string with a closed vocabulary defining the type of the geometry that represents a geographical component of the `GEO_FEATURES` collection, mandatory.

  Three types are allowed:

  a. `Point`, a specific geodesic point, like the centroid of a city or a hospital. It is represented with the string `“POINT”`

  b. `Line`, a feature defining a line like a road, a river, or similar. It is represented with the string `“LINESTRING”`

  c. `Area`, a polygon defining a closed area. It is represented with the string `“POLYGON”`

  If the `GEOMETRY_REP` is going to be including the height (`ALT`) component, a `“Z”` must be added after the string qualifying the `GEOMETRY_TYPE`. In this way, we will have: `“POINT Z”`, `“LINESTRING Z”` and `“POLYGON Z”`

  Other feature types (e.g. Triangular irregular networks, “TIN”) are not supported yet directly, except grids that are detailed in 7.3.
- `GEOMETRY_REP`: Representation of each of the types The way to represent each `GEO_FEATURE_TYPE` will be:
  - A point (`POINT`): `“COORDINATES”`
  - A line (`LINESTRING`): `“COORDINATES, COORDINATES, …”`
  - An area (`POLYGON`): `“(COORDINATES, COORDINATES, …), (COORDINATES, COORDINATES, …)”`

Copyright © SDMx TWG 2020

Where:

- `COORDINATES`: Represents an individual set of coordinates composed by the `X_COORDINATE` (`X`), `Y_COORDINATE` (`Y`), and `ALT` (`Z`) in the following way `“X Y Z”` or `“X Y”` defining a single point of the polygon. Altitude is to be reported in meters.

In an expanded way, `GEO_FEATURE` may be represented in the following ways:

```
POINT (X_COORDINATE Y_COORDINATE): GEO_DESCRIPTION
POINT Z (X_COORDINATE Y_COORDINATE ALT): GEO_DESCRIPTION
LINESTRING (X_COORDINATE Y_COORDINATE, X_COORDINATE Y_COORDINATE, …): GEO_DESCRIPTION
LINESTRING Z (X_COORDINATE Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …): GEO_DESCRIPTION
POLYGON  ((X_COORDINATE Y_COORDINATE, X_COORDINATE Y_COORDINATE, …), (X_COORDINATE Y_COORDINATE, X_COORDINATE Y_COORDINATE, …), …): GEO_DESCRIPTION
POLYGON Z ((X_COORDINATE Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …), (X_COORDINATE Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …), …):
GEO_DESCRIPTION
```

An example of how `GEO_FEATURES` may be represented in an expanded way would be:

```
(POLYGON Z (
    (X_COORDINATE Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …),
    (X_COORDINATE Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …), …),
 POLYGON Z (
    (X_COORDINATE Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …),
    (X_COORDINATE Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …), …),
 POLYGON Z (
    (X_COORDINATE Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …),
    (X_COORDINATE Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …), …), …):
    GEO_DESCRIPTION
```

Accordingly to this logic, an example of an expanded expression representing a value of the `GeospatialInformation` may be the following:

```
“CRS, PRECISION: {(POLYGON Z ((X_COORDINATE Y_COORDINATE ALT,
X_COORDINATE Y_COORDINATE ALT, …), (X_COORDINATE Y_COORDINATE ALT,
X_COORDINATE Y_COORDINATE ALT, …), …), POLYGON Z ((X_COORDINATE
Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …), (X_COORDINATE
Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …), …), POLYGON Z
((X_COORDINATE Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …),
(X_COORDINATE Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …),
…), …): GEO _DESCRIPTION}, {(POLYGON Z ((X_COORDINATE Y_COORDINATE
ALT, X_COORDINATE Y_COORDINATE ALT, …), (X_COORDINATE Y_COORDINATE
ALT, X_COORDINATE Y_COORDINATE ALT, …), …), POLYGON Z ((X_COORDINATE
Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …), (X_COORDINATE
Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …), …), POLYGON Z
((X_COORDINATE Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …),
(X_COORDINATE Y_COORDINATE ALT, X_COORDINATE Y_COORDINATE ALT, …),
…), …): GEO _DESCRIPTION}, …: GEO_DESCRIPTION”
```

Validation rules must be added to the XML Schema to ensure the integrity of the specification according to the proposed syntax.

5.7.3 A Geographic Codelist

Geography is represented by geospatial information. Within SDMX, geospatial information is conceptually represented by the `"GEO_FEATURE_SET"` role/specification. This approach uses a specialized form of SDMX Codelist, named `"GeoCodelist"`, which is a Codelist containing the Geography used to demarcate the geographic extent. This is implemented in two ways:

1. `Geographic`. It is a regular codelist that has been extended to add a geographical feature set to each of its items, typically, this would include all types of administrative geographies;
2. `Grid`. As a codelist that has defined a geographical grid composed of cells representing regular squared portions of the Earth.

A `GeoCodelist` is a Codelist as defined in the SDMX Information Model that has the `GeoType` property added. `GeoType` can take one of two values `"Geographic"` or `"GeoGrid"`.

`"Geographic"` corresponds to the first way to implement a `GeoCodelist`. When the `GeoCodelist` includes a `GeoType="Geographic"` property, a `GeoFeatureSet` property is added to each of the items in the code list to implement a Geographic `GeoCodelist`.

Copyright © SDMx TWG 2020

On the other hand, when `GeoType="GeoGrid"` it is defining a gridded `GeoCodelist`, and it is necessary to add a grid definition to the Codelist identifier using the `gridDefinition` property. The components needed to define a geographical grid are the following:

- `CRS`: The code of the Coordinate Reference System is used to reference the coordinates in the flow, optional. The code of the CRS will be as it appears in the EPSG Geodetic Parameter Registry maintained by the International Association of Oil and Gas Producers. If this component is omitted, by default the CRS will be the World Geodetic System 1984 (WGS 84, EPSG:4326).
- `REFERENCE_CORNER`: A code composed of two characters to define the position of the coordinates that will be used as a starting reference to locate the cells. The possible values of this code can be `UL` (Upper Left), `UR` (Upper Right), `LL` (Lower Left), or `LR` (Lower Right). If this component is omitted the value `LL` (Lower Left) will be taken by default. This element is optional.
- `REFERENCE_COORDINATES`: Represents the starting point to reference the cells of the grid, accordingly to the `CRS` and the `REFERENCE_CORNER`. It is represented by an individual set of coordinates composed by the `X_COORDINATE` (`X`) and `Y_COORDINATE` (`Y`) in the following way `"X,Y"`. This element is mandatory if `GEO_STD` is omitted.
- `CELL_WIDTH`: The size in meters of a horizontal side of the cells in the grid. This element is mandatory if `GEO_STD` is omitted.
- `CELL_HEIGHT`: The size in meters of a vertical side of the cells in the grid. This element is mandatory if `GEO_STD` is omitted .
- `GEO_STD`: A restricted text value expressing that the cells in the grid will provide information about matching codes existing in another reference system that establishes a mechanism to define the grid. This element is optional.

  Accepted values for this component are included in the Geographical Grids Codelist (`CL_GEO_GRIDS`). Examples contained in the mentioned document are:

Value	Description
`GEOHASH`	GeoHash
`GEOREF`	World Geographic Reference System
`MGRS`	Military Grid Reference System
`OLC`	Open Location Code / Plus Code
`QTH`	Maidenhead Locator System /QTH Locator / IAURU Locator
`W3W`	What3words™
`WOEID`	Where On Earth Identifier

The `GRID_DEFINITION` element will contain a regular expression string corresponding to the following format:

```
"CRS: REFERENCE_CORNER; REFERENCE_COORDINATES; CELL_WIDTH,
CELL_HEIGHT: GEO_STD"
```

In an expanded way we would have:

```
"CRS:REFERENCE_CORNER; X_COORDINATE, Y_COORDINATE; CELL_WIDTH,
CELL_HEIGHT: GEO_STD"
```

If the grid will be fully adhering to a standard declared in the GEO_STD, the definition of each code in the code list will be optional. In other case, each item in the code list must be assigned to one cell in the grid, which is made by adding the GEO_CELL element to each item of the code list that will contain a regular expression string composed of the following components:

- `GEO_COL`: The number of the column in the grid starting by zero.
- `GEO_ROW`: The number of the row in the grid starting by zero.
- `GEO_TAG`: An optional text to include additional information to the cell.
- `GEO_CELL` will have values with the following format: `"GEO_COL, GEO_ROW: GEO_TAG"`

When using a gridded `GeoCodelist` we may use the `GEO_TAG` to integrate the cells in the grid to the codes used by other standard defined grids. As an example, `GEO_TAG` can take the values of the Open Location Codes, GeoHash, etc. If this is done, the `GEO_STD` component must have been added to the definition of the grid. If the `GEO_STD` is omitted, the `GEO_TAG` contents will be taken just as free text.

Copyright © SDMx TWG 2020

5.8 Maintenance Agencies and Metadata Providers

All structural metadata in SDMX is owned and maintained by a maintenance agency (Agency identified by `agencyID` in the schemas). Similarly, all reference metadata (i.e., `MetadataSets`) is owned and maintained by a metadata provider organisation (`MetadataProvider` identified by `metadataProviderID` in the schemas). It is vital to the integrity of the structural metadata that there are no conflicts in `agencyID` and `metadataProviderID`. In order to achieve this, SDMX adopts the following rules:

1. Agencies are maintained in an `AgencyScheme` (which is a sub class of `OrganisationScheme`); Metadata Providers are maintained in a `MetadataProviderScheme`.
2. The maintenance agency of the Agency/Metadata Provider Scheme must also be declared in a (different) `AgencyScheme`.
3. The “top-level” agency is SDMX and this agency scheme is maintained by SDMX.
4. Agencies registered in the top-level scheme can themselves maintain a single `AgencyScheme` and a single `MetadataProviderScheme`. SDMX is an agency in the SDMX `AgencyScheme`. Agencies in any `AgencyScheme` can themselves maintain a single `AgencyScheme` and so on.
5. The `AgencyScheme` and `MetadataProvideScheme` cannot be versioned and thus have a fixed version set to ‘1.0’.
6. There can be only one `AgencyScheme` maintained by any one Agency. It has a fixed Id of ‘AGENCIES’. Similarly, only one `MetadataProvideScheme` is maintained by one Agency and has a fixed id of `'METADATA_PROVIDERS'`.
7. The format of the agency identifier is `agencyId.agencyID` etc. The top-level agency in this identification mechanism is the agency registered in the SDMX agency scheme. In other words, SDMX is not a part of the hierarchical ID structure for agencies. SDMX is, itself, a maintenance agency.

This supports a hierarchical structure of `agencyID`.

An example is shown below.

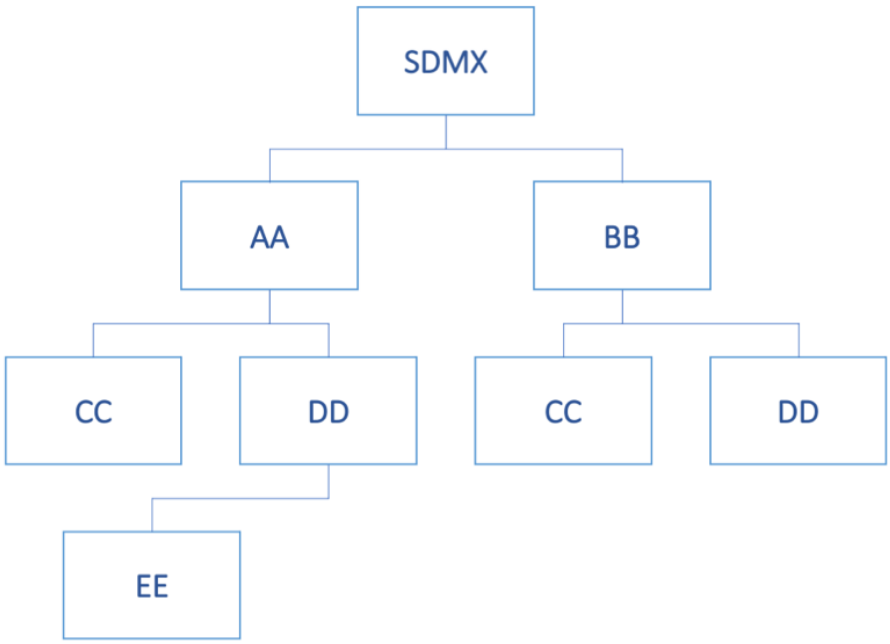


Figure 16. Example of Hierarchic Structure of Agencies

Each agency is identified by its full hierarchy excluding SDMX.

Copyright © SDMx TWG 2020

The XML representing this structure is shown below.

```
<str:AgencySchemes>
  <str:AgencyScheme agencyID="SDMX" id="AGENCIES">
    <com:Name xml:lang="en">Top-level Agency Scheme</com:Name>
    <str:Agency id="AA">
      <com:Name xml:lang="en">AA Name</com:Name>
    </str:Agency>
    <str:Agency id="BB">
      <com:Name xml:lang="en">BB Name</com:Name>
    </str:Agency>
  </str:AgencyScheme>

  <str:AgencyScheme agencyID="AA" id="AGENCIES">
    <com:Name xml:lang="en">AA Agencies</com:Name>
    <str:Agency id="CC">
      <com:Name xml:lang="en">CC Name</com:Name>
    </str:Agency>
    <str:Agency id="DD">
      <com:Name xml:lang="en">DD Name</com:Name>
    </str:Agency>
  </str:AgencyScheme>

  <str:AgencyScheme agencyID="BB" id="AGENCIES">
    <com:Name xml:lang="en">BB Agencies</com:Name>
    <str:Agency id="CC">
      <com:Name xml:lang="en">CC Name</com:Name>
    </str:Agency>
    <str:Agency id="DD">
      <com:Name xml:lang="en">DD Name</com:Name>
    </str:Agency>
  </str:AgencyScheme>

  <str:AgencyScheme agencyID="AA.DD" id="AGENCIES">
    <com:Name xml:lang="en">AA.DD Agencies</com:Name>
    <str:Agency id="EE">
      <com:Name xml:lang="en">EE Name</com:Name>
    </str:Agency>
  </str:AgencyScheme>

</str:AgencySchemes>
```

Figure 17. Example Agency Schemes Showing a Hierarchy

Examples of Structure definitions that show how Agencies are used, are presented below:

```
<str:Codelist agencyID="SDMX" id="CL_FREQ" version="1.0.0"
    urn="urn:sdmx:org.sdmx.infomodel.codelist.Codelist=SDMX:CL_FREQ(1.0.0)">
    <com:Name xml:lang="en">Frequency</com:Name>
  </str:Codelist>
  <str:Codelist agencyID="AA" id="CL_FREQ" version="1.0.0"
    urn="urn:sdmx:org.sdmx.infomodel.codelist.Codelist=AA:CL_FREQ(1.0.0)">
    <com:Name xml:lang="en">Frequency</com:Name>
  </str:Codelist>
  <str:Codelist agencyID="AA.CC" id="CL_FREQ" version="1.0.0"
   urn="urn:sdmx:org.sdmx.infomodel.codelist.Codelist=AA.CC:CL_FREQ(1.0.0)">
    <com:Name xml:lang="en">Frequency</com:Name>
  </str:Codelist>
  <str:Codelist agencyID="BB.CC" id="CL_FREQ" version="1.0.0"
   urn="urn:sdmx:org.sdmx.infomodel.codelist.Codelist=BB.CC:CL_FREQ(1.0.0)">
    <com:Name xml:lang="en">Frequency</com:Name>
  </str:Codelist>
```

Figure 18. Example Showing Use of Agency Identifiers

Each of these maintenance agencies has a Codelist with an identical id `'CL_FREQ'`. However, each is uniquely identified by means of the hierarchic agency structure.

Copyright © SDMx TWG 2020

5.9 Concept Roles

5.9.1 Overview

The DSD Components of Dimension and Attribute can play a specific role in the DSD and it is important to some applications that this role is specified. For instance, the following roles are some examples:

- **Frequency** – in a data set the content of this Component contains information on the frequency of the observation values.
- **Geography** – in a data set the content of this Component contains information on the geographic location of the observation values.

5.9.2 Information Model

The Information Model for this is shown below:

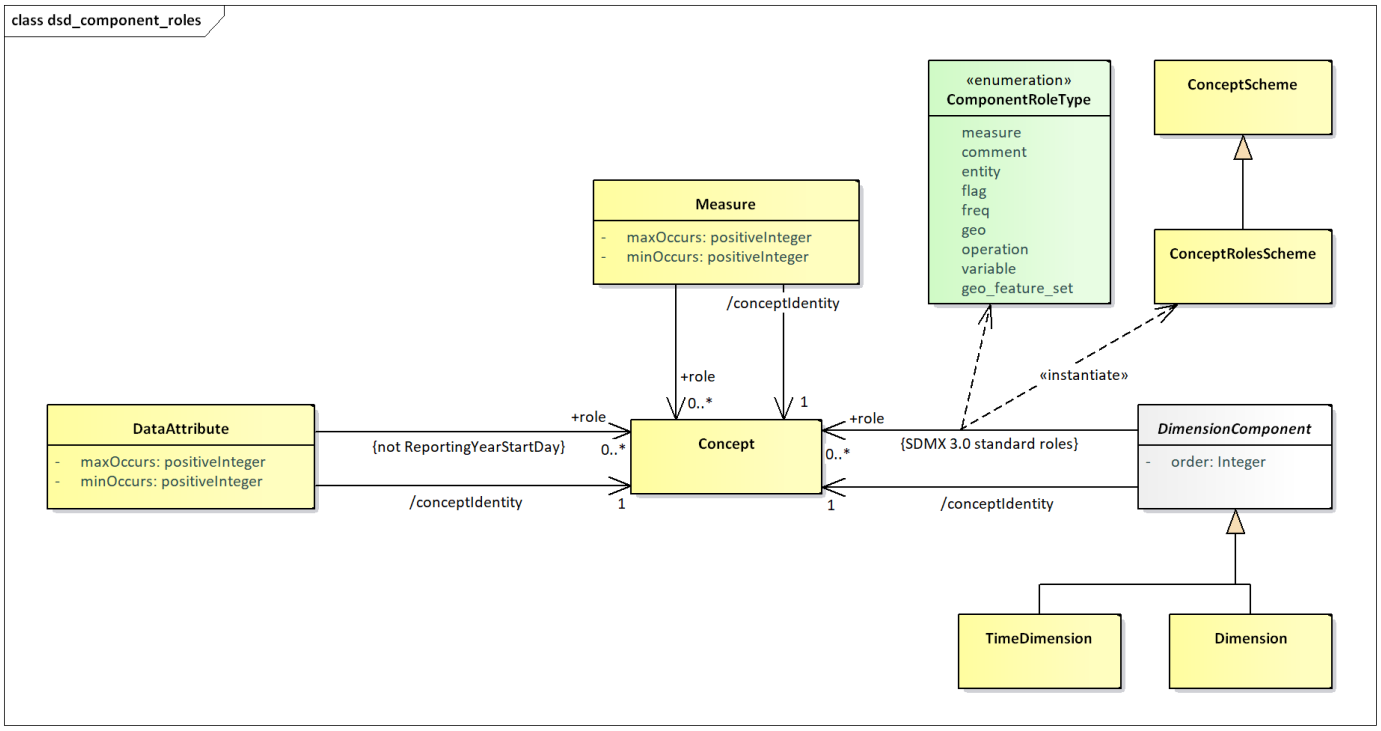


Figure 19. Information Model Extract for Concept Role

It is possible to specify zero or more concept roles for a Dimension, Measure and Data Attribute. The Time Dimension has explicitly defined roles and cannot be further specified with additional concept roles.

Copyright © SDMx TWG 2020

5.9.3 Technical Mechanism

The mechanism for maintain and using concept roles is as follows:

1. A standard Concept Scheme maintained in the Global Registry, with the following identification: `SDMX:CONCEPT_ROLES(1.0.0)`, shall include the default roles, specified by the SDMX SWG (as detailed in 9.5).
2. Any recognized Agency can have a concept scheme that contains concepts that identify concept roles. Indeed, from a technical perspective any agency can have more than one of these schemes, though this is not recommended.
3. The concept scheme that contains the “role” concepts can contain concepts that do not play a role.
4. There is no explicit indication on the Concept whether it is a ‘role’ concept.
5. Therefore, any concept in any concept scheme is capable of being a ‘role’ concept.
6. It is the responsibility of Agencies to ensure their community knows which concepts in which concept schemes play a ‘role’ and the significance and interpretation of this role. In other words, such concepts must be known by applications, there is no technical mechanism that can inform an application on how to process such a ‘role’.
7. If the concept referenced in the Concept Identity in a DSD component (Dimension, Measure Dimension, Attribute) is contained in the concept scheme containing concept roles then the DSD component could play the role implied by the concept, if this is understood by the processing application.
8. If the concept referenced in the Concept Identity in a DSD component (Dimension, Measure Dimension, Attribute) is not contained in the concept scheme containing concept roles, and the DSD component is playing a role, then the concept role is identified by the Concept Role in the schema.

5.9.4 SDMX-ML Examples in a DSD

The standard roles Concept Scheme, is still a normal Concept Scheme, thus it may be used also for the concept identity of a Component, e.g., the `'FREQ'`:

```
<str:Dimension id="FREQ">
  <str:ConceptIdentity>urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=
        SDMX:CONCEPT_ROLES(1.0.0).FREQ</str:ConceptIdentity>
</str:Dimension>
```

Given this is the standard roles Concept Scheme, any application should interpret the above Dimension to have the role of Frequency.

Using a Concept Scheme that is not the standard roles Concept Scheme where it is required to assign a role using the standard roles Concept Scheme. Again, `FREQ` is chosen as the example.

```
<str:Dimension id="FREQ">
  <str:ConceptIdentity>urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=
        SDMX:CONCEPTS(1.0.0).FREQ</str:ConceptIdentity>
  <str:ConceptRole>urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=
        SDMX:CONCEPT_ROLES(1.0.0).FREQ</str:ConceptRole>
</str:Dimension>
```

This explicitly states that this Dimension is playing a role identified by the FREQ concept in the standard roles Concept Scheme. Again, the application must interpret this as a Frequency role.

In other cases where a role from a non-standard roles Concept Scheme is used, then the application has to know how to interpret the provided roles, e.g., like in the case below:

```
<str:Dimension id="FREQ">
  <str:ConceptIdentity>urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=
        SDMX:CONCEPTS(1.0.0).FREQ</str:ConceptIdentity>
  <str:ConceptRole>urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=
        SDMX:MY_CONCEPT_ROLES(1.0.0).FREQ</str:ConceptRole>
</str:Dimension>
```

This is all that is required for interoperability within a community. Having a standard roles Concept Scheme, maintained by the SDMX SWG, allows the SDMX community to have a common understanding of the roles, while also being able to extend the roles in bilateral (or multilateral) agreements, by maintaining their own roles Concept Scheme. This will then ensure there is interoperability between systems that understand the use of these concepts.

Copyright © SDMx TWG 2020

Note that each of the Components (Data Attribute, Measure, Dimension, Time Dimension) has a mandatory identity association (Concept Identity) and if this Concept also identifies the role then it must be interpreted accordingly.

In order for these roles to be extensible and also to enable user communities to maintain community-specific roles, the roles are maintained in a controlled vocabulary which is implemented in SDMX as Concepts in a Concept Scheme. The Component optionally references this Concept if it is required to declare the role explicitly. Note that a Component can play more than one role and therefore multiple “role” concepts can be referenced.

5.9.5 SDMX standard roles Concept Scheme

As of SDMX 3.0, there is a predefined Concept Scheme, with a set of Concepts that are considered the standard roles for SDMX. Beyond that, a user is free to add other roles, using custom Concept Schemes. This predefined Concept Scheme is the result of the SWG guidelines for Concept Roles, plus that for Measure, and includes the following Concepts:

COMMENT	Comment	Descriptive text which can be attached to data or metadata.
`ENTITY`	Entity	Describes the subject of the data set (e.g., a country).
`FLAG`	Flag	Coded attribute in a data set that represents qualitative information for the cell or partial key (e.g. series) value.
`FREQ`	Frequency	Time interval at which the source data are collected.
`GEO`	Geographical	Geographic area to which the measured statistical phenomenon relates.
`OPERATION`	Statistical operation	Signifies statistical operations have been done on the observations.
`VARIABLE`	Variable	Characteristic of a unit being observed that may assume more than one of a set of values to which a numerical measure or a category from a classification can be assigned.
`MEASURE`	Measure	Used for emulating the functionality of the deprecated MeasureDimension.
`GEO_FEATURE_SET`	Geographical Feature Set	Georeferencing information to describe the location or the shape of a statistical unit, recognizable object or geographical area.
`PRIMARY`	Primary Measure	Used for backwards compatibility with SDMX 2.1 and back, or when the “Primary Measure” concept is needed.

Copyright © SDMx TWG 2020

5.10 Constraints

5.10.1 Introduction

A Constraint is a Maintainable Artefact that can be associated to one or more of:

- Data Structure Definition
- Metadata Structure Definition
- Dataflow
- Metadataflow
- Provision Agreement
- Metadata Provision Agreement
- Data Provider or Metadata Provider (this is restricted to a Release Calendar Constraint)
- Simple or Queryable Data Sources
- Dataset
- Metadataset

Note that regardless of the Artefact to which the Constraint is associated, it is constraining the contents of code lists in the DSD to which the constrained object is related. This does not apply, of course, to a Metadata/Data Provider as the latter can be associated, via the (Metadata) Provision Agreement, to many MSDs/DSDs. Hence the reason for the restriction on the type of Constraint that can be attached to a Metadata/Data Provider.

5.10.2 Types of Constraint

The Constraint can be of one of two types:

- Data constraint
- Metadata constraint

The Data Constraint may serve two different perspectives, depending on the way the latter is retrieved. These are:

- Allowed constraint
- Actual constraint

The former (allowed – also valid for Metadata Constraint) is specified by a data or metadata provider or consumer for sharing the allowed data and metadata in the context of their DSD or MSD exchanges, e.g., only Monthly data for a specific Dataflow. The latter (actual) is a dynamic Constraint in response to an availability request (only possible for data).

For Actual Data Constraints, there a few characteristics that are worth noting:

- They can only be retrieved by the availability requests (as specified in the REST API).
- They depend on the data available in an SDMX Web Service and thus they can only be dynamically generated according to that data.
- Although they are Maintainable Artefacts, they cannot change independently of data; thus, they cannot be versioned (they are non-versioned, as explained in section 14).
- Their identifier may also be dynamically generated and thus there is no REST resource based on their identification.

5.10.3 Rules for a Constraint

Scope of a Constraint

A Constraint is used specify the content of a data or metadata source in terms of the component values or the keys.

Copyright © SDMx TWG 2020

In terms of data the components are:

- Dimension
- Time Dimension
- Data Attribute
- Measure
- Metadata Attribute
- DataKeySets: the keys are the content of the KeyDescriptor – i.e., the series keys composed, for each key, by a value for each Dimension.

In terms of reference metadata the components are:

- Metadata Attribute

For a Constraint based on a DSD the Constraint can reference one or more of:

- Data Structure Definition
- Dataflow
- Provision Agreement
- Data Provider

For a Constraint based on an MSD the Constraint can reference one or more of:

- Metadata Structure Definition
- Metadataflow
- Metadata Provision Agreement
- Metadata Provider
- Metadata Set

Furthermore, there can be more than one Constraint specified for a specific object e.g., more than one Constraint for a specific DSD.

In view of the flexibility of constraints attachment, clear rules on their usage are required. These are elaborated below.

Multiple Constraints

There can be many Constraints for any Constrainable Artefact (e.g., DSD), subject to the following restrictions:

CUBE REGION

A Constraint can contain multiple Member Selections (e.g., Dimensions).

- A specific Member Selection (e.g., Dimension `FREQ`) can only be contained in one Cube Region for any one attached object (e.g., a specific DSD or specific Dataflow).
- Component values within a Member Selection may define a validity period. Otherwise, the value is valid for the whole validity of the Cube Region.
- For partial reference resolution purposes (as per the SDMX REST API), the latest non-draft Constraint must be considered.
- A Member Selection may include wildcarding of values (using character `'%'` to represent zero or more occurrences of any character), as well as cascading through hierarchic structures (e.g., parents in Codelist), or localised values (e.g., text for English only). Lack of locale means any language may match. Cascading values are mutual exclusive to localised values, as the former refer to coded values, while the latter refer to uncoded values.
- Any values included in a Member Selection for Components with an array data type (i.e., Measures, Attributes or Metadata Attributes), will be applied as single values and will not be assessed combined with other values to match all possible array values. For example, including the Code `'A'` for an Attribute will allow any instance of the Attribute that includes `'A'`, like `['A', 'B']` or `['A', 'C', 'D']`. Similarly, if Code `'A'` was excluded, all those arrays of values would also be excluded.

Copyright © SDMx TWG 2020

KEY SET

Key Sets will be processed in the order they appear in the Constraint and wildcards can be used (e.g., any key position not reference explicitly is deemed to be “all values”).

As the Key Sets can be “included” or “excluded” it is recommended that Key Sets with wildcards are declared before KeySets with specific series keys. This will minimize the risk that keys are inadvertently included or excluded.

In addition, Attribute, Measure and Metadata Attribute constraints may accompany KeySets, in order to specify the allowed values per Key. Those are expressed following the rules for Cube Regions, as explained above.

Finally, a validity period may be specified per Key.

Inheritance of a Constraint

ATTACHMENT LEVELS OF A CONSTRAINT

There are three levels of constraint attachment for which these inheritance rules apply:

- DSD/MSD – top level
    - Dataflow/Metadataflow – second level
        - Provision Agreement – third level

Note that these rules do not apply to the Simple Datasource or Queryable Datasource; the Constraint(s) attached to these artefacts are resolved for this artefact only and do not take into account Constraints attached to other artefacts (e.g., Provision Agreement, Dataflow, DSD).

It is not necessary for a Constraint to be attached to a higher level artefact. e.g., it is valid to have a Constraint for a Provision Agreement where there are no constraints attached the relevant dataflow or DSD.

CASCADE RULES FOR PROCESSING CONSTRAINTS

The processing of the constraints on either Dataflow/Metadataflow or Provision Agreement must take into account the constraints declared at higher levels. The rules for the lower-level constraints (attached to Dataflow/ Metadataflow and Provision Agreement) are detailed below.

Note that there can be a situation where a constraint is specified at a lower level before a constraint is specified at a higher level. Therefore, it is possible that a higher-level constraint makes a lower-level constraint invalid. SDMX makes no rules on how such a conflict should be handled when processing the constraint for attachment. However, the cascade rules on evaluating constraints for usage are clear – the higher-level constraint takes precedence in any conflicts that result in a less restrictive specification at the lower level.

CUBE REGION

It is not necessary to have a Constraint on the higher-level artefact (e.g., DSD referenced by the Dataflow), but if there is such a Constraint at the higher level(s) then:

- The lower-level Constraint cannot be less restrictive than the Constraint specified for the same Member Selection (e.g. Dimension) at the next higher level, which constrains that Member Selection. For example, if the Dimension `FREQ` is constrained to `A, Q` in a DSD, then the Constraint at the Dataflow or Provision Agreement cannot be `A, Q, M` or even just `M` – it can only further constrain `A, Q`.
- The Constraint at the lower level for any one Member Selection further constrains the content for the same Member Selection at the higher level(s).
- Any Member Selection, which is not referenced in a Constraint, is deemed to be constrained according to the Constraint specified at the next higher level which constraints that Member Selection.
- If there is a conflict when resolving the Constraint in terms of a lower-level Constraint being less restrictive than a higher-level Constraint, then the Constraint at the higher-level is used.

Note that it is possible for a Constraint at a higher level to constrain, say, four Dimensions in a single Constraint, and a Constraint at a lower level to constrain the same four in two, three, or four Constraints.

Copyright © SDMx TWG 2020

KEY SET

It is not necessary to have a Constraint on the higher-level artefact (e.g., DSD referenced by the Dataflow), but if there is such a Constraint at the higher level(s) then:

- The lower-level Constraint cannot be less restrictive than the Constraint specified at the higher level.
- The Constraint at the lower level for any one Member Selection further constrains the keys specified at the higher level(s).
- Any Member Selection, which is not referenced in a Constraint, is deemed to be constrained according to the Constraint specified at the next higher level which constraints that Member Selection.
- If there is a conflict when resolving the keys in the Constraint at two levels, in terms of a lower-level constraint being less restrictive than a higher-level Constraint, then the offending keys specified at the lower level are not deemed part of the Constraint.

Note that a Key in a Key Set can have wildcarded Components. For instance, the Constraint may simply constrain the Dimension `FREQ` to `"A"`, and all keys where the `FREQ="A"` are therefore valid.

The following logic explains how the inheritance mechanism works. Note that this is conceptual logic and actual systems may differ in the way this is implemented.

1. Determine all possible keys that are valid at the higher level.
2. These keys are deemed to be inherited by the lower-level constrained object, subject to the Constraints specified at the lower level.
3. Determine all possible keys that are possible using the Constraints specified at the lower level.
4. At the lower level inherit all keys that match with the higher-level Constraint.
5. If there are keys in the lower-level Constraint that are not inherited then the key is invalid (i.e., it is less restrictive).

Constraints Examples

DATA CONSTRAINT AND CASCADING

The following scenario is used.

A DSD contains the following Dimensions:

- `GEO` – Geography
- `SEX` – Sex
- `AGE` – Age
- `CAS` – Current Activity Status

In the DSD, common code lists are used and the requirement is to restrict these at various levels to specify the actual code that are valid for the object to which the Constraint is attached.

Copyright © SDMx TWG 2020

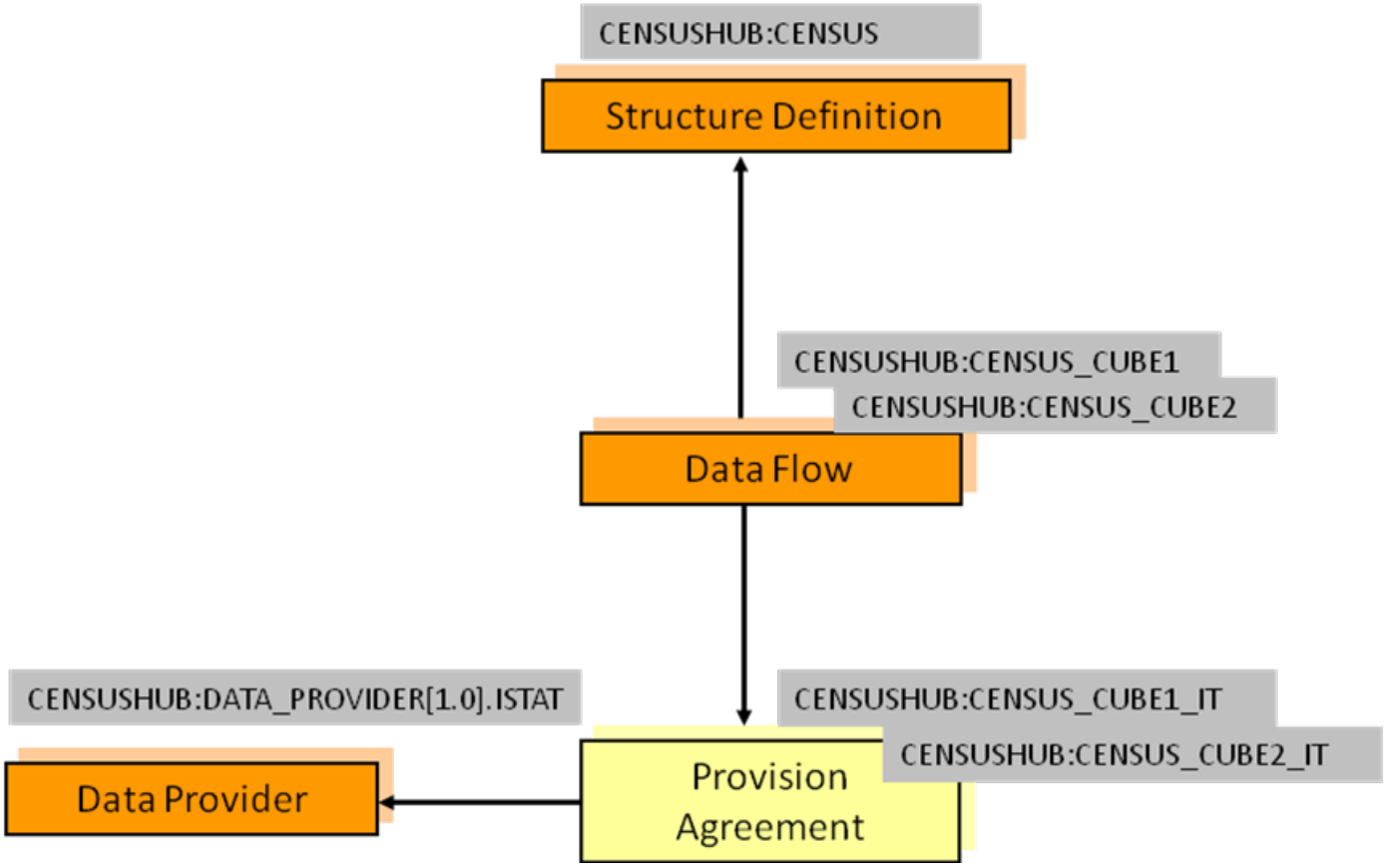


Figure 20. Example Scenario for Constraints

Constraints are declared as follows:

Copyright © SDMx TWG 2020

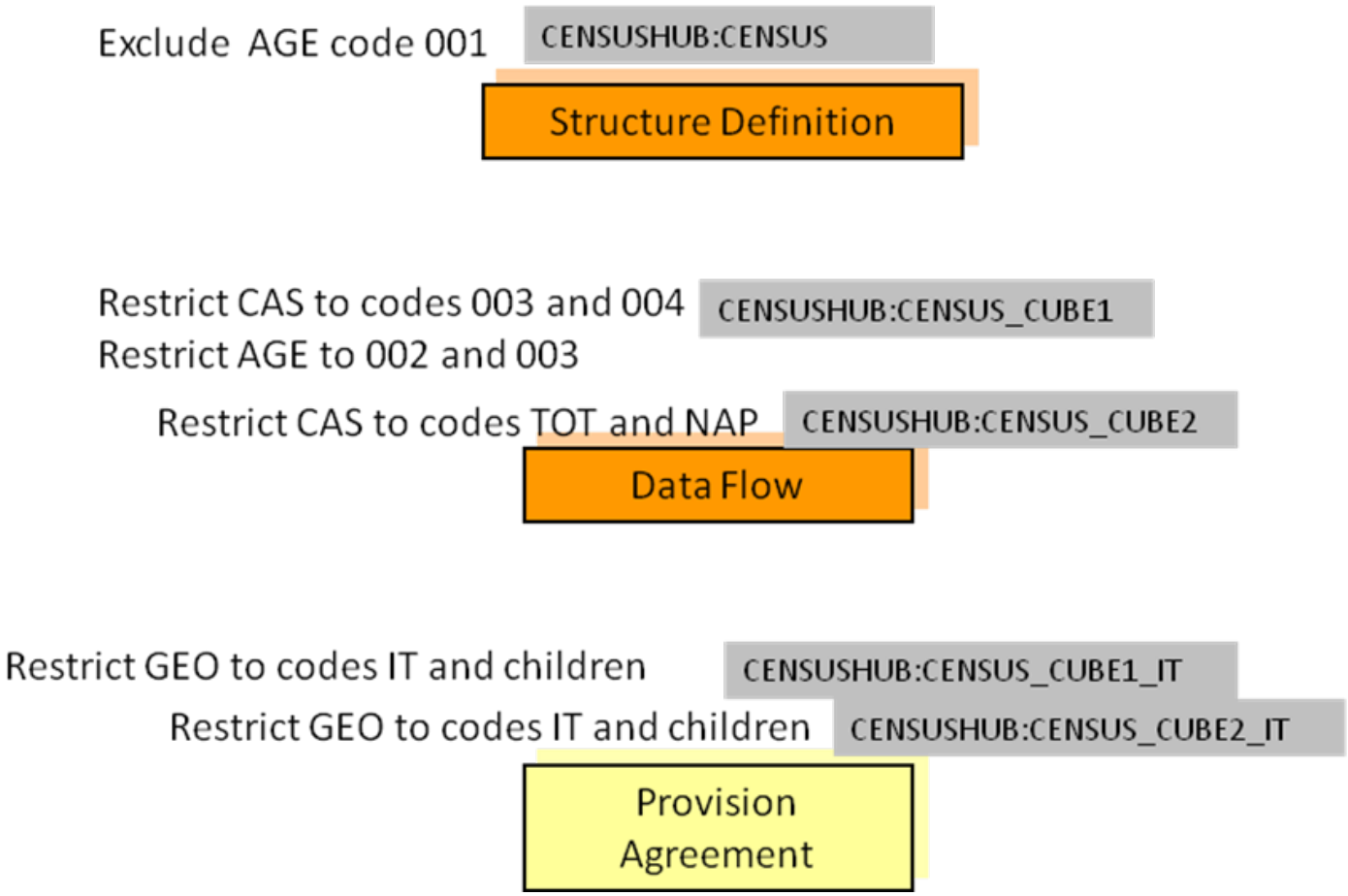


Figure 21. Example Constraints

Notes:

`AGE` is constrained for the DSD and is further restricted for the Dataflow `CENSUS_CUBE1`.

- The same Constraint applies to both Provision Agreements.

The cascade rules elaborated above result as follows:

DSD

- Constrained by eliminating code `001` from the code list for the `AGE` Dimension.

Dataflow `CENSUS_CUBE1`

- Constrained by restricting the code list for the `AGE` Dimension to codes `002` and `003` (note that this is a more restrictive constraint than that declared for the DSD which specifies all codes except code 001).
    - Restricts the `CAS` codes to `003` and `004`.

Dataflow `CENSUS_CUBE2`

- Restricts the code list for the `CAS` Dimension to codes `TOT` and `NAP`.
    - Inherits the `AGE` constraint applied at the level of the DSD.

Provision Agreement `CENSUS_CUBE1_IT`

- Restricts the codes for the `GEO` Dimension to `IT` and its children.
    - Inherits the constraints from Dataflow `CENSUS_CUBE1` for the `AGE` and `CAS` Dimensions.

Copyright © SDMx TWG 2020

Provision Agreement `CENSUS_CUBE2_IT`

- Restricts the codes for the `GEO` Dimension to `IT` and its children.
    - Inherits the constraints from Dataflow `CENSUS_CUBE2` for the `CAS` Dimension.
    - Inherits the `AGE` constraint applied at the level of the DSD.

The Constraints are defined as follows:

DSD Constraint:

```
<str:DataConstraint agencyID="SDMX" id="DATA_CONSTRAINT" version="1.0.0-draft" type="Allowed">
  <com:Name xml:lang="en">SDMX 3.0 Data Constraint sample</com:Name>
  <str:ConstraintAttachment>
    <str:DataStructure>urn:sdmx:org.sdmx.infomodel.datastructure.DataStructure=CENSUSHUB:CENSUS(1.0.0)</str:DataStructure>
  </str:ConstraintAttachment>
  <str:CubeRegion include="true">
    <!-- the ability to exclude values is illustrated – i.e., all values valid except this one -->
    <com:KeyValue id="AGE" include="false">
      <com:Value>001</com:Value>
    </com:KeyValue>
  </str:CubeRegion>
</str:DataConstraint>
```

Dataflow Constraints:

```
<str:DataConstraint agencyID="SDMX" id="DATA_CONSTRAINT_2" version="1.0.0-draft" type="Allowed">
  <com:Name xml:lang="en">SDMX 3.0 Data Constraint sample</com:Name>
  <str:ConstraintAttachment>
    <str:Dataflow>urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=
         CENSUSHUB:CENSUS_CUBE1(1.0.0)</str:Dataflow>
  </str:ConstraintAttachment>
  <str:CubeRegion include="true">
    <com:KeyValue id="AGE" include="true">
      <com:Value>002</com:Value>
      <com:Value>003</com:Value>
    </com:KeyValue>
    <com:KeyValue id="CAS">
      <com:Value>003</com:Value>
      <com:Value>004</com:Value>
    </com:KeyValue>
  </str:CubeRegion>
</str:DataConstraint>

<str:DataConstraint agencyID="SDMX" id="DATA_CONSTRAINT_3" version="1.0.0-draft" type="Allowed">
  <com:Name xml:lang="en">SDMX 3.0 Data Constraint sample</com:Name>
  <str:ConstraintAttachment>
    <str:Dataflow>urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=
         CENSUSHUB:CENSUS_CUBE2(1.0.0)</str:Dataflow>
  </str:ConstraintAttachment>
  <str:CubeRegion include="true">
    <com:KeyValue id="CAS" include="true">
      <com:Value>TOT</com:Value>
      <com:Value>NAP</com:Value>
    </com:KeyValue>
  </str:CubeRegion>
</str:DataConstraint>
```

Provision Agreement Constraint:

```
<str:DataConstraint agencyID="SDMX" id="DATA_CONSTRAINT_4" version="1.0.0-draft" type="Allowed">
  <com:Name xml:lang="en">SDMX 3.0 Data Constraint sample</com:Name>
  <str:ConstraintAttachment>
    <str:ProvisionAgreement>urn:sdmx:org.sdmx.infomodel.registry.
      ProvisionAgreement=CENSUSHUB:CENSUS_CUBE1_IT(1.0.0)
    </str:ProvisionAgreement>
    <str:ProvisionAgreement>urn:sdmx:org.sdmx.infomodel.registry.
      ProvisionAgreement=CENSUSHUB:CENSUS_CUBE2_IT(1.0.0)
    </str:ProvisionAgreement>
  </str:ConstraintAttachment>
  <str:CubeRegion include="true">
    <com:KeyValue id="GEO" include="true">
      <com:Value cascadeValues="true">IT</com:Value>
    </com:KeyValue>
  </str:CubeRegion>
</str:DataConstraint
```

COMBINATION OF CONSTRAINTS

The possible combination of constraining terms are explained in this section, following a few examples.

Copyright © SDMx TWG 2020

Let’s assume a DSD with the following Components:

Type	Component
Dimension	`FREQ`
Dimension	`JD_TYPE`
Dimension	`JD_CATEGORY`
Dimension	`VIS_CTY`
TimeDimension	`TIME_PERIOD`
Attribute	`OBS_STATUS`
Attribute	`UNIT`
Attribute	`COMMENT`
MetadataAttribute	`CONTACT`
Measure	`MULTISELECT`
Measure	`CHOICE`

On the above, let’s assume the following use cases with their constraining requirements:

Use Case 1: A Constraint on allowed values for some Dimensions

- `R1` : Allow monthly and quarterly data
- `R2` : Allow Mexico for vis-à-vis country

This is expressed with the following `CubeRegion` :

Dimension	Values
`FREQ`	`M, Q`
`VIS_CTY`	`MX`

Use Case 2: A Constraint on allowed combinations for some Dimensions

- `R1` : Allow monthly data for Germany
- `R2` : Allow quarterly data for Mexico

This is expressed with the following `DataKeySet` :

Key	Dimension	Values
`Key1`	`FREQ`	`M`
`Key1`	`VIS_CTY`	`DE`
`Key2`	`FREQ`	`Q`
`Key2`	`VIS_CTY`	`MX`

Use Case 3: A Constraint on allowed values for some Dimensions combined with allowed values for some Attributes

- `R1` : Allow monthly and quarterly data
- `R2` : Allow Mexico for vis-à-vis country
- `R3` : Allow present for status

Copyright © SDMx TWG 2020

This may be expressed with the following `CubeRegion`:

Dimensions	Values
FREQ	M, Q
VIS_CTY	MX
OBS_STATUS	A

Use Case 4: A Constraint on allowed combinations for some Dimensions combined with specific Attribute values

- `R1`: Allow monthly data, for Germany, with unit euro
- `R2`: Allow quarterly data, for Mexico, with unit usd

This is may be expressed with the following `DataKeySet`:

Key	Dimension	Values
Key1	FREQ	M
Key1	VIS_CTY	DE
Key1	UNIT	EUR
Key2	FREQ	Q
Key2	VIS_CTY	MX
Key2	UNIT	USD

Use Case 5: A Constraint on allowed values for some Dimensions together with some combination of Dimension values

- `R1`: For annually and quarterly data, for Mexico and Germany, only `A` status is allowed
- `R2`: For monthly data, for Mexico and Germany, only `F` status is allowed

Considering the above examples, the following `CubeRegions` would be created:

CubeRegion	Dimension	Values
CubeRegion1	FREQ	Q, A
CubeRegion1	VIS_CTY	MX, DE
CubeRegion1	OBS_STATUS	A
CubeRegion2	FREQ	M
CubeRegion2	VIS_CTY	MX, DE
CubeRegion2	OBS_STATUS	F

The problem with this approach is that according to the business rule for `Constraints`, only one should be specified per `Component`. Thus, if a software would perform some conflict resolution would end up with empty sets for `FREQ` and `OBS_STATUS` (as they do not share any values).

Nevertheless, there is a much easier approach to that; this is the cascading mechanism of Constraints (as shown in Data Constraint and Cascading). Hence, these rules would be expressed into two levels of Constraints, e.g., DSD and Dataflows:

Copyright © SDMx TWG 2020

DSD `CubeRegion`:

Dimension	Value
FREQ	M, Q, A
VIS_CTY	MX, DE
OBS_STATUS	A, F

Dataflow1 `CubeRegion`:

Dimension	Value
FREQ	Q, A
VIS_CTY	MX, DE
OBS_STATUS	F

Dataflow2 `CubeRegion`:

Dimension	Value
FREQ	M
VIS_CTY	MX, DE
OBS_STATUS	A

Use case 6: A Constraint on allowed values for some Dimensions combined with allowed values for Measures

- `R1`: Allow monthly data, for Germany, with unit euro, and measure choice is 'A'
- `R2`: Allow quarterly data, for Mexico, with unit usd, and measure choice is 'B'

This is may be expressed with the following `DataKeySet`:

Key	Dimension	Value
Key1	FREQ	M
Key1	VIS_CTY	DE
Key1	UNIT	EUR
Key1	CHOICE	A
Key2	FREQ	Q
Key2	VIS_CTY	MX
Key2	UNIT	USD
Key2	CHOICE	B

Use Case 7: A Constraint with wildcards for Codes and removePrefix property

For this example, we assume that the `VIS_CTY` representation has been prefixed with prefix `'AREA_'`. In this Constraint, we need to remove the prefix.

- `R1`: Allow monthly and quarterly data
- `R2`: Allow vis-à-vis countries that start with `M`
- `R3`: Remove the prefix `'AREA_'`

Copyright © SDMx TWG 2020

This may be expressed with the following `CubeRegion`:

Dimension	Value
`FREQ`	`M, Q`
`VIS_CTY (removePrefix='AREA_')`	`M%`

Use Case 8: A Constraint with multilingual support on Attributes

- R1: Allow monthly and quarterly data
- R2: Allow Mexico for vis-à-vis country
- R3: Allow a comment, in English, which includes the term adjusted for status

This may be expressed with the following `CubeRegion`:

Dimension	Value
`FREQ`	`M, Q`
`VIS_CTY`	`MX`
`COMMENT (lang='en')`	`%adjusted%`

Use Case 9: A Constraint on allowed values for Dimensions combined with allowed values for Metadata Attributes

- `R1`: Allow monthly and quarterly data
- `R2`: Allow Mexico for vis-à-vis country
- `R3`: Allow John Doe for contact

This may be expressed with the following `CubeRegion`:

Dimension	Value
`FREQ`	`M, Q`
`VIS_CTY`	`MX`
`CONTACT`	`John Doe`

OTHER CONSTRAINING TERMS

Beyond the cube regions and keysets, there is one more constraining term, i.e., the `ReleaseCalendar`.

The `ReleaseCalendar` is the only term that does not apply on Components; it specifies the schedule of publication or reporting of the dataset or metadataset.

For example, the `ReleaseCalendar` for `Provider` `BIS`, is specified in the three following terms:

- Periodicity: how often data should be reported, e.g., monthly
- Offset: the number of days between the 1st of January and the first release of data, e.g., 10 days
- Tolerance: the maximum allowed of days that data may be considered, without being considered as late, e.g., 5 days

With the above terms, `BIS` would need to report data between the 10th and 15th of every month.

> **Note**
>
> The SDMX 2.1 constraining term `ReferencePeriod` has been deprecated in SDMX 3.0; thus, the `TimeDimension` and any `Dimension` with a time `Representation` can be constrained within a `CubeRegion` or `MetadataTargetRegion`, using the `TimeRangeValue`.

Copyright © SDMx TWG 2020

5.11 Transforming between versions of SDMX

5.11.1 Scope

The scope of this section is to define both best practices and mandatory behaviour for specific aspects of transformation between different versions of SDMX.

5.11.2 Compatibility and new DSD features

The following table provides an overview of the backwards compatibility between SDMX 3.0 and 2.1.

SDMX 3.0 feature	SDMX 2.1 compatibility	Comments
Multiple Measures	Create a Measure Dimension Or Model Measures as Attributes	For Measure Dimensions, all Concepts must reside in the same Concept Scheme
Arrays for values	Cannot be supported	Arrays are always reported in a verbose format, even if one value is reported
Measure Relationship	Can be ignored, as it does not affect dataset format	
Metadata Attributes	Can be created as Data Attributes	Not for extended facets
Multilingual Components	Cannot be supported	
No Measure	Can only be emulated by ignoring the Primary Measure value	
Use extended Codelist	A new Codelist with all Codes must be created	
Sentinel values	Cannot be supported in the DSD	Rules may be supported outside the DSD, in bilateral agreements

The following table illustrates forward compatibility issues from SDMX 2.1 to 3.0.

SDMX 2.1 feature	SDMX 3.0 compatibility	Comments
Measure Dimension	Create a Dimension with role `'MEASURE'` Or Create multiple Measures from the Measure Dimension Concept Scheme	If the dataset has to resemble that of SDMX 2.1 Structure Specific, then the first option must be used
Primary Measure	Create one Measure with role `'PRIMARY'`; use `id="OBS_VALUE"`	

Copyright © SDMx TWG 2020

5.12 Validation and Transformation Language (VTL)

5.12.1 Introduction

The Validation and Transformation Language (VTL) supports the definition of Transformations, which are algorithms to calculate new data starting from already existing ones[1]. The purpose of the VTL in the SDMX context is to enable the:

- definition of validation and transformation algorithms, in order to specify how to calculate new data from existing ones;
- exchange of the definition of VTL algorithms, also together the definition of the data structures of the involved data (for example, exchange the data structures of a reporting framework together with the validation rules to be applied, exchange the input and output data structures of a calculation task together with the VTL Transformations describing the calculation algorithms);
- compilation and execution of VTL algorithms, either interpreting the VTL Transformations or translating them in whatever other computer language is deemed as appropriate.

It is important to note that the VTL has its own information model (IM), derived from the Generic Statistical Information Model (GSIM) and described in the VTL User Guide. The VTL IM is designed to be compatible with more standards, like SDMX, DDI (Data Documentation Initiative) and GSIM, and includes the model artefacts that can be manipulated (inputs and/or outputs of Transformations, e.g. “Data Set”, “Data Structure”) and the model artefacts that allow the definition of the transformation algorithms (e.g. “Transformation”, “Transformation Scheme”).

The VTL language can be applied to SDMX artefacts by mapping the SDMX IM model artefacts to the model artefacts that VTL can manipulate[2]. Thus, the SDMX artefacts can be used in VTL as inputs and/or outputs of Transformations. It is important to be aware that the artefacts do not always have the same names in the SDMX and VTL IMs, nor do they always have the same meaning. The more evident example is given by the SDMX Dataset and the VTL “Data Set”, which do not correspond one another: as a matter of fact, the VTL “Data Set” maps to the SDMX “`Dataflow`”, while the SDMX “`Dataset`” has no explicit mapping to VTL (such an abstraction is not needed in the definition of VTL Transformations). A SDMX “`Dataset`”, however, is an instance of a SDMX “`Dataflow`” and can be the artefact on which the VTL transformations are executed (i.e., the Transformations are defined on `Dataflows` and are applied to `Dataflow` instances that can be `Datasets`).

The VTL programs (Transformation Schemes) are represented in SDMX through the `TransformationScheme` maintainable class which is composed of `Transformation` (nameable artefact). Each `Transformation` assigns the outcome of the evaluation of a VTL expression to a result.

This section does not explain the VTL language or any of the content published in the VTL guides. Rather, this is a description of how the VTL can be used in the SDMX context and applied to SDMX artefacts.

5.12.2 References to SDMX artefacts from VTL statements

Introduction

The VTL can manipulate SDMX artefacts (or objects) by referencing them through pre-defined conventional names (aliases).

The alias of an SDMX artefact can be its URN (Universal Resource Name), an abbreviation of its URN or another user-defined name.

In any case, the aliases used in the VTL Transformations have to be mapped to the SDMX artefacts through the `VtlMappingScheme` and `VtlMapping` classes (see the section of the SDMX IM relevant to the VTL). A `VtlMapping` allows specifying the aliases to be used in the VTL Transformations, Rulesets[3] or User Defined Operators[4] to reference SDMX artefacts. A `VtlMappingScheme` is a container for zero or more `VtlMapping`.

The correspondence between an alias and a SDMX artefact must be one-to-one, meaning that a generic alias identifies one and just one SDMX artefact while a SDMX artefact is identified by one and just one alias. In other words, within a `VtlMappingScheme` an artefact can have just one alias and different artefacts cannot have the same alias.

The references through the URN and the abbreviated URN are described in the following paragraphs.

Copyright © SDMx TWG 2020

References through the URN

This approach has the advantage that in the VTL code the URN of the referenced artefacts is directly intelligible by a human reader but has the drawback that the references are verbose.

The SDMX URN[5] is the concatenation of the following parts, separated by special symbols like dot, equal, asterisk, comma, and parenthesis:

- `SDMXprefix`
- `SDMX-IM-package-name`
- `class-name`
- `agency-id`
- `maintainedobject-id`
- `maintainedobject-version`
- `container-object-id` [6]
- `object-id`

The generic structure of the URN is the following:

```
SDMXprefix.SDMX-IM-package-name.class-name=agency-id:maintainedobject-id
(maintainedobject-version).*container-object-id.object-id
```

The `SDMXprefix` is `"urn:sdmx:org"`, always the same for all SDMX artefacts.

The `SDMX-IM-package-name` is the concatenation of the string `"sdmx.infomodel."` with the package-name, which the artefact belongs to. For example, for referencing a `Dataflow` the `SDMX-IM-package-name` is `"sdmx.infomodel.datastructure"`, because the class Dataflow belongs to the package `"datastructure"`.

The `class-name` is the name of the SDMX object class, which the SDMX object belongs to (e.g., for referencing a `Dataflow` the `class-name` is `"Dataflow"`). The VTL can reference SDMX artefacts that belong to the classes `Dataflow`, `Dimension`, `TimeDimension`, `Measure`, `DataAttribute`, `Concept`, `Codelist`.

The `agency-id` is the acronym of the agency that owns the definition of the artefact, for example for the Eurostat artefacts the `agency-id` is `"ESTAT"`). The `agency-id` can be composite (for example `AgencyA.Dept1.Unit2`).

The `maintainedobject-id` is the name of the maintained object which the artefact belongs to, and in case the artefact itself is maintainable[7], coincides with the name of the artefact. Therefore the `maintainedobject-id` depends on the class of the artefact:

- if the artefact is a `Dataflow`, which is a maintainable class, the `maintainedobject-id` is the `Dataflow` name (`dataflow-id`);
- if the artefact is a `Dimension`, `Measure`, `TimeDimension` or `DataAttribute`, which are not maintainable and belong to the `DataStructure` maintainable class, the `maintainedobject-id` is the name of the `DataStructure` (`dataStructure-id`) which the artefact belongs to;
- if the artefact is a `Concept`, which is not maintainable and belongs to the `ConceptScheme` maintainable class, the `maintainedobject-id` is the name of the `ConceptScheme` (`conceptScheme-id`) which the artefact belongs to;
- if the artefact is a `Codelist`, which is a maintainable class, the `maintainedobject-id` is the `Codelist` name (`codelist-id`).

The `maintainedobject-version` is the version, according to the SDMX versioning rules, of the maintained object which the artefact belongs to (for example, possible versions might be 1.0, 2.3, 1.0.0, 2.1.0 or 3.1.2).

The `container-object-id` does not apply to the classes that can be referenced in VTL Transformations, therefore is not present in their URN

The `object-id` is the name of the non-maintainable artefact (when the artefact is maintainable its name is already specified as the `maintainedobject-id`, see above), in particular it has to be specified:

- if the artefact is a `Dimension`, `TimeDimension`, `Measure` or `DataAttribute` (the `object-id` is the name of one of the artefacts above, which are data structure components)
- if the artefact is a `Concept` (the `object-id` is the name of the `Concept`)

Copyright © SDMx TWG 2020

For example, by using the URN, the VTL Transformation that sums two SDMX `Dataflows` `DF1` and `DF2` and assigns the result to a third persistent `Dataflow` `DFR`, assuming that `DF1`, `DF2` and `DFR` are the `maintainedobject-id` of the three `Dataflows`, that their version is 1.0.0 and their Agency is `AG`, would be written as[8]:

```
'urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=AG:DFR(1.0.0)' <-
'urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=AG:DF1(1.0.0)' +
'urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=AG:DF2(1.0.0)'
```

Abbreviation of the URN

The complete formulation of the URN described above is exhaustive but verbose, even for very simple statements. In order to reduce the verbosity through a simplified identifier and make the work of transformation definers easier, proper abbreviations of the URN are possible. Using this approach, the referenced artefacts remain intelligible in the VTL code by a human reader.

The URN can be abbreviated by omitting the parts that are not essential for the identification of the artefact or that can be deduced from other available information, including the context in which the invocation is made. The possible abbreviations are described below.

- The `SDMXprefix` can be omitted for all the SDMX objects, because it is a prefixed string (urn:sdmx:org), always the same for SDMX objects.
- The `SDMX-IM-package-name` can be omitted as well because it can be deduced from the `class-name` that follows it (the table of the SDMX-IM packages and classes that allows this deduction is in the Registry Specifications Section). In particular, considering the object classes of the artefacts that VTL can reference, the package is:
  - `"datastructure"` for the classes `Dataflow`, `Dimension`, `TimeDimension`, `Measure`, `DataAttribute`,
  - `"conceptscheme"` for the class `Concept`,
  - `"codelist"` for the class `Codelist`.
- The `class-name` can be omitted as it can be deduced from the VTL invocation. In particular, starting from the VTL class of the invoked artefact (e.g. dataset, component, identifier, measure, attribute, variable, valuedomain), which is known given the syntax of the invoking VTL operator[9], the SDMX class can be deduced from the mapping rules between VTL and SDMX (see the section "Mapping between VTL and SDMX" hereinafter)[10].
- If the agency-id is not specified, it is assumed by default equal to the agency-id of the `TransformationScheme`, `UserDefinedOperatorScheme` or `RulesetScheme` from which the artefact is invoked. For example, the `agency-id` can be omitted if it is the same as the invoking `TransformationScheme` and cannot be omitted if the artefact comes from another agency[11]. Take also into account that, according to the VTL consistency rules, the agency of the result of a `Transformation` must be the same as its `TransformationScheme`, therefore the `agency-id` can be omitted for all the results (left part of Transformation statements).
- As for the `maintainedobject-id`, this is essential in some cases while in other cases it can be omitted:
  - if the referenced artefact is a `Dataflow`, which is a maintainable class, the `maintainedobject-id` is the `dataflow-id` and obviously cannot be omitted;
  - if the referenced artefact is a `Dimension`, `TimeDimension`, `Measure`, `DataAttribute`, which are not maintainable and belong to the `DataStructure` maintainable class, the `maintainedobject-id` is the `dataStructure`-id and can be omitted, given that these components are always invoked within the invocation of a Dataflow, whose dataStructure-id can be deduced from the SDMX structural definitions;
  - if the referenced artefact is a `Concept`, which is not maintainable and belong to the `ConceptScheme` maintainable class, the maintained object is the `conceptScheme-id` and cannot be omitted;
  - if the referenced artefact is a `Codelist`, which is a maintainable class, the `maintainedobject-id` is the codelist-id and obviously cannot be omitted.
- When the `maintainedobject-id` is omitted, the `maintainedobject-version` is omitted too. When the `maintainedobject-id` is not omitted and the `maintainedobject-version` is omitted, the version 1.0 is assumed by default.
  - As said, the `container-object-id` does not apply to the classes that can be referenced in VTL Transformations, therefore is not present in their URN
  - The `object-id` does not exist for the artefacts belonging to the `Dataflow`, and `Codelist` classes, while it exists and cannot be omitted for the artefacts belonging to the classes `Dimension`, `TimeDimension`, `Measure`, `DataAttribute` and `Concept`, as for them the `object-id` is the main identifier of the artefact

Copyright © SDMx TWG 2020

The simplified object identifier is obtained by omitting all the first part of the URN, including the special characters, till the first part not omitted.

For example, the full formulation that uses the complete URN shown at the end of the previous paragraph:

```
'urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=AG:DFR(1.0.0)' :=
'urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=AG:DF1(1.0.0)' +
'urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=AG:DF2(1.0.0)'
```

by omitting all the non-essential parts would become simply:

```
DFR := DF1 + DF2
```

The references to the `Codelists` can be simplified similarly. For example, given the non-abbreviated reference to the `Codelist AG:CL_FREQ(1.0.0)`, which is[12]:

```
'urn:sdmx:org.sdmx.infomodel.codelist.Codelist=AG:CL_FREQ(1.0.0)'
```

if the `Codelist` is referenced from a `RulesetScheme` belonging to the agency `AG`, omitting all the optional parts, the abbreviated reference would become simply[13]:

```
CL_FREQ
```

As for the references to the components, it can be enough to specify the `component-Id`, given that the `dataStructure-Id` can be omitted. An example of non-abbreviated reference, if the data structure is `DST1` and the component is `SECTOR`, is the following:

```
'urn:sdmx:org.sdmx.infomodel.datastructure.`DataStructure`=AG:DST1(1.0.0).SECTOR'
```

The corresponding fully abbreviated reference, if made from a `TransformationScheme` belonging to `AG`, would become simply:

```
SECTOR
```

For example, the `Transformation` for renaming the component `SECTOR` of the `Dataflow` `DF1` into `SEC` can be written as[14]:

```
'DFR(1.0.0)' := 'DF1(1.0.0)' [rename SECTOR to SEC]
```

In the references to the `Concepts`, which can exist for example in the definition of the VTL Rulesets, at least the `conceptScheme-id` and the `concept-id` must be specified.

An example of non-abbreviated reference, if the `conceptScheme-id` is `CS1` and the `concept-id` is `SECTOR`, is the following:

```
'urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=AG:CS1(1.0.0).SECTOR'
```

The corresponding fully abbreviated reference, if made from a `RulesetScheme` belonging to `AG`, would become simply:

```
CS1(1.0.0).SECTOR
```

The Codes and in general all the Values can be written without any other specification, for example, the transformation to check if the values of the measures of the `Dataflow` `DF1` are between 0 and 25000 can be written like follows:

```
'DFR(1.0.0)' := between ( 'DF1(1.0.0)', 0, 25000 )
```

The artefact (`Component`, `Concept`, `Codelist` ...) which the Values are referred to can be deduced from the context in which the reference is made, taking also into account the VTL syntax. In the Transformation above, for example, the values 0 and 2500 are compared to the values of the measures of `DF1(1.0.0)`.

User-defined alias

The third possibility for referencing SDMX artefacts from VTL statements is to use user-defined aliases not related to the SDMX URN of the artefact.

This approach gives preference to the use of symbolic names for the SDMX artefacts. As a consequence, in the VTL code the referenced artefacts may become not directly intelligible by a human reader. In any case, the VTL aliases are associated to the

Copyright © SDMx TWG 2020

SDMX URN through the `VtlMappingScheme` and `VtlMapping` classes. These classes provide for structured references to SDMX artefacts whatever kind of reference is used in VTL statements (URN, abbreviated URN or user-defined aliases).

References to SDMX artefacts from VTL Rulesets

The VTL Rulesets allow defining sets of reusable Rules that can be applied by some VTL operators, like the ones for validation and hierarchical roll-up. A “Rule” consists in a relationship between Values belonging to some Value Domains or taken by some Variables, for example: (i) when the Country is USA then the Currency is USD; (ii) the Benelux is composed by Belgium, Luxembourg, Netherlands.

The VTL Rulesets have a signature, in which the Value Domains or the Variables on which the Ruleset is defined are declared, and a body, which contains the Rules.

In the signature, given the mapping between VTL and SDMX better described in the following paragraphs, a reference to a VTL Value Domain becomes a reference to a SDMX `Codelist`, while a reference to a VTL Represented Variable becomes a reference to a SDMX `Concept`, assuming for it a definite representation[15].

In general, for referencing SDMX `Codelists` and `Concepts`, the conventions described in the previous paragraphs apply. In the Ruleset syntax, the elements that reference SDMX artefacts are called `"valueDomain"` and `"variable"` for the Datapoint Rulesets and `"ruleValueDomain"`, `"ruleVariable"`, `"condValueDomain"`, `"condVariable"` for the Hierarchical Rulesets). The syntax of the Ruleset signature allows also to define aliases of the elements above, these aliases are valid only within the specific Ruleset definition statement and cannot be mapped to SDMX.[16]

In the body of the Rulesets, the Codes and in general all the Values can be written without any other specification, because the artefact, which the Values are referred (`Codelist`, `Concept`) to can be deduced from the Ruleset signature.

5.12.3 Mapping between SDMX and VTL artefacts

When the mapping occurs

The mapping methods between the VTL and SDMX object classes allow transforming a SDMX definition in a VTL one and vice-versa for the artefacts to be manipulated.

It should be remembered that VTL programs (i.e. Transformation Schemes) are represented in SDMX through the `TransformationScheme` maintainable class which is composed of `Transformations` (nameable artefacts). Each `Transformation` assigns the outcome of the evaluation of a VTL expression to a result: the input operands of the expression and the result can be SDMX artefacts.

Every time a SDMX object is referenced in a VTL Transformation as an input operand, there is the need to generate a VTL definition of the object, so that the VTL operations can take place. This can be made starting from the SDMX definition and applying a SDMX-VTL mapping method in the direction from SDMX to VTL. The possible mapping methods from SDMX to VTL are described in the following paragraphs and are conceived to allow the automatic deduction of the VTL definition of the object from the knowledge of the SDMX definition.

In the opposite direction, every time an object calculated by means of VTL must be treated as a SDMX object (for example for exchanging it through SDMX), there is the need of a SDMX definition of the object, so that the SDMX operations can take place. The SDMX definition is needed for the VTL objects for which a SDMX use is envisaged[17].

The mapping methods from VTL to SDMX are described in the following paragraphs as well, however they do not allow the complete SDMX definition to be automatically deduced from the VTL definition, more than all because the former typically contains additional information in respect to the latter. For example, the definition of a SDMX DSD includes also some mandatory information not available in VTL (like the concept scheme to which the SDMX components refer, the ‘usage’ and ‘`attributeRelationship`’ for the `DataAttributes` and so on). Therefore the mapping methods from VTL to SDMX provide only a general guidance for generating SDMX definitions properly starting from the information available in VTL, independently of how the SDMX definition it is actually generated (manually, automatically or part and part).

Copyright © SDMx TWG 2020

General mapping of VTL and SDMX data structures

This section makes reference to the VTL "Model for data and their structure"[18] and the correspondent SDMX "Data Structure Definition"[19].

The main type of artefact that the VTL can manipulate is the VTL Data Set, which in general is mapped to the SDMX `Dataflow`. This means that a VTL Transformation, in the SDMX context, expresses the algorithm for calculating a derived `Dataflow` starting from some already existing `Dataflow`s (either collected or derived).[20]

While the VTL Transformations are defined in term of `Dataflow` definitions, they are assumed to be executed on instances of such `Dataflow`s, provided at runtime to the VTL engine (the mechanism for identifying the instances to be processed are not part of the VTL specifications and depend on the implementation of the VTL-based systems). As already said, the SDMX `Datasets` are instances of SDMX `Dataflow`s, therefore a VTL Transformation defined on some SDMX `Dataflow`s can be applied on some corresponding SDMX `Datasets`.

A VTL Data Set is structured by one and just one Data Structure and a VTL Data Structure can structure any number of Data Sets. Correspondingly, in the SDMX context a SDMX `Dataflow` is structured by one and just one `DataStructureDefinition` and one `DataStructureDefinition` can structure any number of `Dataflow`s.

A VTL Data Set has a Data Structure made of Components, which in turn can be Identifiers, Measures and Attributes. Similarly, a SDMX `DataflowDefinition` has a `DataStructureDefinition` made of components that can be `DimensionComponents`, Measure and `DataAttributes`. In turn, a SDMX `DimensionComponent` can be a Dimension or a `TimeDimension`. Correspondingly, in the SDMX implementation of the VTL, the VTL Identifiers can be (optionally) distinguished in similar sub-classes (Simple Identifier, Time Identifier) even if such a distinction is not evidenced in the VTL IM.

The possible mapping options are described in more detail in the following sections.

Mapping from SDMX to VTL data structures

BASIC MAPPING

The main mapping method from SDMX to VTL is called **Basic** mapping. This is considered as the default mapping method and is applied unless a different method is specified through the `VtlMappingScheme` and `VtlDataflowMapping` classes.

When transforming **from SDMX to VTL**, this method consists in leaving the components unchanged and maintaining their names and roles, according to the following table:

SDMX	VTL
`Dimension`	(Simple) Identifier
`TimeDimension`	(Time) Identifier
`Measure`	Measure
`DataAttribute`	Attribute

The SDMX `DataAttributes`, in VTL they are all considered "at data point / observation level" (i.e. dependent on all the VTL Identifiers), because VTL does not have the SDMX `AttributeRelationships`, which defines the construct to which the `DataAttribute` is related (e.g. observation, dimension or set or group of dimensions, whole data set).

With the Basic mapping, one SDMX observation[21] generates one VTL data point.

PIVOT MAPPING

An alternative mapping method from SDMX to VTL is the **Pivot** mapping, which makes sense and is different from the Basic method only for the SDMX data structures that contain a `Dimension` that plays the `role` of measure dimension (like in SDMX 2.1) and just one `Measure`. Through this method, these structures can be mapped to multi-measure VTL data structures. Besides that, a user may choose to use any `Dimension` acting as a list of `Measures` (e.g., a `Dimension` with indicators), either by considering the "Measure" `role` of a `Dimension`, or at will using any coded `Dimension`. Of course, in SDMX 3.0, this can only work when only one `Measure` is defined in the DSD.

Copyright © SDMx TWG 2020

In SDMX 2.1 the `MeasureDimension` was a subclass of `DimensionComponent` like `Dimension` and `TimeDimension`. In the current SDMX version, this subclass does not exist anymore, however a `Dimension` can have the `role` of measure dimension (i.e. a `Dimension` that contributes to the identification of the measures). In SDMX 2.1 a `DataStructure` could have zero or one `MeasureDimensions`, in the current version of the standard, from zero to many `Dimension` may have the `role` of measure dimension. Hereinafter a `Dimension` that plays the `role` of measure dimension is referenced for simplicity as "`MeasureDimension`", i.e. maintaining the capital letters and the courier font even if the `MeasureDimension` is not anymore a class in the SDMX Information Model of the current SDMX version. For the sake of simplicity, the description below considers just one `Dimension` having the `role` of `MeasureDimension` (i.e., the more simple and common case). Nevertheless, it maintains its validity also if in the `DataStructure` there are more dimension with the `role` of `MeasureDimensions`: in this case what is said about the `MeasureDimension` must be applied to the combination of all the `MeasureDimensions` considered as a joint variable[22].

Among other things, the Pivot method provides also backward compatibility with the SDMX 2.1 data structures that contained a `MeasureDimension`.

If applied to SDMX structures that do not contain any `MeasureDimension`, this method behaves like the Basic mapping (see the previous paragraph).

The SDMX structures that contain a `MeasureDimension` are mapped as described below (this mapping is equivalent to a pivoting operation):

- A SDMX simple dimension becomes a VTL (simple) identifier and a SDMX `TimeDimension` becomes a VTL (time) identifier;
- Each possible `Code Cj` of the SDMX `MeasureDimension` is mapped to a VTL Measure, having the same name as the SDMX Code (i.e. `Cj`); the VTL Measure `Cj` is a new VTL component even if the SDMX data structure has not such a `Component`;
- The SDMX `MeasureDimension` is not mapped to VTL (it disappears in the VTL Data Structure);
- The SDMX `Measure` is not mapped to VTL as well (it disappears in the VTL Data Structure);
- An SDMX `DataAttribute` is mapped in different ways according to its `AttributeRelationship`:
  - If, according to the SDMX `AttributeRelationship`, the values of the `DataAttribute` do not depend on the values of the `MeasureDimension`, the SDMX `DataAttribute` becomes a VTL Attribute having the same name. This happens if the `AttributeRelationship` is not specified (i.e. the `DataAttribute` does not depend on any `DimensionComponent` and therefore is at data set level), or if it refers to a set (or a group) of dimensions which does not include the `MeasureDimension`;
  - Otherwise, if, according to the SDMX `AttributeRelationship`, the values of the `DataAttribute` depend on the `MeasureDimension`, the SDMX `DataAttribute` is mapped to one VTL Attribute for each possible Code of the SDMX `MeasureDimension`. By default, the names of the VTL Attributes are obtained by concatenating the name of the SDMX `DataAttribute` and the names of the correspondent Code of the `MeasureDimension` separated by underscore. For example, if the SDMX `DataAttribute` is named `DA` and the possible Codes of the SDMX `MeasureDimension` are named `C1, C2, …, Cn`, then the corresponding VTL Attributes will be named `DA_C1, DA_C2, …, DA_Cn` (if different names are desired, they can be achieved afterwards by renaming the Attributes through VTL operators).
  - Like in the Basic mapping, the resulting VTL Attributes are considered as dependent on all the VTL identifiers (i.e. "at data point / observation level"), because VTL does not have the SDMX notion of Attribute Relationship.

The summary mapping table of the "pivot" mapping from SDMX to VTL for the SDMX data structures that contain a `MeasureDimension` is the following:

SDMX Component	VTL Mapping
Dimension	(Simple) Identifier
`TimeDimension`	(Time) Identifier
`MeasureDimension` & one `Measure`	One Measure for each `Code` of the SDMX `MeasureDimension`
`DataAttribute` (not depending on `MeasureDimension`)	Attribute
`DataAttribute` (depending on `MeasureDimension`)	One Attribute for each `Code` of the SDMX `MeasureDimension`

Using this mapping method, the components of the data structure can change in the conversion from SDMX to VTL and it must be taken into account that the VTL statements can reference only the components of the resulting VTL data structure.

Copyright © SDMx TWG 2020

At observation / data point level, calling `Cj (j=1, … n)` the j-th Code of the `MeasureDimension`:

- The set of SDMX observations having the same values for all the Dimensions except than the `MeasureDimension` become one multi-measure VTL Data Point, having one Measure for each Code Cj of the SDMX `MeasureDimension`;
- The values of the SDMX simple `Dimensions`, `TimeDimension` and `DataAttributes` not depending on the `MeasureDimension` (these components by definition have always the same values for all the observations of the set above) become the values of the corresponding VTL (simple) Identifiers, (time) Identifier and Attributes.
- The value of the Measure of the SDMX observation belonging to the set above and having `MeasureDimension=Cj` becomes the value of the VTL Measure `Cj`
- For the SDMX `DataAttributes` depending on the `MeasureDimension`, the value of the `DataAttribute DA` of the SDMX observation belonging to the set above and having `MeasureDimension=Cj` becomes the value of the VTL Attribute `DA_Cj`

FROM SDMX DATAATTRIBUTES TO VTL MEASURES

- In some cases, it may happen that the `DataAttributes` of the SDMX `DataStructure` need to be managed as Measures in VTL. Therefore, a variant of both the methods above consists in transforming all the SDMX `DataAttributes` in VTL Measures. When `DataAttributes` are converted to Measures, the two methods above are called `Basic_A2M` and `Pivot_A2M` (the suffix “A2M” stands for Attributes to Measures). Obviously, the resulting VTL data structure is, in general, multi-measure and does not contain Attributes.

The `Basic_A2M` and `Pivot_A2M` behaves respectively like the Basic and Pivot methods, except that the final VTL components, which according to the Basic and Pivot methods would have had the role of Attribute, assume instead the role of Measure.

Proper VTL features allow changing the role of specific attributes even after the SDMX to VTL mapping: they can be useful when only some of the `DataAttributes` need to be managed as VTL Measures.

Mapping from VTL to SDMX data structures

BASIC MAPPING

The main mapping method **from VTL to SDMX** is called **Basic** mapping as well.

This is considered as the default mapping method and is applied unless a different method is specified through the `VtlMappingScheme` and `VtlDataflowMapping` classes.

The method consists in leaving the components unchanged and maintaining their names and roles in SDMX, according to the following mapping table, which is the same as the basic mapping from SDMX to VTL, only seen in the opposite direction.

Mapping table:

VTL	SDMX
(Simple) Identifier	`Dimension`
(Time) Identifier	`TimeDimension`
Measure	`Measure`
Attribute	`DataAttribute`

If the distinction between simple identifier and time identifier is not maintained in the VTL environment, the classification between `Dimension` and `TimeDimension` exists only in SDMX, as declared in the relevant `DataStructureDefinition`.

Regarding the Attributes, because VTL considers all of them “at observation level”, the corresponding SDMX `DataAttributes` should be put “at observation level” as well, unless some different information about their `AttributeRelationship` is otherwise available.

Note that the basic mappings in the two directions (from SDMX to VTL and vice-versa) are (almost completely) reversible. In fact, if a SDMX structure is mapped to a VTL structure and then the latter is mapped back to SDMX, the resulting data structure is like the original one (apart for the `AttributeRelationship`, that can be different if the original SDMX structure contains `DataAttributes` that are not at observation level). In reverse order, if a VTL structure is mapped to SDMX and then the latter is mapped back to VTL, the original data structure is obtained.

Copyright © SDMx TWG 2020

As said, the resulting SDMX definitions must be compliant with the SDMX consistency rules. For example, the SDMX DSD must have the `AttributeRelationship` for the `DataAttributes`, which does not exist in VTL.

UNPIVOT MAPPING

An alternative mapping method from VTL to SDMX is the **Unpivot** mapping.

Although this mapping method can be used in any case, it makes major sense in case the VTL data structure has more than one measure component (multi-measures VTL structure). This is used to support the SDMX 2.1 case of a `MeasureDimension` or any other Dimension acting as a list of Measures, under the assumptions explained in section “Pivot Mapping”.

The multi-measures VTL structures are converted to SDMX `Dataflows` having an added `MeasureDimension`, which disambiguates the VTL multiple Measures, and a new Measure in place of the VTL ones, containing the values of the VTL Measures.

The **unpivot** mapping behaves like follows:

- like in the basic mapping, a VTL (simple) identifier becomes a SDMX `Dimension` and a VTL (time) identifier becomes a SDMX `TimeDimension` (as said, a measure identifier cannot exist in multi-measure VTL structures);
- a `MeasureDimension` component called `"measure_name"` is added to the SDMX `DataStructure`;
- a `Measure` component called `"obs_value"` is added to the SDMX `DataStructure`;
- each VTL Measure is mapped to a `Code` of the SDMX `MeasureDimension` having the same name as the VTL Measure (therefore all the VTL Measure Components do not originate Components in the SDMX `DataStructure`);
- a VTL Attribute becomes a SDMX `DataAttribute` having `AttributeRelationship` referred to all the SDMX `DimensionComponents` including the `TimeDimension` and except the `MeasureDimension`.

The summary mapping table of the **unpivot** mapping method is the following:

VTL	SDMX
(Simple) Identifier	`Dimension`
(Time) Identifier	`TimeDimension`
All Measure Components	`MeasureDimension` (having one `Code` for each VTL measure component) and one `Measure`
Attribute	`DataAttribute` depending on all SDMX `Dimensions` including the `TimeDimension` and except the `MeasureDimension`

At observation / data point level:

- a multi-measure VTL Data Point becomes a set of SDMX observations, one for each VTL Measure;
- the values of the VTL Identifiers become the values of the corresponding SDMX `DimensionComponents`, for all the observations of the set above;
- the name of the j-th VTL Measure (e.g. “Cj”) becomes the Code of the SDMX `MeasureDimension` of the j-th observation of the set;
- the value of the j-th VTL Measure becomes the value of the SDMX Measure of the j-th observation of the set;
- the values of the VTL Attributes become the values of the corresponding SDMX `DataAttributes` (in principle for all the observations of the set above).

If desired, this method can be applied also to mono-measure VTL structures, provided that none of the VTL Components has already the role of Measure Identifier. Like in the general case, a `MeasureDimension` component called “measure_name” is added to the SDMX `DataStructure`, in this case it has just one possible `Code`, corresponding to the name of the unique VTL Measure. The original VTL Measure would not become a Component in the SDMX data structure. The value of the VTL Measure would be assigned to the unique SDMX Measure called “obs_value”.

In any case, the resulting SDMX definitions must be compliant with the SDMX consistency rules. For example, the possible `Codes` of the SDMX `MeasureDimension` need to be listed in a SDMX `Codelist`, with proper id, agency and version; moreover, the SDMX DSD must have the `AttributeRelationship` for the `DataAttributes`, which does not exist in VTL.

Copyright © SDMx TWG 2020

FROM VTL MEASURES TO SDMX DATA ATTRIBUTES

More than all for the multi-measure VTL structures (having more than one Measure Component), it may happen that the Measures of the VTL Data Structure need to be managed as `DataAttributes` in SDMX. Therefore, a third mapping method consists in transforming some VTL measures in a corresponding SDMX `Measures` and all the other VTL Measures in SDMX `DataAttributes`. This method is called `M2A` ("M2A" stands for "Measures to `DataAttributes`").

All VTL Measures maintain their names in SDMX. The VTL Measure Components that become SDMX `DataAttributes` are the ones declared as `DataAttributes` in the target SDMX data structure definition.

The mapping table is the following:

VTL	SDMX
(Simple) Identifier	`Dimension`
(Time) Identifier	`TimeDimension`
Some Measures	`Measure`
Other Measures	`DataAttribute`
Attribute	`DataAttribute`

Even in this case, the resulting SDMX definitions must be compliant with the SDMX consistency rules. For example, the SDMX DSD must have the `attributeRelationship` for the `DataAttributes`, which does not exist in VTL.

Declaration of the mapping methods between data structures

In order to define and understand properly VTL Transformations, the applied mapping methods must be specified in the SDMX structural metadata. If the default mapping method (Basic) is applied, no specification is needed.

A customized mapping can be defined through the `VtlMappingScheme` and `VtlDataflowMapping` classes (see the section of the SDMX IM relevant to the VTL). A `VtlDataflowMapping` allows specifying the mapping methods to be used for a specific dataflow, both in the direction from SDMX to VTL (`toVtlMappingMethod`) and from VTL to SDMX (`fromVtlMappingMethod`); in fact a `VtlDataflowMapping` associates the structured URN that identifies a SDMX `Dataflow` to its VTL alias and its mapping methods.

It is possible to specify the `toVtlMappingMethod` and `fromVtlMappingMethod` also for the conventional `dataflow` called "generic_dataflow": in this case the specified mapping methods are intended to become the default ones, overriding the "Basic" methods. In turn, the `toVtlMappingMethod` and `fromVtlMappingMethod` declared for a specific `Dataflow` are intended to override the default ones for such a `Dataflow`.

The `VtlMappingScheme` is a container for zero or more `VtlDataflowMapping` (it may contain also mappings towards artefacts other than dataflows).

Mapping dataflow subsets to distinct VTL Data Sets

Until now it has been assumed to map one SMDX `Dataflow` to one VTL Data Set and vice-versa. This mapping one-to-one is not mandatory according to VTL because a VTL Data Set is meant to be a set of observations (data points) on a logical plane, having the same logical data structure and the same general meaning, independently of the possible physical representation or storage (see VTL 2.0 User Manual page 24), therefore a SDMX `Dataflow` can be seen either as a unique set of data observations (corresponding to one VTL Data Set) or as the union of many sets of data observations (each one corresponding to a distinct VTL Data Set).

As a matter of fact, in some cases it can be useful to define VTL operations involving definite parts of a SDMX `Dataflow` instead than the whole.[23]

Therefore, in order to make the coding of VTL operations simpler when applied on parts of SDMX `Dataflow`s, it is allowed to map distinct parts of a SDMX `Dataflow` to distinct VTL Data Sets according to the following rules and conventions. This kind of mapping is possible both from SDMX to VTL and from VTL to SDMX, as better explained below.[24]

Copyright © SDMx TWG 2020

Given a SDMX `Dataflow` and some predefined `Dimensions` of its `DataStructure`, it is allowed to map the subsets of observations that have the same combination of values for such `Dimensions` to correspondent VTL datasets.

For example, assuming that the SDMX `Dataflow` DF1(1.0.0) has the `Dimensions` `INDICATOR`, `TIME_PERIOD` and `COUNTRY`, and that the user declares the `Dimensions` `INDICATOR` and `COUNTRY` as basis for the mapping (i.e. the mapping dimensions): the observations that have the same values for `INDICATOR` and `COUNTRY` would be mapped to the same VTL dataset (and vice-versa).

In practice, this kind mapping is obtained like follows:

- For a given SDMX `Dataflow`, the user (VTL definer) declares the `DimensionComponents` on which the mapping will be based, in a given order.[25] Following the example above, imagine that the user declares the Dimensions INDICATOR and `COUNTRY`.
- The VTL Data Set is given a name using a special notation also called “ordered concatenation” and composed of the following parts:
  - The reference to the SDMX `Dataflow` (expressed according to the rules described in the previous paragraphs, i.e. URN, abbreviated URN or another alias); for example DF(1.0.0);
  - a slash (“/”) as a separator; [26]
  - The reference to a specific part of the SDMX `Dataflow` above, expressed as the concatenation of the values that the SDMX `DimensionComponents` declared above must have, separated by dots (`“.”`) and written in the order in which these `DimensionComponents` are defined[27]. For example POPULATION.USA would mean that such a VTL Data Set is mapped to the SDMX observations for which the dimension `INDICATOR` is equal to POPULATION and the dimension `COUNTRY` is equal to USA.

In the VTL Transformations, this kind of dataset name must be referenced between single quotes because the slash (`“/”`) is not a regular character according to the VTL rules.

Therefore, the generic name of this kind of VTL datasets would be:

```
'DF(1.0.0)/INDICATORvalue.COUNTRYvalue'
```

Where `DF(1.0.0)` is the `Dataflow` and `INDICATORvalue` and `COUNTRYvalue` are placeholders for one value of the `INDICATOR` and `COUNTRY` dimensions.

Instead the specific name of one of these VTL datasets would be:

```
‘DF(1.0.0)/POPULATION.USA’
```

In particular, this is the VTL dataset that contains all the observations of the `Dataflow` `DF(1.0.0)` for which `INDICATOR = POPULATION` and `COUNTRY = USA`.

Let us now analyse the different meaning of this kind of mapping in the two mapping directions, i.e. from SDMX to VTL and from VTL to SDMX.

As already said, the mapping from SDMX to VTL happens when the SDMX `Dataflows` are operand of VTL Transformations, instead the mapping from VTL to SDMX happens when the VTL Data Sets that is result of Transformations[28] need to be treated as SDMX objects. This kind of mapping can be applied independently in the two directions and the `Dimensions` on which the mapping is based can be different in the two directions: these `Dimensions` are defined in the `ToVtlSpaceKey` and in the `FromVtlSpaceKey` classes respectively.

First, let us see what happens in the **mapping direction from SDMX to VTL**, i.e. when parts of a SDMX Dataflow (e.g. `DF1(1.0.0)`) need to be mapped to distinct VTL Data Sets that are operand of some VTL Transformations.

As already said, each VTL Data Set is assumed to contain all the observations of the SDMX Dataflow having `INDICATOR=INDICATORvalue` and `COUNTRY = COUNTRYvalue`. For example, the VTL dataset `‘DF1(1.0.0)/POPULATION.USA’` would contain all the observations of `DF1(1.0.0)` having `INDICATOR = POPULATION` and `COUNTRY = USA`.

In order to obtain the data structure of these VTL Data Sets from the SDMX one, it is assumed that the SDMX `DimensionComponents` on which the mapping is based are dropped, i.e. not maintained in the VTL data structure; this is possible because their values are fixed for each one of the invoked VTL Data Sets[29]. After that, the mapping method from SDMX to VTL specified for the Dataflow `DF1(1.0.0)` is applied (i.e. basic, pivot …).

Copyright © SDMx TWG 2020

In the example above, for all the datasets of the kind `'DF1(1.0.0)/INDICATORvalue.COUNTRYvalue'`, the dimensions `INDICATOR` and COUNTRY would be dropped so that the data structure of all the resulting VTL Data Sets would have the identifier `TIME_PERIOD` only.

It should be noted that the desired VTL Data Sets (i.e. of the kind `'DF1(1.0.0)/INDICATORvalue.COUNTRYvalue'`) can be obtained also by applying the VTL operator "**sub**" (subspace) to the `Dataflow DF1(1.0.0)`, like in the following VTL expression:

```
'DF1(1.0.0)/POPULATION.USA' :=
DF1(1.0.0) [ sub INDICATOR="POPULATION", COUNTRY="USA" ];

'DF1(1.0.0)/POPULATION.CANADA' :=
DF1(1.0.0) [ sub INDICATOR="POPULATION", COUNTRY="CANADA" ];
… … …
```

In fact the VTL operator "sub" has exactly the same behaviour. Therefore, mapping different parts of a SDMX `Dataflow` to different VTL Data Sets in the direction from SDMX to VTL through the ordered concatenation notation is equivalent to a proper use of the operator `"sub"` on such a Dataflow. [30]

In the direction from SDMX to VTL it is allowed to omit the value of one or more `DimensionComponents` on which the mapping is based, but maintaining all the separating dots (therefore it may happen to find two or more consecutive dots and dots in the beginning or in the end). The absence of value means that for the corresponding Dimension all the values are kept and the Dimension is not dropped.

For example, `'DF(1.0.0)/POPULATION.'` (note the dot in the end of the name) is the VTL dataset that contains all the observations of the `Dataflow DF(1.0.0)` for which `INDICATOR = POPULATION` and `COUNTRY =` any value.

This is equivalent to the application of the VTL `"sub"` operator only to the identifier `INDICATOR`:

```
'DF1(1.0.0)/POPULATION.' :=
DF1(1.0.0) [ sub INDICATOR="POPULATION" ];
```

Therefore the VTL Data Set `'DF1(1.0.0)/POPULATION.'` would have the identifiers `COUNTRY` and `TIME_PERIOD`.

Heterogeneous invocations of the same Dataflow are allowed, i.e. omitting different Dimensions in different invocations.

Let us now analyse **the mapping direction from VTL to SDMX**.

In this situation, distinct parts of a SDMX `Dataflow` are calculated as distinct VTL datasets, under the constraint that they must have the same VTL data structure.

For example, let us assume that the VTL programmer wants to calculate the SDMX `Dataflow DF2(1.0.0)` having the Dimensions `TIME_PERIOD`, `INDICATOR`, and `COUNTRY` and that such a programmer finds it convenient to calculate separately the parts of `DF2(1.0.0)` that have different combinations of values for `INDICATOR` and `COUNTRY`:

- each part is calculated as a VTL derived Data Set, result of a dedicated VTL Transformation; [31]
- the data structure of all these VTL Data Sets has the `TIME_PERIOD` identifier and does not have the `INDICATOR` and `COUNTRY` identifiers.[32]

Under these hypothesis, such derived VTL Data Sets can be mapped to `DF2(1.0.0)` by declaring the `DimensionComponents INDICATOR` and `COUNTRY` as mapping dimensions[33].

The corresponding VTL Transformations, assuming that the result needs to be persistent, would be of this kind[34]:

```
'DF2(1.0.0)/INDICATORvalue.COUNTRYvalue' <- expression
```

Some examples follow, for some specific values of `INDICATOR` and `COUNTRY`:

```
'DF2(1.0.0)/GDPPERCAPITA.USA' <- expression11;
'DF2(1.0.0)/GDPPERCAPITA.CANADA' <- expression12;
… … …
'DF2(1.0.0)/POPGROWTH.USA' <- expression21;
'DF2(1.0.0)/POPGROWTH.CANADA' <- expression22;
… … …
```


Copyright © SDMx TWG 2020

As said, it is assumed that these VTL derived Data Sets have the `TIME_PERIOD` as the only identifier. In the mapping from VTL to SMDX, the Dimensions `INDICATOR` and `COUNTRY` are added to the VTL data structure on order to obtain the SDMX one, with the following values respectively:

VTL dataset	INDICATOR value	COUNTRY value
`'DF2(1.0.0)/GDPPERCAPITA.USA'`	`GDPPERCAPITA`	`USA`
`'DF2(1.0.0)/GDPPERCAPITA.CANADA'`	`GDPPERCAPITA`	`CANADA`
…	…	…
`'DF2(1.0.0)/POPGROWTH.USA'`	`POPGROWTH`	`USA`
`'DF2(1.0.0)/POPGROWTH.CANADA'`	`POPGROWTH`	`CANADA`
…	…	…

It should be noted that the application of this many-to-one mapping from VTL to SDMX is equivalent to an appropriate sequence of VTL Transformations. These use the VTL operator “calc” to add the proper VTL identifiers (in the example, `INDICATOR` and `COUNTRY`) and to assign to them the proper values and the operator “union” in order to obtain the final VTL dataset (in the example `DF2(1.0.0)`), that can be mapped one-to-one to the homonymous SDMX `Dataflow`. Following the same example, these VTL Transformations would be:

```
DF2bis_GDPPERCAPITA_USA := 'DF2(1.0.0)/GDPPERCAPITA.USA'
            [calc identifier INDICATOR := "GDPPERCAPITA",
            identifier COUNTRY := "USA"];

DF2bis_GDPPERCAPITA_CANADA := 'DF2(1.0.0)/GDPPERCAPITA.CANADA'
            [calc identifier INDICATOR:="GDPPERCAPITA",
            identifier COUNTRY:="CANADA"];
… … …
DF2bis_POPGROWTH_USA := 'DF2(1.0.0)/POPGROWTH.USA'
            [calc identifier INDICATOR := "POPGROWTH",
            identifier COUNTRY := "USA"];

DF2bis_POPGROWTH_CANADA' := 'DF2(1.0.0)/POPGROWTH.CANADA'
            [calc identifier INDICATOR := "POPGROWTH",
            identifier COUNTRY := "CANADA"];
… … …
DF2(1.0) <- UNION (DF2bis_GDPPERCAPITA_USA',
    DF2bis_GDPPERCAPITA_CANADA', … ,
    DF2bis_POPGROWTH_USA', DF2bis_POPGROWTH_CANADA' …);
```

In other words, starting from the datasets explicitly calculated through VTL (in the example `'DF2(1.0)/GDPPERCAPITA.USA'` and so on), the first step consists in calculating other (non-persistent) VTL datasets (in the example `DF2bis_GDPPERCAPITA_USA` and so on) by adding the identifiers `INDICATOR` and `COUNTRY` with the desired values (`INDICATORvalue` and `COUNTRYvalue`). Finally, all these non-persistent Data Sets are united and give the final result `DF2(1.0)` [35], which can be mapped one-to-one to the homonymous SDMX `Dataflow` having the dimension components `TIME_PERIOD`, `INDICATOR` and `COUNTRY`.

Therefore, mapping different VTL datasets having the same data structure to different parts of a SDMX `Dataflow`, i.e. in the direction from VTL to SDMX, through the ordered concatenation notation is equivalent to a proper use of the operators “calc” and “union” on such datasets. [36]

It is worth noting that in the direction from VTL to SDMX it is mandatory to specify the value for every Dimension on which the mapping is based (in other word, in the name of the calculated VTL dataset is **not** possible to omit the value of some of the Dimensions).

Copyright © SDMx TWG 2020

Mapping variables and value domains between VTL and SDMX

With reference to the VTL “model for Variables and Value domains”, the following additional mappings have to be considered:

VTL	SDMX
Data Set Component	Although this abstraction exists in SDMX, it does not have an explicit definition and corresponds to a `Component` (either a `DimensionComponent` or a Measure or a `DataAttribute`) belonging to one specific Dataflow[37]
Represented Variable	`Concept` with a definite Representation
Value Domain	`Representation` (see the Structure Pattern in the Base Package)
Enumerated Value Domain / Code List	`Codelist`
Code	`Code` (for enumerated `DimensionComponent`, Measure, `DataAttribute`)
Described Value Domain	non-enumerated `Representation` (having Facets / `ExtendedFacets`, see the Structure Pattern in the Base Package)
Value	Although this abstraction exists in SDMX, it does not have an explicit definition and corresponds to a `Code` of a Codelist (for enumerated Representations) or to a valid `value` (for non-enumerated Representations)
Value Domain Subset / Set	This abstraction does not exist in SDMX
Enumerated Value Domain Subset / Enumerated Set	This abstraction does not exist in SDMX
Described Value Domain Subset / Described Set	This abstraction does not exist in SDMX
Set list	This abstraction does not exist in SDMX

The main difference between VTL and SDMX relies on the fact that the VTL artefacts for defining subsets of Value Domains do not exist in SDMX, therefore the VTL features for referring to predefined subsets are not available in SDMX. These artefacts are the Value Domain Subset (or Set), either enumerated or described, the Set List (list of values belonging to enumerated subsets) and the Data Set Component (aimed at defining the set of values that the Component of a Data Set can take, possibly a subset of the codes of Value Domain).

Another difference consists in the fact that all Value Domains are considered as identifiable objects in VTL either if enumerated or not, while in SDMX the `Codelist` (corresponding to a VTL enumerated Value Domain) is identifiable, while the SDMX non-enumerated `Representation` (corresponding to a VTL non-enumerated Value Domain) is not identifiable. As a consequence, the definition of the VTL Rulesets, which in VTL can refer either to enumerated or non-enumerated value domains, in SDMX can refer only to enumerated Value Domains (i.e. to SDMX `Codelists`).

As for the mapping between VTL variables and SDMX `Concepts`, it should be noted that these artefacts do not coincide perfectly. In fact, the VTL variables are represented variables, defined always on the same Value Domain (`“Representation”` in SDMX) independently of the data set / data structure in which they appear[38], while the SDMX `Concepts` can have different `Representations` in different `DataStructures`.[39] This means that one SDMX `Concept` can correspond to many VTL Variables, one for each representation the `Concept` has.

Therefore, it is important to be aware that some VTL operations (for example the binary operations at data set level) are consistent only if the components having the same names in the operated VTL Data Sets have also the same representation (i.e. the same Value Domain as for VTL). For example, it is possible to obtain correct results from the VTL expression

```
DS_c := DS_a + DS_b (where DS_a, DS_b, DS_c are VTL Data Sets)
```

if the matching components in `DS_a` and `DS_b` (e.g. ref_date, geo_area, sector …) refer to the same general representation. In simpler words, `DS_a` and `DS_b` must use the same values/codes (for ref_date, geo_area, sector …), otherwise the relevant values would not match and the result of the operation would be wrong.

Copyright © SDMx TWG 2020

As mentioned, the property above is not enforced by construction in SDMX, and different representations of the same `Concept` can be not compatible one another (for example, it may happen that geo_area is represented by ISO-alpha-3 codes in `DS_a` and by ISO alpha-2 codes in `DS_b`). Therefore, it will be up to the definer of VTL Transformations to ensure that the VTL expressions are consistent with the actual representations of the correspondent SDMX `Concepts`.

It remains up to the SDMX-VTL definer also the assurance of the consistency between a VTL Ruleset defined on Variables and the SDMX `Components` on which the Ruleset is applied. In fact, a VTL Ruleset is expressed by means of the values of the Variables (i.e. SDMX `Concepts`), i.e. assuming definite representations for them (e.g. ISO-alpha-3 for country). If the Ruleset is applied to SDMX `Components` that have the same name of the `Concept` they refer to but different representations (e.g. ISO-alpha-2 for country), the Ruleset cannot work properly.

5.12.4 Mapping between SDMX and VTL Data Types

VTL Data types

According to the VTL User Guide the possible operations in VTL depend on the data types of the artefacts. For example, numbers can be multiplied but text strings cannot. In the VTL Transformations, the compliance between the operators and the data types of their operands is statically checked, i.e., violations result in compile-time errors.

The VTL data types are sub-divided in scalar types (like integers, strings, etc.), which are the types of the scalar values, and compound types (like Data Sets, `Components`, Rulesets, etc.), which are the types of the compound structures. See below the diagram of the VTL data types, taken from the VTL User Manual:

Copyright © SDMx TWG 2020

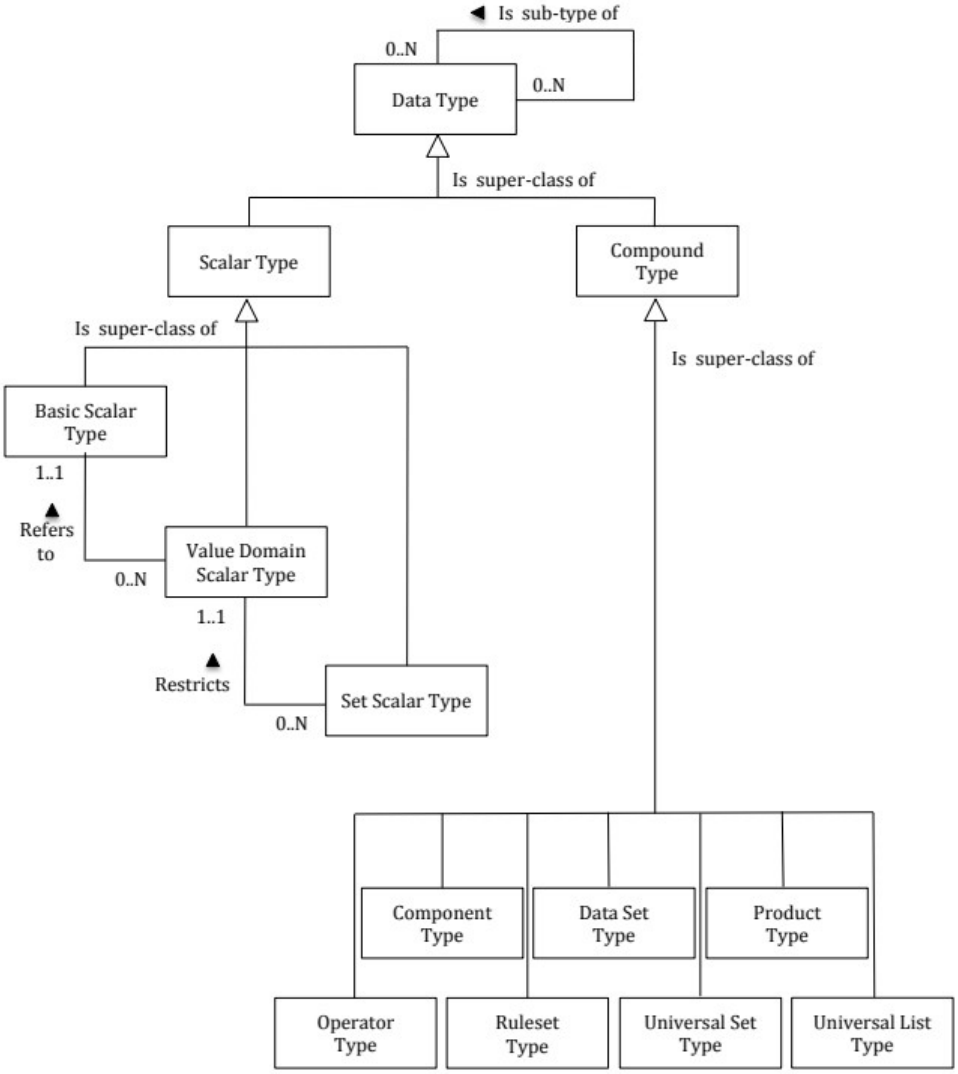


Figure 22. VTL Data Types

The VTL scalar types are in turn subdivided in basic scalar types, which are elementary (not defined in term of other data types) and Value Domain and Set scalar types, which are defined in terms of the basic scalar types.

The VTL basic scalar types are listed below and follow a hierarchical structure in terms of supersets/subsets (e.g. “scalar” is the superset of all the basic scalar types):

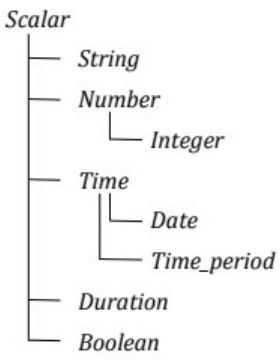


Figure 23. VTL Basic Scalar Types

Copyright © SDMx TWG 2020

VTL basic scalar types and SDMX data types

The VTL assumes that a basic scalar type has a unique internal representation and can have more external representations.

The internal representation is the format used within a VTL system to represent (and process) all the scalar values of a certain type. In principle, this format is hidden and not necessarily known by users. The external representations are instead the external formats of the values of a certain basic scalar type, i.e. the formats known by the users. For example, the internal representation of the dates can be an integer counting the days since a predefined date (e.g. from 01/01/4713 BC up to 31/12/5874897 AD like in Postgres) while two possible external representations are the formats `YYYY-MM-GG` and `MM-GG-YYYY` (e.g. respectively 2010-12-31 and 12-31-2010).

The internal representation is the reference format that allows VTL to operate on more values of the same type (for example on more dates) even if such values have different external formats: these values are all converted to the unique internal representation so that they can be composed together (e.g. to find the more recent date, to find the time span between these dates and so on).

The VTL assumes that a unique internal representation exists for each basic scalar type but does not prescribe any particular format for it, leaving the VTL systems free to using they preferred or already existing internal format. By consequence, in VTL the basic scalar types are abstractions not associated to a specific format.

SDMX data types are conceived instead to support the data exchange, therefore they do have a format, which is known by the users and correspond, in VTL terms, to external representations. Therefore, for each VTL basic scalar type there can be more SDMX data types (the latter are explained in the section “General Notes for Implementers” of this document and are actually much more numerous than the former).

The following paragraphs describe the mapping between the SDMX data types and the VTL basic scalar types. This mapping shall be presented in the two directions of possible conversion, i.e. from SDMX to VTL and vice-versa.

The conversion from SDMX to VTL happens when an SDMX artefact acts as inputs of a VTL Transformation. As already said, in fact, at compile time the VTL needs to know the VTL type of the operands in order to check their compliance with the VTL operators and at runtime it must convert the values from their external (SDMX) representations to the corresponding internal (VTL) ones.

The opposite conversion, i.e. from VTL to SDMX, happens when a VTL result, i.e. a VTL Data Set output of a Transformation, must become a SDMX artefact (or part of it). The values of the VTL result must be converted into the desired (SDMX) external representations (data types) of the SDMX artefact.

Copyright © SDMx TWG 2020

Mapping SDMX data types to VTL basic scalar types

The following table describes the default mapping for converting from the SDMX data types to the VTL basic scalar types.

Copyright © SDMx TWG 2020

SDMX data type (`BasicComponentDataType`)	**Default VTL basic scalar type**
`String` (string allowing any character)	`string`
`Alpha` (string which only allows A-z)	`string`
`AlphaNumeric` (string which only allows A-z and 0-9)	`string`
`Numeric` (string which only allows 0-9, but is not numeric so that it can have leading zeros)	`string`
`BigInteger` (corresponds to XML Schema `xs:integer` datatype; infinite set of integer values)	`integer`
`Integer` (corresponds to XML Schema `xs:int` datatype; between -2147483648 and +2147483647 (inclusive))	`integer`
`Long` (corresponds to XML Schema `xs:long` datatype; between -9223372036854775808 and +9223372036854775807 (inclusive))	`integer`
`Short` (corresponds to XML Schema `xs:short` datatype; between -32768 and -32767 (inclusive))	`integer`
`Decimal` (corresponds to XML Schema `xs:decimal` datatype; subset of real numbers that can be represented as decimals)	`number`
`Float` (corresponds to XML Schema `xs:float` datatype; patterned after the IEEE single-precision 32-bit floating point type)	`number`
`Double` (corresponds to XML Schema `xs:double` datatype; patterned after the IEEE double-precision 64-bit floating point type)	`number`
`Boolean` (corresponds to the XML Schema `xs:boolean` datatype; supports the mathematical concept of binary-valued logic: {true, false})	`boolean`
`URI` (corresponds to the XML Schema `xs:anyURI`; absolute or relative Uniform Resource Identifier Reference)	`string`
`Count` (an integer following a sequential pattern, increasing by 1 for each occurrence)	`integer`
`InclusiveValueRange` (decimal number within a closed interval, whose bounds are specified in the SDMX representation by the facets `minValue` and `maxValue`)	`number`
`ExclusiveValueRange` (decimal number within an open interval, whose bounds are specified in the SDMX representation by the facets `minValue` and `maxValue`)	`number`
`Incremental` (decimal number that increases by a specific interval, defined by the interval facet, typically enforced outside of the XML validation)	`number`
`ObservationalTimePeriod` (superset of `StandardTimePeriod` and `TimeRange`)	`time`
`StandardTimePeriod` (superset of `BasicTimePeriod` and `ReportingTimePeriod`)	`time`
`BasicTimePeriod` (superset of `GregorianTimePeriod` and `DateTime`)	`date`
`GregorianTimePeriod` (superset of `GregorianYear`, `GregorianYearMonth`, and `GregorianDay`)	`date`
`GregorianYear` (`YYYY`)	`date`
`GregorianYearMonth` / `GregorianMonth` (`YYYY-MM`)	`date`
`GregorianDay` (`YYYY-MM-DD`)	`date`
`ReportingTimePeriod` (superset of `ReportingYear`, `ReportingSemester`, `ReportingTrimester`, `ReportingQuarter`, `ReportingMonth`, `ReportingWeek`, `ReportingDay`)	`time_period`
`ReportingYear` (`YYYY-A1` – 1 year period)	`time_period`
`ReportingSemester` (`YYYY-Ss` – 6 month period)	`time_period`
`ReportingTrimester` (`YYYY-Tt` – 4 month period)	`time_period`

Copyright © SDMx TWG 2020

SDMX data type (`BasicComponentDataType`)	Default VTL basic scalar type
`ReportingQuarter` (`YYYY-Qq` – 3 month period)	`time_period`
`ReportingMonth` (`YYYY-Mmm` – 1 month period)	`time_period`
`ReportingWeek` (`YYYY-Www` – 7 day period; following ISO 8601 definition of a week in a year)	`time_period`
`ReportingDay` (`YYYY-Dddd` – 1 day period)	`time_period`
`DateTime` (`YYYY-MM-DDThh:mm:ss`)	`date`
`TimeRange` (`YYYY-MM-DD(Thh:mm:ss)?/<duration>`)	`time`
`Month` (`--MM` ; specifies a month independent of a year)	`string`
`MonthDay` (`--MM-DD` ; specifies a day within a month independent of a year)	`string`
`Day` (`---DD` ; specifies a day independent of a month or year)	`string`
`Time` (`hh:mm:ss` ; time independent of a date)	`string`
`Duration` (corresponds to XML Schema `xs:duration` datatype)	`duration`
`XHTML`	Metadata type – not applicable
`KeyValues`	Metadata type – not applicable
`IdentifiableReference`	Metadata type – not applicable
`DataSetReference`	Metadata type – not applicable

Figure 24. Mappings from SDMX data types to VTL Basic Scalar Types

When VTL takes in input SDMX artefacts, it is assumed that a type conversion according to the table above always happens. In case a different VTL basic scalar type is desired, it can be achieved in the VTL program taking in input the default VTL basic scalar type above and applying to it the VTL type conversion features (see the implicit and explicit type conversion and the “cast” operator in the VTL Reference Manual).

Copyright © SDMx TWG 2020

Mapping VTL basic scalar types to SDMX data types

The following table describes the default conversion from the VTL basic scalar types to the SDMX data types .

VTL basic scalar type	Default SDMX data type (`BasicComponentDataType`)	Default output format
`String`	`String`	Like XML (`xs:string`)
`Number`	`Float`	Like XML (`xs:float`)
`Integer`	`Integer`	Like XML (`xs:int`)
`Date`	`DateTime`	`YYYY-MM-DDT00:00:00Z`
`Time`	`StandardTimePeriod`	`<date>/<date>` (as defined above)
`time_period`	`ReportingTimePeriod` (`StandardReportingPeriod`)	`YYYY-Pppp` (according to SDMX)
`Duration`	`Duration`	Like XML (`xs:duration`) `PnYnMnDTnHnMnS`
`Boolean`	`Boolean`	Like XML (`xs:boolean`) with the values `"true"` or `"false"`

Figure 25. Mappings from VTL Basic Scalar Types to SDMX data types

In case a different default conversion is desired, it can be achieved through the `CustomTypeScheme` and `CustomType` artefacts (see also the section Transformations and Expressions of the SDMX information model).

The custom output formats can be specified by means of the VTL formatting mask described in the section “Type Conversion and Formatting Mask” of the VTL Reference Manual. Such a section describes the masks for the VTL basic scalar types “number”, “integer”, “date”, “time”, “time_period” and “duration” and gives examples. As for the types “string” and “boolean” the VTL conventions are extended with some other special characters as described in the following table.

VTL special characters for the formatting masks:

Number:

VTL Special Character	Description
D	one numeric digit (if the scientific notation is adopted, D is only for the mantissa)
E	one numeric digit (for the exponent of the scientific notation)
. (dot)	possible separator between the integer and the decimal parts.
, (comma)	possible separator between the integer and the decimal parts.

Copyright © SDMx TWG 2020

Time and duration:

VTL Special Character	Description
C	century
Y	year
S	semester
Q	quarter
M	month
W	week
D	day
h	hour digit (by default on 24 hours)
M	minute
S	second
D	decimal of second
P	period indicator (representation in one digit for the duration)
P	number of the periods specified in the period indicator
AM/PM	indicator of AM / PM (e.g. am/pm for “am” or “pm”)
MONTH	uppercase textual representation of the month (e.g., JANUARY for January)
DAY	uppercase textual representation of the day (e.g., MONDAY for Monday)
Month	lowercase textual representation of the month (e.g., january)
Day	lowercase textual representation of the month (e.g., monday)
Month	First character uppercase, then lowercase textual representation of the month (e.g., January)
Day	First character uppercase, then lowercase textual representation of the day using (e.g. Monday)

String:

VTL Special Character	Description
X	any string character
Z	any string character from “A” to “z”
9	any string character from “0” to “9”

Boolean:

VTL Special Character	Description
B	Boolean using “true” for True and “false” for False
1	Boolean using “1” for True and “0” for False
0	Boolean using “0” for True and “1” for False

Copyright © SDMx TWG 2020

Other qualifiers:

VTL Special Character	Description
*	an arbitrary number of digits (of the preceding type)
+	at least one digit (of the preceding type)
( )	optional digits (specified within the brackets)
\	prefix for the special characters that must appear in the mask
N	fixed number of digits used in the preceding textual representation of the month or the day

The default conversion, either standard or customized, can be used to deduce automatically the representation of the components of the result of a VTL Transformation. In alternative, the representation of the resulting SDMX Dataflow can be given explicitly by providing its `DataStructureDefinition`. In other words, the representation specified in the DSD, if available, overrides any default conversion[40].

Null Values

In the conversions from SDMX to VTL it is assumed by default that a missing value in SDMX becomes a NULL in VTL. After the conversion, the NULLs can be manipulated through the proper VTL operators.

On the other side, the VTL programs can produce in output NULL values for Measures and Attributes (Null values are not allowed in the Identifiers). In the conversion from VTL to SDMX, it is assumed that a NULL in VTL becomes a missing value in SDMX.

In the conversion from VTL to SDMX, the default assumption can be overridden, separately for each VTL basic scalar type, by specifying which the value that represents the NULL in SDMX is. This can be specified in the attribute "`nullValue`" of the `CustomType` artefact (see also the section Transformations and Expressions of the SDMX information model). A `CustomType` belongs to a `CustomTypeScheme`, which can be referenced by one or more `TransformationScheme` (i.e. VTL programs). The overriding assumption is applied for all the SDMX Dataflows calculated in the `TransformationScheme`.

Format of the literals used in VTL Transformations

The VTL programs can contain literals, i.e. specific values of certain data types written directly in the VTL definitions or expressions. The VTL does not prescribe a specific format for the literals and leave the specific VTL systems and the definers of VTL Transformations free of using their preferred formats.

Given this discretion, it is essential to know which are the external representations adopted for the literals in a VTL program, in order to interpret them correctly. For example, if the external format for the dates is `YYYY-MM-DD` the date literal 2010-01-02 has the meaning of 2nd January 2010, instead if the external format for the dates is `YYYY-DD-MM` the same literal has the meaning of 1st February 2010.

Hereinafter, i.e. in the SDMX implementation of the VTL, it is assumed that the literals are expressed according to the "default output format" of the table of the previous paragraph ("Mapping VTL basic scalar types to SDMX data types") unless otherwise specified.

A different format can be specified in the attribute "`vtlLiteralFormat`" of the `CustomType` artefact (see also the section Transformations and Expressions of the SDMX information model).

Like in the case of the conversion of NULLs described in the previous paragraph, the overriding assumption is applied, for a certain VTL basic scalar type, if a value is found for the `vtlLiteralFormat` attribute of the `CustomType` of such VTL basic scalar type. The overriding assumption is applied for all the literals of a related VTL `TransformationScheme`.

In case a literal is operand of a VTL Cast operation, the format specified in the Cast overrides all the possible otherwise specified formats.

Copyright © SDMx TWG 2020

Copyright © SDMx TWG 2020

1. The Validation and Transformation Language is a standard language designed and published under the SDMX initiative. VTL is described in the VTL User and Reference Guides available on the SDMX website. ↩
2. In this chapter, in order to distinguish VTL and SDMX model artefacts, the VTL ones are written in the standard font while the SDMX ones in monospace font ↩
3. See also the section “VTL-DL Rulesets” in the VTL Reference Manual. ↩
4. The VTLMappings are used also for User Defined Operators (UDO). Although UDOs are envisaged to be defined on generic operands, so that the specific artefacts to be manipulated are passed as parameters at their invocation, it is also possible that an UDO invokes directly some specific SDMX artefacts. These SDMX artefacts have to be mapped to the corresponding aliases used in the definition of the UDO through the `VtlMappingScheme` and `VtlMapping` classes as well. ↩
5. For a complete description of the structure of the URN see the SDMX 2.1 Standards - Section 5 - Registry Specifications, paragraph 6.2.2 (“Universal Resource Name (URN)”). ↩
6. The container-object-id can repeat and may not be present. ↩
7. i.e., the artefact belongs to a maintainable class ↩
8. Since these references to SDMX objects include non-permitted characters as per the VTL ID notation, they need to be included between single quotes, according to the VTL rules for irregular names. ↩
9. For the syntax of the VTL operators see the VTL Reference Manual ↩
10. In case the invoked artefact is a VTL component, which can be invoked only within the invocation of a VTL data set (SDMX Dataflow), the specific SDMX class-name (e.g. `Dimension`, `TimeDimension`, `Measure` or `DataAttribute`) can be deduced from the data structure of the SDMX Dataflow, which the component belongs to. ↩
11. If the Agency is composite (for example `AgencyA.Dept1.Unit2`), the agency is considered different even if only part of the composite name is different (for example `AgencyA.Dept1.Unit3` is a different Agency than the previous one). Moreover the `agency-id` cannot be omitted in part (i.e., if a `TransformationScheme` owned by `AgencyA.Dept1.Unit2` references an artefact coming from `AgencyA.Dept1.Unit3`, the specification of the `agency-id` becomes mandatory and must be complete, without omitting the possibly equal parts like `AgencyA.Dept1`) ↩
12. Single quotes are needed because this reference is not a VTL regular name. ↩
13. Single quotes are not needed in this case because `CL_FREQ` is a VTL regular name. ↩
14. The result `DFR(1.0.0)` is be equal to `DF1(1.0.0)` save that the component `SECTOR` is called `SEC` ↩
15. Rulesets of this kind cannot be reused when the referenced Concept has a different representation. ↩
16. See also the section “VTL-DL Rulesets” in the VTL Reference Manual. ↩
17. If a calculated artefact is persistent, it needs a persistent definition, i.e. a SDMX definition in a SDMX environment. In addition, possible calculated artefact that are not persistent may require a SDMX definition, for example when the result of a non-persistent calculation is disseminated through SDMX tools (like an inquiry tool). ↩
18. See the VTL 2.0 User Manual ↩
19. See the SDMX Standards Section Information Model ↩
20. Besides the mapping between one SDMX `Dataflow` and one VTL Data Set, it is also possible to map distinct parts of a SDMX `Dataflow` to different VTL Data Set, as explained in a following paragraph. ↩
21. Here an SDMX observation is meant to correspond to one combination of values of the `DimensionComponents`. ↩
22. E.g., if in the data structure there exist 3 Dimensions `C,D,E` having the role of `MeasureDimension`, they should be considered as a joint `MeasureDimension Z=(C,D,E);` therefore when the description says “each possible value Cj of the `MeasureDimension` …” it means “each possible combination of values `(Cj, Dk, Ew)` of the joint `MeasureDimension Z=(C,D,E)`”. ↩
23. A typical example of this kind is the validation, and more in general the manipulation, of individual time series belonging to the same `Dataflow`, identifiable through the `DimensionComponents` of the `Dataflow` except the `TimeDimension`. The coding of these kind of operations might be simplified by mapping distinct time series (i.e. different parts of a SDMX `Dataflow`) to distinct VTL Data Sets. ↩
24. Please note that this kind of mapping is only an option at disposal of the definer of VTL Transformations; in fact it remains always possible to manipulate the needed parts of SDMX `Dataflow`s by means of VTL operators (e.g. “sub”, “filter”, “calc”, “union” …), maintaining a mapping one-to-one between SDMX `Dataflow`s and VTL Data Sets. ↩
25. This definition is made through the `ToVtlSubspace` and `ToVtlSpaceKey` classes and/or the `FromVtlSuperspace` and `FromVtlSpaceKey` classes, depending on the direction of the mapping (“key” means “dimension”). The mapping of `Dataflow` subsets can be applied independently in the two directions, also according to different Dimensions. When no Dimension is declared for a given direction, it is assumed that the option of mapping different parts of a SDMX `Dataflow` to different VTL Data Sets is not used. ↩
26. As a consequence of this formalism, a slash in the name of the VTL Data Set assumes the specific meaning of separator between the name of the `Dataflow` and the values of some of its Dimensions. ↩
27. This is the order in which the dimensions are defined in the `ToVtlSpaceKey` class or in the `FromVtlSpaceKey` class, depending on the direction of the mapping. ↩
28. It should be remembered that, according to the VTL consistency rules, a given VTL dataset cannot be the result of more than one VTL Transformation. ↩
29. If these `DimensionComponents` would not be dropped, the various VTL Data Sets resulting from this kind of mapping would have non-matching values for the Identifiers corresponding to the mapping Dimensions (e.g. `POPULATION` and `COUNTRY`). As a consequence, taking into account that the typical binary VTL operations at dataset level (`+`, `-`, `*`, `/` and so on) are executed on the observations having matching values for the identifiers, it would not be possible to compose the resulting VTL datasets one another (e.g. it would not be possible to calculate the population ratio between USA and CANADA). ↩
30. In case the ordered concatenation notation is used, the VTL Transformation described above, e.g. `'DF1(1.0)/POPULATION.USA' := DF1(1.0) [ sub INDICATOR="POPULATION", COUNTRY="USA"]`, is implicitly executed. In order to test the overall compliance of the VTL program to the VTL consistency rules, it has to be considered as part of the VTL program even if it is not explicitly coded. ↩
31. If the whole DF2(1.0) is calculated by means of just one VTL Transformation, then the mapping between the SDMX Dataflow and the corresponding VTL dataset is one-to-one and this kind of mapping (one SDMX Dataflow to many VTL datasets) does not apply. ↩
32. This is possible as each VTL dataset corresponds to one particular combination of values of INDICATOR and COUNTRY. ↩

Copyright © SDMx TWG 2020

33. The mapping dimensions are defined as `FromVtlSpaceKeys` of the `FromVtlSuperSpace` of the `VtlDataflowMapping` relevant to `DF2(1.0)`. ↩
34. the symbol of the VTL persistent assignment is used (`<-`) ↩
35. The result is persistent in this example but it can be also non persistent if needed. ↩
36. In case the ordered concatenation notation from VTL to SDMX is used, the set of `Transformations` described above is implicitly performed; therefore, in order to test the overall compliance of the VTL program to the VTL consistency rules, these implicit `Transformations` have to be considered as part of the VTL program even if they are not explicitly coded. ↩
37. Through SDMX Constraints, it is possible to specify the values that a Component of a Dataflow can assume. ↩
38. By using represented variables, VTL can assume that data structures having the same variables as identifiers can be composed one another because the correspondent values can match. ↩
39. A `Concept` becomes a `Component` in a `DataStructureDefinition`, and `Components` can have different `LocalRepresentations` in different `DataStructureDefinitions`, also overriding the (possible) base representation of the Concept. ↩
40. The representation given in the DSD should obviously be compatible with the VTL data type. ↩

Copyright © SDMx TWG 2020

5.13 Structure Mapping

5.13.1 Introduction

The purpose of SDMX structure mapping is to transform datasets from one dimensionality to another. In practice, this means that the input and output datasets conform to different Data Structure Definition.

Structure mapping does not alter the observation values and is not intended to perform any aggregations or calculations.

An input series maps to:

1. Exactly one output series; or
2. Multiple output series with different Series Keys, but the same observation values; or
3. Zero output series where no source rule matches the input Component values.

Typical use cases include:

- Transforming received data into a common internal structure;
- Transforming reported data into the data collector's preferred structure;
- Transforming unidimensional datasets[1] to multi-dimensional; and
- Transforming internal datasets with a complex structure to a simpler structure with fewer dimensions suitable for dissemination.

5.13.2 1-1 structure maps

1-1 (pronounced 'one to one') mappings support the simple use case where the value of a Component in the source structure is translated to a different value in the target, usually where different classification schemes are used for the same Concept.

In the example below, ISO 2-character country codes are mapped to their ISO 3-character equivalent.

Country	Alpha-2 code	Alpha-3 code
Afghanistan	AF	AFG
Albania	AL	ALB
Algeria	DZ	DZA
American Samoa	AS	ASM
Andorra	AD	AND
etc…		

Different source values can also map to the same target value, for example when deriving regions from country codes.

Source Component: `REF_AREA`	**Target Component:** `REGION`
FR	EUR
DE	EUR
IT	EUR
ES	EUR
BE	EUR

Copyright © SDMx TWG 2020

5.13.3 N-n structure maps

N-n (pronounced ‘N to N’) mappings describe rules where a specified combination of values in multiple source Components map to specified values in one or more target Components. For example, when mapping a partial Series Key from a highly multidimensional cube (like Balance of Payments) to a single ‘Indicator’ Dimension in a target Data Structure.

Example:

Rule	Source	Target
1	If `FREQUENCY=A` and `ADJUSTMENT=N` and `MATURITY=L`	Set `INDICATOR=A_N_L`
2	If `FREQUENCY=M` and `ADJUSTMENT=S_A1` and `MATURITY=TY12`	Set `INDICATOR=MON_SAX_12`

N-n rules can also set values for multiple source Components.

Rule	Source	Target
1	If `FREQUENCY=A` and `ADJUSTMENT=N` and `MATURITY=L`	Set `INDICATOR=A_N_L`, `STATUS=QXR15`, `NOTE="Unadjusted"`
2	If `FREQUENCY=M` and `ADJUSTMENT=S_A1` and `MATURITY=TY12`	Set `INDICATOR=MON_SAX_12`, `STATUS=MPM12`, `NOTE="Seasonally Adjusted"`

5.13.4 Ambiguous mapping rules

A structure map is ambiguous if the rules result in a dataset containing multiple series with the same Series Key.

A simple example mapping a source dataset with a single dimension to one with multiple dimensions is shown below:

Source	Target	Output Series Key
`SERIES_CODE=XMAN_Z_21`	`INDICATOR=XM`, `FREQ=A`, `ADJUSTMENT=N`, `UNIT_MEASURE=_Z`, `COMP_ORG=21`	`XM:A:N`
`SERIES_CODE=XMAN_Z_34`	`INDICATOR=XM`, `FREQ=A`, `ADJUSTMENT=N`, `UNIT_MEASURE=_Z`, `COMP_ORG=34`	`XM:A:N`

The above behaviour can be okay if the series `XMAN_Z_21` contains observations for different periods of time then the series `XMAN_Z_34`. If however both series contain observations for the same point in time, the output for this mapping will be two observations with the same series key, for the same period in time.

5.13.5 Representation maps

Representation Maps replace the SDMX 2.1 Codelist Maps and are used describe explicit mappings between source and target Component values.

The source and target of a Representation Map can reference any of the following:

1. Codelist
2. Free Text (restricted by type, e.g String, Integer, Boolean)
3. Valuelist

Copyright © SDMx TWG 2020

A Representation Map mapping ISO 2-character to ISO 3-character Codelists would take the following form:

CL_ISO_ALPHA2	CL_ISO_ALPHA3
AF	AFG
AL	ALB
DZ	DZA
AS	ASM
AD	AND
etc…	

A Representation Map mapping free text country names to an ISO 2-character Codelist could be similarly described:

Text	CL_ISO_ALPHA2
“Germany”	DE
“France”	FR
“United Kingdom”	GB
“Great Britain”	GB
“Ireland”	IE
“Eire”	IE
etc…	

Valuelists, introduced in SDMX 3.0, are equivalent to Codelists but allow the maintenance of non-SDMX identifiers. Importantly, their IDs do not need to conform to IDType, but as a consequence are not Identifiable.

When used in Representation Maps, Valuelists allow Non-SDMX identifiers containing characters like £, $, % to be mapped to Code IDs, or Codes mapped to non-SDMX identifiers.

In common with Codelists, each item in a Valuelist has a multilingual name giving it a human-readable label and an optional description. For example:

Value	Locale	Name
$	en	United States Dollar
%	En	Percentage
	fr	Pourcentage

Other characteristics of Representation Maps:

- Support the mapping of multiple source Component values to multiple Target Component values as described in the section on n-to-n mappings; this covers also the case of mapping an Attribute with an array representation to map combinations of values to a single target value;
- Allow source or target mappings for an Item to be optional allowing rules such as ‘A maps to nothing’ or ‘nothing maps to A’; and
- Support for mapping rules where regular expressions or substrings are used to match source Component values. Refer to the Section Regular expression and substring rule.

Copyright © SDMx TWG 2020

5.13.6 Regular expression and substring rules

It is common for classifications to contain meanings within the identifier, for example the code Id `'XULADS'` may refer to a particular seasonality because it starts with the letters `XU`.

With SDMX 2.1 each code that starts with `XU` had to be individually mapped to the same seasonality, and additional mappings added when new `Codes` were added to the `Codelists`. This led to many hundreds or thousands of mappings which can be more efficiently summarised in a single conceptual rule:

```
If starts with 'XU' map to 'Y'
```

These rules are described using either regular expressions, or substrings for simpler use cases.

Regular expressions

Regular expression mapping rules are defined in the Representation Map.

Below is an example set of regular expression rules for a particular component.

Regex	Description	Output
`A`	Rule match if `input = 'A'`	`OUT_A`
`^[A-G]`	Rule match if the input starts with letters A to G	`OUT_B`
`A | B`	Rule match if input is either ‘A’ or ‘B’	`OUT_C`

Like all mapping rules, the output is either a Code, a Value or free text depending on the representation of the Component in the target Data Structure Definition.

If the regular expression contains capture groups, these can be used in the definition of the output value, by specifying `\n` as an output value where `n` is the number of the capture group starting from 1. For example

Regex	Target output	Example Input	Example Output
`([0-9]{4})[0-9]([0-9]{1})`	`\1-Q\2`	200933	2009-Q3

As regular expression rules can be used as a general catch-all if nothing else matches, the ordering of the rules is important. Rules should be tested starting with the highest priority, moving down the list until a match is found.

The following example shows this:

Priority	Regex	Description	Output
1	`A`	Rule match if `input = 'A'`	`OUT_A`
2	`B`	Rule match if `input = 'B'`	`OUT_B`
3	`[A-Z]`	Any character `A-Z`	`OUT_C`

The input `'A'` matches both the first and the last rule, but the first takes precedence having the higher priority. The output is `OUT_A`.

The input `'G'` matches on the last rule which is used as a catch-all or default in this example.

Substrings

Substrings provide an alternative to regular expressions where the required section of an input value can be described using the number of the starting character, and the length of the substring in characters. The first character is at position 1.

Copyright © SDMx TWG 2020

For instance:

Input String	Start	Length	Output
`ABC_DEF_XYZ`	5	3	`DEF`
`XULADS`	1	2	`XU`

Sub-strings can therefore be used for the conceptual rule `If starts with 'XU' map to Y` as shown in the following example:

Start	Length	Source	Target
1	2	`XU`	`Y`

5.13.7 Mapping non-SDMX time formats to SDMX formats

Structure mapping allows non-SDMX compliant time values in source datasets to be mapped to an SDMX compliant time format.

Two types of time input are defined:

1. **Pattern based dates** – a string which can be described using a notation like `dd/mm/yyyy` or is represented as the number of periods since a point in time, for example: `2010M001` (first month in 2010), or `2014D123` (123rd day in 2014); and
2. **Numerical based datetime** – a number specifying the elapsed periods since a fixed point in time, for example Unix Time is measured by the number of milliseconds since 1970.

The output of a time-based mapping is derived from the output Frequency, which is either explicitly stated in the mapping or defined as the value output by a specific Dimension or Attribute in the output mapping. If the output frequency is unknown or if the SDMX format is not desired, then additional rules can be provided to specify the output date format for the given frequency Id. The default rules are:

Frequency	Format	Example
A	`YYYY`	2010
D	`YYYY-MM-DD`	2010-01-01
I	`YYYY-MM-DD-Thh:mm:ss`	2010-01T20:22:00
M	`YYYY-MM`	2010-01
Q	`YYYY-Qn`	2010-Q1
S	`YYYY-Sn`	2010-S1
T	`YYYY-Tn`	2010-T1
W	`YYYY-Wn`	YYYY-W53

In the case where the input frequency is lower than the output frequency, the mapping defaults to end of period, but can be explicitly set to start, end or mid-period.

There are two important points to note:

1. The output frequency determines the output date format, but the default output can be redefined using a Frequency Format mapping to force explicit rules on how the output time period is formatted.
2. To support the use case of changing frequency the structure map can optionally provide a start of year attribute, which defines the year start date in MM-DD format. For example: YearStart=04-01.

Copyright © SDMx TWG 2020

Pattern based dates

Date and time formats are specified by date and time pattern strings based on Java's Simple Date Format. Within date and time pattern strings, unquoted letters from `'A'` to `'Z'` and from `'a'` to `'z'` are interpreted as pattern letters representing the components of a date or time string. Text can be quoted using single quotes `(')` to avoid interpretation. `"''"` represents a single quote. All other characters are not interpreted; they're simply copied into the output string during formatting or matched against the input string during parsing.

Due to the fact that dates may differ per locale, an optional property, defining the locale of the pattern, is provided. This would assist processing of source dates, according to the given locale[2]. An indicative list of examples is presented in the following table:

English (en)	Australia (AU)	en-AU
English (en)	Canada (CA)	en-CA
English (en)	United Kingdom (GB)	en-GB
English (en)	United States (US)	en-US
Estonian (et)	Estonia (EE)	et-EE
Finnish (fi)	Finland (FI)	fi-FI
French (fr)	Belgium (BE)	fr-BE
French (fr)	Canada (CA)	fr-CA
French (fr)	France (FR)	fr-FR
French (fr)	Luxembourg (LU)	fr-LU
French (fr)	Switzerland (CH)	fr-CH
German (de)	Austria (AT)	de-AT
German (de)	Germany (DE)	de-DE
German (de)	Luxembourg (LU)	de-LU
German (de)	Switzerland (CH)	de-CH
Greek (el)	Cyprus (CY)	el-CY
Greek (el)	Greece (GR)	el-GR
Hebrew (iw)	Israel (IL)	iw-IL
Hindi (hi)	India (IN)	hi-IN
Hungarian (hu)	Hungary (HU)	hu-HU
Icelandic (is)	Iceland (IS)	is-IS
Indonesian (in)	Indonesia (ID)	in-ID
Irish (ga)	Ireland (IE)	ga-IE
Italian (it)	Italy (IT)	it-IT

Copyright © SDMx TWG 2020

Examples

- `22/06/1981` would be described as `dd/MM/YYYY`, with locale `en-GB`
- `2008-mars-12` would be described as `YYYY-MMM-DD`, with locale `fr-FR`
- `22 July 1981` would be described as `dd MMMM YYYY`, with locale `en-US`
- `22 Jul 1981` would be described as `dd MMM YYYY`
- `2010 D62` would be described as `YYYYDnn` (day 62 of the year 2010)

The following pattern letters are defined (all other characters from `'A'` to `'Z'` and from `'a'` to `'z'` are reserved):

Letter	Date or Time Component	Presentation	Examples
`G`	Era designator	Text	AD
`yy`	Year short (upper case is Year of Week [1])	Year	96
`yyyy`	Year Full (upper case is Year of Week)	Year	1996
`MM`	Month number in year starting with 1	Month	07
`MMM`	Month name short	Month	Jul
`MMMM`	Month name full	Month	July
`ww`	Week in year	Number	27
`W`	Week in month	Number	2
`DD`	Day in year	Number	189
`dd`	Day in month	Number	10
`F`	Day of week in month	Number	2
`E`	Day name in week	Text	Tuesday; Tue
`U`	Day number of week (1 = Monday, ..., 7 = Sunday)	Number	1
`HH`	Hour in day (0-23)	Number	0
`kk`	Hour in day (1-24)	Number	24
`KK`	Hour in am/pm (0-11)	Number	0
`hh`	Hour in am/pm (1-12)	Number	12
`mm`	Minute in hour	Number	30
`ss`	Second in minute	Number	55
`S`	Millisecond	Number	978
`n`	Number of periods, used after a SDMX Frequency Identifier such as M, Q, D (month, quarter, day)	Number	12

The model is illustrated below:

Copyright © SDMx TWG 2020

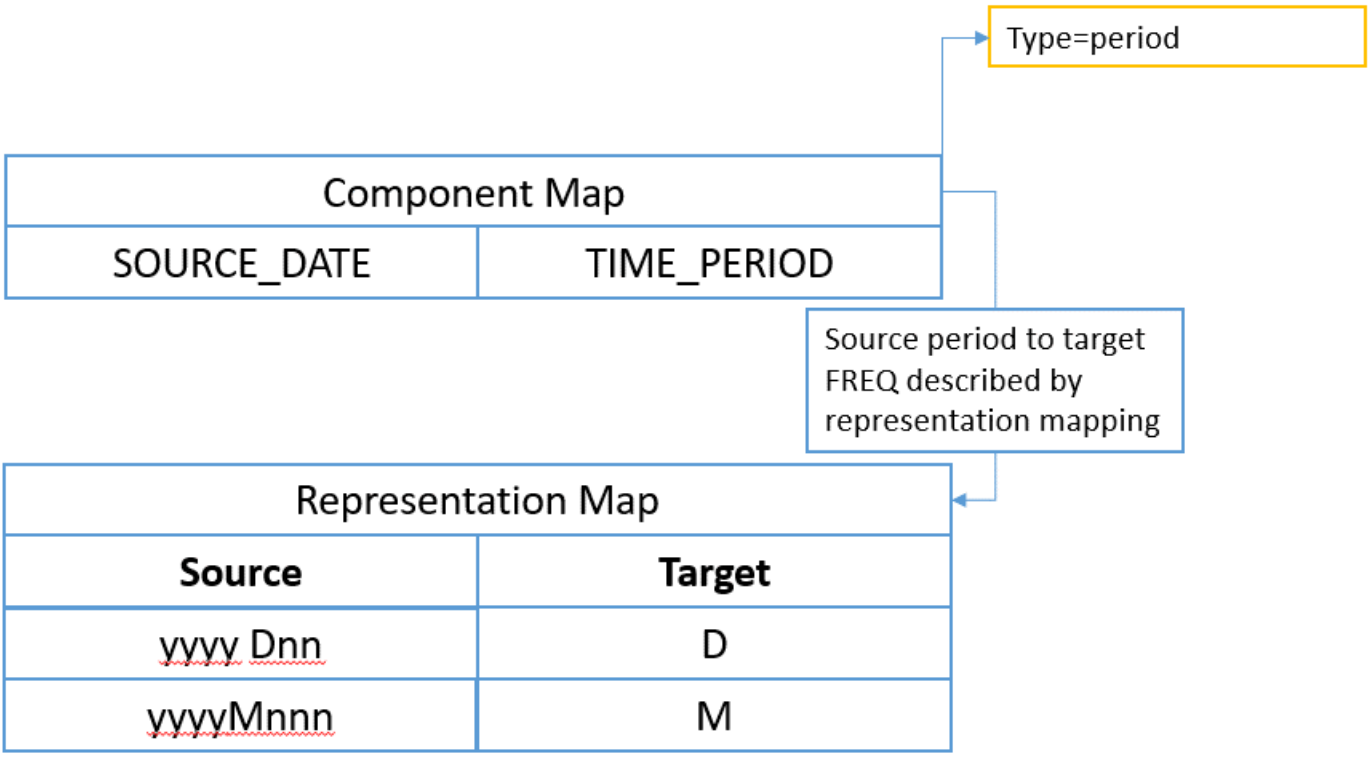


Figure 24. showing the component map mapping the `SOURCE_DATE` Dimension to the `TIME_PERIOD` dimension with the additional information on the component map to describe the time format

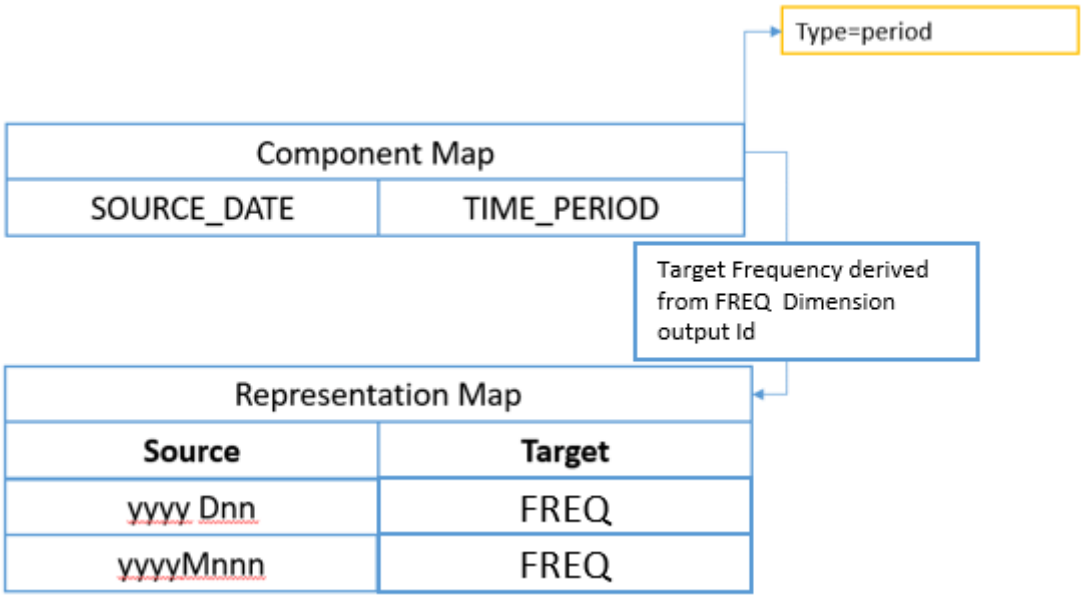


Figure 25. showing an input date format, whose output frequency is derived from the output value of the `FREQ` Dimension

Copyright © SDMx TWG 2020

Numerical based datetime

Where the source datetime input is purely numerical, the mapping rules are defined by the **Base** as a valid SDMX Time Period, and the **Period** which must take one of the following enumerated values:

- day
- second
- millisecond
- microsecond
- nanosecond

Numerical datetime systems	Base	Period
Epoch Time (UNIX): Milliseconds since 01 Jan 1970	1970	millisecond
Windows System Time: Milliseconds since 01 Jan 1601	1601	millisecond

The example above illustrates numerical based datetime mapping rules for two commonly used time standards.

The model is illustrated below:

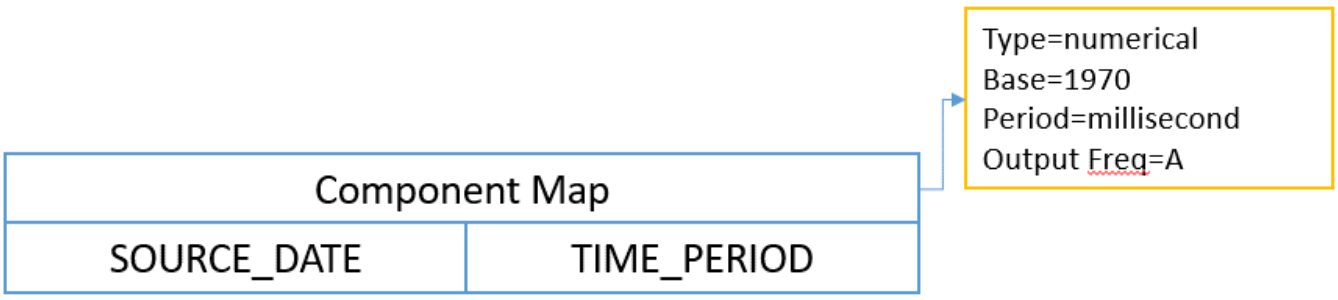


Figure 26. showing the component map mapping the `SOURCE_DATE` Dimension to the `TIME_PERIOD` Dimension with the additional information on the component map to describe the numerical datetime system in use

Mapping more complex time inputs

VTL should be used for more complex time inputs that cannot be interpreted using the pattern based on numerical methods.

5.13.8 Using `TIME_PERIOD` in mapping rules

The source `TIME_PERIOD` Dimension can be used in conjunction with other input Dimensions to create discrete mapping rules where the output is conditional on the time period value.

Copyright © SDMx TWG 2020

The main use case is setting the value of Observation Attributes in the target dataset.

Rule	Source	Target
Rule	Source	Target
:—	:—	:—
1	If `INDICATOR=XULADS` and `TIME_PERIOD=2007`	Set `OBS_CONF=F`
2	If `INDICATOR=XULADS` and `TIME_PERIOD=2008`	Set `OBS_CONF=F`
3	If `INDICATOR=XULADS` and `TIME_PERIOD=2009`	Set `OBS_CONF=F`
4	If `INDICATOR=XULADS` and `TIME_PERIOD=2010`	Set `OBS_CONF=C`

In the example above, `OBS_CONF` is an Observation Attribute.

5.13.9 Time span mapping rules using validity periods

Creating discrete mapping rules for each `TIME_PERIOD` is impractical where rules need to cover a specific span of time regardless of frequency, and for high-frequency data.

Instead, an optional validity period can be set for each mapping.

By specifying validity periods, the example from Section 13.8 can be re-written using two rules as follows:

Rule	Source	Target
1	If `INDICATOR=XULADS`, validity period: `start=2007`, `end=2009`	Set `OBS_CONF=F`
2	If `INDICATOR=XULADS`, validity period: `start=2010`	Set `OBS_CONF=F`

In Rule 1, start period resolves to the start of the 2007 period (`2007-01-01T00:00:00`), and the end period resolves to the very end of 2009 (`2009-12-31T23:59:59`). The rule will hold true regardless of the input data frequency. Any observations reporting data for the Indicator `XULADS` that fall into that time range will have an `OBS_CONF` value of `F`.

In Rule 2, no end period is specified so remains in effect from the start of the period (`2010-01-01T00:00:00`) until the end of time. Any observations reporting data for the Indicator `XULADS` that fall into that time range will have an `OBS_CONF` value of `C`.

5.13.10 Mapping examples

Many to one mapping (N-1)

Source	Map To
`FREQ="A", ADJUSTMENT="N", REF_AREA="PL", COUNTERPART_AREA="W0", REF_SECTOR="S1", COUNTERPART_SECTOR="S1", ACCOUNTING_ENTRY="B", STO="B5G"`	`FREQ="A", REF_AREA="PL", COUNTERPART_AREA="W0", INDICATOR="IND_ABC"`

The Dimensions `FREQ`, `REF_AREA`, and `COUNTERPART_AREA` map from source to target verbatim. The mapping simply specifies:

```
FREQ => FREQ
REF_AREA => REF_AREA
COUNTERPART_AREA => COUNTERPART_AREA
```

No Representation Mapping is required. The source value simply copies across unmodified.

The remaining Dimensions all map to the Indicator Dimension. This is an example of many Dimensions mapping to one Dimension. In this case a Representation Mapping is required, and the mapping first describes the input ‘partial key’ and how this maps to the target indicator:

```
N:S1:S1:B:B5G => IND_ABC
```


Copyright © SDMx TWG 2020

Where the key sequence is based on the order specified in the mapping (i.e `ADJUSTMENT`, `REF_SECTOR`, etc will result in the first value `N` being taken from `ADJUSTMENT` as this was the first item in the source Dimension list.

> **Note**
>
> The key order is NOT based on the Dimension order of the DSD, as the mapping needs to be resilient to the DSD changing.

Mapping other data types to Code Id

In the case where the incoming data type is not a string and not a code identifier i.e. the source Dimension is of type Integer and the target is `Codelist`. This is supported by the `RepresentationMap`. The `RepresentationMap` source can reference a `Codelist`, `Valuelist`, or be free text, the free text can include regular expressions.

The following representation mapping can be used to explicitly map each age to an output code.

Source Input (Free Text)	Desired Output (Code Id)
0	A
1	A
2	A
3	B
4	B

If this mapping takes advantage of regular expressions it can be expressed in two rules:

Regular Expression	Desired Output
`[0-2]`	A
`[3-4]`	B

Observation Attributes for Time Period

This use case is where a specific observation for a specific time period has an attribute value.

Input INDICATOR	Input TIME_PERIOD	Output OBS_CONF
`XULADS`	2008	C
`XULADS`	2009	C
`XULADS`	2010	C

Or using a validity period on the Representation Mapping:

Input INDICATOR	Valid From/ Valid To	Output OBS_CONF
`XULADS`	2008/2010	C

Time mapping

This use case is to create a time period from an input that does not respect SDMX Time Formats.

Copyright © SDMx TWG 2020

The Component Mapping from `SYS_TIME` to `TIME_PERIOD` specifies itself as a time mapping with the following details:

Source Value	Source Mapping	Target Frequency	Output
18/07/1981	`dd/MM/yyyy`	A	1981

When the target frequency is based on another target Dimension value, in this example the value of the `FREQ` Dimension in the target DSD.

Source Value	Source Mapping	Target Frequency Dimension	Output
18/07/1981	`dd/MM/yyyy`	`FREQ`	1981-07-18 (when `FREQ=D`)

When the source is a numerical format

Source Value	Start Period	Interval	Target `FREQ`	Output
1589808220	1970	millisecond	M	2020-05

When the source frequency is lower than the target frequency additional information can be provided for resolve to start of period, end of period, or mid period, as shown in the following example:

Source Value	Source Mapping	Target Frequency Dimension	Output
1981	`yyyy`	D – End of Period	1981-12-31

When the start of year is April 1st the Structure Map has `YearStart=04-01`:

Source Value	Source Mapping	Target Frequency Dimension	Output
1981	`yyyy`	D – End of Period	1982-03-31

---

1. `yyyy` represents the calendar year while `YYYY` represents the year of the week, which is only relevant for 53 week years ↩↩
2. A list of commonly used locales can be found in the Java supported locales ↩

Copyright © SDMx TWG 2020

5.14 ANNEX Semantic Versioning

5.14.1 Introduction to Semantic Versioning

In the world of versioned data modelling exists a dreaded place called “dependency hell.” The bigger your data model through organisational, national or international harmonisation grows and the more artefacts you integrate into your modelling, the more likely you are to find yourself, one day, in this pit of despair.

In systems with many dependencies, releasing new artefact versions can quickly become a nightmare. If the dependency specifications are too tight, you are in danger of version lock (the inability to upgrade an artefact without having to release new versions of every dependent artefact). If dependencies are specified too loosely, you will inevitably be bitten by version promiscuity (assuming compatibility with more future versions than is reasonable). Dependency hell is where you are when version lock and/or version promiscuity prevent you from easily and safely moving your data modelling forward.

As a very successful solution to the similar problem in software development, “Semantic Versioning” semver.org proposes a simple set of rules and requirements that dictate how version numbers are assigned and incremented. These rules make also perfect sense in the world of versioned data modelling and help to solve the “dependency hell” encountered with previous versions of SDMX. SDMX 3.0 applies thus the Semantic Versioning rules on all versioned SDMX artefacts. Once you release a versioned SDMX artefact, you communicate changes to it with specific increments to your version number.

This SDMX 3.0(.0) specification inherits the original semver.org 2.0.0 wording (license: Creative Commons - CC BY 3.0) and applies it to versioned SDMX structural artefacts. Under this scheme, version numbers and the way they change convey meaning about the underlying data structures and what has been modified from one version to the next.

5.14.2 Semantic Versioning Specification for SDMX 3.0(.0)

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119.

In the following, “versioned” artefacts are understood to be semantically versioned SDMX structural artefacts, and X, Y, Z and EXT are understood as placeholders for the version parts MAJOR, MINOR, PATCH, and EXTENSION, as defined in chapter 4.3.

Copyright © SDMx TWG 2020

The following rules apply to versioned artefacts:

- All versioned SDMX artefacts MUST specify a version number.
- The version number of immutable versioned SDMX artefacts MUST take the form `X.Y.Z` where `X`, `Y`, and `Z` are non-negative integers and MUST NOT contain leading zeroes. `X` is the MAJOR version, `Y` is the MINOR version, and `Z` is the PATCH version. Each element MUST increase numerically. For instance: `1.9.0 → 1.10.0 → 1.11.0`.
- Once an SDMX artefact with an `X.Y.Z` version has been shared externally or publicly released, the contents of that version MUST NOT be modified. That artefact version is considered stable. Any modifications MUST be released as a new version.
- MAJOR version zero `(0.y.z)` is for initial modelling. Anything MAY change at any time. The externally released or public artefact SHOULD NOT be considered stable.
- Version `1.0.0` defines the first stable artefact. The way in which the version number is incremented after this release is dependent on how this externally released or public artefact changes.
- PATCH version `Z (x.y.Z | x > 0)` MUST be incremented if only backwards compatible property changes are introduced. A property change is defined as an internal change that does not affect the relationship to other artefacts. These are changes in the artefact's or artefact element's names, descriptions and annotations that MUST NOT alter their meaning.
- MINOR version `Y (x.Y.z | x > 0)` MUST be incremented if a new, backwards compatible element is introduced to a stable artefact. These are additional items in ItemScheme artefacts. It MAY be incremented if substantial new information is introduced within the artefact's properties. It MAY include PATCH level changes. PATCH version MUST be reset to 0 when MINOR version is incremented.
- MAJOR version `X (X.y.z | X > 0)` MUST be incremented if any backwards incompatible changes are introduced to a stable artefact. These often relate to deletions of items in ItemSchemes or to backwards incompatibility introduced due to changes in references to other artefacts. A MAJOR version change MAY also include MINOR and PATCH level changes. PATCH and MINOR version MUST be reset to 0 when MAJOR version is incremented.
- A mutable version, e.g. used for externally released or public drafts or as pre-release, MUST be denoted by appending an EXTENSION that consists of a hyphen and a series of dot separated identifiers immediately following the PATCH version `(x.y.z-EXT)`. Identifiers MUST comprise only ASCII alphanumerics and hyphen `[0-9A-Za-z-]`. Identifiers MUST NOT be empty. Numeric identifiers MUST NOT include leading zeroes. However, to foster harmonisation and general comprehension it is generally recommended to use the standard EXTENSION `-draft`. Extended versions have a lower precedence than the associated stable version. An extended version indicates that the version is unstable and it might not satisfy the intended compatibility requirements as denoted by its associated stable version. When making changes to an SDMX artefact with an extended version number then one is not required to increment the version if those changes are kept within the allowed scope of the version increment from the previous version (if that existed), otherwise also here the before mentioned version increment rules for `X.Y.Z` apply. Concretely, a version X.0.0-EXT will allow for any changes, a version `X.Y.0-EXT` will allow only for MINOR changes and a version `X.Y.Z-EXT` will allow only for any PATCH changes, as defined above. EXTENSION examples: `1.0.0-draft`, `1.0.0-draft.1`, `1.0.0-0.3.7`, `1.0.0-x.7.z.92`.
- Precedence refers to how versions are compared to each other when ordered. Precedence MUST be calculated by separating the version into MAJOR, MINOR, PATCH and EXTENSION identifiers in that order. Precedence is determined by the first difference when comparing each of these identifiers from left to right as follows: MAJOR, MINOR, and PATCH versions are always compared numerically. Example: `1.0.0 < 2.0.0 < 2.1.0 < 2.1.1`. When MAJOR, MINOR, and PATCH are equal, an extended version has lower precedence than a stable version. Example: `1.0.0-draft < 1.0.0`. Precedence for two extended versions with the same MAJOR, MINOR, and PATCH version MUST be determined by comparing each dot separated identifier from left to right until a difference is found as follows: identifiers consisting of only digits are compared numerically and identifiers with letters or hyphens are compared lexically in ASCII sort order. Numeric identifiers always have lower precedence than non-numeric identifiers. A larger set of EXTENSION fields has a higher precedence than a smaller set, if all of the preceding identifiers are equal. Example: `1.0.0-draft < 1.0.0-draft.1 < 1.0.0-draft.prerelease < 1.0.0-prerelease < 1.0.0-prerelease.2 < 1.0.0-prerelease.11 < 1.0.0-rc.1 < 1.0.0`.
- The reasons for version changes MAY be documented in brief form in an artefact's annotation of type "CHANGELOG".

5.14.3 Backus–Naur Form Grammar for Valid SDMX 3.0(.0) Semantic Versions

```
<valid semver> ::= <version core>
                 | <version core> "-" <extension>

<version core> ::= <major> "." <minor> "." <patch>
```


Copyright © SDMx TWG 2020

```
<major> ::= <numeric identifier>
<minor> ::= <numeric identifier>
<patch> ::= <numeric identifier>

<extension> ::= <dot-separated extension identifiers>

<dot-separated extension identifiers> ::= <extension identifier>
                                        | <extension identifier> "." <dot-separated extension identifiers>

<extension identifier> ::= <alphanumeric identifier>
                         | <numeric identifier>**

<alphanumeric identifier> ::= <non-digit>
                            | <non-digit> <identifier characters>
                            | <identifier characters> <non-digit>
                            | <identifier characters> <non-digit> <identifier characters>

<numeric identifier> ::= "0"
                       | <positive digit>
                       | <positive digit> <digits>

<identifier characters> ::= <identifier character>
                          | <identifier character> <identifier characters>

<identifier character> ::= <digit>
                         | <non-digit>

<non-digit> ::= <letter>
              | "-"

<digits> ::= <digit>
           | <digit> <digits>

<digit> ::= "0"
          | <positive digit>

<positive digit> ::= "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

<letter> ::= "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P" | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X" |
"Y" | "Z" | "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h" | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p" | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x" |
"y" | "z"
```


Copyright © SDMx TWG 2020

5.14.4 Dependency Management in SDMX 3.0(.0)

MAJOR, MINOR or PATCH version parts in SDMX 3.0 artefact references CAN be wildcarded using “+” as extension:

- `X+.Y.Z` means the currently latest available version `>= X.Y.Z`
  - Example: `2+.3.1` means the currently latest available version `>= 2.3.1` (even if not backwards compatible)
  - Typical use case: references in SDMX Categorisations
- `X.Y+.Z` means the currently latest available backwards compatible version `>= X.Y.Z`
  - Example: `2.3+.1` means the currently latest available version `>= 2.3.1 and 3.0.0` (all backwards compatible versions `>= 2.3.1`)
  - Typical use case: references in SDMX DSD
- `X.Y.Z+` means the currently latest available forwards and backwards compatible version `>= X.Y.Z`
  - Example: `2.3.1+` means the currently latest available version `>= 2.3.1 and < 2.4.0` (all forwards and backwards compatible versions `>= 2.3.1`)
- Non-versioned and 2-digit version SDMX structural artefacts CAN reference any other non-versioned or versioned (whether SemVer or not) SDMX structural artefacts.
- Semantically versioned artefacts MUST only reference other semantically versioned artefacts.
- Wildcarded references in a stable artefact implicitly target only future stable versions of the referenced artefacts within the defined wildcard scope.
  - Example: The reference to `AGENCY_ID:CODELIST_ID(2.3+.1)` in an artefact `AGENCY_ID:DSD_ID(2.2.1)` resolves to artefact `AGENCY_ID:CODELIST_ID(2.4.3)` if that was currently the latest available stable version.
- Wildcarded references in a version-extended artefact implicitly target future stable and version-extended versions of the referenced artefacts within the defined wildcard scope.
  - Example: The reference to `AGENCY_ID:CODELIST_ID(2.3+.1)` in an artefact `AGENCY_ID:DSD_ID(2.2.1-draft)` resolves to artefact `AGENCY_ID:CODELIST_ID(2.5.0-draft)` if that was currently the latest available version.
- References to specific version-extended artefacts MAY be used, but those cannot be combined with a wildcard.
  - Example: The reference to `AGENCY_ID:CODELIST_ID(2.5.0-draft)` in an artefact `AGENCY_ID:DSD_ID(2.2.1)` resolves to artefact `AGENCY_ID:CODELIST_ID(2.5.0-draft)`, which might be subject to continued backwards compatible changes.

Because both, wildcarded references and references to version-extended artefacts, allow for changes in the referenced artefacts, care needs to be taken when choosing the appropriate references in order to achieve the required limitation in the allowed scope of changes.

For references to non-dependent artefacts, MAJOR, MINOR or PATCH version parts in SDMX 3.0 artefact references CAN alternatively be wildcarded using “*” as replacement:

* means all available versions

5.14.5 Upgrade and conversions of artefacts defined with previous SDMX standard versions to Semantic Versioning

Because SDMX standardises the interactions between statistical systems, which cannot all be upgraded at the same time, the new versioning rules cannot be applied to existing artefacts in EDIFACT, SDMX 1.0, 2.0 or 2.1. SemVer can only be applied to structural artefacts that are newly modelled with the SDMX 3.0 Information Model. Migrating to SemVer means migrating to the SDMX 3.0 Information Model, to its new API version and new versions of its exchange message formats.

To migrate SDMX structural artefacts created previously to SDMX 3.0.0:

If the artefacts do not need versioning, then the new artefacts based on the SDMX 3.0 Information Model SHOULD remain as-is, e.g., a previous artefact with the non-final version 1.0 and that doesn’t need versioning becomes non-versioned, i.e., keeps version 1.0. This will be the case for all AgencyScheme artefacts.

Copyright © SDMx TWG 2020

If artefact versioning is required and SDMX 3.0.0 Semantic Versioning is available within the tools and processes used, then it is recommended to switch to Semantic Versioning with the following steps:

1. Complement the missing version parts with 0s to make the version number SemVer-compliant using the MAJOR.MINOR.PATCH-EXTENSION syntax:

   Example: Version 2 becomes version 2.0.0 and version 3.1 becomes version 3.1.0.

2. Replace the “isFinal=false” property by the version extensions “-draft” (or alternatively “-unstable” or “-nonfinal” depending on the use case).

   Example: Version 1.3 with isFinal=true becomes version 1.3.0 and version 1.3 with isFinal=false becomes version 1.3.0-draft.

If artefact versioning is required but semantic versioning cannot be applied, then version strings used in previous versions of the Standard (e.g., version=1.2) may continue to be used.

Note: Like for other not fully backwards compatible SDMX 3.0 features, also some cases of semantically versioned SDMX 3.0 artefacts cannot be converted back to earlier SDMX versions. This is the case when one or more extensions have been created in parallel to the corresponding stable version. In this case, only the stable version SHOULD be converted to a final version (e.g., 3.2.1 becomes 3.2.1 final, and 3.2.1-draft cannot be converted back).

5.14.6 FAQ for Semantic Versioning

My organisation is new to SDMX and starts to implement 3.0 or starts to implement a new process fully based on SDMX 3.0. Which versioning scheme should be used?

If all counterparts involved in the process and all tools used for its implementation are SDMX 3.0-ready, then it is recommended to:

- in general, use semantic versioning;
- exceptionally, do not use versioning for artefacts that do not require it, e.g. artefacts that never change, that are only used internally or for which communication on changes with external parties or systems is not required.

How should I deal with revisions in the 0.y.z initial modelling phase?

The simplest thing to do is start your initial modelling release at 0.1.0 and then increment the minor version for each subsequent release.

How do I know when to release 1.0.0?

If your data model is being used in production, it should probably already be 1.0.0. If you have a stable artefact on which users have come to depend, you should be 1.0.0. If you’re worrying a lot about backwards compatibility, you should probably already be 1.0.0.

Doesn’t this discourage rapid modelling and fast iteration?

Major version zero is all about rapid modelling. If you’re changing the artefact every day you should either still be in version 0.y.z or on the next (minor or) major version for a separate modelling.

If even the tiniest backwards incompatible changes to the public artefact require a major version bump, won’t I end up at version 42.0.0 very rapidly?

This is a question of responsible modelling and foresight. Incompatible changes should not be introduced lightly to a data model that has a lot of dependencies. The cost that must be incurred to upgrade can be significant. Having to bump major versions to release incompatible changes means you will think through the impact of your changes, and evaluate the cost/benefit ratio involved.

Documenting the version changes in an artefact’s annotation of type “CHANGELOG” is too much work!

It is your responsibility as a professional modeller to properly document the artefacts that are intended for use by others. Managing data model complexity is a hugely important part of keeping a project efficient, and that’s hard to do if nobody knows how to use your data model, or what artefacts are safe to reuse. In the long run, SDMX 3.0 Semantic Versioning can keep everyone and everything running smoothly.

Copyright © SDMx TWG 2020

However, refrain from overdoing. Nobody can and will read too long lists of changes. Thus, keep it to the absolute essence, and mainly use it for short announcements. You can even skip the changelog if the change is impact-less. For all complete reports, a new API feature could be more useful to automatically generate a log of differences between two versions.

What do I do if I accidentally release a backwards incompatible change as a minor version?

As soon as you realise that you've broken the SDMX 3.0 Semantic Versioning specification, fix the problem and release a new minor version that corrects the problem and restores backwards compatibility. Even under this circumstance, it is unacceptable to modify versioned releases. If it's appropriate, document the offending version and inform your users of the problem so that they are aware of the offending version.

What if I inadvertently alter the public artefact in a way that is not compliant with the version number change (i.e. the modification incorrectly introduces a major breaking change in a patch release)?

Use your best judgement. If you have a huge audience that will be drastically impacted by changing the behaviour back to what the public artefact intended, then it may be best to perform a major version release, even though the property change could strictly be considered a patch release. Remember, SDMX 3.0.0 Semantic Versioning is all about conveying meaning by how the version number changes. If these changes are important to your users, use the version number to inform them.

How should I handle deprecating elements?

Deprecating existing elements is a normal part of data modelling and is often required to make forward progress or follow history (changing classifications, evolving reference areas). When you deprecate part of your stable artefact, you should issue a new minor version with the deprecation in place (e.g. add the new country code but still keep the old country code) and with a "CHANGELOG" annotation announcing the deprecation (e.g. the intention to remove the old country code in a future version) . Before you completely remove the functionality in a new major release there should be at least one minor release that contains the deprecation so that users can smoothly transition to the new artefact.

Does SDMX 3.0.0 Semantic Versioning have a size limit on the version string?

No, but use good judgement. A 255 character version string is probably overkill, for example. In addition, specific SDMX implementations may impose their own limits on the size of the string. Remember, it is generally recommended to use the standard extension "-draft".

Is "v1.2.3" a semantic version?

No, `v1.2.3` is not a semantic version. The semantic version is `1.2.3`.

Is there a suggested regular expression (RegEx) to check an SDMX 3.0.0 Semantic Versioning string?

There are two:

One with named groups for those systems that support them (PCRE [Perl Compatible Regular Expressions, i.e. Perl, PHP and R], Python and Go).

Reduced version (without original SemVer "build metadata") from: https://regex101.com/r/Ly7O1x/3/

```
^(?P&lt;major>0|\[1-9\]\d\*)\\(?P&lt;minor>0|\[1-9\]\d\*)\\(?P&lt;patch>0|\[1-9\]\d\*)(?:-(?P&lt;extension>(?:0|\[1-9\]\d\*|\d\*\[a-zA-Z-\]\[0-9a-zA-Z-\]\*)
(?:\\(?:0|\[1-9\]\d\*|\d\*\[a-zA-Z-\]\[0-9a-zA-Z-\]\*))\*))?$
```

And one with numbered capture groups instead (so cg1 = major, cg2 = minor, cg3 = patch and cg4 = extension) that is compatible with ECMA Script (JavaScript), PCRE (Perl Compatible Regular Expressions, i.e. Perl, PHP and R), Python and Go.

Reduced version (without original SemVer "build metadata") from: https://regex101.com/r/vkijKf/1/

```
^(0|\[1-9\]\d\*)\\(0|\[1-9\]\d\*)\\(0|\[1-9\]\d\*)(?:-((?:0|\[1-9\]\d\*|\d\*\[a-zA-Z-\]\[0-9a-zA-Z-\]\*)(?:\\(?:0|\[1-9\]\d\*|\d\*\[a-zA-Z-\]\[0-9a-zA-Z-\]
\*))\*))?$
```

Must I adopt semantic versioning rules when switching to SDMX 3.0?

No. If backwards compatibility with pre-existing tools and processes is required, then it is possible to continue using the previous versioning scheme (with up to two version parts MAJOR.MINOR). Semantic versioning is indicated only for those use cases where a proper artefact versioning is required. If versioning does not apply to some or all of your artefacts, then rather migrate to non-versioned SDMX 3.0 artefacts.

Copyright © SDMx TWG 2020

May I mix artefacts that follow semantic versioning with artefacts that don't?

Artefacts that are not (semantically) versioned may reference artefacts that are semantically versioned, but those are fully safe to use only when not extended. However, the reverse is not true: non-semantically-versioned artefacts do not offer change guarantees, and, therefore, should not be referenced by semantically versioned artefacts.

I have plenty of artefacts. I'm happy with my current versioning policy and I don't want to use SemVer! Can I still migrate to SDMX 3.0, and if so, what do I need to do?

Yes, of course, you can. The introduction of semantic versioning is done in a way which is largely backward compatible with previous versions of the standard, so you can keep your existing 2-digit version numbers (1.0, 1.1, 2.0, etc.) if that is required by your current tools and processes. However, if not using SemVer then pre-SDMX 3.0 final artefacts will be migrated as non-final and mutable in SDMX 3.0. There are also many good reasons to move to SemVer, and the migration is encouraged. Be assured that there will be tools out there that will assist you doing this in an efficient and convenient way.

I have plenty of artefacts versioned `X.Y`. I want to make some of them immutable, and enjoy the benefits provided by semantic versioning. Some other artefacts however must remain mutable (i.e. non final). However, in both cases, I'd like adopt the semantic versioning. What do I need to do?

For artefacts that will be made immutable and are therefore safe to use, simply append a '.0' to the current version (use `X.Y.0`) when migrating to Semantic Versioning. E.g., if the version of your artefact is currently 1.10, then migrate to 1.10.0.

For artefacts that remain mutable, and therefore do not bring the guarantees of semantic versioning, if you want to benefit from the advantages of semantic versioning, then simply append `.0-notfinal` to the version string. So, if the version of your artefact is currently `1.10`, use `1.10.0-notfinal` instead. Indeed, other extensions can be used depending on your use case.

I have adopted SDMX 3.0 with the semantic versioning conventions for the version strings of all my artefacts, regardless of whether these are stable (e.g. `1.0.0`) or unstable (e.g. `1.0.0-notfinal`, `1.0.0-draft`, etc.). However, I still receive artefacts from organizations that have not yet adopted SemVer conventions for the version strings. How should I treat these?

The only artefacts that are safe to use, are those that are semantically versioned. Starting with SDMX 3.0, these artefacts MUST use the SEMVER version string to indicate this fact and the version string of these artefacts MUST be expressed as `X.Y.Z` (e.g. `2.1.0`). Extended versions bring some limited guarantees for changes.

All other artefacts are in principle unsafe. They might be safe in practice but the SDMX standard does not bring any guarantees in that respect, and these artefacts may change in unpredictable ways.

In practice, the migration approach will often mirror the way in which organisations have migrated between earlier SDMX versions. Rarely, the new data models used mixed SDMX standard versions in their dependencies, and if they did then standard conversions were put in place. A typical method is to first migrate the re-used artefacts from the previous SDMX version to SDMX 3.0 and while doing so to apply the appropriate new semantic version string. From that point onwards, you can enjoy the advantages of the new SDMX versioning features for all those artefacts that require appropriate versioning.

Copyright © SDMx TWG 2020

6. REST API (v2.2)

6.1 REST API

6.1.1 SDMX REST API: Developers' documentation

Overview

The SDMX REST API allows implementers to offer programmatic access to statistical data and metadata over HTTP.

It offers a variety of features such as:

- To retrieve structural metadata, using a combination of id, agencyID and version number.
- To retrieve statistical data or reference metadata using keys (with options for wildcarding and support for the OR operator, as well as operators on Component values), data or metadata flows and data or metadata providers.
- To further refine queries for statistical data or reference metadata using time information (start period and end period).
- To retrieve what has changed since your last query (i.e. new or updated data).
- To return the results of a query in various formats.
- For structural metadata, it is possible to instruct the web service to resolve references (for instance, when querying for data structure definitions, it is possible to also retrieve the concepts and code lists used in the returned data structure definitions), as well as artefacts that use the matching artefact (for example, to retrieve the dataflows that use a matching data structure definition).
- For structural metadata, it is possible to retrieve a minimal version of the artefact, for the sake of efficiency (for example, to retrieve all code lists – names, ids, etc – without the codes).
- To build dynamic query filters using the *availability* queries.
- To perform maintenance operations, such as creating, updating or deleting data and metadata.

REST principles

The API is based on REST principles:

- In REST, specific information is known as **Resource**. In SDMX, specific resources would be maintainable artefacts, for example, code lists, concept schemes, data structure definitions, dataflows, etc. Each resource is addressable via a **global identifier** (i.e.: a URI).
- Manipulating resources is done using **methods defined in the HTTP protocol** (e.g.: GET, POST, PUT, DELETE).
- A resource can be represented in various formats (such as the different flavours and versions of the SDMX-ML standard). Selection of the appropriate **representation** is done using HTTP Content Negotiation and the HTTP `Accept` request header.

Copyright © SDMx TWG 2020

Use of operators

Furthemore, the API supports several operators, which will be described in more details in the various sections. These are:

Character	Meaning
`*`	All
`+`	Latest stable version
`~`	Latest version (whether stable, draft or unversioned)
`,`	Or
`.`	Subpart separator
`+`	Multivalues attribute

As can be seen, the `+` is used twice, but in different contexts, thereby removing any ambiguity.

Content

The documentation is organized in the following sections:

- Data queries
- Structure queries
- Reference metadata queries
- Schema queries
- Availability queries
- Query syntax for versions
- Content-Negotiation
- Status codes
- Structure maintenance
- Tips for providers and consumers
- Extending the API

Copyright © SDMx TWG 2020

6.1.2 Data queries

> **Note**
>
> To enhance readability of the large tables on this page, navigation panels are hidden. To return to the introduction, click REST API.

Overview

Data queries allow **retrieving statistical data**. Entire datasets, individual observations, or anything in between, can be retrieved using filters on dimensions (including time), attributes and/or measures. All data matching a query can be retrieved or only the data that has changed since the last time the same query was performed. Using the *includeHistory* parameter, it is also possible to retrieve previous versions of the data. Last but not least, the data retrieved can be packaged in different ways (as time series, cross-sections or as a table), in a variety of formats (JSON, XML, CSV, etc.).

Syntax

Copyright © SDMx TWG 2020

```
protocol://ws-entry-point/data/{context}/{agencyID}/{resourceID}/{version}/{key}?
{c}&{updatedAfter}&{firstNObservations}&{lastNObservations}&{dimensionAtObservation}&{attributes}&{measures}&{includeHistory}
```


Copyright © SDMx TWG 2020

Parameter	Type	Description	Default
context	One of the following: `datastructure`, `dataflow`, `provisionagreement`	Data can be reported against a data structure, a dataflow or a provision agreement. This parameter allows selecting the desired context for data retrieval.	*
agencyID	A string compliant with the SDMX *common:NCNameIDType*	The agency maintaining the artefact for which data have been reported.	*
resourceID	A string compliant with the SDMX *common:IDType*	The id of the artefact for which data have been reported.	*
version	A string compliant with the allowed SDMX versioning schemes	The version of the artefact for which data have been reported.	*
key	A string compliant with the *KeyType* defined in the SDMX Open API specification.	The combination of dimension values identifying the slice of the cube for which data should be returned. Wildcarding is supported via the `*` operator. For example, if the following key identifies the bilateral exchange rates for the daily US dollar exchange rate against the euro, D.USD.EUR.SP00.A, then the following key can be used to retrieve the data for all currencies against the euro: D.`*`.EUR.SP00.A. Any dimension value omitted at the end of the Key is assumed as equivalent to a wildcard, e.g. D.USD is equivalent to D.USD.`*`.`*`.`*`	*
c	Map	Filter data by component value. For example, if a structure defines a frequency dimension (FREQ) and the code A (Annual) is an allowed value for that dimension, the following can be used to retrieve annual data: `c[FREQ]=A`. The same applies to attributes (e.g. `c[CONF_STATUS]=F`) and measures. Multiple values are supported, using a comma (`,`) as separator: `c[FREQ]=A,M`. The comma effectively acts as an `OR` statement (i.e. FREQ is A OR FREQ is M). The plus (`+`) can be used whenever an `AND` statement is required, such as for example, for attributes with multiple values or for time ranges. For example, to indicate that ATTR1 must either be A or (B AND M), use the following: `c[ATTR1]=A,B+M`. Operators may be used too (see table with operators	

Copyright © SDMx TWG 2020

Parameter	Type	Description	Default
		below). This parameter can be used in addition, or instead of, the `key` path parameter. This parameter may be used multiple times (e.g. `c[FREQ]=A,M&c[CONF_STATUS]=F`), but only once per Component.	
updatedAfter	xs:dateTime	The last time the query was performed by the client in the database. If this parameter is used, the returned message should only include the latest version of what has changed in the database since that point in time (updates and revisions). This should include observations that have been added since the last time the query was performed (INSERT), observations that have been revised since the last time the query was performed (UPDATE) and observations that have been deleted since the last time the query was performed (DELETE). If no offset is specified, default to local time of the web service. If the information about when the data has been updated is not available at the observation level, the web service should return either the series that have changed (if the information is attached at the series level) or the dataflows that have changed (if the information is attached at the dataflow level).	
firstNObservations	Positive integer	The maximum number of observations to be returned for each of the matching series, starting from the first observation	
lastNObservations	Positive integer	The maximum number of observations to be returned for each of the matching series, counting back from the most recent observation	
dimensionAtObservation	A string compliant with the SDMX common:NCNameIDType	The ID of the dimension to be attached at the observation level. This parameter allows the client to indicate how the data should be packaged by the service. The options are `TIME_PERIOD` (a *timeseries* view of the data), the `ID of any other dimension` used in that dataflow (a *cross-sectional* view of the data) or the keyword	Depends DSD

Copyright © SDMx TWG 2020

Parameter	Type	Description	Default
		`AllDimensions` (a *flat* / *table* view of the data where the observations are not grouped, neither in time series, nor in sections). In case this parameter is not set, the service is expected to default to TimeDimension, if the data structure definition has one, or else, to default to AllDimensions.	
attributes	String	This parameter specifies the attributes to be returned. Possible options are: `dsd` (all the attributes defined in the data structure definition), `msd` (all the reference metadata attributes), `dataset` (all the attributes attached to the dataset-level), `series` (all the attributes attached to the series-level), `obs` (all the attributes attached to the observation-level), `all` (all attributes), `none` (no attributes), `{attribute_id}`: The ID of one or more attributes the caller is interested in.	`dsd`
measures	String	This parameter specifies the measures to be returned. Possible options are: `all` (all measures), `none` (no measure), `{measure_id}`: The ID of one or more measures the caller is interested in.	`all`
includeHistory	Boolean	This parameter allows retrieving previous versions of the data, as they were disseminated in the past (*history* or *timeline* functionality). When the value is set to `true`, the returned data message should contain one or two datasets per data dissemination, depending on whether a dissemination also deleted observations from the data source. The `validFromDate` and/or `validToDate` attributes of the dataset should be used to indicate the periods of validity for the data contained in the data set. See below for an example on how to handle the `includeHistory` parameter.	`false`

Copyright © SDMx TWG 2020

The following rules apply:

- Multiple values for a parameter must be separated using a comma (`,`).
- Two additional operators are supported for the *version* parameter: the `+`, to indicate the latest stable version of an artefact, and `~`, to indicate the latest version of an artefact regardless of its status (draft vs. stable).
- Default values do not need to be supplied if they are the last element in the path.
- Operators can be used to refine the applicability of the `c` query parameter:

Operator	Meaning	Note
eq	Equals	Default if no operator is specified and there is only one value (e.g. `c[FREQ]=M` is equivalent to `c[FREQ]=eq:M`)
ne	Not equal to	
lt	Less than	
le	Less than or equal to	
gt	Greater than	
ge	Greater than or equal to	
co	Contains	
nc	Does not contain	
sw	Starts with	
ew	Ends with	

Operators appear as prefix to the component value(s) and are separated from it by a `:` (e.g. `c[TIME_PERIOD]=ge:2020-01+le:2020-12`).

Response types

The following media types can be used with *data* queries:

- **application/vnd.sdmx.data+json;version=2.0.0**
- application/vnd.sdmx.data+xml;version=3.0.0
- application/vnd.sdmx.data+csv;version=2.0.0;labels=[id|name|both];timeFormat=[original|normalized];keys=[none|obs|series|both]
- application/vnd.sdmx.genericdata+xml;version=2.1
- application/vnd.sdmx.structurespecificdata+xml;version=2.1
- application/vnd.sdmx.generictimeseriesdata+xml;version=2.1
- application/vnd.sdmx.structurespecifictimeseriesdata+xml;version=2.1
- application/vnd.sdmx.data+json;version=1.0.0
- application/vnd.sdmx.data+csv;version=1.0.0;labels=[id|both];timeFormat=[original|normalized]

The default format is highlighted in **bold**.

SDMX-CSV offers the possibility to set the value for two parameters via the media-type. These parameters are `label` and `timeFormat`; both are optional. The default values for these parameters are marked with * in the above media-type (i.e. `id` and `original` respectively). For additional information about these parameters, please refer to the SDMX-CSV specification.

Copyright © SDMx TWG 2020

Examples of queries

- Retrieve the data matching the supplied path parameters:

```
https://ws-entry-point/data/dataflow/ECB/EXR/1.0.0/M.USD.EUR.SP00.A
```

- Retrieve the data matching the supplied path parameters, and, for the attributes, only retrieve OBS_STATUS and UNIT_MEASURE :

```
https://ws-entry-point/data/dataflow/ECB/EXR/1.0.0/M.USD.EUR.SP00.A?attributes=OBS_STATUS,UNIT_MEASURE
```

- Retrieve the data matching the supplied path parameters, including multiple versions and a wildcard for the second dimension:

```
https://ws-entry-point/data/datastructure/ECB/ECB_EXR1/1.0.0,2.0.0/M.*.EUR.SP00.A
```

- Retrieve the data matching the supplied path parameters (including multiple keys and the latest stable version):

```
https://ws-entry-point/data/dataflow/ECB/EXR/+/M.USD.EUR.SP00.A,A.CHF.EUR.SP00.A?updatedAfter=2009-05-15T14%3A15%3A00%2B01%3A00
```

- Retrieve the *public* data matching the supplied path parameters (all data reported for the latest version of the ECB EXR dataflow), and falling between the supplied the start and end periods:

```
https://ws-entry-point/data/dataflow/ECB/EXR/?c[TIME_PERIOD]=ge:2009-05-01+le:2009-05-31&c[CONF_STATUS]=F
```

- Retrieve the list of indicators about Switzerland (CH) available in the source:

```
https://ws-entry-point/data/?c[REF_AREA]=CH&attributes=none&measures=none
```

- Retrieve the list of observations matching the supplied path parameters, that are above a certain threshold:

```
https://ws-entry-point/data/dataflow/ECB/EXR/1.0.0/M.USD.EUR.SP00.A?c[OBS_VALUE]=ge:10.0&dimensionAtObservation=AllDimensions
```

- Retrieve the list of indicators containing euro in their title:

```
https://ws-entry-point/data/?c[TITLE]=co:euro&attributes=none&measures=none
```

- Retrieving deltas using `updatedAfter`:

```
https://ws-entry-point/data/dataflow/ECB/EXR?updatedAfter=2009-05-15T14%3A15%3A00%2B01%3A00
```

By supplying a percent-encoded timestamp to the `updatedAfter` parameter, it is possible to **only retrieve the latest version of changed values** in the database since a certain point in time (so-called *updates and revisions* or *deltas*).

The response to such a query could include one or more dataset(s) representing:

- The observations that have been **added** since the last time the query was performed.
- The observations that have been **revised** since the last time the query was performed.
- The observations that have been **deleted** since the last time the query was performed.

Developers who update their local databases should make use of the `updatedAfter` parameter as it is likely to significantly **improve performance**. Instead of systematically downloading data that may not have changed, you would only receive the *consolidated* changes to be made in your database since the last time your client performed the same query.

An alternative, less efficient than the solution described above, but more efficient than downloading everything all the time, would be to use HTTP Conditional GET requests (i.e. the `If-Modified-Since` or `If-None-Match` HTTP Request headers). Using this mechanism, everything will be returned but only if something has changed since the previous query.

A sample response message is provided below. In the response the `action` attribute of the `Dataset` element is of importance whereas the `validFromDate` is just there for information purposes.

```
<?xml version="1.0" encoding="UTF-8"?>
<message:GenericData xmlns:message="http://www.sdmx.org/resources/sdmxml/schemas/v2_1/message" xmlns:common="http://www.sdmx.org/resources/sdmxml/schemas/
v2_1/common" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:generic="http://www.sdmx.org/resources/sdmxml/schemas/v2_1/data/generic"
xsi:schemaLocation="http://www.sdmx.org/resources/sdmxml/schemas/v2_1/message http://sdw-wsrest.ecb.europa.eu:80/vocabulary/sdmx/2_1/SDMXMessage.xsd http://
www.sdmx.org/resources/sdmxml/schemas/v2_1/common http://sdw-wsrest.ecb.europa.eu:80/vocabulary/sdmx/2_1/SDMXCommon.xsd http://www.sdmx.org/resources/sdmxml/
schemas/v2_1/data/generic http://sdw-wsrest.ecb.europa.eu:80/vocabulary/sdmx/2_1/SDMXDataGeneric.xsd">
```


Copyright © SDMx TWG 2020

```
    <message:Header>
        <message:ID>388d1c9a-d187-4f6a-8792-e117cf34047f</message:ID>
        <message:Test>false</message:Test>
        <message:Prepared>2016-12-20T16:19:56.398+01:00</message:Prepared>
        <message:Sender id="ECB"/>
        <message:Structure structureID="ECB_RTD1" dimensionAtObservation="TIME_PERIOD">
            <common:Structure>
                <URN>urn:sdmx:org.sdmx.infomodel.datastructure.DataStructure=ECB:ECB_RTD1(1.0)</URN>
            </common:Structure>
        </message:Structure>
    </message:Header>
    <message:DataSet action="Replace" validFromDate="2016-12-20T16:19:56.398+01:00" structureRef="ECB_RTD1">[...]</message:DataSet>
    <message:DataSet action="Delete" validToDate="2016-12-20T16:19:56.440+01:00" structureRef="ECB_RTD1">[...]</message:DataSet>
</message:GenericData>
```

- Retrieving a limited amount of observations using `firstNObservations` and `lastNObservations`:

```
https://ws-entry-point/data/dataflow/ECB/EXR/1.0.0/M.USD.EUR.SP00.A?lastNObservations=2
```

Using the `firstNObservations` and/or `lastNObservations` parameters, it is possible to specify the **maximum number of observations** to be returned for each of the matching series, starting from the first observation (`firstNObservations`) or counting back from the most recent observation (`lastNObservations`). This can be useful for building an overview page, for example, where, for each indicator, you only display 2 values (the current one and the previous one).

- Retrieving how a time series evolved over time using the `includeHistory` parameter:

```
https://ws-entry-point/data/dataflow/ECB/EXR/1.0.0/M.USD.EUR.SP00.A?includeHistory=true
```

Using the `includeHistory` parameter, you can instruct the web service to return **previous versions of the matching data**. This is useful to see how the data have evolved over time, i.e. when new data have been released or when data have been revised or deleted. Possible options are:

- `false`: Only the version currently in production will be returned. This is the default.
- `true`: The version currently in production, as well as all previous versions, will be returned.

Such a query would give you the *history* between that point in time and today.

A sample response message is provided below. In the response the `action` & the `validFromDate` attributes of the `Dataset` element are both equally important. While the `action` attribute indicates what needs to be done, the `validFromDate` attribute allows defining the validity periods of the reported data.

```
<?xml version="1.0" encoding="UTF-8"?>
<message:GenericData xmlns:message="http://www.sdmx.org/resources/sdmxml/schemas/v2_1/message" xmlns:common="http://www.sdmx.org/resources/sdmxml/schemas/
v2_1/common" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:generic="http://www.sdmx.org/resources/sdmxml/schemas/v2_1/data/generic"
xsi:schemaLocation="http://www.sdmx.org/resources/sdmxml/schemas/v2_1/message http://sdw-wsrest.ecb.europa.eu:80/vocabulary/sdmx/2_1/SDMXMessage.xsd http://
www.sdmx.org/resources/sdmxml/schemas/v2_1/common http://sdw-wsrest.ecb.europa.eu:80/vocabulary/sdmx/2_1/SDMXCommon.xsd http://www.sdmx.org/resources/sdmxml/
schemas/v2_1/data/generic http://sdw-wsrest.ecb.europa.eu:80/vocabulary/sdmx/2_1/SDMXDataGeneric.xsd">
    <message:Header>
        <message:ID>a2c99026-00c8-4f9a-a3d4-1891f89bd2b0</message:ID>
        <message:Test>false</message:Test>
        <message:Prepared>2016-12-20T17:16:51.578+01:00</message:Prepared>
        <message:Sender id="ECB"/>
        <message:Structure structureID="ECB_RTD1" dimensionAtObservation="TIME_PERIOD">
            <common:Structure>
                <URN>urn:sdmx:org.sdmx.infomodel.datastructure.DataStructure=ECB:ECB_RTD1(1.0)</URN>
            </common:Structure>
        </message:Structure>
    </message:Header>
    <message:DataSet action="Replace" validFromDate="2014-12-03T15:30:00.000+01:00" structureRef="ECB_RTD1">[...]</message:DataSet>
    <message:DataSet action="Replace" validFromDate="2015-06-02T15:30:00.000+02:00" structureRef="ECB_RTD1">[...]</message:DataSet>
    <message:DataSet action="Replace" validFromDate="2015-10-21T15:30:00.000+02:00" structureRef="ECB_RTD1">[...]</message:DataSet>
    <message:DataSet action="Replace" validFromDate="2016-06-01T15:30:00.000+02:00" structureRef="ECB_RTD1">[...]</message:DataSet>
    <message:DataSet action="Delete" validToDate="2014-11-05T15:30:00.000+01:00" structureRef="ECB_RTD1">[...]</message:DataSet>
</message:GenericData>
```


Copyright © SDMx TWG 2020

6.1.3 Structure queries

> **Note**
>
> To enhance readability of the large tables on this page, navigation panels are hidden. To return to the introduction, click REST API.

Overview

Structure queries allow **retrieving structural metadata**.

Below is a list of popular types of structural metadata, that you will typically find in SDMX web services.

- `Concept schemes`: In order to make sense of some statistical data, we need to know the *concepts* associated with them. For example, on its own, the figure 1.2953 is pretty meaningless, but if we know that this is an exchange rate for the US dollar against the euro on 23 November 2006, it starts making more sense. The various concepts are typically grouped into collections known as *concept schemes*.
- `Codelists`: Some of the concepts can be *free text* (such as a comment about a particular observation value) but others take their values from a *controlled vocabulary list* (such as, for example, a list of countries). These are known as *codelists* in SDMX.
- `Data structure definitions`: All the concepts that describe a particular domain (such as exchange rates or inflation) are grouped into a *data structure definition (DSD)*. In a DSD, concepts are divided into *dimensions* and *attributes*. Dimensions, when combined, allow to uniquely *identify* statistical data. Attributes on the other hand do not help identifying statistical data, but add useful information (like the unit of measure or the number of decimals). In order to perform granular data queries, one must know the concepts that are used as dimensions, as well as their allowed values (as defined in the codelists).
- `Dataflows`: Dataflows represent the data that cover a particular domain (such as, for example, balance of payments). A *dataflow* provides a reference to the data structure definition that applies for a particular domain, thereby indicating how the data for that domain will look like.

Structure queries in SDMX allow you to retrieve structural metadata at various levels of granularity, from all structural metadata available in the source to a single code from a particular version of a particular codelist maintained by a particular agency.

Syntax

```
protocol://ws-entry-point/structure/{artefactType}/{agencyID}/{resourceID}/{version}?{detail}&{references}
```

For item schemes, an additional path parameter (itemID) is permissible.

Copyright © SDMx TWG 2020

```
protocol://ws-entry-point/structure/{artefactType}/{agencyID}/{resourceID}/{version}/{itemID}?{detail}&{references}
```


Copyright © SDMx TWG 2020

Parameter	Type	Description	Default	Multiple values?
artefactType	One of the following types: datastructure, metadatastructure, categoryscheme, conceptscheme, codelist, hierarchy, hierarchyassociation, valuelist, agencyscheme, dataproviderscheme, metadataproviderscheme, dataconsumerscheme, organisationunitscheme, dataflow, metadataflow, reportingtaxonomy, provisionagreement, metadataprovisionagreement, structuremap, representationmap, conceptschememap, categoryschememap, organisationschememap, reportingtaxonomymap, process, categorisation, dataconstraint, metadataconstraint, transformationscheme, rulesetscheme, userdefinedoperatorscheme, customtypescheme, namepersonalisationscheme, vtlmappingscheme	The type of structural metadata to be returned.	*	No
agencyID	A string compliant with the SDMX *common:NCNameIDType*	The agency maintaining the artefact to be returned. It is possible to set more than one agency, using `,` as separator (e.g. BIS,ECB).	*	Yes
resourceID	A string compliant with the SDMX *common:IDType*	The id of the artefact to be returned. It is possible to set more than one id, using `,` as separator (e.g. CL_FREQ,CL_CONF_STATUS).	*	Yes
version	A string compliant with the SDMX *semantic versioning* rules	The version of the artefact to be returned. It is possible to set more than one version, using `,` as separator (e.g. 1.0.0,2.1.7).	~	Yes
itemID	A string compliant with the SDMX *common:NestedNCNameIDType* for conceptscheme and agencyscheme, any type for valuelist or with the SDMX *common:NestedIDType* in all other cases	The id of the item to be returned. It is possible to set more than one id, using `,` as separator (e.g. A,Q,M). **This parameter is valid for item schemes only**	*	Yes
detail	*String*	This attribute specifies the desired amount of information to be returned. For example, it is possible to instruct the web service to return only basic information about the	**full**	No

Copyright © SDMx TWG 2020

Parameter	Type	Description	Default	Multiple values?
		maintainable artefact (i.e.: id, agency id, version and name). Most notably, items of item schemes will not be returned (for example, it will not return the codes in a code list query). Possible values are: (1) `full`: all available information for all returned artefacts should be returned. Returned extended codelists are to be resolved, i.e. include all inherited codes, and must not include the CodelistExtension property. As the inherited codelists must be resolved, they should not be returned a second time as separated codelists. (2) `allstubs`: all returned artefacts should be returned as stubs, i.e. only containing identification information and the artefacts' name. (3) `referencestubs`: same as full with the exception that referenced artefacts should be returned only as stubs, i.e. only containing identification information and the artefacts' name. (4) `allcompletestubs`: all returned artefacts should be returned as complete stubs, i.e. only containing identification information, the artefacts' name, description and annotations. (5) `referencecompletestubs`: same as full with the exception that referenced artefacts should be returned as complete stubs, i.e. only containing identification information, the artefacts' name, description and annotations. (6) `referencepartial`: same as full with the exception that referenced item schemes should only include items used by the artefact to be returned. For example, a concept scheme would only		

Copyright © SDMx TWG 2020

Parameter	Type	Description	Default	Multiple values?
		contain the concepts used in a DSD, and its isPartial flag would be set to true. Likewise, if a dataflow has been constrained, then the codelists referenced by the DSD referenced by the dataflow should only contain the codes referenced by the content constraint. (7) `raw`: same as full with the exception that the returned extended codelists are not resolved and must include the CodelistExtension property, and if referenced codelists or descendants are to be returned then they include also all inherited codelists.		
references	*String*	This attribute instructs the web service to return (or not) the artefacts referenced by the artefact to be returned (for example, the code lists and concepts used by the data structure definition matching the query), as well as the artefacts that use the matching artefact (for example, the dataflows that use the data structure definition matching the query). Possible values are: `none` (no references will be returned), `parents` (the artefacts that use the artefact matching the query), `parentsandsiblings` (the artefacts that use the artefact matching the query, as well as the artefacts referenced by these artefacts), `ancestors` (the artefacts that use the artefact matching the query, up to any level), `children` (artefacts referenced by the artefact to be returned), `descendants` (references of references, up to any level, will also be returned), `all` (the combination of parentsandsiblings and	**none**	Yes

Copyright © SDMx TWG 2020

Parameter	Type	Description	Default	Multiple values?
		descendants). In addition, a `concrete type of resource` may also be used (for example, references=codelist).		

NOTES ABOUT USING `itemID`

As mentioned, `itemID` can be used for item scheme queries only! These are:

- categoryscheme
- conceptscheme
- codelist
- agencyscheme
- dataproviderscheme
- metadataproviderscheme
- dataconsumerscheme
- organisationunitscheme
- reportingtaxonomy
- transformationscheme
- rulesetscheme
- userdefinedoperatorscheme
- customtypescheme
- namepersonalisationscheme
- vtlmappingscheme

Although it is not following the *item scheme* pattern, the *valuelist* is also a collection, i.e. a collection of values.

If `itemID` is set and is a *top-level* id (e.g.: Code A (Annual) in the Frequency Codelist), and such an item exists in the matching item scheme, the item scheme returned should contain only the matching item and its `isPartial` parameter should be set to `true`.

If `itemID` is set and is a *nested* id (e.g.: Category A.1.1, belonging to Category A.1, belonging to Category A in a Category Scheme), and such an item exists in the matching item scheme, the item scheme returned should contain the matching item and its ancestors, and its `isPartial` parameter should be set to `true`.

The `detail` parameter allows specifying the amount of details to be retrieved. When `itemID` is used, the full details of the items requested are the following:

- Identification
- Names
- Descriptions
- Annotations
- Other details except Items

Copyright © SDMx TWG 2020

Since details are not exactly the same for all Item Schemes, the following table provides these details per Artefact type (`@` is used for XML attributes and `/` for XML elements).

Item	flat/nested	Details in Item Query
Code	flat	@id, @urn, @uri, /Annotations, /Name, /Description, /Parent
Concept	flat	@id, @urn, @uri, /Annotations, /Name, /Description, /Parent, /CoreRepresentation, /ISOConceptReference
Agency	flat	@id, @urn, @uri, /Annotations, /Name, /Description, /Contact
DataProvider	flat	@id, @urn, @uri, /Annotations, /Name, /Description, /Contact
DataConsumer	flat	@id, @urn, @uri, /Annotations, /Name, /Description, /Contact
OrganisationUnit	flat	@id, @urn, @uri, /Annotations, /Name, /Description, /Parent, /Contact
Category	nested	@id, @urn, @uri, /Annotations, /Name, /Description
ReportingCategory	nested	@id, @urn, @uri, /Annotations, /Name, /Description, /StructuralMetadata, /ProvisioningMetadata

While most Item Schemes are flat, hence the above table is easy to interpret, for Category Schemes and Reporting Taxonomies (which have a nested structure) the above table indicates that any child Item(s) of the Item(s) requested must not be included in an Item Query request.

Further to the above, the reference resolution mechanism will be applied to all items returned. For example, querying for a category which has two ancestors in the hierarchy of the category scheme, will result into returning three categories (the requested one and its ancestors), as well as all references of those three categories.

APPLICABILITY AND MEANING OF REFERENCES (INCLUDING REFERENCEPARTIAL)

The table below lists the 1st level artefacts (one level up, one level down) that will be returned if the references parameter is set to `all`. Artefacts referenced by the matching artefact are displayed in regular style, artefacts that reference the matching artefact

Copyright © SDMx TWG 2020

are displayed in *Italic* and artefacts that can both reference and be referenced by the matching artefact are displayed in ***bold italic***.

Copyright © SDMx TWG 2020

Maintainable artefact	Artefacts returned
AgencyScheme	*All*
Categorisation	All, *HierarchyAssociation*, *Metadataflow*, *MetadataProvisionAgreement*, *Process*
CategoryScheme	AgencyScheme, *Categorisation*, *CategorySchemeMap*, *Metadataflow*, *MetadataProvisionAgreement*, *Process*
CategorySchemeMap	AgencyScheme, *Categorisation*, CategoryScheme, *HierarchyAssociation*, *Metadataflow*, *MetadataProvisionAg*
Codelist	AgencyScheme, *Categorisation*, ***Codelist***, *ConceptScheme*, *DataStructureDefinition*, *Hierarchy*, *HierarchyAss* *VtlMappingScheme*
ConceptScheme	AgencyScheme, *Categorisation*, Codelist, *ConceptSchemeMap*, *DataStructureDefinition*, *HierarchyAssociation*
ConceptSchemeMap	AgencyScheme, *Categorisation*, ConceptScheme, *HierarchyAssociation*, *Metadataflow*, *MetadataProvisionAg*
CustomTypeScheme	AgencyScheme, *Categorisation*, *HierarchyAssociation*, *Metadataflow*, *MetadataProvisionAgreement*, *Process*,
DataConstraint	AgencyScheme, *Categorisation*, Dataflow, DataProviderScheme, DataStructureDefinition, *HierarchyAssociati*
DataConsumerScheme	AgencyScheme, *Categorisation*, *HierarchyAssociation*, *Metadataflow*, *MetadataProvisionAgreement*, *Organisa*
Dataflow	AgencyScheme, *Categorisation*, *DataConstraint*, DataStructureDefinition, *HierarchyAssociation*, *Metadataflow*
DataProviderScheme	AgencyScheme, *Categorisation*, *DataConstraint*, *HierarchyAssociation*, *Metadataflow*, *MetadataProvisionAgre*
DataStructureDefinition	AgencyScheme, *Categorisation*, Codelist, ConceptScheme, *DataConstraint*, *Dataflow*, *HierarchyAssociation*, *
Hierarchy	AgencyScheme, *Categorisation*, Codelist, *HierarchyAssociation*, *Metadataflow*, *MetadataProvisionAgreement*,
HierarchyAssociation	All, *Categorisation*, *Metadataflow*, *MetadataProvisionAgreement*, *Process*
MetadataConstraint	AgencyScheme, *Categorisation*, *HierarchyAssociation*, ***Metadataflow***, MetadataProviderScheme, ***Metadata***
Metadataflow	All, *Categorisation*, *HierarchyAssociation*, *MetadataConstraint*, *Metadataflow*, *MetadataProvisionAgreement*,
MetadataProviderScheme	AgencyScheme, *Categorisation*, *HierarchyAssociation*, *MetadataConstraint*, *Metadataflow*, *MetadataProvision*
MetadataProvisionAgreement	All, *Categorisation*, *HierarchyAssociation*, *Metadataflow*, *MetadataProvisionAgreement*, *MetadataConstraint*,
MetadataStructureDefinition	AgencyScheme, *Categorisation*, Codelist, ConceptScheme, *DataStructureDefinition*, *HierarchyAssociation*, *Me*
NamePersonalisationScheme	AgencyScheme, *Categorisation*, *Metadataflow*, *MetadataProvisionAgreement*, *Process*, *TranformationScheme*
OrganisationSchemeMap	AgencyScheme, *Categorisation*, DataConsumerScheme, DataProviderScheme, *HierarchyAssociation*, *Metada*
OrganisationUnitScheme	AgencyScheme, *Categorisation*, *Metadataflow*, *MetadataProvisionAgreement*, *OrganisationSchemeMap*, *Proc*
Process	All, *Categorisation*, *HierarchyAssociation*, *Metadataflow*, *MetadataProvisionAgreement*
ProvisionAgreement	AgencyScheme, *Categorisation*, *DataConstraint*, Dataflow, DataProviderScheme, *HierarchyAssociation*, *Meta*
ReportingTaxonomy	AgencyScheme, *Categorisation*, *CategorySchemeMap*, *HierarchyAssociation*, Dataflow, ***Metadataflow***, *Meta*
ReportingTaxonomyMap	AgencyScheme, *Categorisation*, *HierarchyAssociation*, *Metadataflow*, *MetadataProvisionAgreement*, *Process*,
RepresentationMap	AgencyScheme, *Categorisation*, Codelist, *HierarchyAssociation*, *Metadataflow*, *MetadataProvisionAgreement*,
RulesetScheme	AgencyScheme, *Categorisation*, *HierarchyAssociation*, *Metadataflow*, *MetadataProvisionAgreement*, *Process*,
StructureMap	AgencyScheme, *Categorisation*, Dataflow, DataStructureDefinition, *HierarchyAssociation*, *Metadataflow*, *Meta*
TranformationScheme	AgencyScheme, *Categorisation*, CustomTypeScheme, *HierarchyAssociation*, *Metadataflow*, *MetadataProvision*
UserDefinedOperatorScheme	AgencyScheme, *Categorisation*, *HierarchyAssociation*, *Metadataflow*, *MetadataProvisionAgreement*, *Process*,
ValueList	AgencyScheme, *Categorisation*, *ConceptScheme*, *DataStructureDefinition*, *Metadataflow*, *MetadataProvisionA*
VtlMappingScheme	AgencyScheme, *Categorisation*, Codelist, ConceptScheme, Dataflow, *HierarchyAssociation*, *Metadataflow*, *Me*

Copyright © SDMx TWG 2020

Also, when returning only partial references (via `referencepartial`), the filtering applies to any dependency, regardless of its level. The table below describes the impact of using `referencepartial` on the referenced item schemes.

Maintainable artefact	Meaning of detail=referencepartial
AgencyScheme	Only the agencies maintaining the returned structures should be included in the returned agency scheme(s)
CategoryScheme	Only the categories referenced by the returned categorisations or structure sets should be included in the returned category scheme(s)
Codelist	Only the codes referenced by the returned categorisations, data/metadata constraints, hierarchies or structure sets should be included in the returned codelist(s)
ConceptScheme	Only the concepts referenced by the returned categorisations, data structure definitions, metadata structure definitions or structure sets should be included in the returned concept scheme(s)
DataConsumerScheme	Only the data consumers referenced by the returned categorisations, metadata structure definitions or structure sets should be included in the returned data consumer scheme(s)
DataProviderScheme	Only the data providers referenced by the returned categorisations, data constraints or provision agreements or structure sets should be included in the returned data provider scheme(s)
MetadataProviderScheme	Only the data providers referenced by the returned categorisations, metadata constraints, metadata provision agreements or structure sets should be included in the returned metadata provider scheme(s)
HierarchicalCodelist	Only the hierarchies referenced by the returned categorisations or structure sets should be included in the returned hierarchical codelist(s)
OrganisationUnitScheme	Only the organisation units referenced by the returned categorisations or structure sets in the returned organisation unit scheme(s)
ReportingTaxonomy	Only the reporting categories referenced by the returned categorisations or structure sets should be included in the reporting taxonomy

For example, a dataflow only references a data structure definition and, in addition, it may be referenced by one or more data constraints. In case `references` is set to `all` and `detail` is set to `referencepartial`, the following should apply to the returned item schemes:

- The concept scheme(s) should only include the concepts referenced by the data structure referenced by target dataflow;
- The codelist(s) should only include the codes referenced by the data constraint(s) referencing the target dataflow;
- The data provider schemes(s) should only include the data provider(s) that are linked to the target dataflow via a provision agreement
- The agency scheme(s) should only include the agencies maintaining the structural metadata to be returned (dataflow, data constraints, data structures, etc.)

Response types

The following media types can be used with *structure* queries:

- **application/vnd.sdmx.structure+json;version=2.0.0**
- application/vnd.sdmx.structure+json;version=1.0.0
- application/vnd.sdmx.structure+xml;version=3.0.0
- application/vnd.sdmx.structure+xml;version=2.1

The default format is highlighted in **bold**.

Copyright © SDMx TWG 2020

Examples of queries

- To retrieve version 1.0.0 of the DSD with id ECB_EXR1 maintained by the ECB, as well as the code lists and the concepts used in the DSD:

```
https://ws-entry-point/structure/datastructure/ECB/ECB_EXR1/1.0.0?references=children&detail=referencepartial
```

- To retrieve the latest stable version of the DSD with id ECB_EXR1 maintained by the ECB, without the code lists and concepts of the DSD:

```
https://ws-entry-point/structure/datastructure/ECB/ECB_EXR1/+
```

- To retrieve the latest version of all DSDs maintained by the ECB, as well as the dataflows using these DSDs:

```
https://ws-entry-point/structure/datastructure/ECB?references=dataflow
```

- To retrieve the latest version of all code lists maintained by all maintenance agencies, but without the codes:

```
https://ws-entry-point/structure/codelist?detail=allstubs
```

- To retrieve, as stubs, the latest version of all maintainable artefacts maintained by the ECB:

```
https://ws-entry-point/structure/all/ECB?detail=allstubs
```

- To retrieve the category PRICES of the DOMAINS category scheme maintained by the ECB, as well as the categorisations referencing that category:

```
https://ws-entry-point/structure/categoryscheme/ECB/DOMAINS/+/PRICES?references=categorisation
```

- To retrieve the latest version of the CL_FREQ codelists maintained by the BIS or the ECB:

```
https://ws-entry-point/structure/codelist/BIS,ECB/CL_FREQ
```


Copyright © SDMx TWG 2020

6.1.4 Metadata queries

Note

To enhance readability of the large tables on this page, navigation panels are hidden. To return to the introduction, click REST API.

By metadataset

OVERVIEW

These queries enable clients to find metadatasets by the identification of the metadataset, enabling clients to checkout specific reports.

SYNTAX

Copyright © SDMx TWG 2020

```
protocol://ws-entry-point/metadata/metadataset/{providerID}/{metadatasetID}/{version}?{detail}
```


Copyright © SDMx TWG 2020

Parameter	Type	Description	Default	Multiple values
providerID	A string compliant with the SDMX *common:NestedNCNameIDType* for MetadataProvider	The id of the data provider who maintains the Metadata Set. It is possible to set more than one id, using `,` as separator (e.g. UK1,UK2)	*	Yes
metadatasetID	A string compliant with the SDMX *common:NestedNCNameIDType* for MetadataSet	The Id of the Metadata Set to be returned. It is possible to set more than one id, using `,` as separator (e.g. MS1,MS2)	*	Yes
version	A string compliant with the SDMX *semantic versioning* rules	The version of the artefact to be returned. It is possible to set more than one version, using `,` as separator (e.g. 1.0.0,2.1.7).	*	Yes
detail	*String*	This attribute specifies the desired amount of information to be returned. For example, it is possible to instruct the web service to return only basic information about the metadataset (i.e.: id, dataprovider id, version and name). Most notably, metadata attributes of a metadataset will not be returned Possible values are: (1) `full`: all available	**full**	No

Copyright © SDMx TWG 2020

Parameter	Type	Description	Default	Multiple values
		information for all returned metadatasets should be returned. (2) `allstubs`: all returned metadatasets should be returned as stubs, i.e. only containing identification information and the metadataset' name.		

Notes:

- Default values do not need to be supplied if they are the last element in the path.

EXAMPLES

- To retrieve version 1.0.0 of the Metadataset with id QUALITY_REPORT maintained by the ECB

```
https://ws-entry-point/metadata/metadataset/ECB/QUALITY_REPORT/1.0.0
```

- To retrieve all the Metadatasets maintained by the ECB, without the reported metadata attributes

```
https://ws-entry-point/metadata/metadataset/ECB/?detail=allstubs
```

By metadataflow

OVERVIEW

These queries enable clients to find metadatasets by the collection (metadataflow), optionally filtered by the metadata provider.

SYNTAX

Copyright © SDMx TWG 2020

```
protocol://ws-entry-point/metadata/metadataflow/{flowAgencyID}/{flowID}/{flowVersion}/{providerRef}?{detail}
```


Copyright © SDMx TWG 2020

Parameter	Type	Description	Default	Multiple values
flowAgencyID	A string compliant with the SDMX *common:NCNameIDType*	The agency maintaining the metadataflow for which metadata have been reported.	*	Yes
flowID	A string compliant with the SDMX *common:IDType*	The id of the metadataflow for which metadata have been reported.	*	Yes
flowVersion	A string compliant with the *VersionType* defined in the SDMX Open API specification	The version of the metadataflow for which data have been reported.	*	Yes
providerRef	A string identifying the metadata provider. The syntax is agency id, provider id, separated by a ",". For example: AGENCY_ID,PROVIDER_ID. In case the string only contains one out of these 2 elements, it is considered to be the provider id, i.e. all,PROVIDER_ID.	The provider of the data (or metadata) to be retrieved. If not supplied, the returned message will contain data (or metadata) provided by any provider. Its a common use case in SDMX-based web services that the provider id is sufficient to uniquely identify a data provider. Should this not be the case, the agency can be used, in conjunction with the provider id, in order to uniquely identify a data provider. The OR operator is supported using the + character. For example, the following value can be used to indicate that the metadataset should be provided by the Swiss National Bank (CH2) or Central Bank of Norway (NO2): CH2+NO2.	*	Yes
detail	*String*	This attribute specifies the desired amount of information to be returned. For example, it is	**full**	No

Copyright © SDMx TWG 2020

Parameter	Type	Description	Default	Multiple values
		possible to instruct the web service to return only basic information about the metadataset (i.e.: id, dataprovider id, version and name). Most notably, metadata attributes of a metadataset will not be returned Possible values are: (1) `full`: all available information for all returned metadatasets should be returned. (2) `allstubs`: all returned metadatasets should be returned as stubs, i.e. only containing identification information and the metadataset' name.		

Notes:

- Default values do not need to be supplied if they are the last element in the path.

EXAMPLES

- To retrieve all Metadatasets reported by the central bank of France (FR2) against the ECB METHODOLOGY metadataflow (any version)

```
https://ws-entry-point/metadata/metadataflow/ECB/METHODOLOGY/*/FR2
```

- To retrieve Metadatasets from all metadata providers, for the latest stable version of the METHODOLOGY metadataflow, without the reported metadata attributes

```
https://ws-entry-point/metadata/metadataflow/*/METHODOLOGY/+/?detail=allstubs
```

By structure

OVERVIEW

These queries enable clients to request all metadata sets which are reported against one or more structures, as such the syntax for defining which structures to find metadata for follows the same syntax as the structure resource.

SYNTAX

Copyright © SDMx TWG 2020

```
protocol://ws-entry-point/metadata/structure/{artefactType}/{agencyID}/{resourceID}/{version}?{detail}
```


Copyright © SDMx TWG 2020

Parameter	Type	Description	Default	Multiple values?
artefactType	One of the following types: datastructure, metadatastructure, categoryscheme, conceptscheme, codelist, hierarchy, hierarchyassociation, valuelist, organisationscheme, agencyscheme, dataproviderscheme, dataconsumerscheme, organisationunitscheme, dataflow, metadataflow, reportingtaxonomy, provisionagreement, structuremap, representationmap, conceptschememap,categoryschememap, organisationschememap, process, categorisation, dataconstraint, metadataconstraint, transformationscheme, rulesetscheme, userdefinedoperatorscheme, customtypescheme, namepersonalisationscheme, vtlmappingscheme	The type of structural metadata to be returned.	*	No
agencyID	A string compliant with the SDMX *common:NCNameIDType*	The agency maintaining the artefact to be returned. It is possible to set more than one agency, using `,` as separator (e.g. BIS,ECB).	*	Yes
resourceID	A string compliant with the SDMX *common:IDType*	The id of the artefact to be returned. It is possible to set more than one id, using `,` as separator (e.g. CL_FREQ,CL_CONF_STATUS).	*	Yes
version	A string compliant with the SDMX *semantic versioning* rules	The version of the artefact to be returned. It is possible to set more than one version, using `,` as separator (e.g. 1.0.0,2.1.7).	~	Yes
detail	*String*	This attribute specifies the desired amount of information to be returned. For example, it is possible to instruct the web service to return only basic information about the metadataset (i.e.: id, dataprovider id, version and name). Most notably, metadata attributes of a metadataset will not be returned Possible values are: (1) `full`: all available information for all returned metadatasets should be returned. (2) `allstubs`: all returned metadatasets should be returned as	**full**	No

Copyright © SDMx TWG 2020

Parameter	Type	Description	Default	Multiple values?
		stubs, i.e. only containing identification information and the metadataset' name.		

EXAMPLES

- To retrieve metadatasets against version 1.0.0 of the DSD with id ECB_EXR1 maintained by the ECB, as well as the code lists and the concepts used in the DSD:

```
https://ws-entry-point/metadata/datastructure/ECB/ECB_EXR1/1.0.0references=children&detail=referencepartial
```

- To retrieve metadatasets reported against the latest version of the DSD with id ECB_EXR1 maintained by the ECB, without the code lists and concepts of the DSD:

```
https://ws-entry-point/metadata/datastructure/ECB/ECB_EXR1
```

- To retrieve metadatasets against any DSD maintained by the ECB (latest version), as well as the dataflows using these DSDs:

```
https://ws-entry-point/metadata/datastructure/ECB?references=dataflow
```

- To retrieve metadatasets (as stubs) against any Dataflow:

```
https://ws-entry-point/metadata/dataflow/?detail=allStubs
```

Response types

The following media types can be used with *reference metadata* queries:

- **application/vnd.sdmx.metadata+json;version=2.0.0**
- application/vnd.sdmx.metadata+xml;version=3.0.0
- application/vnd.sdmx.metadata+csv;version=1.0.0
- application/vnd.sdmx.genericmetadata+xml;version=2.1
- application/vnd.sdmx.structurespecificmetadata+xml;version=2.1

The default format is highlighted in **bold**.

Copyright © SDMx TWG 2020

6.1.5 Schema queries

Overview

Schema queries allow retrieving **the definition of data validity for a certain context**. The service must take into account the constraints that apply within that context (e.g. dataflow).

This is typically used for **validation and/or communication purposes**, for example as a way to inform providers about the data they are expected to report.

Syntax

Copyright © SDMx TWG 2020

```
protocol://ws-entry-point/schema/{context}/{agencyID}/{resourceID}/{version}?{dimensionAtObservation}
```

Parameter	Type
context	One of `datastructure`, `metadatastructure`, `dataflow`, `metadataflow`, `provisionagreement`, `metadataprovisionagreement`
agencyID	A string compliant with the *SDMX common:NCNameIDType*
resourceID	A string compliant with the SDMX *common:IDType*
version	A string compliant with the SDMX *semantic versioning* rules
dimensionAtObservation	A string compliant with the SDMX *common:NCNameIDType*

Note: Mandatory parameters are highlighted in **bold**.

Copyright © SDMx TWG 2020

Response types

The following media types can be used with *schema* queries:

- **application/vnd.sdmx.schema+json;version=3.0.0**
- application/vnd.sdmx.schema+xml;version=3.0.0
- application/vnd.sdmx.structure+xml;version=3.0.0
- application/vnd.sdmx.structure+json;version=2.0.0
- application/vnd.sdmx.schema+xml;version=2.1
- application/vnd.sdmx.structure+xml;version=2.1
- application/vnd.sdmx.structure+json;version=1.0.0

The default format is highlighted in **bold**.

The *schema* formats are meant to be used for **validation** purposes (i.e. to validate SDMX-ML and SDMX-JSON data files).

The *structure* formats are meant to be used for **communication** purposes. In that case, the response must only include the following types of artefact: (meta)data structures, codelists, concept schemes, agency schemes and, optionally, constraints. The various item schemes must only contain the relevant items: Codelists must only contain the codes that are allowed after applying the constraints up to the specified *context*; Concept schemes must only contain the concepts that are used by the data structure; Agency schemes must only contain agencies maintaining artefacts that are part of the response. Constraints may be added to the response in case multiple components reference the same codelist. For example, let's consider a data structure where two components, reference area and counterparty country, reference the same codelist (containing 10 codes). For reference area, only code A is allowed, while, for counterparty country, only codes B & C are allowed. The codelist returned in the response must only include the 3 allowed codes (A, B and C), as mentioned above. In addition, constraints should also be returned in the response, to indicate which codes are allowed as reference area (A) and which codes are allowed as counterparty country (B and C).

Examples

- Retrieve the data validity (in SDMX-JSON) for the **latest stable version** of the `EXR_WEB` dataflow maintained by the `ECB`. The constraints attached to the dataflow and the data structure used by the dataflow must be considered. The returned codelists may only contain the codes that are allowed after applying the constraints. The returned concept schemes may only contain the concepts used by the data structure. The returned agency schemes may only contain the agencies maintaining artefacts that are part of the response. Special characters have not been percent-encoded for the sake of simplicity.

```
GET /schema/dataflow/ECB/EXR_WEB/+
Accept: application/vnd.sdmx.structure+json;version=2.0.0
```

- Retrieve an XML schema to validate data submissions for the **latest stable version** of the `EXR_WEB` dataflow maintained by the `ECB`. The data are expected to be packaged as time series (i.e. All dimensions are available at the series-level, except for the `TIME_PERIOD` dimension, which is found at the observation-level). The constraints attached to the dataflow and the data structure used by the dataflow must be considered.

```
GET /schema/dataflow/ECB/EXR_WEB/+/?dimensionAtObservation=TIME_PERIOD
Accept: application/vnd.sdmx.schema+xml;version=3.0.0
```


Copyright © SDMx TWG 2020

6.1.6 Data availability queries

> **Note**
>
> To enhance readability of the large tables on this page, navigation panels are hidden. To return to the introduction, click REST API.

Overview

Availability queries allow to see what data are available for a structure (data structure, dataflow or provision agreement), based on a data query. It returns a `DataConstraint`, i.e. structural metadata, and is therefore similar to the other structural metadata queries but the query itself is more akin to a data query.

Syntax

Copyright © SDMx TWG 2020

```
protocol://ws-entry-point/availability/{context}/{agencyID}/{resourceID}/{version}/{key}/{componentId}?{c}&{updatedAfter}&{references}&{mode}
```


Copyright © SDMx TWG 2020

Parameter	Type	Description	Default
context	One of the following: `datastructure`, `dataflow`, `provisionagreement`	Data can be reported against a data structure, a dataflow or a provision agreement. This parameter allows selecting the desired context for data retrieval.	*
agencyID	A string compliant with the SDMX *common:NCNameIDType*	The agency maintaining the artefact for which data have been reported.	*
resourceID	A string compliant with the SDMX *common:IDType*	The id of the artefact for which data have been reported.	*
version	A string compliant with the *VersionType* defined in the SDMX Open API specification	The version of the artefact for which data have been reported.	*
key	A string compliant with the *KeyType* defined in the SDMX Open API specification.	The combination of dimension values identifying the slice of the cube for which data should be returned. Wildcarding is supported via the `*` operator. For example, if the following key identifies the bilateral exchange rates for the daily US dollar exchange rate against the euro, D.USD.EUR.SP00.A, then the following key can be used to retrieve the data for all currencies against the euro: D.*.EUR.SP00.A	*
componentId	A string compliant with the SDMX common: IDType	The id of the Dimension for which to obtain availability information about. In the case where this information is not provided, data availability will be provided for all Dimensions.	*
c	Map	Filter data by component value. For example, if a structure defines a frequency dimension (FREQ) and the code A (Annual) is an allowed value for that dimension, the following can be used to retrieve annual data: `c[FREQ]=A`. The same applies to attributes (e.g. `c[CONF_STATUS]=F`) and measures. Multiple values are supported, using a comma (`,`) as separator: `c[FREQ]=A,M`. In case of attributes that support multiple values, the plus (`+`) can be used to list all values that an attribute must have. For example, to indicate that ATTR1 must either be A or (B AND M), use the following: `c[ATTR1]=A,B+M`. Operators may be used too (see table with operators below). This parameter can be	

Copyright © SDMx TWG 2020

Parameter	Type	Description	Default
		used in addition, or instead of, the `key` path parameter. This parameter may be used multiple times (e.g. `c[FREQ]=A,M&c[CONF_STATUS]=F`)	
updatedAfter	xs:dateTime	The last time the query was performed by the client. If this attribute is used, the returned message should only include the dimension values for the data that have changed since that point in time (updates and revisions). This should include the dimension values for data that have been added since the last time the query was performed (INSERT), data that have been revised since the last time the query was performed (UPDATE) and data that have been deleted since the last time the query was performed (DELETE). If no offset is specified, default to local time of the web service.	
references	String	This attribute instructs the web service to return (or not) the artefacts referenced by the DataConstraint to be returned. Possible values are: “codelist”, “datastructure”, “conceptscheme”, “dataflow”, “dataproviderscheme”, “none”, “all”. The keyword “all” is used to indicate the inclusion of dataflow, datastructure, conceptschemes, dataproviderschemes and codelists. Note, in the case ItemSchemes are returned (i.e. Codelists, ConceptSchemes and DataProviderSchemes), only the items used by the DataConstraint will be included (i.e. concepts used by the constrained dimensions; codes for which data are available; data providers that have provided data available according to the CubeRegion). Additionally for Codelists parent codes will be included in the response if the child codes are in the returned codelist, irrespective of whether they are referenced by the DataConstraint. If this results in a partial list, the isPartial attribute will be set to true.	**none**

Copyright © SDMx TWG 2020

Parameter	Type	Description	Default
mode	String	This attribute instructs the web service to return a DataConstraint which defines a Cube Region containing values which will be returned by executing the query (mode=”exact”) vs a Cube Region showing what values remain valid selections that could be added to the data query (mode=”available”). A valid selection is one which results in one or more series existing for the selected value, based on the current data query selection state defined by the current path parameters.	**exact**

Notes:

- Default values do not need to be supplied if they are the last element in the path.
- Operators can be used to refine the applicability of the `c` query parameter:

Operator	Meaning	Note
eq	Equals	Default if no operator is specified and there is only one value (e.g. `c[FREQ]=M` is equivalent to `c[FREQ]=eq:M`)
ne	Not equal to	
lt	Less than	
le	Less than or equal to	
gt	Greater than	
ge	Greater than or equal to	
co	Contains	
nc	Does not contain	
sw	Starts with	
ew	Ends with	

Operators appear as prefix to the component value(s) and are separated from it by a `:` (e.g. `c[TIME_PERIOD]=ge:2020-01+le:2020-12`).

As already mentioned, the response from the Data Availability API is an SDMX DataConstraint containing a CubeRegion which defines the distinct Values for each Dimension of the data. These distinct values contained in the CubeRegion are determined by the server based on the data query presented to this API. The meaning of the distinct values depends on the response mode.

Copyright © SDMx TWG 2020

RESPONSE MODE

The API response to the data query depends on the **mode** the API is called with. `mode` is a query parameter to the API, and if not provided defaults to `exact`. Depending on the mode, the returned values represent either:

- The **distinct values** which will be contained in the resultant dataset, should the data query be run against the Data API (**mode=exact**);
- The **valid future selections** that could be passed to the Data API based on the current data query selections (**mode=available**).

To highlight the difference between the response mode, consider the following dataset.

Reference Area	Employment Status	Sex
UK	EMP	M
FR	EMP	M
FR	UEMP	F

The client calls the availability API with the query **Reference Area=FR** and **Sex=F**.

With `mode=exact`, the response is:

Dimension	Values
Reference Area	FR
Employment Status	UEMP
SEX	F

Mode exact has found only one series matches this query, and returned to the client the distinct values per dimension. In this case there is only one distinct value per dimension.

With `mode=available`, the response is:

Dimension	Values
Reference Area	FR
Employment Status	UEMP
SEX	**M**, F

Mode available has determined that the UK is no longer a valid selection. If the UK were chosen, no data would be returned for that Reference Area. If the user were to include Employment Status=EMP this would result in no data being returned at all. The difference in this mode is that the service has determined that SEX=M is a valid selection, if this was to be added to the data query it would result in more series being returned.

If the user was to add to their query **Sex=M**, the query becomes **Reference Area=FR** and **Sex=M + F** and the response would change as follows.

With `mode=exact`:

Dimension	Values
Reference Area	FR
Employment Status	EMP, UEMP
SEX	M, F

Copyright © SDMx TWG 2020

The service has detected two series that match the query criteria, and responded with the distinct code values for each dimension for those matches.

With `mode=available`:

Dimension	Values
Reference Area	**UK**, FR
Employment Status	EMP, UEMP
SEX	M, F

Reference Area UK has now become available, this is because if the user were to now select Reference Area=UK they would add additional series to the query response. In addition both Employment Status values are available. The Employment Status dimension currently has no query selections on it, this means the dimension has no filters imposed. The service has determined that the user can impose a filter for either EMP or UEMP and the inclusion of these filters will not result in an empty dataset being returned.

TEMPORAL COVERAGE

For DSDs that have a time dimension, temporal coverage can be included in the DataConstraint's `ReferencePeriod` element. The `ReferencePeriod` is used to indicate the earliest and latest observation dates available for the sub-cube of data based on the current data query.

This information may not be included in the response if the service does not have access to this information.

METRICS

The `DataConstraint` may define up to two additional Annotations used to capture the metrics for the **number of series** and the **number of observations** which will be returned if the data query presented to this API were run against the data API.

The `Metrics` annotations are attached to the `DataConstraint`, and have `AnnotationType` of `sdmx_metrics` an Id of `series_count` or `obs_count` depending on the metric being reported, and the annotation title is used to report the count, which is a positive integer value.

Metrics are only included if the service has the information available to provide the count. A request for metrics may include only series counts, or no metrics at all depending on the service.

AVAILABILITY FOR A SINGLE DIMENSION

If a client application is only interested in the data available for a single dimension of the dataset, for example if the client is building a code picker form for only the 'Reporting Country' dimension, then the client is able to specify that this using the `componentId` path parameter. If this is the case, the CubeRegion will only include information of the distinct values for the specified dimension.

REFERENCED STRUCTURES

The client is able to include referenced structures used by the constraint by using the `references` parameter, for example if a dimension is coded, and references are returned, the dimension's codelist will be included in the response. Importantly the codelist will only include codes which are part of the constraint.

Included references are:

- `Dataflow` - referenced directly by the constraint
- `Data Structure Definition` - referenced by the dataflow
- `Codelist` - which contain only the codes identified in the constraint, with any parent codes also included even if they are not included in the constraint
- `Concept Scheme` - which contain only the concepts used by the constraint
- `Data Provider Scheme` - which contain only the data providers who have provided data for the dataflow

Copyright © SDMx TWG 2020

Response types

The following media types can be used with *availability* queries:

- **application/vnd.sdmx.structure+json;version=2.0.0**
- application/vnd.sdmx.structure+xml;version=3.0.0
- application/vnd.sdmx.structure+xml;version=2.1
- application/vnd.sdmx.structure+json;version=1.0.0

The default format is highlighted in **bold**.

Examples

- To retrieve the distinct values for each Dimension for the entire ECB_EXR1_WEB dataflow. The metadata used to decode the code ids and concept ids into human readable labels is also requested.

```
https://ws-entry-point/availability/dataflow/ECB/ECB_EXR1_WEB/?references=all
```

- For the data query M.*.EUR.SP00.A supplied for the ECB_EXR1_WEB dataflow, the client requests information about which dimension values remain valid for inclusion into the query:

```
https://ws-entry-point/availability/dataflow/ECB/ECB_EXR1_WEB/1.0.0/M.*.EUR.SP00.A?mode=available
```

- For the data query M.*.EUR.SP00.A supplied for the ECB_EXR1_WEB dataflow, the client requests information about which dimension values will be returned as a result of executing the query:

```
https://ws-entry-point/availability/dataflow/ECB/ECB_EXR1_WEB/*/M.*.EUR.SP00.A?mode=exact
```

- As `exact` is the default value for mode, the query above can also be written as follows:

```
https://ws-entry-point/availability/dataflow/ECB/ECB_EXR1_WEB/*/M.*.EUR.SP00.A
```

A typical use case: Executing queries via data query forms

STEP 1: BUILDING THE DATA QUERY FORM

A data query form enables users of the UI to build a data query against a dataflow by selecting dimensions, and choosing filters for the dimension. For example the user is able to click ‘Reporting Country’ and then select the codes ‘United Kingdom’, ‘France’, ‘Germany’. The code labels are presented in the users chosen locale where possible.

The use case can be supported as follows:

1. Query Data Availability API with a query for all data for the dataflow, and include all references:

```
https://ws-entry-point/availability/dataflow/ECB_EXR1_WEB/?references=all
```

2. The response includes the DataConstraint and the Data Structure Definition. We can iterate the dimensions to build the Dimension picker. For each dimension, we can get the concept, as this provides the human readable label (ideally in the chosen locale, if available). The Cube Region constraint provides the available values for the dimension. If the dimension is coded, then the codelist can be used to get the human readable label in the chosen locale. The code will additionally provide any hierarchy information. An HTML checkbox is created for each available dimension value.

STEP 2: UPDATE THE DATA QUERY FORM BASED ON CODE SELECTION STATE

The use case is to disable or enable values for dimension from being selected based on the current query state. The choice to disable a value from selection is based on the answer to the question: If this value is selected will it; a) return no data: b) not add any more data?

Copyright © SDMx TWG 2020

Let’s consider the following dataset:

Reference Area	Employment Status	Sex
UK	EMP	M
FR	EMP	M
FR	UEMP	F

1. The user selects `Reference Area=UK`. `Employment Status=UEMP` and `Sex=F` are no longer valid selections. Selecting either of these two codes and then running the data query will fail to return any data.
2. The user is however able to add to their query the `Reference Area=FR`. Re-running the availability query results in `Employment Status=UEMP` and `Sex=F` becoming valid selections again, as the inclusion of either of these values will result in data being returned.
3. If the user now adds the selection `Sex=F`, then the Reference `Area=UK` is no longer a valid selection.

The use case can be supported as follows:

1. When the user adds or removes a data query filter by checking or unchecking a checkbox, call the Data Availability API with current data query state and `mode=available`.

```
https://ws-entry-point/availability/dataflow/EMPLOYMENT/UK.*.M,FR.*.M?mode=available
```

2. The response will include only the values which remain valid selections. Use this information to enable or disable the dimension values.

BONUS: PREVENT THE CLIENT FROM EXECUTING A DATA QUERY WHICH EXCEEDS SERVER LIMIT

Let’s imagine a situation where the service has imposed a limit on 2000 series per query. If the service receives a data query which results in more series being returned, then the client will receive an error response. The idea is then to prevent a user inadvertently running a query from the web User Interface which exceeds the limit imposed by this service.

The use case can be supported as follows:

1. When the user adds or removes a data query filter by checking or unchecking a checkbox, call the Data Availability API with the current data query state.
2. The response will include the Cube Region constraint. If the Cube Region constraint has an Annotation with type `sdmx_metrics` and an Id of `series_count` then obtain the count via the Annotation’s title. Use this count to either enable or disable the ‘Run Query’ button.

Copyright © SDMx TWG 2020

6.1.7 Query syntax for specific, latest or all artefact versions

The SDMX REST API supports retrievals of specific, latest or all artefact versions according to the SDMX semantic versioning & dependency management rules by means of the SDMX REST API wildcard operators.

Let's use the version form `X.Y.Z-EXT`, where - `X` is a non-negative number representing the MAJOR version part - `Y` is a non-negative number representing the MINOR version part - `Z` is a non-negative number representing the PATCH version part - `-EXT` is a constrained character string representing the version EXTENSION part

Capital letters `X`, `Y`, `Z` and `EXT` are used for the queried version parts and the exactly corresponding response version parts. In case the response version parts could be different from the version parts used in the query, we will use the small characters `x`, `y`, `z` and `ext` instead. Last version parts are omitted as required.

Query for a specific version

- `X` This matches version **`X`**. Note that this version format is deprecated.
- `X.Y` This matches version **`X.Y`**.
- `X.Y.Z` This matches version **`X.Y.Z`**.
- `X.Y.Z-EXT` This matches version **`X.Y.Z-EXT`**.

Query for the latest stable (semantic) version

WITHOUT SPECIFIC MINIMUM VERSION

- `+` This matches the *latest available stable semantic* version of an artefact in form of **`x.y.z`** where `x` > 0. It is equivalent to querying for:
- `+.0.0` or
- `1+.0.0`
- `X.+.0` With `X` > 0. This matches the *latest available stable semantic* version of an artefact in form of **`X.y.z`**. It is equivalent to querying for:
- `X.0+.0`
- `X.Y.+` With `X` > 0. This matches the *latest available stable semantic* version of an artefact in form of **`X.Y.z`**. It is equivalent to querying for:
- `X.Y.0+`

WITH SPECIFIC MINIMUM VERSION

- `X+.Y.Z` With `X` > 0. This matches the *latest available stable semantic* version of an artefact in form of **`x.y.z`** where `x.y.z` >= `X.Y.Z`.
- `X.Y+.Z` With `X` > 0. This matches the *latest available stable semantic* version of an artefact in form of **`X.y.z`** where `y.z` >= `Y.Z`.
- `X.Y.Z+` With `X` > 0. This matches the *latest available stable semantic* version of an artefact in form of **`X.Y.z`** where `z` >= `Z`.

Query for the latest possibly unstable version

WITHOUT SPECIFIC MINIMUM VERSION

- `~` This matches the *latest available* version of an artefact in form of **`x`**, **`x.y`**, **`x.y.z`** or **`x.y.z-ext`**.
- `~.0` This matches the *latest available* version of an artefact in form of **`x.y`**. It is equivalent to querying for:
- `0~.0`
- `~.0.0` This matches the *latest available* version of an artefact in form of **`x.y.z`** or **`x.y.z-ext`**. It is equivalent to querying for:
- `0~.0.0`
- `X.~` This matches the *latest available* version of an artefact in form of **`X.y`**. It is equivalent to querying for:

Copyright © SDMx TWG 2020

- `X.0~`
- `X.~.0` This matches the *latest available* version of an artefact in form of **`X.y.z`** or **`X.y.z-ext`**. It is equivalent to querying for:
- `X.0~.0`
- `X.Y.~` This matches the *latest available* version of an artefact in form of **`X.Y.z`** or **`X.Y.z-ext`**. It is equivalent to querying for:
- `X.Y.0~`

WITH SPECIFIC MINIMUM VERSION

- `X~.Y` This matches the *latest available* version of an artefact in form of **`x.y`** where `x.y` >= `X.Y`.
- `X~.Y.Z` This matches the *latest available* version of an artefact in form of **`x.y.z`** or **`x.y.z-ext`** where `x.y.z` >= `X.Y.Z`.
- `X.Y~` This matches the *latest available* version of an artefact in form of **`X.y`** where `y` >= `Y`.
- `X.Y~.Z` This matches the *latest available* version of an artefact in form of **`X.y.z`** or **`X.y.z-ext`** where `y.z` >= `Y.Z`.
- `X.Y.Z~` This matches the *latest available* version of an artefact in form of **`X.Y.z`** or **`X.Y.z-ext`** where `z` >= `Z`.

Query for all versions

WITHOUT SPECIFIC MINIMUM VERSION

- `*` This matches *all* versions of an artefact in form of **`x`**, **`x.y`**, **`x.y.z`** or **`x.y.z-ext`**
- `*.0` This matches *all* versions of an artefact in form of **`x.y`**. It is equivalent to querying for:
- `0*.0`
- `*.0.0` This matches *all* versions of an artefact in form of **`x.y.z`** or **`x.y.z-ext`**. It is equivalent to querying for:
- `0*.0.0`
- `X.*` This matches *all* versions of an artefact in form of **`X.y`**. It is equivalent to querying for:
- `X.0*`
- `X.*.0` This matches *all* versions of an artefact in form of **`X.y.z`** or **`X.y.z-ext`**. It is equivalent to querying for:
- `X.0*.0`
- `X.Y.*` This matches *all* versions of an artefact in form of **`X.Y.z`** or **`X.Y.z-ext`**. It is equivalent to querying for:
- `X.Y.0*`

WITH SPECIFIC MINIMUM VERSION

- `X*.Y` This matches *all* versions of an artefact in form of **`x.y`** where `x.y` >= `X.Y`.
- `X*.Y.Z` This matches *all* versions of an artefact in form of **`x.y.z`** or **`x.y.z-ext`** where `x.y.z` >= `X.Y.Z`.
- `X.Y*` This matches *all* versions of an artefact in form of **`X.y`** where `y` >= `Y`.
- `X.Y*.Z` This matches *all* versions of an artefact in form of **`X.y.z`** or **`X.y.z-ext`** where `y.z` >= `Y.Z`.
- `X.Y.Z*` This matches *all* versions of an artefact in form of **`X.Y.z`** or **`X.Y.z-ext`** where `z` >= `Z`.

`OR` operator

Any of the version query strings above can be combined with the OR operator (`,`). For example: - `1~.2.0,1.2.0+` - `+,1.2.1*`

Unsupported syntax

The following version query syntax cannot be correctly interpreted and is **not supported**.

Using a positive number in version parts after a part that is fully wildcarded with `+`, `~` or `*`:
- `+.2.3` Use `+.0.0` instead. - `1.+.3` Use `1.+.0` instead. - `~.2` Use `~.0` instead. - `~.2.3` Use `~.0.0` instead. - `1.~.3` Use `1.~.0` instead. - `*.2` Use `*.0` instead. - `*.2.3` Use `*.0.0` instead. - `1.*.3` Use `1.*.0` instead.

Using `+` (querying for stable versions always having 3 parts) with only 2 version parts:
- `+.0` Use `+` or `+.0.0` instead. - `2.3+` Use `2.3+.0` instead.

Copyright © SDMx TWG 2020

Using two or more of the `+`, `~` or `*` wildcard operators within the same version:
- `~.0.*` Use `~` or `~.0.0` instead. - `3.2*.1+` Use `3.2*.1`, `3.2.1+` or `3.2*.1,3.2.1+` instead. - `3.2+.1+` Use `3.2+.1` instead.

Copyright © SDMx TWG 2020

6.1.8 HTTP Content-Negotiation

HTTP Content Negotiation is a mechanism offered by HTTP that allows clients to indicate their preferred representation, language, encoding, etc. for a resource.

Format selection

Using the HTTP Content Negotiation mechanism, the client specifies the desired format for the resource using the Accept header.

Along with official media types (e.g.: text/html, application/xml, etc), the HTTP standard also defines a syntax, which the SDMX RESTful API leverages, allowing a service to define its own types:

```
application/vnd.sdmx.[type]+[format];version=[version]
```

A few examples are listed below:

- application/vnd.sdmx.structure+json;version=2.0.0
- application/vnd.sdmx.data+json;version=2.0.0
- application/vnd.sdmx.structure+xml;version=3.0.0

The full list of media types is available in the *Response types* section of the various query pages:

- Data queries
- Structure queries
- Reference metadata queries
- Schema queries
- Data availability queries

Selection of the appropriate language

The Accept-Language header is used to indicate the language preferences of the client. Multiple values, along with their respective weights, are possible. For example:

```
Accept-Language: ru, en-gb;q=0.8, en;q=0.7
```

Enabling data compression

Compression can be enabled using the appropriate Accept-Encoding header.

Helping web caches and Content Delivery Networks (CDN)

The Vary header is used to indicate the list of headers that are relevant for a particular service.

For example, services that offer data in multiple formats will rely on the HTTP Accept header, but this header will likely be irrelevant for services that support only one format. Using the `Vary` header to indicate which headers are effectively used by the server helps web caches and Content Delivery Networks to build appropriate cache keys.

Copyright © SDMx TWG 2020

6.1.9 Error handling and status information

RESTful web services use HTTP status codes to indicate errors. In principle, any code defined by HTTP can therefore be returned by an SDMX RESTful web service but a few of the most common ones are described below.

Codes below 400 are considered normal (i.e. non-problematic). Codes between 400 and 499 are considered client errors. Codes greater or equal to 500 indicate a server error or problem.

HTTP status code	Description
200	This is the standard response for successful HTTP requests. With `GET` requests, it is used to indicate that the request was successfully processed and that the data are available in the body of the response.
204	If the result from the query is empty, the web service should return this code. `204` is preferred to `404` in this case, as a lack of results matching a query is an “empty but still healthy state”.
304	This is used to indicate that there are no changes since the version specified by the request headers `If-Modified-Since` or `If-None-Match`. The response does not include matching data, as the data downloaded previously is still valid.
400	This error code is used when the query doesn’t comply with the SDMX REST API.
401	For use when authentication is needed but has failed or has not yet been provided.
403	For use when authentication has been provided correctly, but user is not authorized to access the query results.
406	If a client asks for a resource representation that the web service does not offer, a 406 HTTP status code should be returned.
413	The request would result in a response that is larger than the server is willing or able to process. In case the service offers the possibility to users to download the results of large queries at a later stage (for instance, using asynchronous web services), the web service may choose to indicate the (future) location of the file, as part of the error message.
414	This status code (URI Too Long) indicates that the server is refusing to service the request because the request-target is longer than the server is willing to interpret. See the wiki page for information on the URL length.
422	A web service should return this error when a request is syntactically correct but fails a semantic validation or violates agreed business rules.
500	The web service should return this error code when none of the other error codes better describes the reason for the failure of the service to provide a meaningful response.
501	All SDMX web services should implement all the standard interfaces, even if their only function is to return this error message. This eases interoperability between SDMX compliant web services and it also eases the development of generic SDMX web services clients. If the web service has not yet implemented one of the methods defined in the API, then it should return this error.
503	If a web service is temporarily unavailable because of maintenance or for some other similar reasons, then it should return this error code.

Copyright © SDMx TWG 2020

6.1.10 Structure maintenance

Applying the HTTP methods to the SDMX REST API for Structural Metadata maintenance

The following sections describe the foreseen HTTP methods for the maintenance of SDMX Structural Metadata via the REST API.

CREATE

Inserting or creating new SDMX Structural Metadata is detailed in this secion.

Artefact (Structure)

Submitting SDMX Artefacts in bulk, either of the same or of different types, is achieved with an HTTP `POST` method. Creating new Artefact(s) may be issued by: - `POST` one or more Maintainable Artefacts under the proper resource type, e.g. for Codelists: `/structure/codelist/` - `POST` one or more Maintainable Artefacts under the abstract structure resource type, e.g. `/structure/`

Client

In order to create Artefacts, the client:

- MAY set the `Accept` header to indicate the preferred response format;
- MUST set the `Content-type` header according to the format of the submitted Artefact(s);
- MUST include in the request body, one or more Maintainable Artefacts in the SDMX format indicated in the `Content-type` header and of the SDMX type indicated in the resource, i.e.:
- any set of Maintainable Artefacts under resource `/structure/`
- a set of specific type of Maintainable Artefacts under the corresponding resource type, e.g. for Codelists: `/structure/codelist/`

Server

In response to Artefact(s) creation, the server:

- MUST return `201` upon successful creation (or `207` for partial success);
- MUST return a `SubmitStructureResponse` message with the result of the action(s), according to the `Accept` header, or the default, if the `Accept` type is not supported (currently available only in SDMX-ML format);
- MUST set the `Content-type` according to the returned format;
- MAY set the `Location` header to point to the created/primary resource/Artefact (only one instance is allowed);

Response

The `SubmitStructureResponse` message must be returned in any case (success, partial success, failure). This is defined as part of the `RegistryInterface` messages. The details of the message are explained in section **Structure response message**, below.

Copyright © SDMx TWG 2020

Especially when different results occur on the Artefacts (e.g. partial success), the server should act as follows: - Return a multi-status return code (like `207`); - Return a `JSON` or `XML` message with the details of the result (currently available only in SDMX-ML);

Method	Exists	Versioning respected	Is Referenced	References exist/ provided	Response Code
POST	F	T	-	T	`201` (successful) or `207` (partially successful)
POST	F	T	-	F	`409` missing references
POST	F	F	-	I	`409` failure to respect the versioning rules

T: True, F: False, I: Irrelevant, -: Not applicable

NOTE: The rules for versioning in SDMX 3.0 are based on Semantic Versioning. In addition, Artefacts with no version are supported. SDMX 3.0 does not specify rules for legacy versioning - i.e. versioning prior to SDMX 3.0. The versioning rules in that case are up to the organisation using them.

UPDATE

Updating existing SDMX Structural Metadata is detailed in this section.

Replace full Artefacts

Following the current SDMX practices, updating (replacing) means providing the new version of any Maintainable Artefact. According to RFC7231, `PUT` is the proper way to update a resource, as identified by the URL, but `POST` may also be used in case more than one resources need to be updated.

The `POST` method may be used like this:

- under `/structure` for different types of Maintainable Artefacts, i.e. an SDMX Structure message;
- under `/structure/{maintainable}` for Maintainable Artefacts of type `{maintainable}`, i.e. an SDMX Structure message including only a specific type of Structures; e.g. an SDMX message with one or more Dataflows, under `/structure/dataflow`;

The `PUT` method may be used like this:

- under `/structure/{maintainable}/{identifier}` for the Maintainable Artefact of type `{maintainable}`, identified by `{identifier}`, i.e. an SDMX DSD message with `ESTAT:NA_MAIN(1.1)` under `/structure/datastructure/ESTAT/NA_MAIN/1.1`

The result, following the current practices of updates in SDMX, is to **completely replace** the identified Artefact.

Client

Copyright © SDMx TWG 2020

In order to update Artefact(s), the client:

- MAY set the `Accept` header to indicate the preferred response format;
- MUST set the `Content-type` header according to the format of the submitted Artefact(s);
- MUST include in the request body, one or more Maintainable Artefacts in the SDMX format indicated in the `Content-type` header and of the SDMX type indicated in the resource, i.e.:
- for `POST`:
- any set of Maintainable Artefacts under resource `/structure/`
- one or more Maintainable Artefacts of the specific type under the corresponding resource type, e.g. Dataflows under `/structure/dataflow/`
- for `PUT`:
- one Maintainable Artefact, of the specific type under the corresponding resource type and identified according to the resource parameters, e.g., DSD `ESTAT:NA_MAIN(1.1)` under `/structure/datastructure/ESTAT/NA_MAIN/1.1`

Server

In response to Artefact(s) update, the server:

- MUST respond with `200` in case of successful update;
- MUST return a `SubmitStructureResponse` message with the result of the action(s), according to the `Accept` header, or the default, if the `Accept` type is not supported (currently available only in SDMX-ML);
- MUST set the `Content-type` according to the returned format;
- MUST respond with `422` in case of resource type mismatch, i.e. the resource type identified in the URL does not match the Artefact type(s) of the included SDMX Artefact(s), or the identification of the Artefact (in case of `PUT`) does not match the resource path parameters.
- MUST respond with `404` if the resource was not found;

Response

The `SubmitStructureResponse` message must be returned in any case (success, partial success, failure). This is defined as part of the `RegistryInterface` messages. The details of the message are explained in section **Structure response message**, below.

Copyright © SDMx TWG 2020

The following matrix summarises the returned `HTTP` response codes.

Method	Exists	Versioning respected	Is Referenced	References exist/ provided	Return Code
PUT/POST	T	T	F	T	`200` (successful) or `207` (partially successful)
PUT/POST	T	T	F	F	`409`
PUT/POST	T	T	T	T	`200` if update is possible - `409` if references would break
PUT/POST	T	F	-	-	`409` in case of changes that violate the versioning requirements
PUT/POST	-	-	-	-	`422` see section **Server**
PUT	F	-	-	-	`404`

T: True, F: False, I: Irrelevant, -: Not applicable

Partially update Item Schemes

In the case of Item Schemes, it is possible to submit a partial Item Scheme, including the Items that must be updated. The usage of `POST` and `PUT` is the same as in the case of full replacement. The only difference is that the submitted Items Schemes to be partially updated (i.e. not completely replaced) must include the flag `isPartial="true"`.

In this case, any Item included in the submitted partial Item Scheme: - fully replaces the Item, if it existed in the stored Item Scheme; the Item stays in the same position; - is added as a new Item, if it did not exist - in a selected position, if it is submitted relatively to existing Items; - else, at the end of the Item Scheme.

For more details on nested Items (like Categories), please refer to section **Nested Items** below.

In addition to the Items, the Item Scheme details are also updated accordingly, i.e. the names, description, annotations and any other attributes. Names and Descriptions are replaced if they exist for the submitted language (i.e. the value of their `lang` attribute), or simply added, if new. Annotations and other Item Scheme attributes are fully replaced.

Client

Copyright © SDMx TWG 2020

In order to partially update an Item Scheme, the client:

- MAY set the `Accept` header to indicate the preferred response format;
- MUST set the `Content-type` header according to the format of the submitted Item Scheme(s);
- MUST include the flag `isPartial="true"` in the submitted Item Scheme(s);
- MUST include in the request body:
- For `PUT`, **one** Item Scheme in the SDMX format indicated in the `Content-type` header and of the Item Scheme type indicated in the resource, e.g. a Concept Scheme under resource `/conceptscheme`;
- For `POST`, one or more Item Schemes in the SDMX format indicated in the `Content-type` header and
  - of the Item Scheme type indicated in the resource, in case of a specific Item Scheme resource, e.g. a Codelist under resource `/structure/codelist/`
  - of any Item Scheme type, in case of the `/structure` resource.

Server

In response to an Item Scheme partial update, the server:

- MUST respond with `200` in case of successful update;
- MUST return a `SubmitStructureResponse` message with the result of the action, according to the `Accept` header, or the default, if the `Accept` type is not supported (currently available only in SDMX-ML);
- MUST set the `Content-type` according to the returned format;
- MUST respond with `422` in case of resource type mismatch, i.e. the artefact identified in the URL does not match either to the resource type or identification of the included SDMX Artefact.
- MUST respond with `404` if the resource was not found;

Response

The `SubmitStructureResponse` message must be returned in any case (success, partial success, failure). This is defined as part of the `RegistryInterface` messages. The details of the message are explained in section **Structure response message**, below.

The following matrix summarises the returned `HTTP` response codes.

Method	Exists	Versioning respected	Is Referenced	Refs exist/ provided	Response Code
PUT/POST	T	T	F	T	`200` (successful) or `207` (partially successful)
PUT/POST	T	T	F	F	`409`
PUT/POST	T	T	T	T	`200` if update is possible - `409` if references would break
PUT/POST	T	F	-	-	`409` in case of changes that violate the versioning requirements
PUT/POST	-	-	-	-	`422` see section **Server**
PUT	F	-	-	-	`404`

Copyright © SDMx TWG 2020

T: True, F: False, I: Irrelevant, -: Not applicable

DELETE

Always concerns one Maintainable Artefact or one Item. For example: - A fully identified Maintainable Artefact, e.g. `/structure/codelist/SDMX/CL_FREQ/1.0` - A fully identified Item, e.g. `/structure/codelist/SDMX/CL_FREQ/1.0/M`

In case the deleted Item was acting as a parent to other Item(s), then the server should make sure that: - in case of flat Item Schemes the children become orphans, i.e. their parent is removed; - in case of nested Item Schemes, all children are also deleted.

Client

In order to delete an Artefact, the client:

- MAY set the `Accept` header to indicate the preferred response format;
- MUST fully identify exactly **one** Maintainable Artefact or **one** Item, by means of the proper URL;

Server

In response to an Artefact deletion, the server:

- MUST respond with `200` in case of successful deletion;
- MUST return a `SubmitStructureResponse` message with the result of the action, according to the `Accept` header, or the default, if the `Accept` type is not supported (currently available only in SDMX-ML);
- MUST respond with `404` if the resource was not found;

Response

The `SubmitStructureResponse` message must be returned in any case (success, failure). This is defined as part of the `RegistryInterface` messages. The details of the message are explained in section **Structure response message**, below.

The following matrix summarises the returned `HTTP` response codes.

Method	Exists	Versioning respected	Is Referenced	References exist/ provided	Response Code
DELETE	F	-	-	-	`404`
DELETE	T	F	F	I	`200`
DELETE	T	T	I	I	`409` when trying to delete a stable artefact, a deprecation strategy may be followed
DELETE	T	F	T	I	`409`

T: True, F: False, I: Irrelevant, -: Not applicable

ANNEX I - REGISTRYINTERFACE MESSAGES FOR SUBSCRIPTION AND REGISTRATION

Support for Subscriptions and Registrations are not yet part of the SDMX RESTful API. Thus, an intermediate solution for supporting them in SDMX 3.0 (after the deprecation of the SOAP API) is needed. This section describes how to use the RegistryInterface messages for submitting and retrieving Subscriptions and Registrations via the RESTful API.

Introduction

The usage of the RegistryInterface messages is foreseen via the POST verb, under resources: - `/rest/subscription` for Subscriptions - `/rest/registration` for Registrations - `/rest` for any RegistryInterface message

Copyright © SDMx TWG 2020

The available `RegistryInterface` messages are the following: - `SubmitRegistrationsRequest` - `SubmitRegistrationsResponse` - `SubmitSubscriptionsRequest` - `SubmitRegistrationsResponse` - `QueryRegistrationRequest` - `QueryRegistrationResponse` - `QuerySubscriptionRequest` - `QuerySubscriptionResponse`

In addition, the above shall remain as part of the generic `RegistryInterface` message.

Retrieving Subscriptions and Registrations

For Subscriptions: - The client may POST a `QuerySubscriptionRequest` or a `RegistryInterface/QuerySubscriptionRequest` message under resource `/rest/subscription` for retrieving Subscriptions. - As a result, a `QuerySubscriptionResponse` or a `RegistryInterface/QuerySubscriptionResponse` message will be returned as a response to describe the requested Subscriptions.

Similarly, for Registrations: - The client may POST a `QueryRegistrationRequest` or a `RegistryInterface/QueryRegistrationRequest` message under resource `/rest/registration` for retrieving Registrations. - As a result, a `QueryRegistrationResponse` or a `RegistryInterface/QueryRegistrationResponse` message will be returned as a response to describe the requested Registrations.

Maintaining Subscriptions and Registrations

For Subscriptions: - The client may POST a `SubmitSubscriptionsRequest` or a `RegistryInterface/SubmitSubscriptionsRequest` message under resource `/rest/subscription` for inserting/updating/deleting Subscriptions. - As a result, a `SubmitSubscriptionsResponse` or a `RegistryInterface/SubmitSubscriptionsResponse` message will be returned as a response to describe the outcome of the insertion/update/deletion of Subscriptions.

Similarly, for Registrations: - The client may POST a `SubmitRegistrationsRequest` or a `RegistryInterface/SubmitRegistrationsRequest` message under resource `/rest/registration` for inserting/updating/deleting Registrations. - As a result, a `SubmitRegistrationsResponse` or a `RegistryInterface/SubmitRegistrationsResponse` message will be returned as a response to describe the outcome of the insertion/update/deletion of Registrations.

ANNEX II - STRUCTURE RESPONSE MESSAGE

The `SubmitStructureResponse` message must be returned in any case (success, partial success, failure). This is defined as part of the `RegistryInterface` messages. This message includes the following information per submitted Artefact: - The `action`, e.g. `Append`, `Replace` or `Delete` (`Information` is also available) - A reference to a specific Maintainable Artefact - A status message with the result of the action, which contains: - The status, e.g. `Success`, `Failure` or `Warning` - One or more message texts to explain the result (we need only one per Artefact), which in turn contains: - A code (the specific HTTP code, per Artefact) - A multilingual text message

An example is shown below:

```
<reg:SubmissionResult>
  <reg:SubmittedStructure action="Append"> <!-- Append|Delete|Replace -->
    <reg:MaintainableObject>
      <Ref agencyID="SDMX" id="CL_FREQ" version="1.0.0" package="codelist" class="Codelist" />
    </reg:MaintainableObject>
  </reg:SubmittedStructure>
  <reg:StatusMessage status="Success"> <!-- Success|Failure|Warning -->
    <reg:MessageText code="204"> <!-- Could be the HTTP code 200, 201, 204, ... -->
      <com:Text xml:lang="en">Codelist successfully deleted</com:Text>
      <com:Text xml:lang="fr">Codelist supprimé avec succès</com:Text>
    </reg:MessageText>
  </reg:StatusMessage>
</reg:SubmissionResult>
```

Especially when different results occur on the Artefacts (e.g. partial success), the following should occur: - Return a multi-status return code (like `207`); - Return a JSON/XML message with the results details (currently available only in SDMX-ML);

In the case of a multi-status response, the `SubmitStructureResponse` message will include the corresponding code per Artefact, e.g.:

```
<reg:SubmissionResult>
  <reg:SubmittedStructure action="Replace"> <!-- Append|Delete|Replace -->
    <reg:MaintainableObject>
      <Ref agencyID="SDMX" id="CODELIST" version="1.0" package="codelist" class="Codelist"/>
    </reg:MaintainableObject>
  </reg:SubmittedStructure>
  <reg:StatusMessage status="Success"> <!-- Success|Failure|Warning -->
    <reg:MessageText code="200">  <!-- Could be the HTTP code 200, 204, ... -->
      <com:Text xml:lang="en">Successfully SDMX:CODELIST(1.0) updated Codelist</com:Text>
      <com:Text xml:lang="de">Codelist SDMX:CODELIST(1.0) erfolgreich aktualisiert</com:Text>
    </reg:MessageText>
  </reg:StatusMessage>
</reg:SubmissionResult>
```


Copyright © SDMx TWG 2020

```
<reg:SubmissionResult>
  <reg:SubmittedStructure action="Replace">
    <reg:MaintainableObject>
      <Ref agencyID="SDMX" id="CL_FREQ" version="1.1.1" package="codelist" class="Codelist" />
    </reg:MaintainableObject>
  </reg:SubmittedStructure>
  <reg:StatusMessage status="Failure">
    <reg:MessageText code="409">
      <com:Text xml:lang="en">Codelist SDMX:CODELIST(1.1.1) failed to update, due to semantic versioning violation</com:Text>
    </reg:MessageText>
  </reg:StatusMessage>
</reg:SubmissionResult>
```

ANNEX III - NESTED ITEMS

This section aims at explaining the particularities of nested Items for a subset of the available Item Schemes, namely: - CategoryScheme (Category) - ReportingTaxonomy (ReportingCategory)

In all the above cases, Items may contain other Items in a tree-like hierarchy. As a result, the resource for an Item within such an hierarchy need to include the full path of that Item in order to exactly identify it. For example, for the following Category Scheme (excerpt of SDMX:STAT_SUBJECT_MATTER(1.0) from the Global SDMX Registry):

```
<str:CategoryScheme agencyID="SDMX" id="STAT_SUBJECT_MATTER" version="1.0">
  <com:Name xml:lang="en">SDMX Statistical Subject-Matter Domains</com:Name>
  <str:Category id="DEMO_SOCIAL_STAT">
    <com:Name xml:lang="en">Demographic and social statistics</com:Name>
  </str:Category>
  <str:Category id="ECO_STAT">
    <com:Name xml:lang="en">Economic statistics</com:Name>
    <str:Category id="MACROECO_STAT">
      <com:Name xml:lang="en">Macroeconomic statistics</com:Name>
    </str:Category>
    <str:Category id="SECTORAL_STAT">
      <com:Name xml:lang="en">Sectoral statistics</com:Name>
      <str:Category id="AGRI_FOREST_FISH">
        <com:Name xml:lang="en">Agriculture, forestry, fisheries</com:Name>
      </str:Category>
      <str:Category id="ENERGY">
        <com:Name xml:lang="en">Energy</com:Name>
      </str:Category>
    </str:Category>
    <str:Category id="GOV_FINANCE_PUBLIC_SECTOR">
      <com:Name xml:lang="en">Government finance, fiscal and public sector statistics</com:Name>
    </str:Category>
  </str:Category>
  <str:Category id="ENVIRONMENT_MULTIDOMAIN_STAT">
    <com:Name xml:lang="en">Environment and multi-domain statistics</com:Name>
  </str:Category>
</str:CategoryScheme>
```

In order to get Item/Category `ENERGY` we need to request the following resource: `categoryscheme/SDMX/CAT/1.0/ECO_STAT.SECTORAL_STAT.ENERGY` Instead of the identifier of the Item, the full path of identifiers that lead to that Item are required, i.e.: `ECO_STAT` -> `SECTORAL_STAT` -> `ENERGY`.

Similarly, when trying to delete that Item (Category) the same resource must be used. The behaviour of the server, then, should be that the Category and all its children (if any) are deleted. In the above example, deleting the Category `ECO_STAT.SECTORAL_STAT` would result into deleting also `ECO_STAT.SECTORAL_STAT.ENERGY` and `ECO_STAT.SECTORAL_STAT.AGRI_FOREST_FISH`.

When submitting a nested Item Scheme for partial update (using `PUT` or `POST` with `isPartial="true"`) then all root level Categories appearing in the body are going to fully replace existing root level Categories. This means that all Categories comprising any root Category are going to be replaced by the new Categories submitted. Let's assume submitting this partial Category Scheme:

```
<str:CategoryScheme agencyID="SDMX" id="STAT_SUBJECT_MATTER" version="1.0.0" isPartial="true">
  <com:Name xml:lang="en">SDMX Statistical Subject-Matter Domains</com:Name>
  <str:Category id="ECO_STAT">
    <com:Name xml:lang="en">Economic statistics</com:Name>
    <str:Category id="MACROECO_STAT">
      <com:Name xml:lang="en">Macroeconomic statistics</com:Name>
    </str:Category>
  </str:Category>
</str:CategoryScheme>
```

When applied to the initial Category Scheme, this update would mean that Category `ECO_STAT` only has one child Category, i.e. the resulting Category Scheme would be:

```
<str:CategoryScheme agencyID="SDMX" id="STAT_SUBJECT_MATTER" version="1.0.0">
  <com:Name xml:lang="en">SDMX Statistical Subject-Matter Domains</com:Name>
  <str:Category id="DEMO_SOCIAL_STAT">
    <com:Name xml:lang="en">Demographic and social statistics</com:Name>
```


Copyright © SDMx TWG 2020

```
  </str:Category>
  <str:Category id="ECO_STAT">
    <com:Name xml:lang="en">Economic statistics</com:Name>
    <str:Category id="MACROECO_STAT">
      <com:Name xml:lang="en">Macroeconomic statistics</com:Name>
    </str:Category>
  </str:Category>
  <str:Category id="ENVIRONMENT_MULTIDOMAIN_STAT">
    <com:Name xml:lang="en">Environment and multi-domain statistics</com:Name>
  </str:Category>
</str:CategoryScheme>
```

Moreover, in order to add a new Category under `SECTORAL_STAT` in the initial Category Scheme, then the whole new tree of `ECO_STAT` would need to be submitted, i.e.:

```
<str:CategoryScheme agencyID="SDMX" id="STAT_SUBJECT_MATTER" version="1.0.0">
  <com:Name xml:lang="en">SDMX Statistical Subject-Matter Domains</com:Name>
  <str:Category id="ECO_STAT">
    <com:Name xml:lang="en">Economic statistics</com:Name>
    <str:Category id="MACROECO_STAT">
      <com:Name xml:lang="en">Macroeconomic statistics</com:Name>
    </str:Category>
    <str:Category id="SECTORAL_STAT">
      <com:Name xml:lang="en">Sectoral statistics</com:Name>
      <str:Category id="AGRI_FOREST_FISH">
        <com:Name xml:lang="en">Agriculture, forestry, fisheries</com:Name>
      </str:Category>
      <str:Category id="NEW_SECTORAL_CATEGORY">
        <com:Name xml:lang="en">New</com:Name>
      </str:Category>
      <str:Category id="ENERGY">
        <com:Name xml:lang="en">Energy</com:Name>
      </str:Category>
    </str:Category>
    <str:Category id="GOV_FINANCE_PUBLIC_SECTOR">
      <com:Name xml:lang="en">Government finance, fiscal and public sector statistics</com:Name>
    </str:Category>
  </str:Category>
</str:CategoryScheme>
```


Copyright © SDMx TWG 2020

ANNEX IV - SUMMARY OF HTTP RESPONSE CODES

Method	Exists	Versioning respected	Is Referenced	References exist/ provided	Response Code
POST	F	T	-	T	`201` (successful) or `207` (partially successful)
POST	F	T	-	F	`409` missing references
POST	F	F	-	I	`409` failure to respect the versioning rules
PUT/POST	T	T	F	T	`200` (successful) or `207` (partially successful)
PUT/POST	T	T	F	F	`409`
PUT/POST	T	T	T	T	`200` if update is possible - `409` if references would break
PUT/POST	T	F	-	-	`409` in case of changes that violate the versioning requirements
PUT/POST	-	-	-	-	`422` see section **Server**
PUT	F	-	-	-	`404`
DELETE	F	-	-	-	`404`
DELETE	T	F	F	I	`200`
DELETE	T	T	I	I	`409` when trying to delete a stable artefact, a deprecation strategy may be followed
DELETE	T	F	T	I	`409`

T: True, F: False, I: Irrelevant, -: Not applicable

Copyright © SDMx TWG 2020

ANNEX V - EXAMPLES

Difference between replace and partially update

To explain the updating semantics, the difference between replacing and partially updating the same SDMX Artefact shall be illustrated in the following example. For the sake of simplicity, a Codelist will be utilised, i.e. let's assume:

```
<str:Codelist agencyID="SDMX" id="CL_DECIMALS" version="1.0">
   <com:Name>Code list for Decimals (DECIMALS)</com:Name>
   <com:Description xml:lang="en">It provides a list of values showing the
     number of decimal digits used in the data.</com:Description>
   <str:Code id="0">
      <com:Name>Zero</com:Name>
   </str:Code>
   <str:Code id="1">
      <com:Name>One</com:Name>
   </str:Code>
   <str:Code id="2">
      <com:Name>Two</com:Name>
   </str:Code>
</str:Codelist>
```

Let's assume that we `PUT` the following Codelist, under `/codelist/SDMX/CL_DECIMALS/1.0`, or `POST` it under `/codelist/`:

```
<str:Codelist agencyID="SDMX" id="CL_DECIMALS" version="1.0">
   <com:Name>Code list for Decimals (DECIMALS)</com:Name>
   <com:Description xml:lang="en">It provides a list of values showing the
     number of decimal digits used in the data.</com:Description>
   <str:Code id="0">
      <com:Name>No decimal</com:Name>
   </str:Code>
   <str:Code id="1">
      <com:Name>One</com:Name>
   </str:Code>
 </str:Codelist>
```

This will result into replacing the original Codelist with the one submitted. Hence, in the new Codelist only two Codes will exist; the first one (Code `0`) with an updated name.

If, instead, we added the flag `isPartial="true"` in the above Codelist, i.e.:

```
<str:Codelist agencyID="SDMX" id="CL_DECIMALS" version="1.0" isPartial="true">
   <com:Name>Code list for Decimals (DECIMALS)</com:Name>
   <com:Description xml:lang="en">It provides a list of values showing the
     number of decimal digits used in the data.</com:Description>
   <str:Code id="0">
      <com:Name>No decimal</com:Name>
   </str:Code>
   <str:Code id="1">
      <com:Name>One</com:Name>
   </str:Code>
 </str:Codelist>
```

Then the result would be a bit different. The new Codelist will still have three Codes, but the first one (Code `0`) would have an updated name, i.e.:

```
<str:Codelist agencyID="SDMX" id="CL_DECIMALS" version="1.0">
   <com:Name>Code list for Decimals (DECIMALS)</com:Name>
   <com:Description xml:lang="en">It provides a list of values showing the
     number of decimal digits used in the data.</com:Description>
   <str:Code id="0">
      <com:Name>No decimal</com:Name>
   </str:Code>
   <str:Code id="1">
      <com:Name>One</com:Name>
   </str:Code>
   <str:Code id="2">
      <com:Name>Two</com:Name>
   </str:Code>
</str:Codelist>
```


Copyright © SDMx TWG 2020

6.1.11 Useful tips

Security considerations

This section describes useful security measures for SDMX REST services.

AUTHENTICATION

Authentication refers to the process of uniquely identifying an entity. In the context of a web service, we need to distinguish between service authentication and client authentication.

Clients of web services have a high interest in ensuring that they are connected to the service they intend to consume and not to a rogue service masquerading as a trusted entity. To support this, use SSL/TLS and offer clients the possibility to consume the web service over HTTPS. This allows the client to validate that the certificate matches the domain name of the service, is issued by a trusted authority, and has not expired.

When restrictions apply to the data and metadata published, it is important for the service provider to be able to uniquely identify the client. To support this requirement, use HTTP basic authentication over SSL/TLS. If stronger authentication is required, use SSL client certificates instead.

CONFIDENTIALITY

Confidentiality refers to the process of guaranteeing that resources cannot be accessed by unauthorised users. This requirement is a key requirement for the SDMX web services when restrictions apply to the data and metadata published, as both clients and services have a high interest in ensuring their data is not illegally accessed.

For these web services, use SSL/TLS to support confidentiality during the transfer between the service and the client. SSL/TLS supports this requirement using a combination of symmetric and asymmetric encryption.

INTEGRITY

Integrity refers to the process of guaranteeing that resources cannot be accidentally or maliciously modified. Support this requirement using SSL/TLS. SSL/TLS supports this requirement by calculating a message digest.

Tips for data consumers

CONNECT TO THE SERVICE VIA HTTPS

So, you want to keep a local database up to date with data from your favourite data providers? Or maybe, you just want to build a few visualisations with data coming from various web services? Great, that's what SDMX web services are all about! But are you sure it's a good idea to do so without first checking the identity of the data providers? If not, then make sure you connect to the web service over https.

KNOW YOUR MIME TYPE

If you don't specify the format you want (using the `HTTP Accept header`), the behaviour is unpredictable as the service decides the format to be returned and this decision may change over time. The same applies if you specify a generic mime type, such as `application/xml`. That change could break your client. So, don't rely on the default, and be sure to specify the desired format using the HTTP Accept header with a fully qualified mime-type (e.g.: `application/vnd.sdmx.genericdata+xml;version=2.1`).

GET ONLY WHAT YOU NEED

Let's say you want to get some exchange rates from a web service. And let's say you check regularly to see whether the rates have been updated. Do you want to get everything all the time, included all the data that have not changed, or do you want something more efficient? If it's the latter, make sure to use the `updatedAfter` parameter. In the same vein, if you just want the data and don't need the associated metadata, say so using the `attributes` parameter… For additional information, see the section about the data queries.

OPT FOR FAT-FREE MESSAGES

Plain text formats (XML, JSON, CSV, etc.) compress very well. Compressed data are typically several times smaller than uncompressed ones. In our tests, the compressed files were between 3 and … 47 times smaller than the uncompressed ones, depending on the type of query and the selected formats. So, in order to optimise network transfer times, request **compressed**

Copyright © SDMx TWG 2020

representations of the data, using the appropriate HTTP `Accept-Encoding header`. See the section about HTTP Content Negotiation for additional information.

CHECK THE RESPONSE STATUS CODE

The response status code is used to report potential issues. So, check it. It's not always 200 (or 42). See the section about the status codes for additional information.

Tips for data providers

OFFER ACCESS OVER HTTPS

Clients of web services have a high interest in ensuring that they are connected to the service they intend to consume and not to a rogue service masquerading as a trusted entity. To support this, please offer the possibility to **consume your service over HTTPS**, and make sure your **SSL/TLS certificate** is/remains valid (i.e.: the domain/subdomains should match, the certificate should not be expired, etc.).

CHECK THE MIME-TYPE AND SUPPORT VARIOUS FORMATS

By default, SDMX 2.1 REST services are expected to return the most recent version of the SDMX-ML Generic Data format they support, while SDMX 3.0 REST services are expected to return the most recent version of SDMX-JSON. However, clients can use the HTTP Accept header to specify the format they want, so check it and make sure you don't return XML if they asked for JSON ;-). Also, if you don't offer the requested format, simply return an HTTP 406 status code. This being said, if you offer them a **choice between various formats**, you will most likely improve the usability and accessibility of your service, as different clients have different needs. See the section about HTTP Content Negotiation for a list of SDMX formats.

SUPPORT DATA COMPRESSION

Plain text formats (XML, JSON, CSV, etc.) compress very well. Compressed data are typically several times smaller than uncompressed ones. In our tests, the compressed files were between 3 and … 47 times smaller than the uncompressed ones, depending on the type of query and the selected formats. So, in order to optimise network transfer times, offer the possibility to **get compressed representations** of the data, using the appropriate HTTP `Accept-Encoding header`. See the section about HTTP Content Negotiation for additional information.

SUPPORT CROSS-ORIGIN QUERIES

Clients written in Python, Java, .Net, etc. typically won't have any problems getting the data. But what about the HTML5/JavaScript clients running within a browser? Because of the Same-origin policy, these won't be able to access your web service without additional work, unless you enable the sharing of resources across domains. One way of doing that, is to configure your service to **support CORS headers**. Just make sure the CORS headers are also added in case of errors, else the JavaScript clients will be left clueless about why an error happened.

SUPPORT CACHING HEADERS

The SDMX API allows a client to retrieve all data matching a query. In addition, using the `updatedAfter` parameter, it also allows a client to only retrieve what has changed since the last time the query was issued. This is a very powerful feature but it requires smart clients, able to handle partial updates. For the others, there is a 3rd option: retrieve **all data matching the query, but only if some of the matching data have changed**. To do so, simply add support for the caching mechanism offered by HTTP. In that scenario, a client sends an `If-Modified-Since` header (or an `If-None-Match`), along with a timestamp (or an ETag), and the web service returns the data, if something has changed, or an `HTTP 304 Not modified` status code otherwise.

If you add support for caching headers, don't forget to indicate, via the `Vary` header, which HTTP headers should be taken into account by the cache to build the cache key. Typical examples would be the HTTP `Accept` and `Accept-Encoding` header, as you don't want the cache to return SDMX-ML when the client has requested SDMX-JSON…

Copyright © SDMx TWG 2020

VERSION YOUR API

The SDMX-REST API follows semantic versioning. This means that the type and impact of changes is expressed using an API number made of 3 digits (e.g. v1.2.3).

- The first digit (1 in the above example) indicates that changes (either new features or bug fixes) are not backward compatible.
- The second digit (2 in the above example) indicates that features have been added in a backward compatible manner.
- The third digit (3 in the above example) indicates that bugs have been fixed in a backward compatible manner.

For clients of an SDMX-REST API, this means that only an increment of the first digit might be problematic, while any change to the second or third digit should be transparent to them (i.e. clients might not be able to leverage new features but at least they should not break).

It is considered best practice to version an API, to avoid breaking clients in case of backward incompatible changes. As only changes to the first digit represent changes that are not backward compatible, it is recommended to **version the API based on the value of the first digit only**, e.g. `https://ws-entry-point/v1/`.

BE LIBERAL IN WHAT YOU ACCEPT

It may happen that a client sends a request containing both valid and invalid information. For example, it could be that a data query contains codes that exist in the codelist (which is valid), as well as codes that don’t exist in the codelist (which is invalid). In such cases, it is recommended to be lenient and return the data matching the query, instead of throwing an error because of the invalid codes.

The same applies when all codes supplied are invalid. Instead of throwing an HTTP client error code, it is recommended to return a 204 (No content), to emphasize that an absence of results is an empty but still healthy state.

Copyright © SDMx TWG 2020

6.1.12 Extending the SDMX REST API

One powerful aspect of REST APIs is that they can be easily extended to accommodate more resources or parameters. Extending a RESTful API does not render it incompatible to its predecessor, as long as you respect the existing resources and parameters and implement them according to the specification.

The purpose of this page is to show how the SDMX REST API can be extended, for example by adding the possibility to select the representation using query parameters.

Extending query parameters

The easiest extension one can perform on a REST API is adding custom query parameters. Using path parameters is possible as well but more complex as the order of path parameters is significant. Remember though that the parameters must not clash with the parameters of the original API specification, i.e. the SDMX REST API in our case. Of course, apart from adding new query parameters, their values may also be extended instead.

As a general rule, any extension to the SDMX REST API has a lower order of precedence compared to the standard functionality. For the specific example of the representation as query parameters, described in this section, the HTTP `Accept` header is higher in the order of precedence. This means that for any vendor specific mime-type in the HTTP `Accept` header, the representation query parameters should be ignored.

Apart from the behaviour and order of precedence in regard to the standard REST API specifications, the default behaviour of the extension should be documented as well.

Use case: Ease the selection of the desired representation

REST APIS AND WEB BROWSERS

REST APIs have become popular, not only for their simplicity and power, but also for being intuitive and easy to use through a web browser. Nevertheless, there is one important aspect of the SDMX REST API that is not available to end users via browsers, like the format and version of the returned resource.

Currently, the SDMX REST API uses the HTTP content negotiation mechanism to allow clients to select the resource representation to be returned.

The problem is that end users, on their browser, cannot set the HTTP `Accept` header, unless they use a dedicated extension (aka plugin) such as Postman; hence, they are always presented with the default representation that the specific web service implementation has. For this use case, some providers of SDMX REST web services have extended their API to make it easier for end users to select the required representation.

QUERY PARAMETERS FOR REPRESENTATION

As an example of extending the SDMX REST API, a proposal for new query parameters is presented here. Since the normative way of the SDMX REST specifications for requesting representation is the HTTP content negotiation mechanism and, in principle, the API should not specify the same functionality in alternative ways, this proposal should be treated as an example of how to extend the API and not as a normative part of the standard.

The *mime-types* for the `Accept` header, as specified by the SDMX REST API, may be found here. They follow the following pattern: `application/vnd.sdmx.<representation>+<format>;version=<version>`, where:

- `<representation>` is any of the available SDMX message types, e.g. structurespecific, compact, data, schema, etc.
- `<format>` is the technical format, e.g. xml, json, and
- `<version>` is the version of the representation.

In the example presented here, the `<format>` will be omitted, since two parameters can sufficiently describe the required representation. Hence, two new query parameters may be specified to allow for a similar functionality, i.e. `formatVersion` and `format`. The latter is proposed as a more generic term compared to representation, in order to include the mime-type `<format>` part.

Copyright © SDMx TWG 2020

The `format` query parameter

The `format`, as explained above, represents a combination of the representation and technical format of the messages. Hence, it includes the following possible values:

- `genericdata` for the SDMX-ML Generic Dataset
- `structurespecificdata` for the SDMX-ML Structure Specific Dataset
- `generictimeseriesdata` for the SDMX-ML Generic Time Series Dataset
- `structurespecifictimeseriesdata` for the SDMX-ML Structure Specific Time Series Dataset
- `genericmetadata` for the SDMX-ML Generic Reference Metadataset
- `structurespecificmetadata` for the SDMX-ML Structure Specific Reference Metadataset
- `structure` for all SDMX-ML Structural Metadata
- `schema` for all SDMX XML Generated Schema
- `jsondata` for the SDMX-JSON Dataset
- `csvdata` for the SDMX-CSV Dataset

The `formatVersion` query parameter

The `formatVersion` represents the version of the message format, for any format specified by the `format` query parameter.

The versions per the available formats, may be found in the section HTTP content negotiation.

DEFAULT BEHAVIOUR

The default behaviour of this extension is as follows:

- Omitting both attributes: falls back to the standard's default behaviour.
- Omitting `formatVersion`: the Web Service may define the default version; following the SDMX general behaviour for versions, the latest available should be used.
- Omitting `format`: the attribute `formatVersion` should be also ignored, if specified.

EXAMPLES

To retrieve an SDMX-ML 2.1 Structure Specific dataset, using the representation query parameters:

```
https://ws-entry-point/data/dataflow/ECB/EXR/+/M.USD.EUR.SP00.A?format=structurespecificdata&formatVersion=2.1
```

To retrieve an SDMX-ML 2.1 Codelist, using the representation query parameters:

```
https://ws-entry-point/structure/codelist/ECB/CL_FREQ?format=structure&formatVersion=2.1
```

To retrieve a json dataset, using the representation query parameters:

```
https://ws-entry-point/data/dataflow/ECB/EXR/+/M.USD.EUR.SP00.A?format=jsondata&formatVersion=1.0.0
```


Copyright © SDMx TWG 2020

7. Transmission Formats

7.1 SDMx-ML (v3.1)

7.1.1 Introduction

Background

HISTORY AND VERSION 3.0 DEVELOPMENTS

The SDMX Technical Standards Version 1.0 established an information model which described aggregated statistical data sets and the structural metadata needed to exchange them in a standard fashion. This drew on the earlier example of the GESMES/TS standard. Based on the SDMX information model, several formats were developed: XML formats for exchange of structural metadata, data sets, and queries for these (SDMX-ML), and EDIFACT formats for the structural metadata and data sets (SDMX-EDI). These standards supported a number of exchange patterns, characterized as “bilateral”, “gateway”, and “data-sharing” models, as described in the Framework document in the Version 1.0 standards package.

Versions 2.0 and 2.1 built on this foundation to provide a higher degree of support for all of these models, with an emphasis on data sharing in the form of a set of standard registry services interfaces. Support for new types of metadata exchange and reporting were added, with a focus on “reference metadata” concerned with quality, methodology, and other issues. Further, the ability to provide metadata about the relationships between data sets and structures was expanded, providing more support for data cubes. Overall, the scope of the Version 2.1 SDMX Technical Standards was much broader, and included a larger set of message types in the SDMX-ML formats.

For Version 3.0, the SDMX-ML specification has been rationalised and tightened while maintaining the fundamentals and principles established in 2.1. The documents for describing structural metadata remain the same except where there have been changes to the information model either to enhance existing structures, add new ones, or remove those deprecated such as the Structure Set.

The deprecation of the SOAP web services specification in Version 3.0 has resulted in the removal of all SDMX-ML Query documents. Standardisation on the REST API and HTTP GET for registry structure queries means separate ‘query request’ documents are no longer required.

The opportunity has also been taken to rationalise the plethora of alternative data documents accumulated from Versions 1.0, 2.0 and 2.1. All, including Generic, Utility, Compact and Cross-Sectional have been deprecated except for Structure Specific Data which becomes the sole standard XML format for data updates, revisions and bilateral exchanges in Version 3.0.

THE XML DESIGN

All of these document types will share a common “envelope” at the message level (“SDMXMessage.xsd”), as well as a set of common low-level components (“SDMXCommon[subset].xsd”) so that header information and basic structure of a message will always be the same.

- Schema and sub-modules for describing all types of structural metadata – for data sets (data structure definitions), for metadata sets (metadata structure definitions), for related groups of metadata and data structures, and for all types of structural objects involved in registry-based exchanges (“SDMXStructure[*subset*].xsd”)
- Structure specific data schema for for updates, revisions/bilateral exchange (“SDMXDataStructureSpecific.xsd”)
- Generic schema and sub-modules for registry interfaces (“SDMXRegistry [*subset*].xsd”)
- Generic schema for reference metadata sets (“SDMXMetadataGeneric.xsd”)

FOSTERING THE USE OF A STANDARD SDMX-ML

In addition to these different formats, standard mappings and corresponding transformation tools have been developed for the creation of data structure-specific schemas from structure descriptions, to transform XML data instances from one XML data description format to another, and from these formats into the corresponding SDMX-ML messages. This level of free tools support will foster the early use of SDMX and permit the data to be easily used across all processes, which is otherwise a difficult requirement to meet. Ultimately, it is the fact that all formats share a common information model that enables this approach to meet the wide set of SDMX requirements.

Copyright © SDMx TWG 2020

NORMATIVE REFERENCES

- W3C XML Schema Definition Language, version 1.0, World Wide Web Consortium
- W3C Extensible Markup Language, version 1.0, Third Edition, World Wide Web Consortium

CONFORMANCE

Data of this document is normative, providing rules for the creation of conformant SDMX-ML XML instances and W3C XML Schemas.

Copyright © SDMx TWG 2020

DESIGN OVERVIEW

SCOPE AND REQUIREMENTS

To understand the relationships between the several document types, it is important to have some familiarity with the requirements they are designed to fulfil.

- Large amounts of data must be captured in a reasonably compact format, because of the potential size of databases being exchanged.
- It must be possible to send incremental updates, rather than entire, complete databases. The validation of such exchanges demands not that an entire data set be exchanged, but only that enough information be sent to ensure accurate updating and revision processes.
- Structural information as well as data will need to be transmitted.
- There must be a reliable transformation to and from the GESMES/TS EDIFACT syntax.
- It should be possible to present natural-language information in multiple, equivalent languages.
- To support web services and similar technological approaches, there is a requirement to send queries to information sources as well as data and structures.
- Users (and registry services) may not know about a specific data structure, and will need to be able to handle data across data structures, and even (for, say, a comparison service) to put data structured according to multiple data structures in a single XML instance.
- The XML should conform to the Information Model as closely as possible, and promote simple mapping from the XML instances to model based objects
- The XML should behave as “normally” as possible within standard XML tools such as web development environments, parsers, guided editing tools, etc.
- Data should be able to be structured in one or two levels, with the two level format allowing for a single dimension which serves to disambitguate the observations. In addtion, it should still be possible to restrict this more generalized structure to time series only formats. However, doing so should not add any additional burden in terms of how data is processed.
- Data structure and metadata structure specific formats should be able to be generically processed, without having to understand the entire structure; which is to say that the structure-specific schemas should serve to validate the information, but not be required to process it.
- XML formats should promote re-use of common semantics, concepts, and codelists to the greatest possible extent, while still recognizing the agency which maintains a specific resource (a codelist, a data structure, a data set, etc.)
- XML formats must support interactions of applications with standard registry services, based on standard interfaces. These must function both as web services, and as services operating over http and similar protocols.
- XML formats must support the reporting of reference metadata which is not structural in nature, but which constitutes a primary information flow of metadata attached to other parts of the statistical collection, reporting, processing, exchange, and dissemination. Quality initiatives, methodological metadata, administrative metadata, and similar types of metadata reporting must be supported, and must be user-configurable.
- XML formats for describing the relationships between groups of metadata sets and data sets, by mapping concepts and codelists between these structures, and by allowing for common querying of data and metadata described with not only a single structural definition, but with a related set of structural definitions, based on these mappings.
- Allow for time-related concepts which are not related to the time of the observation to be used in data structures.
- Allow for simple, un-coded incremental identifiers in data structure definitions, to be used to dis-ambiguate data series/observations which do not have a simple 1-to-1 relationship with the time period of the observation.
- Allow for un-coded identifiers and descriptors to be associated with data structure definitions which establish an external entity or identifier to disambiguate between otherwise identical series/observations (ie, when a data set describes a group of organisations, or a set of accounts, which might otherwise have identical key values).
- Allow for non-numeric observation values (usually but not always coded)
- Allow “cube”-based systems (such as OLAP) to interoperate with less sophisticated systems, without necessarily losing the richness of metadata found in the more sophisticated systems.

Copyright © SDMx TWG 2020

This is a very broad set of requirements, and in examining these it becomes evident that some of the requirements are very much at cross-purposes. It is almost impossible to design a single XML document type for any single function (exchange of data, exchange of reference metadata, querying, etc.) which will satisfy all of these requirements. At the same time, it was very much felt that whatever design was adopted should have a clear relationship with the information model.

DESIGN APPROACH

The basic design for the XML Schema started with the information model. Just as the model does, the structural metadata objects are built up from the abstract classes on which they are based. This approach means that more advanced XML tools that understand such inheritance can use these constructs to more easily deserialize the XML instances into object based on the models.

Another fundamental approach was to make constructs as consistent as possible so that tools built to process the XML could make use of reuse. This approach extends not only to the basic information model properties such as identifications and version, but also to areas such as referencing and query messages.

Through the years of the standard, it was identified that users typically implemented on the previously named CompactData message and/or the GenericData message. It was determined that the UtilityData message was rarely used,and the CrossSectionalData message was used, but often in inconsistent manners due to the ambiguity in its definition. Therefore, the approach for this version was to harmonize these data formats as much as possible. The result is two basic data formats; a generic format and a data structure definition specific format. As much as possible these formats have been structured so that they are very similar in structure. In addition to the harmonisation of the generic and structure specific data formats, the requirements of the CompactData and CrossSectionalData messages have been combined into one format. This format is now flexible in that it allows for any single dimension to exist at the observation. In order to accommodate existing applications that did only use the CompactData message due to its time series orientation, time series only variations of the structure specific and generic data messages have been created. Care was taken to ensure that these time series only variations could be processed just as the general format counterparts are so as to introduce no additional requirements.

These same principles were applied to the reference metadata messages as well. In addition, the base structure specific schemas from which data and metadata structure specific variations are created, have been structured so that they enforce a specific structure and can be easily processed in a generic manner.

Another new approach that has been introduced in this version is to remove some of the generalities from the messages that existed to allow for web services to be more specific as to the contract of their functions. As opposed to have a single query message that can accommodate any structural metadata query and data and reference metadata queries serving as the input into a codelist query function–a specific message is created that only allows for what a codelist query should reasonably be expected to handle.

SDMX-ML PACKAGING: NAMESPACE MODULES

In the proposed XML Schema design, there is a packaging scheme based on the idea that XML namespaces can be used as “modules”, so that any given user or application need only be familiar with a subset of the entire library in order to use it. This approach fit very well with the design described above, and is often used in major XML standards for other domains.

The other major benefit of namespaces – especially in light of the requirement that maintenance agencies be tracked across the potential reuse of the structures and data they maintained – is that it allows SDMX to own certain namespace modules, and allows other maintenance agencies to own namespaces specific to the key-families or metadata structure definitions they also maintain.

The result is a set of namespace packages which agree with the design approach described above. Some large schemas are divided into sub-modules, all within the same namespace, for ease of use. Each module or sub-module is a single instance of the W3C XML Schema Language’s schema element, associated with the appropriate XML namespace. Where these modules and sub-modules have dependencies on one another, they use the XML Schema importing mechanism to draw on constructs described in another module or sub-module. Further, the namespaces themselves reflect a sort of relationship. For example, all data constructs

Copyright © SDMx TWG 2020

are in a namespace based on http://www.sdmx.org/resources/sdmxml/schemas/v3_0/data. These relationships in the namespace are meant to reflect the relationship of the constructs.

- An SDMX Namespace Module containing the common message constructs, including the common header information (“SDMXMessage.xsd”) - uses all other SDMX-ML namespace modules
- An SDMX Namespace Module containing the message footer constructs, including the common header information (“SDMXMessageFooter.xsd”) - used only by the message namespace.
- An SDMX Namespace Module containing the descriptions of structural metadata such as key families, concepts, and codelists (“SDMXStructure [*sub-module*].xsd”)
- An SDMX Namespace Module containing constructs shared in common across all of the SDMX message types (“SDMXCommon[*sub-module*].xsd”) – needed for all other SDMX-ML namespace modules (also included for convenience is the XML namespace [“xml.xsd”] provided by the W3C for including the xml:lang attribute in schemas).
- An SDMX Namespace Module for structured (data structure-specific) data (“SDMXDataStructureSpecific.xsd”)
- A set of namespaced modules created and maintained by those who create data structure-specific schemas – not maintained by SDMX
- An SDMX Namespace Module providing a generic format for reporting of reference metadata, regardless of metadata structure definition (“SDMXMetadataGeneric.xsd”).
- An SDMX Namespace Module providing standard interfaces for interactions with a set of registry services (“SDMXRegistry[*sub-module*].xsd”).

The following sections describe in detail the proposed XML formats, which should be examined alongside the documentation provided. These proposed schemas are divided into the generic schemas, for which a complete set of schema definitions can be provided, and data structure-specific schemas, for which a core structure is provided (with schema code), plus a guide to how a specific data structure or metadata structure definition can be mapped onto the core structure.

Related Documents

To all for more manageable documents, the remainder of this document has been divided into seven different parts, each a separate document. These parts are as follows:

1. Message: This contains the description of the message namespaces schemas
2. Common: This details the common namespace schemas
3. Structure: This details the structure namespace (structural metadata) schemas
4. Data: This details the data and metadata namespaces, as well as detailing the rules for creating strucutre specific schemas
5. Registry: This details the registry namespace schemas
6. Samples: This provides a brief explanation of the currently available sample files.

Copyright © SDMx TWG 2020

7.1.2 Message

Introduction

At the core of the SDMX XML messages are the message namespaces. These namespaces define the general structure of all messages and define the specific messages that are available for exchange in SDMX.

There are two namespaces associated with the messages. The main namespace schema which defines every message in SDMX is http://www.sdmx.org/resources/sdmxml/schemas/v3_0/message. Associated with this is another sub-namespace, http://www.sdmx.org/resources/sdmxml/schemas/v3_0/message/footer. This namespace defines footer level information that is available in messages which might require non-standard payload information to be transmitted.

In general, every message in SDMX follows common format of:

- Header
- Payload (0..n)
- Footer (0..1)

The header and payload elements exist in the message namespace, but the content of the payload is defined in the namespaces that are specific the functionality of the messages. Note that the header follows a common construct, which is then restricted according to the requirements of the message which is using it.

Schema Documentation

MESSAGE NAMESPACE

http://www.sdmx.org/resources/sdmxml/schemas/v3_0/message

Summary

Referenced Namespaces:

Namespace	Prefix
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/common	common
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/data/structurespecific	dsd
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/message/footer	footer
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/metadata/generic	metadata
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/registry	registry
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/structure	structure
http://www.w3.org/2001/XMLSchema	xs

Contents:

- 7 Global Elements
- 16 Complex Types
- 1 Simple Type

Global Elements

Structure (StructureType): Structure is a message that contains structural metadata. It may contain any of the following; categorisations, category schemes, code lists, concepts (concept schemes), constraints (attachment and content) data flows, hierarchical code lists, metadata flows, metadata structure definitions, organisation schemes, processes, reporting taxonomies, and structure sets.

Copyright © SDMx TWG 2020

StructureSpecificData (StructureSpecificDataType): StructureSpecificData is used to convey data structure specific according to data structure definition. The payload of this message (i.e. the data sets) will be based on XML schemas which are specific to the data structure definition and the orientation (i.e. the observation dimension) of the data.

GenericMetadata (GenericMetadataType): GenericMetadata contains reported metadata in a format which supports any metadata structure definition.

RegistryInterface (RegistryInterfaceType): RegistryInterface is used to conduct all interactions with the SDMX Registry Services.

SubmitStructureRequest (SubmitStructureRequestType): SubmitStructureRequest is used to submit structure definitions to the repository. The structure resources (key families, agencies, concepts and concept schemes, code lists, etc.) to be submitted may be communicated in-line or be supplied in a referenced SDMX-ML Structure messages external to the registry. A response will indicate status and contain any relevant error information.

SubmitStructureResponse (SubmitStructureResponseType): SubmitStructureResponse is returned by the registry when a structure submission request is received. It indicates the status of the submission, and carries any error messages which are generated, if relevant.

Error (ErrorType): Error is used to communicate that an error has occurred when responding to a request in an non-registry environment. The content will be a collection of error messages.

Complex Types

***MessageType*:** MessageType is an abstract type which is used by all of the messages, to allow inheritance of common features. Every message consists of a mandatory header, followed by optional payload (which may occur multiple times), and finally an optional footer section for conveying error, warning, and informational messages.

Content:

```
Header, {any element with namespace of
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/message}*, Footer?
```

Element Documentation:

Name	Type	Documentation
Header	*BaseHeaderType*	
Footer	FooterType	Footer is used to communicate information such as error and warnings after the payload of a message.

StructureType: StructureType defines the contents of a structure message. The payload is optional since this message may be returned from a web service with only information in the footer.

Derivation:

```
MessageType (restriction)
   StructureType
```

Content:

```
Header, Structures?, Footer?
```

Element Documentation:

Name	Type	Documentation
Header	StructureHeaderType	
Structures	StructuresType	
Footer	FooterType	Footer is used to communicate information such as error and warnings after the payload of a message.

Copyright © SDMx TWG 2020

StructureSpecificDataType: StructureSpecificDataType defines the structure of the structure specific data message. Note that the data set payload type is abstract, and therefore it will have to be assigned a type in an instance. This type must be derived from the base type referenced. This base type defines a general structure which can be followed to allow for generic processing of the data even if the exact details of the data structure specific format are not known.

Derivation:

```
MessageType (restriction)
   StructureSpecificDataType
```

Content:

```
Header, DataSet*, Footer?
```

Element Documentation:

Name	Type	Documentation
Header	StructureSpecificDataHeaderType	
DataSet	*DataSetType*	
Footer	FooterType	Footer is used to communicate information such as error and warnings after the payload of a message.

GenericMetadataType: GenericMetadataType defines the contents of a generic metadata message.

Derivation:

```
MessageType (restriction)
   GenericMetadataType
```

Content:

```
Header, MetadataSet*, Footer?
```

Element Documentation:

Name	Type	Documentation
Header	GenericMetadataHeaderType	
MetadataSet	MetadataSetType	
Footer	FooterType	Footer is used to communicate information such as error and warnings after the payload of a message.

RegistryInterfaceType: This is a type which describes a structure for holding all of the various dedicated registry interface message types.

Derivation:

```
MessageType (restriction)
   RegistryInterfaceType
```

Content:

```
Header, (SubmitRegistrationsRequest | SubmitRegistrationsResponse |
QueryRegistrationRequest | QueryRegistrationResponse |
SubmitStructureRequest | SubmitStructureResponse |
SubmitSubscriptionsRequest | SubmitSubscriptionsResponse |
QuerySubscriptionRequest | QuerySubscriptionResponse |
NotifyRegistryEvent)?, Footer?
```


Copyright © SDMx TWG 2020

Element Documentation:

Copyright © SDMx TWG 2020

Name	Type	Documentation
Header	BasicHeaderType	
SubmitRegistrationsRequest	SubmitRegistrationsRequestType	SubmitRegistrationsRequest is sent to the registry by an agency or data/metadata provider to request one or more registrations for a data set or metadata set. The data source to be registered must be accessible to the registry services at an indicated URL, so that it can be processed by those services.
SubmitRegistrationsResponse	SubmitRegistrationsResponseType	SubmitRegistrationsResponse is sent to the agency or data/metadata provider in response to a submit registrations request. It indicates the success or failure of each registration request, and contains any error messages generated by the registration service.
QueryRegistrationRequest	QueryRegistrationRequestType	QueryRegistrationRequest is used to query the contents of a registry for data sets and metadata sets. It specifies whether the result set should include metadata sets, data sets, or both. The search can be characterized by providing constraints including reference periods, data regions, and data keys.
QueryRegistrationResponse	QueryRegistrationResponseType	QueryRegistrationResponse is sent as a response to any query of the contents of a registry. The result set contains a set of links to data and/or metadata If the result set is null, or there is some other problem with the query, then appropriate error messages and statuses will be returned.
SubmitStructureRequest	SubmitStructureRequestType	SubmitStructureRequest is used to submit structure definitions to the repository. The structure resources (key families, agencies, concepts and concept schemes, code lists, etc.) to be submitted may be communicated in-line or be supplied in a referenced SDMX-ML Structure messages external to the registry. A response will indicate status and contain any relevant error information.
SubmitStructureResponse	SubmitStructureResponseType	SubmitStructureResponse is returned by the registry when a structure submission request is received. It indicates the status of the submission, and carries any error messages which are generated, if relevant.
SubmitSubscriptionsRequest	SubmitSubscriptionsRequestType	SubmitSubscriptionsRequest contains one or more requests submitted to the registry to subscribe to registration and change events for specific registry resources.
SubmitSubscriptionsResponse	SubmitSubscriptionsResponseType	SubmitSubscriptionsResponse is the response to a submit subscriptions request. It contains information which describes the success or failure of each subscription request, providing any error messages in the event of failure. If successful, it returns the registry URN of the subscription, and the subscriber-assigned ID.
QuerySubscriptionRequest	QuerySubscriptionRequestType	QuerySubscriptionRequest is used to query the registry for the subscriptions of a given organisation.
QuerySubscriptionResponse	QuerySubscriptionResponseType	QuerySubscriptionResponse is sent as a response to a subscription query. If the query is successful, the details of all subscriptions for the requested organisation are sent.
NotifyRegistryEvent	NotifyRegistryEventType	NotifyRegistryEvent is sent by the registry services to subscribers, to notify them of specific registration and change events. Basic information about the event, such as the object that triggered it, the time of the event, the action that took place, and the subscription that triggered the notification are always sent. Optionally, the details of the changed object may also be provided.

Copyright © SDMx TWG 2020

Name	Type	Documentation
Footer	FooterType	Footer is used to communicate information such as error and warnings after the payload of a message.

SubmitStructureRequestType: SubmitStructureRequestType defines the structure of a registry submit structure request document.

Derivation:

```
MessageType (restriction)
   RegistryInterfaceType (restriction)
         SubmitStructureRequestType
```

Content:

```
Header, SubmitStructureRequest
```

Element Documentation:

Name	Type	Documentation
Header	BasicHeaderType	
SubmitStructureRequest	SubmitStructureRequestType	SubmitStructureRequest is used to submit structure definitions to the repository. The structure resources (key families, agencies, concepts and concept schemes, code lists, etc.) to be submitted may be communicated in-line or be supplied in a referenced SDMX-ML Structure messages external to the registry. A response will indicate status and contain any relevant error information.

SubmitStructureResponseType: SubmitStructureResponseType defines the structure of a registry submit registration response document.

Derivation:

```
MessageType (restriction)
   RegistryInterfaceType (restriction)
         SubmitStructureResponseType
```

Content:

```
Header, SubmitStructureResponse, Footer?
```

Element Documentation:

Name	Type	Documentation
Header	BasicHeaderType	
SubmitStructureResponse	SubmitStructureResponseType	SubmitStructureResponse is returned by the registry when a structure submission request is received. It indicates the status of the submission, and carries any error messages which are generated, if relevant.
Footer	FooterType	Footer is used to communicate information such as error and warnings after the payload of a message.

ErrorType: ErrorType describes the structure of an error response.

Content:

Copyright © SDMx TWG 2020

```
ErrorMessage+
```

Element Documentation:

Name	Type	Documentation
ErrorMessage	CodedStatusMessageType	ErrorMessage contains the error message. It can occur multiple times to communicate message for multiple errors, or to communicate the error message in parallel languages. If both messages for multiple errors and parallel language messages are used, then each error message should be given a code in order to distinguish message for unique errors.

***BaseHeaderType*:** BaseHeaderType in an abstract base type that defines the basis for all message headers. Specific message formats will refine this

Content:

```
ID, Test, Prepared, Sender, Receiver*, Name*, Structure*,
DataProvider?, MetadataProvider?, DataSetAction?, DataSetID*,
Extracted?, ReportingBegin?, ReportingEnd?, EmbargoDate?, Source*
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
ID	IDType	ID identifies an identification for the message, assigned by the sender.
Test	xs:boolean	Test indicates whether the message is for test purposes or not.
Prepared	HeaderTimeType	Prepared is the date the message was prepared.
Sender	SenderType	Sender is information about the party that is transmitting the message.
Receiver	PartyType	Receiver is information about the party that is the intended recipient of the message.
Name	TextType	Name provides a name for the transmission. Multiple instances allow for parallel language values.
Structure	*PayloadStructureType*	Structure provides a reference to the structure (either explicitly or through a structure usage reference) that describes the format of data or reference metadata. In addition to the structure, it is required to also supply the namespace of the structure specific schema that defines the format of the data/metadata. For cross sectional data, additional information is also required to state which dimension is being used at the observation level. This information will allow the structure specific schema to be generated. For generic format messages, this is used to simply reference the underlying structure. It is not mandatory in these cases and the generic data/metadata sets will require this reference explicitly.
DataProvider	DataProviderReferenceType	DataProvider identifies the provider of the data for a data message.
MetadataProvider	MetadataProviderReferenceType	MetadataProvider identifies the provider of the metadata for a metadata message.
DataSetAction	ActionType	DataSetAction code provides a code for determining whether the enclosed message is an Update or Delete message (not to be used with the UtilityData message).
DataSetID	IDType	DataSetID provides an identifier for a contained data set.
Extracted	xs:dateTime	Extracted is a time-stamp from the system rendering the data.
ReportingBegin	ObservationalTimePeriodType	ReportingBegin provides the start of the time period covered by the message (in the case of data).
ReportingEnd	ObservationalTimePeriodType	ReportingEnd provides the end of the time period covered by the message (in the case of data).
EmbargoDate	xs:dateTime	EmbargoDate holds a time period before which the data included in this message is not available.
Source	TextType	Source provides human-readable information about the source of the data.

StructureHeaderType: StructureHeaderType defines the structure for structural metadata messages.

Derivation:

```
BaseHeaderType (restriction)
   StructureHeaderType
```

Content:

```
ID, Test, Prepared, Sender, Receiver*, Name*, Source*
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
ID	IDType	ID identifies an identification for the message, assigned by the sender.
Test	xs:boolean	Test indicates whether the message is for test purposes or not.
Prepared	HeaderTimeType	Prepared is the date the message was prepared.
Sender	SenderType	Sender is information about the party that is transmitting the message.
Receiver	PartyType	Receiver is information about the party that is the intended recipient of the message.
Name	TextType	Name provides a name for the transmission. Multiple instances allow for parallel language values.
Source	TextType	Source provides human-readable information about the source of the data.

StructureSpecificDataHeaderType: StructureSpecificDataHeaderType defines the header structure for a structure specific data message.

Derivation:

```
BaseHeaderType (restriction)
   StructureSpecificDataHeaderType
```

Content:

```
ID, Test, Prepared, Sender, Receiver*, Name*, Structure+,
DataProvider?, DataSetAction?, DataSetID*, Extracted?, ReportingBegin?,
ReportingEnd?, EmbargoDate?, Source*
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
ID	IDType	ID identifies an identification for the message, assigned by the sender.
Test	xs:boolean	Test indicates whether the message is for test purposes or not.
Prepared	HeaderTimeType	Prepared is the date the message was prepared.
Sender	SenderType	Sender is information about the party that is transmitting the message.
Receiver	PartyType	Receiver is information about the party that is the intended recipient of the message.
Name	TextType	Name provides a name for the transmission. Multiple instances allow for parallel language values.
Structure	StructureSpecificDataStructureType	Structure provides a reference to the structure (either explicitly or through a structure usage reference) that describes the format of data or reference metadata. In addition to the structure, it is required to also supply the namespace of the structure specific schema that defines the format of the data/metadata. For cross sectional data, additional information is also required to state which dimension is being used at the observation level. This information will allow the structure specific schema to be generated. For generic format messages, this is used to simply reference the underlying structure. It is not mandatory in these cases and the generic data/metadata sets will require this reference explicitly.
DataProvider	DataProviderReferenceType	DataProvider identifies the provider of the data for a data message.
DataSetAction	ActionType	DataSetAction code provides a code for determining whether the enclosed message is an Update or Delete message (not to be used with the UtilityData message).
DataSetID	IDType	DataSetID provides an identifier for a contained data set.
Extracted	xs:dateTime	Extracted is a time-stamp from the system rendering the data.
ReportingBegin	ObservationalTimePeriodType	ReportingBegin provides the start of the time period covered by the message (in the case of data).
ReportingEnd	ObservationalTimePeriodType	ReportingEnd provides the end of the time period covered by the message (in the case of data).
EmbargoDate	xs:dateTime	EmbargoDate holds a time period before which the data included in this message is not available.
Source	TextType	Source provides human-readable information about the source of the data.

GenericMetadataHeaderType: GenericMetadataHeaderType defines the header format for generic reference metadata.

Derivation:

```
BaseHeaderType (restriction)
   GenericMetadataHeaderType
```

Content:

```
ID, Test, Prepared, Sender, Receiver*, Name*, Structure+,
MetadataProvider?, DataSetAction?, DataSetID*, Extracted?, Source*
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
ID	IDType	ID identifies an identification for the message, assigned by the sender.
Test	xs:boolean	Test indicates whether the message is for test purposes or not.
Prepared	HeaderTimeType	Prepared is the date the message was prepared.
Sender	SenderType	Sender is information about the party that is transmitting the message.
Receiver	PartyType	Receiver is information about the party that is the intended recipient of the message.
Name	TextType	Name provides a name for the transmission. Multiple instances allow for parallel language values.
Structure	GenericMetadataStructureType	Structure provides a reference to the structure (either explicitly or through a structure usage reference) that describes the format of data or reference metadata. In addition to the structure, it is required to also supply the namespace of the structure specific schema that defines the format of the data/metadata. For cross sectional data, additional information is also required to state which dimension is being used at the observation level. This information will allow the structure specific schema to be generated. For generic format messages, this is used to simply reference the underlying structure. It is not mandatory in these cases and the generic data/metadata sets will require this reference explicitly.
MetadataProvider	MetadataProviderReferenceType	MetadataProvider identifies the provider of the metadata for a metadata message.
DataSetAction	ActionType	DataSetAction code provides a code for determining whether the enclosed message is an Update or Delete message (not to be used with the UtilityData message).
DataSetID	IDType	DataSetID provides an identifier for a contained data set.
Extracted	xs:dateTime	Extracted is a time-stamp from the system rendering the data.
Source	TextType	Source provides human-readable information about the source of the data.

BasicHeaderType: BasicHeaderType defines the most basic header information used in simple message exchanges.

Derivation:

```
BaseHeaderType (restriction)
   BasicHeaderType
```

Content:

```
ID, Test, Prepared, Sender, Receiver
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
ID	IDType	ID identifies an identification for the message, assigned by the sender.
Test	xs:boolean	Test indicates whether the message is for test purposes or not.
Prepared	HeaderTimeType	Prepared is the date the message was prepared.
Sender	SenderType	Sender is information about the party that is transmitting the message.
Receiver	PartyType	Receiver is information about the party that is the intended recipient of the message.

PartyType: PartyType defines the information which is sent about various parties such as senders and receivers of messages.

Attributes:

```
id
```

Content:

```
Name*, Contact*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id attribute holds the identification of the party.

Element Documentation:

Name	Type	Documentation
Name	TextType	Name is a human-readable name of the party.
Contact	ContactType	Contact provides contact information for the party in regard to the transmission of the message.

SenderType: SenderType extends the basic party structure to add an optional time zone declaration.

Derivation:

```
PartyType (extension)
   SenderType
```

Attributes:

```
id
```

Content:

```
Name*, Contact*, Timezone?
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id attribute holds the identification of the party.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Name	TextType	Name is a human-readable name of the party.
Contact	ContactType	Contact provides contact information for the party in regard to the transmission of the message.
Timezone	TimezoneType	Timezone specifies the time zone of the sender, and if specified can be applied to all un-time zoned time values in the message. In the absence of this, any dates without time zone are implied to be in an indeterminate “local time”.

ContactType: ContactType provides defines the contact information about a party.

Content:

```
Name*, Department*, Role*, (Telephone | Fax | X400 | URI |
Email)*
```

Element Documentation:

Name	Type	Documentation
Name	TextType	Name contains a human-readable name for the contact.
Department	TextType	Department is designation of the organisational structure by a linguistic expression, within which the contact person works.
Role	TextType	Role is the responsibility of the contact person with respect to the object for which this person is the contact.
Telephone	xs:string	Telephone holds the telephone number for the contact person.
Fax	xs:string	Fax holds the fax number for the contact person.
X400	xs:string	X400 holds the X.400 address for the contact person.
URI	xs:anyURI	URI holds an information URL for the contact person.
Email	xs:string	Email holds the email address for the contact person.

Simple Types

HeaderTimeType: Provides a union type of xs:date and xs:dateTime for the header fields in the message.

Union of:

`xs:dateTime, xs:date.`

MESSAGE FOOTER NAMESPACE

http://www.sdmx.org/resources/sdmxml/schemas/v3_0/message/footer

Summary

Referenced Namespaces:

Namespace	Prefix
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/common	common
http://www.w3.org/2001/XMLSchema	xs

Copyright © SDMx TWG 2020

Contents:

- 1 Global Element
- 2 Complex Types
- 1 Simple Type

Global Elements

Footer (FooterType): Footer is used to communicate information such as error and warnings after the payload of a message.

Complex Types

FooterType: FooterType describes the structure of a message footer. The footer is used to convey any error, information, or warning messages. This is to be used when the message has payload, but also needs to communicate additional information. If an error occurs and no payload is generated, an Error message should be returned.

Content:

```
Message+
```

Element Documentation:

Name	Type	Documentation
Message	FooterMessageType	Message contains a single error, information, or warning message. A code is provided along with an optional severity. The text of the message can be expressed in multiple languages.

FooterMessageType: FooterMessageType defines the structure of a message that is contained in the footer of a message. It is a status message that have a severity code of Error, Information, or Warning added to it.

Derivation:

```
StatusMessageType (restriction)
   CodedStatusMessageType (extension)
         FooterMessageType
```

Attributes:

```
code, severity?
```

Content:

```
Text+
```

Attribute Documentation:

Name	Type	Documentation
code	xs:string	The code attribute holds an optional code identifying the underlying error that generated the message. This should be used if parallel language descriptions of the error are supplied, to distinguish which of the multiple error messages are for the same underlying error.
severity	SeverityCodeType	

Element Documentation:

Name	Type	Documentation
Text	TextType	Text contains the text of the message, in parallel language values.

Copyright © SDMx TWG 2020

Simple Types

SeverityCodeType:

Derived by restriction of `xs:string` .

Enumerations:

Value	Documentation
Error	Error indicates the status message coresponds to an error.
Warning	Warning indicates that the status message corresponds to a warning.
Information	Information indicates that the status message corresponds to an informational message.

Copyright © SDMx TWG 2020

7.1.3 Common Namespace

Introduction

The common namespace defines a collection of constructs that are reused across the various components of SDMX. Most important of these are the referencing mechanism. The goal of the reference construct was to define a generic structure that could be processed uniformly regardless of the context where the reference was used. But it was also important that references be required to be complete whenever possible.

Any object can be referenced either explicitly with a URN or by a set of complete reference fields. To meet the previously stated requirements, and very general mechanism was created based on the URN structure of SDMX objects for these reference fields.

There was also a requirement that the references be able to be refined to meet particular needs outside of the common namespace. An example of this is in the metadata structure specific schemas. It is a requirement that if an target object is specified as having to come from a particular scheme, that a type based on the reference structure be created that enforced the requirement.

Typically, this would not have been an issues as all of the components which make up the references are of atomic types, and therefore can be expressed as XML attributes which are easily refined and restricted since the XML Schema design principles in SDMX always treats attributes as unqualified.

However, the requirement to allow both a URN and/or a complete set of reference field necessitate that these properties be contained in elements. The fact that they are elements typically would mean that the only way a refinement outside of the namespace could happen was if the element were global and allowed for substitutions. This however would mean that every distinct type of referenced object would have a unique set of elements. This moved away from the requirement that the structure be easy to process regardless of context.

The solution to this problem was to deviate from the normal SDMX XML Schema design principle of always using qualified elements and allowing for these to be unqualified. Doing so allows other namespace to derive from these types and place further restrictions on what can be referenced. The deviation from this principle was justified in that it met the all of the requirements and was not deemed to major of a shift since these properties normally would have been expressed as unqualified attributes if it weren’t for the complete reference requirement.

Schema Documentation

COMMON NAMESPACE

http://www.sdmx.org/resources/sdmxml/schemas/v3_0/common

Summary

Referenced Namespaces:

Namespace	Prefix
http://www.w3.org/1999/xhtml	xhtml
http://www.w3.org/2001/XMLSchema	xs

Contents:

- 6 Global Elements
- 30 Complex Types
- 206 Simple Types

Global Elements

Name (TextType): Name is a reusable element, used for providing a human-readable name for an object.

Description (TextType): Description is a reusable element, used for providing a longer human-readable description of an object.

Copyright © SDMx TWG 2020

Text (TextType): Text is a reusable element, used for providing a language specific text value for general purposes (i.e. not for a name or description).

StructuredText (XHTMLType): StructuredText is a reusable element, used for providing a language specific text value structured as XHTML.

Annotations (AnnotationsType): Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotableType may reference it.

Link (LinkType): Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the metadata report can be dynamically inserted in the structure metadata.

Complex Types

ValueType: ValueType is an abstract class that is the basis for any component value that cannot be simply represented as a space-normalized value (e.g. in an XML attribute). Although its content is mixed, it should be restricted so that only character data or the Text or Structured text is used. See StringValueType, IntValueType, ObservationalTimeValueType, TextValueType, and StructuredTextValueType for details.

Content:

```
{text} x (Text* | StructuredText*)?
```

Element Documentation:

Name	Type	Documentation
Text	TextType	Text is a reusable element, used for providing a language specific text value for general purposes (i.e. not for a name or description).
StructuredText	XHTMLType	StructuredText is a reusable element, used for providing a language specific text value structured as XHTML.

BooleanValueType: BooleanValueType is a refinement of SimpleValueType limiting the content to be a boolean.

Derivation:

```
*ValueType* (restriction)
   BooleanValueType
```

Content:

StringValueType: StringValueType is a refinement of SimpleValueType limiting the content to be a string. This can be further refined with facets, patterns, etc.

Derivation:

```
ValueType (restriction)
   StringValueType
```

Content:

IntValueType: IntValueType is a refinement of SimpleValueType limiting the content to be an integer. This can be further refined ranges, etc.

Derivation:

```
ValueType (restriction)
   IntValueType
```

Content:

DoubleValueType: DoubleValueType is a refinement of SimpleValueType limiting the content to be a double. This can be further refined ranges, etc.

Copyright © SDMx TWG 2020

Derivation:

```
ValueType (restriction)
   DoubleValueType
```

Content:

ObservationalTimePeriodValueType: ObservationalTimePeriodValueType is a refinement of SimpleValueType limiting the content to be an observational time period.

Derivation:

```
ValueType (restriction)
   ObservationalTimePeriodValueType
```

Content:

TextValueType: TextValueType is a restriction of ValueType that allows multiple Text elements to expressed a text value in multiple languages. The content of this should be restricted in its use to only allow a langue code (xml:lang) to be used once within an element of this type.

Derivation:

```
ValueType (restriction)
   TextValueType
```

Content:

`Text*`

Element Documentation:

Name	Type	Documentation
Text	TextType	Text is a reusable element, used for providing a language specific text value for general purposes (i.e. not for a name or description).

StructuredTextValueType: StructuredTextValueType is a restriction of ValueType that allows multiple StructuredText (XHTML mixed content) elements to expressed a text value in multiple languages. The content of this should be restricted in its use to only allow a langue code (xml:lang) to be used once within an element of this type.

Derivation:

```
ValueType (restriction)
   StructuredTextValueType
```

Content:

`StructuredText*`

Element Documentation:

Name	Type	Documentation
StructuredText	XHTMLType	StructuredText is a reusable element, used for providing a language specific text value structured as XHTML.

TextType: TextType provides for a set of language-specific alternates to be provided for any human-readable constructs in the instance.

Derivation:

```
xs:anySimpleType (restriction)
   xs:string (extension)
         TextType
```


Copyright © SDMx TWG 2020

Attributes:

`xml:lang?`

Content:

Attribute Documentation:

Name	Type	Documentation
xml:lang (default: en)	xs:language	The xml:lang attribute specifies a language code for the text. If not supplied, the default language is assumed to be English.

StatusMessageType: StatusMessageType describes the structure of an error or warning message. A message contains the text of the message, as well as an optional language indicator and an optional code.

Attributes:

`code?`

Content:

`Text+`

Attribute Documentation:

Name	Type	Documentation
code	xs:string	The code attribute holds an optional code identifying the underlying error that generated the message. This should be used if parallel language descriptions of the error are supplied, to distinguish which of the multiple error messages are for the same underlying error.

Element Documentation:

Name	Type	Documentation
Text	TextType	Text contains the text of the message, in parallel language values.

EmptyType: EmptyType is an empty complex type for elements where the presence of the tag indicates all that is necessary.

Content:

`{Empty}`

CodedStatusMessageType: CodedStatusMessageType describes the structure of an error or warning message which required a code.

Derivation:

```
StatusMessageType (restriction)
   CodedStatusMessageType
```

Attributes:

`code`

Content:

`Text+`

Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
code	xs:string	The code attribute holds an optional code identifying the underlying error that generated the message. This should be used if parallel language descriptions of the error are supplied, to distinguish which of the multiple error messages are for the same underlying error.

Element Documentation:

Name	Type	Documentation
Text	TextType	Text contains the text of the message, in parallel language values.

AnnotableType: AnnotableType is an abstract base type used for all annotable artefacts. Any type that provides for annotations should extend this type.

Content:

`Annotations?`

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotableType may reference it.

AnnotationsType: AnnotationsType provides for a list of annotations to be attached to data and structure messages.

Content:

`Annotation+`

Element Documentation:

Name	Type	Documentation
Annotation	AnnotationType	

AnnotationType: AnnotationType provides for non-documentation notes and annotations to be embedded in data and structure messages. It provides optional fields for providing a title, a type description, a URI, and the text of the annotation.

Attributes:

`id?`

Content:

`AnnotationTitle?, AnnotationType?, AnnotationURL*, AnnotationText*, AnnotationValue?`

Attribute Documentation:

Name	Type	Documentation
id	xs:string	The id attribute provides a non-standard identification of an annotation. It can be used to disambiguate annotations.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
AnnotationTitle	xs:string	AnnotationTitle provides a title for the annotation.
AnnotationType	xs:string	AnnotationType is used to distinguish between annotations designed to support various uses. The types are not enumerated, as these can be specified by the user or creator of the annotations. The definitions and use of annotation types should be documented by their creator.
AnnotationURL	AnnotationURLType	AnnotationURL is a URI - typically a URL - which points to an external resource which may contain or supplement the annotation. These can be localised by indicating a language for the resource. If a language is not specified, the resource is assumed to not be localised. If a specific behavior is desired, an annotation type should be defined which specifies the use of this field more exactly.
AnnotationText	TextType	AnnotationText holds a language-specific string containing the text of the annotation.
AnnotationValue	xs:string	AnnotationValue holds a non-localised value for the annotation.

AnnotationURLType: AnnotationURLType defines an external resource. These can indicate localisation by specifying a language for the resource.

Derivation:

```
xs:anySimpleType (restriction)
   xs:anyURI (extension)
         AnnotationURLType
```

Attributes:

`xml:lang?`

Content:

Attribute Documentation:

Name	Type	Documentation
xml:lang	xs:language	Indicates the language of the resources at the URL, if it is localised. If this does not exist, the resource is not localised.

LinkType:

Attributes:

`rel, url, urn?, type?`

Content:

`{Empty}`

Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
rel	xs:string	The type of object that is being linked to.
url	xs:anyURI	The url of the object being linked.
urn	xs:anyURI	A SDMX registry urn of the object being linked (if applicable).
type	xs:string	The type of link (e.g. PDF, text, HTML, reference metadata).

IdentifiableType: IdentifiableType is an abstract base type for all identifiable objects.

Derivation:

```
*AnnotableType* (extension)
   *IdentifiableType*
```

Attributes:

`id?, urn?, uri?`

Content:

`Annotations?, Link*`

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the metadata report can be dynamically inserted in the structure metadata.

NameableType: NameableType is an abstract base type for all nameable objects.

Derivation:

```
*AnnotableType* (extension)
   *IdentifiableType* (extension)
         *NameableType*
```

Attributes:

`id?, urn?, uri?`

Content:

Copyright © SDMx TWG 2020

`Annotations?, Link*, Name+, Description*`

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the metadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

VersionableType: VersionableType is an abstract base type for all versionable objects.

Derivation:

```
*AnnotableType* (extension)
   *IdentifiableType* (extension)
         *NameableType* (extension)
               *VersionableType*
```

Attributes:

`id?, urn?, uri?, version?, validFrom?, validTo?`

Content:

`Annotations?, Link*, Name+, Description*`

Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the metadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

MaintainableBaseType: MaintainableBaseType is an abstract type that only serves the purpose of forming the base for the actual MaintainableType. The purpose of this type is to restrict the VersionableType to require the id attribute.

Derivation:

```
*AnnotableType* (extension)
   *IdentifiableType* (extension)
         *NameableType* (extension)
               *VersionableType* (restriction)
                     *MaintainableBaseType*
```

Attributes:

`id, urn?, uri?, version?, validFrom?, validTo?`

Content:

`Annotations?, Link*, Name+, Description*`

Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MaintainableUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the metadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

MaintainableType: MaintainableType is an abstract base type for all maintainable objects.

Derivation:

```
*AnnotableType* (extension)
   *IdentifiableType* (extension)
         *NameableType* (extension)
               *VersionableType* (restriction)
                     *MaintainableBaseType* (extension)
                           *MaintainableType*
```

Attributes:

`id, urn?, uri?, version?, validFrom?, validTo?, agencyID, isExternalReference?, serviceURL?, structureURL?`

Content:

`Annotations?, Link*, Name+, Description*`

Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MaintainableUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the metadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

ReferencePeriodType: Specifies the inclusive start and end times.

Attributes:

`startTime, endTime`

Content:

`{Empty}`

Attribute Documentation:

Name	Type	Documentation
startTime	xs:dateTime	The startTime attributes contains the inclusive start date for the reference period.
endTime	xs:dateTime	The endTime attributes contains the inclusive end date for the reference period.

QueryableDataSourceType: QueryableDataSourceType describes a data source which is accepts an standard SDMX Query message and responds appropriately.

Attributes:

`isRESTDatasource, isWebServiceDatasource`

Content:

`DataURL, WSDLURL?, WADLURL?`

Attribute Documentation:

Name	Type	Documentation
isRESTDatasource	xs:boolean	The isRESTDatasource attribute indicates, if true, that the queryable data source is accessible via the REST protocol.
isWebServiceDatasource	xs:boolean	The isWebServiceDatasource attribute indicates, if true, that the queryable data source is accessible via Web Services protocols.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
DataURL	xs:anyURI	DataURL contains the URL of the data source.
WSDLURL	xs:anyURI	WSDLURL provides the location of a WSDL instance on the internet which describes the queryable data source.
WADLURL	xs:anyURI	WADLURL provides the location of a WADL instance on the internet which describes the REST protocol of the queryable data source.

XHTMLType: XHTMLType allows for mixed content of text and XHTML tags. When using this type, one will have to provide a reference to the XHTML schema, since the processing of the tags within this type is strict, meaning that they are validated against the XHTML schema provided.

Attributes:

`xml:lang?`

Content:

`{text} x {any element with namespace of http://www.w3.org/1999/xhtml}*`

Attribute Documentation:

Name	Type	Documentation
xml:lang (default: en)	xs:language	

PayloadStructureType: PayloadStructureType is an abstract base type used to define the structural information for data or metadata sets. A reference to the structure is provided (either explicitly or through a reference to a structure usage).

Attributes:

`structureID, schemaURL?, namespace?, dimensionAtObservation?, explicitMeasures?, serviceURL?, structureURL?`

Content:

`(ProvisionAgreement | StructureUsage | Structure)`

Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
structureID	xs:ID	The structureID attribute uniquely identifies the structure for the purpose of referencing it from the payload. This is only used in structure specific formats. Although it is required, it is only useful when more than one data set is present in the message.
schemaURL	xs:anyURI	The schemaURL attribute provides a location from which the structure specific schema can be located.
namespace	xs:anyURI	The namespace attribute is used to provide the namespace for structure-specific formats. By communicating this information in the header, it is possible to generate the structure specific schema while processing the message.
dimensionAtObservation	ObservationDimension Type	The dimensionAtObservation is used to reference the dimension at the observation level for data messages. This can also be given the explicit value of “AllDimensions” which denotes that the cross sectional data is in the flat format.
explicitMeasures	xs:boolean	The explicitMeasures indicates whether explicit measures are used in the cross sectional format. This is only applicable for the measure dimension as the dimension at the observation level or the flat structure.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Element Documentation:

Name	Type	Documentation
ProvisionAgreement	ProvisionAgreementReferenceType	ProvisionAgreement references a provision agreement which the data or metadata is reported against.
StructureUsage	StructureUsageReferenceType	StructureUsage references a flow which the data or metadata is reported against.
Structure	StructureReferenceType	Structure references the structure which defines the structure of the data or metadata set.

DataStructureType: DataStructureType is an abstract base type the forms the basis for the structural information for a data set.

Derivation:

```
*PayloadStructureType* (restriction)
   *DataStructureType*
```

Attributes:

`structureID, schemaURL?, namespace?, dimensionAtObservation?, explicitMeasures?, serviceURL?, structureURL?`

Copyright © SDMx TWG 2020

Content:

`(ProvisionAgreement | StructureUsage | Structure)`

Attribute Documentation:

Name	Type	Documentation
structureID	xs:ID	The structureID attribute uniquely identifies the structure for the purpose of referencing it from the payload. This is only used in structure specific formats. Although it is required, it is only useful when more than one data set is present in the message.
schemaURL	xs:anyURI	The schemaURL attribute provides a location from which the structure specific schema can be located.
namespace	xs:anyURI	The namespace attribute is used to provide the namespace for structure-specific formats. By communicating this information in the header, it is possible to generate the structure specific schema while processing the message.
dimensionAtObservation	ObservationDimension Type	The dimensionAtObservation is used to reference the dimension at the observation level for data messages. This can also be given the explicit value of “AllDimensions” which denotes that the cross sectional data is in the flat format.
explicitMeasures	xs:boolean	The explicitMeasures indicates whether explicit measures are used in the cross sectional format. This is only applicable for the measure dimension as the dimension at the observation level or the flat structure.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Element Documentation:

Name	Type	Documentation
ProvisionAgreement	ProvisionAgreementReferenceType	ProvisionAgreement references a provision agreement which the data is reported against.
StructureUsage	DataflowReferenceType	StructureUsage references a dataflow which the data is reported against.
Structure	DataStructureReferenceType	Structure references the data structure definition which defines the structure of the data.

StructureSpecificDataStructureType: StructureSpecificDataStructureType defines the structural information for a structured data set. In addition to referencing the data structure or dataflow which defines the structure of the data, the namespace for the data structure specific schema as well as which dimension is used at the observation level must be provided. It is also necessary to state whether the format uses explicit measures, although this is technically only applicable is the dimension at the observation level is the measure dimension or the flat data format is used.

Copyright © SDMx TWG 2020

Derivation:

```
PayloadStructureType (restriction)
   DataStructureType (restriction)
         StructureSpecificDataStructureType
```

Attributes:

`structureID, schemaURL?, namespace, dimensionAtObservation, explicitMeasures?, serviceURL?, structureURL?`

Content:

`(ProvisionAgreement | StructureUsage | Structure)`

Attribute Documentation:

Name	Type	Documentation
structureID	xs:ID	The structureID attribute uniquely identifies the structure for the purpose of referencing it from the payload. This is only used in structure specific formats. Although it is required, it is only useful when more than one data set is present in the message.
schemaURL	xs:anyURI	The schemaURL attribute provides a location from which the structure specific schema can be located.
namespace	xs:anyURI	The namespace attribute is used to provide the namespace for structure-specific formats. By communicating this information in the header, it is possible to generate the structure specific schema while processing the message.
dimensionAtObservation	ObservationDimension Type	The dimensionAtObservation is used to reference the dimension at the observation level for data messages. This can also be given the explicit value of “AllDimensions” which denotes that the cross sectional data is in the flat format.
explicitMeasures (default: false)	xs:boolean	The explicitMeasures indicates whether explicit measures are used in the cross sectional format. This is only applicable for the measure dimension as the dimension at the observation level or the flat structure.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
ProvisionAgreement	ProvisionAgreementReferenceType	ProvisionAgreement references a provision agreement which the data is reported against.
StructureUsage	DataflowReferenceType	StructureUsage references a dataflow which the data is reported against.
Structure	DataStructureReferenceType	Structure references the data structure definition which defines the structure of the data.

MetadataStructureType: MetadataStructureType is an abstract base type the forms the basis of the structural information for any metadata message. A reference to the metadata structure definition or a metadataflow must be provided. This can be used to determine the structure of the message.

Derivation:

```
PayloadStructureType (restriction)
   MetadataStructureType
```

Attributes:

`structureID, schemaURL?, namespace?, serviceURL?, structureURL?`

Content:

`(ProvisionAgreement | StructureUsage | Structure)`

Attribute Documentation:

Name	Type	Documentation
structureID	xs:ID	The structureID attribute uniquely identifies the structure for the purpose of referencing it from the payload. This is only used in structure specific formats. Although it is required, it is only useful when more than one data set is present in the message.
schemaURL	xs:anyURI	The schemaURL attribute provides a location from which the structure specific schema can be located.
namespace	xs:anyURI	The namespace attribute is used to provide the namespace for structure-specific formats. By communicating this information in the header, it is possible to generate the structure specific schema while processing the message.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
ProvisionAgreement	ProvisionAgreementReferenceType	ProvisionAgreement references a provision agreement which the metadata is reported against.
StructureUsage	MetadataflowReferenceType	StructureUsage references a metadataflow which the metadata is reported against.
Structure	MetadataStructureReferenceType	Structure references the metadata structure definition which defines the structure of the metadata.

GenericMetadataStructureType: GenericMetadataStructureType defines the structural information for a generic metadata message.

Derivation:

```
PayloadStructureType (restriction)
   MetadataStructureType (restriction)
         GenericMetadataStructureType
```

Attributes:

`structureID, schemaURL?, serviceURL?, structureURL?`

Content:

`(ProvisionAgreement | StructureUsage | Structure)`

Attribute Documentation:

Name	Type	Documentation
structureID	xs:ID	The structureID attribute uniquely identifies the structure for the purpose of referencing it from the payload. This is only used in structure specific formats. Although it is required, it is only useful when more than one data set is present in the message.
schemaURL	xs:anyURI	The schemaURL attribute provides a location from which the structure specific schema can be located.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Element Documentation:

Name	Type	Documentation
ProvisionAgreement	ProvisionAgreementReferenceType	ProvisionAgreement references a provision agreement which the metadata is reported against.
StructureUsage	MetadataflowReferenceType	StructureUsage references a metadataflow which the metadata is reported against.
Structure	MetadataStructureReferenceType	Structure references the metadata structure definition which defines the structure of the metadata.

Copyright © SDMx TWG 2020

Simple Types

AlphaNumericType: AlphaNumericType is a reusable simple type that allows for only mixed-case alphabetical and numeric characters.

Derived by restriction of `xs:string`.
Regular Expression Pattern: `[A-Za-z0-9]+`

AlphaType: AlphaType is a reusable simple type that allows for only mixed-case alphabetical characters. This is derived from the AlphaNumericType.

Derived by restriction of `AlphaNumericType` .
Regular Expression Pattern: `[A-Za-z]+`

NumericType: NumericType is a reusable simple type that allows for only numeric characters. This is not to be confused with an integer, as this may be used to numeric strings which have leading zeros. These leading zeros are not ignored. This is derived from the AlphaNumericType.

Derived by restriction of `AlphaNumericType`.
Regular Expression Pattern: `[0-9]+`

ObservationalTimePeriodType: ObservationalTimePeriodType specifies a distinct time period or point in time in SDMX. The time period can either be a Gregorian calendar period, a standard reporting period, a distinct point in time, or a time range with a specific date and duration.

Union of:

`xs:gYear, xs:gYearMonth, xs:date, xs:dateTime, ReportingYearType, ReportingSemesterType, ReportingTrimesterType, ReportingQuarterType, ReportingMonthType, ReportingWeekType, ReportingDayType, TimeRangeType.`

StandardTimePeriodType: StandardTimePeriodType defines the set of standard time periods in SDMX. This includes the reporting time periods and the basic date type (i.e. the calendar time periods and the dateTime format).

Union of:

`xs:gYear, xs:gYearMonth, xs:date, xs:dateTime, ReportingYearType, ReportingSemesterType, ReportingTrimesterType, ReportingQuarterType, ReportingMonthType, ReportingWeekType, ReportingDayType.`

BasicTimePeriodType: BasicTimePeriodType contains the basic dates and calendar periods. It is a combination of the Gregorian time periods and the date time type..

Union of:

`xs:gYear, xs:gYearMonth, xs:date, xs:dateTime.`

GregorianTimePeriodType: GregorianTimePeriodType defines the set of standard calendar periods in SDMX.

Union of:

`xs:gYear, xs:gYearMonth, xs:date.`

ReportingTimePeriodType: ReportingTimePeriodType defines standard reporting periods in SDMX, which are all in relation to the start day (day-month) of a reporting year which is specified in the specialized reporting year start day attribute. If the reporting year start day is not defined, a day of January 1 is assumed. The reporting year must be expressed as the year at the beginning of the period. Therefore, if the reporting year runs from April to March, any given reporting year is expressed as the year for April. The general format of a report period can be described as `[year]-[period][time zone]`?, where the type of period is designated with a single character followed by a number representing the period. Note that all periods allow for an optional time zone offset. See the details of each member type for the specifics of its format.

Union of:

Copyright © SDMx TWG 2020

`ReportingYearType, ReportingSemesterType, ReportingTrimesterType, ReportingQuarterType, ReportingMonthType, ReportingWeekType, ReportingDayType.`

BaseReportPeriodType: BaseReportPeriodType is a simple type which frames the general pattern of a reporting period for validation purposes. This regular expression is only a general validation which is meant to validate the following structure `[year]-[period][time zone]`?. This type is meant to be derived from for further validation.

Derived by restriction of `xs:string`.
Regular Expression Pattern: `\d{4}\\([ASTQ]\d{1}|[MW]\d{2}|[D]\d{3})(Z|((\\|\\)\d{2}:\d{2}))?`

ReportPeriodValidTimeZoneType: ReportPeriodValidTimeZoneType is a derivation of the BaseReportPeriodType which validates that the time zone provided in the base type is valid. The base type will have provided basic validation already. The patterns below validate that the time zone is “Z” or that it is between -14:00 and +14:00, or that there is no time zone provided. This type is meant to be derived from for further validation.

Derived by restriction of `BaseReportPeriodType`.
Regular Expression Pattern: `.+Z.{5}.*(\\|\\)(14:00|((0[0-9]|1[0-3]):[0-5][0-9])).{5}[^\\\\Z]+`

ReportingYearType: ReportingYearType defines a time period of 1 year (P1Y) in relation to a reporting year which has a start day (day-month) specified in the specialized reporting year start day attribute. In the absence of a start day for the reporting year, a day of January 1 is assumed. In this case a reporting year will coincide with a calendar year. The format of a reporting year is YYYY-A1 (e.g. 2000-A1). Note that the period value of 1 is fixed.

Derived by restriction of `ReportPeriodValidTimeZoneType`.
Regular Expression Pattern: `.{5}A1.*`

ReportingSemesterType: ReportingSemesterType defines a time period of 6 months (P6M) in relation to a reporting year which has a start day (day-month) specified in the specialized reporting year start day attribute. In the absence of a start day for the reporting year, a day of January 1 is assumed. The format of a reporting semester is YYYY-Ss (e.g. 2000-S1), where s is either 1 or 2.

Derived by restriction of `ReportPeriodValidTimeZoneType`.
Regular Expression Pattern: `.{5}S[1-2].*`

ReportingTrimesterType: ReportingTrimesterType defines a time period of 4 months (P4M) in relation to a reporting year which has a start day (day-month) specified in the specialized reporting year start day attribute. In the absence of a start day for the reporting year, a day of January 1 is assumed. The format of a reporting trimester is YYYY-Tt (e.g. 2000-T1), where s is either 1, 2, or 3.

Derived by restriction of `ReportPeriodValidTimeZoneType`.
Regular Expression Pattern: `.{5}T[1-3].*`

ReportingQuarterType: ReportingQuarterType defines a time period of 3 months (P3M) in relation to a reporting year which has a start day (day-month) specified in the specialized reporting year start day attribute. In the absence of a start day for the reporting year, a day of January 1 is assumed. The format of a reporting quarter is YYYY-Qq (e.g. 2000-Q1), where q is a value between 1 and 4.

Derived by restriction of `ReportPeriodValidTimeZoneType`.
Regular Expression Pattern: `.{5}Q[1-4].*`

ReportingMonthType: ReportingMonthType defines a time period of 1 month (P1M) in relation to a reporting year which has a start day (day-month) specified in the specialized reporting year start day attribute. In the absence of a start day for the reporting year, a day of January 1 is assumed. In this case a reporting month will coincide with a calendar month. The format of a reporting month is YYYY-Mmm (e.g. 2000-M01), where mm is a two digit month (i.e. 01-12).

Derived by restriction of `ReportPeriodValidTimeZoneType`.
Regular Expression Pattern: `.{5}M(0[1-9]|1[0-2]).*`

ReportingWeekType: ReportingWeekType defines a time period of 7 days (P7D) in relation to a reporting year which has a start day (day-month) specified in the specialized reporting year start day attribute. A standard reporting week is based on the ISO 8601 definition of a week date, in relation to the reporting period start day. The first week is defined as the week with the first Thursday

Copyright © SDMx TWG 2020

on or after the reporting year start day. An equivalent definition is the week starting with the Monday nearest in time to the reporting year start day. There are other equivalent definitions, all of which should be adjusted based on the reporting year start day. In the absence of a start day for the reporting year, a day of January 1 is assumed. The format of a reporting week is YYYY-Www (e.g. 2000-W01), where mm is a two digit week (i.e. 01-53).

Derived by restriction of `ReportPeriodValidTimeZoneType`.
Regular Expression Pattern: `.{5}W(0[1-9]|[1-4][0-9]|5[0-3]).*`

ReportingDayType: ReportingDayType defines a time period of 1 day (P1D) in relation to a reporting year which has a start day (day-month) specified in the specialized reporting year start day attribute. In the absence of a start day for the reporting year, a day of January 1 is assumed. The format of a reporting day is YYYY-Dddd (e.g. 2000-D001), where ddd is a three digit day (i.e. 001-366).

Derived by restriction of `ReportPeriodValidTimeZoneType`.
Regular Expression Pattern: `.{5}D(0[0-9][1-9]|[1-2][0-9][0-9]|3[0-5][0-9]|36[0-6]).*`

BaseTimeRangeType: BaseTimeRangeType is a simple type which frames the general pattern for a time range in SDMX. A time range pattern is generally described as [xs:date or xs:dateTime]\xs:duration], where the referenced types are defined by XML Schema. This type is meant to be derived from for further validation.

Derived by restriction of `xs:string`.
Regular Expression Pattern: `\d{4}\\\d{2}\\\d{2}(T\d{2}:\d{2}:\d{2}(\\\d+)?)?(Z|((\\|\\)\d{2}:\d{2}))?/P.+`

RangeValidMonthDayType: RangeValidMonthDayType is a derivation of the BaseTimeRangeType which validates that the day provided is valid for the month, without regard to leap years. The base type will have provided basic validation already. The patterns below validate that there are up to 29 days in February, up to 30 days in April, June, September, and November and up to 31 days in January, March, May, July, August, October, and December. This type is meant to be derived from for further validation.

Derived by restriction of `BaseTimeRangeType`.
Regular Expression Pattern: `.{5}02\\(0[1-9]|[1-2][0-9]).+.{5}(04|06|09|11)\\(0[1-9]|[1-2][0-9]|30).+.{5}(01|03|05|07|08|10|12)\\(0[1-9]|[1-2][0-9]|3[0-1]).+`

RangeValidLeapYearType: RangeValidLeapYearType is a derivation of the RangeValidMonthDayType which validates that a date of February 29 occurs in a valid leap year (i.e. if the year is divisible 4 and not by 100, unless it is also divisible by 400). This type is meant to be derived from for further validation.

Derived by restriction of `RangeValidMonthDayType`.
Regular Expression Pattern: `((\d{2}(04|08|12|16|20|24|28|32|36|40|44|48|52|56|60|64|68|72|76|80|84|88|92|96))|((00|04|08|12|16|20|24|28|32|36|40|44|48|52|56|60|64|68|72|76|80|84|88|92|96)00))\\02\\29.+.{5}02\\(([0-1][0-9])|(2[^9])).+.{5}((0[1,3-9])|1[0-2]).+`

RangeValidTimeType: RangeValidTimeType is a derivation of the RangeValidLeapYearType which validates that the time (if provided) is validly formatted. The base type will have provided basic validation already. The patterns below validate that the time falls between 00:00:00 and 24:00:00. Note that as the XML dateTime type does, seconds are required. It is also permissible to have fractions of seconds, but only within the boundaries of the range specified. This type is meant to be derived from for further validation.

Derived by restriction of `RangeValidLeapYearType`.
Regular Expression Pattern:
`.{10}T(24:00:00(\\[0]+)?|((([0-1][0-9])|(2[0-3])):[0-5][0-9]:[0-5][0-9](\\\d+)?))(/|Z|\\|\\).+[^T]+/.+`

RangeValidTimeZoneType: RangeValidMonthDayType is a derivation of the RangeValidTimeType which validates that the time zone provided in the base type is valid. The base type will have provided basic validation already. The patterns below validate that the time zone is “Z” or that it is between -14:00 and +14:00, or that there is no time zone provided. This type is meant to be derived from for further validation.

Derived by restriction of `RangeValidTimeType`.
Regular Expression Pattern: `.+Z/.+.{10}.*(\\|\\)(14:00|((0[0-9]|1[0-3]):[0-5][0-9]))/.+.{10}[^\\\\Z]+`

TimeRangeValidDateDurationType: TimeRangeValidDateDurationType is an abstract derivation of the RangeValidTimeType which validates that duration provided is generally valid, up to the time component.

Copyright © SDMx TWG 2020

Derived by restriction of `RangeValidTimeZoneType`.
Regular Expression Pattern: `.+/P(\d+Y)?(\d+M)?(\d+D)?(T.+)?`

TimeRangeType: TimeRangeType defines the structure of a time range in SDMX. The pattern of a time range can be generally described as `[start date]\\duration]`, where start date is an date or dateTime type as defined in XML Schema and duration is a time duration as defined in XML Schema. Note that it is permissible for a time zone offset to be provided on the date or date time.

Derived by restriction of `TimeRangeValidDateDurationType`.
Regular Expression Pattern: `.+/P.*T(\d+H)?(\d+M)?(\d+(.\d+)?S)?.+/P[^T]+`

TimezoneType: TimezoneType defines the pattern for a time zone. An offset of -14:00 to +14:00 or Z can be specified.

Derived by restriction of `xs:string`.
Regular Expression Pattern: `Z(\\|\\)(14:00|((0[0-9]|1[0-3]):[0-5][0-9]))`

OccurrenceType: OccurrenceType is used to express the maximum occurrence of an object. It combines an integer, equal or greater than 1, and the literal text, “unbounded”, for objects which have no upper limit on its occurrence.

Union of:

`MaxOccursNumberType, UnboundedCodeType.`

MaxOccursNumberType: MaxOccursNumberType is a base type used to restrict an integer to be greater than 1, for the purpose of expressing the maximum number of occurrences of an object.

Derived by restriction of `xs:nonNegativeInteger`.
Minimum (inclusive): `1`
Fraction Digits: `0`

UnboundedCodeType: UnboundedCodeType provides single textual value of “unbounded”, for use in OccurrentType.

Derived by restriction of `xs:string`.

Enumerations:

Value	Documentation
unbounded	Object has no upper limit on occurrences.

ActionType: ActionType provides a list of actions, describing the intention of the data transmission from the sender’s side. Each action provided at the data or metadata set level applies to the entire data set for which it is given. Note that the actions indicated in the Message Header are optional, and used to summarize specific actions indicated with this data type for all registry interactions. The “Informational” value is used when the message contains information in response to a query, rather than being used to invoke a maintenance activity.

Derived by restriction of `xs:NMTOKEN`.

Copyright © SDMx TWG 2020

Enumerations:

Value	Documentation
Append	Append - this is an incremental update for an existing data/metadata set or the provision of new data or documentation (attribute values) formerly absent. If any of the supplied data or metadata is already present, it will not replace that data or metadata. This corresponds to the “Update” value found in version 1.0 of the SDMX Technical Standards.
Replace	Replace - data/metadata is to be replaced, and may also include additional data/metadata to be appended. The replacement occurs at the level of the observation - that is, it is not possible to replace an entire series.
Delete	Delete - data/metadata is to be deleted. Deletion occurs at the lowest level object. For instance, if a delete data message contains a series with no observations, then the entire series will be deleted. If the series contains observations, then only those observations specified will be deleted. The same basic concept applies for attributes. If a series or observation in a delete message contains attributes, then only those attributes will be deleted.
Information	Informational - data/metadata is being exchanged for informational purposes only, and not meant to update a system.

WildCardValueType: WildCardValueType is a single value code list, used to include the ‘%’ character - indicating that an entire field is wild carded.

Derived by restriction of `xs:string`.

Enumerations:

Value	Documentation
%	Indicates a wild card value.

CascadeSelectionType:

Union of:

`xs:boolean, ExcludeRootType.`

ExcludeRootType: ExcludeRootType is a single enumerated value that indicates that child values should be included, but that the actual root should not.

Derived by restriction of `xs:string`.

Enumerations:

Value	Documentation
excluderoot	

ObservationDimensionType: ObservationDimensionType allows for the dimension at the observation level to be specified by either providing the dimension identifier or using the explicit value “AllDimensions”.

Union of:

`NCNameIDType, ObsDimensionsCodeType`.

ObsDimensionsCodeType: ObsDimensionsCodeType is an enumeration containing the values “TimeDimension” and “AllDimensions”

Derived by restriction of `xs:string`.

Copyright © SDMx TWG 2020

Enumerations:

Value	Documentation
AllDimensions	AllDimensions notes that the cross sectional format shall be flat; that is all dimensions should be contained at the observation level.
TIME_PERIOD	TIME_PERIOD refers to the fixed identifier for the time dimension.

DataType: DataTypeType provides an enumerated list of the types of data formats allowed as the for the representation of an object.

Derived by restriction of `xs:NMTOKEN`.

Copyright © SDMx TWG 2020

Enumerations:

Copyright © SDMx TWG 2020

Value	Documentation
String	A string datatype corresponding to W3C XML Schema’s xs:string datatype.
Alpha	A string datatype which only allows for the simple alphabetic character set of A-Z, a-z.
AlphaNumeric	A string datatype which only allows for the simple alphabetic character set of A-Z, a-z plus the simple numeric character set of 0-9.
Numeric	A string datatype which only allows for the simple numeric character set of 0-9. This format is not treated as an integer, and therefore can having leading zeros.
BigInteger	An integer datatype corresponding to W3C XML Schema’s xs:integer datatype.
Integer	An integer datatype corresponding to W3C XML Schema’s xs:int datatype.
Long	A numeric datatype corresponding to W3C XML Schema’s xs:long datatype.
Short	A numeric datatype corresponding to W3C XML Schema’s xs:short datatype.
Decimal	A numeric datatype corresponding to W3C XML Schema’s xs:decimal datatype.
Float	A numeric datatype corresponding to W3C XML Schema’s xs:float datatype.
Double	A numeric datatype corresponding to W3C XML Schema’s xs:double datatype.
Boolean	A datatype corresponding to W3C XML Schema’s xs:boolean datatype.
URI	A datatype corresponding to W3C XML Schema’s xs:anyURI datatype.
Count	A simple incrementing Integer type. The isSequence facet must be set to true, and the interval facet must be set to “1”.
InclusiveValueRange	This value indicates that the startValue and endValue attributes provide the inclusive boundaries of a numeric range of type xs:decimal.
ExclusiveValueRange	This value indicates that the startValue and endValue attributes provide the exclusive boundaries of a numeric range, of type xs:decimal.
Incremental	This value indicates that the value increments according to the value provided in the interval facet, and has a true value for the isSequence facet.
ObservationalTimePeriod	Observational time periods are the superset of all time periods in SDMX. It is the union of the standard time periods (i.e. Gregorian time periods, the reporting time periods, and date time) and a time range.
StandardTimePeriod	Standard time periods is a superset of distinct time period in SDMX. It is the union of the basic time periods (i.e. the Gregorian time periods and date time) and the reporting time periods.
BasicTimePeriod	BasicTimePeriod time periods is a superset of the Gregorian time periods and a date time.
GregorianTimePeriod	Gregorian time periods correspond to calendar periods and are represented in ISO-8601 formats. This is the union of the year, year month, and date formats.
GregorianYear	A Gregorian time period corresponding to W3C XML Schema’s xs:gYear datatype, which is based on ISO-8601.
GregorianYearMonth	A time datatype corresponding to W3C XML Schema’s xs:gYearMonth datatype, which is based on ISO-8601.
GregorianDay	A time datatype corresponding to W3C XML Schema’s xs:date datatype, which is based on ISO-8601.
ReportingTimePeriod	Reporting time periods represent periods of a standard length within a reporting year, where to start of the year (defined as a month and day) must be defined elsewhere or it is assumed to be January 1. This is the union of the reporting year, semester, trimester, quarter, month, week, and day.

Copyright © SDMx TWG 2020

Value	Documentation
ReportingYear	A reporting year represents a period of 1 year (P1Y) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingYearType.
ReportingSemester	A reporting semester represents a period of 6 months (P6M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingSemesterType.
ReportingTrimester	A reporting trimester represents a period of 4 months (P4M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingTrimesterType.
ReportingQuarter	A reporting quarter represents a period of 3 months (P3M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingQuarterType.
ReportingMonth	A reporting month represents a period of 1 month (P1M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingMonthType.
ReportingWeek	A reporting week represents a period of 7 days (P7D) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingWeekType.
ReportingDay	A reporting day represents a period of 1 day (P1D) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingDayType.
DateTime	A time datatype corresponding to W3C XML Schema’s xs:dateTime datatype.
TimeRange	TimeRange defines a time period by providing a distinct start (date or date time) and a duration.
Month	A time datatype corresponding to W3C XML Schema’s xs:gMonth datatype.
MonthDay	A time datatype corresponding to W3C XML Schema’s xs:gMonthDay datatype.
Day	A time datatype corresponding to W3C XML Schema’s xs:gDay datatype.
Time	A time datatype corresponding to W3C XML Schema’s xs:time datatype.
Duration	A time datatype corresponding to W3C XML Schema’s xs:duration datatype.
GeospatialInformation	A string used to describe geographical features like points (e.g., locations of places, landmarks, buildings, etc.), lines (e.g., rivers, roads, streets, etc.), or areas (e.g., geographical regions, countries, islands, land lots, etc.). A mix of different features is possible too, e.g., combining polygons and geographical points to describe a country and the location of its capital.
XHTML	This value indicates that the content of the component can contain XHTML markup.
KeyValues	This value indicates that the content of the component will be data key (a set of dimension references and values for the dimensions).
IdentifiableReference	This value indicates that the content of the component will be complete reference (either URN or full set of reference fields) to an Identifiable object in the SDMX Information Model.
DataSetReference	This value indicates that the content of the component will be reference to a data provider, which is actually a formal reference to a data provider and a data set identifier value.

BasicComponentDataType: BasicComponentDataType provides an enumerated list of the types of characters allowed in the textType attribute for all non-target object components.

Derived by restriction of `DataType`.

Copyright © SDMx TWG 2020

Enumerations:

Copyright © SDMx TWG 2020

Value	Documentation
String	A string datatype corresponding to W3C XML Schema’s xs:string datatype.
Alpha	A string datatype which only allows for the simple alphabetic charcter set of A-Z, a-z.
AlphaNumeric	A string datatype which only allows for the simple alphabetic character set of A-Z, a-z plus the simple numeric character set of 0-9.
Numeric	A string datatype which only allows for the simple numeric character set of 0-9. This format is not treated as an integer, and therefore can having leading zeros.
BigInteger	An integer datatype corresponding to W3C XML Schema’s xs:integer datatype.
Integer	An integer datatype corresponding to W3C XML Schema’s xs:int datatype.
Long	A numeric datatype corresponding to W3C XML Schema’s xs:long datatype.
Short	A numeric datatype corresponding to W3C XML Schema’s xs:short datatype.
Decimal	A numeric datatype corresponding to W3C XML Schema’s xs:decimal datatype.
Float	A numeric datatype corresponding to W3C XML Schema’s xs:float datatype.
Double	A numeric datatype corresponding to W3C XML Schema’s xs:double datatype.
Boolean	A datatype corresponding to W3C XML Schema’s xs:boolean datatype.
URI	A datatype corresponding to W3C XML Schema’s xs:anyURI datatype.
Count	A simple incrementing Integer type. The isSequence facet must be set to true, and the interval facet must be set to “1”.
InclusiveValueRange	This value indicates that the startValue and endValue attributes provide the inclusive boundaries of a numeric range of type xs:decimal.
ExclusiveValueRange	This value indicates that the startValue and endValue attributes provide the exclusive boundaries of a numeric range, of type xs:decimal.
Incremental	This value indicates that the value increments according to the value provided in the interval facet, and has a true value for the isSequence facet.
ObservationalTimePeriod	Observational time periods are the superset of all time periods in SDMX. It is the union of the standard time periods (i.e. Gregorian time periods, the reporting time periods, and date time) and a time range.
StandardTimePeriod	Standard time periods is a superset of distinct time period in SDMX. It is the union of the basic time periods (i.e. the Gregorian time periods and date time) and the reporting time periods.
BasicTimePeriod	BasicTimePeriod time periods is a superset of the Gregorian time periods and a date time.
GregorianTimePeriod	Gregorian time periods correspond to calendar periods and are represented in ISO-8601 formats. This is the union of the year, year month, and date formats.
GregorianYear	A Gregorian time period corresponding to W3C XML Schema’s xs:gYear datatype, which is based on ISO-8601.
GregorianYearMonth	A time datatype corresponding to W3C XML Schema’s xs:gYearMonth datatype, which is based on ISO-8601.
GregorianDay	A time datatype corresponding to W3C XML Schema’s xs:date datatype, which is based on ISO-8601.
ReportingTimePeriod	Reporting time periods represent periods of a standard length within a reporting year, where to start of the year (defined as a month and day) must be defined elsewhere or it is assumed to be January 1. This is the union of the reporting year, semester, trimester, quarter, month, week, and day.

Copyright © SDMx TWG 2020

Value	Documentation
ReportingYear	A reporting year represents a period of 1 year (P1Y) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingYearType.
ReportingSemester	A reporting semester represents a period of 6 months (P6M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingSemesterType.
ReportingTrimester	A reporting trimester represents a period of 4 months (P4M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingTrimesterType.
ReportingQuarter	A reporting quarter represents a period of 3 months (P3M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingQuarterType.
ReportingMonth	A reporting month represents a period of 1 month (P1M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingMonthType.
ReportingWeek	A reporting week represents a period of 7 days (P7D) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingWeekType.
ReportingDay	A reporting day represents a period of 1 day (P1D) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingDayType.
DateTime	A time datatype corresponding to W3C XML Schema’s xs:dateTime datatype.
TimeRange	TimeRange defines a time period by providing a distinct start (date or date time) and a duration.
Month	A time datatype corresponding to W3C XML Schema’s xs:gMonth datatype.
MonthDay	A time datatype corresponding to W3C XML Schema’s xs:gMonthDay datatype.
Day	A time datatype corresponding to W3C XML Schema’s xs:gDay datatype.
Time	A time datatype corresponding to W3C XML Schema’s xs:time datatype.
Duration	A time datatype corresponding to W3C XML Schema’s xs:duration datatype.
GeospatialInformation	A string used to describe geographical features like points (e.g., locations of places, landmarks, buildings, etc.), lines (e.g., rivers, roads, streets, etc.), or areas (e.g., geographical regions, countries, islands, land lots, etc.). A mix of different features is possible too, e.g., combining polygons and geographical points to describe a country and the location of its capital.
XHTML	This value indicates that the content of the component can contain XHTML markup.

SimpleDataType: SimpleDataType restricts BasicComponentDataType to specify the allowable data types for a data structure definition component. The XHTML representation is removed as a possible type.

Derived by restriction of `BasicComponentDataType`.

Copyright © SDMx TWG 2020

Enumerations:

Copyright © SDMx TWG 2020

Value	Documentation
String	A string datatype corresponding to W3C XML Schema’s xs:string datatype.
Alpha	A string datatype which only allows for the simple alphabetic charcter set of A-Z, a-z.
AlphaNumeric	A string datatype which only allows for the simple alphabetic character set of A-Z, a-z plus the simple numeric character set of 0-9.
Numeric	A string datatype which only allows for the simple numeric character set of 0-9. This format is not treated as an integer, and therefore can having leading zeros.
BigInteger	An integer datatype corresponding to W3C XML Schema’s xs:integer datatype.
Integer	An integer datatype corresponding to W3C XML Schema’s xs:int datatype.
Long	A numeric datatype corresponding to W3C XML Schema’s xs:long datatype.
Short	A numeric datatype corresponding to W3C XML Schema’s xs:short datatype.
Decimal	A numeric datatype corresponding to W3C XML Schema’s xs:decimal datatype.
Float	A numeric datatype corresponding to W3C XML Schema’s xs:float datatype.
Double	A numeric datatype corresponding to W3C XML Schema’s xs:double datatype.
Boolean	A datatype corresponding to W3C XML Schema’s xs:boolean datatype.
URI	A datatype corresponding to W3C XML Schema’s xs:anyURI datatype.
Count	A simple incrementing Integer type. The isSequence facet must be set to true, and the interval facet must be set to “1”.
InclusiveValueRange	This value indicates that the startValue and endValue attributes provide the inclusive boundaries of a numeric range of type xs:decimal.
ExclusiveValueRange	This value indicates that the startValue and endValue attributes provide the exclusive boundaries of a numeric range, of type xs:decimal.
Incremental	This value indicates that the value increments according to the value provided in the interval facet, and has a true value for the isSequence facet.
ObservationalTimePeriod	Observational time periods are the superset of all time periods in SDMX. It is the union of the standard time periods (i.e. Gregorian time periods, the reporting time periods, and date time) and a time range.
StandardTimePeriod	Standard time periods is a superset of distinct time period in SDMX. It is the union of the basic time periods (i.e. the Gregorian time periods and date time) and the reporting time periods.
BasicTimePeriod	BasicTimePeriod time periods is a superset of the Gregorian time periods and a date time.
GregorianTimePeriod	Gregorian time periods correspond to calendar periods and are represented in ISO-8601 formats. This is the union of the year, year month, and date formats.
GregorianYear	A Gregorian time period corresponding to W3C XML Schema’s xs:gYear datatype, which is based on ISO-8601.
GregorianYearMonth	A time datatype corresponding to W3C XML Schema’s xs:gYearMonth datatype, which is based on ISO-8601.
GregorianDay	A time datatype corresponding to W3C XML Schema’s xs:date datatype, which is based on ISO-8601.
ReportingTimePeriod	Reporting time periods represent periods of a standard length within a reporting year, where to start of the year (defined as a month and day) must be defined elsewhere or it is assumed to be January 1. This is the union of the reporting year, semester, trimester, quarter, month, week, and day.

Copyright © SDMx TWG 2020

Value	Documentation
ReportingYear	A reporting year represents a period of 1 year (P1Y) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingYearType.
ReportingSemester	A reporting semester represents a period of 6 months (P6M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingSemesterType.
ReportingTrimester	A reporting trimester represents a period of 4 months (P4M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingTrimesterType.
ReportingQuarter	A reporting quarter represents a period of 3 months (P3M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingQuarterType.
ReportingMonth	A reporting month represents a period of 1 month (P1M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingMonthType.
ReportingWeek	A reporting week represents a period of 7 days (P7D) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingWeekType.
ReportingDay	A reporting day represents a period of 1 day (P1D) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingDayType.
DateTime	A time datatype corresponding to W3C XML Schema’s xs:dateTime datatype.
TimeRange	TimeRange defines a time period by providing a distinct start (date or date time) and a duration.
Month	A time datatype corresponding to W3C XML Schema’s xs:gMonth datatype.
MonthDay	A time datatype corresponding to W3C XML Schema’s xs:gMonthDay datatype.
Day	A time datatype corresponding to W3C XML Schema’s xs:gDay datatype.
Time	A time datatype corresponding to W3C XML Schema’s xs:time datatype.
Duration	A time datatype corresponding to W3C XML Schema’s xs:duration datatype.
GeospatialInformation	A string used to describe geographical features like points (e.g., locations of places, landmarks, buildings, etc.), lines (e.g., rivers, roads, streets, etc.), or areas (e.g., geographical regions, countries, islands, land lots, etc.). A mix of different features is possible too, e.g., combining polygons and geographical points to describe a country and the location of its capital.

TimeDataType: TimeDataType restricts SimpleDataType to specify the allowable data types for representing a time value.

Derived by restriction of `SimpleDataType`.

Copyright © SDMx TWG 2020

Enumerations:

Value	Documentation
ObservationalTimePeriod	Observational time periods are the superset of all time periods in SDMX. It is the union of the standard time periods (i.e. Gregorian time periods, the reporting time periods, and date time) and a time range.
StandardTimePeriod	Standard time periods is a superset of distinct time period in SDMX. It is the union of the basic time periods (i.e. the Gregorian time periods and date time) and the reporting time periods.
BasicTimePeriod	BasicTimePeriod time periods is a superset of the Gregorian time periods and a date time.
GregorianTimePeriod	Gregorian time periods correspond to calendar periods and are represented in ISO-8601 formats. This is the union of the year, year month, and date formats.
GregorianYear	A Gregorian time period corresponding to W3C XML Schema's xs:gYear datatype, which is based on ISO-8601.
GregorianYearMonth	A time datatype corresponding to W3C XML Schema's xs:gYearMonth datatype, which is based on ISO-8601.
GregorianDay	A time datatype corresponding to W3C XML Schema's xs:date datatype, which is based on ISO-8601.
ReportingTimePeriod	Reporting time periods represent periods of a standard length within a reporting year, where to start of the year (defined as a month and day) must be defined elsewhere or it is assumed to be January 1. This is the union of the reporting year, semester, trimester, quarter, month, week, and day.
ReportingYear	A reporting year represents a period of 1 year (P1Y) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingYearType.
ReportingSemester	A reporting semester represents a period of 6 months (P6M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingSemesterType.
ReportingTrimester	A reporting trimester represents a period of 4 months (P4M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingTrimesterType.
ReportingQuarter	A reporting quarter represents a period of 3 months (P3M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingQuarterType.
ReportingMonth	A reporting month represents a period of 1 month (P1M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingMonthType.
ReportingWeek	A reporting week represents a period of 7 days (P7D) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingWeekType.
ReportingDay	A reporting day represents a period of 1 day (P1D) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingDayType.
DateTime	A time datatype corresponding to W3C XML Schema's xs:dateTime datatype.
TimeRange	TimeRange defines a time period by providing a distinct start (date or date time) and a duration.

UrnPrefixPart: This is validates the first part of the URN (urn:sdmx.org.infomodel.\<package>.\<class=). It is intended to be further restricted.

Derived by restriction of `xs:string`.
Regular Expression Pattern: `urn:sdmx:org\\sdmx\\infomodel\\[a-z]+\\[A-Za-z]+=[^=]+`

UrnClassesPart: This refines the prefix to make specific restrictions of package and class values. Note that only one of the patterns must match. It is intended to be further restricted.

Copyright © SDMx TWG 2020

Derived by restriction of `UrnPrefixPart`.
Regular Expression Pattern: `.+\\base\\Agency=.+.+\\base\\AgencyScheme=.+.+\\base\\Any=.+.+\\base\\DataConsumer=.+.+\\base\\DataConsumerScheme=.+.+\\base\\DataProvider=.+.+\\base\\DataProviderScheme=.+.+\\base\\MetadataProvider=.+.+\\base\\MetadataProviderScheme=.+.+\\base\\OrganisationUnit=.+.+\\base\\OrganisationUnitScheme=.+.+\\categoryscheme\\Categorisation=.+.+\\categoryscheme\\Category=.+.+\\categoryscheme\\CategoryScheme=.+.+\\categoryscheme\\ReportingCategory=.+.+\\categoryscheme\\ReportingTaxonomy=.+.+\\codelist\\Code=.+.+\\codelist\\Codelist=.+.+\\codelist\\HierarchicalCode=.+.+\\codelist\\Hierarchy=.+.+\\codelist\\HierarchyAssociation=.+.+\\codelist\\Level=.+.+\\codelist\\ValueList=.+.+\\conceptscheme\\Concept=.+.+\\conceptscheme\\ConceptScheme=.+.+\\datastructure\\AttributeDescriptor=.+.+\\datastructure\\DataAttribute=.+.+\\datastructure\\Dataflow=.+.+\\datastructure\\DataStructure=.+.+\\datastructure\\Dimension=.+.+\\datastructure\\DimensionDescriptor=.+.+\\datastructure\\GroupDimensionDescriptor=.+.+\\datastructure\\Measure=.+.+\\datastructure\\MeasureDescriptor=.+.+\\datastructure\\TimeDimension=.+.+\\metadatastructure\\MetadataAttribute=.+.+\\metadatastructure\\Metadataflow=.+.+\\metadatastructure\\MetadataSet=.+.+\\metadatastructure\\MetadataStructure=.+.+\\process\\Process=.+.+\\process\\ProcessStep=.+.+\\process\\Transition=.+.+\\registry\\DataConstraint=.+.+\\registry\\MetadataConstraint=.+.+\\registry\\MetadataProvisionAgreement=.+.+\\registry\\ProvisionAgreement=.+.+\\structuremapping\\CategorySchemeMap=.+.+\\structuremapping\\ConceptSchemeMap=.+.+\\structuremapping\\DatePatternMap=.+.+\\structuremapping\\EpochMap=.+.+\\structuremapping\\FrequencyFormatMapping=.+.+\\structuremapping\\OrganisationSchemeMap=.+.+\\structuremapping\\ReportingTaxonomyMap=.+.+\\structuremapping\\RepresentationMap=.+.+\\structuremapping\\StructureMap=.+.+\\transformation\\CustomType=.+.+\\transformation\\CustomTypeScheme=.+.+\\transformation\\NamePersonalisation=.+.+\\transformation\\NamePersonalisationScheme=.+.+\\transformation\\Ruleset=.+.+\\transformation\\RulesetScheme=.+.+\\transformation\\Transformation=.+.+\\transformation\\TransformationScheme=.+.+\\transformation\\UserDefinedOperator=.+.+\\transformation\\UserDefinedOperatorScheme=.+.+\\transformation\\VtlCodelistMapping=.+.+\\transformation\\VtlConceptMapping=.+.+\\transformation\\VtlDataflowMapping=.+.+\\transformation\\VtlMappingScheme=.+`

UrnAgencyPart: This restricts the prefix and classes patterns to validate the agency part of the URN (=\<agency_id>:).

Derived by restriction of `UrnClassesPart`.
Regular Expression Pattern: `.+=([A-Za-z][A-Za-z0-9\_\\]*(\\[A-Za-z][A-Za-z0-9\_\\]*)*):[^:]+`

WildcardUrnAgencyPart: This restricts the prefix and classes patterns to validate the agency part of a wildcarded URN reference (=\<agency_id>:).

Derived by restriction of `UrnClassesPart`.
Regular Expression Pattern: `.+=([A-Za-z][A-Za-z0-9\_\\]*(\\[A-Za-z][A-Za-z0-9\_\\]*)*):[^:]+.+=\\:[^:]+`

UrnMaintainableIdPart: This refines the prefix, classes, and agency patterns to validate the maintainable ID part of the URN (:\<maintainable_id(\<version_number>)). Note that it does not restrict the version part as it is intended to be further restricted.

Derived by restriction of `UrnAgencyPart`.
Regular Expression Pattern: `.+:([A-Za-z0-9\_@\$\\]+)\\[0-9A-Za-z\\\\\\]+\\[^(\\\\)]*`

WildcardUrnMaintainableIdPart: This refines the prefix, classes, and agnecy patterns to validate the maintainable ID part of a wildcarded URN reference (:\<maintainable_id(\<version_number>)). Note that it does not restrict the version part as it is intended to be further restricted.

Derived by restriction of `WildcardUrnAgencyPart`.
Regular Expression Pattern: `.+:([A-Za-z0-9\_@\$\\]+)\\[0-9A-Za-z\\\\\\\\]+\\[^(\\\\)]*.+:\\\\[0-9A-Za-z\\\\\\\\]+\\[^(\\\\)]*`

UrnVersionPart: This refines the prefix, classes, agency, and maintainable id patterns to validate the version number part of the URN ((\<version_number)). It accounts for both legacy and semantic versioning, but not wildcarding (for referencing). It is meant to be further refined, although all parts after this are optional.

Derived by restriction of `UrnMaintainableIdPart`.
Regular Expression Pattern: `.+\\(0|[1-9]\d*)(\\(0|[1-9]\d*))?\\.*.+\\(0|[1-9]\d*)(\\(0|[1-9]\d*)){2}(\\(([A-Za-z\\]|([A-Za-z\\][A-Za-z0-9\\]+)|([A-Za-z0-9\\]+[A-Za-z\\][A-Za-z0-9\\]*))|(0|[1-9][0-9]*))(\\(([A-Za-z\\]|([A-Za-z\\][A-Za-z0-9\\]+)|([A-Za-z0-9\\]+[A-Za-z\\][A-Za-z0-9\\]*))|(0|[1-9][0-9]*)))*)?\\.*`

UrnReferenceVersionPart: This refines the prefix, classes, agency, and maintainable id patterns to validate the version number part of a URN reference ((\<version_number)). It accounts for both legacy and semantic versioning (including late binding). It is meant to be further refined, although all parts after this are optional.

Copyright © SDMx TWG 2020

Derived by restriction of `UrnMaintainableIdPart`.
Regular Expression Pattern: `.+\\(0|[1-9]\d*)(\\(0|[1-9]\d*))?\\.*.+\\(0|[1-9]\d*)(\\(0|[1-9]\d*)){2}(\\(([A-Za-z\\]|([A-Za-z\\][A-Za-z0-9\\]+)|([A-Za-z0-9\\]+[A-Za-z\\][A-Za-z0-9\\]*))|(0|[1-9][0-9]*))(\\(([A-Za-z\\]|([A-Za-z\\][A-Za-z0-9\\]+)|([A-Za-z0-9\\]+[A-Za-z\\][A-Za-z0-9\\]*))|(0|[1-9][0-9]*)))*)?\\.*.+\\((0|[1-9]\d*)\\?)(\\((0|[1-9]\d*))){2}\\.*.+\\((0|[1-9]\d*))(\\((0|[1-9]\d*)\\?))(\\((0|[1-9]\d*)))\\.*.+\\((0|[1-9]\d*)\\?)(\\((0|[1-9]\d*)))(\\((0|[1-9]\d*)\\?))\\.*`

WildcardUrnVersionPart: This refines the prefix, classes, agency, and maintainable id patterns to validate the version number part of a wildcarded URN reference ((\<version_number)). It accounts for both legacy and semantic versioning (including late binding). It is meant to be further refined, although all parts after this are optional.

Derived by restriction of `WildcardUrnMaintainableIdPart`.
Regular Expression Pattern: `.+\\(0|[1-9]\d*)(\\(0|[1-9]\d*))?\\.*.+\\(0|[1-9]\d*)(\\(0|[1-9]\d*)){2}(\\(([A-Za-z\\]|([A-Za-z\\][A-Za-z0-9\\]+)|([A-Za-z0-9\\]+[A-Za-z\\][A-Za-z0-9\\]*))|(0|[1-9][0-9]*))(\\(([A-Za-z\\]|([A-Za-z\\][A-Za-z0-9\\]+)|([A-Za-z0-9\\]+[A-Za-z\\][A-Za-z0-9\\]*))|(0|[1-9][0-9]*)))*)?\\.*.+\\((0|[1-9]\d*)\\?)(\\((0|[1-9]\d*))){2}\\.*.+\\((0|[1-9]\d*))(\\((0|[1-9]\d*)\\?))(\\((0|[1-9]\d*)))\\.*.+\\((0|[1-9]\d*)\\?)(\\((0|[1-9]\d*)))(\\((0|[1-9]\d*)\\?))\\.*.+\\\\\\.*`

UrnType: The completes the refinement of the prefix, classes, agency, maintainable id, and version number patterns to validate the last, and optional, nested component part of the URN (e.g. (\<version_number>)\<containerobject-id>.\<object-id>*). The nested patterns provide a complete validation of the URN pattern.

Derived by restriction of `UrnVersionPart`.
Regular Expression Pattern: `.+\\(\\[A-Za-z0-9\_@\$\\]+(\\[A-Za-z0-9\_@\$\\]+)*)?`

UrnReferenceType: The completes the refinement of the prefix, classes, agency, maintainable id, and version number patterns to validate the last, and optional, nested component part of a URN reference (e.g. (\<version_number>)\<containerobject-id>.\<object-id>*). The nested patterns provide a complete validation of the URN pattern.

Derived by restriction of `UrnReferenceVersionPart`.
Regular Expression Pattern: `.+\\(\\[A-Za-z0-9\_@\$\\]+(\\[A-Za-z0-9\_@\$\\]+)*)?`

WildcardUrnType: The completes the refinement of the prefix, classes, agency, maintainable id, and version number patterns to validate the last, and optional, nested component part of a wildcarded URN reference (e.g. (\<version_number>)\<containerobject-id>.\<object-id>*). The nested patterns provide a complete validation of the URN pattern.

Derived by restriction of `WildcardUrnVersionPart`.
Regular Expression Pattern: `.+\\(\\[A-Za-z0-9\_@\$\\]+(\\[A-Za-z0-9\_@\$\\]+)*)?.+\\(\\\\(\\\\)*)?`

MaintainableUrnType: Restricts the URN so that the pattern ends after the version part.

Derived by restriction of `UrnType`.
Regular Expression Pattern: `.+\\`

MaintainableUrnReferenceType: Restricts the URN reference so that the pattern ends after the version part.

Derived by restriction of `UrnReferenceType`.
Regular Expression Pattern: `.+\\`

ComponentUrnType: Restricts the URN so that the pattern ends after the first component part.

Derived by restriction of `UrnType`.
Regular Expression Pattern: `.+\\\\[A-Za-z0-9\_@\$\\]+`

ComponentUrnReferenceType: Restricts the URN reference so that the pattern ends after the first component part.

Derived by restriction of `UrnReferenceType`.
Regular Expression Pattern: `.+\\\\[A-Za-z0-9\_@\$\\]+`

AgencyUrnType: Urn type for an agency.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\base\\Agency=.+:AGENCIES\\1\\0\\.+`

AgencySchemeUrnType: Urn type for an agency scheme.

Copyright © SDMx TWG 2020

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\base\\AgencyScheme=.+:AGENCIES\\1\\0\\`

DataConsumerUrnType: Urn type for a data consumer.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\base\\DataConsumer=.+:DATA_CONSUMERS\\1\\0\\.+`

DataConsumerSchemeUrnType: Urn type for a data consumer scheme.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\base\\DataConsumerScheme=.+:DATA_CONSUMERS\\1\\0\\`

DataProviderUrnType: Urn type for a data provider.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\base\\DataProvider=.+:DATA_PROVIDERS\\1\\0\\.+`

DataProviderSchemeUrnType: Urn type for a data provider scheme.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\base\\DataProviderScheme=.+:DATA_PROVIDERS\\1\\0\\`

MetadataProviderUrnType: Urn type for a metadata provider.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\base\\MetadataProvider=.+:METADATA_PROVIDERS\\1\\0\\.+`

MetadataProviderSchemeUrnType: Urn type for a metadata provider scheme.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\base\\MetadataProviderScheme=.+:METADATA_PROVIDERS\\1\\0\\`

OrganisationUnitUrnType: Urn type for an organisation unit.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\base\\OrganisationUnit=.+\\1\\0\\.+`

OrganisationUnitSchemeUrnType: Urn type for an organisation unit scheme.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\base\\OrganisationUnitScheme=.+\\1\\0\\`

CategorisationUrnType: Urn type for a categorisation.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\categoryscheme\\Categorisation=.+`

CategoryUrnType: Urn type for a category.

Derived by restriction of `UrnType`.
Regular Expression Pattern: `.+\\categoryscheme\\Category=.+`

CategorySchemeUrnType: Urn type for a category scheme.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\categoryscheme\\CategoryScheme=.+`

ReportingCategoryUrnType: Urn type for a reporting category.

Derived by restriction of `UrnType`.
Regular Expression Pattern: `.+\\categoryscheme\\ReportingCategory=.+`

ReportingTaxonomyUrnType: Urn type for a reporting taxonomy.

Copyright © SDMx TWG 2020

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\categoryscheme\\ReportingTaxonomy=.+`

CodeUrnType: Urn type for a code.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\codelist\\Code=.+`

CodelistUrnType: Urn type for a codelist.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\codelist\\Codelist=.+`

HierarchicalCodeUrnType: Urn type for a hierarchical code.

Derived by restriction of `UrnType`.
Regular Expression Pattern: `.+\\codelist\\HierarchicalCode=.+`

HierarchyUrnType: Urn type for a hierarchy.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\codelist\\Hierarchy=.+`

HierarchyAssociationUrnType: Urn type for a hierarchy association.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\codelist\\HierarchyAssociation=.+`

LevelUrnType: Urn type for a level.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\codelist\\Level=.+`

ValueListUrnType: Urn type for a value list.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\codelist\\ValueList=.+`

ConceptUrnType: Urn type for a concept.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\conceptscheme\\Concept=.+`

ConceptSchemeUrnType: Urn type for a concept scheme.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\conceptscheme\\ConceptScheme=.+`

AttributeDescriptorUrnType: Urn type for an attribute descriptor.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\datastructure\\AttributeDescriptor=.+`

DataAttributeUrnType: Urn type for a data attribute.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\datastructure\\DataAttribute=.+`

DataflowUrnType: Urn type for a dataflow.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\datastructure\\Dataflow=.+`

DataStructureUrnType: Urn type for a data structure.

Copyright © SDMx TWG 2020

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\datastructure\\DataStructure=.+`

DimensionUrnType: Urn type for a dimension.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\datastructure\\Dimension=.+`

DimensionDescriptorUrnType: Urn type for a dimension descriptor.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\datastructure\\DimensionDescriptor=.+`

GroupDimensionDescriptorUrnType: Urn type for a group dimension descriptor.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\datastructure\\GroupDimensionDescriptor=.+`

MeasureUrnType: Urn type for a measure.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\datastructure\\Measure=.+`

MeasureDescriptorUrnType: Urn type for a measure descriptor.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\datastructure\\MeasureDescriptor=.+`

TimeDimensionUrnType: Urn type for a time dimension.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\datastructure\\TimeDimension=.+`

MetadataAttributeUrnType: Urn type for a metadata attribute.

Derived by restriction of `UrnType`.
Regular Expression Pattern: `.+\\metadatastructure\\MetadataAttribute=.+`

MetadataflowUrnType: Urn type for a metadataflow.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\metadatastructure\\Metadataflow=.+`

MetadataSetUrnType: Urn type for a metadata set.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\metadatastructure\\MetadataSet=.+`

MetadataStructureUrnType: Urn type for a metadata structure.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\metadatastructure\\MetadataStructure=.+`

ProcessUrnType: Urn type for a process.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\process\\Process=.+`

ProcessStepUrnType: Urn type for a process step.

Derived by restriction of `UrnType`.
Regular Expression Pattern: `.+\\process\\ProcessStep=.+`

TransitionUrnType: Urn type for a transition.

Copyright © SDMx TWG 2020

Derived by restriction of `UrnType`.
Regular Expression Pattern: `.+\\process\\Transition=.+`

DataConstraintUrnType: Urn type for a data constraint.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\registry\\DataConstraint=.+`

MetadataConstraintUrnType: Urn type for a metadata constraint.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\registry\\MetadataConstraint=.+`

MetadataProvisionAgreementUrnType: Urn type for a metadata provision agreement.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\registry\\MetadataProvisionAgreement=.+`

ProvisionAgreementUrnType: Urn type for a provision agreement.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\registry\\ProvisionAgreement=.+`

CategorySchemeMapUrnType: Urn type for a category scheme map.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\structuremapping\\CategorySchemeMap=.+`

ConceptSchemeMapUrnType: Urn type for a concept scheme map.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\structuremapping\\ConceptSchemeMap=.+`

DatePatternMapUrnType: Urn type for a date pattern map.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\structuremapping\\DatePatternMap=.+`

EpochMapUrnType: Urn type for a epoch map.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\structuremapping\\EpochMap=.+`

FrequencyFormatMappingUrnType: Urn type for a frequency format mapping.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\structuremapping\\FrequencyFormatMapping=.+`

OrganisationSchemeMapUrnType: Urn type for a organisation scheme map.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\structuremapping\\OrganisationSchemeMap=.+`

ReportingTaxonomyMapUrnType: Urn type for a reporting taxonomy map.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\structuremapping\\ReportingTaxonomyMap=.+`

RepresentationMapUrnType: Urn type for a representation map.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\structuremapping\\RepresentationMap=.+`

StructureMapUrnType: Urn type for a structure map.

Copyright © SDMx TWG 2020

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\structuremapping\\StructureMap=.+`

CustomTypeUrnType: Urn type for a custom type.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\transformation\\CustomType=.+`

CustomTypeSchemeUrnType: Urn type for a custom type scheme.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\transformation\\CustomTypeScheme=.+`

NamePersonalisationUrnType: Urn type for a name personalisation.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\transformation\\NamePersonalisation=.+`

NamePersonalisationSchemeUrnType: Urn type for a name personalisation scheme.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\transformation\\NamePersonalisationScheme=.+`

RulesetUrnType: Urn type for a ruleset.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\transformation\\Ruleset=.+`

RulesetSchemeUrnType: Urn type for a ruleset scheme.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\transformation\\RulesetScheme=.+`

TransformationUrnType: Urn type for a transformation.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\transformation\\Transformation=.+`

TransformationSchemeUrnType: Urn type for a transformation scheme.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\transformation\\TransformationScheme=.+`

UserDefinedOperatorUrnType: Urn type for a user defined operator.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\transformation\\UserDefinedOperator=.+`

UserDefinedOperatorSchemeUrnType: Urn type for a user defined operator scheme.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\transformation\\UserDefinedOperatorScheme=.+`

VtlMappingUrnType: Urn type for a VTL mapping.

Derived by restriction of `ComponentUrnType`.
Regular Expression Pattern: `.+\\transformation\\VtlCodelistMapping=.+.+\\transformation\\VtlConceptMapping=.+.+\\transformation\\VtlDataflowMapping=.+`

VtlMappingSchemeUrnType: Urn type for a VTL mapping scheme.

Derived by restriction of `MaintainableUrnType`.
Regular Expression Pattern: `.+\\transformation\\VtlMappingScheme=.+`

StructureOrUsageReferenceType: A reference type for a structure or structure usage.

Copyright © SDMx TWG 2020

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\datastructure\\DataStructure=.+.+\\datastructure\\Dataflow=.+.+\\metadatastructure\\MetadataStructure=.+.+\\metadatastructure\\Metadataflow=.+`

StructureUsageReferenceType: A reference type for structure usage.

Derived by restriction of `StructureOrUsageReferenceType`.
Regular Expression Pattern: `.+\\datastructure\\Dataflow=.+.+\\metadatastructure\\Metadataflow=.+`

StructureReferenceType: A reference type for a structure.

Derived by restriction of `StructureOrUsageReferenceType`.
Regular Expression Pattern: `.+\\datastructure\\DataStructure=.+.+\\metadatastructure\\MetadataStructure=.+`

AnyCodelistReferenceType: A reference type for a codelist or value list.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\codelist\\Codelist=.+.+\\codelist\\ValueList=.+`

OrganisationSchemeReferenceType: A reference type for any type of organisation scheme.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern:
`.+\\base\\AgencyScheme=.+:AGENCIES\\.+\\.+\\base\\DataConsumerScheme=.+:DATA_CONSUMERS\\.+\\.+\\base\\DataProviderScheme=.+:DATA_PROVIDERS\\.+\\.+\\base\\MetadataProviderScheme=.+:METADATA_PROVIDERS\\.+\\.+\\base\\OrganisationUnitScheme=.+`

OrganisationReferenceType: A reference type for any type of organisation.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\base\\Agency=.+:AGENCIES\\.+\\.+.+\\base\\DataConsumer=.+:DATA_CONSUMERS\\.+\\.+.+\\base\\DataProvider=.+:DATA_PROVIDERS\\.+\\.+.+\\base\\MetadataProvider=.+:METADATA_PROVIDERS\\.+\\.+.+\\base\\OrganisationUnit=.+`

ConstraintReferenceType: A reference for any type of constraint.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\registry\\DataConstraint=.+.+\\registry\\MetadataConstraint=.+`

AgencyReferenceType: A reference type for an agency.

Derived by restriction of `OrganisationReferenceType`.
`Regular Expression Pattern: .+\\base\\Agency=.+:AGENCIES\\.+\\.+`

AgencySchemeReferenceType: A reference type for an agency scheme.

Derived by restriction of `OrganisationSchemeReferenceType`.
Regular Expression Pattern: `.+\\base\\AgencyScheme=.+:AGENCIES\\.+\\`

DataConsumerReferenceType: A reference type for a data consumer.

Derived by restriction of `OrganisationReferenceType`.
Regular Expression Pattern: `.+\\base\\DataConsumer=.+:DATA_CONSUMERS\\.+\\.+`

DataConsumerSchemeReferenceType: A reference type for a data consumer scheme.

Derived by restriction of `OrganisationSchemeReferenceType`.
Regular Expression Pattern: `.+\\base\\DataConsumerScheme=.+:DATA_CONSUMERS\\.+\\`

DataProviderReferenceType: A reference type for a data provider.

Derived by restriction of `OrganisationReferenceType`.
Regular Expression Pattern: `.+\\base\\DataProvider=.+:DATA_PROVIDERS\\.+\\.+`

DataProviderSchemeReferenceType: A reference type for a data provider scheme.

Copyright © SDMx TWG 2020

Derived by restriction of `OrganisationSchemeReferenceType`.
Regular Expression Pattern: `.+\\base\\DataProviderScheme=.+:DATA_PROVIDERS\\.+\\`

MetadataProviderReferenceType: A reference type for a metadata provider.

Derived by restriction of `OrganisationReferenceType`.
Regular Expression Pattern: `.+\\base\\MetadataProvider=.+:METADATA_PROVIDERS\\.+\\.+`

MetadataProviderSchemeReferenceType: A reference type for a metadata provider scheme.

Derived by restriction of `OrganisationSchemeReferenceType`.
Regular Expression Pattern: `.+\\base\\MetadataProviderScheme=.+:METADATA_PROVIDERS\\.+\\`

OrganisationUnitReferenceType: A reference type for an organisation unit.

Derived by restriction of `OrganisationReferenceType`.
Regular Expression Pattern: `.+\\base\\OrganisationUnit=.+`

OrganisationUnitSchemeReferenceType: A reference type for an organisation unit scheme.

Derived by restriction of `OrganisationSchemeReferenceType`.
Regular Expression Pattern: `.+\\base\\OrganisationUnitScheme=.+`

CategorisationReferenceType: A reference type for a categorisation.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\categoryscheme\\Categorisation=.+`

CategoryReferenceType: A reference type for a category.

Derived by restriction of `UrnReferenceType`.
Regular Expression Pattern: `.+\\categoryscheme\\Category=.+`

CategorySchemeReferenceType: A reference type for a category scheme.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\categoryscheme\\CategoryScheme=.+`

ReportingCategoryReferenceType: A reference type for a reporting category.

Derived by restriction of `UrnReferenceType`.
Regular Expression Pattern: `.+\\categoryscheme\\ReportingCategory=.+`

ReportingTaxonomyReferenceType: A reference type for a reporting taxonomy.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\categoryscheme\\ReportingTaxonomy=.+`

CodeReferenceType: A reference type for a code.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: .+\codelist\Code=.+

CodelistReferenceType: A reference type for a codelist.

Derived by restriction of `AnyCodelistReferenceType`.
Regular Expression Pattern: `.+\\codelist\\Codelist=.+`

HierarchicalCodeReferenceType: A reference type for a hierarchical code.

Derived by restriction of `UrnReferenceType`.
Regular Expression Pattern: `.+\\codelist\\HierarchicalCode=.+`

HierarchyReferenceType: A reference type for a hierarchy.

Copyright © SDMx TWG 2020

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\codelist\\Hierarchy=.+`

HierarchyAssociationReferenceType: A reference type for a hierarchy association.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\codelist\\HierarchyAssociation=.+`

LevelReferenceType: A reference type for a level.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\codelist\\Level=.+`

ValueListReferenceType: A reference type for a value list.

Derived by restriction of `AnyCodelistReferenceType`.
Regular Expression Pattern: `.+\\codelist\\ValueList=.+`

ConceptReferenceType: A reference type for a concept.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\conceptscheme\\Concept=.+`

ConceptSchemeReferenceType: A reference type for a concept scheme.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\conceptscheme\\ConceptScheme=.+`

AttributeDescriptorReferenceType: A reference type for an attribute descriptor.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\datastructure\\AttributeDescriptor=.+`

DataAttributeReferenceType: A reference type for a data attribute.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\datastructure\\DataAttribute=.+`

DataflowReferenceType: A reference type for a dataflow.

Derived by restriction of `StructureUsageReferenceType`.
Regular Expression Pattern: `.+\\datastructure\\Dataflow=.+`

DataStructureReferenceType: A reference type for a data structure.

Derived by restriction of `StructureReferenceType`.
Regular Expression Pattern: `.+\\datastructure\\DataStructure=.+`

DimensionReferenceType: A reference type for a dimension.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\datastructure\\Dimension=.+`

DimensionDescriptorReferenceType: A reference type for a dimension descriptor.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\datastructure\\DimensionDescriptor=.+`

GroupDimensionDescriptorReferenceType: A reference type for a group dimension descriptor.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\datastructure\\GroupDimensionDescriptor=.+`

MeasureReferenceType: A reference type for a measure.

Copyright © SDMx TWG 2020

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\datastructure\\Measure=.+`

MeasureDescriptorReferenceType: A reference type for a measure descriptor.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\datastructure\\MeasureDescriptor=.+`

TimeDimensionReferenceType: A reference type for a time dimension.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\datastructure\\TimeDimension=.+`

MetadataAttributeReferenceType: A reference type for a metadata attribute.

Derived by restriction of `UrnReferenceType`.
Regular Expression Pattern: `.+\\metadatastructure\\MetadataAttribute=.+`

MetadataflowReferenceType: A reference type for a metadataflow.

Derived by restriction of `StructureUsageReferenceType`.
Regular Expression Pattern: `.+\\metadatastructure\\Metadataflow=.+`

MetadataSetReferenceType: A reference type for a metadata set.

Derived by restriction of `StructureReferenceType`.
Regular Expression Pattern: `.+\\metadatastructure\\MetadataSet=.+`

MetadataStructureReferenceType: A reference type for a metadata structure.

Derived by restriction of `StructureReferenceType`.
Regular Expression Pattern: `.+\\metadatastructure\\MetadataStructure=.+`

ProcessReferenceType: A reference type for a process.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\process\\Process=.+`

ProcessStepReferenceType: A reference type for a process step.

Derived by restriction of `UrnReferenceType`.
Regular Expression Pattern: `.+\\process\\ProcessStep=.+`

TransitionReferenceType: A reference type for a transition.

Derived by restriction of `UrnReferenceType`.
Regular Expression Pattern: `.+\\process\\Transition=.+`

DataConstraintReferenceType: A reference type for a data constraint.

Derived by restriction of `ConstraintReferenceType`.
Regular Expression Pattern: `.+\\registry\\DataConstraint=.+`

MetadataConstraintReferenceType: A reference type for a metadata constraint.

Derived by restriction of `ConstraintReferenceType`.
Regular Expression Pattern: `.+\\registry\\MetadataConstraint=.+`

MetadataProvisionAgreementReferenceType: A reference type for a metadata provision agreement.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\registry\\MetadataProvisionAgreement=.+`

ProvisionAgreementReferenceType: A reference type for a provision agreement.

Copyright © SDMx TWG 2020

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\registry\\ProvisionAgreement=.+`

CategorySchemeMapReferenceType: A reference type for a category scheme map.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\structuremapping\\CategorySchemeMap=.+`

ConceptSchemeMapReferenceType: A reference type for a concept scheme map.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\structuremapping\\ConceptSchemeMap=.+`

DatePatternMapReferenceType: A reference type for a date pattern map.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\structuremapping\\DatePatternMap=.+`

EpochMapReferenceType: A reference type for an epoch map.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\structuremapping\\EpochMap=.+`

FrequencyFormatMappingReferenceType: A reference type for a frequency format mapping.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\structuremapping\\FrequencyFormatMapping=.+`

OrganisationSchemeMapReferenceType: A reference type for a organisation scheme map.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\structuremapping\\OrganisationSchemeMap=.+`

ReportingTaxonomyMapReferenceType: A reference type for a reporting taxonomy map.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\structuremapping\\ReportingTaxonomyMap=.+`

RepresentationMapReferenceType: A reference type for a representation map.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\structuremapping\\RepresentationMap=.+`

StructureMapReferenceType: A reference type for a structure map.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\structuremapping\\StructureMap=.+`

CustomTypeReferenceType: A reference type for a custom type.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\transformation\\CustomType=.+`

CustomTypeSchemeReferenceType: A reference type for a custom type scheme.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\transformation\\CustomTypeScheme=.+`

NamePersonalisationReferenceType: A reference type for a name personalisation.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\transformation\\NamePersonalisation=.+`

NamePersonalisationSchemeReferenceType: A reference type for a name personalisation scheme.

Copyright © SDMx TWG 2020

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\transformation\\NamePersonalisationScheme=.+`

RulesetReferenceType: A reference type for a ruleset.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\transformation\\Ruleset=.+`

RulesetSchemeReferenceType: A reference type for a ruleset scheme.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\transformation\\RulesetScheme=.+`

TransformationReferenceType: A reference type for a transformation.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\transformation\\Transformation=.+`

TransformationSchemeReferenceType: A reference type for a transformation scheme.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\transformation\\TransformationScheme=.+`

UserDefinedOperatorReferenceType: A reference type for a user defined operator.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\transformation\\UserDefinedOperator=.+`

UserDefinedOperatorSchemeReferenceType: A reference type for a user defined operator scheme.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\transformation\\UserDefinedOperatorScheme=.+`

VtlMappingReferenceType: A reference type for a VTL mapping.

Derived by restriction of `ComponentUrnReferenceType`.
Regular Expression Pattern: `.+\\transformation\\VtlCodelistMapping=.+.+\\transformation\\VtlConceptMapping=.+.+\\transformation\\VtlDataflowMapping=.+`

VtlMappingSchemeReferenceType: A reference type for a VTL mapping scheme.

Derived by restriction of `MaintainableUrnReferenceType`.
Regular Expression Pattern: `.+\\transformation\\VtlMappingScheme=.+`

VersionReferenceType: VersionReferenceType defines the structure of version number in a reference. When semantic versioning is used, the major, minor, or patch version parts can be wildcarded using “+” as an extension. For example, 2+.3.1 means the currently latest available version >= 2.3.1 (even if not backwards compatible). Note that wildcarded semantic version references cannot be combined with version extended reference (e.g. 2.3+.1-draft is not permissible). Version-extended (e.g. 1.3.1-draft) and legacy version numbers (e.g. 1 or 1.0) are also supported.

Union of:

LegacyVersionNumberType, SemanticVersionNumberType, SemanticVersionReferenceType.

SemanticVersionReferenceType: SemanticVersionReferenceType is a simple type for referencing semantic version numbers. It allows for the wildcarding of only one the major, minor, or patch version parts using “+”.

Derived by restriction of `xs:string`.
Regular Expression Pattern: `((0|[1-9]\d*)\\?)(\\((0|[1-9]\d*))){2}((0|[1-9]\d*))(\\((0|[1-9]\d*)\\?))(\\((0|[1-9]\d*)))((0|[1-9]\d*)\\?)(\\((0|[1-9]\d*)))(\\((0|[1-9]\d*)\\?))`

WildcardVersionType: WildcardVersionType combines the VersionType and WildcardType to allow a reference to either a specific version of an object, or to wildcard the version in the reference by specifying the ‘*’ value.

Union of:

Copyright © SDMx TWG 2020

LegacyVersionNumberType, SemanticVersionNumberType, SemanticVersionReferenceType, WildcardType.

WildcardType: WildcardType is a single value code list, used to include the '*' character - indicating that the identification component is wildcarded.

Derived by restriction of `xs:string`.

Enumerations:

Value	Documentation
*	Indicates that any value of the identifier component is allowed.

NestedIDType: NestedIDType is the least restrictive form of an identifier used throughout all SDMX-ML messages. It allows for a hierarchical identifier, with each portion separated by the '.' character. For the identifier portions, valid characters include A-Z, a-z, @, 0-9, _, -, \$.

Derived by restriction of `xs:string`.
Regular Expression Pattern: `[A-Za-z0-9\_@\$\\]+(\\[A-Za-z0-9\_@\$\\]+)*`

TwoLevelIDType: TwoLevelIDType defines an identifier with exactly two levels.

Derived by restriction of `NestedIDType`.
Regular Expression Pattern: `[A-Za-z0-9\_@\$\\]+\\[A-Za-z0-9\_@\$\\]+`

IDType: IDType provides a type which is used for restricting the characters in codes and IDs throughout all SDMX-ML messages. Valid characters include A-Z, a-z, @, 0-9, _, -, \$.

Derived by restriction of `NestedIDType`.
Regular Expression Pattern: `[A-Za-z0-9\_@\$\\]+`

NCNameIDType: NCNameIDType restricts the IDType, so that the id may be used to generate valid XML components. IDs created from this type conform to the W3C XML Schema NCNAME type, and therefore can be used as element or attribute names.

Derived by restriction of `IDType`.
Regular Expression Pattern: `[A-Za-z][A-Za-z0-9\_\\]*`

NestedNCNameIDType: NestedNCNameIDType restricts the NestedIDType, so that the id may be used to generate valid XML components. IDs created from this type conform to the W3C XML Schema NCNAME type, and therefore can be used as element or attribute names.

Derived by restriction of `NestedIDType`.
Regular Expression Pattern: `[A-Za-z][A-Za-z0-9\_\\]*(\\[A-Za-z][A-Za-z0-9\_\\]*)*`

SingleNCNameIDType: SingleNCNameIDType restricts the NestedNCNameIDType to allow only one level. Note that this is the same pattern as the NCNameIDType, but can be used when the base type to be restricted is a nested NCNameIDType (where as the NCNameIDType could only restrict the IDType).

Derived by restriction of `NestedNCNameIDType`.
Regular Expression Pattern: `[A-Za-z][A-Za-z0-9\_\\]*`

VersionType: VersionType is used to communicate version information. Semantic versioning, based on 3 or 4 version parts (major.minor.patch[-extension]) is supported. The legacy SDMX version format is also support.

Union of:

`LegacyVersionNumberType, SemanticVersionNumberType.`

SemanticVersionNumberType: SemanticVersionNumberType is a simple type for validating semantic version in the format (major.minor.patch[-extension]).

Derived by restriction of `xs:string`.
Regular Expression Pattern: `(0|[1-9]\d*)(\\(0|[1-9]\d*)){2}(\\(([A-Za-z\\]|([A-Za-z\\][A-Za-z0-9\\]+)|([A-Za-z0-9\\]+[A-Za-z\\]`

Copyright © SDMx TWG 2020

`[A-Za-z0-9\\]*))|(0|[1-9][0-9]*))(\\(([A-Za-z\\]|([A-Za-z\\][A-Za-z0-9\\]+)|([A-Za-z0-9\\]+[A-Za-z\\][A-Za-z0-9\\]*))|(0|[1-9][0-9]*)))*)?`

LegacyVersionNumberType: LegacyVersionNumberType describes the version number format previously supported in SDMX. The format is restricted to allow for simple incrementing and sorting of version number. The version consists of a set of maximum 2 numeric components, separated by the '.' character. When processing version, each numeric component (the number preceding and following any '.' character) should be parsed as an integer. Thus, a version of 1.3 and 1.03 would be equivalent, as both the '3' component and the '03' component would parse to an integer value of 3.

Derived by restriction of `xs:string`.
Regular Expression Pattern: `(0|[1-9]\d*)(\\(0|[1-9]\d*))?`

Copyright © SDMx TWG 2020

7.1.4 Structure Namespace

Introduction

The structure namespace contains the definition of all structural metadata constructs. These constructs are intended to be very tightly coupled with the information model to ease the burden of implementers on translating the information from the XML messages into objects based on the information model.

The conformance with the information was achieved through derivation by extensions, restrictions, and substitutions. Because of some the limitations of XML Schema in these areas, it was often necessary to create intermediate type which formed the basis of the final types which make up the information that is actually exchanged in SDMX messages. The intermediate types are all abstract, so they are not explicitly used in a message. They do however serve the purpose of creating a strong relation of the schemas to the information model.

Schema Documentation

STRUCTURE NAMESPACE

http://www.sdmx.org/resources/sdmxml/schemas/v3_0/structure

Summary

Referenced Namespaces:

Namespace	Prefix
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/common	common
http://www.w3.org/2001/XMLSchema	xs

Contents:

38 Global Elements
235 Complex Types
19 Simple Types

Global Elements

Structures (StructuresType): Structures contains constructs for all structural metadata components.

Item (ItemType): Item is an abstract element that serves as a substitution head for all items in an item scheme, including those items nested within other items. Concrete instances of this must use a concrete instance of ItemType.

Substitutions: Category, Code, *GeoRefCode*, GeoFeatureSetCode, GeoGridCode, Concept, *Organisation*, Agency, DataConsumer, DataProvider, MetadataProvider, OrganisationUnit, ReportingCategory, Transformation, VtlMapping, NamePersonalisation, Ruleset, UserDefinedOperator, CustomType

Grouping (GroupingType): Grouping is an abstract element that serves as a substitution head for all structure groupings. Groupings contain a collection of component lists for a structure. Concrete instances of this must use a concrete instance of GroupingType.

Substitutions: DataStructureComponents, MetadataStructureComponents

ComponentList (ComponentListType): ComponentList is an abstract element that serves as a substitution head for all component lists. Concrete instances of this must use a concrete instance of ComponentListType.

Substitutions: AttributeList, DimensionList, Group, MeasureList, MetadataAttributeList

Component (ComponentType): Component is an abstract element that serves as a substitution head for all components. Concrete instances of this must use a concrete instance of ComponentType.

Substitutions: Attribute, MetadataAttributeUsage, Dimension, TimeDimension, GroupDimension, Measure, MetadataAttribute

Copyright © SDMx TWG 2020

Category (CategoryType): Category represents a set of nested categories which describe a simple classification hierarchy.

Substitution For: *Item*

Code (CodeType): Code describes a code in a codelist. In addition to the identification and description of the code, basic presentational information is also available. Presentational information not present may be added through the use of annotations.

Substitution For: *Item*

Substitutions: *GeoRefCode*, GeoFeatureSetCode, GeoGridCode

GeoRefCode (GeoRefCodeType): GeoRefCode is the abstract base from which specific types of geographic codes will be derived.

Substitution For: Code

Substitutions: GeoFeatureSetCode, GeoGridCode

GeoFeatureSetCode (GeoFeatureSetCodeType): Is a geographic code in a geographic codelist. It adds a value to a code that folows a pattern to represent a geo feature set.

Substitution For: *GeoRefCode*

GeoGridCode (GeoGridCodeType): GeoGridCode is a code the represents a geographic grid cell that belongs to a specific grid definition.

Substitution For: *GeoRefCode*

Concept (ConceptType): Concept describes the details of a concept within a concept scheme.

Substitution For: *Item*

DataStructureComponents (DataStructureComponentsType): DataStructureComponents defines the grouping of the sets of metadata concepts that have a defined structural role in the data structure definition. Note that for any component or group defined in a data structure definition, its id must be unique. This applies to the identifiers explicitly defined by the components as well as those inherited from the concept identity of a component. For example, if two dimensions take their identity from concepts with same identity (regardless of whether the concepts exist in different schemes) one of the dimensions must be provided a different explicit identifier. Although there are XML schema constraints to help enforce this, these only apply to explicitly assigned identifiers. Identifiers inherited from a concept from which a component takes its identity cannot be validated against this constraint. Therefore, systems processing data structure definitions will have to perform this check outside of the XML validation. There are also two reserved identifiers in a data structure definition; TIME_PERIOD, and REPORTING_YEAR_START_DAY. These identifiers may not be used outside of their respective defintions (TimeDimension and ReportingYearStartDay). This applies to both the explicit identifiers that can be assigned to the components or groups as well as an identifier inherited by a component from its concept identity. For example, if an ordinary dimension (i.e. not the time dimension) takes its concept identity from a concept with the identifier TIME_PERIOD, that dimension must provide a different explicit identifier.

Substitution For: *Grouping*

AttributeList (AttributeListType): AttributeList describes the attribute descriptor for the data structure definition. It is a collection of metadata concepts that define the attributes of the data structure definition.

Substitution For: *ComponentList*

Attribute (AttributeType): Attribute describes the definition of a data attribute, which is defined as a characteristic of an object or entity.

Substitution For: *Component*

MetadataAttributeUsage (MetadataAttributeUsageType): MetadataAttributeUsage refines the details of how a metadata attribute from the metadata structure referenced from the data structure is used. By default, metadata attributes can be expressed at any level of the data. This allows an attribute relationship to be defined in order restrict the reporing of a metadata attribute to a specific part of the data.

Substitution For: *Component*

Copyright © SDMx TWG 2020

DimensionList (DimensionListType): DimensionList describes the key descriptor for the data structure definition. It is an ordered set of metadata concepts that, combined, classify a statistical series, such as a time series, and whose values, when combined (the key) in an instance such as a data set, uniquely identify a specific series.

Substitution For: *ComponentList*

Dimension (DimensionType): Dimension describes the structure of a dimension, which is defined as a statistical concept used (most probably together with other statistical concepts) to identify a statistical series, such as a time series, e.g. a statistical concept indicating certain economic activity or a geographical reference area.

Substitution For: *Component*

TimeDimension (TimeDimensionType): TimeDimension is a special dimension which designates the period in time in which the data identified by the full series key applies.

Substitution For: *Component*

Group (GroupType): Group describes a group descriptor in a data structure definition. It is a set metadata concepts (and possibly their values) that define a partial key derived from the key descriptor in a data structure definition.

Substitution For: *ComponentList*

GroupDimension (GroupDimensionType): GroupDimension is a component which contains only a reference to a dimension in the key descriptor (DimensionList). Although it is conventional to declare dimensions in the same order as they are declared in the ordered key, there is no requirement to do so - the ordering of the values of the key are taken from the order in which the dimensions are declared. Note that the id of a dimension may be inherited from its underlying concept - therefore this reference value may actually be the id of the concept.

Substitution For: *Component*

MeasureList (MeasureListType): MeasureList describes the measure descriptor for a data structure.

Substitution For: *ComponentList*

Measure (MeasureType): Measure defines the structure of a measure, which is the concept that is the value of the phenomenon to be measured in a data set.

Substitution For: *Component*

MetadataStructureComponents (MetadataStructureComponentsType): MetadataStructureComponents defines the grouping of the sets of the components that make up the metadata structure definition.

Substitution For: *Grouping*

MetadataAttributeList (MetadataAttributeListType): MetadataAttributeList defines the set of metadata attributes that can be defined as a hierarchy, for reporting reference metadata about a target object. The identification of metadata attributes must be unique at any given level of the metadata structure. Although there are XML schema constraints to help enforce this, these only apply to explicitly assigned identifiers. Identifiers inherited from a concept from which a metadata attribute takes its identity cannot be validated against this constraint. Therefore, systems processing metadata structure definitions will have to perform this check outside of the XML validation.

Substitution For: *ComponentList*

MetadataAttribute (MetadataAttributeType): MetadataAttribute defines the a metadata attribute, which is the value of an attribute, such as the instance of a coded or uncoded attribute in a metadata structure definition.

Substitution For: *Component*

Organisation (OrganisationType): Organisation is an abstract substitution head for a generic organisation.

Substitution For: *Item*

Substitutions: Agency, DataConsumer, DataProvider, MetadataProvider, OrganisationUnit

Copyright © SDMx TWG 2020

Agency (AgencyType): Agency is an organisation which maintains structural metadata such as classifications, concepts, data structures, and metadata structures.

Substitution For: *Organisation*

DataConsumer (DataConsumerType): DataConsumer describes an organisation using data as input for further processing.

Substitution For: *Organisation*

DataProvider (DataProviderType): DataProvider describes an organisation that produces data.

Substitution For: *Organisation*

MetadataProvider (MetadataProviderType): MetadataProvider describes an organisation that produces metadata .

Substitution For: *Organisation*

OrganisationUnit (OrganisationUnitType): OrganisationUnit describes a generic organisation, which serves not predefined role in SDMX.

Substitution For: *Organisation*

ReportingCategory (ReportingCategoryType): ReportingCateogry defines a reporting category, which is used to group structure usages into useful sub-packages.

Substitution For: *Item*

Transformation (TransformationType): Transformation describes the details of a single transformation within a transformation scheme.

Substitution For: *Item*

VtlMapping (VtlMappingType): VtlMapping details a mapping between SDMX and VTL transformation.

Substitution For: *Item*

NamePersonalisation (NamePersonalisationType): NamePersonalisation details a name personalisation that is used in a transformation.

Substitution For: *Item*

Ruleset (RulesetType): Ruleset details a ruleset within a ruleset scheme.

Substitution For: *Item*

UserDefinedOperator (UserDefinedOperatorType): UserDefinedOperator details a user defined operators within a user defined operator scheme.

Substitution For: *Item*

CustomType (CustomTypeType): CustomType details a custom type within a custom type scheme.

Substitution For: *Item*

Complex Types

StructuresType: StructuresType describes the structure of the container for all structural metadata components. The structural components may be explicitly detailed, or referenced from an external structure document or registry service. Best practices dictate that, at a minimum, any structural component that is referenced by another structural component be included by reference.

Content:

```
ALL(AgencySchemesCategorisationsCategorySchemeMapsCategorySchemesCodelistsConceptSchemeMapsConceptSchemesCustomTypeSchemesDataConstraintsDataConsumerSchemesDa
taflowsDataProviderSchemesDataStructuresGeographicCodelistsGeoGridCodelistsHierarchiesHierarchyAssociationsMetadataConstraintsMetadataflowsMetadataProviderSch
emesMetadataProvisionAgreementsMetadataStructuresNamePersonalisationSchemesOrganisationSchemeMapsOrganisationUnitSchemesProcessesProvisionAgreementsReportingT
axonomiesReportingTaxonomyMapsRepresentationMapsRulesetSchemesStructureMapsTransformationSchemesUserDefinedOperatorSchemesValueListsVtlMappingSchemes)
```


Copyright © SDMx TWG 2020

Element Documentation:

Copyright © SDMx TWG 2020

Name	Type	Documentation
AgencySchemes	AgencySchemesType	AgencySchemes contains a collection of agency scheme descriptions. The agency schemes may be detailed in full, or referenced from an external structure document or registry service.
Categorisations	CategorisationsType	Categorisations contains a collection of structural object categorisations. This container may contain categorisations for many types of objects. The categorisations may be detailed in full, or referenced from an external structure document or registry service.
CategorySchemeMaps	CategorySchemeMapsType	CategorySchemeMaps contains a collection of category scheme map descriptions. The category scheme maps may be detailed in full, or referenced from an external structure document or registry service.
CategorySchemes	CategorySchemesType	CategorySchemes contains a collection of category scheme descriptions. The category schemes may be detailed in full, or referenced from an external structure document or registry service.
Codelists	CodelistsType	Codelists contains a collection of code list descriptions. The code lists may be detailed in full, or referenced from an external structure document or registry service.
ConceptSchemeMaps	ConceptSchemeMapsType	ConceptSchemeMaps contains a collection of concept scheme map descriptions. The concept scheme maps may be detailed in full, or referenced from an external structure document or registry service.
ConceptSchemes	ConceptSchemesType	ConceptSchemes contains a collection of concept scheme descriptions. The concept schemes described are contained within schemes. The concepts may be detailed in full, or referenced from an external structure document or registry service.
CustomTypeSchemes	CustomTypeSchemesType	CustomTypeSchemes contains a collection of custom type schemes. The scheme may be detailed in full, or referenced from an external structure document or registry service.
DataConstraints	DataConstraintsType	DataConstraints contains a collection of data constraint descriptions. The constraints may be detailed in full, or referenced from an external structure document or registry service.
DataConsumerSchemes	DataConsumerSchemesType	DataConsumerSchemes contains a collection of data consumer scheme descriptions. The data consumer schemes may be detailed in full, or referenced from an external structure document or registry service.
Dataflows	DataflowsType	Dataflows contains a collection of data flow descriptions. The data flows may be detailed in full, or referenced from an external structure document or registry service.
DataProviderSchemes	DataProviderSchemesType	DataProviderSchemes contains a collection of data provider scheme descriptions. The data provider schemes may be detailed in full, or referenced from an external structure document or registry service.
DataStructures	DataStructuresType	DataStructures contains a collection of data structure definitions. The data structure definitions may be detailed in

Copyright © SDMx TWG 2020

Name	Type	Documentation
		full, or referenced from an external structure document or registry service.
GeographicCodelists	GeographicCodelistsT ype	GeographicCodelists contains a collection of geographi codelist descriptions. The codelists may be detailed in full, or referenced from an external structure document or registry service.
GeoGridCodelists	GeoGridCodelistsType	GeoGridCodelists contains a collection of geographic grid codelist descriptions. The codelists may be detailed in full, or referenced from an external structure document or registry service.
Hierarchies	HierarchiesType	Hierarchies contains a collection of hierarchical code list descriptions. The hierarchical code lists may be detailed in full, or referenced from an external structure document or registry service.
HierarchyAssociation s	HierarchyAssociation sType	HierarchyAssociations contains a collection of hierarchy associations. The associations may be detailed in full, or referenced from an external structure document or registry service.
MetadataConstraints	MetadataConstraintsT ype	MetadataConstraints contains a collection of metadata constraint descriptions. The constraints may be detailed in full, or referenced from an external structure document or registry service.
Metadataflows	MetadataflowsType	Metadataflows contains a collection of metadata flow descriptions. The metadata flows may be detailed in full, or referenced from an external structure document or registry service.
MetadataProviderSche mes	MetadataProviderSche mesType	MetadataProviderSchemes contains a collection of metadata provider scheme descriptions. The meatadata provider schemes may be detailed in full, or referenced from an external structure document or registry service.
MetadataProvisionAgr eements	MetadataProvisionAgr eementsType	ProvisionAgreements contains a collection of provision agreements. The provision agreements may be detailed in full, or referenced from an external structure document or registry service.
MetadataStructures	MetadataStructuresTy pe	MetadataStructures contains a collection of metadata structure definition descriptions. The metadata structure definitions may be detailed in full, or referenced from an external structure document or registry service.
NamePersonalisationS chemes	NamePersonalisationS chemesType	NamePersonalisationSchemes contains a collection of name personalisation schemes. The scheme may be detailed in full, or referenced from an external structure document or registry service.
OrganisationSchemeMa ps	OrganisationSchemeMa psType	OrganisationSchemeMaps contains a collection of organisation scheme map descriptions. The organisation scheme maps may be detailed in full, or referenced from an external structure document or registry service.
OrganisationUnitSche mes	OrganisationUnitSche mesType	OrganisationUnitSchemes contains a collection of organisation unit scheme descriptions. The organisation unit schemes may be detailed in full, or referenced from an external structure document or registry service.

Copyright © SDMx TWG 2020

Name	Type	Documentation
Processes	ProcessesType	Processes contains a collection of process descriptions. The processes may be detailed in full, or referenced from an external structure document or registry service.
ProvisionAgreements	ProvisionAgreementsT ype	ProvisionAgreements contains a collection of provision agreements. The provision agreements may be detailed in full, or referenced from an external structure document or registry service.
ReportingTaxonomies	ReportingTaxonomiesT ype	ReportingTaxonomies contains a collection of reporting taxonomy descriptions. The reporting taxonomies may be detailed in full, or referenced from an external structure document or registry service.
ReportingTaxonomyMap s	ReportingTaxonomyMap sType	ReportingTaxonomyMaps contains a collection of reporting taxonomy map descriptions. The reporting taxonomy maps may be detailed in full, or referenced from an external structure document or registry service.
RepresentationMaps	RepresentationMapsTy pe	RepresentationMaps contains a collection of representation map descriptions. The representation maps may be detailed in full, or referenced from an external structure document or registry service.
RulesetSchemes	RulesetSchemesType	RulesetSchemes contains a collection of ruleset schemes. The scheme may be detailed in full, or referenced from an external structure document or registry service.
StructureMaps	StructureMapsType	StructureMaps contains a collection of structure map descriptions. The structure maps may be detailed in full, or referenced from an external structure document or registry service.
TransformationScheme s	TransformationScheme sType	TransformationSchemes contains a collection of transformation schemes. The transformation schemes may be detailed in full, or referenced from an external structure document or registry service.
UserDefinedOperatorS chemes	UserDefinedOperatorS chemesType	UserDefinedOperatorSchemes contains a collection of user defined operator schemes. The scheme may be detailed in full, or referenced from an external structure document or registry service.
ValueLists	ValueListsType	ValueLists contains a collection of value list descriptions. The value lists may be detailed in full, or referenced from an external structure document or registry service.
VtlMappingSchemes	VtlMappingSchemesTyp e	VtlMappingSchemes contains a collection of VTL mapping schemes. The scheme may be detailed in full, or referenced from an external structure document or registry service.

AgencySchemesType: AgencySchemesType describes the structure of the agency schemes container. It contains one or more agency scheme, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
AgencyScheme+
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
AgencyScheme	AgencySchemeType	AgencyScheme provides the details of an agency scheme, in which agencies are described.

CategorisationsType: CategorisationsType describes the structure of the categorisations container. It contains one or more categorisation of a specific object type, which can be explicitly detailed or referenced from an external structure document or registry service. This container may contain categorisations for multiple types of structural objects.

Content:

```
Categorisation+
```

Element Documentation:

Name	Type	Documentation
Categorisation	CategorisationType	Categorisation allows for the association of an identifiable object to a category, providing for the classifications of the reference identifiable object. This must either contain the full details of the categorisation, or provide a name and identification information and reference the full details from an external structure document or registry service.

CategorySchemeMapsType: CategorySchemeMapsType describes the structure of the category scheme maps container. It contains one or more category scheme map, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
CategorySchemeMap+
```

Element Documentation:

Name	Type	Documentation
CategorySchemeMap	CategorySchemeMapType	CategorySchemeMap provides the details of a category scheme map, which descibes mappings between categories in different schemes.

CategorySchemesType: CategorySchemesType describes the structure of the category schemes container. It contains one or more category scheme, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
CategoryScheme+
```

Element Documentation:

Name	Type	Documentation
CategoryScheme	CategorySchemeType	CategoryScheme provides the details of a category scheme, which is the descriptive information for an arrangement or division of categories into groups based on characteristics, which the objects have in common. This provides for a simple, leveled hierarchy or categories.

ConceptSchemeMapsType: ConceptSchemeMapsType describes the structure of the concept scheme maps container. It contains one or more concept scheme map, which can be explicitly detailed or referenced from an external structure document or registry service.

Copyright © SDMx TWG 2020

Content:

```
ConceptSchemeMap+
```

Element Documentation:

Name	Type	Documentation
ConceptSchemeMap	ConceptSchemeMapType	ConceptSchemeMap provides the details of a concept scheme map, which descibes mappings between concepts in different schemes.

ConceptSchemesType: ConceptSchemesType describes the structure of the concept schemes container. It contains one or more concept schemes, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
ConceptScheme*
```

Element Documentation:

Name	Type	Documentation
ConceptScheme	ConceptSchemeType	ConceptScheme provides the details of a concept scheme, which is the descriptive information for an arrangement or division of concepts into groups based on characteristics, which the objects have in common. It contains a collection of concept definitions, that may be arranged in simple hierarchies.

CodelistsType: CodelistsType describes the structure of the codelists container. It contains one or more codelist, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
Codelist+
```

Element Documentation:

Name	Type	Documentation
Codelist	CodelistType	Codelist provides the details of a codelist, which is defined as a list from which some statistical concepts (coded concepts) take their values.

CustomTypeSchemesType: CustomTypeSchemesType describes the structure of the custom type schemes container. It contains one or more custom type scheme, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
CustomTypeScheme+
```

Element Documentation:

Name	Type	Documentation
CustomTypeScheme	CustomTypeSchemeType	CustomTypeScheme provides the details of a custom type scheme, in which user defined operators are described.

DataConstraintsType: DataConstraintsType describes the structure of the data constraints container. It contains one or more data constraint, which can be explicitly detailed or referenced from an external structure document or registry service. This container may contain both attachment and content constraints.

Copyright © SDMx TWG 2020

Content:

```
DataConstraint+
```

Element Documentation:

Name	Type	Documentation
DataConstraint	DataConstraintType	DataConstraint specifies a sub set of the definition of the allowable or available content of a data set in terms of the content or in terms of the set of key combinations.

DataConsumerSchemesType: DataConsumerSchemesType describes the structure of the data consumer schemes container. It contains one or more data consumer scheme, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
DataConsumerScheme+
```

Element Documentation:

Name	Type	Documentation
DataConsumerScheme	DataConsumerSchemeType	DataConsumerScheme provides the details of an data consumer scheme, in which data consumers are described.

DataflowsType: DataflowsType describes the structure of the data flows container. It contains one or more data flow, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
Dataflow+
```

Element Documentation:

Name	Type	Documentation
Dataflow	DataflowType	Dataflow provides the details of a data flow, which is defined as the structure of data that will be provided for different reference periods.

DataProviderSchemesType: DataProviderSchemesType describes the structure of the data provider schemes container. It contains one or more data provider scheme, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
DataProviderScheme+
```

Element Documentation:

Name	Type	Documentation
DataProviderScheme	DataProviderSchemeType	DataProviderScheme provides the details of an data provider scheme, in which data providers are described.

DataStructuresType: DataStructuresType describes the structure of the data structure definitions container. It contains one or more data structure definition, which can be explicitly detailed or referenced from an external structure document or registry service.

Copyright © SDMx TWG 2020

Content:

```
DataStructure+
```

Element Documentation:

Name	Type	Documentation
DataStructure	DataStructureType	DataStructure provides the details of a data structure definition, which is defined as a collection of metadata concepts, their structure and usage when used to collect or disseminate data.

GeographicCodelistsType: GeographicCodelistsType describes the structure of the geographic code lists container. It contains one or more geographic codelist, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
GeographicCodelist+
```

Element Documentation:

Name	Type	Documentation
GeographicCodelist	GeographicCodelistType	GeographiCodelist provides the details of a geographic codelists container, which comprises a set of GeoFeatureSetCodes, by adding a value in the Code that follows a pattern to represent a geo feature set.

GeoGridCodelistsType: GeoGridCodelistsType describes the structure of the codelists container. It contains one or more geographic grid codelist, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
GeoGridCodelist+
```

Element Documentation:

Name	Type	Documentation
GeoGridCodelist	GeoGridCodelistType	GeoGridCodelist provides the details of a geographic grid code list, which comprises a set of GridCodes, which are related to the gridDefinition specified in the GeoGridCodelist.

HierarchiesType: HierarchiesType describes the structure of the hierarchies container. It contains one or more hierarchy, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
Hierarchy+
```

Element Documentation:

Name	Type	Documentation
Hierarchy	HierarchyType	Hierarchy provides the details of a hierarchy, which is defined as an organised collection of codes that may participate in many parent/child relationships with other codes in the list.

HierarchyAssociationsType: HiearchyAssociationsType describes the structure of the hierarchy assoications container. It contains one or more hiearchy associations, which can be explicitly detailed or referenced from an external structure document or registry service.

Copyright © SDMx TWG 2020

Content:

```
HierarchyAssociation+
```

Element Documentation:

Name	Type	Documentation
HierarchyAssociation	HierarchyAssociationType	HierarchyAssociation provides the details of a hiearchy assoication, which associates a hiearchy with an identifiable object in the context of another object.

MetadataConstraintsType: MetadataConstraintsType describes the structure of the metadata constraints container. It contains one or more metadata constraint, which can be explicitly detailed or referenced from an external structure document or registry service. This container may contain both attachment and content constraints.

Content:

```
MetadataConstraint+
```

Element Documentation:

Name	Type	Documentation
MetadataConstraint	MetadataConstraintType	MetadataConstraint specifies a sub set of the definition of the allowable content of a metadata set.

MetadataflowsType: MetadataflowsType describes the structure of the metadata flows container. It contains one or more metadata flow, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
Metadataflow+
```

Element Documentation:

Name	Type	Documentation
Metadataflow	MetadataflowType	Metadataflow provides the details of a metadata flow, which is defined as the structure of reference metadata that will be provided for different reference periods

MetadataProviderSchemesType: MetadataProviderSchemesType describes the structure of the metadata provider schemes container. It contains one or more metadata provider scheme, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
MetadataProviderScheme+
```

Element Documentation:

Name	Type	Documentation
MetadataProviderScheme	MetadataProviderSchemeType	MetadataProviderScheme provides the details of an metadata provider scheme, in which metadata providers are described.

MetadataProvisionAgreementsType: MetadataProvisionAgreementsType describes the structure of the metadata provision agreements container. It contains one or more metadata provision agreement, which can be explicitly detailed or referenced from an external structure document or registry service.

Copyright © SDMx TWG 2020

Content:

```
MetadataProvisionAgreement+
```

Element Documentation:

Name	Type	Documentation
MetadataProvisionAgreement	MetadataProvisionAgreementType	MetadataProvisionAgreement provides the details of a metadata provision agreement, which is an agreement for a metadata provider to report reference metadata against a flow.

MetadataStructuresType: MetadataStructuresType describes the structure of the metadata structure definitions container. It contains one or more metadata structure definition, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
MetadataStructure+
```

Element Documentation:

Name	Type	Documentation
MetadataStructure	MetadataStructureType	MetadataStructure provides the details of a metadata structure definition, which is defined as a collection of metadata concepts, their structure and usage when used to collect or disseminate reference metadata. A metadata structure definition performs several functions: it groups sets of objects into “targets” against which reference metadata may be reported. Targets define the structure of the reference metadata “keys” which identify specific types of reported metadata, and describe the valid values for populating the keys. Also, metadata structure definitions provide a presentational organization of concepts for reporting purposes. The structure of reference metadata is derived from this presentational structure.

NamePersonalisationSchemesType: NamePersonalisationSchemesType describes the structure of the name personalisation schemes container. It contains one or more name personalisation scheme, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
NamePersonalisationScheme+
```

Element Documentation:

Name	Type	Documentation
NamePersonalisationScheme	NamePersonalisationSchemeType	NamePersonalisationScheme provides the details of a name personalisation scheme, in which name personalisations are described.

OrganisationSchemeMapsType: OrganisationSchemeMapsType describes the structure of the organisation scheme maps container. It contains one or more organisation scheme map, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
OrganisationSchemeMap+
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
OrganisationSchemeMap	OrganisationSchemeMapType	OrganisationSchemeMap provides the details of a organisation scheme map, which descibes mappings between organisations in different schemes.

OrganisationUnitSchemesType: OrganisationUnitSchemesType describes the structure of the organisation unit schemes container. It contains one or more organisation unit scheme, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
OrganisationUnitScheme+
```

Element Documentation:

Name	Type	Documentation
OrganisationUnitScheme	OrganisationUnitSchemeType	OrganisationUnitScheme provides the details of an organisation unit scheme, in which organisation units are described.

ProcessesType: ProcessesType describes the structure of the processes container. It contains one or more process, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
Process+
```

Element Documentation:

Name	Type	Documentation
Process	ProcessType	Process provides the details of a process, which is a scheme which defines or documents the operations performed on data in order to validate data or to derive new information according to a given set of rules. It is not meant to support process automation, but serves as a description of how processes occur. The primary use for this structural mechanism is the attachment of reference metadata regarding statistical processing. This must either contain the full details of the category scheme, or provide a name and identification information and reference the full details from an external structure document or registry service.

ProvisionAgreementsType: ProvisionAgreementsType describes the structure of the provision agreements container. It contains one or more provision agreement, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
ProvisionAgreement+
```

Element Documentation:

Name	Type	Documentation
ProvisionAgreement	ProvisionAgreementType	ProvisionAgreement provides the details of a provision agreement, which is an agreement for a data provider to report data against a flow.

Copyright © SDMx TWG 2020

ReportingTaxonomiesType: ReportingTaxonomiesType describes the structure of the reporting taxonomies container. It contains one or more reporting taxonomy, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
ReportingTaxonomy+
```

Element Documentation:

Name	Type	Documentation
ReportingTaxonomy	ReportingTaxonomyType	ReportingTaxonomy provides the details of a reporting taxonomy, which is a scheme which defines the composition structure of a data report where each component can be described by an independent data or metadata flow definition.

ReportingTaxonomyMapsType: ReportingTaxonomyMapsType describes the structure of the reporting taxonomy maps container. It contains one or reporting taxonomy map, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
ReportingTaxonomyMap+
```

Element Documentation:

Name	Type	Documentation
ReportingTaxonomyMap	ReportingTaxonomyMapType	ReportingTaxonomyMap provides the details of a reporting taxonomy map, which descibes mappings between reporting taxonomies.

RepresentationMapsType: RepresentationMapsType describes the structure of the representation maps container. It contains one or more representation map, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
RepresentationMap+
```

Element Documentation:

Name	Type	Documentation
RepresentationMap	RepresentationMapType	RepresentationMap provides the details of a represenation map, which describes mappings between various component represenations.

RulesetSchemesType: RulesetSchemesType describes the structure of the ruleset schemes container. It contains one or more ruleset scheme, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
RulesetScheme+
```

Element Documentation:

Name	Type	Documentation
RulesetScheme	RulesetSchemeType	RulesetScheme provides the details of a ruleset scheme, in which rulesets are described.

Copyright © SDMx TWG 2020

StructureMapsType: StructureMapsType describes the structure of the structure maps container. It contains one or more structure maps, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
StructureMap+
```

Element Documentation:

Name	Type	Documentation
StructureMap	StructureMapType	StructureMap provides the details or a structure map, which describes mapping between data structures or dataflows.

TransformationSchemesType: TransformationSchemesType describes the structure of the transformations container. It contains one or more transformation schemes, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
TransformationScheme+
```

Element Documentation:

Name	Type	Documentation
TransformationScheme	TransformationSchemeType	TransformationScheme provides the details of a transformation scheme, in which transformations are described.

UserDefinedOperatorSchemesType: UserDefinedOperatorSchemesType describes the structure of the user defined operator schemes container. It contains one or more user defined operator scheme, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
UserDefinedOperatorScheme+
```

Element Documentation:

Name	Type	Documentation
UserDefinedOperatorScheme	UserDefinedOperatorSchemeType	UserDefinedOperatorScheme provides the details of a user defined operator scheme, in which user defined operators are described.

ValueListsType: ValueListsType describes the structure of the value lists container. It contains one or more value list, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
ValueList+
```

Element Documentation:

Name	Type	Documentation
ValueList	ValueListType	ValueList provides the details of a value list, which is a closed set of values that can occur for a dimension, measure, or attribute. This may be a simple list of values, or a list of values with names and descriptions (similar to a codelist).

Copyright © SDMx TWG 2020

VtlMappingSchemesType: VtlMappingSchemesType describes the structure of the VTL mappings schemes container. It contains one or more VTL mapping schemes, which can be explicitly detailed or referenced from an external structure document or registry service.

Content:

```
VtlMappingScheme+
```

Element Documentation:

Name	Type	Documentation
VtlMappingScheme	VtlMappingSchemeType	VtlMappingScheme provides the details of a VTL mapping scheme, in which VTL mappings are described.

ItemSchemeType: ItemSchemeType is an abstract base type for all item scheme objects. It contains a collection of items. Concrete instances of this type should restrict the actual types of items allowed within the scheme.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, Item*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MaintainableUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Item	*ItemType*	Item is an abstract element that serves as a substitution head for all items in an item scheme, including those items nested within other items. Concrete instances of this must use a concrete instance of ItemType.

ItemBaseType: ItemBaseType is an abstract base type that forms the basis for the ItemType. It requires that at least an id be supplied for an item.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

ItemType: ItemType is an abstract base type for all items with in an item scheme. Concrete instances of this type may or may not utilize the nested item, but if so should restrict the actual types of item allowed.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, (Parent | *Item+*)?
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Parent	NestedIDType	
Item	*ItemType*	Item is an abstract element that serves as a substitution head for all items in an item scheme, including those items nested within other items. Concrete instances of this must use a concrete instance of ItemType.

UnnestedItemType: UnnestedItemType is an abstract base type for all items with in an item scheme that do not contain nested items.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           UnnestedItemType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

StructureType: StructureType is an abstract base type for all structure objects. Concrete instances of this should restrict to a concrete grouping.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 StructureType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, *Grouping?*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MaintainableUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Grouping	*GroupingType*	Grouping is an abstract element that serves as a substitution head for all structure groupings. Groupings contain a collection of component lists for a structure. Concrete instances of this must use a concrete instance of GroupingType.

GroupingType: GroupType is an abstract base type for specific structure groupings. It contains a collection of component lists. Concrete instances of this should restrict to specific concrete component lists.

Content:

```
ComponentList*
```

Element Documentation:

Name	Type	Documentation
ComponentList	*ComponentListType*	ComponentList is an abstract element that serves as a substitution head for all component lists. Concrete instances of this must use a concrete instance of ComponentListType.

ComponentListType: ComponentListType is an abstract base type for all component lists. It contains a collection of components. Concrete types should restrict this to specific concrete components.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         ComponentListType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*, *Component**
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Component	*ComponentType*	Component is an abstract element that serves as a substitution head for all components. Concrete instances of this must use a concrete instance of ComponentType.

ComponentBaseType: ComponentBaseType is an abstract type that only serves the purpose of forming the base for the actual ComponentType. It only restricts the format of the id attribute to the NCNameIDType.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds an explicit identification of the component. If this identifier is not supplied, then it is assumed to be the same as the identifier of the concept referenced from the concept identity. Because structures require that every component be given a unique identifier, it may be necessary to assign an explicit identifier when more than one component in a structure reference concepts with same identifier. It is important to note that this applies strictly to the identifier of concept and not the URN. Therefore if two concepts with the same identifier from different concept schemes are referenced in the same structure, one of the components will have to provide a unique explicit identifier. The type of this identifier is restricted to the common:NCNameIDType. This is necessary, since component identifiers are used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.

ComponentType: ComponentType is an abstract base type for all components. It contains information pertaining to a component, including an optional reference to a concept, an optional role played by the concept, an optional text format description, and an optional local representation.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*, ConceptIdentity?, LocalRepresentation?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds an explicit identification of the component. If this identifier is not supplied, then it is assumed to be the same as the identifier of the concept referenced from the concept identity. Because structures require that every component be given a unique identifier, it may be necessary to assign an explicit identifier when more than one component in a structure reference concepts with same identifier. It is important to note that this applies strictly to the identifier of concept and not the URN. Therefore if two concepts with the same identifier from different concept schemes are referenced in the same structure, one of the components will have to provide a unique explicit identifier. The type of this identifier is restricted to the common:NCNameIDType. This is necessary, since component identifiers are used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
ConceptIdentity	ConceptReferenceType	ConceptIdentity allows for the referencing of a concept in a concept scheme. The component takes its semantic from this concept, and if an id is not specified, it takes its identification as well. If a representation (LocalRepresentation) is not supplied, then the representation of the component is also inherited from the concept. Note that in the case of the component representation being inherited from the concept, the allowable representations for the component still apply. Therefore, if a component references a concept with a core representation that is not allowed for the concept, that representation must be locally overridden. For components which can specify a concept role, it is implied that the concept which is referenced also identifies a role for the component.
LocalRepresentation	*RepresentationType*	LocalRepresentation references item schemes that may be used to create the representation of a component. The type of this must be refined such that a concrete item scheme reference is used.

StructureUsageType: StructureUsageType is an abstract base type for all structure usages. It contains a reference to a structure. Concrete instances of this type should restrict the type of structure referenced.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
```


Copyright © SDMx TWG 2020

```
                              MaintainableType (extension)
                                    StructureUsageType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, Structure?
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MaintainableUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Structure	StructureReferenceType	Structure references the structure (data structure or metadata structure definition) which the structure usage is based upon. Implementations will have to refine the type to use a concrete structure reference (i.e. either a data structure or metadata structure definition reference).

RepresentationType: RepresentationType is an abstract type that defines a representation. Because the type of item schemes that are allowed as the an enumeration vary based on the object in which this is defined, this type is abstract to force that the enumeration reference be restricted to the proper type of item scheme reference.

Attributes:

```
minOccurs?, maxOccurs?
```

Content:

```
(TextFormat | (Enumeration, EnumerationFormat?))
```

Attribute Documentation:

Name	Type	Documentation
minOccurs (default: 1)	xs: nonNegativeInteger	The minOccurs attribute indicates the minimum number of value that must be reported for the component.
maxOccurs	OccurenceType	The maxOccurs attribute indicates the maximum number of values that can be reported for the component.

Element Documentation:

Name	Type	Documentation
TextFormat	TextFormatType	TextFormat describes an uncoded textual format.
Enumeration	AnyCodelistReferenceType	Enumeration references an item scheme that enumerates the allowable values for this representation.
EnumerationFormat	CodedTextFormatType	EnumerationFormat describes the facets of the item scheme enumeration. This is for the most part, informational.

TextFormatType: TextFormatType defines the information for describing a full range of text formats and may place restrictions on the values of the other attributes, referred to as “facets”.

Attributes:

Copyright © SDMx TWG 2020

```
textType?, isSequence?, interval?, startValue?, endValue?,
timeInterval?, startTime?, endTime?, minLength?, maxLength?, minValue?,
maxValue?, decimals?, pattern?, isMultiLingual?
```

Content:

```
SentinelValue*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Copyright © SDMx TWG 2020

Name	Type	Documentation
textType (default: String)	DataType	The textType attribute provides a description of the datatype. If it is not specified, any valid characters may be included in the text field (it corresponds to the xs:string datatype of W3C XML Schema) within the constraints of the facets.
isSequence	xs:boolean	The isSequence attribute indicates whether the values are intended to be ordered, and it may work in combination with the interval, startValue, and endValue attributes or the timeInterval, startTime, and endTime, attributes. If this attribute holds a value of true, a start value or time and a numeric or time interval must supplied. If an end value is not given, then the sequence continues indefinitely.
interval	xs:decimal	The interval attribute specifies the permitted interval (increment) in a sequence. In order for this to be used, the isSequence attribute must have a value of true.
startValue	xs:decimal	The startValue attribute is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates the starting point of the sequence. This value is mandatory for a numeric sequence to be expressed.
endValue	xs:decimal	The endValue attribute is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates that ending point (if any) of the sequence.
timeInterval	xs:duration	The timeInterval attribute indicates the permitted duration in a time sequence. In order for this to be used, the isSequence attribute must have a value of true.
startTime	StandardTimePeriodType	The startTime attribute is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates the start time of the sequence. This value is mandatory for a time sequence to be expressed.
endTime	StandardTimePeriodType	The endTime attribute is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates that ending point (if any) of the sequence.
minLength	xs:positiveInteger	The minLength attribute specifies the minimum and length of the value in characters.
maxLength	xs:positiveInteger	The maxLength attribute specifies the maximum length of the value in characters.
minValue	xs:decimal	The minValue attribute is used for inclusive and exclusive ranges, indicating what the lower bound of the range is. If this is used with an inclusive range, a valid value will be greater than or equal to the value specified here. If the inclusive and exclusive data type is not specified (e.g. this facet is used with an integer data type), the value is assumed to be inclusive.
maxValue	xs:decimal	The maxValue attribute is used for inclusive and exclusive ranges, indicating what the upper bound of the range is. If this is used with an inclusive range, a valid value will be less than or equal to the value specified here. If the inclusive and exclusive data type is not specified (e.g. this facet is used with an integer data type), the value is assumed to be inclusive.

Copyright © SDMx TWG 2020

Name	Type	Documentation
decimals	xs:positiveInteger	The decimals attribute indicates the number of characters allowed after the decimal separator.
pattern	xs:string	The pattern attribute holds any regular expression permitted in the similar facet in W3C XML Schema.
isMultiLingual (default: true)	xs:boolean	The isMultiLingual attribute indicates for a text format of type “string”, whether the value should allow for multiple values in different languages.

Element Documentation:

Name	Type	Documentation
SentinelValue	SentinelValueType	SentinelValue defines a value that has a special meaning within the text format representation of a component.

SentinelValueType: SentinelValueType defines the structure of a sentinel value. A sentinel is a value that has a special meaning within the text format representation of a component. The value is associated with a multi-lingual name and description.

Attributes:

```
value
```

Content:

```
Name+, Description*
```

Attribute Documentation:

Name	Type	Documentation
value	xs:anySimpleType	The sentinel value being described.

Element Documentation:

Name	Type	Documentation
Name	TextType	Name is a reusable element, used for providing a human-readable name for an object.
Description	TextType	Description is a reusable element, used for providing a longer human-readable description of an object.

BasicComponentTextFormatType: BasicComponentTextFormatType is a restricted version of the TextFormatType that restricts the text type to the representations allowed for all components except for target objects.

Derivation:

```
TextFormatType (restriction)
   BasicComponentTextFormatType
```

Attributes:

```
textType?, isSequence?, interval?, startValue?, endValue?,
timeInterval?, startTime?, endTime?, minLength?, maxLength?, minValue?,
maxValue?, decimals?, pattern?, isMultiLingual?
```

Content:

```
SentinelValue*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Copyright © SDMx TWG 2020

Name	Type	Documentation
textType (default: String)	BasicComponentDataType	The textType attribute provides a description of the datatype. If it is not specified, any valid characters may be included in the text field (it corresponds to the xs:string datatype of W3C XML Schema) within the constraints of the facets.
isSequence	xs:boolean	The isSequence attribute indicates whether the values are intended to be ordered, and it may work in combination with the interval, startValue, and endValue attributes or the timeInterval, startTime, and endTime, attributes. If this attribute holds a value of true, a start value or time and a numeric or time interval must supplied. If an end value is not given, then the sequence continues indefinitely.
interval	xs:decimal	The interval attribute specifies the permitted interval (increment) in a sequence. In order for this to be used, the isSequence attribute must have a value of true.
startValue	xs:decimal	The startValue attribute is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates the starting point of the sequence. This value is mandatory for a numeric sequence to be expressed.
endValue	xs:decimal	The endValue attribute is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates that ending point (if any) of the sequence.
timeInterval	xs:duration	The timeInterval attribute indicates the permitted duration in a time sequence. In order for this to be used, the isSequence attribute must have a value of true.
startTime	StandardTimePeriodType	The startTime attribute is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates the start time of the sequence. This value is mandatory for a time sequence to be expressed.
endTime	StandardTimePeriodType	The endTime attribute is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates that ending point (if any) of the sequence.
minLength	xs:positiveInteger	The minLength attribute specifies the minimum and length of the value in characters.
maxLength	xs:positiveInteger	The maxLength attribute specifies the maximum length of the value in characters.
minValue	xs:decimal	The minValue attribute is used for inclusive and exclusive ranges, indicating what the lower bound of the range is. If this is used with an inclusive range, a valid value will be greater than or equal to the value specified here. If the inclusive and exclusive data type is not specified (e.g. this facet is used with an integer data type), the value is assumed to be inclusive.
maxValue	xs:decimal	The maxValue attribute is used for inclusive and exclusive ranges, indicating what the upper bound of the range is. If this is used with an inclusive range, a valid value will be less than or equal to the value specified here. If the inclusive and exclusive data type is not specified (e.g. this facet is used with an integer data type), the value is assumed to be inclusive.

Copyright © SDMx TWG 2020

Name	Type	Documentation
decimals	xs:positiveInteger	The decimals attribute indicates the number of characters allowed after the decimal separator.
pattern	xs:string	The pattern attribute holds any regular expression permitted in the similar facet in W3C XML Schema.
isMultiLingual (default: true)	xs:boolean	The isMultiLingual attribute indicates for a text format of type “string”, whether the value should allow for multiple values in different languages.

Element Documentation:

Name	Type	Documentation
SentinelValue	SentinelValueType	SentinelValue defines a value that has a special meaning within the text format representation of a component.

SimpleComponentTextFormatType: SimpleComponentTextFormatType is a restricted version of the BasicComponentTextFormatType that does not allow for multi-lingual values.

Derivation:

```
TextFormatType (restriction)
   BasicComponentTextFormatType (restriction)
         SimpleComponentTextFormatType
```

Attributes:

```
textType?, isSequence?, interval?, startValue?, endValue?,
timeInterval?, startTime?, endTime?, minLength?, maxLength?, minValue?,
maxValue?, decimals?, pattern?
```

Content:

```
SentinelValue*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Copyright © SDMx TWG 2020

Name	Type	Documentation
textType (default: String)	SimpleDataType	The textType attribute provides a description of the datatype. If it is not specified, any valid characters may be included in the text field (it corresponds to the xs:string datatype of W3C XML Schema) within the constraints of the facets.
isSequence	xs:boolean	The isSequence attribute indicates whether the values are intended to be ordered, and it may work in combination with the interval, startValue, and endValue attributes or the timeInterval, startTime, and endTime, attributes. If this attribute holds a value of true, a start value or time and a numeric or time interval must supplied. If an end value is not given, then the sequence continues indefinitely.
interval	xs:decimal	The interval attribute specifies the permitted interval (increment) in a sequence. In order for this to be used, the isSequence attribute must have a value of true.
startValue	xs:decimal	The startValue attribute is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates the starting point of the sequence. This value is mandatory for a numeric sequence to be expressed.
endValue	xs:decimal	The endValue attribute is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates that ending point (if any) of the sequence.
timeInterval	xs:duration	The timeInterval attribute indicates the permitted duration in a time sequence. In order for this to be used, the isSequence attribute must have a value of true.
startTime	StandardTimePeriodType	The startTime attribute is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates the start time of the sequence. This value is mandatory for a time sequence to be expressed.
endTime	StandardTimePeriodType	The endTime attribute is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates that ending point (if any) of the sequence.
minLength	xs:positiveInteger	The minLength attribute specifies the minimum and length of the value in characters.
maxLength	xs:positiveInteger	The maxLength attribute specifies the maximum length of the value in characters.
minValue	xs:decimal	The minValue attribute is used for inclusive and exclusive ranges, indicating what the lower bound of the range is. If this is used with an inclusive range, a valid value will be greater than or equal to the value specified here. If the inclusive and exclusive data type is not specified (e.g. this facet is used with an integer data type), the value is assumed to be inclusive.
maxValue	xs:decimal	The maxValue attribute is used for inclusive and exclusive ranges, indicating what the upper bound of the range is. If this is used with an inclusive range, a valid value will be less than or equal to the value specified here. If the inclusive and exclusive data type is not specified (e.g. this facet is used with an integer data type), the value is assumed to be inclusive.
decimals	xs:positiveInteger	

Copyright © SDMx TWG 2020

Name	Type	Documentation
		The decimals attribute indicates the number of characters allowed after the decimal separator.
pattern	xs:string	The pattern attribute holds any regular expression permitted in the similar facet in W3C XML Schema.

Element Documentation:

Name	Type	Documentation
SentinelValue	SentinelValueType	SentinelValue defines a value that has a special meaning within the text format representation of a component.

CodedTextFormatType: CodedTextFormatType is a restricted version of the SimpleComponentTextFormatType that only allows factets and text types applicable to codes. Although the time facets permit any value, an actual code identifier does not support the necessary characters for time. Therefore these facets should not contain time in their values.

Derivation:

```
TextFormatType (restriction)
   BasicComponentTextFormatType (restriction)
         SimpleComponentTextFormatType (restriction)
               CodedTextFormatType
```

Attributes:

```
textType?, isSequence?, interval?, startValue?, endValue?,
timeInterval?, startTime?, endTime?, minLength?, maxLength?, minValue?,
maxValue?, pattern?
```

Content:

```
{Empty}
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Copyright © SDMx TWG 2020

Name	Type	Documentation
textType	CodeDataType	The textType attribute provides a description of the datatype. If it is not specified, any valid characters may be included in the text field (it corresponds to the xs:string datatype of W3C XML Schema) within the constraints of the facets.
isSequence	xs:boolean	The isSequence attribute indicates whether the values are intended to be ordered, and it may work in combination with the interval, startValue, and endValue attributes or the timeInterval, startTime, and endTime, attributes. If this attribute holds a value of true, a start value or time and a numeric or time interval must supplied. If an end value is not given, then the sequence continues indefinitely.
interval	xs:integer	The interval attribute specifies the permitted interval (increment) in a sequence. In order for this to be used, the isSequence attribute must have a value of true.
startValue	xs:integer	The startValue attribute is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates the starting point of the sequence. This value is mandatory for a numeric sequence to be expressed.
endValue	xs:integer	The endValue attribute is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates that ending point (if any) of the sequence.
timeInterval	xs:duration	The timeInterval attribute indicates the permitted duration in a time sequence. In order for this to be used, the isSequence attribute must have a value of true.
startTime	StandardTimePeriodType	The startTime attribute is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates the start time of the sequence. This value is mandatory for a time sequence to be expressed.
endTime	StandardTimePeriodType	The endTime attribute is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates that ending point (if any) of the sequence.
minLength	xs:positiveInteger	The minLength attribute specifies the minimum and length of the value in characters.
maxLength	xs:positiveInteger	The maxLength attribute specifies the maximum length of the value in characters.
minValue	xs:integer	The minValue attribute is used for inclusive and exclusive ranges, indicating what the lower bound of the range is. If this is used with an inclusive range, a valid value will be greater than or equal to the value specified here. If the inclusive and exclusive data type is not specified (e.g. this facet is used with an integer data type), the value is assumed to be inclusive.
maxValue	xs:integer	The maxValue attribute is used for inclusive and exclusive ranges, indicating what the upper bound of the range is. If this is used with an inclusive range, a valid value will be less than or equal to the value specified here. If the inclusive and exclusive data type is not specified (e.g. this facet is used with an integer data type), the value is assumed to be inclusive.
pattern	xs:string	The pattern attribute holds any regular expression permitted in the similar facet in W3C XML Schema.

Copyright © SDMx TWG 2020

NonFacetedTextFormatType: NonFacetedTextFormatType is a restricted version of the SimpleComponentTextFormatType that does not allow for any facets.

Derivation:

```
TextFormatType (restriction)
   BasicComponentTextFormatType (restriction)
         SimpleComponentTextFormatType (restriction)
               NonFacetedTextFormatType
```

Attributes:

```
textType?
```

Content:

```
{Empty}
```

Attribute Documentation:

Name	Type	Documentation
textType	SimpleDataType	The textType attribute provides a description of the datatype. If it is not specified, any valid characters may be included in the text field (it corresponds to the xs:string datatype of W3C XML Schema) within the constraints of the facets.

TimeTextFormatType: TimeTextFormat is a restricted version of the SimpleComponentTextFormatType that only allows time based format and specifies a default ObservationalTimePeriod representation and facets of a start and end time.

Derivation:

```
TextFormatType (restriction)
   BasicComponentTextFormatType (restriction)
         SimpleComponentTextFormatType (restriction)
               TimeTextFormatType
```

Attributes:

```
textType?, startTime?, endTime?
```

Content:

```
SentinelValue*
```

Attribute Documentation:

Name	Type	Documentation
textType (default: ObservationalTimePeriod)	TimeDataType	The textType attribute provides a description of the datatype. If it is not specified, any valid characters may be included in the text field (it corresponds to the xs:string datatype of W3C XML Schema) within the constraints of the facets.
startTime	StandardTimePeriodType	The startTime attribute is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates the start time of the sequence. This value is mandatory for a time sequence to be expressed.
endTime	StandardTimePeriodType	The endTime attribute is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates that ending point (if any) of the sequence.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
SentinelValue	SentinelValueType	SentinelValue defines a value that has a special meaning within the text format representation of a component.

CategorisationBaseType: CategorisationBaseType defines the base refinement of the CategorisationType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 CategorisationBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	CategorisationUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

CategorisationType: CategorisationType is defines the structure for a categorisation. A source object is referenced via an object reference and the target category is referenced via the target category.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 CategorisationBaseType (extension)
                                       CategorisationType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, (Source, Target)?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	CategorisationUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Source	UrnReferenceType	Source is a reference to an object to be categorized.
Target	CategoryReferenceType	Target is reference to the category that the referenced object is to be mapped to.

CategorySchemeType: CategorySchemeType describes the structure of a category scheme. A category scheme is the descriptive information for an arrangement or division of categories into groups based on characteristics, which the objects have in common. This provides for a simple, leveled hierarchy or categories.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       CategorySchemeType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, Category*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds the identification of the category scheme. The type of this id is restricted to the common:NCNNameIDType. This is necessary, since the category scheme may be used to create simple types in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	CategorySchemeUrnTyp e	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Category	CategoryType	Category represents a set of nested categories which describe a simple classification hierarchy.

CategoryType: CategoryType describes the details of a category. A category is defined as an item at any level in a classification. The Category element represents a set of nested categories which are child categories.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           CategoryType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, Category*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	CategoryUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Category	CategoryType	Category represents a set of nested categories which describe a simple classification hierarchy.

CodelistBaseType: CodelistType defines the structure of a codelist. A codelist is defined as a list from which some statistical concepts (coded concepts) take their values.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       CodelistBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, (Code | GeoFeatureSetCode
| GeoGridCode)*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds the identification of the code list. The type of this id is restricted to the common:NCNNameIDType. This is necessary, since the code list may be used to create simple types in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	CodelistUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Code	CodeType	Code describes a code in a codelist. In addition to the identification and description of the code, basic presentational information is also available. Presentational information not present may be added through the use of annotations.

CodelistType:

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       CodelistBaseType (extension)
                                             CodelistType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, (Code | GeoFeatureSetCode
| GeoGridCode)*, CodelistExtension*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds the identification of the code list. The type of this id is restricted to the common:NCNNameIDType. This is necessary, since the code list may be used to create simple types in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	CodelistUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Code	CodeType	Code describes a code in a codelist. In addition to the identification and description of the code, basic presentational information is also available. Presentational information not present may be added through the use of annotations. **Substitutions:** GeoFeatureSetCode, GeoGridCode
CodelistExtension	CodelistExtensionTyp e	CodelistExtension allows for the extension of codelists by referencing the codelists to be extended and providing inclusion/ exclusion rules for selecting the codes to extend. The order of these is important as it is indicates the order of precedence of the extended codelists for conflict resolution of codes. However, the prefix property can be used to ensure uniqueness of inherited codes in the extending codelist, in case conflicting codes must be included.

CodeType: CodeType describes the structure of a code. A code is defined as a language independent set of letters, numbers or symbols that represent a concept whose meaning is described in a natural language. Presentational information not present may be added through the use of annotations.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           CodeType
```

Attributes:

```
id, urn?, uri?
```

Content:

Annotations?, Link*, Name+, Description*, Parent?

Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	CodeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms. It may be used in place of a short description.
Description	TextType	Description provides a plain text, human-readable description of the code. This may be provided in multiple, parallel language-equivalent forms.
Parent	SingleNCNameIDType	Parent provides the ability to describe simple hierarchies within a single codelist, by referencing the id value of another code in the same codelist.

CodelistExtensionType: CodelistExtensionType defines the structure of a codelist to be extended by the codelist defining the extension. It provides a reference to the extended codelist and selection criteria to indicate the codes to be included in the extending codelist.

Attributes:

```
prefix?
```

Content:

Codelist, (InclusiveCodeSelection | ExclusiveCodeSelection)?

Attribute Documentation:

Name	Type	Documentation
prefix	xs:string	A reference to a codelist may contain a prefix. If a prefix is provided, this prefix will be applied to all the codes in the codelist before they are imported into the extended codelist.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Codelist	CodelistReferenceType	
InclusiveCodeSelection	CodeSelectionType	Selection of codes from the referenced (extended) codelist to be included in the extending codelist.
ExclusiveCodeSelection	CodeSelectionType	Selection of codes from the referenced (extended) codelist to be excluded in the extending codelist.

CodeSelectionType: CodeSelectionType defines the structure for code selection to be used as inclusive or exclusive extensions.

Content:

MemberValue+

Element Documentation:

Name	Type	Documentation
MemberValue	MemberValueType	An explicit or wildcard selection of a code(s) from the codelist selected for inclusion/exclusion. If a wildcard expression is used, it is evaluated to determine codes selected for inclusion/exclusion. Otherwise, each member value is a distinct code. If the extended list is hierarchical, this can indicate whether child codes are to be included.

MemberValueType: Allows for a ditinct reference or a wildcard expression for selecting codes from a codelist.

Derivation:

```
xs:anySimpleType (restriction)
   xs:string (restriction)
         WildcardedMemberValueType (extension)
               MemberValueType
```

Attributes:

```
cascadeValues?
```

Content:

Attribute Documentation:

Name	Type	Documentation
cascadeValues	CascadeSelectionType	Indicates whether child codes should be selected when the codelist is hierarchical. Possible values are true (include the selected and child codes), false (only include the selected code(s)), and excluderoot (include the children but not the selected code(s)).

GeoCodelistBaseType: GeoCodelistBaseType is an abstract base refinement of a codelist that restricts the cods to be derived from the abstract GeoRefCode.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       CodelistBaseType (extension)
```


Copyright © SDMx TWG 2020

```
                                          CodelistType
(restriction)
                                                GeoCodelistBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, *GeoRefCode**,
CodelistExtension*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds the identification of the code list. The type of this id is restricted to the common:NCNNameIDType. This is necessary, since the code list may be used to create simple types in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	CodelistUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
GeoRefCode	*GeoRefCodeType*	GeoRefCode is the abstract base from which specific types of geographic codes will be derived.
CodelistExtension	CodelistExtensionType	CodelistExtension allows for the extension of codelists by referencing the codelists to be extended and providing inclusion/exclusion rules for selecting the codes to extend. The order of these is important as it is indicates the order of precedence of the extended codelists for conflict resolution of codes. However, the prefix property can be used to ensure uniqueness of inherited codes in the extending codelist, in case conflicting codes must be included.

GeoCodelistType: GeoCodelistType is an abstract refinement of a codelist from which specific types of geographic codelists will be derived.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       CodelistBaseType (extension)
                                             CodelistType
(restriction)
                                                   GeoCodelistBaseType
(extension)
                                                         GeoCodelistType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?, geoType
```

Content:

```
Annotations?, Link*, Name+, Description*, *GeoRefCode**,
CodelistExtension*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds the identification of the code list. The type of this id is restricted to the common:NCNNameIDType. This is necessary, since the code list may be used to create simple types in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	CodelistUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.
geoType	GeoCodelistTypeType	The type of geographic codelist. The will be refined and provided a fixed value in the specific geographic codelist type implementations.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
GeoRefCode	*GeoRefCodeType*	GeoRefCode is the abstract base from which specific types of geographic codes will be derived.
CodelistExtension	CodelistExtensionType	CodelistExtension allows for the extension of codelists by referencing the codelists to be extended and providing inclusion/exclusion rules for selecting the codes to extend. The order of these is important as it is indicates the order of precedence of the extended codelists for conflict resolution of codes. However, the prefix property can be used to ensure uniqueness of inherited codes in the extending codelist, in case conflicting codes must be included.

GeoRefCodeType: GeoRefCodeType is the abstract base type defining the structure of geographic codes.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           CodeType (extension)
                                 GeoRefCodeType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, Parent?
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	CodeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms. It may be used in place of a short description.
Description	TextType	Description provides a plain text, human-readable description of the code. This may be provided in multiple, parallel language-equivalent forms.
Parent	SingleNCNameIDType	Parent provides the ability to describe simple hierarchies within a single codelist, by referencing the id value of another code in the same codelist.

GeographicCodelistType: GeographicCodelistType defines the sturcture of a geographic codelist. It comprises a set of GeoFeatureSetCodes, by adding a value in the Code that follows a pattern to represent a geo feature set.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       CodelistBaseType (extension)
                                             CodelistType
(restriction)
                                                   GeoCodelistBaseType
(extension)
                                                         GeoCodelistType
(restriction)
                                                               GeographicCodelistType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?, geoType
```

Content:

```
Annotations?, Link*, Name+, Description*, GeoFeatureSetCode*,
CodelistExtension*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds the identification of the code list. The type of this id is restricted to the common:NCNameIDType. This is necessary, since the code list may be used to create simple types in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	CodelistUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.
geoType (fixed: GeographicCodelist)	GeoCodelistTypeType	The type of geographic codelist. The will be refined and provided a fixed value in the specific geographic codelist type implementations.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
GeoFeatureSetCode	GeoFeatureSetCodeTyp e	Is a geographic code in a geographic codelist. It adds a value to a code that folows a pattern to represent a geo feature set.
CodelistExtension	CodelistExtensionTyp e	CodelistExtension allows for the extension of codelists by referencing the codelists to be extended and providing inclusion/exclusion rules for selecting the codes to extend. The order of these is important as it is indicates the order of precedence of the extended codelists for conflict resolution of codes. However, the prefix property can be used to ensure uniqueness of inherited codes in the extending codelist, in case conflicting codes must be included.

GeoFeatureSetCodeType: GeoFeatureSetCodeType defines the structure of a geogrphic code.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           CodeType (extension)
                                 GeoRefCodeType (extension)
                                       GeoFeatureSetCodeType
```

Attributes:

```
id, urn?, uri?, value
```

Content:

```
Annotations?, Link*, Name+, Description*, Parent?
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	CodeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
value	xs:string	The geo feature set of the Code, which represents a set of points defining a feature in a format defined a predefined pattern (see section 6).

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms. It may be used in place of a short description.
Description	TextType	Description provides a plain text, human-readable description of the code. This may be provided in multiple, parallel language-equivalent forms.
Parent	SingleNCNameIDType	Parent provides the ability to describe simple hierarchies within a single codelist, by referencing the id value of another code in the same codelist.

GeoGridCodelistBaseType: GeoGridCodelistBaseType is the abstract base refinement for a geographic grid codelist.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       CodelistBaseType (extension)
                                             CodelistType
(restriction)
                                                   GeoCodelistBaseType
(extension)
                                                         GeoCodelistType
(restriction)
                                                               GeoGridCodelistBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?, geoType
```

Content:

```
Annotations?, Link*, Name+, Description*, GeoGridCode*,
CodelistExtension*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds the identification of the code list. The type of this id is restricted to the common:NCNNameIDType. This is necessary, since the code list may be used to create simple types in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	CodelistUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.
geoType (fixed: GeoGridCodelist)	GeoCodelistTypeType	The type of geographic codelist. The will be refined and provided a fixed value in the specific geographic codelist type implementations.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
GeoGridCode	GeoGridCodeType	GeoGridCode is a code the represents a geographic grid cell that belongs to a specific grid definition.
CodelistExtension	CodelistExtensionType	CodelistExtension allows for the extension of codelists by referencing the codelists to be extended and providing inclusion/exclusion rules for selecting the codes to extend. The order of these is important as it is indicates the order of precedence of the extended codelists for conflict resolution of codes. However, the prefix property can be used to ensure uniqueness of inherited codes in the extending codelist, in case conflicting codes must be included.

GeoGridCodelistType: GeoGridCodelistType defines the structure of a geographic grid code list. These define a geographical grid composed of cells representing regular squared portions of the Earth.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       CodelistBaseType (extension)
                                             CodelistType
(restriction)
                                                   GeoCodelistBaseType
(extension)
                                                         GeoCodelistType
(restriction)
                                                               GeoGridCodelistBaseType
(extension)
                                                                     GeoGridCodelistType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?, geoType
```

Content:

```
Annotations?, Link*, Name+, Description*, GeoGridCode*,
CodelistExtension*, GridDefinition
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds the identification of the code list. The type of this id is restricted to the common:NCNNameIDType. This is necessary, since the code list may be used to create simple types in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	CodelistUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.
geoType (fixed: GeoGridCodelist)	GeoCodelistTypeType	The type of geographic codelist. The will be refined and provided a fixed value in the specific geographic codelist type implementations.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
GeoGridCode	GeoGridCodeType	GeoGridCode is a code the represents a geographic grid cell that belongs to a specific grid definition.
CodelistExtension	CodelistExtensionType	CodelistExtension allows for the extension of codelists by referencing the codelists to be extended and providing inclusion/exclusion rules for selecting the codes to extend. The order of these is important as it is indicates the order of precedence of the extended codelists for conflict resolution of codes. However, the prefix property can be used to ensure uniqueness of inherited codes in the extending codelist, in case conflicting codes must be included.
GridDefinition	xs:string	Contains a regular expression string corresponding to the grid definition for the GeoGrid Codelist.

GeoGridCodeType: GeoGridCodeType defines the structure of a geographic grid cell.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           CodeType (extension)
                                 GeoRefCodeType (extension)
                                       GeoGridCodeType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, Parent?, GeoCell
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	CodeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms. It may be used in place of a short description.
Description	TextType	Description provides a plain text, human-readable description of the code. This may be provided in multiple, parallel language-equivalent forms.
Parent	SingleNCNameIDType	Parent provides the ability to describe simple hierarchies within a single codelist, by referencing the id value of another code in the same codelist.
GeoCell	xs:string	The value used to assign the Code to one cell in the grid.

ValueListBaseType: ValueListBaseType defines the base refinement of the ValueListType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ValueListBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	ValueListUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

ValueListType: ValueListType defines the structure of value list. These represent a closed set of values the can occur for a dimension, measure, or attribute. These may be values, or values with names and descriptions (similar to a codelist).

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ValueListBaseType (extension)
                                       ValueListType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, ValueItem*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	ValueListUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
ValueItem	ValueItemType	

ValueItemType: ValueItemType defines the structure of a value item. A value must be provided, and a longer name and description can be provided to provide additiona meaning to the value (similar to a code in a code list).

Derivation:

```
AnnotableType (extension)
   ValueItemType
```

Attributes:

```
id
```

Content:

```
Annotations?, Name*, Description*
```

Attribute Documentation:

Name	Type	Documentation
id	xs:string	

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Name	TextType	Name is a reusable element, used for providing a human-readable name for an object.
Description	TextType	Description is a reusable element, used for providing a longer human-readable description of an object.

ConceptSchemeType: ConceptSchemeType describes the structure of a concept scheme. A concept scheme is the descriptive information for an arrangement or division of concepts into groups based on characteristics, which the objects have in common. It contains a collection of concept definitions, that may be arranged in simple hierarchies.

Derivation:

Copyright © SDMx TWG 2020

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       ConceptSchemeType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, Concept*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds the identification of the concept scheme. The type of this id is restricted to the common:NCNNameIDType. This is necessary, since the concept scheme may be used to create simple types in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	ConceptSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Concept	ConceptType	Concept describes the details of a concept within a concept scheme.

ConceptBaseType: ConceptBaseType is an abstract base type the forms the basis of the ConceptType by requiring a name and id, and restricting the content of the id.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           ConceptBaseType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, Parent?
```

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds the identification of the concept. The type of this id is restricted to the common:NCNameIDType. This is necessary, since concept id may be used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	ConceptUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Parent	SingleNCNameIDType	Parent captures the semantic relationships between concepts which occur within a single concept scheme. This identifies the concept of which the current concept is a qualification (in the ISO 11179 sense) or subclass.

ConceptType: ConceptType describes the details of a concept. A concept is defined as a unit of knowledge created by a unique combination of characteristics. If a concept does not specify a TextFormat or a core representation, then the representation of the concept is assumed to be represented by any set of valid characters (corresponding to the xs:string datatype of W3C XML Schema).

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           ConceptBaseType (extension)
                                 ConceptType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, Parent?,
CoreRepresentation?, ISOConceptReference?
```

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds the identification of the concept. The type of this id is restricted to the common:NCNameIDType. This is necessary, since concept id may be used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	ConceptUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Parent	SingleNCNameIDType	Parent captures the semantic relationships between concepts which occur within a single concept scheme. This identifies the concept of which the current concept is a qualification (in the ISO 11179 sense) or subclass.
CoreRepresentation	ConceptRepresentatio n	
ISOConceptReference	ISOConceptReferenceT ype	Provides a reference to an ISO 11179 concept.

ConceptRepresentation: ConceptRepresentation defines the core representation that are allowed for a concept. The text format allowed for a concept is that which is allowed for any non-target object component.

Derivation:

```
RepresentationType (restriction)
   ConceptRepresentation
```

Attributes:

```
minOccurs?, maxOccurs?
```

Content:

```
(TextFormat | (Enumeration, EnumerationFormat?))
```

Attribute Documentation:

Name	Type	Documentation
minOccurs (default: 1)	xs: nonNegativeInteger	The minOccurs attribute indicates the minimum number of value that must be reported for the component.
maxOccurs	OccurenceType	The maxOccurs attribute indicates the maximum number of values that can be reported for the component.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
TextFormat	BasicComponentTextFormatType	TextFormat describes an uncoded textual format.
Enumeration	AnyCodelistReference Type	Enumeration references a codelist which enumerates the possible values that can be used as the representation of this concept.
EnumerationFormat	CodedTextFormatType	EnumerationFormat describes the facets of the item scheme enumeration. This is for the most part, informational.

ISOConceptReferenceType: ISOConceptReferenceType provides a reference to and ISO 11179 concept.

Content:

```
ConceptAgency, ConceptSchemeID, ConceptID
```

Element Documentation:

Name	Type	Documentation
ConceptAgency	xs:string	
ConceptSchemeID	xs:string	
ConceptID	xs:string	

ConstraintBaseType: ConstraintBaseType is an abstract base type that forms the basis of the main abstract ConstraintType. It requires that a name be provided.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ConstraintBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MaintainableUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

ConstraintType: ConstraintType is an abstract base type that specific types of constraints (data and metadata) restrict and extend to describe their details. These constraint types both define a constraint attachment and a release calendar.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ConstraintBaseType (extension)
                                       ConstraintType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, role
```

Content:

```
Annotations?, Link*, Name+, Description*, ConstraintAttachment?,
ReleaseCalendar?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MaintainableUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
role	ConstraintRoleType	The role attribute indicates whether this constraint states what data is actually present for the constraint attachment, or if it defines what content is allowed.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
ConstraintAttachment	*ConstraintAttachment Type*	ConstraintAttachment describes the collection of constrainable artefacts that the constraint is attached to.
ReleaseCalendar	ReleaseCalendarType	ReleaseCalendar defines dates on which the constrained data is to be made available.

DataConstraintBaseType: DataConstraintBaseType is an abstract base refinement of ConstraintType. The constraint attachment is restricted to constrainable artefacts related to data.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ConstraintBaseType (extension)
                                       ConstraintType (restriction)
                                             DataConstraintBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, role
```

Content:

```
Annotations?, Link*, Name+, Description*, ConstraintAttachment?,
ReleaseCalendar?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	DataConstraintUrnTyp e	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
role	ConstraintRoleType	The role attribute indicates whether this constraint states what data is actually present for the constraint attachment, or if it defines what content is allowed.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
ConstraintAttachment	DataConstraintAttachmentType	ConstraintAttachment describes the collection of constrainable artefacts that the constraint is attached to.
ReleaseCalendar	ReleaseCalendarType	ReleaseCalendar defines dates on which the constrained data is to be made available.

DataConstraintType: DataConstraintType defines the structure of a data constraint. A data constraint can specify either the available set of keys (DataKeySet) or set of component values (CubeRegion) in a data source, or the allowable keys that can be constructed from a data structure definition. Multiple such constraints may be present for a constrained artefact. For instance, there may be a constraing that specifies the values allowed for the data source (role is “Allowed”) which can be used for validation or for constructing a partial code list, whilst another constraing can specify the actual content of a data source (role is “Actual”).

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ConstraintBaseType (extension)
                                       ConstraintType (restriction)
                                             DataConstraintBaseType
(extension)
                                                   DataConstraintType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, role
```

Content:

```
Annotations?, Link*, Name+, Description*, ConstraintAttachment?,
ReleaseCalendar?, DataKeySet*, CubeRegion[0..2]
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	DataConstraintUrnTyp e	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
role	ConstraintRoleType	The role attribute indicates whether this constraint states what data is actually present for the constraint attachment, or if it defines what content is allowed.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
ConstraintAttachment	DataConstraintAttachmentType	ConstraintAttachment describes the collection of constrainable artefacts that the constraint is attached to.
ReleaseCalendar	ReleaseCalendarType	ReleaseCalendar defines dates on which the constrained data is to be made available.
DataKeySet	DataKeySetType	DataKeySet defines a full, distinct set of dimension values and the attribute values associated with the key.
CubeRegion	CubeRegionType	CubeRegion defines a slice of the data set (dimensions and attribute values) for the constrained artefact. A set of included or excluded regions can be described.

MetadataConstraintBaseType: MetadataConstraintBaseType is an abstract base refinement of ConstraintType. The constraint attachment is restricted to constrainable artefacts related to metadata, and the only possible role is “Allowed”.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ConstraintBaseType (extension)
                                       ConstraintType (restriction)
                                             MetadataConstraintBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, role
```

Content:

```
Annotations?, Link*, Name+, Description*, ConstraintAttachment?,
ReleaseCalendar?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MetadataConstraintUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
role (fixed: Allowed)	ConstraintRoleType	The role attribute indicates whether this constraint states what data is actually present for the constraint attachment, or if it defines what content is allowed.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
ConstraintAttachment	MetadataConstraintAttachmentType	ConstraintAttachment describes the collection of constrainable artefacts that the constraint is attached to.
ReleaseCalendar	ReleaseCalendarType	ReleaseCalendar defines dates on which the constrained data is to be made available.

MetadataConstraintType: MetadataConstraintType defines the structure of a metadata constraint. A metadata constraint can specify allowed attribute values for metadata described by the constrained artefact.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ConstraintBaseType (extension)
                                       ConstraintType (restriction)
                                             MetadataConstraintBaseType
(extension)
                                                   MetadataConstraintType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, role
```

Content:

```
Annotations?, Link*, Name+, Description*, ConstraintAttachment?,
ReleaseCalendar?, MetadataTargetRegion[0..2]
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MetadataConstraintUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
role (fixed: Allowed)	ConstraintRoleType	The role attribute indicates whether this constraint states what data is actually present for the constraint attachment, or if it defines what content is allowed.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
ConstraintAttachment	MetadataConstraintAttachmentType	ConstraintAttachment describes the collection of constrainable artefacts that the constraint is attached to.
ReleaseCalendar	ReleaseCalendarType	ReleaseCalendar defines dates on which the constrained data is to be made available.
MetadataTargetRegion	MetadataTargetRegionType	MetadataTargetRegion describes the values allowed for metadata attributes.

ReleaseCalendarType: ReleaseCalendarType describes information about the timing of releases of the constrained data. All of these values use the standard “P7D” - style format.

Content:

```
Periodicity, Offset, Tolerance
```

Element Documentation:

Name	Type	Documentation
Periodicity	xs:string	Periodicity is the period between releases of the data set.
Offset	xs:string	Offset is the interval between January first and the first release of data within the year.
Tolerance	xs:string	Tolerance is the period after which the release of data may be deemed late.

DataKeySetType: DataKeySetType defines a collection of full or partial data keys (dimension values).

Attributes:

```
isIncluded
```

Content:

```
Key+
```

Attribute Documentation:

Name	Type	Documentation
isIncluded	xs:boolean	The isIncluded attribute indicates whether the keys defined in this key set are inclusive or exclusive to the constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Key	DataKeyType	Key contains a set of dimension values which identify a full set of data.

ConstraintAttachmentType: ConstraintAttachmentType describes a collection of references to constrainable artefacts.

Content:

```
(DataProvider | MetadataProvider | MetadataSet+ | SimpleDataSource+
| (DataStructure+, QueryableDataSource*) | (MetadataStructure+,
QueryableDataSource*) | (Dataflow+, QueryableDataSource*) |
(Metadataflow+, QueryableDataSource*) | (ProvisionAgreement+,
QueryableDataSource*) | (MetadataProvisionAgreement+,
QueryableDataSource*))
```


Copyright © SDMx TWG 2020

Element Documentation:

Copyright © SDMx TWG 2020

Name	Type	Documentation
DataProvider	DataProviderReferenceType	DataProvider is reference to a data provider to which the constraint is attached. If this is used, then only the release calendar is relevant. The referenced is provided as a URN and/or a full set of reference fields.
MetadataProvider	MetadataProviderReferenceType	MetadataProvider is reference to a metadata provider to which the constraint is attached. If this is used, then only the release calendar is relevant. The referenced is provided as a URN and/or a full set of reference fields.
MetadataSet	MetadataSetReferenceType	MetadataSet is reference to a metadata set to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields.
SimpleDataSource	xs:anyURI	SimpleDataSource describes a simple data source, which is a URL of a SDMX-ML data or metadata message.
DataStructure	DataStructureReferenceType	DataStructure is reference to a data structure definition to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields. A constraint which is attached to more than one data structure must only express key sets and/or cube regions where the identifiers of the dimensions are common across all structures to which the constraint is attached.
QueryableDataSource	QueryableDataSourceType	QueryableDataSource describes a queryable data source to which the constraint is attached.
MetadataStructure	MetadataStructureReferenceType	MetadataStructure is reference to a metadata structure definition to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields. A constraint which is attached to more than one metadata structure must only express key sets and/or target regions where the identifiers of the target objects are common across all structures to which the constraint is attached.
QueryableDataSource	QueryableDataSourceType	QueryableDataSource describes a queryable data source to which the constraint is attached.
Dataflow	DataflowReferenceTyp e	Dataflow is reference to a data flow to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields. A constraint can be attached to more than one dataflow, and the dataflows do not necessarily have to be usages of the same data structure. However, a constraint which is attached to more than one data structure must only express key sets and/or cube regions where the identifiers of the dimensions are common across all structures to which the constraint is attached.
QueryableDataSource	QueryableDataSourceType	QueryableDataSource describes a queryable data source to which the constraint is attached.
Metadataflow	MetadataflowReferenceType	Metadataflow is reference to a metadata flow to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields. A constraint can be attached to more than one metadataflow, and the metadataflows do not necessarily have to be usages of the same metadata structure. However, a constraint which is attached to more than one metadata structure must only express key sets and/or target regions where the identifiers of the target objects are common across all structures to which the constraint is attached.
QueryableDataSource		

Copyright © SDMx TWG 2020

Name	Type	Documentation
	QueryableDataSourceType	QueryableDataSource describes a queryable data source to which the constraint is attached.
ProvisionAgreement	ProvisionAgreementReferenceType	ProvisionAgreementReference is reference to a provision agreement to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields. A constraint can be attached to more than one provision aggreement, and the provision agreements do not necessarily have to be references structure usages based on the same structure. However, a constraint which is attached to more than one provision agreement must only express key sets and/or cube/target regions where the identifier of the components are common across all structures to which the constraint is attached.
QueryableDataSource	QueryableDataSourceType	QueryableDataSource describes a queryable data source to which the constraint is attached.
MetadataProvisionAgreement	MetadataProvisionAgreementReferenceType	ProvisionAgreementReference is reference to a provision agreement to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields. A constraint can be attached to more than one provision aggreement, and the provision agreements do not necessarily have to be references structure usages based on the same structure. However, a constraint which is attached to more than one provision agreement must only express key sets and/or cube/target regions where the identifier of the components are common across all structures to which the constraint is attached.
QueryableDataSource	QueryableDataSourceType	QueryableDataSource describes a queryable data source to which the constraint is attached.

DataConstraintAttachmentType: DataConstraintAttachmentType restricts the base ConstraintAttachmentType to only allow artefacts related to data.

Derivation:

```
ConstraintAttachmentType (restriction)
   DataConstraintAttachmentType
```

Content:

```
(DataProvider | SimpleDataSource+ | (DataStructure+,
QueryableDataSource*) | (Dataflow+, QueryableDataSource*) |
(ProvisionAgreement+, QueryableDataSource*))
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
DataProvider	DataProviderReferenceType	DataProvider is reference to a data provider to which the constraint is attached. If this is used, then only the release calendar is relevant. The referenced is provided as a URN and/or a full set of reference fields.
SimpleDataSource	xs:anyURI	SimpleDataSource describes a simple data source, which is a URL of a SDMX-ML data or metadata message.
DataStructure	DataStructureReferenceType	DataStructure is reference to a data structure definition to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields. A constraint which is attached to more than one data structure must only express key sets and/or cube regions where the identifiers of the dimensions are common across all structures to which the constraint is attached.
QueryableDataSource	QueryableDataSourceType	QueryableDataSource describes a queryable data source to which the constraint is attached.
Dataflow	DataflowReferenceType	Dataflow is reference to a data flow to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields. A constraint can be attached to more than one dataflow, and the dataflows do not necessarily have to be usages of the same data structure. However, a constraint which is attached to more than one data structure must only express key sets and/or cube regions where the identifiers of the dimensions are common across all structures to which the constraint is attached.
QueryableDataSource	QueryableDataSourceType	QueryableDataSource describes a queryable data source to which the constraint is attached.
ProvisionAgreement	ProvisionAgreementReferenceType	ProvisionAgreementReference is reference to a provision agreement to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields. A constraint can be attached to more than one provision aggreement, and the provision agreements do not necessarily have to be references structure usages based on the same structure. However, a constraint which is attached to more than one provision agreement must only express key sets and/or cube/target regions where the identifier of the components are common across all structures to which the constraint is attached.
QueryableDataSource	QueryableDataSourceType	QueryableDataSource describes a queryable data source to which the constraint is attached.

MetadataConstraintAttachmentType: MetadataConstraintAttachmentType restricts the base ConstraintAttachmentType to only allow artefacts related to metadata.

Derivation:

```
ConstraintAttachmentType (restriction)
   MetadataConstraintAttachmentType
```

Content:

```
(MetadataProvider | MetadataSet+ | SimpleDataSource+ |
(MetadataStructure+, QueryableDataSource*) | (Metadataflow+,
QueryableDataSource*) | (MetadataProvisionAgreement+,
QueryableDataSource*))
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
MetadataProvider	MetadataProviderReferenceType	MetadataProvider is reference to a metadata provider to which the constraint is attached. If this is used, then only the release calendar is relevant. The referenced is provided as a URN and/or a full set of reference fields.
MetadataSet	MetadataSetReferenceType	MetadataSet is reference to a metadata set to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields.
SimpleDataSource	xs:anyURI	SimpleDataSource describes a simple data source, which is a URL of a SDMX-ML data or metadata message.
MetadataStructure	MetadataStructureReferenceType	MetadataStructure is reference to a metadata structure definition to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields. A constraint which is attached to more than one metadata structure must only express key sets and/or target regions where the identifiers of the target objects are common across all structures to which the constraint is attached.
QueryableDataSource	QueryableDataSourceType	QueryableDataSource describes a queryable data source to which the constraint is attached.
Metadataflow	MetadataflowReferenceType	Metadataflow is reference to a metadata flow to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields. A constraint can be attached to more than one metadataflow, and the metadataflows do not necessarily have to be usages of the same metadata structure. However, a constraint which is attached to more than one metadata structure must only express key sets and/or target regions where the identifiers of the target objects are common across all structures to which the constraint is attached.
QueryableDataSource	QueryableDataSourceType	QueryableDataSource describes a queryable data source to which the constraint is attached.
MetadataProvisionAgreement	MetadataProvisionAgreementReferenceType	ProvisionAgreementReference is reference to a provision agreement to which the constraint is attached. The referenced is provided as a URN and/or a full set of reference fields. A constraint can be attached to more than one provision aggreement, and the provision agreements do not necessarily have to be references structure usages based on the same structure. However, a constraint which is attached to more than one provision agreement must only express key sets and/or cube/target regions where the identifier of the components are common across all structures to which the constraint is attached.
QueryableDataSource	QueryableDataSourceType	QueryableDataSource describes a queryable data source to which the constraint is attached.

RegionType: RegionType is an abstract type which defines a generic constraint region. This type can be refined to define regions for data or metadata sets. A region is defined by a collection of key values - each of which a collection of values for a component which disambiguates data (i.e. dimensions of a dataset). For each region, a collection of attribute values can be provided. Taken together, the key values and attributes serve to identify or describe a subset of a data or metadata set. Finally, the region can flagged as being included or excluded, although this flag only makes sense when the region is used in a particular context.

Derivation:

Copyright © SDMx TWG 2020

```
AnnotableType (extension)
   RegionType
```

Attributes:

```
include?, validFrom?, validTo?
```

Content:

```
Annotations?, KeyValue*, Component*
```

Attribute Documentation:

Name	Type	Documentation
include (default: true)	xs:boolean	The include attribute indicates that the region is to be included or excluded within the context in which it is defined. For example, if the regions is defined as part of a content constraint, the exclude flag would mean the data identified by the region is not present.
validFrom	StandardTimePeriodType	
validTo	StandardTimePeriodType	

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
KeyValue	*MemberSelectionType*	KeyValue contains a reference to a component which disambiguates the data (i.e. a dimension) and provides a collection of values for the component. The collection of values can be flagged as being inclusive or exclusive to the region being defined. Any key component that is not included is assumed to be wild carded, which is to say that the cube includes all possible values for the un-referenced key components. Further, this assumption applies to the values of the components as well. The values for any given component can only be sub-setted in the region by explicit inclusion or exclusion. For example, a dimension X which has the possible values of 1, 2, 3 is assumed to have all of these values if a key value is not defined. If a key value is defined with an inclusion attribute of true and the values of 1 and 2, the only the values of 1 and 2 for dimension X are included in the definition of the region. If the key value is defined with an inclusion attribute of false and the value of 1, then the values of 2 and 3 for dimension X are included in the definition of the region. Note that any given key component must only be referenced once in the region.
Component	*MemberSelectionType*	Component contains a reference to a component (data attribute, metadata attribute, or measure) and provides a collection of values for the referenced component. This serves to state that for the key which defines the region, the components that are specified here have or do not have (depending on the include attribute of the value set) the values provided. It is possible to provide a component reference without specifying values, for the purpose of stating the component is absent (include = false) or present with an unbounded set of values. As opposed to key components, which are assumed to be wild carded if absent, no assumptions are made about the absence of a component. Only components which are explicitly stated to be present or absent from the region will be know. All unstated components for the set cannot be assumed to absent or present.

MemberSelectionType: MemberSelectionType is an abstract base type which is used to provide a set of values for a referenced component. Implementations of this type will be based on a particular component type and refine the allowed values to reflect the types of values that are possible for that type of component.

Attributes:

```
id, include?, removePrefix?, validFrom?, validTo?
```

Content:

```
(Value+ | TimeRange)?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NestedNCNameIDType	The id attribute provides the identifier for the component for which values are being provided. This base type allows for a nested identifier to be provided, for the purpose of referencing a nested component (i.e. a metadata attribute). However, specific implementations will restrict this representation to only allow single level identifiers where appropriate.
include (default: true)	xs:boolean	The include attribute indicates whether the values provided for the referenced component are to be included or excluded from the region in which they are defined.
removePrefix	xs:boolean	The removePrefix attribute indicates whether codes should keep or not the prefix, as defined in the extension of codelist.
validFrom	StandardTimePeriodType	
validTo	StandardTimePeriodType	

Element Documentation:

Name	Type	Documentation
Value	SimpleComponentValueType	Value provides a simple value for the component, such as a coded, numeric, or simple text value. This type of component value is applicable for dimensions and attributes.
TimeRange	TimeRangeValueType	TimeValue provides a value for a component which has a time representation. This is repeatable to allow for a range to be specified, although a single value can also be provided. An operator is available on this to indicate whether the specified value indicates an exact value or the beginning/end of a range (inclusive or exclusive).

DataKeyType: DataKeyType is a region which defines a distinct full or partial data key. The key consists of a set of values, each referencing a dimension and providing a single value for that dimension. The purpose of the key is to define a subset of a data set (i.e. the observed value and data attribute) which have the dimension values provided in this definition. Any dimension not stated explicitly in this key is assumed to be wild carded, thus allowing for the definition of partial data keys.

Derivation:

```
AnnotableType (extension)
   RegionType (restriction)
         DataKeyType
```

Attributes:

```
include?, validFrom?, validTo?
```

Content:

```
Annotations?, KeyValue*, Component*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
include (fixed: true)	xs:boolean	The include attribute has a fixed value of true for a distinct key, since such a key is always assumed to identify existing data or metadata.
validFrom	StandardTimePeriodType	
validTo	StandardTimePeriodType	

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
KeyValue	DataKeyValueType	KeyValue contains a reference to a component which disambiguates the data (i.e. a dimension) and provides a collection of values for the component. The collection of values can be flagged as being inclusive or exclusive to the region being defined. Any key component that is not included is assumed to be wild carded, which is to say that the cube includes all possible values for the un-referenced key components. Further, this assumption applies to the values of the components as well. The values for any given component can only be sub-setted in the region by explicit inclusion or exclusion. For example, a dimension X which has the possible values of 1, 2, 3 is assumed to have all of these values if a key value is not defined. If a key value is defined with an inclusion attribute of true and the values of 1 and 2, the only the values of 1 and 2 for dimension X are included in the definition of the region. If the key value is defined with an inclusion attribute of false and the value of 1, then the values of 2 and 3 for dimension X are included in the definition of the region. Note that any given key component must only be referenced once in the region.
Component	DataComponentValueSetType	Component contains a reference to a component (data attribute, metadata attribute, or measure) and provides a collection of values for the referenced component. This serves to state that for the key which defines the region, the components that are specified here have or do not have (depending on the include attribute of the value set) the values provided. It is possible to provide a component reference without specifying values, for the purpose of stating the component is absent (include = false) or present with an unbounded set of values. As opposed to key components, which are assumed to be wild carded if absent, no assumptions are made about the absence of a component. Only components which are explicitly stated to be present or absent from the region will be know. All unstated components for the set cannot be assumed to absent or present.

DataKeyValueType: DataKeyValueType is a type for providing a dimension value for the purpose of defining a distinct data key. Only a single value can be provided for the dimension.

Derivation:

```
MemberSelectionType (restriction)
   DataKeyValueType
```

Attributes:

Copyright © SDMx TWG 2020

```
id, include?, removePrefix?
```

Content:

```
Value
```

Attribute Documentation:

Name	Type	Documentation
id	SingleNCNameIDType	The id attribute provides the identifier for the component for which values are being provided. This base type allows for a nested identifier to be provided, for the purpose of referencing a nested component (i.e. a metadata attribute). However, specific implementations will restrict this representation to only allow single level identifiers where appropriate.
include (fixed: true)	xs:boolean	The include attribute indicates whether the values provided for the referenced component are to be included or excluded from the region in which they are defined.
removePrefix	xs:boolean	The removePrefix attribute indicates whether codes should keep or not the prefix, as defined in the extension of codelist.

Element Documentation:

Name	Type	Documentation
Value	SimpleKeyValueType	Value provides a simple value for the component, such as a coded, numeric, or simple text value. This type of component value is applicable for dimensions and attributes.

DataComponentValueSetType: DataComponentValueSetType defines the structure for providing values for a data attributes, measures, or metadata attributes. If no values are provided, the component is implied to include/excluded from the region in which it is defined, with no regard to the value of the component. Note that for metadata attributes which occur within other metadata attributes, a nested identifier can be provided. For example, a value of CONTACT.ADDRESS.STREET refers to the metadata attribute with the identifier STREET which exists in the ADDRESS metadata attribute in the CONTACT metadata attribute, which is defined at the root of the report structure.

Derivation:

```
MemberSelectionType (restriction)
   DataComponentValueSetType
```

Attributes:

```
id, include?, removePrefix?
```

Content:

```
(Value+ | TimeRange)?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NestedNCNameIDType	The id attribute provides the identifier for the component for which values are being provided. This base type allows for a nested identifier to be provided, for the purpose of referencing a nested component (i.e. a metadata attribute). However, specific implementations will restrict this representation to only allow single level identifiers where appropriate.
include (default: true)	xs:boolean	The include attribute indicates whether the values provided for the referenced component are to be included or excluded from the region in which they are defined.
removePrefix	xs:boolean	The removePrefix attribute indicates whether codes should keep or not the prefix, as defined in the extension of codelist.

Element Documentation:

Name	Type	Documentation
Value	DataComponentValueType	Value provides a simple value for the component, such as a coded, numeric, or simple text value. This type of component value is applicable for dimensions and attributes.
TimeRange	TimeRangeValueType	TimeValue provides a value for a component which has a time representation. This is repeatable to allow for a range to be specified, although a single value can also be provided. An operator is available on this to indicate whether the specified value indicates an exact value or the beginning/end of a range (inclusive or exclusive).

CubeRegionType: CubeRegionType defines the structure of a data cube region. This is based on the abstract RegionType and simply refines the key and attribute values to conform with what is applicable for dimensions and attributes, respectively. See the documentation of the base type for more details on how a region is defined.

Derivation:

```
AnnotableType (extension)
   RegionType (restriction)
         CubeRegionType
```

Attributes:

```
include?
```

Content:

```
Annotations?, KeyValue*, Component*
```

Attribute Documentation:

Name	Type	Documentation
include (default: true)	xs:boolean	The include attribute indicates that the region is to be included or excluded within the context in which it is defined. For example, if the regions is defined as part of a content constraint, the exclude flag would mean the data identified by the region is not present.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
KeyValue	CubeRegionKeyType	KeyValue contains a reference to a component which disambiguates the data (i.e. a dimension) and provides a collection of values for the component. The collection of values can be flagged as being inclusive or exclusive to the region being defined. Any key component that is not included is assumed to be wild carded, which is to say that the cube includes all possible values for the un-referenced key components. Further, this assumption applies to the values of the components as well. The values for any given component can only be sub-setted in the region by explicit inclusion or exclusion. For example, a dimension X which has the possible values of 1, 2, 3 is assumed to have all of these values if a key value is not defined. If a key value is defined with an inclusion attribute of true and the values of 1 and 2, the only the values of 1 and 2 for dimension X are included in the definition of the region. If the key value is defined with an inclusion attribute of false and the value of 1, then the values of 2 and 3 for dimension X are included in the definition of the region. Note that any given key component must only be referenced once in the region.
Component	ComponentValueSetType	Component contains a reference to a component (data attribute, metadata attribute, or measure) and provides a collection of values for the referenced component. This serves to state that for the key which defines the region, the components that are specified here have or do not have (depending on the include attribute of the value set) the values provided. It is possible to provide a component reference without specifying values, for the purpose of stating the component is absent (include = false) or present with an unbounded set of values. As opposed to key components, which are assumed to be wild carded if absent, no assumptions are made about the absence of a component. Only components which are explicitly stated to be present or absent from the region will be know. All unstated components for the set cannot be assumed to absent or present.

MetadataTargetRegionType: MetadataTargetRegionType defines the structure of a metadata target region. A metadata target region must define the report structure and the metadata target from that structure on which the region is based. This type is based on the abstract RegionType and simply refines the key and attribute values to conform with what is applicable for target objects and metadata attributes, respectively. See the documentation of the base type for more details on how a region is defined.

Derivation:

```
AnnotableType (extension)
   RegionType (restriction)
         MetadataTargetRegionType
```

Attributes:

```
include?, validFrom?, validTo?
```

Content:

```
Component*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
include (default: true)	xs:boolean	The include attribute indicates that the region is to be included or excluded within the context in which it is defined. For example, if the regions is defined as part of a content constraint, the exclude flag would mean the data identified by the region is not present.
validFrom	StandardTimePeriodType	
validTo	StandardTimePeriodType	

Element Documentation:

Name	Type	Documentation
Component	MetadataAttributeValueSetType	Component contains a reference to a component (data attribute, metadata attribute, or measure) and provides a collection of values for the referenced component. This serves to state that for the key which defines the region, the components that are specified here have or do not have (depending on the include attribute of the value set) the values provided. It is possible to provide a component reference without specifying values, for the purpose of stating the component is absent (include = false) or present with an unbounded set of values. As opposed to key components, which are assumed to be wild carded if absent, no assumptions are made about the absence of a component. Only components which are explicitly stated to be present or absent from the region will be know. All unstated components for the set cannot be assumed to absent or present.

CubeRegionKeyType: CubeRegionKeyType is a type for providing a set of values for a dimension for the purpose of defining a data cube region. A set of distinct value can be provided, or if this dimension is represented as time, and time range can be specified.

Derivation:

```
MemberSelectionType (restriction)
   CubeRegionKeyType
```

Attributes:

```
id, include?, removePrefix?, validFrom?, validTo?
```

Content:

```
(Value+ | TimeRange)
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	SingleNCNameIDType	The id attribute provides the identifier for the component for which values are being provided. This base type allows for a nested identifier to be provided, for the purpose of referencing a nested component (i.e. a metadata attribute). However, specific implementations will restrict this representation to only allow single level identifiers where appropriate.
include (default: true)	xs:boolean	The include attribute indicates whether the values provided for the referenced component are to be included or excluded from the region in which they are defined.
removePrefix	xs:boolean	The removePrefix attribute indicates whether codes should keep or not the prefix, as defined in the extension of codelist.
validFrom	StandardTimePeriodType	
validTo	StandardTimePeriodType	

Element Documentation:

Name	Type	Documentation
Value	CubeKeyValueType	Value provides a simple value for the component, such as a coded, numeric, or simple text value. This type of component value is applicable for dimensions and attributes.
TimeRange	TimeRangeValueType	TimeValue provides a value for a component which has a time representation. This is repeatable to allow for a range to be specified, although a single value can also be provided. An operator is available on this to indicate whether the specified value indicates an exact value or the beginning/end of a range (inclusive or exclusive).

ComponentValueSetType: ComponentValueSetType defines the structure for providing values for a data attributes, measures, or metadata attributes. If no values are provided, the component is implied to include/excluded from the region in which it is defined, with no regard to the value of the component. Note that for metadata attributes which occur within other metadata attributes, a nested identifier can be provided. For example, a value of CONTACT.ADDRESS.STREET refers to the metadata attribute with the identifier STREET which exists in the ADDRESS metadata attribute in the CONTACT metadata attribute, which is defined at the root of the report structure.

Derivation:

```
MemberSelectionType (restriction)
   ComponentValueSetType
```

Attributes:

```
id, include?, removePrefix?
```

Content:

```
(Value+ | TimeRange)?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NestedNCNameIDType	The id attribute provides the identifier for the component for which values are being provided. This base type allows for a nested identifier to be provided, for the purpose of referencing a nested component (i.e. a metadata attribute). However, specific implementations will restrict this representation to only allow single level identifiers where appropriate.
include (default: true)	xs:boolean	The include attribute indicates whether the values provided for the referenced component are to be included or excluded from the region in which they are defined.
removePrefix	xs:boolean	The removePrefix attribute indicates whether codes should keep or not the prefix, as defined in the extension of codelist.

Element Documentation:

Name	Type	Documentation
Value	SimpleComponentValue Type	Value provides a simple value for the component, such as a coded, numeric, or simple text value. This type of component value is applicable for dimensions and attributes.
TimeRange	TimeRangeValueType	TimeValue provides a value for a component which has a time representation. This is repeatable to allow for a range to be specified, although a single value can also be provided. An operator is available on this to indicate whether the specified value indicates an exact value or the beginning/end of a range (inclusive or exclusive).

MetadataAttributeValueSetType: MetadataAttributeValueSetType defines the structure for providing values for a metadata attribute. If no values are provided, the attribute is implied to include/excluded from the region in which it is defined, with no regard to the value of the metadata attribute.

Derivation:

```
MemberSelectionType (restriction)
   MetadataAttributeValueSetType
```

Attributes:

```
id, include?, removePrefix?
```

Content:

```
(Value+ | TimeRange)?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NestedNCNameIDType	The id attribute provides the identifier for the component for which values are being provided. This base type allows for a nested identifier to be provided, for the purpose of referencing a nested component (i.e. a metadata attribute). However, specific implementations will restrict this representation to only allow single level identifiers where appropriate.
include (default: true)	xs:boolean	The include attribute indicates whether the values provided for the referenced component are to be included or excluded from the region in which they are defined.
removePrefix	xs:boolean	The removePrefix attribute indicates whether codes should keep or not the prefix, as defined in the extension of codelist.

Element Documentation:

Name	Type	Documentation
Value	SimpleComponentValue Type	Value provides a simple value for the component, such as a coded, numeric, or simple text value. This type of component value is applicable for dimensions and attributes.
TimeRange	TimeRangeValueType	TimeValue provides a value for a component which has a time representation. This is repeatable to allow for a range to be specified, although a single value can also be provided. An operator is available on this to indicate whether the specified value indicates an exact value or the beginning/end of a range (inclusive or exclusive).

SimpleComponentValueType: SimpleValueType contains a simple value for a component, and if that value is from a code list, the ability to indicate that child codes in a simple hierarchy are part of the value set of the component for the region.

Derivation:

```
xs:anySimpleType (restriction)
   xs:string (extension)
         SimpleComponentValueType
```

Attributes:

```
cascadeValues?, xml:lang?, validFrom?, validTo?
```

Content:

Attribute Documentation:

Name	Type	Documentation
cascadeValues (default: false)	CascadeSelectionType	The cascadeValues attribute, if true, indicates that if the value is taken from a code all child codes in a simple hierarchy are understood be included in the region.
xml:lang	xs:language	The xml:lang attribute specifies a language code for the value. This is used when the component value support multi-lingual values.
validFrom	StandardTimePeriodType	
validTo	StandardTimePeriodType	

Copyright © SDMx TWG 2020

CubeKeyValueType: CubeKeyValueType derives from the SimpleValueType, but does not allow for a locale (xml:lang).

Derivation:

```
xs:anySimpleType (restriction)
   xs:string (extension)
         SimpleComponentValueType (restriction)
               CubeKeyValueType
```

Attributes:

```
cascadeValues?, validFrom?, validTo?
```

Content:

Attribute Documentation:

Name	Type	Documentation
cascadeValues (default: false)	CascadeSelectionType	The cascadeValues attribute, if true, indicates that if the value is taken from a code all child codes in a simple hierarchy are understood be included in the region.
validFrom	StandardTimePeriodType	
validTo	StandardTimePeriodType	

DataComponentValueType: DataComponentValueType derives from the SimpleValueType, but does not allow for validity dates.

Derivation:

```
xs:anySimpleType (restriction)
   xs:string (extension)
         SimpleComponentValueType (restriction)
               DataComponentValueType
```

Attributes:

```
cascadeValues?, xml:lang?
```

Content:

Attribute Documentation:

Name	Type	Documentation
cascadeValues (default: false)	CascadeSelectionType	The cascadeValues attribute, if true, indicates that if the value is taken from a code all child codes in a simple hierarchy are understood be included in the region.
xml:lang	xs:language	The xml:lang attribute specifies a language code for the value. This is used when the component value support multi-lingual values.

SimpleKeyValueType: SimpleKeyValueType derives from the SimpleValueType, but does not allow for the cascading of value in the hierarchy, as keys are meant to describe a distinct full or partial key.

Derivation:

```
xs:anySimpleType (restriction)
   xs:string (extension)
         SimpleComponentValueType (restriction)
               SimpleKeyValueType
```

Content:

Copyright © SDMx TWG 2020

TimeRangeValueType: TimeRangeValueType allows a time period value to be expressed as a range. It can be expressed as the period before a period, after a period, or between two periods. Each of these properties can specify their inclusion in regards to the range.

Attributes:

```
validFrom?, validTo?
```

Content:

```
(BeforePeriod | AfterPeriod | (StartPeriod, EndPeriod))
```

Attribute Documentation:

Name	Type	Documentation
validFrom	StandardTimePeriodTy pe	
validTo	StandardTimePeriodTy pe	

Element Documentation:

Name	Type	Documentation
BeforePeriod	TimePeriodRangeType	BeforePeriod is the period before which the period is meant to cover. This date may be inclusive or exclusive in the range.
AfterPeriod	TimePeriodRangeType	AfterPeriod is the period after which the period is meant to cover. This date may be inclusive or exclusive in the range.
StartPeriod	TimePeriodRangeType	StartPeriod is the start date or the range that the queried date must occur within. This date may be inclusive or exclusive in the range.
EndPeriod	TimePeriodRangeType	EndPeriod is the end period of the range. This date may be inclusive or exclusive in the range.

TimePeriodRangeType: TimePeriodRangeType defines a time period, and indicates whether it is inclusive in a range.

Derivation:

```
xs:anySimpleType (restriction)
   ObservationalTimePeriodType (extension)
         TimePeriodRangeType
```

Attributes:

```
isInclusive?
```

Content:

Attribute Documentation:

Name	Type	Documentation
isInclusive (default: true)	xs:boolean	The isInclusive attribute, when true, indicates that the time period specified is included in the range.

DataStructureBaseType: DataStructureBaseType describes base refinement of the StructureType for a data structure definition. A data structure definition is defined as a collection of metadata concepts, their structure and usage when used to collect or disseminate data.

Derivation:

Copyright © SDMx TWG 2020

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 StructureType (restriction)
                                       DataStructureBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, DataStructureComponents?
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	DataStructureUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
DataStructureComponents	DataStructureComponentsType	DataStructureComponents defines the grouping of the sets of metadata concepts that have a defined structural role in the data structure definition. Note that for any component or group defined in a data structure definition, its id must be unique. This applies to the identifiers explicitly defined by the components as well as those inherited from the concept identity of a component. For example, if two dimensions take their identity from concepts with same identity (regardless of whether the concepts exist in different schemes) one of the dimensions must be provided a different explicit identifier. Although there are XML schema constraints to help enforce this, these only apply to explicitly assigned identifiers. Identifiers inherited from a concept from which a component takes its identity cannot be validated against this constraint. Therefore, systems processing data structure definitions will have to perform this check outside of the XML validation. There are also two reserved identifiers in a data structure definition; TIME_PERIOD, and REPORTING_YEAR_START_DAY. These identifiers may not be used outside of their respective defintions (TimeDimension and ReportingYearStartDay). This applies to both the explicit identifiers that can be assigned to the components or groups as well as an identifier inherited by a component from its concept identity. For example, if an ordinary dimension (i.e. not the time dimension) takes its concept identity from a concept with the identifier TIME_PERIOD, that dimension must provide a different explicit identifier.

DataStructureType: DataStructureType defines the structure for a data structure definition. A data structure definition is defined as a collection of metadata concepts, their structure and usage when used to collect or disseminate data.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 StructureType (restriction)
                                       DataStructureBaseType
(extension)
                                             DataStructureType
```

Attributes:

Copyright © SDMx TWG 2020

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, DataStructureComponents?,
Metadata?
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	DataStructureUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
DataStructureComponents	DataStructureComponentsType	DataStructureComponents defines the grouping of the sets of metadata concepts that have a defined structural role in the data structure definition. Note that for any component or group defined in a data structure definition, its id must be unique. This applies to the identifiers explicitly defined by the components as well as those inherited from the concept identity of a component. For example, if two dimensions take their identity from concepts with same identity (regardless of whether the concepts exist in different schemes) one of the dimensions must be provided a different explicit identifier. Although there are XML schema constraints to help enforce this, these only apply to explicitly assigned identifiers. Identifiers inherited from a concept from which a component takes its identity cannot be validated against this constraint. Therefore, systems processing data structure definitions will have to perform this check outside of the XML validation. There are also two reserved identifiers in a data structure definition; TIME_PERIOD, and REPORTING_YEAR_START_DAY. These identifiers may not be used outside of their respective defintions (TimeDimension and ReportingYearStartDay). This applies to both the explicit identifiers that can be assigned to the components or groups as well as an identifier inherited by a component from its concept identity. For example, if an ordinary dimension (i.e. not the time dimension) takes its concept identity from a concept with the identifier TIME_PERIOD, that dimension must provide a different explicit identifier.
Metadata	MetadataStructureReferenceType	A data structure definition may be related to a metadata structure definition in order to use its metadata attributes as part of the data. Note that the referenced metadata set cannot contain nested metadata attributes, as these are not supported in the data. By default all metadata attributes can be associated at any level of the data. However, a metadata attribute usage can be used to provide a specific attribute relationshp for a given metadata attribute.

DataStructureComponentsBaseType: DataStructureComponentsBaseType is an abstract base type the serves as the basis for the DataStructureComponentsType. This type is necessary to allow for valid substitutions of component lists.

Derivation:

```
GroupingType (restriction)
   DataStructureComponentsBaseType
```


Copyright © SDMx TWG 2020

Content:

```
{Empty}
```

DataStructureComponentsType: DataStructureComponentsType describes the structure of the grouping to the sets of metadata concepts that have a defined structural role in the data structure definition. At a minimum at least one dimension must be defined.

Derivation:

```
GroupingType (restriction)
   DataStructureComponentsBaseType (extension)
         DataStructureComponentsType
```

Content:

```
DimensionList, Group*, AttributeList?, MeasureList?
```

Element Documentation:

Name	Type	Documentation
DimensionList	DimensionListType	DimensionList describes the key descriptor for the data structure definition. It is an ordered set of metadata concepts that, combined, classify a statistical series, such as a time series, and whose values, when combined (the key) in an instance such as a data set, uniquely identify a specific series.
Group	GroupType	Group describes a group descriptor in a data structure definition. It is a set metadata concepts (and possibly their values) that define a partial key derived from the key descriptor in a data structure definition.
AttributeList	AttributeListType	AttributeList describes the attribute descriptor for the data structure definition. It is a collection of metadata concepts that define the attributes of the data structure definition.
MeasureList	MeasureListType	MeasureList describes the measure descriptor for a data structure.

AttributeListBaseType: AttributeListBaseType is an abstract base type used as the basis for the AttributeListType.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         ComponentListType (restriction)
               AttributeListBaseType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id (fixed: AttributeDescriptor)	IDType	The id attribute is provided in this case for completeness. However, its value is fixed to AttributeDescriptor.
urn	AttributeDescriptorUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.

AttributeListType: AttributeListType describes the attribute descriptor for the data structure definition.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         ComponentListType (restriction)
               AttributeListBaseType (extension)
                     AttributeListType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*, (Attribute | MetadataAttributeUsage)+
```

Attribute Documentation:

Name	Type	Documentation
id (fixed: AttributeDescriptor)	IDType	The id attribute is provided in this case for completeness. However, its value is fixed to AttributeDescriptor.
urn	AttributeDescriptorUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Attribute	AttributeType	Attribute describes the definition of a data attribute, which is defined as a characteristic of an object or entity.
MetadataAttributeUsage	MetadataAttributeUsageType	MetadataAttributeUsage refines the details of how a metadata attribute from the metadata structure referenced from the data structure is used. By default, metadata attributes can be expressed at any level of the data. This allows an attribute relationship to be defined in order restrict the reporing of a metadata attribute to a specific part of the data.

AttributeBaseType: AttributeBaseType is an abstract base type that serves as the basis for the AttributeType. It restricts the text format base to a text format valid for data components (that does not allow for XHTML representation). The local representation is restricted to the values defined in codelist. The concept role is restricted to the values valid for a data attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     AttributeBaseType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*, ConceptIdentity, LocalRepresentation?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds an explicit identification of the component. If this identifier is not supplied, then it is assumed to be the same as the identifier of the concept referenced from the concept identity. Because structures require that every component be given a unique identifier, it may be necessary to assign an explicit identifier when more than one component in a structure reference concepts with same identifier. It is important to note that this applies strictly to the identifier of concept and not the URN. Therefore if two concepts with the same identifier from different concept schemes are referenced in the same structure, one of the components will have to provide a unique explicit identifier. The type of this identifier is restricted to the common:NCNameIDType. This is necessary, since component identifiers are used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	DataAttributeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
ConceptIdentity	ConceptReferenceType	ConceptIdentity allows for the referencing of a concept in a concept scheme. The component takes its semantic from this concept, and if an id is not specified, it takes its identification as well. If a representation (LocalRepresentation) is not supplied, then the representation of the component is also inherited from the concept. Note that in the case of the component representation being inherited from the concept, the allowable representations for the component still apply. Therefore, if a component references a concept with a core representation that is not allowed for the concept, that representation must be locally overridden. For components which can specify a concept role, it is implied that the concept which is referenced also identifies a role for the component.
LocalRepresentation	AttributeRepresentationType	LocalRepresentation references item schemes that may be used to create the representation of a component. The type of this must be refined such that a concrete item scheme reference is used.

AttributeType: AttributeType describes the structure of a data attribute, which is defined as a characteristic of an object or entity. The attribute takes its semantic, and in some cases it representation, from its concept identity. An attribute can be coded by referencing a code list from its coded local representation. It can also specify its text format, which is used as the representation of the attribute if a coded representation is not defined. Neither the coded or uncoded representation are necessary, since the attribute may take these from the referenced concept. An attribute specifies its relationship with other data structure components and is given an assignment status. These two properties dictate where in a data message the attribute will be attached, and

Copyright © SDMx TWG 2020

whether or not the attribute will be required to be given a value. A set of roles defined in concept scheme can be assigned to the attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     AttributeBaseType (extension)
                           AttributeType
```

Attributes:

```
id?, urn?, uri?, usage?
```

Content:

```
Annotations?, Link*, ConceptIdentity, LocalRepresentation?,
ConceptRole*, AttributeRelationship, MeasureRelationship?
```

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds an explicit identification of the component. If this identifier is not supplied, then it is assumed to be the same as the identifier of the concept referenced from the concept identity. Because structures require that every component be given a unique identifier, it may be necessary to assign an explicit identifier when more than one component in a structure reference concepts with same identifier. It is important to note that this applies strictly to the identifier of concept and not the URN. Therefore if two concepts with the same identifier from different concept schemes are referenced in the same structure, one of the components will have to provide a unique explicit identifier. The type of this identifier is restricted to the common:NCNameIDType. This is necessary, since component identifiers are used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	DataAttributeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
usage (default: optional)	UsageType	The usage attribute indicates whether an attribute value must be available for any corresponding existing observation.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
ConceptIdentity	ConceptReferenceType	ConceptIdentity allows for the referencing of a concept in a concept scheme. The component takes its semantic from this concept, and if an id is not specified, it takes its identification as well. If a representation (LocalRepresentation) is not supplied, then the representation of the component is also inherited from the concept. Note that in the case of the component representation being inherited from the concept, the allowable representations for the component still apply. Therefore, if a component references a concept with a core representation that is not allowed for the concept, that representation must be locally overridden. For components which can specify a concept role, it is implied that the concept which is referenced also identifies a role for the component.
LocalRepresentation	AttributeRepresentationType	LocalRepresentation references item schemes that may be used to create the representation of a component. The type of this must be refined such that a concrete item scheme reference is used.
ConceptRole	ConceptReferenceType	ConceptRole references concepts which define roles which this attribute serves.
AttributeRelationship	AttributeRelationshipType	AttributeRelationship describes how the value of this attribute varies with the values of other components. These relationships will be used to determine the attachment level of the attribute in the various data formats.
MeasureRelationship	MeasureRelationshipType	MeasureRelationship identifies the measures that the attribute applies to. If this is not used, the attribute is assumed to apply to all measures.

AttributeRelationshipType: AttributeRelationshipType defines the structure for stating the relationship between an attribute and other data structure definition components.

Content:

```
(Dataflow | Dimension+ | Group | Observation)
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Dataflow	EmptyType	This means that the value of the attribute varies per dataflow. It is the data modeller's responsibility to design or use non-overlapping dataflows that do not have observations in common, otherwise the integrity of dataflow-specific attribute values is not assured by the model, e.g. when querying those data through its DSD.
Dimension	OptionalLocalDimensionReferenceType	This is used to reference dimensions in the data structure definition on which the value of this attribute depends. An attribute using this relationship can be either a group, series (or section), or observation level attribute. The attachment level of the attribute will be determined by the data format and which dimensions are referenced.
Group	IDType	This is used as a convenience to referencing all of the dimension defined by the referenced group. The attribute will also be attached to this group.
Observation	EmptyType	This is used to specify that the value of the attribute is dependent upon the observed value. An attribute with this relationship will always be treated as an observation level attribute.

OptionalLocalDimensionReferenceType:

Derivation:

```
xs:anySimpleType (restriction)
   xs:string (restriction)
         NestedIDType (restriction)
               IDType (restriction)
                     NCNameIDType (extension)
                           OptionalLocalDimensionReferenceType
```

Attributes:

```
optional?
```

Content:

```
Attribute Documentation:
```

Name	Type	Documentation
optional (default: false)	xs:boolean	

MeasureRelationshipType: MeasureRelationshipType allows for the description of an attributes relationship to one or more measures

Content:

```
Measure+
```

Element Documentation:

Name	Type	Documentation
Measure	NCNameIDType	This is a reference to a measure defined in this data structure definition.

MetadataAttributeUsageBaseType: MetadataAttributeUsageBaseType is the abstract base refinement of a metadata attribute usage. Since this is a usage of metadata attribute already defined in metadata structure, the typical id, concept identity, and representation are excluded in place of a local refernce to the metadata attribute being used.

Copyright © SDMx TWG 2020

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     MetadataAttributeUsageBaseType
```

Attributes:

```
urn?, uri?
```

Content:

```
Annotations?, Link?
```

Attribute Documentation:

Name	Type	Documentation
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.

MetadataAttributeUsageType: MetadataAttributeUsageType defines the structure of how a metadata attribute is used in a data structure. This is a local reference to a metadata attribute from the metadata structure referenced by the data structure. An attribute relationship can be defined in order to describe the relationship of the metadata attribute to the data structure components.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     MetadataAttributeUsageBaseType (extension)
                           MetadataAttributeUsageType
```

Attributes:

```
urn?, uri?
```

Content:

```
Annotations?, Link?, MetadataAttributeReference, AttributeRelationship
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
MetadataAttributeReference	NCNameIDType	MetadataAttributeReference is a local reference to a metadata attribute defined in the metadata structure referenced by this data structure.
AttributeRelationship	AttributeRelationshipType	AttributeRelationship defines the relationship between the referenced metadata attribute and the components of the data structure.

DimensionListBaseType: DimensionListBaseType is an abstract base type used as the basis for the DimensionListType.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         ComponentListType (restriction)
               DimensionListBaseType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*
```

Attribute Documentation:

Name	Type	Documentation
id (fixed: DimensionDescriptor)	IDType	The id attribute is provided in this case for completeness. However, its value is fixed to DimensionDescriptor.
urn	DimensionDescriptorUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.

DimensionListType: DimensionListType describes the key descriptor for a data structure definition. The order of the declaration of child dimensions is significant: it is used to describe the order in which they will appear in data formats for which key values are supplied in an ordered fashion (exclusive of the time dimension, which is not represented as a member of the ordered key). Any data structure definition which uses the time dimension should also declare a frequency dimension, conventionally the first dimension in the key (the set of ordered non-time dimensions). If is not necessary to assign a time dimension, as data can be organised in any fashion required.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         ComponentListType (restriction)
               DimensionListBaseType (extension)
                     DimensionListType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*, Dimension+, TimeDimension?
```

Attribute Documentation:

Name	Type	Documentation
id (fixed: DimensionDescriptor)	IDType	The id attribute is provided in this case for completeness. However, its value is fixed to DimensionDescriptor.
urn	DimensionDescriptorUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Dimension	DimensionType	Dimension describes the structure of a dimension, which is defined as a statistical concept used (most probably together with other statistical concepts) to identify a statistical series, such as a time series, e.g. a statistical concept indicating certain economic activity or a geographical reference area.
TimeDimension	TimeDimensionType	TimeDimension is a special dimension which designates the period in time in which the data identified by the full series key applies.

BaseDimensionBaseType: BaseDimensionBaseType is an abstract base type that serves as the basis for any dimension. It restricts the text format base to a text format valid for data components (that does not allow for XHTML representation).

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     BaseDimensionBaseType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*, ConceptIdentity, LocalRepresentation?
```

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds an explicit identification of the component. If this identifier is not supplied, then it is assumed to be the same as the identifier of the concept referenced from the concept identity. Because structures require that every component be given a unique identifier, it may be necessary to assign an explicit identifier when more than one component in a structure reference concepts with same identifier. It is important to note that this applies strictly to the identifier of concept and not the URN. Therefore if two concepts with the same identifier from different concept schemes are referenced in the same structure, one of the components will have to provide a unique explicit identifier. The type of this identifier is restricted to the common:NCNameIDType. This is necessary, since component identifiers are used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
ConceptIdentity	ConceptReferenceType	ConceptIdentity allows for the referencing of a concept in a concept scheme. The component takes its semantic from this concept, and if an id is not specified, it takes its identification as well. If a representation (LocalRepresentation) is not supplied, then the representation of the component is also inherited from the concept. Note that in the case of the component representation being inherited from the concept, the allowable representations for the component still apply. Therefore, if a component references a concept with a core representation that is not allowed for the concept, that representation must be locally overridden. For components which can specify a concept role, it is implied that the concept which is referenced also identifies a role for the component.
LocalRepresentation	*DataStructureRepresentationType*	LocalRepresentation references item schemes that may be used to create the representation of a component. The type of this must be refined such that a concrete item scheme reference is used.

BaseDimensionType: BaseDimensionType is an abstract base type which defines the basic structure of all dimensions.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     BaseDimensionBaseType (extension)
                           BaseDimensionType
```

Attributes:

```
id?, urn?, uri?, position?
```

Content:

```
Annotations?, Link*, ConceptIdentity, LocalRepresentation?,
ConceptRole*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds an explicit identification of the component. If this identifier is not supplied, then it is assumed to be the same as the identifier of the concept referenced from the concept identity. Because structures require that every component be given a unique identifier, it may be necessary to assign an explicit identifier when more than one component in a structure reference concepts with same identifier. It is important to note that this applies strictly to the identifier of concept and not the URN. Therefore if two concepts with the same identifier from different concept schemes are referenced in the same structure, one of the components will have to provide a unique explicit identifier. The type of this identifier is restricted to the common:NCNameIDType. This is necessary, since component identifiers are used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
position	xs:int	The order of the dimensions in the key descriptor (DimensionList element) defines the order of the dimensions in the data structure. This position attribute explicitly specifies the position of the dimension in the data structure. It is optional and if specified must be consistent with the position of the dimension in the key descriptor.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
ConceptIdentity	ConceptReferenceType	ConceptIdentity allows for the referencing of a concept in a concept scheme. The component takes its semantic from this concept, and if an id is not specified, it takes its identification as well. If a representation (LocalRepresentation) is not supplied, then the representation of the component is also inherited from the concept. Note that in the case of the component representation being inherited from the concept, the allowable representations for the component still apply. Therefore, if a component references a concept with a core representation that is not allowed for the concept, that representation must be locally overridden. For components which can specify a concept role, it is implied that the concept which is referenced also identifies a role for the component.
LocalRepresentation	*DataStructureRepresentationType*	LocalRepresentation references item schemes that may be used to create the representation of a component. The type of this must be refined such that a concrete item scheme reference is used.
ConceptRole	ConceptReferenceType	ConceptRole references concepts which define roles which this dimension serves.

Copyright © SDMx TWG 2020

DimensionType: DimensionType describes the structure of an ordinary dimension, which is defined as a statistical concept used (most probably together with other statistical concepts) to identify a statistical series, such as a time series, e.g. a statistical concept indicating certain economic activity or a geographical reference area. The dimension takes its semantic, and in some cases it representation, from its concept identity. A dimension can be coded by referencing a code list from its coded local representation. It can also specify its text format, which is used as the representation of the dimension if a coded representation is not defined. Neither the coded or uncoded representation are necessary, since the dimension may take these from the referenced concept.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     BaseDimensionBaseType (extension)
                           BaseDimensionType (restriction)
                                 DimensionType
```

Attributes:

```
id?, urn?, uri?, position?
```

Content:

```
Annotations?, Link*, ConceptIdentity, LocalRepresentation?,
ConceptRole*
```

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds an explicit identification of the component. If this identifier is not supplied, then it is assumed to be the same as the identifier of the concept referenced from the concept identity. Because structures require that every component be given a unique identifier, it may be necessary to assign an explicit identifier when more than one component in a structure reference concepts with same identifier. It is important to note that this applies strictly to the identifier of concept and not the URN. Therefore if two concepts with the same identifier from different concept schemes are referenced in the same structure, one of the components will have to provide a unique explicit identifier. The type of this identifier is restricted to the common:NCNameIDType. This is necessary, since component identifiers are used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	DimensionUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
position	xs:int	The order of the dimensions in the key descriptor (DimensionList element) defines the order of the dimensions in the data structure. This position attribute explicitly specifies the position of the dimension in the data structure. It is optional and if specified must be consistent with the position of the dimension in the key descriptor.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
ConceptIdentity	ConceptReferenceType	ConceptIdentity allows for the referencing of a concept in a concept scheme. The component takes its semantic from this concept, and if an id is not specified, it takes its identification as well. If a representation (LocalRepresentation) is not supplied, then the representation of the component is also inherited from the concept. Note that in the case of the component representation being inherited from the concept, the allowable representations for the component still apply. Therefore, if a component references a concept with a core representation that is not allowed for the concept, that representation must be locally overridden. For components which can specify a concept role, it is implied that the concept which is referenced also identifies a role for the component.
LocalRepresentation	SimpleDataStructureRepresentationType	LocalRepresentation references item schemes that may be used to create the representation of a component. The type of this must be refined such that a concrete item scheme reference is used.
ConceptRole	ConceptReferenceType	ConceptRole references concepts which define roles which this dimension serves.

TimeDimensionType: TimeDimensionType describes the structure of a time dimension. The time dimension takes its semantic from its concept identity (usually the TIME_PERIOD concept), yet is always has a fixed identifier (TIME_PERIOD). The time dimension always has a fixed text format, which specifies that its format is always the in the value set of the observational time period (see common:ObservationalTimePeriodType). It is possible that the format may be a sub-set of the observational time period value set. For example, it is possible to state that the representation might always be a calendar year. See the enumerations of the textType attribute in the LocalRepresentation/TextFormat for more details of the possible sub-sets. It is also possible to facet this representation with start and end dates. The purpose of such facts is to restrict the value of the time dimension to occur within the specified range. If the time dimension is expected to allow for the standard reporting periods (see common:ReportingTimePeriodType) to be used, then it is strongly recommended that the reporting year start day attribute also be included in the data structure definition. When the reporting year start day attribute is used, any standard reporting period values will be assumed to be based on the start day contained in this attribute. If the reporting year start day attribute is not included and standard reporting periods are used, these values will be assumed to be based on a reporting year which begins January 1.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     BaseDimensionBaseType (extension)
                           BaseDimensionType (restriction)
                                 TimeDimensionType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*, ConceptIdentity, LocalRepresentation
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id (fixed: TIME_PERIOD)	NCNameIDType	The id attribute holds an explicit identification of the component. If this identifier is not supplied, then it is assumed to be the same as the identifier of the concept referenced from the concept identity. Because structures require that every component be given a unique identifier, it may be necessary to assign an explicit identifier when more than one component in a structure reference concepts with same identifier. It is important to note that this applies strictly to the identifier of concept and not the URN. Therefore if two concepts with the same identifier from different concept schemes are referenced in the same structure, one of the components will have to provide a unique explicit identifier. The type of this identifier is restricted to the common:NCNameIDType. This is necessary, since component identifiers are used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	TimeDimensionUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
ConceptIdentity	ConceptReferenceType	ConceptIdentity allows for the referencing of a concept in a concept scheme. The component takes its semantic from this concept, and if an id is not specified, it takes its identification as well. If a representation (LocalRepresentation) is not supplied, then the representation of the component is also inherited from the concept. Note that in the case of the component representation being inherited from the concept, the allowable representations for the component still apply. Therefore, if a component references a concept with a core representation that is not allowed for the concept, that representation must be locally overridden. For components which can specify a concept role, it is implied that the concept which is referenced also identifies a role for the component.
LocalRepresentation	TimeDimensionRepresentationType	LocalRepresentation references item schemes that may be used to create the representation of a component. The type of this must be refined such that a concrete item scheme reference is used.

GroupBaseType: GroupBaseType is an abstract base type that forms the basis for the GroupType.

Derivation:

Copyright © SDMx TWG 2020

```
AnnotableType (extension)
   IdentifiableType (extension)
         ComponentListType (restriction)
               GroupBaseType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	GroupDimensionDescriptorUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.

GroupType: GroupType describes the structure of a group descriptor in a data structure definition. A group may consist of a of partial key, or collection of distinct cube regions or key sets to which attributes may be attached. The purpose of a group is to specify attributes values which have the same value based on some common dimensionality. All groups declared in the data structure must be unique - that is, you may not have duplicate partial keys. All groups must be given unique identifiers.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         ComponentListType (restriction)
               GroupBaseType (extension)
                     GroupType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, GroupDimension+
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	GroupDimensionDescriptorUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
GroupDimension	GroupDimensionType	GroupDimension is a component which contains only a reference to a dimension in the key descriptor (DimensionList). Although it is conventional to declare dimensions in the same order as they are declared in the ordered key, there is no requirement to do so - the ordering of the values of the key are taken from the order in which the dimensions are declared. Note that the id of a dimension may be inherited from its underlying concept - therefore this reference value may actually be the id of the concept.

GroupDimensionBaseType: GroupDimensionBaseType is an abstract base type which refines the base ComponentType in order to form the basis for the GroupDimensionType.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     GroupDimensionBaseType
```

Content:

```
{Empty}
```

GroupDimensionType: GroupDimensionType defines a dimension component with a group key descriptor component list. Although technically a component, this is essentially a reference to a dimension defined in the key descriptor. Therefore, the identification, name, and description, concept identity and representation properties that are typically available for a component are not allowed here, as they are all inherited from the referenced dimension.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     GroupDimensionBaseType (extension)
                           GroupDimensionType
```

Content:

```
DimensionReference
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
DimensionReference	NCNameIDType	DimensionReference provides a reference to a dimension defined in the key descriptor of the data structure definition in which this group key descriptor is defined.

MeasureListType: MeasureListType describes the structure of the measure descriptor for a data structure definition.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         ComponentListType (restriction)
               MeasureListType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*, Measure+
```

Attribute Documentation:

Name	Type	Documentation
id (fixed: MeasureDescriptor)	IDType	The id is the identifier for the object.
urn	MeasureDescriptorUrn Type	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Measure	MeasureType	Measure defines the structure of a measure, which is the concept that is the value of the phenomenon to be measured in a data set.

MeasureBaseType: MeasureBaseType is an abstract base type that refines ComponentType to rerstrict the represenations to those which are applicable for a measure.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     MeasureBaseType
```


Copyright © SDMx TWG 2020

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*, ConceptIdentity, LocalRepresentation?
```

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds an explicit identification of the component. If this identifier is not supplied, then it is assumed to be the same as the identifier of the concept referenced from the concept identity. Because structures require that every component be given a unique identifier, it may be necessary to assign an explicit identifier when more than one component in a structure reference concepts with same identifier. It is important to note that this applies strictly to the identifier of concept and not the URN. Therefore if two concepts with the same identifier from different concept schemes are referenced in the same structure, one of the components will have to provide a unique explicit identifier. The type of this identifier is restricted to the common:NCNameIDType. This is necessary, since component identifiers are used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	MeasureUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
ConceptIdentity	ConceptReferenceType	ConceptIdentity allows for the referencing of a concept in a concept scheme. The component takes its semantic from this concept, and if an id is not specified, it takes its identification as well. If a representation (LocalRepresentation) is not supplied, then the representation of the component is also inherited from the concept. Note that in the case of the component representation being inherited from the concept, the allowable representations for the component still apply. Therefore, if a component references a concept with a core representation that is not allowed for the concept, that representation must be locally overridden. For components which can specify a concept role, it is implied that the concept which is referenced also identifies a role for the component.
LocalRepresentation	MeasureRepresentationType	LocalRepresentation references item schemes that may be used to create the representation of a component. The type of this must be refined such that a concrete item scheme reference is used.

Copyright © SDMx TWG 2020

MeasureType: MeasureType defines the structure of a measure descriptor. In addition to the identifying concept and representation, a usage status and max occurs can be defined.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     MeasureBaseType (extension)
                           MeasureType
```

Attributes:

```
id?, urn?, uri?, usage?
```

Content:

```
Annotations?, Link*, ConceptIdentity, LocalRepresentation?,
ConceptRole*
```

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds an explicit identification of the component. If this identifier is not supplied, then it is assumed to be the same as the identifier of the concept referenced from the concept identity. Because structures require that every component be given a unique identifier, it may be necessary to assign an explicit identifier when more than one component in a structure reference concepts with same identifier. It is important to note that this applies strictly to the identifier of concept and not the URN. Therefore if two concepts with the same identifier from different concept schemes are referenced in the same structure, one of the components will have to provide a unique explicit identifier. The type of this identifier is restricted to the common:NCNameIDType. This is necessary, since component identifiers are used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	MeasureUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
usage (default: optional)	UsageType	The usage attribute indicates whether a measure value must be available for any corresponding existing observation.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
ConceptIdentity	ConceptReferenceType	ConceptIdentity allows for the referencing of a concept in a concept scheme. The component takes its semantic from this concept, and if an id is not specified, it takes its identification as well. If a representation (LocalRepresentation) is not supplied, then the representation of the component is also inherited from the concept. Note that in the case of the component representation being inherited from the concept, the allowable representations for the component still apply. Therefore, if a component references a concept with a core representation that is not allowed for the concept, that representation must be locally overridden. For components which can specify a concept role, it is implied that the concept which is referenced also identifies a role for the component.
LocalRepresentation	MeasureRepresentationType	LocalRepresentation references item schemes that may be used to create the representation of a component. The type of this must be refined such that a concrete item scheme reference is used.
ConceptRole	ConceptReferenceType	ConceptRole references concepts which define roles which this measure serves.

DataStructureRepresentationType: DataStructureRepresentationType is an abstract base type which defines the allowable representations for any data structure definition component. The enumeration must be restricted to the proper type for item scheme for a given component.

Derivation:

```
RepresentationType (restriction)
   DataStructureRepresentationType
```

Attributes:

```
maxOccurs?
```

Content:

```
(TextFormat | (Enumeration, EnumerationFormat?))
```

Attribute Documentation:

Name	Type	Documentation
maxOccurs	OccurenceType	The maxOccurs attribute indicates the maximum number of values that can be reported for the component.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
TextFormat	SimpleComponentTextFormatType	TextFormat describes an uncoded textual format.
Enumeration	AnyCodelistReference Type	Enumeration references an item scheme that enumerates the allowable values for this representation.
EnumerationFormat	CodedTextFormatType	EnumerationFormat describes the facets of the item scheme enumeration. This is for the most part, informational.

AttributeRepresentationType: AttributeRepresentationType defines the representation for a data attribute. A data attribute can be text (including XHTML and multi-lingual values), a simple value, or an enumerated value.

Derivation:

```
RepresentationType (restriction)
   AttributeRepresentationType
```

Attributes:

```
minOccurs?, maxOccurs?
```

Content:

```
(TextFormat | (Enumeration, EnumerationFormat?))
```

Attribute Documentation:

Name	Type	Documentation
minOccurs (default: 1)	xs: nonNegativeInteger	The minOccurs attribute indicates the minimum number of value that must be reported for the component.
maxOccurs (default: 1)	OccurenceType	The maxOccurs attribute indicates the maximum number of values that can be reported for the component.

Element Documentation:

Name	Type	Documentation
TextFormat	BasicComponentTextFormatType	TextFormat describes an uncoded textual format.
Enumeration	AnyCodelistReference Type	Enumeration references an item scheme that enumerates the allowable values for this representation.
EnumerationFormat	CodedTextFormatType	EnumerationFormat describes the facets of the item scheme enumeration. This is for the most part, informational.

MeasureRepresentationType: MeasureRepresentationType defines the representation for a measure. A measure can be text (including XHTML and multi-lingual values), a simple value, or an enumerated value.

Derivation:

```
RepresentationType (restriction)
   MeasureRepresentationType
```

Attributes:

```
minOccurs?, maxOccurs?
```


Copyright © SDMx TWG 2020

Content:

```
(TextFormat | (Enumeration, EnumerationFormat?))
```

Attribute Documentation:

Name	Type	Documentation
minOccurs (default: 1)	xs: nonNegativeInteger	The minOccurs attribute indicates the minimum number of value that must be reported for the component.
maxOccurs (default: 1)	OccurenceType	The maxOccurs attribute indicates the maximum number of values that can be reported for the component.

Element Documentation:

Name	Type	Documentation
TextFormat	BasicComponentTextFormatType	TextFormat describes an uncoded textual format.
Enumeration	AnyCodelistReference Type	Enumeration references an item scheme that enumerates the allowable values for this representation.
EnumerationFormat	CodedTextFormatType	EnumerationFormat describes the facets of the item scheme enumeration. This is for the most part, informational.

SimpleDataStructureRepresentationType: SimpleDataStructureRepresentationType defines the representation for any non-time dimension data structure definition component.

Derivation:

```
RepresentationType (restriction)
   DataStructureRepresentationType (restriction)
         SimpleDataStructureRepresentationType
```

Content:

```
(TextFormat | (Enumeration, EnumerationFormat?))
```

Element Documentation:

Name	Type	Documentation
TextFormat	SimpleComponentTextFormatType	TextFormat describes an uncoded textual format.
Enumeration	CodelistReferenceTyp e	Enumeration references an item scheme that enumerates the allowable values for this representation.
EnumerationFormat	CodedTextFormatType	EnumerationFormat describes the facets of the item scheme enumeration. This is for the most part, informational.

TimeDimensionRepresentationType: TimeDimensionRepresentationType defines the representation for the time dimension. Enumerated values are not allowed.

Derivation:

```
RepresentationType (restriction)
   DataStructureRepresentationType (restriction)
         SimpleDataStructureRepresentationType (restriction)
               TimeDimensionRepresentationType
```

Content:

Copyright © SDMx TWG 2020

```
TextFormat
```

Element Documentation:

Name	Type	Documentation
TextFormat	TimeTextFormatType	TextFormat describes an uncoded textual format.

DataflowType: DataflowType describes the structure of a data flow. A data flow is defined as the structure of data that will provided for different reference periods. If this type is not referenced externally, then a reference to a data structure must be provided.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 StructureUsageType (restriction)
                                       DataflowType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, Structure?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	DataflowUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Structure	DataStructureReferenceType	Structure provides a reference to the data structure definition which defines the structure of all data for this flow.

HierarchyBaseType: HierarchyBaseType is an abstract base class that is the basis for the HierarchyType. It requires that a name be supplied.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 HierarchyBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	HierarchyUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

HierarchyType: HierarchyType describes the structure of a hierarchical codelist. A hierarchical code list is defined as an organised collection of codes that may participate in many parent/child relationships with other codes in the list.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 HierarchyBaseType (extension)
                                       HierarchyType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, hasFormalLevels
```

Content:

```
Annotations?, Link*, Name+, Description*, Level?, HierarchicalCode+
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	HierarchyUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
hasFormalLevels	xs:boolean	If “true”, this indicates a hierarchy where the structure is arranged in levels of detail from the broadest to the most detailed level. If “false”, this indicates a hierarchy structure where the items in the hierarchy have no formal level structure.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Level	LevelType	In a formally leveled hierarchy, Level describes a group of codes which are characterised by homogeneous coding, and where the parent of each code in the group is at the same higher level of the hierarchy. In a value based hierarchy Level describes information about the codes at the specified nesting level. This structure is recursive to indicate the hierarchy of the levels.
HierarchicalCode	HierarchicalCodeType	HierarchicalCode is used to assemble the codes from the codelist(s) referenced into a hierarchy.

HierarchicalCodeBaseType: HierarchicalCodeBaseType is an abstract base type the creates the basis for the HierarchicalCodeType. It removes the urn and uri.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         HierarchicalCodeBaseType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id attribute allows for an id to be assigned to the use of the particular code at that specific point in the hierarchy. This value is unique within the hierarchy being created, and is used to map the hierarchy against external structures.
urn	HierarchicalCodeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.

HierarchicalCodeType: HierarchicalCodeType describes the structure of a hierarchical code. A hierarchical code provides for a reference to a code that is referenced within the hierarchical code list via either a complete reference to a code through either a URN or full set of reference fields. Codes are arranged in a hierarchy by this reference. Note that it is possible to reference a single code such that it has multiple parents within the hierarchy. Further, the hierarchy may or may not be a leveled one.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         HierarchicalCodeBaseType (extension)
               HierarchicalCodeType
```

Attributes:

```
id, urn?, uri?, validFrom?, validTo?
```

Content:

```
Annotations?, Link*, Code, HierarchicalCode*, Level?
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id attribute allows for an id to be assigned to the use of the particular code at that specific point in the hierarchy. This value is unique within the hierarchy being created, and is used to map the hierarchy against external structures.
urn	HierarchicalCodeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
validFrom	xs:dateTime	The validFrom attriubte indicates the point in time in which the hiearchical code became effective. This can be used to track the historicity of codes changing over time.
validTo	xs:dateTime	The validTo attriubte indicates the point in time in which the hiearchical code became no longer effective. This can be used to track the historicity of codes changing over time.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Code	CodeReferenceType	Code provides a complete, explicit reference to a code through either its URN, or a complete reference to the codelist and code.
HierarchicalCode	HierarchicalCodeType	HierarchicalCode is used to nest referenced codes into a value based hierarchy.
Level	IDType	Level references a formal level defined within the hierarchy which defines this hierarchical code. This is only necessary if the nesting depth of the hierarchical code does not correspond to the nesting depth of the level to which it belongs (i.e. the hieararchical code is to skip down a level). Otherwise, the code is assumed to exist at the level in which the nesting depth of the level matches the nesting depth of the code.

LevelBaseType: LevelBaseType is an abstract base type that makes up the basis for the LevelType. It requires a name and id.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               LevelBaseType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	LevelUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

LevelType: LevelType describes a level in a hierarchical codelist. Where level is defined as a group where codes can be characterised by homogeneous coding, and where the parent of each code in the group is at the same higher level of the hierarchy.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               LevelBaseType (extension)
                     LevelType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, CodingFormat?, Level?
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	LevelUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
CodingFormat	CodingTextFormatType	CodingFormat specifies the text formatting of the codes in this level. This includes facets such as the expected characters and the length of the codes.
Level	LevelType	Level describes the next level down in the hierarchy.

CodingTextFormatType:

Derivation:

```
TextFormatType (restriction)
   BasicComponentTextFormatType (restriction)
         SimpleComponentTextFormatType (restriction)
               CodingTextFormatType
```

Attributes:

```
textType?, isSequence?, interval?, startValue?, endValue?, minLength?,
maxLength?, minValue?, maxValue?, pattern?
```

Content:

```
{Empty}
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
textType	SimpleCodeDataType	The textType attribute provides a description of the datatype. If it is not specified, any valid characters may be included in the text field (it corresponds to the xs:string datatype of W3C XML Schema) within the constraints of the facets.
isSequence	xs:boolean	The isSequence attribute indicates whether the values are intended to be ordered, and it may work in combination with the interval, startValue, and endValue attributes or the timeInterval, startTime, and endTime, attributes. If this attribute holds a value of true, a start value or time and a numeric or time interval must supplied. If an end value is not given, then the sequence continues indefinitely.
interval	xs:integer	The interval attribute specifies the permitted interval (increment) in a sequence. In order for this to be used, the isSequence attribute must have a value of true.
startValue	xs:positiveInteger	The startValue attribute is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates the starting point of the sequence. This value is mandatory for a numeric sequence to be expressed.
endValue	xs:positiveInteger	The endValue attribute is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates that ending point (if any) of the sequence.
minLength	xs:positiveInteger	The minLength attribute specifies the minimum and length of the value in characters.
maxLength	xs:positiveInteger	The maxLength attribute specifies the maximum length of the value in characters.
minValue	xs:positiveInteger	The minValue attribute is used for inclusive and exclusive ranges, indicating what the lower bound of the range is. If this is used with an inclusive range, a valid value will be greater than or equal to the value specified here. If the inclusive and exclusive data type is not specified (e.g. this facet is used with an integer data type), the value is assumed to be inclusive.
maxValue	xs:positiveInteger	The maxValue attribute is used for inclusive and exclusive ranges, indicating what the upper bound of the range is. If this is used with an inclusive range, a valid value will be less than or equal to the value specified here. If the inclusive and exclusive data type is not specified (e.g. this facet is used with an integer data type), the value is assumed to be inclusive.
pattern	xs:string	The pattern attribute holds any regular expression permitted in the similar facet in W3C XML Schema.

HierarchyAssociationBaseType: HierarchyAssociationBaseType defines the base refinement of the HierarchyAssociationType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 HierarchyAssociationBaseType
```


Copyright © SDMx TWG 2020

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	HierarchyAssociation UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

HierarchyAssociationType: HierarchyAssociationType defines the structure of a hiearchy association, which links a hierarchy with and identifiable object in the context of another object (e.g. a dimension within the context of a dataflow).

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 HierarchyAssociationType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, LinkedHierarchy,
LinkedObject, ContextObject?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MaintainableUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
LinkedHierarchy	HierarchyReferenceType	The associated hierarchy.
LinkedObject	UrnReferenceType	Associates the Identifiable Artefact that needs the Hierarchy.
ContextObject	UrnReferenceType	The context within which the association is performed.

MetadataflowBaseType: MetadataflowBaseType is an abstract base type that serves as the basis for the MetadataflowType. It restricts the structure to reference a metadata structure definition.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 StructureUsageType (restriction)
                                       MetadataflowBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, Structure?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MetadataflowUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Structure	MetadataStructureReferenceType	Structure provides a reference to the metadata structure definition describing the structure of all reference metadata for this flow.

MetadataflowType: MetadataflowType describes the structure of a metadata flow. A dataflow is defined as the structure of reference metadata that will be provided for different reference periods. If this type is not referenced externally, then a reference to a metadata structure definition must be provided

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 StructureUsageType (restriction)
                                       MetadataflowBaseType
(extension)
                                             MetadataflowType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, Structure?, Target+
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MetadataflowUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Structure	MetadataStructureReferenceType	Structure provides a reference to the metadata structure definition describing the structure of all reference metadata for this flow.
Target	WildcardUrnType	References identifiable structures to which the refernece metadata described by the referenced metadata structure should be restricted to. These references may include wildcards for parts of the reference.

MetadataStructureType: MetadataStructureType is used to describe a metadata structure definition, which is defined as a collection of metadata concepts, their structure and usage when used to collect or disseminate reference metadata.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 StructureType (restriction)
                                       MetadataStructureType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, MetadataStructureComponents?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MetadataStructureUrn Type	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
MetadataStructureComponents	MetadataStructureComponentsType	MetadataStructureComponents defines the grouping of the sets of the components that make up the metadata structure definition.

MetadataStructureComponentsBaseType: MetadataStructureComponentsBaseType is an abstract base type that forms the basis for the MetadataStructureComponentsType.

Derivation:

```
GroupingType (restriction)
   MetadataStructureComponentsBaseType
```

Content:

```
{Empty}
```

MetadataStructureComponentsType: MetadataStructureComponentsType describes the structure of the grouping of the sets of the components that make up the metadata structure definition.

Derivation:

```
GroupingType (restriction)
   MetadataStructureComponentsBaseType (extension)
         MetadataStructureComponentsType
```

Content:

```
MetadataAttributeList
```

Element Documentation:

Name	Type	Documentation
MetadataAttributeList	MetadataAttributeListType	MetadataAttributeList defines the set of metadata attributes that can be defined as a hierarchy, for reporting reference metadata about a target object. The identification of metadata attributes must be unique at any given level of the metadata structure. Although there are XML schema constraints to help enforce this, these only apply to explicitly assigned identifiers. Identifiers inherited from a concept from which a metadata attribute takes its identity cannot be validated against this constraint. Therefore, systems processing metadata structure definitions will have to perform this check outside of the XML validation.

Copyright © SDMx TWG 2020

MetadataAttributeListType: MetadataAttributeListType describes the structure of a meta data attribute list. It comprises a set of metadata attributes that can be defined as a hierarchy.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         ComponentListType (restriction)
               MetadataAttributeListType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*, MetadataAttribute+
```

Attribute Documentation:

Name	Type	Documentation
id (fixed: MetadataAttributeDescriptor)	IDType	The id attribute is provided in this case for completeness. However, its value is fixed to MetadataAttributeDescriptor.
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
MetadataAttribute	MetadataAttributeTyp e	MetadataAttribute defines the a metadata attribute, which is the value of an attribute, such as the instance of a coded or uncoded attribute in a metadata structure definition.

MetadataAttributeBaseType: MetadataAttributeBaseType is an abstract base type the serves as the basis for the MetadataAttributeType.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     MetadataAttributeBaseType
```

Attributes:

```
id?, urn?, uri?
```


Copyright © SDMx TWG 2020

Content:

```
Annotations?, Link*, ConceptIdentity, LocalRepresentation?
```

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds an explicit identification of the component. If this identifier is not supplied, then it is assumed to be the same as the identifier of the concept referenced from the concept identity. Because structures require that every component be given a unique identifier, it may be necessary to assign an explicit identifier when more than one component in a structure reference concepts with same identifier. It is important to note that this applies strictly to the identifier of concept and not the URN. Therefore if two concepts with the same identifier from different concept schemes are referenced in the same structure, one of the components will have to provide a unique explicit identifier. The type of this identifier is restricted to the common:NCNameIDType. This is necessary, since component identifiers are used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	MetadataAttributeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
ConceptIdentity	ConceptReferenceType	ConceptIdentity allows for the referencing of a concept in a concept scheme. The component takes its semantic from this concept, and if an id is not specified, it takes its identification as well. If a representation (LocalRepresentation) is not supplied, then the representation of the component is also inherited from the concept. Note that in the case of the component representation being inherited from the concept, the allowable representations for the component still apply. Therefore, if a component references a concept with a core representation that is not allowed for the concept, that representation must be locally overridden. For components which can specify a concept role, it is implied that the concept which is referenced also identifies a role for the component.
LocalRepresentation	MetadataAttributeRepresentationType	LocalRepresentation references item schemes that may be used to create the representation of a component. The type of this must be refined such that a concrete item scheme reference is used.

MetadataAttributeType: MetadataAttributeType describes the structure of a metadata attribute. The metadata attribute takes its semantic, and in some cases it representation, from its concept identity. A metadata attribute may be coded (via the local

Copyright © SDMx TWG 2020

representation), uncoded (via the text format), or take no value. In addition to this value, the metadata attribute may also specify subordinate metadata attributes. If a metadata attribute only serves the purpose of containing subordinate metadata attributes, then the isPresentational attribute should be used. Otherwise, it is assumed to also take a value. If the metadata attribute does take a value, and a representation is not defined, it will be inherited from the concept it takes its semantic from. The optional id on the metadata attribute uniquely identifies it within the metadata structured definition. If this id is not supplied, its value is assumed to be that of the concept referenced from the concept identity. Note that a metadata attribute (as identified by the id attribute) definition must be unique across the entire metadata structure definition.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         ComponentBaseType (extension)
               ComponentType (restriction)
                     MetadataAttributeBaseType (extension)
                           MetadataAttributeType
```

Attributes:

```
id?, urn?, uri?, minOccurs?, maxOccurs?, isPresentational?
```

Content:

```
Annotations?, Link*, ConceptIdentity, LocalRepresentation?,
MetadataAttribute*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds an explicit identification of the component. If this identifier is not supplied, then it is assumed to be the same as the identifier of the concept referenced from the concept identity. Because structures require that every component be given a unique identifier, it may be necessary to assign an explicit identifier when more than one component in a structure reference concepts with same identifier. It is important to note that this applies strictly to the identifier of concept and not the URN. Therefore if two concepts with the same identifier from different concept schemes are referenced in the same structure, one of the components will have to provide a unique explicit identifier. The type of this identifier is restricted to the common:NCNameIDType. This is necessary, since component identifiers are used to create XML elements and attributes in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	MetadataAttributeUrn Type	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
minOccurs (default: 1)	xs: nonNegativeInteger	The minOccurs attribute indicates the minimum number of times this metadata attribute must occur within its parent object.
maxOccurs (default: 1)	OccurenceType	The maxOccurs attribute indicates the maximum number of times this metadata attribute can occur within its parent object.
isPresentational (default: false)	xs:boolean	The isPresentational attribute indicates whether the metadata attribute should allow for a value. A value of true, meaning the metadata attribute is presentational means that the attribute only contains child metadata attributes, and does not contain a value. If this attribute is not set to true, and a representation (coded or uncoded) is not defined, then the representation of the metadata attribute will be inherited from the concept from which it takes its identity.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
ConceptIdentity	ConceptReferenceType	ConceptIdentity allows for the referencing of a concept in a concept scheme. The component takes its semantic from this concept, and if an id is not specified, it takes its identification as well. If a representation (LocalRepresentation) is not supplied, then the representation of the component is also inherited from the concept. Note that in the case of the component representation being inherited from the concept, the allowable representations for the component still apply. Therefore, if a component references a concept with a core representation that is not allowed for the concept, that representation must be locally overridden. For components which can specify a concept role, it is implied that the concept which is referenced also identifies a role for the component.
LocalRepresentation	MetadataAttributeRepresentationType	LocalRepresentation references item schemes that may be used to create the representation of a component. The type of this must be refined such that a concrete item scheme reference is used.
MetadataAttribute	MetadataAttributeTyp e	MetadataAttribute defines the a metadata attribute, which is the value of an attribute, such as the instance of a coded or uncoded attribute in a metadata structure definition.

MetadataAttributeRepresentationType: MetadataAttributeRepresentationType defines the possible local representations of a metadata attribute.

Derivation:

```
RepresentationType (restriction)
   MetadataAttributeRepresentationType
```

Content:

```
(TextFormat | (Enumeration, EnumerationFormat?))
```

Element Documentation:

Name	Type	Documentation
TextFormat	BasicComponentTextFormatType	TextFormat describes an uncoded textual format.
Enumeration	CodelistReferenceTyp e	Enumeration references an item scheme that enumerates the allowable values for this representation.
EnumerationFormat	CodedTextFormatType	EnumerationFormat describes the facets of the item scheme enumeration. This is for the most part, informational.

OrganisationSchemeBaseType: OrganisationSchemeBaseType is an abstract base type for any organisation scheme.

Derivation:

Copyright © SDMx TWG 2020

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       OrganisationSchemeBaseType
```

Attributes:

```
id, urn?, uri?, validFrom?, validTo?, agencyID, isExternalReference?,
serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MaintainableUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

OrganisationSchemeType: OrganisationSchemeType describes the structure of an organisation scheme.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       OrganisationSchemeBaseType
(extension)
                                             OrganisationSchemeType
```

Attributes:

```
id, urn?, uri?, validFrom?, validTo?, agencyID, isExternalReference?,
serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, *Organisation**
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MaintainableUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Organisation	*OrganisationType*	Organisation is an abstract substitution head for a generic organisation.

BaseOrganisationType: BaseOrganisationType is an abstract base type the forms the basis for the OrganisationType.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           BaseOrganisationType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, Parent?
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Parent	IDType	If the particular organisation scheme allows, an organisation may reference a parent organisation defined in the same scheme. This does not affect the identification of the organisation, but rather only serves to state the relationship.

OrganisationType: OrganisationType in an abstract type which describes the structure of the details of an organisation. In addition to the basic organisation identification, contact details can be provided.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           BaseOrganisationType (extension)
                                 OrganisationType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, Parent?, Contact*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Parent	IDType	If the particular organisation scheme allows, an organisation may reference a parent organisation defined in the same scheme. This does not affect the identification of the organisation, but rather only serves to state the relationship.
Contact	ContactType	Contact describes a contact for the organisation,

AgencySchemeType: AgencySchemeType defines a specific type of organisation scheme which contains only maintenance agencies. The agency scheme maintained by a particular maintenance agency is always provided a fixed identifier and is never versioned. Therefore, agencies can be added or removed without have to version the scheme. Agencies schemes have no hierarchy, meaning that no agency may define a relationship with another agency in the scheme. In fact, the actual parent agency for an agency in a scheme is the agency which defines the scheme.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       OrganisationSchemeBaseType
(extension)
                                             OrganisationSchemeType
(restriction)
                                                   AgencySchemeType
```

Attributes:

```
id, urn?, uri?, validFrom?, validTo?, agencyID, isExternalReference?,
serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, Agency*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id (fixed: AGENCIES)	IDType	The id is the identifier for the object.
urn	AgencySchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Agency	AgencyType	Agency is an organisation which maintains structural metadata such as classifications, concepts, data structures, and metadata structures.

DataConsumerSchemeType: DataConsumerSchemeType defines a type of organisation scheme which contains only data consumers. The data consumer scheme maintained by a particular maintenance agency is always provided a fixed identifier and is never versioned. Therefore, consumers can be added or removed without have to version the scheme. This scheme has no hierarchy, meaning that no organisation may define a relationship with another organisation in the scheme.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       OrganisationSchemeBaseType
(extension)
                                             OrganisationSchemeType
(restriction)
                                                   DataConsumerSchemeType
```

Attributes:

```
id, urn?, uri?, validFrom?, validTo?, agencyID, isExternalReference?,
serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, DataConsumer*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id (fixed: DATA_CONSUMERS)	IDType	The id is the identifier for the object.
urn	DataConsumerSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
DataConsumer	DataConsumerType	DataConsumer describes an organisation using data as input for further processing.

DataProviderSchemeType: DataProviderSchemeType defines a type of organisation scheme which contains only data providers. The data provider scheme maintained by a particular maintenance agency is always provided a fixed identifier and is never versioned. Therefore, providers can be added or removed without have to version the scheme. This scheme has no hierarchy, meaning that no organisation may define a relationship with another organisation in the scheme

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       OrganisationSchemeBaseType
(extension)
                                             OrganisationSchemeType
(restriction)
                                                   DataProviderSchemeType
```

Attributes:

```
id, urn?, uri?, validFrom?, validTo?, agencyID, isExternalReference?,
serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, DataProvider*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id (fixed: DATA_PROVIDERS)	IDType	The id is the identifier for the object.
urn	DataProviderSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
DataProvider	DataProviderType	DataProvider describes an organisation that produces data.

MetadataProviderSchemeType: MetadataProviderSchemeType defines a type of organisation scheme which contains only metadata providers. The metadata provider scheme maintained by a particular maintenance agency is always provided a fixed identifier and is never versioned. Therefore, providers can be added or removed without have to version the scheme. This scheme has no hierarchy, meaning that no organisation may define a relationship with another organisation in the scheme

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       OrganisationSchemeBaseType
(extension)
                                             OrganisationSchemeType
(restriction)
                                                   MetadataProviderSchemeType
```

Attributes:

```
id, urn?, uri?, validFrom?, validTo?, agencyID, isExternalReference?,
serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, MetadataProvider*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id (fixed: METADATA_PROVIDERS)	IDType	The id is the identifier for the object.
urn	MetadataProviderSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
MetadataProvider	MetadataProviderType	MetadataProvider describes an organisation that produces metadata .

OrganisationUnitSchemeType: OrganisationUnitSchemeType defines a type of organisation scheme which simply defines organisations and there parent child relationships. Organisations in this scheme are assigned no particular role, and may in fact exist within the other type of organisation schemes as well.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       OrganisationSchemeBaseType
(extension)
                                             OrganisationSchemeType
(restriction)
                                                   OrganisationUnitSchemeType
```

Attributes:

```
id, urn?, uri?, validFrom?, validTo?, agencyID, isExternalReference?,
serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, OrganisationUnit*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	OrganisationUnitSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
OrganisationUnit	OrganisationUnitType	OrganisationUnit describes a generic organisation, which serves not predefined role in SDMX.

AgencyType: AgencyType defines the structure of an agency description. The contacts defined for the organisation are specific to the agency role the organisation is serving.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           BaseOrganisationType (extension)
                                 OrganisationType (restriction)
                                       AgencyType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, Contact*
```

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	The id attribute holds the identification of the agency. The type of this id is restricted to the common:NCNNameIDType. This is necessary, since the agency identifier will be used as part of the name for simple types in data and metadata structure specific schemas and therefore must be compliant with the NCName type in XML Schema (see common:NCNameIDType for further details).
urn	AgencyUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Contact	ContactType	Contact describes a contact for the organisation,

DataConsumerType: DataConsumerType defines the structure of a data consumer description. The contacts defined for the organisation are specific to the data consumer role the organisation is serving.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           BaseOrganisationType (extension)
                                 OrganisationType (restriction)
                                       DataConsumerType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, Contact*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	DataConsumerUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Contact	ContactType	Contact describes a contact for the organisation,

DataProviderType: DataProviderType defines the structure of a data provider description. The contacts defined for the organisation are specific to the data provider role the organisation is serving.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           BaseOrganisationType (extension)
                                 OrganisationType (restriction)
                                       DataProviderType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, Contact*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	DataProviderUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Contact	ContactType	Contact describes a contact for the organisation,

MetadataProviderType: MetadataProviderType defines the structure of a metadata provider description. The contacts defined for the organisation are specific to the metadata provider role the organisation is serving.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           BaseOrganisationType (extension)
                                 OrganisationType (restriction)
                                       MetadataProviderType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, Contact*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MetadataProviderUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Contact	ContactType	Contact describes a contact for the organisation,

OrganisationUnitType: OrganisationUnitType defines the structure of an organisation unit description. In addition to general identification and contact information, an organisation unit can specify a relationship with another organisation unit from the same scheme which is its parent organisation.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           BaseOrganisationType (extension)
                                 OrganisationType (restriction)
                                       OrganisationUnitType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, Parent?, Contact*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	OrganisationUnitUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Parent	IDType	If the particular organisation scheme allows, an organisation may reference a parent organisation defined in the same scheme. This does not affect the identification of the organisation, but rather only serves to state the relationship.
Contact	ContactType	Contact describes a contact for the organisation,

ContactType: ContactType describes the structure of a contact's details.

Attributes:

```
id?
```

Content:

```
Name*, Department*, Role*, (Telephone | Fax | X400 | URI |
Email)*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id attribute is used to carry user id information for the contact.

Element Documentation:

Name	Type	Documentation
Name	TextType	Name is a reusable element, used for providing a human-readable name for an object.
Department	TextType	Department is designation of the organisational structure by a linguistic expression, within which the contact person works.
Role	TextType	Role is the responsibility of the contact person with respect to the object for which this person is the contact.
Telephone	xs:string	Telephone holds the telephone number for the contact person.
Fax	xs:string	Fax holds the fax number for the contact person.
X400	xs:string	X400 holds the X.400 address for the contact person.
URI	xs:anyURI	URI holds an information URL for the contact person.
Email	xs:string	Email holds the email address for the contact person.

Copyright © SDMx TWG 2020

ProvisionAgreementBaseType: ProvisionAgreementBaseType defines the base refinement of the ProvisionAgreementType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ProvisionAgreementBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	ProvisionAgreementUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

ProvisionAgreementType: ProvisionAgreementType describes the structure of a provision agreement. A provision agreement defines an agreement for a data provider to report data against a dataflow. Attributes which describe how the registry must behave when data or metadata is registered against this provision agreement are supplied.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ProvisionAgreementBaseType
(extension)
                                       ProvisionAgreementType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, Dataflow, DataProvider
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	ProvisionAgreementUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Dataflow	DataflowReferenceType	Dataflow provides a reference to a pre-existing dataflow in the registry. The reference is provided via a URN and/or a full set of reference fields.
DataProvider	DataProviderReferenceType	DataProvider provides a reference to a pre-existing data provider in the registry. The reference is provided via a URN and/or a full set of reference fields.

MetadataProvisionAgreementBaseType: MetadataProvisionAgreementBaseType defines the base refinement of the MetadataProvisionAgreementType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 MetadataProvisionAgreementBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MetadataProvisionAgreementUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

MetadataProvisionAgreementType: ProvisionAgreementType describes the structure of a provision agreement. A provision agreement defines an agreement for a data provider to report data or reference metadata against a flow. Attributes which describe how the registry must behave when data or metadata is registered against this provision agreement are supplied.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 MetadataProvisionAgreementBaseType
(extension)
                                       MetadataProvisionAgreementType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, Metadataflow,
MetadataProvider, Target*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MetadataProvisionAgreementUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Metadataflow	MetadataflowReferenceType	Metadataflow provides a reference to a pre-existing metadataflow in the registry. The reference is provided via a URN and/or a full set of reference fields.
MetadataProvider	MetadataProviderReferenceType	MetadataProvider provides a reference to a pre-existing metadata provider in the registry. The reference is provided via a URN and/or a full set of reference fields.
Target	WildcardUrnType	References identifiable structures to which the refernece metadata described by the metadata structure used by the metadaflow should be restricted to. These references may include wildcards for parts of the reference.

ProcessBaseType: ProcessBaseType defines the base refinement of the ProcessType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ProcessBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	ProcessUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

ProcessType: ProcessType describes the structure of a process, which is a scheme which defines or documents the operations performed on data in order to validate data or to derive new information according to a given set of rules. Processes occur in order, and will continue in order unless a transition dictates another step should occur.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ProcessBaseType (extension)
                                       ProcessType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, ProcessStep*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	ProcessUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
ProcessStep	ProcessStepType	ProcessStep defines a process step, which is a specific operation, performed on data in order to validate or to derive new information according to a given set of rules.

ProcessStepBaseType: ProcessStepBaseType is an abstract base type used as the basis for the ProcessStepType.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ProcessStepBaseType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	ProcessStepUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

ProcessStepType: ProcessStepType describes the structure of a process step. A nested process step is automatically sub-ordinate, and followed as the next step. If the following step is conditional, it should be referenced in a transition.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ProcessStepBaseType (extension)
                     ProcessStepType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, Input*, Output*,
Computation?, Transition*, ProcessStep*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	ProcessStepUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Input	InputOutputType	Input references an object which is an input to the process step.
Output	InputOutputType	Output references an object which is an output form the process step.
Computation	ComputationType	Computation describes the computations involved in the process, in any form desired by the user (these are informational rather than machine-actionable), and so may be supplied in multiple, parallel-language versions.
Transition	TransitionType	Transition describes the next process steps. Each transition in a process step should be evaluated, allowing for multiple process step branches from a single process step.
ProcessStep	ProcessStepType	ProcessStep defines a process step, which is a specific operation, performed on data in order to validate or to derive new information according to a given set of rules.

TransitionBaseType: TransitionBaseType defines the base refinement of the TransitionType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         TransitionBaseType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	TransitionUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.

TransitionType: TransitionType describes the details of a transition, which is an expression in a textual or formalised way of the transformation of data between two specific operations performed on the data.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         TransitionBaseType (extension)
               TransitionType
```

Attributes:

```
id?, urn?, uri?, localID?
```

Content:

```
Annotations?, Link*, TargetStep, Condition+
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	TransitionUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
localID	IDType	The localID attribute is an optional identification for the transition within the process.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
TargetStep	NestedIDType	TargetStep references a process step within the process that should be transitioned to, should the conditions described be met.
Condition	TextType	Condition is a textual description of the conditions to be met in order for the target step to be proceeded to. It is informational only (not machine-actionable), and may be supplied in multiple, parallel-language form.

Copyright © SDMx TWG 2020

ComputationType: ComputationType describes a computation in a process.

Derivation:

```
AnnotableType (extension)
   ComputationType
```

Attributes:

```
localID?, softwarePackage?, softwareLanguage?, softwareVersion?
```

Content:

```
Annotations?, Description+
```

Attribute Documentation:

Name	Type	Documentation
localID	IDType	The localID attribute is an optional identification for the computation within the process.
softwarePackage	xs:string	The softwarePackage attribute holds the name of the software package that is used to perform the computation.
softwareLanguage	xs:string	The softwareLanguage attribute holds the coding language that the software package used to perform the computation is written in.
softwareVersion	xs:string	The softwareVersion attribute hold the version of the software package that is used to perform that computation.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Description	TextType	Description describe the computation in any form desired by the user (these are informational rather than machine-actionable), and so may be supplied in multiple, parallel-language versions,

InputOutputType: InputOutputType describes the structure of an input or output to a process step. It provides a reference to the object that is the input or output.

Derivation:

```
AnnotableType (extension)
   InputOutputType
```

Attributes:

```
localID?
```

Content:

```
Annotations?, ObjectReference
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
localID	IDType	The localID attribute is an optional identification for the input or output within the process.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
ObjectReference	UrnReferenceType	ObjectReference is an abstract substitution head that references the object that is an input or output. It is substituted with a concrete reference to an explicit object type.

ReportingTaxonomyType: ReportingTaxonomyType describes the structure of a reporting taxonomy, which is a scheme which defines the composition structure of a data report where each component can be described by an independent structure or structure usage description.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       ReportingTaxonomyType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, ReportingCategory*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	ReportingTaxonomyUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
ReportingCategory	ReportingCategoryType	ReportingCateogry defines a reporting category, which is used to group structure usages into useful sub-packages.

ReportingCategoryBaseType: ReportingCategoryBaseType is an abstract base type that serves as the basis for the ReportingCategoryType.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           ReportingCategoryBaseType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, ReportingCategory*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	ReportingCategoryUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
ReportingCategory	ReportingCategoryType	ReportingCateogry defines a reporting category, which is used to group structure usages into useful sub-packages.

ReportingCategoryType: ReportingCategoryType describes the structure of a reporting category, which groups structure usages into useful sub-packages. Sub ordinate reporting categories can be nested within the category definition.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           ReportingCategoryBaseType (extension)
                                 ReportingCategoryType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, ReportingCategory*,
(StructuralMetadata* | ProvisioningMetadata*)
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	ReportingCategoryUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
ReportingCategory	ReportingCategoryType	ReportingCateogry defines a reporting category, which is used to group structure usages into useful sub-packages.
StructuralMetadata	StructureReferenceType	StructuralMetadata provides a reference for data structure definition and metadata structure definition references which are grouped in the reporting category. It is assumed that all structural metadata objects referenced from a category will be of the same type.
ProvisioningMetadata	StructureUsageReferenceType	ProvisioningMetadata provides a reference for dataflow and metadataflow references which are grouped in the reporting category. It is assumed that all provisioning metadata objects referenced from a category will be of the same type.

ItemSchemeMapBaseType: ItemSchemeMapBaseType is an abstract base type which forms the basis for the ItemSchemeMapType.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ItemSchemeMapBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MaintainableUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

ItemSchemeMapType: ItemSchemeMapType is an abstract base type which forms the basis for mapping items between item schemes of the same type.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ItemSchemeMapBaseType (extension)
                                       ItemSchemeMapType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, Source, Target, ItemMap*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MaintainableUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Source	MaintainableUrnReferenceType	Source provides a reference to the item scheme which items are mapped from.
Target	MaintainableUrnReferenceType	Target provides a reference to the item scheme which items are mapped to.
ItemMap	SingleValueMappingType	

OrganisationSchemeMapType: OrganisationSchemeMapType defines the structure of a map which identifies relationships between organisations in different organisation schemes.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ItemSchemeMapBaseType (extension)
                                       ItemSchemeMapType
(restriction)
                                             OrganisationSchemeMapType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, Source, Target, ItemMap*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	OrganisationSchemeMapUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Source	OrganisationSchemeReferenceType	Source provides a reference to the item scheme which items are mapped from.
Target	OrganisationSchemeReferenceType	Target provides a reference to the item scheme which items are mapped to.
ItemMap	SingleValueMappingTy pe	

CategorySchemeMapType: CategorySchemeMapType defines the structure of a map which identifies relationships between categories in different category schemes.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ItemSchemeMapBaseType (extension)
                                       ItemSchemeMapType
(restriction)
                                             CategorySchemeMapType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, Source, Target, ItemMap*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	CategorySchemeMapUrn Type	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Source	CategorySchemeReferenceType	Source provides a reference to the item scheme which items are mapped from.
Target	CategorySchemeReferenceType	Target provides a reference to the item scheme which items are mapped to.
ItemMap	SingleValueMappingTy pe	

ConceptSchemeMapType: ConceptSchemeMapType defines the structure of a map which identifies relationships between concepts in different concept schemes.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ItemSchemeMapBaseType (extension)
                                       ItemSchemeMapType
(restriction)
                                             ConceptSchemeMapType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, Source, Target, ItemMap*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	ConceptSchemeMapUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Source	ConceptSchemeReferenceType	Source provides a reference to the item scheme which items are mapped from.
Target	ConceptSchemeReferenceType	Target provides a reference to the item scheme which items are mapped to.
ItemMap	SingleValueMappingTy pe	

ReportingTaxonomyMapType: ReportingTaxonomyMapType defines the structure of a map which identifies relationships between reporting categories in different reporting taxonomies.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 ItemSchemeMapBaseType (extension)
                                       ItemSchemeMapType
(restriction)
                                             ReportingTaxonomyMapType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, Source, Target, ItemMap*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	ReportingTaxonomyMapUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Source	ReportingTaxonomyReferenceType	Source provides a reference to the item scheme which items are mapped from.
Target	ReportingTaxonomyReferenceType	Target provides a reference to the item scheme which items are mapped to.
ItemMap	SingleValueMappingTy pe	

StructureMapBaseType: StructureMapBaseType defines the base refinement of the StructureMapType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 StructureMapBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	StructureMapUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

StructureMapType: StructureMapType defines the structure for mapping components of one structure to components of another structure. A structure may be referenced directly meaning the map applies wherever the structure is used, or it may be a reference via a structure usage meaning the map only applies within the context of that usage. Using the related structures, one can make extrapolations between maps. For example, if key families, A, B, and C, are all grouped in a related structures container, then a map from data structure A to C and a map from data structure B to C could be used to infer a relation between data structure A to C.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 StructureMapBaseType (extension)
                                       StructureMapType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, Source, Target, EpochMap*,
DatePatternMap*, FrequencyFormatMapping*, ComponentMap*,
FixedValueMap*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	StructureMapUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Source	StructureOrUsageReferenceType	Source provides a reference to a structure (data or metadata) or a structure usage (dataflow or metadataflow) from which components defined by the actual structure are to mapped.
Target	StructureOrUsageReferenceType	Target provides a reference to a structure (data or metadata) or a structure usage (dataflow or metadataflow) to which components from the source are to mapped.
EpochMap	EpochMapType	Provides the ability to map source to target date formats. The source date is described as the number of epochs since a point in time, where the duration of each epoch is defined, e.g. number of milliseconds since 1970.
DatePatternMap	DatePatternMapType	Provides the ability to map source to target date formats. The source date is described as a pattern (for example MM-YYYY).
FrequencyFormatMapping	FrequencyFormatMappingType	
ComponentMap	ComponentMapType	ComponentMap defines the relationship between the components of the source and target structures, including information on how the value from the source component relates to values in the target component.
FixedValueMap	FixedValueMapType	FixedValueMap defines a fixed value for a source or target component in the mapping.

FixedValueMapType: FixedValueMapType defines the structure for providing a fixed value for a source or target component.

Derivation:

```
AnnotableType (extension)
   FixedValueMapType
```

Content:

```
Annotations?, (Source | Target), Value*
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Source	IDType	
Target	IDType	
Value	xs:string	The fixed value for the component.

ComponentMapType: ComponentMapType defines the structure for relating a component in a source structure to a component in a target structure.

Derivation:

```
AnnotableType (extension)
   ComponentMapType
```

Content:

```
Annotations?, Source+, Target+, RepresentationMap?
```

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Source	IDType	
Target	IDType	
RepresentationMap	RepresentationMapReferenceType	

FrequencyFormatMappingBaseType: FrequencyFormatMappingBaseType defines the base refinement of the FrequencyFormatMappingType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         FrequencyFormatMappingBaseType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	FrequencyFormatMappingUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.

FrequencyFormatMappingType:

Derivation:

```
AnnotableType (extension)
   IdentifiableType (restriction)
         FrequencyFormatMappingBaseType (extension)
               FrequencyFormatMappingType
```

Attributes:

```
id?, urn?, uri?
```

Content:

```
Annotations?, Link*, FrequencyId, DatePattern
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	FrequencyFormatMappingUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
FrequencyId	IDType	
DatePattern	xs:string	

DateMapType:

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         DateMapType
```

Attributes:

```
id?, urn?, uri?, resolvePeriod?
```

Content:

```
Annotations?, Link*, (Source, Target)+, ( (FrequencyDimension,
MappedFrequencies*) | TargetFrequencyID)
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
resolvePeriod	ResolvePeriodType	Indicates the point in time to resolve to when mapping from low fequency periods to higher frequency periods.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Source	IDType	
Target	IDType	
FrequencyDimension	IDType	
MappedFrequencies	IDType	
TargetFrequencyID	IDType	

EpochMapBaseType: EpochMapBaseType defines the base refinement of the EpochMapType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         DateMapType (restriction)
               EpochMapBaseType
```

Attributes:

```
id?, urn?, uri?, resolvePeriod?
```

Content:

```
Annotations?, Link*, (Source, Target)+, ( (FrequencyDimension,
MappedFrequencies*) | TargetFrequencyID)
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	EpochMapUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
resolvePeriod	ResolvePeriodType	Indicates the point in time to resolve to when mapping from low fequency periods to higher frequency periods.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Source	IDType	
Target	IDType	
FrequencyDimension	IDType	
MappedFrequencies	IDType	
TargetFrequencyID	IDType	

EpochMapType:

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         DateMapType (restriction)
               EpochMapBaseType (extension)
                     EpochMapType
```

Attributes:

```
id?, urn?, uri?, resolvePeriod?, basePeriod, epochPeriod
```

Content:

```
Annotations?, Link*, (Source, Target)+, ( (FrequencyDimension,
MappedFrequencies*) | TargetFrequencyID)
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	EpochMapUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
resolvePeriod	ResolvePeriodType	Indicates the point in time to resolve to when mapping from low fequency periods to higher frequency periods.
basePeriod	xs:string	
epochPeriod	EpochPeriodType	

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Source	IDType	
Target	IDType	
FrequencyDimension	IDType	
MappedFrequencies	IDType	
TargetFrequencyID	IDType	

DatePatternMapBaseType: DatePatternMapBaseType defines the base refinement of the DatePatternMapType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         DateMapType (restriction)
               DatePatternMapBaseType
```

Attributes:

```
id?, urn?, uri?, resolvePeriod?
```

Content:

```
Annotations?, Link*, (Source, Target)+, ( (FrequencyDimension,
MappedFrequencies*) | TargetFrequencyID)
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	DatePatternMapUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
resolvePeriod	ResolvePeriodType	Indicates the point in time to resolve to when mapping from low fequency periods to higher frequency periods.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Source	IDType	
Target	IDType	
FrequencyDimension	IDType	
MappedFrequencies	IDType	
TargetFrequencyID	IDType	

DatePatternMapType:

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         DateMapType (restriction)
               DatePatternMapBaseType (extension)
                     DatePatternMapType
```

Attributes:

```
id?, urn?, uri?, resolvePeriod?, sourcePattern, locale
```

Content:

```
Annotations?, Link*, (Source, Target)+, ( (FrequencyDimension,
MappedFrequencies*) | TargetFrequencyID)
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	DatePatternMapUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
resolvePeriod	ResolvePeriodType	Indicates the point in time to resolve to when mapping from low fequency periods to higher frequency periods.
sourcePattern	xs:string	Describes the source date using conventions for describing years, months, days, etc.
locale	xs:string	The locale on which the input will be parsed according to the pattern.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Source	IDType	
Target	IDType	
FrequencyDimension	IDType	
MappedFrequencies	IDType	
TargetFrequencyID	IDType	

RepresentationMapBaseType: RepresentationMapBaseType defines the base refinement of the RepresentationMapType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 RepresentationMapBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	RepresentationMapUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

RepresentationMapType:

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 RepresentationMapBaseType
(extension)
                                       RepresentationMapType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*, (SourceCodelist |
SourceDataType)+, (TargetCodelist | TargetDataType)+,
RepresentationMapping*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	RepresentationMapUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
SourceCodelist	AnyCodelistReferenceType	
SourceDataType	DataType	
TargetCodelist	AnyCodelistReferenceType	
TargetDataType	DataType	
RepresentationMapping	ValueMappingType	

SingleValueMappingType: SingleValueMappingType defines a mapping with a single source and optional target.

Derivation:

```
AnnotableType (extension)
   SingleValueMappingType
```

Attributes:

```
validFrom?, validTo?
```

Content:

```
Annotations?, SourceValue, TargetValue?
```

Attribute Documentation:

Name	Type	Documentation
validFrom	xs:date	
validTo	xs:date	

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
SourceValue	MappedValueType	There should be a source value for each source represention (e.g. codelist, data type). Source values can be pattern matched by using regular expression or substrings using start/end indexes.
TargetValue	xs:string	The target value(s) is always an absolute string. However, if source value is a regular expression, the target value can output the capture group from the source.

ValueMappingType: ValueMappingType defines a mapping of multiple sources to multiple targets.

Derivation:

```
AnnotableType (extension)
   ValueMappingType
```

Attributes:

```
validFrom?, validTo?
```

Content:

```
Annotations?, SourceValue+, TargetValue*
```

Attribute Documentation:

Name	Type	Documentation
validFrom	xs:date	
validTo	xs:date	

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
SourceValue	MappedValueType	There should be a source value for each source represention (e.g. codelist, data type). Source values can be pattern matched by using regular expression or substrings using start/end indexes.
TargetValue	xs:string	The target value(s) is always an absolute string. However, if source value is a regular expression, the target value can output the capture group from the source.

MappedValueType:

Derivation:

```
xs:anySimpleType (restriction)
   xs:string (extension)
         MappedValueType
```


Copyright © SDMx TWG 2020

Attributes:

```
isRegEx?, startIndex?, endIndex?
```

Content:

Attribute Documentation:

Name	Type	Documentation
isRegEx	xs:boolean	
startIndex	xs:int	
endIndex	xs:int	

VtlDefinitionSchemeType: VtlDefinitionSchemeType is an abstract extension of the ItemSchemeType for VTL schemes.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (extension)
                                       VtlDefinitionSchemeType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?, vtlVersion
```

Content:

```
Annotations?, Link*, Name+, Description*, *Item**
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MaintainableUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.
vtlVersion	xs:string	Identifies the VTL version to which the items in the defined scheme comply. Note that definition schemes can only reference definition schemes using the same VTL version.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Item	*ItemType*	Item is an abstract element that serves as a substitution head for all items in an item scheme, including those items nested within other items. Concrete instances of this must use a concrete instance of ItemType.

TransformationSchemeBaseType: TransformationSchemeBaseType is an abstract base type for the TransformationSchemeType. It restricts the item types to be only transformations.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (extension)
                                       VtlDefinitionSchemeType
(restriction)
                                             TransformationSchemeBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?, vtlVersion
```

Content:

```
Annotations?, Link*, Name+, Description*, Transformation*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	TransformationSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.
vtlVersion	xs:string	Identifies the VTL version to which the items in the defined scheme comply. Note that definition schemes can only reference definition schemes using the same VTL version.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Transformation	TransformationType	Transformation describes the details of a single transformation within a transformation scheme.

TransformationSchemeType: TransformationSchemeType describes the structure of a transformation scheme. A transformation scheme contains a set of transformations to be executed together (in the same run). It can contain any number of transformations that produce any number of results.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (extension)
                                       VtlDefinitionSchemeType
(restriction)
                                             TransformationSchemeBaseType
(extension)
                                                   TransformationSchemeType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?, vtlVersion
```

Content:

```
Annotations?, Link*, Name+, Description*, Transformation*,
VtlMappingScheme?, NamePersonalisationScheme?, CustomTypeScheme?,
RulesetScheme*, UserDefinedOperatorScheme*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	TransformationSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.
vtlVersion	xs:string	Identifies the VTL version to which the items in the defined scheme comply. Note that definition schemes can only reference definition schemes using the same VTL version.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Transformation	TransformationType	Transformation describes the details of a single transformation within a transformation scheme.
VtlMappingScheme	VtlMappingSchemeReferenceType	References a VTL mapping scheme which defines aliases for given SDMX artefacts that are used in the transformations as well as the mapping methods used when converting between SDMX and VTL data structures. All aliases must be defined in the referenced scheme. This also must be used if the basic mapping methods are not used.
NamePersonalisationScheme	NamePersonalisationSchemeReferenceType	References a name personalisation scheme, which defines the overriding of some standard VTL names (to be assigned to some measures and/or attributes of the data structure) with some corresponding personalised names. This must be used if transformations within a transformation scheme personalise standard names. All personalisations must be defined in the referenced scheme.
CustomTypeScheme	CustomTypeSchemeReferenceType	References a custom type scheme which defines custom conversions of VTL scalar types to SDMX data types. This must be used if custom type conversions are used in the transformations defined in a transformation scheme. All custom conversions must be defined in the referenced scheme.
RulesetScheme	RulesetSchemeReferenceType	References a ruleset scheme that defines one or more previously defined rulesets which can be invoked by VTL operators. If a transformation defined in a transformation scheme refers to a ruleset, the scheme in which the ruleset is defined must be referenced here.
UserDefinedOperatorScheme	UserDefinedOperatorSchemeReferenceType	References a user defined operator scheme that defines one or more user defined operators used by the transformations defined in a transformation scheme. If a transformation in a transformation scheme refers to a user defined operator, the scheme in which the user defined operator is defined must be referenced here.

TransformationBaseType: TransformationBaseType defines the base refinement of the TransformationType. Its purpose is to retrict the urn attribute.

Derivation:

Copyright © SDMx TWG 2020

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           UnnestedItemType (restriction)
                                 TransformationBaseType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	TransformationUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

TransformationType: TransformationType defines the structure of a transformation. A transformation is an assignment of a VTL expression to a specific result.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           UnnestedItemType (restriction)
                                 TransformationBaseType (extension)
                                       TransformationType
```

Attributes:

```
id, urn?, uri?, isPersistent
```

Content:

Copyright © SDMx TWG 2020

```
Annotations?, Link*, Name+, Description*, Expression, Result
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	TransformationUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
isPersistent	xs:boolean	Indicates if the the result is permanently stored. A persistent result (value of true) can be used by transformation defined in other transformation schemes, but a non-persistent result (value of false) can only be used by transformations within the same transformation scheme.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Expression	xs:string	The right-hand side of the VTL statement. This is expression that is executed for this transformation. It include references to operands and other artefacts. The expression may contain references to SDMX artefacts using the reduced URN format; see Section 6 SDMX Standards (“SDMX Technical Notes”), 10.2.3 (“Abbreviation of the URN”).
Result	xs:string	The left-hand side of the VTL statement. This identifies the result artefact, which may be used in subsequent transformations. If the result is an SDMX artefact, the is expressed using the alias; see Section 6 SDMX Standards (“SDMX Technical Notes”), 10.2.3 (“Abbreviation of the URN”).

VtlMappingSchemeType: VtlMappingSchemeType defines a set of mappings between SDMX and VTL.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (restriction)
                                       VtlMappingSchemeType
```

Attributes:

Copyright © SDMx TWG 2020

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?
```

Content:

```
Annotations?, Link*, Name+, Description*, VtlMapping*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	VtlMappingSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
VtlMapping	VtlMappingType	VtlMapping details a mapping between SDMX and VTL transformation.

VtlMappingBaseType: VtlMappingBaseType defines the base refinement of the VtlMappingType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           UnnestedItemType (restriction)
                                 VtlMappingBaseType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	VtlMappingUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

VtlMappingType: VtlMappingType defines the structure of a single mapping between the reference to a SDMX artefact made from VTL transformations, rulesets, user defined operators and the corresponding SDMX structure identifier. These are used to assign an alternative names to SDMX Dataflows, Codelists, Concept Schemes, or Concepts. Although are distinct sub-classes in the Information Model, this structure serves to express them all. The references SDMX artefact serves to distinguish which type of sub-class (VtlDatflowMapping or VtlCodelistMapping, or VtlConceptMapping) is being described. When this is used to assign an alias for a SDMX Dataflow, this can also be used to indicate the methods used to convert the data structure from SDMX to VTL and vice-versa. Finally, this can be used to override the deault Basic mapping methods used for Dataflows by utilizing the GenericDataflow element in place of a reference to a specific Dataflow.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           UnnestedItemType (restriction)
                                 VtlMappingBaseType (extension)
                                       VtlMappingType
```

Attributes:

```
id, urn?, uri?, alias
```

Content:

```
Annotations?, Link*, Name+, Description*, ( ( (Dataflow |
GenericDataflow), ToVtlMapping?, FromVtlMapping?) | Codelist |
Concept)
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	VtlMappingUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
alias	xs:string	The alias used to refer to the reference SDMX artefact in the transformations. This must be unique within the mapping scheme in which it is defined.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Dataflow	DataflowReferenceType	A reference to the SDMX Dataflow that the mapping is defined for. This is used to create a VtlDataflowMapping. In this case, it is possible to specify the mapping details to and from VTL.
GenericDataflow	EmptyType	Indicates that the ToVtlMapping and FromVtlMapping are the default mapping methods to be applied, overriding the Basic methods if necessary. The mapping methods in this case would be applied to any dataflows that are not explicitly defined in other VtlMappings.
ToVtlMapping	ToVtlMappingType	Describes the mapping from a SDMX dataflow to a VTL data structure.
FromVtlMapping	FromVtlMappingType	Describes the mapping to a SDMX dataflow from a VTL data structure.
Codelist	CodelistReferenceType	A reference to the SDMX Codelist that the mapping is defined for. This is used to create a VtlCodelistMapping.
Concept	ConceptReferenceType	A reference to the SDMX Concept that the mapping is defined for. This is used to create a VtlConceptMapping.

ToVtlMappingType: ToVtlMappingType defines the mapping method and filter used when mapping from SDMX to VTL.

Attributes:

```
method?
```

Content:

```
ToVtlSubSpace?
```

Attribute Documentation:

Name	Type	Documentation
method	ToVtlMappingMethodType	The mapping method used when mapping from SDMX to VTL. This is typically a StandardToVtlMappingMethodType, but can be any other value to allow for non-standard methods. The implied default is Basic.

Element Documentation:

Name	Type	Documentation
ToVtlSubSpace	SpaceKeyType	Identfies a sub space of the mapped dataflow that the mapping applies to. This is a collection of references to the dimensions that make up the space.

FromVtlMappingType: FromVtlMappingType defines the mapping method and filter used when mapping from VTL to SDMX.

Copyright © SDMx TWG 2020

Attributes:

```
method?
```

Content:

```
FromVtlSuperSpace?
```

Attribute Documentation:

Name	Type	Documentation
method	FromVtlMappingMethodType	The mapping method used when mapping from VTL to SDMX. This is typically a StandardFromVtlMappingMethodType, but can be any other value to allow for non-standard methods. The implied default is Basic for single-measure VTL data structures and Unpivot for multi-meausre VTL data structures.

Element Documentation:

Name	Type	Documentation
FromVtlSuperSpace	SpaceKeyType	Identfies a super space of the mapped dataflow that the mapping applies to. This is a collection of references to the dimensions that make up the space.

SpaceKeyType: SpaceKey defines the structure of a super- or sub- space for a SDMX Dataflow. It is a collection of references to the dimensions that make up the space.

Content:

```
Key+
```

Element Documentation:

Name	Type	Documentation
Key	SingleNCNameIDType	A reference to a dimension by its identifier.

NamePersonalisationSchemeType: NamePersonalisationSchemeType defines a set of personalisations of VTL standard names that are used in a set of transformations.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (extension)
                                       VtlDefinitionSchemeType
(restriction)
                                             NamePersonalisationSchemeType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?, vtlVersion
```

Content:

```
Annotations?, Link*, Name+, Description*, NamePersonalisation*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	NamePersonalisationSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.
vtlVersion	xs:string	Identifies the VTL version to which the items in the defined scheme comply. Note that definition schemes can only reference definition schemes using the same VTL version.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
NamePersonalisation	NamePersonalisationType	NamePersonalisation details a name personalisation that is used in a transformation.

NamePersonalisationBaseType: NamePersonalisationBaseType defines the base refinement of the NamePersonalisationType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           UnnestedItemType (restriction)
                                 NamePersonalisationBaseType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	NamePersonalisationUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

NamePersonalisationType: NamePersonalisationType defines the structure of a name personalisation. A name personalisation is is used in place of a standard VTL name in some VTL operations.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           UnnestedItemType (restriction)
                                 NamePersonalisationBaseType
(extension)
                                       NamePersonalisationType
```

Attributes:

```
id, urn?, uri?, vtlArtefact
```

Content:

```
Annotations?, Link*, Name+, Description*, VtlDefaultName,
PersonalisedName
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	NamePersonalisationUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
vtlArtefact	xs:string	Identifies the type of VTL model artefact that is being personalised. In VTL 2.0, this is valuedomain or variable.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
VtlDefaultName	xs:string	Provides the VTL standard name that is being personalised.
PersonalisedName	xs:string	Provides the personalised name that is used in place of the VTL standard name in the transformation expressions.

RulesetSchemeBaseType: RulesetSchemeBaseType is an abstract base type for the RulesetSchemeType. It restricts the item types to be only rulesets.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (extension)
                                       VtlDefinitionSchemeType
(restriction)
                                             RulesetSchemeBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?, vtlVersion
```

Content:

```
Annotations?, Link*, Name+, Description*, Ruleset*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	RulesetSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.
vtlVersion	xs:string	Identifies the VTL version to which the items in the defined scheme comply. Note that definition schemes can only reference definition schemes using the same VTL version.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Ruleset	RulesetType	Ruleset details a ruleset within a ruleset scheme.

RulesetSchemeType: RulesetSchemeType defines a collection of rulesets that are used in transformations.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (extension)
                                       VtlDefinitionSchemeType
(restriction)
                                             RulesetSchemeBaseType
(extension)
                                                   RulesetSchemeType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?, vtlVersion
```

Content:

```
Annotations?, Link*, Name+, Description*, Ruleset*, VtlMappingScheme?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	RulesetSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.
vtlVersion	xs:string	Identifies the VTL version to which the items in the defined scheme comply. Note that definition schemes can only reference definition schemes using the same VTL version.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
Ruleset	RulesetType	Ruleset details a ruleset within a ruleset scheme.
VtlMappingScheme	VtlMappingSchemeReferenceType	References a VTL mapping scheme which defines aliases for given SDMX artefacts that are used in the rulesets. Rulesets defined on value domains reference Codelists or Concept Schemes (the latter in VTL are considered as the Value Domains of the variables corresponding to the SDMX Measure Dimensions). The rulesets defined on variables reference Concepts (for which a definite representation is assumed). Therefore, a ruleset should only refer to Codelists, Concept Schemes, and Concepts.

RulesetBaseType: RulesetBaseType defines the base refinement of the RulesetType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           UnnestedItemType (restriction)
                                 RulesetBaseType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	RulesetUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

RulesetType: RulesetType defines the structure of a ruleset. A ruleset is a persistent set of rules which can be invoked by using appropriate VTL operators.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           UnnestedItemType (restriction)
                                 RulesetBaseType (extension)
                                       RulesetType
```

Attributes:

```
id, urn?, uri?, rulesetType, rulesetScope
```

Content:

```
Annotations?, Link*, Name+, Description*, RulesetDefinition
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	RulesetUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
rulesetType	xs:string	The VTL type of the ruleset. In VTL 2.0, this is datapoint or hierarchical
rulesetScope	xs:string	This model artefact on which the ruleset is defined. In VTL 2.0, this is value domain or variable.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
RulesetDefinition	xs:string	A VTL statement for the definition of a ruleset. This must conform to the syntax of the VTL definition language.

UserDefinedOperatorSchemeBaseType: UserDefinedOperatorSchemeBaseType is an abstract base type for the UserDefinedOperatorSchemeType. It restricts the item types to be only user defined operators.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (extension)
                                       VtlDefinitionSchemeType
(restriction)
                                             UserDefinedOperatorSchemeBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?, vtlVersion
```

Content:

```
Annotations?, Link*, Name+, Description*, UserDefinedOperator*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UserDefinedOperatorSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.
vtlVersion	xs:string	Identifies the VTL version to which the items in the defined scheme comply. Note that definition schemes can only reference definition schemes using the same VTL version.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
UserDefinedOperator	UserDefinedOperatorType	UserDefinedOperator details a user defined operators within a user defined operator scheme.

UserDefinedOperatorSchemeType: UserDefinedOperatorSchemeType defines a collection of user defined operators that are used in transformations.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (extension)
                                       VtlDefinitionSchemeType
(restriction)
                                             UserDefinedOperatorSchemeBaseType
(extension)
                                                   UserDefinedOperatorSchemeType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?, vtlVersion
```

Content:

```
Annotations?, Link*, Name+, Description*, UserDefinedOperator*,
VtlMappingScheme?, RulesetScheme*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UserDefinedOperatorSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.
vtlVersion	xs:string	Identifies the VTL version to which the items in the defined scheme comply. Note that definition schemes can only reference definition schemes using the same VTL version.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
UserDefinedOperator	UserDefinedOperatorType	UserDefinedOperator details a user defined operators within a user defined operator scheme.
VtlMappingScheme	VtlMappingSchemeReferenceType	References a VTL mapping scheme which defines aliases for given SDMX artefacts that are used in the user defined operators. Although the VTL user defined operators are conceived to be defined on generic operands, so that the specific artefacts to be manipulated are passed as parameters at the invocation, it is also possible that they reference specific SDMX artefacts like Dataflows, Codelists and ConceptSchemes. In this case, the mapping schemes referenced here define the mappings to those artefacts.
RulesetScheme	RulesetSchemeReferenceType	References a ruleset scheme defining rulesets utilized in the user defined operators.

UserDefinedOperatorBaseType: UserDefinedOperatorBaseType defines the base refinement of the UserDefinedOperatorType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           UnnestedItemType (restriction)
                                 UserDefinedOperatorBaseType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UserDefinedOperatorUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

UserDefinedOperatorType: UserDefinedOperatorType defines the structure of a user defined operator. A user defined operator is a custom VTL operator (not existing in the standard library) that extends the VTL standard library for specific purposes. In addition to its identification and name, and definition of the operator must be provided.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           UnnestedItemType (restriction)
                                 UserDefinedOperatorBaseType
(extension)
                                       UserDefinedOperatorType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, OperatorDefinition
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	UserDefinedOperatorUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
OperatorDefinition	xs:string	A VTL statement for the definition of a new operator: it specifies the operator name, its parameters and their data types, the VTL expression that defines its behaviour.

CustomTypeSchemeType: CustomTypeSchemeType defines a collection of custom types that are used in transformations.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (extension)
                                 ItemSchemeType (extension)
                                       VtlDefinitionSchemeType
(restriction)
                                             CustomTypeSchemeType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, isPartial?, vtlVersion
```

Content:

```
Annotations?, Link*, Name+, Description*, CustomType*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	CustomTypeSchemeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
isPartial (default: false)	xs:boolean	The isPartial, if true, indicates that only the relevant portion of the item scheme is being communicated. This is used in cases where a codelist is returned for a data structure in the context of a constraint.
vtlVersion	xs:string	Identifies the VTL version to which the items in the defined scheme comply. Note that definition schemes can only reference definition schemes using the same VTL version.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
CustomType	CustomTypeType	CustomType details a custom type within a custom type scheme.

CustomTypeBaseType: CustomTypeBaseType defines the base refinement of the CustomTypeType. Its purpose is to retrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           UnnestedItemType (restriction)
                                 CustomTypeBaseType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	CustomTypeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

CustomTypeType: CustomTypeType defines the structure of a custom type. A custom type specifies a custom conversion for a VTL scalar type to a resulting data type. This conversion overrides the default conversion.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (restriction)
               ItemBaseType (extension)
                     ItemType (restriction)
                           UnnestedItemType (restriction)
                                 CustomTypeBaseType (extension)
                                       CustomTypeType
```

Attributes:

```
id, urn?, uri?
```

Content:

```
Annotations?, Link*, Name+, Description*, VtlScalarType, DataType,
VtlLiteralFormat?, OutputFormat?, NullValue?
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	CustomTypeUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
VtlScalarType	xs:string	Identifies the VTL scalar type that is to be converted to an resulting output data type.
DataType	CustomTypeDataType	Identifies the resulting output data type the VTL scalar type is to be converted to. If this is an SDMX data type, it must use the proper SimpleDataType enumeration value. For all other data types, a string value can be used to identify the type.
VtlLiteralFormat	xs:string	The format in which the literals of the VTL scalar type are expressed in the transformations. This is only needed if the format is different than the output format expressed by means of the VTL type.
OutputFormat	xs:string	The format the VTL scalar type has to assume (e.g. YYYY-MM-DD; see VTL specifications), both for the literals in the VTL expressions and for the conversion to the output.
NullValue	xs:string	The value to be produced in the output of the conversion when a component has a null value.

Simple Types

CodeDataType: CodeDataType is a restriction of the basic data types that are applicable to codes. Although some of the higher level time period formats are perimitted, it should be noted that any value which contains time (which includes a time zone offset) is not allowable as a code identifier.

Derived by restriction of SimpleDataType .

Copyright © SDMx TWG 2020

Enumerations:

Copyright © SDMx TWG 2020

Value	Documentation
String	A string datatype corresponding to W3C XML Schema’s xs:string datatype.
Alpha	A string datatype which only allows for the simple aplhabetic charcter set of A-Z, a-z.
AlphaNumeric	A string datatype which only allows for the simple alphabetic character set of A-Z, a-z plus the simple numeric character set of 0-9.
Numeric	A string datatype which only allows for the simple numeric character set of 0-9. This format is not treated as an integer, and therefore can having leading zeros.
BigInteger	An integer datatype corresponding to W3C XML Schema’s xs:integer datatype.
Integer	An integer datatype corresponding to W3C XML Schema’s xs:int datatype.
Long	A numeric datatype corresponding to W3C XML Schema’s xs:long datatype.
Short	A numeric datatype corresponding to W3C XML Schema’s xs:short datatype.
Boolean	A datatype corresponding to W3C XML Schema’s xs:boolean datatype.
URI	A datatype corresponding to W3C XML Schema’s xs:anyURI datatype.
Count	A simple incrementing Integer type. The isSequence facet must be set to true, and the interval facet must be set to “1”.
InclusiveValueRange	This value indicates that the startValue and endValue attributes provide the inclusive boundaries of a numeric range of type xs:decimal.
ExclusiveValueRange	This value indicates that the startValue and endValue attributes provide the exclusive boundaries of a numeric range, of type xs:decimal.
Incremental	This value indicates that the value increments according to the value provided in the interval facet, and has a true value for the isSequence facet.
ObservationalTimePeriod	Observational time periods are the superset of all time periods in SDMX. It is the union of the standard time periods (i.e. Gregorian time periods, the reporting time periods, and date time) and a time range.
StandardTimePeriod	Standard time periods is a superset of distinct time period in SDMX. It is the union of the basic time periods (i.e. the Gregorian time periods and date time) and the reporting time periods.
BasicTimePeriod	BasicTimePeriod time periods is a superset of the Gregorian time periods and a date time.
GregorianTimePeriod	Gregorian time periods correspond to calendar periods and are represented in ISO-8601 formats. This is the union of the year, year month, and date formats.
GregorianYear	A Gregorian time period corresponding to W3C XML Schema’s xs:gYear datatype, which is based on ISO-8601.
GregorianYearMonth	A time datatype corresponding to W3C XML Schema’s xs:gYearMonth datatype, which is based on ISO-8601.
GregorianDay	A time datatype corresponding to W3C XML Schema’s xs:date datatype, which is based on ISO-8601.
ReportingTimePeriod	Reporting time periods represent periods of a standard length within a reporting year, where to start of the year (defined as a month and day) must be defined elsewhere or it is assumed to be January 1. This is the union of the reporting year, semester, trimester, quarter, month, week, and day.
ReportingYear	A reporting year represents a period of 1 year (P1Y) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingYearType.
ReportingSemester	A reporting semester represents a period of 6 months (P6M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingSemesterType.

Copyright © SDMx TWG 2020

Value	Documentation
ReportingTrimester	A reporting trimester represents a period of 4 months (P4M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingTrimesterType.
ReportingQuarter	A reporting quarter represents a period of 3 months (P3M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingQuarterType.
ReportingMonth	A reporting month represents a period of 1 month (P1M) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingMonthType.
ReportingWeek	A reporting week represents a period of 7 days (P7D) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingWeekType.
ReportingDay	A reporting day represents a period of 1 day (P1D) from the start date of the reporting year. This is expressed as using the SDMX specific ReportingDayType.
Month	A time datatype corresponding to W3C XML Schema's xs:gMonth datatype.
MonthDay	A time datatype corresponding to W3C XML Schema's xs:gMonthDay datatype.
Day	A time datatype corresponding to W3C XML Schema's xs:gDay datatype.
Duration	A time datatype corresponding to W3C XML Schema's xs:duration datatype.

WildcardedMemberValueType: WildcardedMemberValueType allows for an optional wildcard characters ('%') in an identifier.

Derived by restriction of xs:string .
Regular Expression Pattern: [A-Za-z0-9_@\$-%]+

GeoCodelistTypeType: GeoCodelistTypeType defines an enumeration of the speicfic types of geographic codelists.

Derived by restriction of xs:string .

Enumerations:

Value	Documentation
GeographicCodelist	
GeoGridCodelist	

ConstraintRoleType: ConstraintRoleType defines a list of roles for a content constraint. A constraint can state which data is present or which content is allowed for the constraint attachment.

Derived by restriction of xs:string .

Enumerations:

Value	Documentation
Allowed	The constraint contains the allowed values for attachable object.
Actual	The constraints contains the actual data present for the attachable object.

SimpleOperatorType: SimpleOperatorType provides an enumeration of simple operators to be applied to any value.

Derived by restriction of xs:string .

Copyright © SDMx TWG 2020

Enumerations:

Value	Documentation
notEqual	(!=) - value must not be equal to the value supplied.
equal	(=) - value must be exactly equal to the value supplied.

RangeOperatorType: RangeOperatorType provides an enumeration of range operators to be applied to an ordered value.

Derived by restriction of xs:string .

Enumerations:

Value	Documentation
greaterThanOrEqual	(>=) - value must be greater than or equal to the value supplied.
lessThanOrEqual	(\<=) - value must be less than or equal to the value supplied.
greaterThan	(>) - value must be greater than the value supplied.
lessThan	(\<) - value must be less than the value supplied.

TextSearchOperatorType: TextSearchOperatorType provides an enumeration of text search operators.

Derived by restriction of xs:string .

Enumerations:

Value	Documentation
contains	The text being searched must contain the supplied text.
startsWith	The text being searched must start with the supplied text.
endsWith	The text being searched must end with the supplied text.
doesNotContain	The text being searched cannot contain the supplied text.
doesNotStartWith	The text being searched cannot start with the supplied text.
doesNotEndWith	The text being searched cannot end with the supplied text.

OrderedOperatorType: OrderedOperatorType combines the SimpleOperatorType and the RangeOperatorType to provide a full range or operators for any ordered value.

Union of:

SimpleOperatorType, RangeOperatorType.

TextOperatorType:

Union of:

SimpleOperatorType, TextSearchOperatorType.

TimeOperatorType: TimeOperatorType derives from the OrderedOperatorType to remove the notEqual operator.

Derived by restriction of OrderedOperatorType .

Copyright © SDMx TWG 2020

Enumerations:

Value	Documentation
equal	
greaterThanOrEqual	
lessThanOrEqual	
greaterThan	
lessThan	

UsageType: An enumeration of optional | mandatory to indicate the usage of an attribute or measure.

Derived by restriction of xs:string .

Enumerations:

Value	Documentation
mandatory	
optional	

SimpleCodeDataType: SimpleCodeDataType restricts SimpleDataType to specify the allowable data types for a simple code. The possible values are simply Alpha, AlphaNumeric, or Numeric.

Derived by restriction of SimpleDataType .

Enumerations:

Value	Documentation
Alpha	A string datatype which only allows for the simple aplhabetic charcter set of A-Z, a-z.
AlphaNumeric	A string datatype which only allows for the simple alphabetic character set of A-Z, a-z plus the simple numeric character set of 0-9.
Numeric	A string datatype which only allows for the simple numeric character set of 0-9. This format is not treated as an integer, and therefore can having leading zeros.

EpochPeriodType: EpochPeriodType defines an enumeration of epoch period types.

Derived by restriction of xs:string .

Enumerations:

Value	Documentation
nanosecond	
millisecond	
microsecond	
second	
day	

ResolvePeriodType: ResolvePeriodType defines an enumeration of how date periods should be resolved.

Derived by restriction of xs:string .

Copyright © SDMx TWG 2020

Enumerations:

Value	Documentation
startOfPeriod	
endOfPeriod	
midPeriod	

StandardToVtlMappingMethodType: A simple type enumerating the standard mapping methods when converting from data structures from SDMX to VLT.

Derived by restriction of xs:string .

Enumerations:

Value	Documentation
Basic	The default mapping method. See Section 6 SDMX Standards (“SDMX Technical Notes”), 10.3.3.1 (“Basic Mapping”).
Pivot	Method for mapping multi-measure data. See Section 6 SDMX Standards (“SDMX Technical Notes”), 10.3.3.2 (“Pivot Mapping”).
Basic-A2M	The basic mapping method, using attributes to measures. See Section 6 SDMX Standards (“SDMX Technical Notes”), 10.3.3.3 (“From SDMX DataAttributes to VTL Measures”).
Pivot-A2M	The pivot mapping method, using attributes to measures. See Section 6 SDMX Standards (“SDMX Technical Notes”), 10.3.3.3 (“From SDMX DataAttributes to VTL Measures”).

ToVtlMappingMethodType: A simple type that allows for a standard to VTL mapping method enumeration value or a string value for other mapping methods.

Union of:

StandardToVtlMappingMethodType, xs:string.

StandardFromVtlMappingMethodType: A simple type enumerating the standard mapping methods when converting from data structures from VTL to SDMX.

Derived by restriction of xs:string .

Enumerations:

Value	Documentation
Basic	The default mapping method, applicable only when the VLT data structure has just one measure component. See Section 6 SDMX Standards (“SDMX Technical Notes”), 10.3.4.1 (“Basic Mapping”).
Unpivot	The mapping method to be used when the VTL data structure has more than one measure component. See Section 6 SDMX Standards (“SDMX Technical Notes”), 10.3.4.2 (“Unpivot Mapping”).
M2A	Mapping of multi-measure VTL where on measure is mapped to the SDMX primary measure and the remaining measures are mapped as data attributes. See Section 6 SDMX Standards (“SDMX Technical Notes”), 10.3.4.3 (“From VTL Measures to SDMX DataAttributes”).

FromVtlMappingMethodType: A simple type that allows for standard from VTL mapping method enumeration value or a string value for other mapping methods.

Union of:

StandardFromVtlMappingMethodType, xs:string.

Copyright © SDMx TWG 2020

CustomTypeDataType: A simple type that allows for a SDMX simple data type enumeration value or a string value for other data types.

Union of:

SimpleDataType, xs:string.

Copyright © SDMx TWG 2020

7.1.5 Data

Introduction

The first change in the data and metadata message is one of terminology. In order to foster consistency in the standard, the names and namespaces of the data and metadata message have been changed. The namespaces now have a uniform format of /data/format and /metadata/format, where format is either generic or structured (i.e. the structure specific formats). This also applies to the message names as well, where the names follow the pattern of FormatData (e.g. StructuredData and GenericMetadata).

The various data messages which existed in the standard have been harmonised into two general formats; generic and structured data. The generic data is much like it has been previously, only slightly modified to conform with the harmonised view on data. The structured data combined the principles of the previous versions compact and cross sectional formats into one more generalised format.

The major shift from the previous version is now that all data can be exchanged as either an un-grouped collection of observations, each specifying a full key or it can be exchanged as data grouped into series with any single dimension placed at the observation level. This in effect, combined the abilities of the time series and cross sectional formats in the a single data format. This format also allows for multiple measure to be expressed when the observation dimension is the measure dimension defined in the data structured. The key differences from the previous versions is that this all is possible in a single data structure definition specific format, as well as being possible in the generic format as well. The goal was to make the structures as homogenous as possible.

Another shift from the previous version in the manner in which the base data structure specific format is defined. In previous versions there was not structure defined, so one had to understand all of the requirements and read the specification in order to understand what could be expected in any structure specific format. In this version, that issue has been addressed. The base structures now impose a strict format on the data structure specific schemas. This is achieved much like the referencing structure in the common namespace through the use of unqualified elements. By the elements not existing in a namespace, the structure specific schemas can place the necessary restrictions of them while still being forced to adhere to prescribed structure. This means that not only are the structures able to perform the validation that is required, but that the messages will be much simpler to process as the format will always use the same element names.

Finally, in order to allow for systems which wish to not process the new more flexible data format, time series only variations of both the generic and structured data sets exist. It is important to note that these structures are derived via restriction from the more generalised format. This means that a data set in the general format with the time dimension at the observation level will have exactly the same content as a time series only data set. The result of this is that there is no additional burden for processing the time series specific format it a system can process the more generalised format. These time series only data sets allow for time series only data messages to exist. These messages make it simple for system which, in the previous version of the standard, only used the time series formats to continue to do so.

These same principles have been applied to the reference data messages as well. In the previous version of the standard there were major differences between the generic and metadata structure-specific formats; some of the differences caused some metadata sets were incompatible between versions. In this version the structures of the generic and metadata structure specific formats have been harmonised to the point where they are nearly identical. Yet, the structured format still provides the strong validation against the metadata structure that is intended. And, as with the data, the base metadata structure specific format now imposes a stricter structure on the generated schemas, making the structure specific instance simpler to process in a generic manner.

Schema Documentation

STRUCTURE SPECIFIC DATA NAMESPACE

http://www.sdmx.org/resources/sdmxml/schemas/v3_0/data/structurespecific

Copyright © SDMx TWG 2020

Summary

Referenced Namespaces:

Namespace	Prefix
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/common	common
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/metadata/generic	metadata
http://www.w3.org/2001/XMLSchema	xs

Contents:

6 Complex Types

Complex Types

***DataSetType*:** DataSetType is the abstract type which defines the base structure for any data structure definition specific data set. A derived data set type will be created that is specific to a data structure definition and the details of the organisation of the data (i.e. which dimension is the observation dimension). Data is organised into either a collection of series (grouped observations) or a collection of un-grouped observations. The derived data set type will restrict this choice to be either grouped or un-grouped observations. If this dimension is “AllDimensions” then the derived data set type must consist of a collection of un-grouped observations; otherwise the data set will contain a collection of series with the observations in the series disambiguated by the specified dimension at the observation level. This data set is capable of containing data (observed values) and/or documentation (data and metadata attribute values) and can be used for incremental updates and deletions (i.e. only the relevant updates or deletes are exchanged). It is assumed that each series or un-grouped observation will be distinct in its purpose. For example, if series contains both data and documentation, it assumed that each series will have a unique key. If the series contains only data or only documentation, then it is possible that another series with the same key might exist, but with not with the same purpose (i.e. to provide data or documentation) as the first series. This base type is designed such that derived types can be processed in a generic manner; it assures that data structure definition specific data will have a consistent structure. The group, series, obs, and atts elements are unqualified, meaning that they are not qualified with a namespace in an instance. This means that in the derived data set types, the elements will always be the same, regardless of the target namespace of the schemas which defines these derived types. This allows for consistent processing of the structure without regard to what the namespace might be for the data structure definition specific schema.

Derivation:

```
AnnotableType (extension)
   DataSetType
```

Attributes:

```
structureRef, setID?, action?, reportingBeginDate?, reportingEndDate?,
validFromDate?, validToDate?, publicationYear?, publicationPeriod?
```

Content:

```
Annotations?, DataProvider?, (Atts | Group | Series | Obs)\*,
Metadata?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
structureRef	xs:IDREF	The structureRef contains a reference to a structural specification in the header of a data or reference metadata message. The structural specification details which structure the data or reference metadata conforms to, as well as providing additional information such as how the data is structure (e.g. which dimension occurs at the observation level for a data set).
setID	IDType	The setID provides an identification of the data or metadata set.
action	ActionType	The action attribute indicates whether the file is appending, replacing, or deleting.
reportingBeginDate	BasicTimePeriodType	The reportingBeginDate indicates the inclusive start time of the data reported in the data or metadata set.
reportingEndDate	BasicTimePeriodType	The reportingEndDate indicates the inclusive end time of the data reported in the data or metadata set.
validFromDate	xs:dateTime	The validFromDate indicates the inclusive start time indicating the validity of the information in the data or metadata set.
validToDate	xs:dateTime	The validToDate indicates the inclusive end time indicating the validity of the information in the data or metadata set.
publicationYear	xs:gYear	The publicationYear holds the ISO 8601 four-digit year.
publicationPeriod	ObservationalTimePeriodType	The publicationPeriod specifies the period of publication of the data or metadata in terms of whatever provisioning agreements might be in force (i.e., “Q1 2005” if that is the time of publication for a data set published on a quarterly basis).

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
DataProvider	DataProviderReferenceType	DataProvider contains a reference to the provider for the data set.
Atts	*AttsType*	Atts contains a set of data or metadata attribute values with an attachment level of none (i.e. data set level) or reported against a partial set of dimension values.
Group	*GroupType*	Group contains a references to a defined group in the data structure definition along with its key (if necessary) and values for the attributes which are associated with the group. An attribute is associated to a group by either an explicit group relationship or by a group attachment when the attribute has a relationship with a dimension which is a member of this group.
Series	*SeriesType*	Series contains a collection of observations that share a common key (set of dimension values). The key of a series is every dimension defined in the data structure definition, save the dimension at the observation level. In addition to the key and observations, the series contains values for data and metadata attributes which have a relationship with any dimension that is part of the series key, so long as the attribute does not specify an attachment group or also has a relationship with the dimension declared to be at the observation level.
Obs	*ObsType*	Obs is an un-grouped observation. This observation has a key which is a set of values for all dimensions declared in the data structure definition. In addition to the key, the value of the observation can be provided along with values for all data and metadata attributes which have an association with the observation or any dimension (so long as it does not specify a group attachment).
Metadata	MetadataSetType	Allows for attachment of reference metadata against to the data set.

***GroupType*:** GroupType is the abstract type which defines a structure which is used to communicate attribute values for a group defined in a data structure definition. The group can consist of either a subset of the dimensions defined by the data structure definition, or an association to an attachment constraint, which in turn defines key sets to which attributes can be attached. In the case that the group is based on an attachment constraint, only the identification of group is provided. It is expected that a system which is processing this will relate that identifier to the key sets defined in the constraint and apply the values provided for the attributes appropriately. Data structure definition schemas will drive types based on this for each group defined in the data structure definition. Both the dimension values which make up the key (if applicable) and the attribute values associated with the group will be represented with XML attributes. This is specified in the content model with the declaration of anyAttributes in the “local” namespace. The derived group type will refine this structure so that the attributes are explicit. The XML attributes will be given a name based on the attribute’s identifier. These XML attributes will be unqualified (meaning they do not have a namespace associated with them). The dimension XML attributes will be required while the attribute XML attributes will be optional. To allow for generic processing, it is required that the only unqualified XML attributes in the derived group type be for the group dimensions and data or metadata attributes declared in the data structure definition. If additional attributes are required, these should be qualified with a namespace so that a generic application can easily distinguish them as not being meant to represent a data structure definition dimension or attribute.

Derivation:

```
AnnotableType (extension)
   GroupType
```

Attributes:

Copyright © SDMx TWG 2020

```
type?
```

Content:

```
Annotations?, Comp*, Metadata?
```

Attribute Documentation:

Name	Type	Documentation
type	IDType	The type attribute reference the identifier of the group as defined in the data structure definition. This is optional, but derived group types will provide a fixed value for this so that it always available in the post validation information set. This allows the group to be processed in a generic manner.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Comp	*CompType*	Comp contains the details of group level attributes that have complex representation and cannot be expressed as XML attributes.
Metadata	MetadataSetType	Allows for attachment of reference metadata against to the group.

***SeriesType*:** SeriesType is the abstract type which defines a structure which is used to group a collection of observations which have a key in common. The key for a series is every dimension defined in the data structure definition, save the dimension declared to be at the observation level for this data set. In addition to observations, values can be provided for data and metadata attributes which are associated with the dimensions which make up this series key (so long as the attributes do not specify a group attachment or also have an relationship with the observation dimension). It is possible for the series to contain only observations or only attribute values, or both. Data structure definition schemas will derive a type based on this that is specific to the data structure definition and the variation of the format being expressed in the schema. Both the dimension values which make up the key and the attribute values associated with the key dimensions will be represented with XML attributes. This is specified in the content model with the declaration of anyAttributes in the “local” namespace. The derived series type will refine this structure so that the attributes are explicit. The XML attributes will be given a name based on the attribute’s identifier. These XML attributes will be unqualified (meaning they do not have a namespace associated with them). The dimension XML attributes will be required while the attribute XML attributes will be optional. To allow for generic processing, it is required that the only unqualified XML attributes in the derived group type be for the series dimensions and attributes declared in the data structure definition. If additional attributes are required, these should be qualified with a namespace so that a generic application can easily distinguish them as not being meant to represent a data structure definition dimension or attribute.

Derivation:

```
AnnotableType (extension)
   SeriesType
```

Attributes:

```
TIME_PERIOD?
```

Content:

```
Annotations?, Comp*, Obs*, Metadata?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
TIME_PERIOD	ObservationalTimePeriodType	The TIME_PERIOD attribute is an explict attribute for the time dimension. This is declared in the base schema since it has a fixed identifier and representation. The derived series type will either require or prohibit this attribute, depending on whether time is the observation dimension. If the time dimension specifies a more specific representation of time the derived type will restrict the type definition to the appropriate type.

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Comp	*CompType*	Comp contains the details of series level attributes that have complex representation and cannot be expressed as XML attributes.
Obs	*ObsType*	
Metadata	MetadataSetType	Allows for attachment of reference metadata against to the series.

***CompType*:** CompType is the abstract base for any component value (e.g. a data or metadata attribute, or a measure) that cannot be represented as an XML attribute. For example, a repeated value, a text value in multiple languages, or a value with structured text (XHTML) cannot be expressed as an XML attribute. This type is meant to be restricted based on the component to restrict the cardinality and type of its Value element to conform to the compoent definition. The type of the value element should be restricted to common:SimpleValueType, common:TextValueType, or common:StructuredValueType. In addition, the id attribute should be restricted to be a fixed value with the component identifier. This restricted type based on the component can then be used on Comp elements by using the xsi:type to state the component being expressed and refine the contents of the element to the values allowed by the component.

Derivation:

```
AnnotableType (extension)
   CompType
```

Attributes:

```
id
```

Content:

```
Annotations?, Value*
```

Attribute Documentation:

Name	Type	Documentation
id	NCNameIDType	

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Value	*ValueType*	

***ObsType*:** ObsType is the abstract type which defines the structure of a grouped or un-grouped observation. The observation must be provided a key, which is either a value for the dimension which is declared to be at the observation level if the observation is grouped, or a full set of values for all dimensions in the data structure definition if the observation is un-grouped. This key should disambiguate the observation within the context in which it is defined (e.g. there should not be another observation with the same dimension value in a series). The observation can contain an observed value and/or attribute values. Data structure definition schemas will derive a type or types based on this that is specific to the data structure definition and the variation of the format being expressed in the schema. The dimension value(s) which make up the key and the data and metadata attribute values associated with the key dimension(s) or the primary measure will be represented with XML attributes. This is specified in the content model with the declaration of anyAttributes in the “local” namespace. The derived observation type will refine this structure so that the attributes are explicit. The XML attributes will be given a name based on the attribute’s identifier. These XML attributes will be unqualified (meaning they do not have a namespace associated with them). The dimension XML attribute(s) will be required while the attribute XML attributes will be optional. To allow for generic processing, it is required that the only unqualified XML attributes in the derived observation type be for the observation dimension(s) and attributes declared in the data structure definition. If additional attributes are required, these should be qualified with a namespace so that a generic application can easily distinguish them as not being meant to represent a data structure definition dimension or attribute.

Derivation:

```
AnnotableType (extension)
   ObsType
```

Attributes:

```
type?, TIME_PERIOD?
```

Content:

```
Annotations?, Comp*, Metadata?
```

Attribute Documentation:

Name	Type	Documentation
type	IDType	The type attribute is used when the derived format requires that explicit measure be used. In this case, the derived type based on the measure will fix this value to be the identification of the measure concept. This will not be required, but since it is fixed it will be available in the post validation information set which will allow for generic processing of the data. If explicit measures are not used, then the derived type will prohibit the use of this attribute.
TIME_PERIOD	ObservationalTimePeriodType	The TIME_PERIOD attribute is an explicit attribute for the time dimension. This is declared in the base schema since it has a fixed identifier and representation. The derived series type will either require or prohibit this attribute, depending on whether time is the observation dimension. If the time dimension specifies a more specific representation of time the derived type will restrict the type definition to the appropriate type.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Comp	*CompType*	Comp contains the details of observation measures or attributes that have complex representation and cannot be expressed as XML attributes.
Metadata	MetadataSetType	Allows for attachment of reference metadata against to the observation.

***AttsType*:** AttsType is the abstract type which defines a structure which is used to group a collection of data or metadata attributes which have a key in common. The key for a attribute collection is a subset of the dimension defined in the data structure definition. This is also used for data set level attributes (i.e. those with an attribute relationship of none). In this case, the subset of dimensions is empty. Data structure definition schemas will derive a type based on this that is specific to the data structure definition. The dimension values which make up the key will be represented with local (non-namespace qualified) XML attributes. The metadata attribute values associated with the key dimensions will be expressed as XML local (non-namespace qualified) attributes if they are simple values (e.g. enumerated, dates, numbers) and are not repeatable. Metadata attributes that are repeatable, or do not have simple values (e.g. text) will be expressed using the Comp element. This dimensions and simple attributes are specified in the content model with the declaration of anyAttributes in the “local” namespace. The derived series type will refine this structure so that the attributes are explicit. The XML attributes will be given a name based on the attribute’s identifier. These XML attributes will be unqualified (meaning they do not have a namespace associated with them). The dimension XML attributes will be required while the attribute XML attributes will be optional. To allow for generic processing, it is required that the only unqualified XML attributes in the derived group type be for the series dimensions and attributes declared in the data structure definition. If additional attributes are required, these should be qualified with a namespace so that a generic application can easily distinguish them as not being meant to represent a data structure definition dimension or attribute.

Derivation:

```
AnnotableType (extension)
   AttsType
```

Attributes:

```
TIME_PERIOD?
```

Content:

```
Annotations?, Comp*
```

Attribute Documentation:

Name	Type	Documentation
TIME_PERIOD	ObservationalTimePeriodType	The TIME_PERIOD attribute is an explict attribute for the time dimension. This is declared in the base schema since it has a fixed identifier and representation. The derived series type will either require or prohibit this attribute, depending on whether time is the observation dimension. If the time dimension specifies a more specific representation of time the derived type will restrict the type definition to the appropriate type.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Comp	*CompType*	Comp contains the details of the data or metadata attributes that have complex representation and cannot be expressed as XML attributes.

GENERIC METADATA NAMESPACE

http://www.sdmx.org/resources/sdmxml/schemas/v3_0/metadata/generic

Summary

Referenced Namespaces:

Namespace	Prefix
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/common	common
http://www.w3.org/2001/XMLSchema	xs

Contents:

1 Global Element
3 Complex Types

Global Elements

Attribute (AttributeType): Att elements hold the reported values for a given metadata attribute. These values conform to the definition of the meatadata attribute in the metadata structure definition.

Complex Types

***MetadataSetBaseType*:** MetadataSetBaseType defines the base refinement of the MetadataSetType. Its purpose is to restrict the urn attribute.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 MetadataSetBaseType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?
```

Content:

```
Annotations?, Link*, Name+, Description*
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MetadataSetUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.

MetadataSetType: MetadataSetType describes the structure for a metadata set, which contains a collection of reported metadata against a set of targets. The targets should conform to the restrictions described by the metadata provision or the metadataflow. Note that this is maintainble, and as such must specify in agency. In this case, the agency is the metadata provider. If a metadata provision agreement is referenced, it is assumed that the metadata provider described in the provision will be the same as the agency for this set.

Derivation:

```
AnnotableType (extension)
   IdentifiableType (extension)
         NameableType (extension)
               VersionableType (restriction)
                     MaintainableBaseType (extension)
                           MaintainableType (restriction)
                                 MetadataSetBaseType (extension)
                                       MetadataSetType
```

Attributes:

```
id, urn?, uri?, version?, validFrom?, validTo?, agencyID,
isExternalReference?, serviceURL?, structureURL?, action?,
reportingBeginDate?, reportingEndDate?, publicationYear?,
publicationPeriod?
```

Content:

```
Annotations?, Link*, Name+, Description*, (
(MetadataProvisionAgreement | Metadataflow), Target+, Attribute+)?
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id is the identifier for the object.
urn	MetadataSetUrnType	The urn attribute holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
uri	xs:anyURI	The uri attribute holds a URI that contains a link to a resource with additional information about the object, such as a web page. This uri is not a SDMX message.
version	VersionType	This version attribute holds a version number (see common:VersionType definition for details). If not supplied, artefact is considered to be un-versioned.
validFrom	xs:dateTime	The validFrom attribute provides the inclusive start date for providing supplemental validity information about the version.
validTo	xs:dateTime	The validTo attribute provides the inclusive end date for providing supplemental validity information about the version.
agencyID	NestedNCNameIDType	The agencyID must be provided, and identifies the maintenance agency of the object.
isExternalReference (default: false)	xs:boolean	The isExternalReference attribute, if true, indicates that the actual object is not defined the corresponding element, rather its full details are defined elsewhere - indicated by either the registryURL, the repositoryURL, or the structureURL. The purpose of this is so that each structure message does not have to redefine object that are already defined elsewhere. If the isExternalReference attribute is not set, then it is assumed to be false, and the object should contain the full definition of its contents. If more than one of the registryURL, the repositoryURL, and the structureURL are supplied, then the application processing the object can choose the method it finds best suited to retrieve the details of the object.
serviceURL	xs:anyURI	The serviceURL attribute indicates the URL of an SDMX SOAP web service from which the details of the object can be retrieved. Note that this can be a registry or and SDMX structural metadata repository, as they both implement that same web service interface.
structureURL	xs:anyURI	The structureURL attribute indicates the URL of a SDMX-ML structure message (in the same version as the source document) in which the externally referenced object is contained. Note that this may be a URL of an SDMX RESTful web service which will return the referenced object.
action	ActionType	The action attribute indicates whether the file is appending, replacing, or deleting.
reportingBeginDate	BasicTimePeriodType	The reportingBeginDate indicates the inclusive start time of the data reported in the data or metadata set.
reportingEndDate	BasicTimePeriodType	The reportingEndDate indicates the inclusive end time of the data reported in the data or metadata set.
publicationYear	xs:gYear	The publicationYear holds the ISO 8601 four-digit year.
publicationPeriod	ObservationalTimePeriodType	The publicationPeriod specifies the period of publication of the data or metadata in terms of whatever provisioning agreements might be in force (i.e., “Q1 2005” if that is the time of publication for a data set published on a quarterly basis).

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Link	LinkType	Allows for the linking of other resources to identifiable objects. For example, if there is reference metadata associated with a structure, a link to the meatadata report can be dynamically inserted in the structure metadata.
Name	TextType	Name provides for a human-readable name for the object. This may be provided in multiple, parallel language-equivalent forms.
Description	TextType	Description provides for a longer human-readable description of the object. This may be provided in multiple, parallel language-equivalent forms.
MetadataProvisionAgreement	MetadataProvisionAgreementReferenceType	Metadataflow provides a reference to the.
Metadataflow	MetadataflowReferenceType	Metadataflow provides a reference to the metadataflow the metadata set is being reported against.
Target	WildcardUrnType	Target references the target structures for which metadata is being reported. These must conform with the constraints defined by the metadata provision agreement and/or the metadataflow.
Attribute	AttributeType	Att elements hold the reported metadata attribute values being reported in the metadata set. These conform to the metadata structure defintion

AttributeType: AttributeType defines the structure for a reported metadata attribute. A value for the attribute can be supplied as either a single value (enumerated or non-enumerated single value), or multi-lingual text values (either structured or unstructured). Optional child attributes ar also available if the metadata attribute definition defines nested metadata attributes.

Derivation:

```
AnnotableType (extension)
   AttributeType
```

Attributes:

```
id
```

Content:

```
Annotations?, (Value+ | Text+ | StructuredText+)?, Attribute*
```

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id attribute identifies the metadata attribute that the value is being reported for.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Annotations	AnnotationsType	Annotations is a reusable element the provides for a collection of annotations. It has been made global so that restrictions of types that extend AnnotatableType may reference it.
Value	xs:anySimpleType	Value holds any simple value (enumerated or not) for the metadata attribute. It can be repeated if this metadata attribute allows for multiple values.
Text	TextType	Text is used to supply parallel multi-lingual textual values for the reported metadata attribute. This will be used if the text format of the metadata attribute has a type of string and the multi-lingual value is set to true.
StructuredText	XHTMLType	StructuredText is used to supply parallel multi-lingual structured (as XHTML) textual values for the reported metadata attribute. This will be used if the text format of the metadata attribute has a type of XHTML and the multi-lingual value is set to true.
Attribute	AttributeType	Att contains the reported metadata attribute values for the child metadata attributes.

Mapping to Structure-Specific Schemas

GENERAL

Data structure-specific schemas are each based on one single core construct found in the structure-specific namespace;

Data - http://www.SDMX.org/resources/SDMXML/schemas/v3_0/data/structurespecific

Basic Terminology

In the subsequent sections, the following namespace prefixes are used:

Namespace	Prefix
http://www.w3.org/2001/XMLSchema	xs
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/common	common
http://www.sdmx.org/resources/sdmxml/schemas/ v3_0/data/structurespecific	dsd
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/metadata/generic	metadata

It is assumed that in order to use this guide, the reader is familiar with schema terminology. However, for convenience the following is list of the terminology used here:

Schema: Refers to the format specific schema in general, and in particular the root xs:schema element of that schema file.

Global Element: Refers to an element definition at the top level of the schema (i.e. an xs:element element in the root xs:schema element). It will define a name and type (name and type attributes) and possibly a substitution group (substitutionGroup attribute).

Local Element: Refers to an element definition within a complex type (i.e. an xs:element element contained within a xs:sequence element that is contained in a xs:complexType element). A local element must define a name and type (name and type attributes) and may also specify a minimum and maximum occurrence (minOccurs and maxOccurs attribute).

Qualified/Unqualified Element: A qualified element is an element that must be referred to by the namespace in which it was defined. An unqualified element does not have a namespace associated with it. The structure specific schemas make use of unqualified elements to that the structure specific schemas can restrict the base content to meet the specific needs of the structure, while maintaining as much of the original document structure as possible.

Element Reference: Refers to an element definition within a complex type that is a reference to a global element (i.e. an xs:element element contained within a xs:sequence element that is contained in a xs:complexType element). An element

Copyright © SDMx TWG 2020

reference must reference a global element (via its ref attribute) and may also specify a minimum and maximum occurrence (minOccurs and maxOccurs attribute).

Complex Type: Refers to a complex type definition. In this context, all complex type definitions occur at the top level of the schema (i.e. an xs:complexType element in the root xs:schema element). A complex type must define a name (name attribute) and may be made abstract (via the abstract attribute's boolean value).

Simple Type: Refers to a simple type definition. In this context, all simple type definitions occur at the top level of the schema (i.e. an xs:simpleType element in the root xs:schema element). In this context, a simple type will always be defined via a restriction (an xs:restriction element in the xs:simpleType element). The restriction will reference a base type.

Anonymous Type: A complex or simple type definition which occurs within an element definition. The method is sometimes referred to a the "Russian-doll" technique as it creates nested constructs. Anonymous types are not given names and cannot be abstract. The can however, be derived from other types.

Content Group: A group which defines a content model for reuse. This is contained in the xs:group element, and is defined at the root of the schema. It allows for a common sequence or choice of elements to be reused across multiple types without having to redefine the sequence or choice in each type.

Uniqueness Constraint: A uniqueness constraint is defined within an element and is used to force descendent elements to be unique based on some criteria of it fields (elements or attributes). This is defined in an \<xs:unique> element, and has content of an \<xs:selector> and multiple \<xs:field> elements. The selector designates the descendants that must be unique (with an xpath attribute) and the field specifies which property of the selected element must be unique (also with an xpath attribute)

Extension: Refers to the definition of a complex type that is an extension of another complex type. The extension will always make a reference to a base. In the schema, this is defined within the xs:complexType element as a child xs:complexContent element containing an xs:extension element (with a base attribute).

Restriction: Refers to the definition of a simple or complex type that is a restriction of another type of the same variety. The restriction will always make a reference to a base. In the schema, this is defined with an xs:restriction element (with a base attribute).

Sequence: Refers to a sequence of elements that may be defined as the root of a complex type content model, or as part of the content of a choice or another sequence. This is defined as an xs:sequence element. The sequence may specify a minimum and maximum occurrence (minOccurs and maxOccurs attribute).

Choice: Refers to a choice of elements that may be defined as the root of a complex type content model, or as part of the content of a sequence or another choice. This is defined as an xs:choice element. The sequence may specify a minimum and maximum occurrence (minOccurs and maxOccurs attribute).

Facet: Refers to a single detail of a simple type restriction. This is represented by elements such as xs:minInclusive, xs:totalDigits, xs:minLength, and is contained in the xs:restriction element of a simple type definition. The value of the facet is contained in a value attribute of the particular element.

Enumeration: Refers to an enumerated value of a simple type definition. It is represented by an xs:enumeration element contained within an xs:restriction element of a simple type definition. An enumeration defines a value (in the value attribute) and documentation (in xs:documentation elements contained in an xs:annotation element).

XML Attribute: Refers to the definition of an XML attribute for a complex type (i.e. and xs:attribute element in a xs:complexType element). An attribute must define a name and type (name and type attributes) and may also specify a usage (use attribute).

NAMESPACE RULES

Each format specific schema will specify its namespace in the target namespace of the schema (the targetNamespace attribute of the schema). This document also assumes that the root namespace (that which is defined by the xmlns attribute) of the schema will be the same as the target namespace. Therefore any types or global elements referenced in these descriptions without a namespace prefix are assumed to be in the format specific namespace.

The format specific schemas will incorporate the core format namespace and the common namespace by importing the schemas (via the xs:import element). If necessary, additional namespaces may be imported and referenced.

Copyright © SDMx TWG 2020

For the purpose of the descriptions here, the default element form for the schema (as specified in the elementFormDefault attribute of the schema) is “qualified”, and the default attribute form (as specified in the attributeFormDefault attribute of the schema) is “unqualified”.

GENERAL RULES

The following section details the general rules which apply to all structure specific schema creation.

Component Name Determination

When required to create an XML element or attribute, the name for a component is always its identifier. However, the identifier may be inherited. Therefore, the general rules is as follows:

1. If the component defines an identifier, the element or attribute name is the value of that identifier
2. Otherwise, the element or attribute name is the identifier of the concept from which it takes its semantic (Note that this is technically the component identifer).

Representation Determination

Every component has a representation associated with it, whether it is defined as a local representation in the component definition, or it is inherited from the concept from which the component takes it semantic (as defined in the concept identity of the component).

The representation of a component is determined by the following precedence:

1. The local representation defined by the component
2. The core representation defined in the concept from which the component takes its semantic
3. A default representation of an un-faceted text format with a data type of String.

The representation will either define a text format, or an enumeration with an enumeration format.

A text format consists of a data type and an optional collection of facets. It is the combination of these which determine the exact nature of the component representation. An enumeration consists of a reference to a codelist, hierarchy, or value list, for which an enumerated list of possible values can be created.

Simple / Primitive Type Determination

For any given representation, there exist rules for determining the simple or primitive type which should be used to validate the value. There are no specific requirements to how a simple type is named or if it is referenced or used as an anonymous type. This section simply serves to state the requirements of the type for a component based on its determined representation.

For example, a dimension may inherit its representation for a concept, and the data type of a representation data type may be a String. The simplest solution would be to use the xs:string primitive type. However, an implementer may have chosen to generate simple types for all concepts to avoid having to look up the concept core representation for very component. In this case, the type may be given a name based on the concept and be a simple derivation from the xs:string type that places no further restrictions. The result would be that the type that is actually used for the dimension, although named after the concept, is effectively the required xs:string. These rules are meant to allow such flexibility in how types are created. The only requirement is that the type meet the requirements stated here.

Representation with Codelist Enumeration

A representation which defines an enumeration from a codelist or hierarchy will result in a simple type that is a restriction of the common:IDType. The simple type will define enumerations for each code in the codelist or hierarchy, accounting for extensions. The value for these enumerations will be identifier of the item. If desired, the names of the item may be placed in the documentation of the enumeration, but this is not required. Example:

Copyright © SDMx TWG 2020

\<xs:simpleType name=”ESTAT.CL_COUNTRY.1.0”>

> \<xs:restriction base=”common:IDType”>
>
> \<xs:enumeration value=”BE”>
>
> \<xs:annotation>
>
> \<xs:documentation xml:lang=”en”>Belgium\</xs:documentation>
>
> \</xs:annotation>
>
> \</xs:enumeration>

Representation with Value List Enumeration

A representation which defines an enumeration from a value list will result in a simple type that is a restriction of the xs:string data type. The simple type will define enumerations for each value item in the value list. The value for these enumerations will be identifier of the item. If desired, the names of the item may be placed in the documentation of the enumeration, but this is not required.

Representation with Simple Text Format

A representation which defines a simple text format will result in a simple type or primitive type. The representation is simple if none of the following conditions are true:

- representation max occurs is greater than 1
- text format data type is XHMTL
- text format is multi-lingual

Copyright © SDMx TWG 2020

IF the representation is not simple, see the rules in the following section for complext text formats. If the representation is simple, the first step is to determine the base type from the text format data type:

Copyright © SDMx TWG 2020

SDMX Data Type	XML Schema Data Type
String	xs:string
AlphaNumeric	common:AlphaNumericType
Alpha	common:AlphaType
Numeric	common:NumericType
BigInteger	xs:integer
Integer	xs:int
Long	xs:long
Short	xs:short
Decimal	xs:decimal
Float	xs:float
Double	xs:double
Boolean	xs:boolean
URI	xs:anyURI
Count	xs:integer
InclusiveValueRange	xs:decimal
ExclusiveValueRange	xs: decimal
Incremental	xs: decimal
ObservationalTimePeriod	common:ObservationalTimePeriodType
StandardTimePeriod	common:StandardTimePeriodType
BasicTimePeriod	common:BasicTimePeriodType
GregorianTimePeriod	common:GregorianTimePeriodType
GregorianYear	xs:gYear
GregorianYearMonth	xs:gYearMonth
GregorianDay	xs:date
ReportingTimePeriod	common:ReportingTimePeriodType
ReportingYear	common:ReportingYearType
ReportingSemester	common:ReportingSemesterType
ReportingTrimester	common:ReportingTrimesterType
ReportingQuarter	common:ReportingQuarterType
ReportingMonth	common:ReportingMonthType
ReportingWeek	common:ReportingWeekType
ReportingDay	common:ReportingDayType
DateTime	xs:dateTime
TimeRange	common:TimeRangeType

Copyright © SDMx TWG 2020

SDMX Data Type	XML Schema Data Type
Month	xs:gMonth
MonthDay	xs:gMonthDay
Day	xs:gDay
Time	xs:time
Duration	xs:duration
GeospatialInformation	xs:string
XHTML	See the following section for complex representations

If the text format does not specify any further facets, then the determined type is the listed type or a type which derives from the listed type without placing any addition restrictions on it. However, if one or more facets are specified, then a simple type based on the listed type is necessary. The simple type derives via restriction from the listed type and adds facets according to the following table (the values are mapped as is):

SDMX Facet	XML Schema Facet
minLength	xs:minLength
maxLength	xs:maxLength
minValue[1]	if ExclusiveValueRange: xs:minExclusives, else: xs:minInclusive
maxValue[2]	if ExclusiveValueRange: xs:maxExclusives, else: xs:maxInclusive
decimals[2]	xs:fractionDigits
pattern	xs:pattern

Any other facets are informational only, and will not affect the determined type.

Representation with Complex Text Format

A representation which defines a complex text format will result in a complex type. The representation is complex if any of the following conditions are true:

- representation max occurs is greater than 1
- text format data type is XHMTL
- text format is multi-lingual

Copyright © SDMx TWG 2020

The resulting complex type will be derived via restriction of the common:ValueType. In simple cases, there are pre-defined types that can be used:

- if the text format data type is XHTML, common:StructuredTextValueType can be used; note that this type cannot be further restricted – all other facets are ignored
- if the text format is multi-lingual, common:TextValueType can be used; note that this type cannot be further restricted – all other facets are ignored
- if the text format has no additional facets and the data type is:
- Boolean, common:BooleanValueType can be used
- String, common:StringValueType can be used
- Integer, common:IntValueType can be used
- Double, common:DoubleValueType can be used
- ObservationalTimePeriod, common:ObservationalTimePeriodValueType can be used

If a pre-defined type cannot be used, one will have to be created. The complex type must define a simple content restriction of the common:ValueType. The restriction should define an anonymous simple type based on the text format data type and facets as described in the previous section.

Type Names

These rules will only dictate type names where absolutely necessary. In all other cases, it is the decision of the implementer as to how to name or use the type. It is also the implementer's requirement to ensure that any type name is properly unique within its scope. To assist in this, the following recommendations are offered for naming types such that they are unique.

- It the type is an enumeration from an item scheme, the recommended name is [Item Scheme Class].[Maintenance Agency].[Item Scheme ID].[Item Scheme Version]
- If the type is based on a text format of a concept core representation, the recommended name is Concept.[Maintenance Agency].[Concept Scheme ID].[Concept Scheme Version].[Concept ID]
- If the type is based on a text format of a component local representation, and;
- The component id is required to be unique for all components within the scope of the structure which defines it (e.g. a dimension), the recommended name is [Component ID]
- The component id is only required to be unique within the component list and which defines it (e.g. a metadata attribute), the recommend name is [Component List ID].[Parent Component ID]*.[Component ID]

Type Reuse

It is possible that organisations that manage a large number of structure specific schemas my wish to take advantage of the reuse of previously defined type in order to simply the structure specific schema creation and lessen the number of schema elements which are created. The structure specific formats are designed in such a way that this would be allowed without any adverse affects.

For example, an organisation my create predefined types for all of codelists and concept schemes which their structures utilize. These could be contained in a common schema with any namespace deemed appropriate. This would allow the structure specific schemas generation process to recognize the reused components and not be concerned with regenerating types. The logical flow for setting the representation of a component might be as follows:

> Does the component define a local type?
>
> Yes: Is that type enumerated?
>
> Yes: Type is the qualified type name for the item scheme
>
> No: Generate simple type for text format
>
> No: Type is the qualified name for the concept from which the component takes its semantic.

Copyright © SDMx TWG 2020

Only the constructs that will be detailed in the data and metadata structure-specific rules below are required to be in the specified target namespace of the structure-specific schema. So long as any other generated type conforms to the rules specified, it may exist in any namespace.

DATA STRUCTURE SPECIFIC SCHEMA

Separate schemas will be created for the data structure depending on which dimension occurs at the observation level. The recommended target namespace of the data structured specific schema is: [Data Structure URN]:ObsLevelDim:[Observation Dimensions].

The rules for generating the data structure specific-schema are broken into sections based on the level within the structure (i.e. data set, group, series, attributes, observation). Each section will state the rules for each variation of the structure specific format.

DataSetType

A complex type named DataSetType must be created. Its content model will be derived via restriction. The base type of the restriction is dsd:DataSetType. The complex type content model will be as follows:

1. A sequence consisting of:
    a. An element reference to common:Annotations, with a minimum occurrence of 0
    b. A local element named DataProvider with the type common:DataProviderReferenceType, a form of unqualified and a minimum occurrence of 0
    c. A choice with a minimum occurrence of 0 and a maximum occurrence of unbounded consisting of:
        i. A local element named Atts with a form of unqualified and a type of AttsType (as defined in the AttsType section which follows)
        ii. If the data structure defines groups, a local element named Group with a form of unqualified. The type of this element should be the type that is described in the GroupType section which follows.
        iii. If the dimension at the observation level is not AllDimensions, a local element named Series with a form of unqualified and a type of SeriesType (as defined in the SeriesType section which follows)
        iv. If the dimension at the observation level is AllDimensions, a local element named Obs with a form of unqualified and a type of ObsType (as defined in the ObsType section which follows)
    d. If any metadata attribute usages defined in the data structure that declares an attribute relationship of dataflow, a local element named Metadata with the type metadata:MetadataSetType a form of unqualified, and a minimum occurences of 0

GroupType

If the data structure definition defines only one group, a complex type with its name taken from the identifier of the lone group must be defined. This type is used for the Group element in the DataSetType. Its content model will be derived via restriction of the dsd:GroupType. The complex type content model will be as follows:

1. A sequence consisting of:
    a. An element reference to common:Annotations, with a minimum occurrence of 0
    b. If any attributes defined in the data structure that declares an attribute relationship with the group, a Comp element with a form of unqualified, a minimum occurrence of 0, a maximum occurrence of unbounded, and a type of dsd:CompType
    c. If any metadata attribute usages defined in the data structure that declares an attribute relationship with the group, a local element named Metadata with the type metadata:MetadataSetType a form of unqualified, and a minimum occurences of 0
2. An attribute for each dimension referenced by the group. The XML attribute name and type are defined according to the general rules defined in the previous section, and the usage is required
3. An attribute for each data attribute with simple representation defined in the data structure that declares an attribute relationship with the group or specifies the group as an attachment group. The XML attribute name and type are defined according to the general rules defined in the previous section, and the usage is optional
4. An attribute named type with a type of common:IDType, usage of optional, and a fixed value of the identifier of the group

Copyright © SDMx TWG 2020

If the data structure definition defines more than one group, an abstract complex type with name GroupType must be created. This type is used for the Group element in the DataSetType. Its content model will be derived via restriction of the dsd:GroupType. The complex type content model will be as follows:

1. A sequence consisting of:
    a. An element reference to common:Annotations, with a minimum occurrence of 0
    b. If any attributes defined in the data structure that declares an attribute relationship with a group, a Comp element with a form of unqualified, a minimum occurrence of 0, a maximum occurrence of unbounded, and a type of dsd:CompType
    c. If any metadata attribute usages defined in the data structure that declares an attribute relationship with any group, a local element named Metadata with the type metadata:MetadataSetType a form of unqualified, and a minimum occurences of 0
2. An attribute named type with a type of Group.ID, and a usage of optional
3. An anyAttribute declaration with a namespace of ##local

A simple type named Group.ID must be created. This should restrict the common:IDType. For each group defined by the data structure definition, an enumeration will be created within the restriction with a value of the group identifier.

For each group defined in the data structure definition, a complex type with its name taken from the group identifier is defined. Its content model will be derived via restriction of the previously defined GroupType. The complex type content model will be as follows:

1. A sequence consisting of:
    a. An element reference to common:Annotations, with a minimum occurrence of 0
    b. If any attributes with complex representation defined in the data structure declares an attribute relationship with the group, a Comp element with a form of unqualified, a minimum occurrence of 0, a maximum occurrence of unbounded, and a type of dsd:CompType
    c. If any metadata attribute usages defined in the data structure that declares an attribute relationship with the group, a local element named Metadata with the type metadata:MetadataSetType a form of unqualified, and a minimum occurences of 0
2. An attribute for each dimension referenced by the group. The XML attribute name and type are defined according to the general rules defined in the previous section, and the usage is required
3. An attribute for each data attribute with simple representation defined in the data structure that declares an attribute relationship with the group or specifies the group as an attachment group. The XML attribute name and type are defined according to the general rules defined in the previous section, and the usage is optional
4. An attribute named type with a type of Group.ID, usage of optional, and a fixed value of the identifier of the group

Copyright © SDMx TWG 2020

SeriesType

If the dimension at the observation is not AllDimensions, a complex type name SeriesType must be created. Its content model will be derived via restriction of dsd:SeriesType. The complex type content model will be as follows:

1. A sequence consisting of:
    a. An element reference to common:Annotations, with a minimum occurrence of 0
    b. If any attributes with complex representation defined in the data structure declares an attribute relationship with a dimension that is not at the observation level, a Comp element with a form of unqualified, a minimum occurrence of 0, a maximum occurrence of unbounded, and a type of dsd:CompType
    c. A local element named Obs with a form of unqualified, a minimum occurrence of 0, a maximum occurrence of unbounded, and a type of ObsType (as defined in the ObsType section which follows)
    d. If any metadata attribute usages defined in the data structure that declares an attribute relationship with the series, a local element named Metadata with the type metadata:MetadataSetType a form of unqualified, and a minimum occurences of 0
2. An attribute named TIME_PERIOD with a type of common:ObservationalTimePeriod. If the dimension at the observation level is the time dimension (TIME_PERIOD) or there is no time diemsnion defined by the data structure, a usage of prohibited; otherwise a usage of required
3. An attribute for each dimension defined by the data structure definition, except for the dimension at the observation level and the time dimension (TIME_PERIOD). The XML attribute name and type are defined according to the general rules defined in the previous section, and the usage is required
4. An attribute for each data attribute defined with simple representation in the data structure that declares an attribute relationship with any dimension outside of the dimension at the observation level (so long as it does not also declare an attachment group). The XML attribute name and type are defined according to the general rules defined in the previous section, and the usage is optional

AttsType

A a complex type named AttsType must be created. Its content model will be derived via restriction of dsd:AttsType. The complex type content model will be as follows:

1. A sequence consisting of:
    a. An element reference to common:Annotations, with a minimum occurrence of 0
    b. If any attributes with complex representation are defined in the data structure, a Comp element with a form of unqualified, a minimum occurrence of 0, a maximum occurrence of unbounded, and a type of dsd:CompType
2. If there is no dimension (TIME_PERIOD) defined by the data structure, an attribute named TIME_PERIOD with a type of common:ObservationalTimePeriod, and a usage of prohibited
3. An attribute for all dimension defined by the data structure definition. The XML attribute name and type are defined according to the general rules defined in the previous section, and the usage is optional
4. An attribute for each data attribute defined with simple representation in the data structure. The XML attribute name and type are defined according to the general rules defined in the previous section, and the usage is optional

Copyright © SDMx TWG 2020

ObsType

A complex type name ObsType must be created. Its content model will be derived via restriction of the base type dsd:ObsType. The complex type content model will be as follows:

1. A sequence consisting of:
    a. An element reference to common:Annotations, with a minimum occurrence of 0
    b. If any measures with complex representations are defined in the data structure, any attributes with complex representation that declare an attribute relationship with the observation, a Comp element with a form of unqualified, a minimum occurrence of 0, a maximum occurrence of unbounded, and a type of dsd:CompType
    c. If any metadata attribute usages defined in the data structure that declares an attribute relationship with the observation, a local element named Metadata with the type metadata:MetadataSetType a form of unqualified, and a minimum occurences of 0
2. An attribute named TIME_PERIOD with a type of common:TimePeriodType. If the dimension at the observation level is the time dimension (TIME_PERIOD) or all dimensions and the time dimension is defined by the data structure, a usage of required; otherwise a usage of prohibited
3. If the dimension at the observation level is not all dimensions or the time dimension (TIME_PERIOD), an attribute for the dimension at the observation level. The XML attribute name and type are defined according to the general rules defined in the previous section, and the usage is required

1. If the dimension at the observation level is all dimensions, an attribute for each dimension defined by the data structure definition, except for the time dimension (TIME_PERIOD). The XML attribute name and type are defined according to the general rules defined in the previous section, and the usage is required

1. An attribute for each measure with simple representation defined by the data structure definition. The XML attribute name and type is defined according to the general rules defined in the previous section, and the usage is optional
2. An attribute for each data attribute with simple representation defined in the data structure that declares an attribute relationship with the obseration. The XML attribute name and type are defined according to the general rules defined in the previous section, and the usage is optional

CompType

For every measure and data attribute with complex representation defined by the data structure definition, a complex type must be derived from the restriction of the dsd:CompType. The complex type content model will be as follows:

1. A sequence consisting of:
    a. An element reference to common:Annotations, with a minimum occurrence of 0
    b. A Value element with a form of unqualified, with a minimum occurrence of 0, a maximum occurrence defined by the representation, and a type based on the complex representation type defined according to the general rules defined in the previous section
2. An attribute named id with a type of common: NCNameIDType, usage of required, and a fixed value of the identifier of the measure or attribute

Special data functions

UPDATES

D messages allow for incremental updating of data. This purpose is noted in the action for the data set, which is either inherited from the header of the data message or explicitly stated at the data set level.

A dataset with an action of Append is assumed to be an incremental update. This means that one the information provided explicitly in the message should be altered. Any data attribute or observation value that is to be changed must be provided. However, the absence of an observation value or a data attribute at any level does not imply deletion; instead it is simply implied that the value is to remain unchanged. Therefore, it is valid and acceptable to send a data message with an action of Append

Copyright © SDMx TWG 2020

which contains only a Series elements with attribute values. In this case, the values for the attributes will be updated. Note that it is not permissible to update data attributes using partial keys (outside of those associated with defined groups). In order to update an attribute, a full key must always be provided even if the message format does not require this.

DELETES

Data messages allow for incremental deletion of data. This purpose is noted in the action for the data set, which is either inherited from the header of the data message or explicitly stated at the data set level.

A dataset with an action of Delete is assumed to be an incremental deletion. The deletion is assumed to take place of the lowest level of detail provided in the message. For example, if a delete message is sent with only a data set element, the entire data set will be deleted. On the other hand, if that data set contains a data attribute, only that data attribute value will be deleted. This same dynamic continues through the data set hierarchy. A data set containing only a series with no data attributes or observations will result in that entire series (all observations and data attributes) being deleted. If the series contains data attributes, only the supplied data attributes for that series will be deleted. Finally, if a series contains observations, then only the specified observations will be deleted. If an entire observation is to be deleted (value and data attributes), only the observation dimension should be provided. If only the observation value or particular data attributes are to be deleted, then these should be specified for the observation. Note that a group can only be used to delete the data attributes associated with it. Although the format might not require it, a full key must be provided to delete a series or observation. It is not permissible to wild card a key in order to delete more than one series or observation. Finally, to delete a data attribute or observation value it is recommended that the value to be deleted be supplied; however, it is only required that any valid value be provided.

---

1. Note that these options only apply to numeric representations and should be ignored if the data type is non-numeric ↩

Copyright © SDMx TWG 2020

7.1.6 Registry

Introduction

The registry schemas have been updated to reflect the various changes in the standard and to introduce new constructs for managing subscriptions. In addition, the constructs that had existed for querying structural metadata have been removed as this is now handled by the query messages.

As was done with the query, the registry message set now contains distinct messages for each operation. The messages will eventually allow the registry to function in more standard web service manner.

Schema Documentation

REGISTRY NAMESPACE

http://www.sdmx.org/resources/sdmxml/schemas/v3_0/registry

Summary

Referenced Namespaces:

Namespace	Prefix
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/common	common
http://www.sdmx.org/resources/sdmxml/schemas/v3_0/structure	structure
http://www.w3.org/2001/XMLSchema	xs

Contents:

39 Complex Types
5 Simple Types

Complex Types

RegistrationType: Registration provides the information needed for data and reference metadata set registration. A data source must be supplied here if not already provided in the provision agreement. The data set or metadata set must be associated with a provision agreement, a metadata flow, or a dataflow definition. If possible, the provision agreement should be specified. Only in cases where this is not possible should the dataflow or metadata flow be used.

Attributes:

```
id?, validFrom?, validTo?, lastUpdated?, indexTimeSeries?,
indexDataSet?, indexAttributes?, indexReportingPeriod?
```

Content:

```
ProvisionAgreement, Datasource
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
id	IDType	The id attribute holds a registry assigned identification for the registration. This must be provided in a response message (unless an error occurred while submitting a new registration), and should be included when updating or deleting a registration.
validFrom	xs:dateTime	The validFrom attribute provides an inclusive start date for providing supplemental validity information about the registration, so that data visibility from the registry can be controlled by the registrant.
validTo	xs:dateTime	The validFrom attribute provides an inclusive end date for providing supplemental validity information about the registration, so that data visibility from the registry can be controlled by the registrant.
lastUpdated	xs:dateTime	The lastUpdated attribute provides a timestamp for the last time the data source was updated.
indexTimeSeries (default: false)	xs:boolean	The indexTimeSeries, if true, indicates that the registry must index all time series for the registered data. The default value is false, and the attribute will always be assumed false when the provision agreement references a metadata flow.
indexDataSet (default: false)	xs:boolean	The indexDataSet, if true, indicates that the registry must index the range of actual (present) values for each dimension of the data set or identifier component of the metadata set (as indicated in the set's structure definition). The default value is false.
indexAttributes (default: false)	xs:boolean	The indexAttributes, if true, indicates that the registry must index the range of actual (present) values for each attribute of the data set or the presence of the metadata attributes of the metadata set (as indicated in the set's structure definition). The default value is false.
indexReportingPeriod (default: false)	xs:boolean	The indexReportingPeriod, if true, indicates that the registry must index the time period ranges for which data is present for the data source. The default value is false, and the attribute will always be assumed false when the provision agreement references a metadata flow.

Element Documentation:

Name	Type	Documentation
ProvisionAgreement	ProvisionAgreementReferenceType	ProvisionAgreement provides a reference to the provision agreement that the data is being registered against.
Datasource	DataSourceType	Datasource identifies the data source(s) where the registered data can be retrieved.

DataSourceType: DataSourceType specifies the properties of a data or metadata source. Either a simple data source, a queryable data source, or both must be specified.

Content:

```
(SimpleDataSource | QueryableDataSource)[1..2]
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
SimpleDataSource	xs:anyURI	SimpleDatasource describes a data source that is an SDMX-ML data or metadata message. It requires only the URL of the data.
QueryableDataSource	QueryableDataSourceType	QueryableDatasource describes a data source that can be queried using the SDMX REST interfaces.

SimpleDataSourceType: SimpleDataSourceType describes a simple data source. The URL of the data is contained in the content.

Derivation:

```
xs:anySimpleType (restriction)
   xs:anyURI (extension)
         SimpleDataSourceType
```

Attributes:

```
TYPE?
```

Content:

Attribute Documentation:

Name	Type	Documentation
TYPE (fixed: SIMPLE)	xs:string	TYPE is a fixed attribute that is used to ensure only one simple data source may be provided, when it is referenced in a uniqueness constraint.

QueryableDataSourceType: QueryableDataSourceType describes a queryable data source, and add a fixed attribute for ensuring only one queryable source can be provided.

Derivation:

```
QueryableDataSourceType (extension)
   QueryableDataSourceType
```

Attributes:

```
isRESTDatasource, isWebServiceDatasource, TYPE?
```

Content:

```
DataURL, WSDLURL?, WADLURL?
```

Attribute Documentation:

Name	Type	Documentation
isRESTDatasource	xs:boolean	The isRESTDatasource attribute indicates, if true, that the queryable data source is accessible via the REST protocol.
isWebServiceDatasource	xs:boolean	The isWebServiceDatasource attribute indicates, if true, that the queryable data source is accessible via Web Services protocols.
TYPE (fixed: QUERY)	xs:string	TYPE is a fixed attribute that is used to ensure only one queryable data source may be provided, when it is referenced in a uniqueness constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
DataURL	xs:anyURI	DataURL contains the URL of the data source.
WSDLURL	xs:anyURI	WSDLURL provides the location of a WSDL instance on the internet which describes the queryable data source.
WADLURL	xs:anyURI	WADLURL provides the location of a WADL instance on the internet which describes the REST protocol of the queryable data source.

IdentifiableQueryType: IdentifiableQueryType describes the structure of a query for an identifiable object.

Attributes:

```
id?
```

Content:

```
{Empty}
```

Attribute Documentation:

Name	Type	Documentation
id (default: %)	IDQueryType	The id attribute is used to query for an object based on its identifier. This is either an explicit value, or completely wild cared with the “%” value.

VersionableQueryType: VersionableQueryType describes the structure of a query for a versionable object.

Derivation:

```
IdentifiableQueryType (extension)
   VersionableQueryType
```

Attributes:

```
id?, version?
```

Content:

```
{Empty}
```

Attribute Documentation:

Name	Type	Documentation
id (default: %)	IDQueryType	The id attribute is used to query for an object based on its identifier. This is either an explicit value, or completely wild cared with the “%” value.
version (default: *)	VersionQueryType	The version attribute is used to query for an object based on its version. This can be and explicit value, wild-carded (“%”), or late-bound (“*”). A wild carded version will match any version of the object where as a late-bound version will only match the latest version.

MaintainableQueryType: MaintainableQueryType describes the structure of a query for a maintainable object.

Derivation:

```
IdentifiableQueryType (extension)
   VersionableQueryType (extension)
         MaintainableQueryType
```


Copyright © SDMx TWG 2020

Attributes:

```
id?, version?, agencyID?
```

Content:

```
{Empty}
```

Attribute Documentation:

Name	Type	Documentation
id (default: %)	IDQueryType	The id attribute is used to query for an object based on its identifier. This is either an explicit value, or completely wild cared with the “%” value.
version (default: *)	VersionQueryType	The version attribute is used to query for an object based on its version. This can be and explicit value, wild-carded (“%”), or late-bound (“*”). A wild carded version will match any version of the object where as a late-bound version will only match the latest version.
agencyID (default: %)	NestedIDQueryType	The agencyID attribute is used to query for an object based on its maintenance agency’s identifier. This is either an explicit value, or completely wild cared with the “%” value.

StatusMessageType: StatusMessageType carries the text of error messages and/or warnings in response to queries or requests.

Attributes:

```
status
```

Content:

```
MessageText*
```

Attribute Documentation:

Name	Type	Documentation
status	StatusType	The status attribute carries the status of the query or request.

Element Documentation:

Name	Type	Documentation
MessageText	StatusMessageType	MessageText contains the text of the error and/or warning message. It can occur multiple times to communicate message for multiple warnings or errors.

SubmitRegistrationsRequestType: SubmitRegistrationsRequestType defines the payload of a request message used to submit additions, updates, or deletions of data/metadata set registrations.

Content:

```
RegistrationRequest+
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
RegistrationRequest	RegistrationRequestType	RegistrationRequest provides the details of a requested registration and the action to take on it. A reference to a provision agreement that exists in the registry must be provide along with a simple and/or queryable data source. The id should only be provided when updating or deleting a registration.

RegistrationRequestType: RegistrationRequestType describes the structure of a single registration request. It contains the details of a registration and an action field to indicate the action to be taken on the contained registration.

Attributes:

```
action
```

Content:

```
Registration
```

Attribute Documentation:

Name	Type	Documentation
action	ActionType	The action attribute indicates whether this is an addition, a modification, or a deletion of a registration.

Element Documentation:

Name	Type	Documentation
Registration	RegistrationType	Registration contains the details of the data/metadata set registration to be added, updated, or deleted.

SubmitRegistrationsResponseType: SubmitRegistrationsResponseType describes the structure of a registration response. For each submitted registration in the request, a registration status is provided. The status elements should be provided in the same order as the submitted registrations, although each status will echo the request to ensure accurate processing of the responses.

Content:

```
RegistrationStatus+
```

Element Documentation:

Name	Type	Documentation
RegistrationStatus	RegistrationStatusType	RegistrationStatus provided the status details for the submitted registration. It echoes the original submission and provides status information about the request.

RegistrationStatusType: RegistrationStatusType describes the structure of a registration status.

Content:

```
Registration, StatusMessage
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Registration	RegistrationType	Registration, at the very least echoes the submitted registration. It the request was to create a new registration and it was successful, an id must be supplied.
StatusMessage	StatusMessageType	StatusMessage provides that status for the registration request, and if necessary, any error or warning information.

QueryRegistrationRequestType: QueryRegistrationRequestType describes the structure of a registration query request. The type of query (data, reference metadata, or both) must be specified. It is possible to query for registrations for a particular provision agreement, data provider, or structure usage, or to query for all registrations in the registry. In addition the search can be refined by providing constraints in the form of explicit time periods, constraint regions, and key sets. When constraint regions and key sets are provided they will be effectively processed by first matching all data for the included keys and regions (all available data if there are none) and then removing any data matching the excluded keys or regions.

Attributes:

```
returnConstraints?
```

Content:

```
QueryType, (All | ProvisionAgreement | DataProvider | Dataflow |
Metadataflow), ReferencePeriod?, (DataKeySet | CubeRegion |
MetadataTargetRegion)*
```

Attribute Documentation:

Name	Type	Documentation
returnConstraints (default: false)	xs:boolean	The returnConstraints attribute determines whether information about the constraints on the data or metadata sets returned should also be sent the results.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
QueryType	QueryTypeType	QueryType defines the type of sets (data, metadata, or both) that are being queried for.
All	EmptyType	All indicates that all registrations meeting the other criteria of the query should be returned.
ProvisionAgreement	ProvisionAgreementReferenceType	ProvisionAgreement provides a reference to a provision agreement in the registry, for which all registered sets meeting the other criteria of this query should be returned. The reference is provided as a URN and/or a complete set of reference fields.
DataProvider	DataProviderReferenceType	DataProvider provides a reference to a data provider in the registry, for which all registered sets meeting the other criteria of this query should be returned. The reference is provided as a URN and/or a complete set of reference fields.
Dataflow	DataflowReferenceTyp e	Dataflow provides a reference to a data flow in the registry, for which all registered sets meeting the other criteria of this query should be returned. The reference is provided as a URN and/or a complete set of reference fields.
Metadataflow	MetadataflowReferenceType	Metadataflow provides a reference to a metadata flow in the registry, for which all registered sets meeting the other criteria of this query should be returned. The reference is provided as a URN and/or a complete set of reference fields
ReferencePeriod	ReferencePeriodType	ReferencePeriod provides an inclusive start and end date for the data or metadata being sought.
DataKeySet	DataKeySetType	DataKeySet is used to provide a set of included or excluded keys which serves to refine the data being sought.
CubeRegion	CubeRegionType	CubeRegion is used to provide sets of include or excluded values for dimensions when querying for data.
MetadataTargetRegion	MetadataTargetRegion Type	MetadataTargetRegion is used to provide sets of included or excluded values for identifier components when querying for metadata.

QueryRegistrationResponseType: QueryRegistrationResponseType describes the structure of a registration query response. It provides a status for the request, and if successful, the resulting data and/or metadata results.

Content:

```
StatusMessage, QueryResult*
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
StatusMessage	StatusMessageType	StatusMessage provides that status for the registration query request, and if necessary, any error or warning information.
QueryResult	QueryResultType	QueryResult contains a result for a successful registration query. It can occur multiple times, for each registration the meets the conditions specified in the query.

QueryResultType: QueryResultType describes the structure of a query result for a single data source. Either a data result or metadata result is detailed, depending on the data source.

Attributes:

```
timeSeriesMatch
```

Content:

```
(DataResult | MetadataResult)
```

Attribute Documentation:

Name	Type	Documentation
timeSeriesMatch	xs:boolean	The timeSeriesMatch attribute is true when the result is an exact match with the key found in the registry - that is, when the registered data source provides a matching key. It is set to false when the data source is registered with cube-region constraints, or in any other circumstance when it cannot be established that the sought-for keys have been exactly matched. This is always true when the resulting data source is the source of a metadata set.

Element Documentation:

Name	Type	Documentation
DataResult	ResultType	
MetadataResult	ResultType	

ResultType: ResultType contains the details about a data or metadata source, through the complete registration information. In addition, a reference to the content constraints for the data source may be provided, if the query requested this information.

Content:

```
Registration, Constraint*
```

Element Documentation:

Name	Type	Documentation
Registration	RegistrationType	Registration provides the details of a matching registration. In addition to the data source and provision agreement information, the id of the registration must also be provided.
Constraint	ConstraintReferenceType	Constraint provides a reference to a data or metadata constraint in the registry for the resulting data source (or possibly constraints base on the registration provision agreement, data provider, structure usage, or structure). The reference is provided for by a URN and/or a complete set of reference fields.

Copyright © SDMx TWG 2020

SubmitStructureRequestType: SubmitStructureRequestType describes the structure of a structure submission. Structural components are provided either in-line or referenced via a SDMX-ML Structure message external to the registry. A default action and external reference resolution action are all provided for each of the contained components, but can be overridden on a per component basis.

Attributes:

```
action?, externalDependencies?
```

Content:

```
(StructureLocation | Structures), SubmittedStructure*
```

Attribute Documentation:

Name	Type	Documentation
action (default: Append)	ActionType	The action attribute indicates the default action (append-add, replace-update, delete, or no action-informational) to be taken on all structural components in either the external SDMX-ML Structure message or the in-line components. The default action is Append. The Replace action is not applicable to final structures in the repository, and will produce an error condition, as these can be versioned but not modified. To submit a later version of a structural object, the object should include the incremented version number.
externalDependencies (default: false)	xs:boolean	The externalDependencies attribute indicates the default resolution of external dependencies. This should be set to true if the repository is expected to use external reference URLs in the structural components to retrieve any externally referenced objects that is used by a non-external object.

Element Documentation:

Name	Type	Documentation
StructureLocation	xs:anyURI	StructureLocation provides the location of a SDMX-ML Structure message, external to the repository that can be retrieved by the repository submission service.
Structures	StructuresType	Structures allows for the inline definition of structural components for submission.
SubmittedStructure	SubmittedStructureType	SubmittedStructure contains a reference to one of the structural maintainable artefacts detailed in the external SDMX-ML Structure message or in-line and provides an override for the default action. This should only be used if the action to be performed on the referenced structural object is different than the default action. For example, one may want to append all structural components of a structure message, save one codelist. This codelist could be referenced in a submitted structure element and given an action of Informational.

SubmittedStructureType: SubmittedStructureType generally references a submitted structural object. When used in a submit structure request, its purpose is to override the default action or external dependency resolution behavior. If neither of these indicators are set, then it will be ignored. In a submit structure response, it is used to reference a submitted object for the purpose of providing a status for the submission. In this case, the action attribute should be populated in order to echo the requested action.

Attributes:

```
action?, externalDependencies?
```


Copyright © SDMx TWG 2020

Content:

```
MaintainableObject
```

Attribute Documentation:

Name	Type	Documentation
action	ActionType	The action attribute will indicate the action to be taken on the referenced structural object. This should be provided when this structure is used in a submit structure response.
externalDependencies	xs:boolean	The externalDependencies attribute should be set to true if the repository is expected to use external reference URLs in the structural components to retrieve objects on which the referenced object has dependencies. (Thus, if a data structure referenced here is being submitted to the repository, and the structure message has URLs which point to the locations of the codelists it uses, then this attribute should be set to true). This should not be provided when this structure is used in a submit structure response.

Element Documentation:

Name	Type	Documentation
MaintainableObject	MaintainableUrnReferenceType	

SubmitStructureResponseType: SubmitStructureResponseType describes the structure of a response to a structure submission. For each submitted structure, a Result will be returned.

Content:

```
SubmissionResult+
```

Element Documentation:

Name	Type	Documentation
SubmissionResult	SubmissionResultType	SubmissionResult provides a status for each submitted structural object.

SubmissionResultType: SubmissionResultType provides the status of the structural object submission request. It will identify the object submitted, report back the action requested, and convey the status and any error messages which are relevant to the submission.

Content:

```
SubmittedStructure, StatusMessage
```

Element Documentation:

Name	Type	Documentation
SubmittedStructure	SubmittedStructureType	SubmittedStructure provides a reference to the submitted structural object and echoes back the action requested for it.
StatusMessage	StatusMessageType	StatusMessage provides that status for the submission of the structural object, and if necessary, any error or warning information.

SubmitSubscriptionsRequestType: SubmitSubscriptionsRequestType defines the payload of a request message used to submit additions, updates, or deletions of subscriptions. Subscriptions are submitted to the registry to subscribe to registration and change events for specific registry resources.

Copyright © SDMx TWG 2020

Content:

```
SubscriptionRequest+
```

Element Documentation:

Name	Type	Documentation
SubscriptionRequest	SubscriptionRequestT ype	

SubscriptionType: SubscriptionType describes the details of a subscription to a registration or change event for registry resources. When it occurs as the content of a response message, the registry URN must be provide, unless the response is a failure notification for the creation of a new subscription.

Content:

```
Organisation, RegistryURN?, NotificationMailTo*, NotificationHTTP*,
SubscriberAssignedID?, ValidityPeriod, EventSelector
```

Element Documentation:

Name	Type	Documentation
Organisation	OrganisationReferenceType	Organisation provides a reference to the organisation that owns this subscription. The reference is provided via a URN and/or a complete set of reference fields.
RegistryURN	xs:anyURI	RegistryURN is used to identify the subscription in the case of deletion or modification. This should be provided in all response messages, unless the response it a notification of the failure to create a newly submitted subscription - in which case there will be no registry assigned URN.
NotificationMailTo	NotificationURLType	NotificationMailTo holds an e-mail address (the “mailto:” protocol). Multiple email address can be notified for a single subscription.
NotificationHTTP	NotificationURLType	NotificationHTTP holds an http address to which notifications can be addressed as POSTs. Multiple http address may be notified for a single subscription event.
SubscriberAssignedID	IDType	SubscriberAssignedID allows the subscriber to specify an identification which will be returned as part of the notification for the subscribed events. This should be used if multiple new requests are made, so that the responses can be accurately correlated to the requests.
ValidityPeriod	ValidityPeriodType	Validity period sets a start and end date for the subscription.
EventSelector	EventSelectorType	EventSelector indicates an event or events for the subscription.

SubscriptionRequestType: SubscriptionRequestType describes the structure of a single subscription request. It contains subscription details and an action field to indicate the action to be taken on the contained subscription. Note that if the action is update or delete, then the registry supplied URN for the subscription must be included.

Attributes:

```
action
```

Content:

```
Subscription
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
action	ActionType	The action attribute indicates whether this is an addition, a modification, or a deletion of a subscription.

Element Documentation:

Name	Type	Documentation
Subscription	SubscriptionType	Subscription contains the details of the subscription to be added, updated, or deleted.

SubmitSubscriptionsResponseType: SubmitSubscriptionsResponseType describes the structure of the response to a new subscription submission. A status is provided for each subscription in the request.

Content:

```
SubscriptionStatus+
```

Element Documentation:

Name	Type	Documentation
SubscriptionStatus	SubscriptionStatusType	SubscriptionStatus contains information which describes the success or failure of a subscription request, providing any error messages in the event of failure. The statuses should occur in the same order as the requests when responding to a message with multiple subscription requests. If a subscriber-assigned identification for the subscription is provided, it will be returned to allow for accurate matching of the responses to the requests. A registry assigned URN will be returned for any successfully created, updated, or deleted subscription.

SubscriptionStatusType: SubscriptionStatusType describes the structure a status for a single subscription request.

Content:

```
SubscriptionURN?, SubscriberAssignedID?, StatusMessage
```

Element Documentation:

Name	Type	Documentation
SubscriptionURN	xs:anyURI	SubscriptionURN contains the registry generated URN for the subscription, and will be returned for any successfully created, updated, or deleted subscription.
SubscriberAssignedID	IDType	SubscriberAssignedID is the id assigned by the subscriber to the subscription. If it existed in the subscription request, it will be returned.
StatusMessage	StatusMessageType	StatusMessage provides that status for the subscription request, and if necessary, any error or warning information.

QuerySubscriptionRequestType: QuerySubscriptionRequestType describes the structure of a query for subscriptions. Subscriptions for a given organisation may be retrieved.

Content:

```
Organisation
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Organisation	OrganisationReferenceType	Organisation provides a reference to the data consumer for which the subscription details should be returned.

QuerySubscriptionResponseType: QuerySubscriptionResponseType describes the structure of a subscription query response. A status will describe the success or failure of the request (and provide error or warning messages if necessary). If the query was successful, details of all of the organisation's subscriptions will be provided.

Content:

```
StatusMessage, Subscription*
```

Element Documentation:

Name	Type	Documentation
StatusMessage	StatusMessageType	StatusMessage provides that status for the query subscription request, and if necessary, any error or warning information.
Subscription	SubscriptionType	Subscription contains the details of a subscription for the organisation. This may occur multiple times for each subscription.

NotifyRegistryEventType: NotifyRegistryEventType describes the structure a registry notification, in response to a subscription to a registry event. At a minimum, the event time, a reference to the change object, a reference to the underlying subscription triggering the notification, and the action that took place on the object are sent. In addition, the full details of the object may be provided at the discretion of the registry. In the event that the details are not sent, it will be possible to query for the details of the changed object using the reference provided.

Content:

```
EventTime, (ObjectURN | RegistrationID), SubscriptionURN, EventAction,
(StructuralEvent | RegistrationEvent)?
```

Element Documentation:

Name	Type	Documentation
EventTime	xs:dateTime	EventTime specifies the time of the triggering event.
ObjectURN	xs:anyURI	ObjectURN provides the URN of the object on which the event occurred, unless the event is related to a registration, in which case the RegistrationID element should be used.
RegistrationID	IDType	RegistrationID provides the id of the registration that underwent an event.
SubscriptionURN	xs:anyURI	SubscriptionURN provides the registry/repository URN of the subscription that is the cause of this notification being sent.
EventAction	ActionType	EventAction indicates the nature of the event - whether the action was an addition, a modification, or a deletion.
StructuralEvent	StructuralEventType	StructuralEvent is used to provide the details of the structural object that has changed.
RegistrationEvent	RegistrationEventTyp e	RegistrationEvent is used to provide the details or the registration object that has changed.

NotificationURLType: NotificationURLType describes the structure of an http or email address. The content holds the addresses while an attribute indicates whether or not is expects the contents in a SOAP message.

Copyright © SDMx TWG 2020

Derivation:

xs:anySimpleType (restriction)
  xs:anyURI (extension)
    NotificationURLType

Attributes:

```
isSOAP?
```

Content:

Attribute Documentation:

Name	Type	Documentation
isSOAP (default: false)	xs:boolean	The isSOAP attribute, if true, indicates the provided URL expects the notification to be sent in a SOAP message.

ValidityPeriodType: ValidityPeriodType specifies inclusive start and end-dates for the subscription period.

Content:

```
StartDate, EndDate
```

Element Documentation:

Name	Type	Documentation
StartDate	xs:date	StartDate is an inclusive start date for the subscription.
EndDate	xs:date	EndDate is an inclusive end date for the subscription.

EventSelectorType: EventSelectorType describes the details of the events for a subscription. It allows subscribers to specify registry and repository events for which they wish to receive notifications.

Content:

```
(StructuralRepositoryEvents | DataRegistrationEvents |
MetadataRegistrationEvents)[1..3]
```

Element Documentation:

Name	Type	Documentation
StructuralRepository Events	StructuralRepository EventsType	StructuralRepositoryEvents details structural events for the subscription.
DataRegistrationEven ts	DataRegistrationEven tsType	DataRegistrationEvents details the data registration events for the subscription.
MetadataRegistration Events	MetadataRegistration EventsType	MetadataRegistrationEvents details the metadata registration events for the subscription.

StructuralRepositoryEventsType: StructuralRepositoryEventsType details the structural events for the subscription. At least one maintenance agency must be specified, although it may be given a wildcard value (meaning the subscription is for the structural events of all agencies). This can also be a list of agencies to allow the subscription to subscribe the events of more than one agency. It should be noted that when doing so, all of the subsequent objects are assumed to apply to every agency in the list. The subscription is then refined by detailing the structural objects maintained by the agency for which the subscription should apply. It is possible to explicitly select all object events, all objects of given types, or to individually list out specific objects. Note that for any object, it is also possible to provide an explicit URN to reference a distinct object. In this case, the reference to maintenance

Copyright © SDMx TWG 2020

agency described above is ignored. Although it is not required, if specific objects are being referenced by explicit URNs, it is good practice to list the agencies.

Attributes:

```
TYPE?
```

Content:

```
AgencyID+, (AllEvents | (AgencyScheme | DataConsumerScheme |
DataProviderScheme | OrganisationUnitScheme | Dataflow | Metadataflow
| CategoryScheme | Categorisation | Codelist | HierarchicalCodelist
| ConceptScheme | MetadataStructureDefinition | KeyFamily |
StructureSet | ReportingTaxonomy | Process | AttachmentConstraint |
ContentConstraint | ProvisionAgreement | TransformationScheme |
NameAliasScheme | NamePersonalisationScheme | RulesetScheme |
UserDefinedOperatorScheme)+)
```

Attribute Documentation:

Name	Type	Documentation
TYPE (fixed: STRUCTURE)	xs:string	TYPE is a fixed attribute that is used to ensure only of each event selector may be provided, when it is referenced in a uniqueness constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Copyright © SDMx TWG 2020

Name	Type	Documentation
AgencyID	NestedIDQueryType	AgencyID specifies a maintenance agency for the object or objects indicated in the other fields. This can be either a specific ID, or a single wildcard character (“%”). A wild card character can be used to select all agencies, allowing a subscription to all events for particular object types. This can occur multiple times to list a of group of maintenance agencies, creating event subscriptions for all of the subsequent objects for each agency in the group. Note that if multiple agencies are supplied, then the wildcard character should not be used for any of them.
AllEvents	EmptyType	AllEvents creates a subscription to structural events for all structural objects maintained by the agencies referenced.
AgencyScheme	VersionableObjectEventType	AgencyScheme is used to subscribe to changes of agency schemes. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
DataConsumerScheme	VersionableObjectEventType	DataConsumerScheme is used to subscribe to changes of data consumer schemes. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
DataProviderScheme	VersionableObjectEventType	DataProviderScheme is used to subscribe to changes of data provider schemes. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
OrganisationUnitScheme	VersionableObjectEventType	OrganisationUnitScheme is used to subscribe to changes of organisation unit schemes. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
Dataflow	VersionableObjectEventType	Dataflow is used to subscribe to changes of data flows. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
Metadataflow	VersionableObjectEventType	Metadataflow is used to subscribe to changes of metadata flows. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making

Copyright © SDMx TWG 2020

Name	Type	Documentation
		separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
CategoryScheme	VersionableObjectEventType	CategoryScheme is used to subscribe to changes of category schemes. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
Categorisation	IdentifiableObjectEventType	Categorisation is used to subscribe to changes of categorizations. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id field can be selected.
Codelist	VersionableObjectEventType	Codelist is used to subscribe to changes of code lists. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
HierarchicalCodelist	VersionableObjectEventType	HierarchicalCodelist is used to subscribe to changes of hierarchical code lists. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
ConceptScheme	VersionableObjectEventType	ConceptScheme is used to subscribe to changes of concept schemes. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
MetadataStructureDefinition	VersionableObjectEventType	MetadataStructureDefinition is used to subscribe to changes of metadata structure definitions. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
KeyFamily	VersionableObjectEventType	KeyFamily is used to subscribe to changes of key families. The maintenance agencies of the object are those identified in the

Copyright © SDMx TWG 2020

Name	Type	Documentation
		AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
StructureSet	VersionableObjectEventType	StructureSet is used to subscribe to changes of structure sets. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
ReportingTaxonomy	VersionableObjectEventType	ReportingTaxonomy is used to subscribe to changes of reporting taxonomies. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
Process	VersionableObjectEventType	Process is used to subscribe to changes of processes. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
AttachmentConstraint	VersionableObjectEventType	AttachmentConstraint is used to subscribe to changes of attachment constraints. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
ContentConstraint	VersionableObjectEventType	ContentConstraint is used to subscribe to changes of content constraints. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
ProvisionAgreement	VersionableObjectEventType	ProvisionAgreement is used to subscribe to changes of a provision agreement. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
TransformationScheme	VersionableObjectEventType	TransformationScheme is used to subscribe to changes of a transformation scheme. The maintenance agencies of the object

Copyright © SDMx TWG 2020

Name	Type	Documentation
		are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
NameAliasScheme	VersionableObjectEventType	NameAliasScheme is used to subscribe to changes of a name alias scheme. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
NamePersonalisationScheme	VersionableObjectEventType	NamePersonalisationScheme is used to subscribe to changes of a name personalisation scheme. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
RulesetScheme	VersionableObjectEventType	RulesetScheme is used to subscribe to changes of a ruleset scheme. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.
UserDefinedOperatorScheme	VersionableObjectEventType	UserDefinedOperatorScheme is used to subscribe to changes of a user defined operator scheme. The maintenance agencies of the object are those identified in the AgencyID collection, effectively making separate version of this query for each agency specified. The agency is ignored if the content of this is a URN, which references an explicit object. Otherwise, either all objects of this type or specific object according to the id and version fields can be selected.

IdentifiableObjectEventType: IdentifiableObjectEventType describes the structure of a reference to an identifiable object's events. Either all instances of the object matching the inherited criteria, a specific instance, or specific instances of the object may be selected.

Content:

```
(All | URN | ID)
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
All	EmptyType	All subscribes to the events all instances of the containing object meeting the other criteria specified.
URN	xs:anyURI	URN subscribes to the events of the specific instance of the object type referenced by this URN. Note that when this field is used, the agency information inherited from the structural repository event container is ignored.
ID	IDQueryType	ID subscribes to the events of the specific instance of the object type where the value provided here matches the id of the object. The default value is the wildcard character(“%”).

VersionableObjectEventType: VersionableObjectEventType describes the structure of a reference to a versionable object’s events. Either all instances of the object matching the inherited criteria, a specific instance, or specific instances of the object may be selected.

Content:

```
(All | URN | (ID, Version))
```

Element Documentation:

Name	Type	Documentation
All	EmptyType	All subscribes to the events all instances of the containing object meeting the other criteria specified.
URN	xs:anyURI	URN subscribes to the events of the specific instance of the object type referenced by this URN. Note that when this field is used, the agency information inherited from the structural repository event container is ignored.
ID	IDQueryType	ID subscribes to the events of the specific instance of the object type where the value provided here matches the id of the object and the value provided in the version field matches the version of the object. The default value is the wildcard character(“%”).
Version	VersionQueryType	Version subscribes to the events of the specific instance of the object type where the value provided in the id field matches the id of the object and the value here matches the version of the object. The default value is the wildcard character(“%”). Note that in addition to being wild-carded, this can also be bound to the latest version of the object with the late-bound character(“*”).

DataRegistrationEventsType: DataRegistrationEventsType details the data registration events for the subscription. It is possible to subscribe to all data registration events in the repository, or specific events for; single registrations, provision agreements, data providers, data flows, key families, and categories that categorize data flows or key families.

Attributes:

```
TYPE?
```

Content:

```
(AllEvents | (RegistrationID | ProvisionAgreement | DataProvider |
DataflowReference | KeyFamilyReference | Category)+)
```


Copyright © SDMx TWG 2020

Attribute Documentation:

Name	Type	Documentation
TYPE (fixed: DATA)	xs:string	TYPE is a fixed attribute that is used to ensure only of each event selector may be provided, when it is referenced in a uniqueness constraint.

Element Documentation:

Name	Type	Documentation
AllEvents	EmptyType	AllEvents subscribes to all data registration events in the repository.
RegistrationID	IDType	RegistrationID subscribes to all the data registration events for the unique registration referenced.
ProvisionAgreement	ProvisionAgreementReferenceType	ProvisionAgreementReference subscribes to all data registration events for the explicitly referenced provision agreement.
DataProvider	DataProviderReferenceType	DataProviderReference subscribes to all data registration events for the explicitly referenced data provider.
DataflowReference	MaintainableEventTyp e	DataflowReference subscribes to all data registration events for the data flows referenced by this object. This may reference one or more data flows, as the specific references fields allow for a wild-carded value.
KeyFamilyReference	MaintainableEventTyp e	KeyFamilyReference subscribes to all data registration events for the key families referenced by this object. This may reference one or more key families, as the specific references fields allow for a wild-carded value.
Category	CategoryReferenceTyp e	Category subscribes to all data registration events for any data flows or key families that are categorized by the referenced category.

MetadataRegistrationEventsType: MetadataRegistrationEventsType details the metadata registration events for the subscription. It is possible to subscribe to all metadata registration events in the repository, or specific events for; single registrations, provision agreements, data providers, metadata flows, metadata structure definitions, and categories that categorize metadata flows or metadata structure definitions.

Attributes:

```
TYPE?
```

Content:

```
(AllEvents | (RegistrationID | ProvisionAgreement | DataProvider |
MetadataflowReference | MetadataStructureDefinitionReference |
Category)+)
```

Attribute Documentation:

Name	Type	Documentation
TYPE (fixed: METADATA)	xs:string	TYPE is a fixed attribute that is used to ensure only of each event selector may be provided, when it is referenced in a uniqueness constraint.

Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
AllEvents	EmptyType	AllEvents subscribes to all metadata registration events in the repository.
RegistrationID	IDType	RegistrationID subscribes to all the metadata registration events for the unique registration referenced.
ProvisionAgreement	ProvisionAgreementReferenceType	ProvisionAgreementReference subscribes to all metadata registration events for the explicitly referenced provision agreement.
DataProvider	DataProviderReferenceType	DataProvider subscribes to all metadata registration events for the explicitly referenced data provider.
MetadataflowReference	MaintainableEventTyp e	MetadataflowReference subscribes to all metadata registration events for the metadata flows referenced by this object. This may reference one or more metadata flows, as the specific references fields allow for a wild-carded value.
MetadataStructureDefinitionReference	MaintainableEventTyp e	MetadataStructureDefinitionReference subscribes to all metadata registration events for the metadata structure definitions referenced by this object. This may reference one or more metadata structure definitions, as the specific references fields allow for a wild-carded value.
Category	CategoryReferenceTyp e	Category subscribes to all metadata registration events for any metadata flows or metadata structure definitions that are categorized by the referenced category.

MaintainableEventType: MaintainableEventType provides a reference to a maintainable object's event with either the URN of the specific object, or a set of potentially wild-carded reference fields.

Content:

```
(URN | Ref)
```

Element Documentation:

Name	Type	Documentation
URN	xs:anyURI	URN provides an explicit reference to a single object.
Ref	MaintainableQueryTyp e	Ref provides a reference to potentially many object through the use of possible wild-carded reference fields.

StructuralEventType: StructuralEventType provides the details of a structure event, specifically the object that changed.

Content:

```
Structures
```


Copyright © SDMx TWG 2020

Element Documentation:

Name	Type	Documentation
Structures	StructuresType	Structures contains the details of the structural object that has triggered the event. Although this container allows for multiple structural object, it should only contain the one changed object.

RegistrationEventType: This provides the details of a data or metadata registration event for the purposes of notification.

Content:

```
Registration
```

Element Documentation:

Name	Type	Documentation
Registration	RegistrationType	Registration provides the changed details of the data or metadata registration.

Simple Types

IDQueryType: IDQueryType is a simple type that allows for an identifier to be substituted with a wild card character (“%”).

Union of:

IDType, WildCardValueType.

NestedIDQueryType: NestedIDQueryType is a simple type that allows for a nested identifier to be substituted with a wild card character (“%”).

Union of:

NestedIDType, WildCardValueType.

VersionQueryType: VersionQueryType is a simple type that allows for a version number to be substituted with a wild card character (“%”) or a late bound character (“*”).

Union of:

LegacyVersionNumberType, SemanticVersionNumberType, SemanticVersionReferenceType, WildcardType, WildCardValueType.

StatusType: StatusType provides an enumeration of values that detail the status of queries or requests.

Derived by restriction of xs:NMTOKEN .

Enumerations:

Value	Documentation
Success	Query or request successful.
Warning	Query or request successful, but with warnings.
Failure	Query or request not successful.

QueryTypeType: QueryType provides an enumeration of values which specify the objects in the result-set for a registry query.

Derived by restriction of xs:NMTOKEN .

Copyright © SDMx TWG 2020

Enumerations:

Value	Documentation
DataSets	Only references data sets should be returned.
MetadataSets	Only references to metadata sets should be returned.
AllSets	References to both data sets and metadata sets should be returned.

Copyright © SDMx TWG 2020

7.1.7 Samples

Introduction

This document provides a brief overview of the SDMX-ML XML sample files.

All samples use the xsi:schemaLocation attribute to define the location of the XSD schema files for the relevant namespaces. The samples assume that the schemas are held locally in a directory called ‘schemas’ and are referenced as a relative file path. For example:

```
xsi:schemaLocation="http://www.sdmx.org/resources/sdmxml/schemas/v3_0/message
../../schemas/SDMXMessage.xsd"
```

Copies of the SDMX-ML XSD schemas files will need to be in the appropriate directory in order to validate correctly.

Structure Samples

The Structure Samples provide examples of SDMX-ML messages within the *structure* namespace which is used for all structural metadata artefacts.

CODELIST

codelist.xml

A simple enumerated code list directly equivalent to those in version 2.1 and earlier.

This example shows the SDMX:CL_AGE ‘Age’ cross domain code list.

codelist – extended.xml

Extended code list.

Illustration of the feature introduced in SDMX 3.0 allowing code lists to be defined as an extension of others. One use case envisaged is adding additional codes to standard cross domain code lists without needing to take and maintain an explicit copy. The sample message has two codelists:

- SDMX:CL_AGE, a simple enumerated codelist; and
- EXAMPLE:CL_EXTENDED_AGE, an extension of SDMX:CL_AGE.

CL_EXTENDED_AGE defines two additional codes “I” and “S”. In addition, the ExclusiveCodeSelection option to exclude the “Y” code from the CL_AGE code list. The result is CL_AGE, minus “Y”, plus “I” and “S”. The InclusiveCodeSelection option could have been used if a specific set of codes were needed from CL_AGE.

Copyright © SDMx TWG 2020

codelist – discriminated union.xml

Discriminated union of code lists.

This sample illustrates the discriminated union of code lists use case where a classification or breakdown has multiple “variants” which are all valid or mutually exclusive. Economic activity has been used with ISIC4 and NACE2 examples of two mutually exclusive variants. In the sample:

- Code list extension is used to create a new EXAMPLE:CL_ACTIVITY code list consisting of both the SDMX:CL_ACTIVITY_NACE2 and SDMX:CL_ACTIVITY_ISIC4 codelists.
- The code list extension prefix feature has been used to prefix the NACE codes with “NACE2_”, and similarly the ISIC codes with “ISIC4_”. This ensures there is no ambiguity where the same code appear in both NACE and ISIC code lists.
- A data structure definition has been created with an enumerated ACTIVITY dimension using the EXAMPLE:CL_ACTIVITY code list.
- Two data flows are created referencing the data structure definition. One data flow has a data constraint attached with a CubeRegion for ACTIVITY and Value = “NACE2_%”. The other has a similar data constraint with Value = “ISIC4_%”. The “%” is the wildcard character for constraints introduced in version 3.0.

The discriminated unions are achieved by requesting either of the data flows with references=”all” and detail=”referencepartial”. The result being CL_ACTIVITY with the extensions resolved and the relevant data constraint applied. Thus CL_ACTIVITY will only contain codes prefixed according to the data flow: either beginning “NACE2_” or “ISIC4_”

valuelist.xml

A simple enumerated value list.

Value lists were introduced in version 3.0 to allow the definition of enumerations where the codes do not need to comply with the strict SDMX rules for identifiers. Thus a “value” can be any string of characters. In SDMX-ML valuelists are treated as a class of enumeration so appear in the structure message under Codelists alongside simple and extended codelists, and specialised geospatial codelist variants. Currency is used as the example with the values being currency symbols including “\$”, “£”, “¥” and “ ”.

CONCEPT SCHEME

conceptscheme.xml

The example illustrates a single concept scheme ECB:ECB_CONCEPTS containing multiple individual concepts.

DATA STRUCTURE DEFINITION

ECB EXR.xml

The example illustrates a data structure definition (DSD) for ECB:EXR exchange rates. The concept of primary measure has been deprecated in SDMX 3.0 and the DSD’s MeasureList can contain multiple measures. In this case, a single measure OBS_VALUE is defined.

DATAFLOW

dataflow.xml

A simple data flow for the ECB:EXR data set.

Copyright © SDMx TWG 2020

GEOSPATIAL

geospatial geocomponents.xml

An example of the use of the “GeospatialInformation” representation type. The GeospatialInformation type can be assigned to a Dimension, DataAttribute, MetadataAttribute or a Measure in conjunction with the “GEO” role. Values of are included directly in data messages or metadata reports and must conform to the Geo Feature Set syntax defined in Section 6 of the Technical Specifications.

In the sample:

- A simple illustrative data structure definition has been defined with IDENTIFIER and TIME_PERIOD dimensions, plus a series attribute AREA.
- The AREA attribute has the SDMX Concept Role “GEO” defined identifying it generally as a geospatial component following guidelines.
- The AREA attribute also carries a LocalRepresentation with textType=”GeospatialInformation”.

geospatial geographiccodelist.xml

An example of the definition of GeographicCodelists, a specialised form of code list introduced in SDMX 3.0. GeographicCodelists are used in the same was as simple code lists to assign a closed enumerated list of values to a component. The difference being that the definition of each GeoFeatureSetCode explicitly specifies the geospatial features to which it relates as a string expression using the Geo Feature Set syntax set out in Section 6 of the Technical Specifications. As with any other coded component, data sets only contain the codes and not the Feature Set information itself. Thus, GeoFeatureSetCodes act as pointers to the Geo Feature Set information held in the GeographicCodelist.

In the sample:

- A single illustrative GeographicCodelist has been defined which must be under Codelists in the structure message.
- The GeographicCodelist geoType attribute must be given the value of “GeographicCodelist”.
- Three illustrative GeoFeatureSetCodes have been defined with IDs “A1”, “A2” and “A3”.
- The GeoFeatureSetCode Value attribute specifies the geospatial features to which the code relates as a string expression. Note that the values shown are conceptual examples and are not syntactically valid Geo Feature Set expressions.

geospatial geogridcodelist.xml

An example of the definition of GeoGridCodelists, the second specialised form of codelist for modelling gridded geographies.
Like GeographicCodelists, GeoGridCodelists act in the same way as simple code lists to assign a closed enumerated list of values to a component.
GeoGridCodelists define a geographical grid by setting the X and Y coordinates of a reference point, and the dimensions of each cell. In addition, each GeoGridCode specifies to which cell in the grid it relates using row and column numbers.

In the sample:

- A single illustrative GeoGridCodelist has been defined which must be under Codelists in the structure message.
- The GridDefinition element defines the grid for the code list using a string expression conforming to the Geo Grid syntax set out in Section 6 of the Technical Specifications.
- Each GeoGridCode has an ID and a Name in common with the codes of simple code lists, but also has a GeoCell element containing the row and column of the grid cell as a string expression using the Grid Cell syntax set out in Section 6 of the Technical Specifications.

Copyright © SDMx TWG 2020

VTL TRANSFORMATIONS

VTL Sample 1.xml

Illustrates the use of VTL Rulesets to validate an SDMX dataflow. The validation process adds additional identifiers and measures to the resulting VTL dataset indicating, on a row by row basis the validation rule applied and the outcome in terms of a valid / invalid. If invalid, details of the error found and its severity are added.

A second transformation takes the resulting dataset filtering out high-severity validation errors and persists the result back to a SDMX dataflow using VTL mappings.

VTL Sample 2.xml

Illustrates aggregation of an SDMX dataflow using VTL hierarchical rulesets, filtering and user defined operators.

After the initial aggregation, unaggregated observations removed are removed from the result dataset. A user defined operator is subsequently applied to calculate a percentage measure with the final persistent result mapped from VTL to a SDMX dataflow.

VTL Sample 3.xml

Illustrates the calculation of a GDP per capita dataset from an input mapped SDMX dataflow containing GDP and population indicators on a country by country basis.

Data Samples

SDMX-ML 3.0 has a single format for data transmission - the Structure Specific Data message. Alternative formats from version 2.1 and earlier such as Generic, Utility and Cross Sectional are deprecated.

The Structure Specific Data message is characterised by the XML elements and attributes being derived from the data structure definition of the data set. This provides two main benefits:

1. The message content is relatively compact.
2. It is possible to use an XML schema to validate that the data set contains the correct values as defined by the data structure definition and any constraints attached to the dataflow, data structure definition or data provision agreement.

The samples therefore include both the Structure Specific Data XML, and a corresponding example validation schema XSD. Successful schema validation confirms that the XML is both a valid Structure Specific Data message, and the content is valid according to the SDMX structural metadata.

AGGREGATED TIME SERIES WITH SIMPLE DATA ATTRIBUTES

ECB EXR.xml

Structure specific data message.

Excerpt from the ECB’s Exchange Rates dataset.

In this example which is representative of aggregated time-series datasets with a single measure, observations are grouped together under series, which in turn are grouped under the dataset element. The dimension values and those of series-level attributes are expressed as XML attributes on each series element. Similarly, the TIME_PERIOD, OBS_VALUE and any observation-level attributes are expressed as XML attributes on each obs element. Note that a structure specific namespace ns1 is defined using xsi:schemalocation to reference the validation schema location, in this case ECB_EXR_Dataflow.xsd.

Copyright © SDMx TWG 2020

ECB EXR Dataflow.xsd

Structure specific schema for validating the dataset.

The schema defines a complex type called DataSetType derived from the data set's data structure definition and constraints, in this case those attached to the dataflow.

AGGREGATED TIME SERIES WITH COMPLEX DATA ATTRIBUTES

Complex attributes include those with multilingual text and arrays of values.

EXB EXR CA.xml

Structure specific data message.

A modification of the ECB's Exchange Rates dataset illustrating the following complex attribute use cases:

- TITLE – multi-lingual series-level array attribute
- SOURCE_AGENCY – coded series-level array attribute specifying a minimum of three values
- SOURCE_PUB – uncoded string series-level array attribute with maximum length of 350 characters
- OBS_STATUS – coded observation-level array attribute

Note that a structure specific namespace ns1 is defined using xsi:schemalocation to reference the validation schema location, in this case ECB_EXR_Dataflow.xsd.

ECB EXR CA Dataflow.xsd

Structure specific schema for validating the dataset.

The schema defines a complex type called DataSetType derived from the data set's data structure definition and constraints, in this case those attached to the dataflow.

Complex types based on the abstract dsd:CompType are defined for each complex attribute.

ECB EXR CA DSD.xml

Structure message describing the Data Structure Definition for the above dataflow illustrating, in particular, the definition of complex attributes:

The XML attribute isMultiLingual="true" on the `<str:TextFormat…>` element specifies a multi-lingual string representation.

Setting the maxOccurs XML attribute to a number greater than 1 (or unbounded) on the `<str:LocalRepresentation…>` element specifies an array.

Copyright © SDMx TWG 2020

7.2 SDMx-JSON (v2.1)

7.2.1 SDMX-JSON (v2.0)

SDMX-JSON is a new data exchange format for data dissemination on the web. SDMX-JSON format conforms to the JSON standard specification, and it supports the SDMX 2.1 Information Model. SDMX-JSON is streamable from a supporting web service in order to limit the amount of server resources. It should provide fast responses from a web service: few roundtrips, small message sizes and fast parsing. In addition, the format aims to ease the development of client applications with a simple but generic message structure that supports different types of data structures. The SDMX-JSON format may be used in a wide variety of programming languages and application programs. Testing and development of the format has focused on JavaScript applications running in web browsers.

The SDMX-JSON Data Message works together with the SDMX RESTful Web Services API. It will assist developers to write software that requests data responses in JSON from SDMX RESTful API and processes data and metadata in the JSON response. The current version of SDMX-JSON Data Message focuses on data visualisation and it supports all features of the SDMX RESTful API for data queries. SDMX RESTful API also supports reference metadata queries and structural metadata queries. These are not yet supported by the SDMX-JSON format.

Copyright © SDMx TWG 2020

7.2.2 Structure Message

SDMX-JSON Structure Message

SDMX-JSON Structure Message is an SDMX data exchange format based on JSON. JSON (JavaScript Object Notation) is a widely used standardised lightweight data-interchange format. SDMX-JSON Structure Message focuses on data discovery, works together with the SDMX RESTful Web Services API and it supports all features of that API for structure queries.

The current SDMX-JSON Structure Message format 2.0.0 supports the SDMX 3.0 Information Model and works together with the SDMX 3.0 RESTful Web Services API for structure queries for various types of visualisations. It adheres to the HATEOAS principles. Queries can ask for specific structural SDMX artefacts, and to include related structural SDMX artefacts.

See: - docs: for the normative documentation - samples: for message samples - tools: for the JSON schema usable for validation purposes

Copyright © SDMx TWG 2020

Introduction to SDMX-JSON Structure Message 2.0.0

See the SDMX-JSON Data Message docs for a brief introduction of the SDMX information model. For additional information on the SDMX information model, please refer to the SDMX documentation.

Samples, tools and other SDMX-JSON resources are available in the public Github repository.

Before we start, let's clarify a few more things about this guide:

- New fields may be introduced in later versions. Therefore consuming applications should tolerate the addition of new fields with ease.
- The ordering of properties in objects is undefined. The properties may appear in any order and consuming applications should not rely on any specific ordering. It is safe to consider a nulled property and the absence of a property as the same thing.
- Not all properties appear in all contexts. For example response with error messages may not contain a data property.

Copyright © SDMx TWG 2020

Field Guide to SDMX-JSON Structure Message 2.0.0 Objects (aligned with SDMX 3.0.0)

MESSAGE

Message is the top level object and it contains the requested information (“data”) as well as the meta-information describing the (technical) context of the message and, possibly, error information.

- meta - *Object optional*. A *meta* object that contains non-standard meta-information and basic technical information about the message, such as when it was prepared and who has sent it.
- data - *Object optional*. *Data* contains the message’s “primary data”.
- errors - *Array optional*. *Errors* field is an array of *error* objects. When appropriate provides a list of error messages in addition to RESTful web services HTTP error status codes.

The members data and errors MUST NOT coexist in the same message.

Example

```
{
    "meta": {
        # meta object #
    },
    "data": {
        # data object #
    },
    "errors": [
        {
            # error object #
        }
    ]
}
```

META

Object optional. Used to include non-standard meta-information and basic technical information about the message, such as when it was prepared and who has sent it. Any members MAY be specified within `meta` objects.

- schema - *String optional*. Contains the URL to the schema allowing to validate the message. This also allows identifying the version of SDMX-JSON format used in this message. **Providing the link to the SDMX-JSON schema is recommended.**
- id - *String*. Unique string that identifies the message for further references.
- test - *Boolean optional*. Indicates whether the message is for test purposes or not. False for normal messages.
- prepared - *String*. A timestamp indicating when the message was prepared. Values must follow the ISO 8601 syntax for combined dates and times, including time zone.
- contentLanguages - *Array optional*. Array of strings containing the identifier of all languages used anywhere in the message for localized elements, and thus the languages of the intended audience, representing in an array format the same information than the http Content-Language response header, e.g. “en, fr-fr”. See IETF Language Tags. The array’s first element indicates the main language used in the message for localized elements. **The usage of this property is recommended.**
- name - *String optional*. Human-readable (best-language-match) name for the transmission.
- names - *Object optional*. Human-readable localised *names* for the transmission.
- sender - *Object*. *Sender* contains information about the party that is transmitting the message.
- receiver - *Object optional*. *Receiver* contains information about the party that is receiving the message. This can be useful if the WS requires authentication.
- links - *Array optional*. *Links* field is an array of *link objects*. If appropriate, a collection of links to additional external resources for the header.

See the section on localised text elements on how the message deals with languages.

Copyright © SDMx TWG 2020

Example

```
"meta": {
    "schema": "https://raw.githubusercontent.com/sdmx-twg/sdmx-json/master/data-message/tools/schemas/sdmx-json-data-schema.json",
    "copyright": "Copyright 2017 Statistics hotline",
    "id": "b1804c51-1ee3-45a9-bb75-795cd4e06489",
    "prepared": "2018-01-03T12:54:12",
    "test": false,
    "contentLanguages": [ "en", "fr-fr" ],
    "name": "Transmission name",
    "names": {
        # name object #
    },
    "sender": {
        # sender object #
    },
    "receiver": {
        # receiver object #
    },
    "links": [
        {
            # link object #
        }
    ]
}
```

sender

Object. Information about the party that is transmitting the message. Sender contains the following fields:

- id - *String*. A unique identifier of the party.
- name - *String nullable*. A human-readable (best-language-match) name of the sender.
- names - *Object optional*. A list of human-readable localised *names* of the sender.
- contacts - *Array optional*. A collection of *contacts*. Provides contact information for the party in regard to the transmission of the message.

See the section on localised text elements on how the message deals with languages.

Example

```
"sender": {
    "id": "ECB",
    "name": "European Central Bank",
    "names": {
        # name object #
    },
    "contacts": [
        {
            # contact objects #
        }
    ]
}
```

names

Object containing all appropriate localised names, one per object property:

- One or more of: IETF Language Tag according to RFC 5646 documentation for specifying locals in HTTP - *String*. The localised name.

See the section on localised text elements on how the message deals with languages.

Example

```
{
  "en": "This is an English name",
  "en-GB": "This is a British name",
  "fr": "C'est un nom français"
}
```

contact

Copyright © SDMx TWG 2020

Object. A collection of contact details. Each object in the collection may contain the following field:

- id - *String*. Identifier for the resource.
- name - *String optional*. Human-readable (best-language-match) name of the contact.
- names - *Object optional*. Human-readable localised *names* of the contact.
- department - *String optional*. Human-readable (best-language-match) name of the organisational structure for the contact.
- departments - *Object optional*. Human-readable localised *names* of the organisational structure for the contact.
- role - *String optional*. Human-readable (best-language-match) name of the responsibility of the contact.
- roles - *Object optional*. Human-readable localised *names* of the responsibility of the contact.
- telephones - *Array optional*. An array of telephone numbers for the contact.
- faxes - *Array optional*. An array of fax numbers for the contact person.
- uris - *Array optional*. An array of uris. Each uri holds an information URL for the contact.
- emails - *Array optional*. An array of email addresses for the contact person.
- x400s - *Array optional*. An array of X.400 addresses for the contact person.

See the section on localised text elements on how the message deals with languages.

Example


```
{
  "id": "HOTLINE",
  "name": "Statistics hotline",
  "names": {
    "en": "Statistics hotline",
    "fr": "Service d'assistance téléphonique des Statistiques"
  },
  "department": "Statistics hotline",
  "departments": {
    "en": "Statistics hotline",
    "fr": "Service d'assistance téléphonique des Statistiques"
  },
  "role": "Statistics hotline",
  "roles": {
    "en": "Statistics hotline",
    "fr": "Service d'assistance téléphonique des Statistiques"
  },
  "telephones": ["+00 0 00 00 00 00"],
  "faxes": ["+00 0 00 00 00 01"],
  "uris": ["www.xyz.org"],
  "emails": ["statistics@xyz.org"]
}
```


receiver

Object optional. Information about the party that is receiving the message. This can be useful if the WS requires authentication. Receiver contains the same fields as sender.

link

See the section on linking mechanism for all information on links.

Copyright © SDMx TWG 2020

DATA

Object optional. Header contains the message’s “primary data”.

Copyright © SDMx TWG 2020

- *[Artefact type] - Array optional*. This field is an array of objects of one of the corresponding SDMX Information Model artefact types: *dataStructure*, *metadataStructure*, *categoryScheme*, *conceptScheme*, *codelist*, *geographicCodelists*, *geoGridCodelists*, *valueLists*, *hierarchy*, *hierarchyAssociation*, *agencyScheme*, *dataProviderScheme*, *dataProviderScheme*, *metadataProviderScheme*, *organisationUnitScheme*, *dataflow*, *metadataflow*, *reportingTaxonomy*, *provisionAgreement*, *metadataProvisionAgreement*, *structureMap*, *representationMap*, *conceptSchemeMap*, *categorySchemeMap*, *organisationSchemeMap*, *reportingTaxonomyMap*, *process*, *categorisation*, *dataConstraint*, *metadataConstraint*, *customTypeScheme*, *vtlMappingScheme*, *namePersonalisationScheme*, *rulesetScheme*, *transformationScheme* and

Copyright © SDMx TWG 2020

userDefinedOperatorScheme. Each of the corresponding object properties is allowed at maximum one time. Contains the requested structural information according to the definition of this artefact. For more information, please see:

- *Common SDMX artefact properties*
- *Common properties of SDMX artefacts of base type “ItemScheme”*
- *dataStructures*
- *metadataStructures*
- *categorySchemes*
- *conceptSchemes*
- *codelists*
- *geographicCodelists*
- *geoGridCodelists*
- *valueLists*
- *hierarchies*
- *hierarchyAssociations*
- *agencySchemes*
- *dataProviderSchemes*
- *dataConsumerSchemes*
- *metadataProviderSchemes*
- *organisationUnitSchemes*
- *dataflows*
- *metadataflows*
- *reportingTaxonomies*
- *provisionAgreements*
- *metadataProvisionAgreements*
- *structureMaps*
- *representationMaps*
- *conceptSchemeMaps*
- *categorySchemeMaps*
- *organisationSchemeMaps*
- *reportingTaxonomyMaps*
- *processes*
- *categorisations*
- *dataConstraints*
- *metadataConstraints*
- *customTypeSchemes*
- *vtlMappingSchemes*
- *namePersonalisationSchemes*
- *rulesetSchemes*
- *transformationSchemes*
- *userDefinedOperatorSchemes*

Copyright © SDMx TWG 2020

Example

```
"data": {
    "dataStructures": [
        {
            # dataStructureDefinition object #
        }
    ],
    "dataflows": [
        {
            # dataflow object #
        }
    ]
}
```

Common SDMX artefact properties

All SDMX artefact types share the following common object properties:

- id - *String*. Identifier for the resource.
- agencyID - *String optional*. ID of the agency maintaining this resource.
- version - *String optional*. Version of this resource. Can be a legacy version with two version parts (e.g. '1.0') for fully mutable artefacts, or a semantic version according to SDMX versioning rules with 3 parts (e.g. '1.0.0') for immutable artefacts or with 4 parts (e.g. '1.0.0-draft') for artefacts allowed to change within a certain scope.
- name - *String optional*. Human-readable (best-language-match) name of the resource.
- names - *Object optional*. Human-readable localised *names* of the resource.
- description - *String optional*. Human-readable (best-language-match) description of the resource.
- descriptions - *Object optional*. Human-readable localised descriptions (see *names*) of the resource.
- validFrom - *String optional*. A timestamp from which the version is valid. Values must follow the ISO 8601 syntax for combined dates and times, including time zone.
- validTo - *String optional*. A timestamp from which the version is superseded. Values must follow the ISO 8601 syntax for combined dates and times, including time zone.
- isExternalReference - *Boolean optional*. If set to "true" it indicates that the content of the resource is held externally.
- annotations - *Array optional*. Provides a list of annotation objects.
- links - *Array optional*. A collection of links to additional resources for the resource. See the section *link*. **It is recommended to systematically include as the first link a self-referencing hyperlink (link with "rel"="self") to indicate the URL address of the resource, and its URN.**

See the section on localised text elements on how the message deals with languages.

Copyright © SDMx TWG 2020

Example


```
{
    "id": "MOBILE_NAVI_PUB",
    "agencyID": "ECB.DISS",
    "version": "1.0.0",
    "name": "Economic concepts",
    "names": {
        "en": "Economic concepts",
        "fr": "Concepts économiques"
    },
    "description": "This is the description of Economic concepts",
    "descriptions": {
        "en": "This is the description of Economic concepts",
        "fr": "Ceci est la description des concepts économiques"
    },
    "validFrom": "2012-05-04",
    "validTo": "2015-05-04",
    "isExternalReference": false,
    "annotations":[
        {
            # annotation object#
        }
    ],
    "links": [
        {
            # link object#
        }
    ]
}
```


annotation

Object optional. Provides all information about an annotation.

- id - *String optional*. ID provides a non-standard identification of an annotation. It can be used to disambiguate annotations.
- title - *String optional*. Provides a non-localised title for the annotation.
- type - *String optional*. Type is used to distinguish between annotations designed to support various uses. The types are not enumerated, and these can be freely specified by the creator of the annotations. The definitions and use of annotation types should be documented by their creator.
- value - *String optional*. Provides a non-localised value text for the annotation.
- text - *String optional*. A human-readable (best-language-match) text of the annotation.
- texts - *Object optional*. A list of human-readable localised texts (see *names*) of the annotation.
- links - *Array optional*. *Links* field is an array of *link* objects. If appropriate, a link to an additional external resource which may contain or supplement the annotation.

See the section on localised text elements on how the message deals with languages.

Example


```
{
    "id": "74747",
    "title": "Sample annotation",
    "type": "reference",
    "value": "123456",
    "text": "Sample annotation text",
    "texts": {
        "en": "Sample annotation text",
        "fr": "Exemple de texte d'annotation"
    },
    "links": [
        {
            # link object #
        }
    ]
}
```


Common properties of SDMX artefacts of base type “ItemScheme”

All SDMX artefacts of base type “ItemScheme” (CategoryScheme, ConceptScheme, Codelist, GeographicCodelist, GeoGridCodelist, AgencyScheme, DataProviderScheme, MetadataProviderSchemes, DataConsumerScheme, OrganisationUnitScheme,

Copyright © SDMx TWG 2020

ReportingTaxonomy, CustomTypeScheme, VtlMappingScheme, NamePersonalisationScheme, RulesetScheme, UserDefinedOperatorScheme) share the *Common SDMX artefact properties*.

In addition, they share the following common object properties:

- isPartial - *Boolean optional*. If set to true, it indicates that the resource contains only a sub-set of items.
- categories / concepts / codes / geoFeatureSetCodes / geoGridCodes / agencies / dataProviders / dataConsumers / metadataProviders / organisationUnits / reportingCategories / customTypes / vtlMappings / namePersonalisations / rulesets / transformations / userDefinedOperators - *Array optional*. Provides a list of *items* if the resource inherits from the ItemScheme. **Note that the order of items is significant. In the use case of a submission of a partial list is is necessary to include preceding and succeeding items to allow determining the correct positioning of the submitted items.**

Example

```
{
    "id": "MY_DOMAINS",
    "isPartial": false,
    "categories": [
        {
            # item object #
        }
    ]
}
```

item

Object optional. Abstract generic item within the ItemScheme (if the resource is a CategoryScheme, ConceptScheme, Codelist, GeographicCodelist, GeoGridCodelist, AgencyScheme, DataProviderScheme, MetadataProviderSchemes, DataConsumerScheme, OrganisationUnitScheme, ReportingTaxonomy, CustomTypeScheme, VtlMappingScheme, NamePersonalisationScheme, RulesetScheme or UserDefinedOperatorScheme).

- id - *String*. Identifier for the item.
- name - *String optional*. Human-readable (best-language-match) name of the item.
- names - *Object optional*. Human-readable localised *names* of the item.
- description - *String optional*. Human-readable (best-language-match) description of the item. The description is typically longer than the text provided for the name field.
- descriptions - *Object optional*. Human-readable localised descriptions (see *names*) of the item. A descriptions is typically longer than the text provided for the name field.
- parent - *String optional*. Contains the ID or the URN for the parent of the item (which is itself an item) enabling the reconstruction of the ordered item hierarchy. Prohibited in certain item scheme types.
- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources for the item. See the section link.
- categories / concepts / codes / geoFeatureSetCodes / geoGridCodes / agencies / dataProviders / dataConsumers / metadataProviders / organisationUnits / reportingCategories / customTypes / vtlMappings / namePersonalisations / rulesets / transformations / userDefinedOperators - *Array optional*. Provides a list of child items of the item. **Note that the order of items is significant. In the use case of a submission of a partial list is is necessary to include preceding and succeeding items to allow determining the correct positioning of the submitted items.**

See the section on localised text elements on how the message deals with languages.

Copyright © SDMx TWG 2020

Example

```
{
    "id": "01",
    "name": "Population and migration",
    "names": {
        "en": "Population and migration",
        "fr": "Population et migration"
    },
    "description": "Description for Population and migration",
    "descriptions": {
        "en": "Description pour Population et migration",
        "fr": "Description for Population and migration"
    },
    "parent": "T",
    "links":[
        {
            # link object#
        }
    ],
    "annotations": [
        {
            # annotation object #
        }
    ],
    "categories": [
        {
            # item object (recursive) #
        }
    ]
}
```

dataStructure

Object. Describes the structure of a data structure definition. A data structure definition is defined as a collection of metadata concepts, their structure and usage when used to collect or disseminate data.

See *Common SDMX artefact properties*.

- dataStructureComponents - *Object optional*. The *dataStructureComponents* object defines the grouping of the sets of structural metadata concepts that have a defined structural role in the data structure definition, like dimensions and time dimension, measures, attributes and relationships with external metadata attributes. Note that for any component or group defined in a data structure definition, its id must be unique. This applies to the identifiers explicitly defined by the components as well as those inherited from the concept identity of a component. For example, if two dimensions take their identity from concepts with same identity (regardless of whether the concepts exist in different schemes) one of the dimensions must be provided a different explicit identifier. Although there are XML schema constraints to help enforce this, these only apply to explicitly assigned identifiers. Identifiers inherited from a concept from which a component takes its identity cannot be validated against this constraint. Therefore, systems processing data structure definitions will have to perform this check outside of the XML validation. There are also three reserved identifiers in a data structure definition: TIME_PERIOD, REPORTING_PERIOD_START_DAY and REPORTING_PERIOD_END_DAY. These identifiers may not be used outside of their respective definitions (TimeDimension and Attribute). This applies to both the explicit identifier that can be assigned to the components or groups as well as an identifier inherited by a component from its concept identity. For example, if an ordinary dimension (i.e. not the time dimension) takes its concept identity from a concept with the identifier TIME_PERIOD, that dimension must provide a different explicit identifier.
- metadata - *String optional*. The URN of a a metadata structure definition. A data structure definition may be related to a metadata structure definition in order to use its metadata attributes as part of the data. Note that the referenced metadata set cannot contain nested metadata attributes, as these are not supported in the data. By default all metadata attributes can be associated at any level of the data. However, a metadata attribute usage can be used to provide a specific attribute relationships for a given metadata attribute.

Example

```
{
    "id": "DSD1",
    "version": "1.0.0",
    "agencyID": "SDMX",
    "dataStructureComponents": {
        # dataStructureComponents object #
    },
    "metadata": "urn:sdmx:org.sdmx.infomodel.metadatastructuredefinition.MetadataStructureDefinition=OECD:METADATA(1.0.0)"
}
```


Copyright © SDMx TWG 2020

dataStructureComponents

Object optional. DataStructureComponents describes the structure of the grouping to the sets of structural concepts that have a defined structural role in the data structure definition. At a minimum at least one dimension must be defined.

- attributeList - *Object optional*. The *attributeList* object is a collection of structural concepts that define the attributes of the data structure definition. Attributes can relate to one or more measures, and be reported at the level of dataflow, several or all dimensions (observation).
- dimensionList - *Object*. The *dimensionList* object is an ordered set of structural concepts that, combined, classify a statistical series, such as a time series, and whose values, when combined (the key) in an instance such as a data set, uniquely identify a specific series.
- groups - *Array optional*. Array of *group* objects that are sets of structural concepts (and possibly their values) that define a partial key derived from the key descriptor in a data structure definition.
- measureList - *Object optional*. The *measureList* object is a collection of structural concepts that define the measures of the data structure definition.

Example

```
{
    "attributeList": {
        # attributeList object #
    },
    "dimensionList": {
        # dimensionList object #
    },
    "groups": [
        {
            # group object #
        }
    ],
    "measureList": {
        # measureList object #
    }
}
```

attributeList

Object optional. AttributeList describes the attributes in the data structure definition.

- id - *String*. Identifier for the attributeList. It is provided only for completeness. However, its value is fixed to “AttributeDescriptor”.
- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.
- attributes - *Array optional*. The *attribute* object describes the definition of a data attribute, which is defined as a characteristic of an object or entity. The attribute list may contain the specialized data attribute that states the month and day at which the reporting year begins or ends, and which provides important context to the time dimension when the value of the time dimension is one of the standard reporting periods. This attribute provides a reference point from which the actual calendar dates covered by these periods can be determined. If this attribute does not occur in a data set, then the reporting year start/end day will be assumed to be January 1/December 31. This attribute can be recognised by its identifier “REPORTING_PERIOD_START_DAY” or “REPORTING_PERIOD_END_DAY”.
- metadataAttributeUsages - *Array optional*. The *metadataAttributeUsage* object refines the details of how a metadata attribute from the metadata structure referenced from the data structure is used. By default, metadata attributes can be expressed at any level of the data. This allows an attribute relationship to be defined in order restrict the reporting of a metadata attribute to a specific part of the data.

Copyright © SDMx TWG 2020

Example

```
{
    "id": "AttributeDescriptor",
    "attributes": [
        {
            # attribute object #
        }
    ],
    "metadataAttributeUsages": [
        {
            # metadataAttributeUsage object #
        }
    ]
}
```

attribute

Object optional. Attribute describes the structure of a data attribute, which is defined as a characteristic of an object or entity. The attribute takes its semantic, and in some cases it representation, from its concept identity. An attribute can be coded by referencing a code list from its coded local representation. It can also specify its data format, which is used as the representation of the attribute if a coded representation is not defined. Neither the coded or uncoded representation are necessary, since the attribute may take these from the referenced concept. An attribute specifies its relationship with other data structure components and is given an assignment status. These two properties dictate where in a data message the attribute will be attached, and whether or not the attribute will be required to be given a value. A set of roles defined in concept scheme can be assigned to the attribute.

- id - *String optional*. Identifier for the attribute.
- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.
- usage - Constant *String* `mandatory` or `optional` *optional*. Indication whether reporting a given attribute is mandatory or optional. Default: `optional`, except for REPORTING_YEAR_START_DAY and REPORTING_YEAR_END_DAY attributes, for which the default is `mandatory`.
- attributeRelationship - *Object*. The *attributeRelationship* object describes how the value of this attribute varies with the values of other components. These relationships will be used to determine the attachment level of the attribute in the various data formats.
- measureRelationship - *Array* of *String*s *optional*. The measureRelationship array identifies the measures that the attribute applies to. If this is not used, the attribute is assumed to apply to all measures. If used, it contains one or more identifiers of (a) local measure(s).
- conceptIdentity - *String*. Urn reference to a concept where the identification of the concept scheme which defines it is contained in another context. The reporting year start and end day attributes take their semantics from their respective concept identity (usually the REPORTING_YEAR_START_DAY and REPORTING_YEAR_END_DAY concepts), yet always have a fixed identifier (REPORTING_YEAR_START_DAY and REPORTING_YEAR_END_DAY).
- conceptRoles - *Array* of *String*s *optional*. ConceptRole references (through URNs) the concepts which define roles which this attribute serves. If the concept from which the attribute takes its identity also defines a role the concept serves, then the isConceptRole indicator can be set to true on the concept identity rather than repeating the reference here.
- localRepresentation - *Object optional*. The *localRepresentation* object defines the representation for the attribute. The representation for the reporting year start or end day attribute that has the conceptRole with concept ID “REPORTING_PERIOD_START_DAY”/”REPORTING_PERIOD_END_DAY” and states the month and day at which the reporting year begins or ends, does not allow for enumerated values and its text format is fixed to be a day and month in the ISO 8601 format of ‘–MM-DD’.

Copyright © SDMx TWG 2020

Example

```
{
    "id": "OBS_STATUS",
    "usage": "optional",
    "attributeRelationship": {
        # attributeRelationship object #
    },
    "measureRelationship": [
        "MEASURE1", "MEASURE2"
    ],
    "conceptIdentity": "urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=ECB:ECB_CONCEPTS(1.0).OBS_STATUS",
    "conceptRoles": [
        "urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=ECB:ECB_CONCEPTS(1.0).OBS_STATUS"
    ],
    "localRepresentation": {
        # localRepresentation object #
    }
}
```

Reporting year start day attribute:

```
json
{
    "id": "REPORTING_YEAR_START_DAY",
    "usage": "mandatory",
    "attributeRelationship": {
        "dataflow": {}
    },
    "conceptIdentity": "urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=ECB:ECB_CONCEPTS(1.0).REPORTING_YEAR_START_DAY",
    "conceptRoles": [
        "urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=ECB:ECB_CONCEPTS(1.0).REPORTING_YEAR_START_DAY"
    ],
    "localRepresentation": {
        "format": {
            "dataType": "MonthDay"
        },
        "maxOccurs": 1
    }
}json
```

attributeRelationship

Copyright © SDMx TWG 2020

Object optional. AttributeRelationship defines the structure for stating the relationship between an attribute and other data structure definition components.

- dataflow - Empty *Object optional*. This means that the value of the attribute **varies** per dataflow. It is the data modeller's responsibility to design or use non-overlapping dataflows that do not have observations in common, otherwise the integrity of dataflow-specific attribute values is not assured by the model, e.g. when querying those data through its DSD. The level at which the unique attribute value will be presented in data messages depends on the data format and which dimensions are referenced.
- dimensions - *Array* of *Strings optional*. One or more identifiers of (a) local dimension(s). This is used to reference dimensions in the data structure definition with which the value of this attribute may vary. An attribute using this relationship can be either a group, series (or section), or observation level attribute. The level at which the attribute values will be presented in data messages depends on the data format and which dimensions are referenced. The array cannot be empty.
- areDimensionsOptional - *Array* of *Booleans optional*. This is used to indicate those dimensions to which the attributes might optionally not be attached. The array structure must follow that for the dimensions. The array cannot be empty.
- group - *String optional*. Identifier of a local GroupKey Descriptor. This is used as a convenience to referencing all of the dimension defined by the referenced group. The level at which the attribute values will be presented in data messages depends on the data format and which dimensions are referenced. If the group (level) is available in the data format used then the attribute values should be presented at that group level.
- observation - Empty *Object optional*. This is used to specify that the value of the attribute may vary with any of the local dimensions and thus is dependent upon the observed value. An attribute with this relationship will its values always have presented at observation level.

As with any other attribute, this relationship should be carefully selected also for the reporting year start or end day attribute as it will determine what type of data the data structure definition will allow. For example, if an attribute relationship of `dataflow` is specified, this will mean that the data sets for this dataflow can only contain data with standard reporting periods where the all reporting periods have the same start day. In this case, data reported as standard reporting periods from two entities with different fiscal year start days could not be contained in the same data set.

Example

```
{
    "dataflow": {}
}

{
    "dimensions": [
        "FREQ", "CURRENCY"
    ]
    "areDimensionsOptional": [
        false, true
    ]
}

{
    "group": "MY_GROUP"
}

{
    "observation": {}
}
```

localRepresentation

Copyright © SDMx TWG 2020

Object optional. LocalRepresentation defines the representation for the attribute. A data attribute can be text (including XHTML and multi-lingual values), a simple value, or an enumerated value.

- enumeration - *String optional*. Urn reference to an item scheme (such as a codelist) or a value list. Dimensions cannot reference a value list.
- enumerationFormat - *Object optional*. The *enumerationFormat* object defines a restricted version of a *format* that only allows facets and text types applicable to items (codes) in the referenced scheme. Although the time facets permit any value, an actual code identifier does not support the necessary characters for time. Therefore these facets should not contain time in their values.
- format - *Object optional*. As an exclusive alternative to an item scheme reference the *format* object defines the information for describing a full range of data formats and may place restrictions on the component's values.
- minOccurs - *Non-negative Integer optional*. Indicates the minimum number of values that can be reported for the component. If missing than there is no lower limit on its occurrences. The default is 1.
- maxOccurs - *Positive Integer*/*String optional*. Indicates the maximum number of values that can be reported for the component. If set to the string "unbounded" than there is no upper limit on its occurrences. The default is 1.

The representation for the reporting year start/end day attribute that has the identifier "REPORTING_PERIOD_START_DAY"/"REPORTING_PERIOD_END_DAY" and states the month and day at which the reporting year begins/ends, does not allow for enumerated values and its text format is fixed to be a day and month in the ISO 8601 format of '–MM-DD'. Its *format* has thus only one single property *dataType* fixed to "MonthDay". This type exists solely for the purpose of fixing the representation of the reporting year start/end day attribute. Its maxOccurs property is a *Positive Integer* and is fixed to 1.

Example

```
{
    "enumeration": "urn:sdmx:org.sdmx.infomodel.codelist.Codelist=ECB:CL_ORGANISATION(1.0)",
    "enumerationFormat": {
        # enumerationFormat object #
    }
}

{
    "format": {
        # format object #
    }
}

Local representation of the reporting year start/end day attributes:

{
    "format": {
        "dataType": "MonthDay"
    },
    "maxOccurs": 1
}
```

enumerationFormat

Copyright © SDMx TWG 2020

Object optional. A restricted version of the *format* that only allows facets and text types applicable to codes. Although the time facets permit any value, an actual code identifier does not support the necessary characters for time. Therefore these facets should not contain time in their values.

- dataType - *String optional*. Describes the type of data format allowed for the representation of the component. Only the following dataTypes are supported: “String”, “Alpha”, “AlphaNumeric”, “Numeric”, “BigInteger”, “Integer”, “Long”, “Short”, “Boolean”, “URI”, “Count”, “InclusiveValueRange”, “ExclusiveValueRange”, “Incremental”, “ObservationalTimePeriod”, “StandardTimePeriod”, “BasicTimePeriod”, “GregorianTimePeriod”, “GregorianYear”, “GregorianYearMonth”, “GregorianDay”, “ReportingTimePeriod”, “ReportingYear”, “ReportingSemester”, “ReportingTrimester”, “ReportingQuarter”, “ReportingMonth”, “ReportingWeek”, “ReportingDay”, “Month”, “MonthDay”, “Day” and “Duration”. Time `dimensions` (having the id and role “TIME_PERIOD”) only support the types “ObservationalTimePeriod”, “StandardTimePeriod”, “BasicTimePeriod”, “GregorianTimePeriod”, “GregorianYear”, “GregorianYearMonth”, “GregorianDay”, “ReportingTimePeriod”, “ReportingYear”, “ReportingSemester”, “ReportingTrimester”, “ReportingQuarter”, “ReportingMonth”, “ReportingWeek” and “ReportingDay”. The default data type is “String” except for time `dimensions`, which takes “ObservationalTimePeriod” as default.
- isSequence - *Boolean optional*. Indicates whether the values are intended to be ordered, and it may work in combination with the interval, startValue, and endValue attributes or the timeInterval, startTime, and endTime, attributes. If this attribute holds a value of ‘true’, a start value or time and a numeric or time interval must supplied. If an end value is not given, then the sequence continues indefinitely.
- interval - *Integer optional*. Specifies the permitted interval (increment) in a sequence. In order for this to be used, the isSequence attribute must have a value of ‘true’.
- startValue - *Number optional*. Is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates the starting point of the sequence. This value is mandatory for a numeric sequence to be expressed.
- endValue - *Number optional*. Is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates that ending point (if any) of the sequence.
- timeInterval - *String optional*. Indicates the permitted duration in a time sequence. It complies with the time duration specification of the ISO 8601 standard. In order for this to be used, the isSequence attribute must have a value of ‘true’.
- startTime - *String optional*. Is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates the start time of the sequence. This value is mandatory for a time sequence to be expressed. It must be a valid standard time period (gYear, gYearMonth, date, dateTime and SDMX time periods).
- endTime - *String optional*. Is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates that ending point (if any) of the sequence. It must be a valid standard time period (gYear, gYearMonth, date, dateTime and SDMX time periods).
- minLength - *Positive integer optional*. Specifies the minimum length of the value in characters.
- maxLength - *Positive integer optional*. Specifies the maximum length of the value in characters.
- minValue - *Number optional*. Is used for inclusive and exclusive ranges, indicating what the lower bound of the range is. If this is used with an inclusive range, a valid value will be greater than or equal to the value specified here. By default, the minValue is assumed to be inclusive.
- maxValue - *Number optional*. Is used for inclusive and exclusive ranges, indicating what the upper bound of the range is. If this is used with an inclusive range, a valid value will be less than or equal to the value specified here. By default, the maxValue is assumed to be inclusive.
- pattern - *String optional*. Holds any standard regular expression.

Example

```
{
    "dataType": "String",
    "pattern": "^[0-9][0-9]$"
}
```

format

Copyright © SDMx TWG 2020

Object optional. Format defines the information for describing a range of data formats restricted to the representations allowed for all components except for target objects.

- dataType - *String optional*. Describes the type of data format allowed for the representation of the component. Only the following dataTypes are supported: “String”, “Alpha”, “AlphaNumeric”, “Numeric”, “BigInteger”, “Integer”, “Long”, “Short”, “Decimal”, “Float”, “Double”, “Boolean”, “URI”, “Count”, “InclusiveValueRange”, “ExclusiveValueRange”, “Incremental”, “ObservationalTimePeriod”, “StandardTimePeriod”, “BasicTimePeriod”, “GregorianTimePeriod”, “GregorianYear”, “GregorianYearMonth”, “GregorianDay”, “ReportingTimePeriod”, “ReportingYear”, “ReportingSemester”, “ReportingTrimester”, “ReportingQuarter”, “ReportingMonth”, “ReportingWeek”, “ReportingDay”, “DateTime”, “TimeRange”, “Month”, “MonthDay”, “Day”, “Time”, “Duration”, “GeospatialInformation” and “XHTML”. `Dimensions` do not support the type “XHTML”. Time `dimensions` (having the id and role “TIME_PERIOD”) only support the types “ObservationalTimePeriod”, “StandardTimePeriod”, “BasicTimePeriod”, “GregorianTimePeriod”, “GregorianYear”, “GregorianYearMonth”, “GregorianDay”, “ReportingTimePeriod”, “ReportingYear”, “ReportingSemester”, “ReportingTrimester”, “ReportingQuarter”, “ReportingMonth”, “ReportingWeek”, “ReportingDay”, “DateTime”, “TimeRange”. The default data type is “String” except for time `dimensions`, which takes “ObservationalTimePeriod” as default.
- isSequence - *Boolean optional*. Indicates whether the values are intended to be ordered, and it may work in combination with the interval, startValue, and endValue attributes or the timeInterval, startTime, and endTime, attributes. If this attribute holds a value of ‘true’, a start value or time and a numeric or time interval must supplied. If an end value is not given, then the sequence continues indefinitely.
- interval - *Integer optional*. Specifies the permitted interval (increment) in a sequence. In order for this to be used, the isSequence attribute must have a value of ‘true’.
- startValue - *Number optional*. Is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates the starting point of the sequence. This value is mandatory for a numeric sequence to be expressed.
- endValue - *Number optional*. Is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates that ending point (if any) of the sequence.
- timeInterval - *String optional*. Indicates the permitted duration in a time sequence. It complies with the time duration specification of the ISO 8601 standard. In order for this to be used, the isSequence attribute must have a value of ‘true’.
- startTime - *String optional*. Is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates the start time of the sequence. This value is mandatory for a time sequence to be expressed. It must be a valid standard time period (gYear, gYearMonth, date, dateTime and SDMX time periods).
- endTime - *String optional*. Is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates that ending point (if any) of the sequence. It must be a valid standard time period (gYear, gYearMonth, date, dateTime and SDMX time periods).
- minLength - *Positive integer optional*. Specifies the minimum length of the value in characters.
- maxLength - *Positive integer optional*. Specifies the maximum length of the value in characters.
- minValue - *Number optional*. Is used for inclusive and exclusive ranges, indicating what the lower bound of the range is. If this is used with an inclusive range, a valid value will be greater than or equal to the value specified here. By default, the minValue is assumed to be inclusive.
- maxValue - *Number optional*. Is used for inclusive and exclusive ranges, indicating what the upper bound of the range is. If this is used with an inclusive range, a valid value will be less than or equal to the value specified here. By default, the maxValue is assumed to be inclusive.
- decimals - *Positive integer optional*. Indicates the number of characters allowed after the decimal separator.
- pattern - *String optional*. Holds any standard regular expression.
- isMultiLingual - *Boolean optional*. **Only for `measures` and `attributes`.** This indicates for a text format of type “String” or “XHTML”, whether the uncoded component value should allow for multiple values in different languages. The default is `false`.
- sentinelValues - *Array* of *Object*s *optional*. When present, the sentinelValues array indicates that sentinel values are defined for the data format. Each *sentinelValue* object indicates a reserved value in an otherwise open value domain that holds a specific meaning. For example, a value of -1 can be defined to indicate a non-applicable value.

Copyright © SDMx TWG 2020

Example

```
{
    "dataType": "String",
    "maxLength": 1050,
    "pattern": "^[A-Za-z][A-Za-z0-9_-]*$",
    "isMultilingual": true,
    "sentinelValues": [
        # sentinelValue object #
    ]
}
```

sentinelValue

Object. It defines a reserved value (within the value domain of the data format) along with its meaning.

- value - *Number* or *String*. The sentinel value being described.
- name - *String*. Human-readable (best-language-match) name (or meaning) of the sentinel value.
- names - *Object optional*. Human-readable localised *names* (or meanings) of the sentinel value.
- description - *String optional*. Human-readable (best-language-match) description for the sentinel value.
- descriptions - *Object optional*. Human-readable localised descriptions (see *names*) for the sentinel value.

Example

```
{
    "value": -1,
    "name": "Special meaning",
    "names": {
        "en": "Special meaning",
        "fr": "Signification particulière"
    },
    "description": "Description for special meaning.",
    "descriptions": { "en": "Description for special meaning.",
              "fr": "Description de signification particulière." }
}
```

metadataAttributeUsage

Object. MetadataAttributeUsage defines how a metadata attribute is used in a data structure. This is a local reference to a metadata attribute from the metadata structure referenced by this data structure. An attribute relationship can be defined in order to describe the relationship of the metadata attribute to the data structure components.

- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.
- metadataAttributeReference - *String*. MetadataAttributeReference is a local (nested ID) reference to a metadata attribute defined in the metadata structure referenced by this data structure.
- attributeRelationship - *Object*. The *attributeRelationship* object defines the relationship between the referenced metadata attribute and the components of the data structure.

Example

```
{
    "metadataAttributeReference": "ATTR1.ATTR1-1",
    "attributeRelationship": {
        # attributeRelationship object #
    }
}
```

dimensionList

Object optional. DimensionList describes the key descriptor for a data structure definition. The order of the declaration of child dimensions is significant: it is used to describe the order in which they will appear in data formats for which key values are supplied in an ordered fashion (exclusive of the time dimension, which is not represented as a member of the ordered key). Any

Copyright © SDMx TWG 2020

data structure definition which uses the time dimension should also declare a frequency dimension, conventionally the first dimension in the key (the set of ordered non-time dimensions). It is not necessary to assign a time dimension, as data can be organised in any fashion required.

- id - *String optional*. Identifier for the dimensionList. However, this value is fixed to “DimensionDescriptor”.
- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.
- dimensions - *Array optional* of *dimension* objects that describe the structure of a dimension, which is defined as a statistical concept used (most probably together with other statistical concepts) to identify a statistical series, such as a time series, e.g. a statistical concept indicating certain economic activity or a geographical reference area. The list must not include the time dimension.
- timeDimension - *Object optional*. The *timeDimension* object describes a special dimension which designates the period in time in which the data identified by the full series key applies.

Example

```
{
    "id": "DimensionDescriptor",
    "dimensions": [
        {
            # dimension object #
        }
    ],
    "timeDimension": {
        # timeDimension object #
    },
}
```

dimension

Object optional. Dimension describes the structure of an ordinary dimension, which is defined as a statistical concept used (most probably together with other statistical concepts) to identify a statistical series, such as a time series, e.g. a statistical concept indicating certain economic activity or a geographical reference area. The dimension takes its semantic, and in some cases it representation, from its concept identity. A dimension can be coded by referencing a code list from its coded local representation. It can also specify its text format, which is used as the representation of the dimension if a coded representation is not defined. Neither the coded or uncoded representation are necessary, since the dimension may take these from the referenced concept.

- id - *String optional*. Identifier for the dimension. If not provided, the id is taken from the identifying concept.
- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.
- position - *Integer optional*. Positive integer (minimum: 0). The order of the dimensions in the key descriptor (DimensionList element) defines the order of the dimensions in the data structure, starting at 0. This position attribute explicitly specifies the position of the dimension in the data structure. It is optional and if specified must be consistent with the position of the dimension in the key descriptor.
- conceptIdentity - *String*. Urn reference to a concept where the identification of the concept scheme which defines it is contained in another context.
- conceptRoles - *Array* of *Strings optional*. ConceptRole references concepts (through URNs) which define roles which this dimension serves. If the concept from which the dimension takes its identity also defines a role the concept serves, then the isConceptRole indicator can be set to true on the concept identity rather than repeating the reference here.
- localRepresentation - *Object optional*. The *localRepresentation* object defines the representation for the dimension. Note that for dimensions the maxOccurs property must be 1, thus cannot be changed. Also the isMultiLingual format property cannot be set to true for dimensions.

Copyright © SDMx TWG 2020

Example

```
{
    "id": "FREQ",
    "position": 0,
    "conceptIdentity": "urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=ECB:ECB_CONCEPTS(1.0).FREQ",
    "conceptRoles": [
        "urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=ECB:ECB_CONCEPTS(1.0).FREQ"
    ],
    "localRepresentation": {
        # localRepresentation object #
    }
}
```

timeDimension

Object optional. TimeDimension describes the structure of a time dimension. The time dimension takes its semantic from its concept identity (usually the TIME_PERIOD concept), yet is always has a fixed identifier (TIME_PERIOD). The time dimension always has a fixed text format, which specifies that its format is always the in the value set of the observational time period (see common:ObservationalTimePeriodType). It is possible that the format may be a sub-set of the observational time period value set. For example, it is possible to state that the representation might always be a calendar year. See the enumerations of the dataType attribute in the localRepresentation/format for more details of the possible sub-sets. It is also possible to facet this representation with start and end dates. The purpose of such facts is to restrict the value of the time dimension to occur within the specified range. If the time dimension is expected to allow for the standard reporting periods (see common:ReportingTimePeriodType) to be used, then it is strongly recommended that the reporting year start day attribute also be included in the data structure definition. When the reporting year start day attribute is used, any standard reporting period values will be assumed to be based on the start day contained in this attribute. If the reporting year start day attribute is not included and standard reporting periods are used, these values will be assumed to be based on a reporting year which begins January 1.

- id - *String optional*. Identifier for the time dimension. Fixed to “TIME_PERIOD”.
- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.lowing dimension to occur in any order.
- conceptIdentity - *String*. Urn reference to a concept where the identification of the concept scheme which defines it is contained in another context.
- localRepresentation - *Object*. The *localRepresentation* object has only one required property *format* of type *timeDimensionFormat* which defines the representation for the time dimension.

Example

```
{
    "id": "TIME_PERIOD",
    "conceptIdentity": "urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=ECB:ECB_CONCEPTS(1.0).TIME_PERIOD",
    "localRepresentation": {
        "format": {
            # timeDimension format object #
        }
    }
}
```

timeDimensionFormat

Copyright © SDMx TWG 2020

Object optional. The timeDimension format only allows time based format and specifies a default ObservationalTimePeriod representation and facets of a start and end time.

- endTime - *String optional*. End time for the time dimension.
- startTime - *String optional*. Start time for the time dimension.
- dataType - *String optional*. Any of the following values: ObservationalTimePeriod, StandardTimePeriod, BasicTimePeriod, GregorianTimePeriod, GregorianYear, GregorianYearMonth, GregorianDay, ReportingTimePeriod, ReportingYear, ReportingSemester, ReportingTrimester, ReportingQuarter, ReportingMonth, ReportingWeek, ReportingDay, DateTime, TimeRange.
- sentinelValues - *Array optional*. When present, the sentinelValues array indicates that sentinel values are defined for the text format. Each *sentinelValue* object indicates a reserved value in an otherwise open value domain that holds a specific meaning.

Example

```
{
    "endTime": "2050",
    "startTime": "1960",
    "dataType": "ObservationalTimePeriod",
    "sentinelValues": [
        # sentinelValue object #
    ]
}
```

group

Object optional. Group describes the structure of a group descriptor in a data structure definition. A group consist of a of partial key to which attributes may be attached. The purpose of a group is to specify attributes values which have the same value based on some common dimensionality. All groups declared in the data structure must be unique - that is, you may not have duplicate partial keys. All groups must be given unique identifiers.

- id - *String*. Identifier for the group.
- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.
- groupDimensions - *Array optional*. Contains local references (by ID) to dimensions in the key descriptor (DimensionList). Although it is conventional to declare dimensions in the same order as they are declared in the ordered key, there is no requirement to do so - the ordering of the values of the key are taken from the order in which the dimensions are declared. Note that the id of a dimension may be inherited from its underlying concept - therefore this reference value may actually be the id of the concept.

Example

```
{
    "id": "GROUP1",
    "groupDimensions": [
        "CURRENCY",
        "CURRENCY_DENOM"
    ]
}
```

measureList

Object optional. MeasureList describes the structure of the measure descriptor for a data structure definition.

- id - *String optional*. Identifier for the measureList. Fixed to “MeasureDescriptor”.
- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.
- measures - *Array*. The *measure* object describes the definition of a measure, which is the concept that is the value of the phenomenon to be measured in a data set. Although this may take its semantic from any concept, for a unique measure the common identifier “OBS_VALUE” is recommended.

Copyright © SDMx TWG 2020

Example

```
{
    "id": "MeasureDescriptor",
    "measures": [
        {
            # measure object #
        }
    ]
}
```

measure

Object optional. Measure defines the structure of a measure, which is the concept that is the value of the phenomenon to be measured in a data set. In addition to the identifying concept and representation, a usage status and max occurs can be defined. In case of a unique measure, conventionally the use of the “OBS_VALUE” concept is recommended. A measure can be coded by referencing a code list from its coded local representation. It can also specify its text format, which is used as the representation of the measure if a coded representation is not defined. Neither the coded or uncoded representation are necessary, since the measure may take these from the referenced concept.

- id - *String optional*. Identifier for the measure.
- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.
- conceptIdentity - *String*. Urn reference to a concept where the identification of the concept scheme which defines it is contained in another context.
- conceptRoles - *Array* of *Strings optional*. ConceptRole references (through URNs) the concepts which define roles which this measure serves. If the concept from which the measure takes its identity also defines a role the concept serves, then the isConceptRole indicator can be set to true on the concept identity rather than repeating the reference here.
- localRepresentation - *Object optional*. The *localRepresentation* object defines the representation for the measure.
- usage - Constant *String* `mandatory` or `optional` *optional*. Indication whether reporting a given measure is mandatory or optional. Default: `optional`.

Example

```
{
    "id": "OBS_VALUE",
    "conceptIdentity": "urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=ECB:ECB_CONCEPTS(1.0).OBS_VALUE",
    "conceptRoles": [
        "urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=ECB:ECB_CONCEPTS(1.0).OBS_VALUE"
    ],
    "localRepresentation": {
        # localRepresentation object #
    },
    "usage": "optional"
}
```

metadataStructure

Object. MetadataStructure provides the details of a metadata structure definition, which is defined as a collection of metadata concepts and their structure when used to collect or disseminate reference metadata. A metadata structure definition performs several functions: it defines a presentational organization of metadata concepts against which reference metadata may be reported. The structure of a reference metadata message is derived from this presentational structure.

See *Common SDMX artefact properties*.

- metadataStructureComponents - *Object optional*. The *dataStructureComponents* object defines the grouping of the sets of the components that make up the metadata structure definition.

Copyright © SDMx TWG 2020

Example

```
{
    "id": "DSD1",
    "version": "1.0.0",
    "agencyID": "SDMX",
    "metadadataStructureComponents": {
        # metadataStructureComponent object #
    }
}
```

metadataStructureComponents

Object. MetadataStructureComponents describes the structure of one set of components, the metadata attribute list, that make up the metadata structure definition.

- metadataAttributeList - *Object optional*. The *metadataAttributeList* defines the set of metadata attributes that can be defined as a hierarchy, for reporting reference metadata about a target object. The identification of metadata attributes must be unique at any given level of the metadata structure. Although there are XML schema constraints to help enforce this, these only apply to explicitly assigned identifiers. Identifiers inherited from a concept from which a metadata attribute takes its identity cannot be validated against this constraint. Therefore, systems processing metadata structure definitions will have to perform this check outside of the XML validation.

Example

```
{
    "metadataAttributeList": {
        # metadataAttributeList object #
    }
}
```

metadataAttributeList

Object. MetadataAttributeList describes the structure of a meta data attribute list. It comprises a set of metadata attributes that can be defined as a hierarchy.

- id - *String*. The id attribute is provided in this case for completeness. However, its value is fixed to “MetadataAttributeDescriptor”.
- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.
- metadataAttributes - *Array optional*. The *metadataAttribute* object defines the a metadata attribute, which is the value of an attribute, such as the instance of a coded or uncoded attribute in a metadata structure definition.

Example

```
{
    "id": "MetadataAttributeDescriptor",
    "metadataAttributes": [
        {
            # metadataattribute object #
        }
    ]
}
```

metadataAttribute

Object. MetadataAttribute describes the structure of a metadata attribute. The metadata attribute takes its semantic, and in some cases it representation, from its concept identity. A metadata attribute may be coded (via the local representation), uncoded (via the text format), or take no value. In addition to this value, the metadata attribute may also specify subordinate metadata attributes. If a metadata attribute only serves the purpose of containing subordinate metadata attributes, then the isPresentational attribute should be used. Otherwise, it is assumed to also take a value. If the metadata attribute does take a value, and a representation is not defined, it will be inherited from the concept it takes its semantic from. The optional id on the metadata

Copyright © SDMx TWG 2020

attribute uniquely identifies it within the metadata structured definition. If this id is not supplied, its value is assumed to be that of the concept referenced from the concept identity. Note that a metadata attribute (as identified by the id attribute) definition must be unique across the entire metadata structure definition (including target identifier, identifier component, and report structure ids). A metadata attribute may be used at different levels, but the content (value and/or child metadata attributes and their cardinality) of the metadata attribute cannot change.

- id - *String optional*. Identifier for the metadata attribute.
- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.
- conceptIdentity - *String*. Urn reference to a concept where the identification of the concept scheme which defines it is contained in another context.
- localRepresentation - *Object optional*. The *localRepresentation* object defines the representation for the attribute. Note that the localRepresentation's `minOccurs` and `maxOccurs` properties are prohibited for this component type. The number of values that can be reported for a metadata attribute is always 1.
- minOccurs - *Non-negative integer optional*. Indicates the minimum number of times this metadata attribute must occur within its parent object. If missing than there is no lower limit on its occurrences. The default is 1.
- maxOccurs - *Positive integer/String optional*. Indicates the maximum number of times this metadata attribute can occur within its parent object. If set to the string "unbounded" than there is no upper limit on its occurrences. The default is 1.
- isPresentational - *Boolean optional*. The isPresentational attribute indicates whether the metadata attribute should allow for a value. A value of true, meaning the metadata attribute is presentational means that the attribute only contains child metadata attributes, and does not contain a value. If this attribute is not set to true, and a representation (coded or uncoded) is not defined, then the representation of the metadata attribute will be inherited from the concept from which it takes its identity. The default is false.
- metadataAttributes - *Array optional*. The *metadataAttribute* object defines the a child metadata attribute.

Example

```
{
    "id": "META_ATTR",
    "conceptIdentity": "urn:sdmx:org.sdmx.infomodel.conceptscheme.Concept=OECD:OECD_CONCEPTS(1.0).META_ATTR",
    "localRepresentation": {
        # localRepresentation object #
    },
    "minOccurs": 2,
    "maxOccurs": 5,
    "isPresentational": true,
    "metadataAttributes": [
        {
            # metadataattribute object #
        }
    ]
}
```

categoryScheme

Object. Describes the structure of a category scheme. A category scheme is the descriptive information for an arrangement or division of categories into groups based on characteristics, which the objects have in common. This provides for a simple, leveled hierarchy or categories.

See *Common SDMX artefact properties*. See *common properties of SDMX artefacts of base type "ItemScheme"*. There are no additional parameters. The start, end and parent properties are not used.

Copyright © SDMx TWG 2020

Example


```
{
    "id": "TOPICS",
    "version": "1.0.0",
    "agencyID": "SDMX",
    "name": "Topics",
    "names": {
        "en-GB-oed": "Topics",
        "fr": "Thèmes"
    },
    "isPartial": true,
    "categories": [
        {
            "id": "TOPIC1",
            "name": "Topic 1",
            "names": {
                "en-GB-oed": "Topic 1",
                "fr": "Thème 1"
            },
            "categories": [
                {
                    "id": "SUBTOPIC11",
                    "name": "Topic 11",
                    "names": {
                        "en-GB-oed": "Topic 11",
                        "fr": "Thème 11"
                    }
                }
            ]
        }
    ]
}
```


conceptScheme

Object. Describes the structure of a concept scheme. A concept scheme is the descriptive information for an arrangement or division of concepts into groups based on characteristics, which the objects have in common. It contains a collection of concept definitions, that may be arranged in simple hierarchies.

See *Common SDMX artefact properties*. See *common properties of SDMX artefacts of base type "ItemScheme"*.

In addition, `conceptScheme`'s *item* artefacts share the following common object properties:

- coreRepresentation - *Object optional*. The *coreRepresentation* object defines the core representation that are allowed for a concept. The text format allowed for a concept is that which is allowed for any non-target object component.
- isoConceptReference - *Object optional*. The *isoConceptReference* object provides a reference to an ISO 11179 concept.
- parent - *String optional*. Urn reference to a local concept. Parent captures the semantic relationships between concepts which occur within a single concept scheme. This identifies the concept of which the current concept is a qualification (in the ISO 11179 sense) or subclass. The start and end properties are not used.

Example


```
{
    "id": "CS_BOP",
    "name": "Balance of Payments Concept Scheme",
    "names": {
        "en": "Balance of Payments Concept Scheme",
        "fr": "Schéma des concepts de Balance des paiements"},
    "agencyId": "IMF",
    "version": "1.9.0",
    "concepts": [
        {
            "id": "FREQ",
            "name": "Frequency",
            "names": {
                "en": "Frequency",
                "fr": "Fréquence"
            },
            "coreRepresentation": {
                # coreRepresentation object #
            },
            "isoConceptReference": {
                # isoConceptReference object #
            },
            "parent": "COMMON_CONCEPTS"
        }
    ]
}
```


Copyright © SDMx TWG 2020

coreRepresentation

Object optional. A core representation for a concept. It is either a reference to a codelist which enumerates the possible values that can be used as the representation of this concept, or a text format.

- enumeration - *String optional*. Urn reference to a codelist or valuelist which enumerates the possible values that can be used as the representation of this concept. This must be a valid SDMX Registry URN (see SDMX Registry Specification for details) of a codelist.
- enumerationFormat - *Object optional*. To be used only with a codelist reference. The *enumerationFormat* object defines a restricted version of a text format that only allows facets and text types applicable to codes. Although the time facets permit any value, an actual code identifier does not support the necessary characters for time. Therefore these facets should not contain time in their values.
- format - *Object optional*. As an exclusive alternative to a codelist reference the *format* object defines the information for describing a full range of data formats and may place restrictions on the component's values.
- minOccurs - *Non-negative Integer optional*. Indicates the minimum number of values that can be reported for the component. If missing than there is no lower limit on its occurrences. The default is 1.
- maxOccurs - *Positive Integer/String optional*. Indicates the maximum number of values that can be reported for the component. If set to the string "unbounded" than there is no upper limit on its occurrences. The default is 1.

Example

```
{
    "enumeration": "urn:sdmx:org.sdmx.infomodel.codelist.Codelist=SDMX:CL_FREQ(2.0)",
    "enumerationFormat": {
        # enumerationFormat object #
    }
}

{
    "format": {
        # format object #
    }
}
```

isoConceptReference

Object. Provides a reference to an ISO 11179 concept.

- conceptAgency - *String*.
- conceptID - *String*.
- conceptSchemeID - *String*.

codelist

Object. Defines the structure of a codelist. A codelist is defined as a list from which some statistical concepts (coded concepts) take their values.

See *Common SDMX artefact properties*. See *common properties of SDMX artefacts of base type "ItemScheme"*.

Note that the SDMX Information Model does not foresee a `codes` property for codelist items (`code`), hierarchies being expressed through the `parent` property for codelist items, which contains the ID of the parent code. However, for retrieval use cases, implementers can choose to resolve the parent-child relationships also into recursive `code` properties of `code` . A `code` describes the structure of a code. A code is defined as a language independent set of letters, numbers or symbols that represent a concept whose meaning is described in a natural language. Presentational information not present may be added through the use of annotations.

In addition, `codelist` 's *item* artefacts share the following common object properties:

- parent - *String optional*. The ID of the parent code. Parent provides the ability to describe simple hierarchies within a single codelist, by referencing the ID value of another code in the same codelist. The start and end properties are not used.

Copyright © SDMx TWG 2020

Example


```
{
    "id": "CODELIST1",
    "version": "1.0.0",
    "agencyID": "SDMX",
    "name": "Code list 1",
    "names": {
        "en": "Code list 1",
        "fr": "Liste de codes 1"
    },
    "isPartial": true,
    "codes": [
        {
            "id": "CODE1",
            "name": "Code 1",
            "names": {
                "en": "Code 1",
                "fr": "Code 1"
            }
        },
        {
            "id": "CODE11",
            "name": "Code 11",
            "names": {
                "en": "Code 11",
                "fr": "Code 11"
            }
            "parent": "CODE1"
        }
    ]
}
```


geographicCodelist

Object. Describes the structure of a geographic codelist. It comprises a set of GeoFeatureSetCodes, by adding a value in the Code that follows a pattern to represent a geo feature set.

See *Common SDMX artefact properties*.

See the schema file for more information.

geoGridCodelist

Object. Describes the structure of a geographic grid code list. These define a geographical grid composed of cells representing regular squared portions of the Earth.

See *Common SDMX artefact properties*.

See the schema file for more information.

valueList

Object. Describes the structure of value list. These represent a closed set of values the can occur for a dimension, measure, or attribute. These may be values, or values with names and descriptions (similar to a codelist).

See *Common SDMX artefact properties*.

See the schema file for more information.

hierarchy

Object. Describes the structure of a hierarchy. A hierarchy is defined as an organised collection of codes that may participate in many parent/child relationships with other codes in the list.

See *Common SDMX artefact properties*.

See the schema file for more information.

hierarchyAssociation

Object. Describes the structure of a hierarchyAssociation. A hierarchyAssociation associates a hierarchy with an identifiable object in the context of another object.

See *Common SDMX artefact properties*.

Copyright © SDMx TWG 2020

See the schema file for more information.

agencyScheme

Object. Defines a specific type of organisation scheme which contains only maintenance agencies. The agency scheme maintained by a particular maintenance agency is always provided a fixed identifier and version, and is never final. Therefore, agencies can be added or removed without have to version the scheme. Agencies schemes have no hierarchy, meaning that no agency may define a relationship with another agency in the scheme. In fact, the actual parent agency for an agency in a scheme is the agency which defines the scheme.

See *Common SDMX artefact properties*. See *common properties of SDMX artefacts of base type "ItemScheme"*.

The `agencyScheme`'s *item* artefacts are agencies. Agency is an organisation which maintains structural metadata such as classifications, concepts, data structures, and metadata structures. In addition, agencies include the following object properties:

- contacts - *Array optional*. A collection of *contacts*. Provides contact information for the agency. The contacts defined for the organisation are specific to the agency role the organisation is serving.

See the schema file for more information.

Example


```
{
    "id": "AGENCIES",
    "version": "1.0.0",
    "agencyID": "SDMX",
    "isExternalReference": false,
    "name": "SDMX Agency Scheme",
    "names": {
        "en": "SDMX Agency Scheme",
        "fr": "Schéma des agences SDMX"
    },
    "links": [
        {
            "href": "https://web-service-root/agencyScheme/SDMX/AGENCIES/1.0",
            "rel": "self",
            "urn": "urn:sdmx:org.sdmx.infomodel.base.AgencyScheme=SDMX:AGENCIES(1.0)"
        }
    ],
    "isPartial": true,
    "agencies": [
        {
            "id": "SDMX",
            "name": "SDMX",
            "names": {
                "en": "SDMX",
                "fr": "SDMX"
            },
            "contacts": [
                {
                    "id": "SDMX",
                    "name": "SDMX",
                    "names": {
                        "en": "SDMX",
                        "fr": "SDMX"
                    },
                    "uris": [
                        "sdmx.org"
                    ]
                }
            ]
        }
    ]
}
```


dataProviderScheme

Object. Defines a type of organisation scheme which contains only data providers. The data provider scheme maintained by a particular maintenance agency is always provided a fixed identifier and version, and is never final. Therefore, providers can be added or removed without have to version the scheme. This scheme has no hierarchy, meaning that no organisation may define a relationship with another organisation in the scheme.

See *Common SDMX artefact properties*. See *common properties of SDMX artefacts of base type "ItemScheme"*.

See the schema file for more information.

Copyright © SDMx TWG 2020

dataConsumerScheme

Object. Defines a type of organisation scheme which contains only data consumers. The data consumer scheme maintained by a particular maintenance agency is always provided a fixed identifier and version, and is never final. Therefore, consumers can be added or removed without have to version the scheme. This scheme has no hierarchy, meaning that no organisation may define a relationship with another organisation in the scheme.

See *Common SDMX artefact properties*. See *common properties of SDMX artefacts of base type "ItemScheme"*.

See the schema file for more information.

metadataProviderScheme

Object. Defines a type of organisation scheme which contains only metadata providers. The metadata provider scheme maintained by a particular maintenance agency is always provided a fixed identifier and version, and is never final. Therefore, providers can be added or removed without have to version the scheme. This scheme has no hierarchy, meaning that no organisation may define a relationship with another organisation in the scheme.

See *Common SDMX artefact properties*. See *common properties of SDMX artefacts of base type "ItemScheme"*.

See the schema file for more information.

organisationUnitScheme

Object. Defines a type of organisation scheme which simply defines organisations and there parent child relationships. Organisations in this scheme are assigned no particular role, and may in fact exist within the other type of organisation schemes as well.

See *Common SDMX artefact properties*. See *common properties of SDMX artefacts of base type "ItemScheme"*.

See the schema file for more information.

dataflow

Object. Describes the structure of a data flow. A data flow is defined as the structure of data that will provided for different reference periods. If this type is not referenced externally, then a reference to a data structure definition must be provided.

See *Common SDMX artefact properties*.

In addition, `dataflow` has the following property:

- structure - *String optional*. Urn reference to the data structure definition which defines the structure of all data for this flow.

Example


```
{
    "id": "EXR",
    "version": "1.0.0",
    "agencyID": "ECB",
    "isExternalReference": false,
    "name": "Exchange Rates",
    "names": {
        "en": "Exchange Rates",
        "fr": "Taux de change"
    },
    "links": [
        {
            "href": "/dataflow/ECB/EXR/1.0",
            "rel": "self",
            "urn": "urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=ECB:EXR(1.0)",
        }
    ],
    "structure": "urn:sdmx:org.sdmx.infomodel.datastructure.DataStructure=ECB:ECB_EXR1(1.0)"
}
```


metadataflow

Object. Describes the structure of a metadata flow. A dataflow is defined as the structure of reference metadata that will be provided for different reference periods. If this type is not referenced externally, then a reference to a metadata structure definition must be provided.

Copyright © SDMx TWG 2020

See *Common SDMX artefact properties*.

See the schema file for more information.

reportingTaxonomy

Object. Describes the structure of a reporting taxonomy, which is a scheme which defines the composition structure of a data report where each component can be described by an independent structure or structure usage description.

See *Common SDMX artefact properties*.

See the schema file for more information.

provisionAgreement

Object. Describes the structure of a provision agreement. A provision agreement defines an agreement for a data provider to report data against a flow.

See *Common SDMX artefact properties*.

See the schema file for more information.

metadataProvisionAgreement

Object. Describes the structure of a metadata provision agreement. A metadata provision agreement defines an agreement for a metadata provider to report reference metadata against a flow.

See *Common SDMX artefact properties*.

See the schema file for more information.

structureMap

Object. Describes the structure of a structureMap. StructureMap allows mapping between data structures or dataflows.

See *Common SDMX artefact properties*.

See the schema file for more information.

representationMap

Object. Describes the structure of a representationMap. RepresentationMap allows mapping between representations.

See *Common SDMX artefact properties*.

See the schema file for more information.

conceptSchemeMap

Object. Describes the structure of a conceptSchemeMap. ConceptSchemeMap allows mapping between concepts in different concept schemes.

See *Common SDMX artefact properties*.

See the schema file for more information.

categorySchemeMap

Object. Describes the structure of a categorySchemeMap. CategorySchemeMap allows mapping between categories in different category schemes.

See *Common SDMX artefact properties*.

See the schema file for more information.

organisationSchemeMap

Object. Describes the structure of a organisationSchemeMap. OrganisationSchemeMap allows mapping between organisations in different organisation schemes.

Copyright © SDMx TWG 2020

See *Common SDMX artefact properties*.

See the schema file for more information.

reportingTaxonomyMap

Object. Describes the structure of a reportingTaxonomyMap. ReportingTaxonomyMap allows mapping between reporting categories in different reporting taxonomies.

See *Common SDMX artefact properties*.

See the schema file for more information.

process

Object. Describes the structure of a process, which is a scheme which defines or documents the operations performed on data in order to validate data or to derive new information according to a given set of rules. Processes occur in order, and will continue in order unless a transition dictates another step should occur.

See *Common SDMX artefact properties*.

See the schema file for more information.

categorisation

Object. Defines the structure for a categorisation. A source object is referenced via an object reference and the target category is referenced via the target category.

See *Common SDMX artefact properties*.

In addition, `categorisation` has the following properties:

- source - *String optional*. Source is a urn reference to an object to be categorized.
- target - *String optional*. Target is a urn reference to the category that the referenced object is to be mapped to.

Example


```
{
    "id": "53A341E8-D48B-767E-D5FF-E2E3E0E2BB19",
    "version": "1.0.0",
    "agencyID": "ECB",
    "isExternalReference": false,
    "name": "Categorise: DATAFLOWECB:EXR(1.0)",
    "names": {
        "en": "Categorise: DATAFLOWECB:EXR(1.0)",
        "fr": "Catégoriser: DATAFLOWECB:EXR(1.0)"
    },
    "links": [
        {
            "href": "/categorisation/ECB/53A341E8-D48B-767E-D5FF-E2E3E0E2BB19/1.0",
            "rel": "self",
            "urn": "urn:sdmx:org.sdmx.infomodel.categoryscheme.Categorisation=ECB:53A341E8-D48B-767E-D5FF-E2E3E0E2BB19(1.0)"
        }
    ],
    "source": "urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=ECB:EXR(1.0)",
    "target": "urn:sdmx:org.sdmx.infomodel.categoryscheme.CategoryScheme=ECB:MOBILE_NAVI(1.0).07"
}
```


dataConstraint

Object. DataConstraint specifies a sub set of the definition of the allowable or available content of a data set in terms of the content or in terms of the set of key combinations. The inclusion of a key or region in a constraint is determined by first processing the included key sets, and then removing those keys defined in the excluded key sets. If no included key sets are defined, then it is assumed that all possible keys or regions are included, and any excluded key or regions are removed from this complete set.

See *Common SDMX artefact properties*.

See the schema file for more information.

Copyright © SDMx TWG 2020

In addition, `dataConstraint` has the following properties:

- role - *String*. The type attribute indicates whether this constraint states what data is actually present for the constraint attachment (“Actual”), or if it defines what content is allowed (“Allowed”). Actual data constraints cannot be managed (retrieved or uploaded) through standard structure queries since they are to be generated dynamically through data availability queries according to the real current data availability. Actual data constraints should thus not be (semantically) versioned.
- constraintAttachment - *Object optional*. The *constraintAttachment* object describes the collection of constrainable artefacts that the constraint is attached to.
- cubeRegions - *Array optional*. A list of of *cubeRegion* objects. CubeRegion describes a set of dimension values which define a region and attributes which relate to the region for the purpose of describing a constraint.
- dataKeySets - *Array optional*. A list of of *dataKeySet* objects. DataKeySet defines a collection of full or partial data keys.
- releaseCalendar - *Object optional*. The *releaseCalendar* defines dates on which the constrained data is to be made available.

Example

```
{
    "role": "Allowed",
    "id": "EXR_CONSTRAINTS",
    "version": "1.0.0",
    "agencyID": "ECB",
    "isExternalReference": false,
    "name": "Constraints for the EXR dataflow",
    "names": {
        "en": "Constraints for the EXR dataflow",
        "fr": "Constraintes pour le dataflow EXR"
    },
    "links": [
        {
            "href": "/constraint/ECB/EXR_CONSTRAINTS/1.0",
            "rel": "self",
            "urn": "urn:sdmx:org.sdmx.infomodel.registry.ContentConstraint=ECB:EXR_CONSTRAINTS(1.0)"
        }
    ],
    "constraintAttachment": {
        # constraintAttachment object #
    },
    "cubeRegions": [
        {
            # cubeRegion object #
        }
    ],
    "dataKeySets": [
        {
            # dataKeySet object #
        }
    ],
    "releaseCalendar": {
        # releaseCalendar object #
    }
}
```

metadataConstraint

Object. MetadataConstraint specifies a sub set of the definition of the allowable content of a metadata set.

See *Common SDMX artefact properties*.

See the schema file for more information.

In addition, `metadataConstraint` has the following properties:

- role - *String*. The role attribute is fixed to “Allowed” and indicates that this constraint defines what content is allowed.
- constraintAttachment - *Object optional*. The *constraintAttachment* object describes the collection of constrainable artefacts that the constraint is attached to.
- metadataTargetRegions - *Array optional*. A list of of *metadataTargetRegion* objects which describes the values allowed for metadata attributes.
- releaseCalendar - *Object optional*. The *releaseCalendar* defines dates on which the constrained data is to be made available.

Copyright © SDMx TWG 2020

Example


```
{
    "role": "Allowed",
    "id": "EXR_CONSTRAINTS",
    "version": "1.0.0",
    "agencyID": "ECB",
    "isExternalReference": false,
    "name": "Constraints for the EXR dataflow",
    "names": {
        "en": "Constraints for the EXR dataflow",
        "fr": "Constraintes pour le dataflow EXR"
    },
    "links": [
        {
            "href": "/constraint/ECB/EXR_CONSTRAINTS/1.0",
            "rel": "self",
            "urn": "urn:sdmx:org.sdmx.infomodel.registry.ContentConstraint=ECB:EXR_CONSTRAINTS(1.0)"
        }
    ],
    "constraintAttachment": {
        # constraintAttachment object #
    },
    "metadataTargetRegions": [
        {
            # metadataTargetRegion object #
        }
    ],
    "releaseCalendar": {
        # releaseCalendar object #
    }
}
```


constraintAttachment

Object. ConstraintAttachment describes a collection of references to constrainable artefacts.

constraintAttachment properties for DataConstraints:

- dataProvider - *String optional*. DataProvider is a URN reference to a data provider to which the constraint is attached. If this is used, then only the release calendar is relevant.
- dataStructures - *Array optional* of *string*s. URN references to data structure definitions to which the constraint is attached. A constraint which is attached to more than one data structure must only express key sets and/or cube regions where the identifiers of the dimensions are common across all structures to which the constraint is attached.
- dataflows - *Array optional* of *string*s. Urn references to data flows to which the constraint is attached. A constraint can be attached to more than one dataflow, and the dataflows do not necessarily have to be usages of the same data structure. However, a constraint which is attached to more than one data structure must only express key sets and/or cube regions where the identifiers of the dimensions are common across all structures to which the constraint is attached.
- provisionAgreements - *Array optional* of *string*s. Urn references to provision agreements to which the constraint is attached. A constraint can be attached to more than one data provision agreement, and the data provision agreements do not necessarily have to be references structure usages based on the same structure. However, a constraint which is attached to more than one data provision agreement must only express key sets and/or cube/target regions where the identifier of the components are common across all structures to which the constraint is attached.

Copyright © SDMx TWG 2020

constraintAttachment properties for MetadataConstraints:

- metadataProvider - *String optional*. MetadataProvider is a URN reference to a metadata provider to which the constraint is attached. If this is used, then only the release calendar is relevant.
- metadataSets - *Array optional*. URN references to metadata sets to which the constraint is attached.
- metadataStructures - *Array optional* of *string*s. URN references to metadata structure definitions to which the constraint is attached. A constraint which is attached to more than one metadata structure must only express key sets and/or target regions where the identifiers of the target objects are common across all structures to which the constraint is attached.
- metadataflows - *Array optional* of *string*s. Urn references to metadata flows to which the constraint is attached. A constraint can be attached to more than one metadataflow, and the metadataflows do not necessarily have to be usages of the same metadata structure. However, a constraint which is attached to more than one metadata structure must only express key sets and/or target regions where the identifiers of the target objects are common across all structures to which the constraint is attached.
- metadataProvisionAgreements - *Array optional* of *string*s. Urn references to metadata provision agreements to which the constraint is attached. A constraint can be attached to more than one metadata provision agreement, and the metadata provision agreements do not necessarily have to be references structure usages based on the same structure. However, a constraint which is attached to more than one metadata provision agreement must only express key sets and/or cube/target regions where the identifier of the components are common across all structures to which the constraint is attached.

constraintAttachment properties for any constraint:

- simpleDataSources - *Array optional* of *string*s. URLs of SDMX-ML data or metadata messages.
- queryableDataSources - *Object optional*. The *queryableDataSource* object describes a queryable data source to which the constraint is attached. Used only with one of the dataStructures, dataflows, dataProvisionAgreements, metadataStructures, metadataflows and metadataProvisionAgreements properties.

Copyright © SDMx TWG 2020

Example

```
{
    "dataProvider": "urn:sdmx:org.sdmx.infomodel...."
}

{
    "dataStructures": [
        "urn:sdmx:org.sdmx.infomodel.datastructure.DataStructure=ECB:ECB_EXR1(1.0)"
    ],
    "queryableDataSources": [
        {
            # queryableDataSource object #
        }
    ]
}

{
    "dataflows": [
        "urn:sdmx:org.sdmx.infomodel.datastructure.Dataflow=ECB:EXR(1.0)"
    ],
    "queryableDataSources": [
        {
            # queryableDataSource object #
        }
    ]
}

{
    "provisionAgreements": [
        "urn:sdmx:org.sdmx.infomodel...."
    ],
    "queryableDataSources": [
        {
            # queryableDataSource object #
        }
    ]
}

{
    "metadataProvider": "urn:sdmx:org.sdmx.infomodel...."
}

{
    "metadataSets": [
        "urn:sdmx:org.sdmx.infomodel....", "urn:sdmx:org.sdmx.infomodel...."
    ]
}

{
    "metadataStructures": [
        "urn:sdmx:org.sdmx.infomodel.metadatastructure.MetadataStructure=ECB:ECB_EXR1_M(1.0)"
    ],
    "queryableDataSources": [
        {
            # queryableDataSource object #
        }
    ]
}

{
    "metadataflows": [
        "urn:sdmx:org.sdmx.infomodel.metadatastructure.Metadataflow=ECB:EXR_M(1.0)"
    ],
    "queryableDataSources": [
        {
            # queryableDataSource object #
        }
    ]
}

{
    "metadataProvisionAgreements": [
        "urn:sdmx:org.sdmx.infomodel...."
    ],
    "queryableDataSources": [
        {
            # queryableDataSource object #
        }
    ]
}

{
    "simpleDataSources": [
        "/data/EXR/M..EUR.SP00.A"
    ]
}
```

queryableDataSource

Copyright © SDMx TWG 2020

Object. QueryableDataSource describes a data source which accepts an standard SDMX Query message and responds appropriately.

- isRESTDatasource - *Boolean*.
- isWebServiceDatasource - *Boolean*.
- dataURL - *String*. DataURL contains the URL of the data source.
- wadlURL - *String optional*. WADLURL provides the location of a WADL instance on the internet which describes the REST protocol of the queryable data source.
- wsdlURL - *String optional*. WSDLURL provides the location of a WSDL instance on the internet which describes the queryable data source.

Example


```
{
    "isRESTDatasource": true,
    "isWebServiceDatasource": true,
    "dataURL": "/data/EXR/M..EUR.SP00.A"
}
```


cubeRegion

Object. CubeRegion defines the structure of a data cube region. This is based on the abstract RegionType and simply refines the key and attribute values to conform with what is applicable for dimensions and attributes, respectively. See the documentation of the base type for more details on how a region is defined.

- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.
- include - *Boolean optional*. Default: `true`. The include attribute indicates that the region is to be included or excluded within the context in which it is defined. For example, if the regions is defined as part of a content constraint, the exclude flag would mean the data identified by the region is not present.
- components - *Array optional* of *ComponentValueSet* objects containing a reference to a component (data attribute, metadata attribute, or measure) and providing a collection of values for the referenced component. This serves to state that for the key which defines the region, the components that are specified here have or do not have (depending on the include attribute of the value set) the values provided. It is possible to provide a component reference without specifying values, for the purpose of stating the component is absent (include = false) or present with an unbounded set of values. As opposed to key components, which are assumed to be wild carded if absent, no assumptions are made about the absence of a component. Only components which are explicitly stated to be present or absent from the region will be know. All unstated components for the set cannot be assumed to absent or present.
- keyValues - *Array optional* of *CubeRegionKey* objects containing a reference to a component which disambiguates the data (i.e. a dimension) and providing a collection of values for the component. The collection of values can be flagged as being inclusive or exclusive to the region being defined. Any key component that is not included is assumed to be wild carded, which is to say that the cube includes all possible values for the un-referenced key components. Further, this assumption applies to the values of the components as well. The values for any given component can only be sub-setted in the region by explicit inclusion or exclusion. For example, a dimension X which has the possible values of 1, 2, 3 is assumed to have all of these values if a key value is not defined. If a key value is defined with an inclusion attribute of true and the values of 1 and 2, the only the values of 1 and 2 for dimension X are included in the definition of the region. If the key value is defined with an inclusion attribute of false and the value of 1, then the values of 2 and 3 for dimension X are included in the definition of the region. Note that any given key component must only be referenced once in the region.

Copyright © SDMx TWG 2020

Example

```
{
    "include": true,
    "components": [
        {
            # ComponentValueSet object #
        }
    ],
    "keyValues": [
        {
            # CubeRegionKey object #
        }
    ]
}
```

ComponentValueSet

Object. ComponentValueSet defines the structure for providing values for a data attributes, measures, or metadata attributes. If no values are provided, the component is implied to include/excluded from the region in which it is defined, with no regard to the value of the component. Note that for metadata attributes which occur within other metadata attributes, a nested identifier can be provided. For example, a value of CONTACT.ADDRESS.STREET refers to the metadata attribute with the identifier STREET which exists in the ADDRESS metadata attribute in the CONTACT metadata attribute, which is defined at the root of the report structure.

- id - *String*.
- include - *Boolean optional*. The include attribute indicates whether the values provided for the referenced component are to be included are excluded from the region in which they are defined.
- removePrefix - *Boolean optional*. The removePrefix attribute indicates whether codes should keep or not the prefix, as defined in the extension of codelist.
- timeRange - *Object optional*. A *TimeRangeValue* object.
- values - Non-empty *array optional* of *String*s and *SimpleComponentValue* objects. Only one of timeRange or values properties is allowed.

Example

```
{
    "id": "EXR_TYPE",
    "include": true,
    "removePrefix": false,
    "values": {
        "NRP0", "NN00", "NRD0", "NRU1"
        # SimpleComponentValue object #
    }
}

{
    "id": "EXR_TYPE",
    "include": true,
    "removePrefix": false,
    "timeRange": {
        # TimeRangeValue object #
    }
}
```

TimeRangeValue

Copyright © SDMx TWG 2020

Object. TimeRangeValue allows a time period value to be expressed as a range. It can be expressed as the period before a period, after a period, or between two periods. Each of these properties can specify their inclusion in regards to the range.

- afterPeriod - *Object optional*. A *TimePeriodRange* object. AfterPeriod is the period after which the period is meant to cover. This date may be inclusive or exclusive in the range.
- beforePeriod - *Object optional*. A *TimePeriodRange* object. BeforePeriod is the period before which the period is meant to cover. This date may be inclusive or exclusive in the range.
- endPeriod - *Object optional*. A *TimePeriodRange* object. EndPeriod is the end period of the range. This date may be inclusive or exclusive in the range.
- startPeriod - *Object optional*. A *TimePeriodRange* object. StartPeriod is the start date or the range that the queried date must occur within. This date may be inclusive or exclusive in the range.

Example

```
{
    "afterPeriod": {
        # TimePeriodRange object #
    },
    "beforePeriod": {
        # TimePeriodRange object #
    },
    "endPeriod": {
        # TimePeriodRange object #
    },
    "startPeriod": {
        # TimePeriodRange object #
    }
}
```

TimePeriodRange

Object. TimePeriodRange defines a time period, and indicates whether it is inclusive in a range.

- period - *String optional*. Specifies a distinct time period or point in time in SDMX. The time period can either be a Gregorian calendar period, a standard reporting period, a distinct point in time, or a time range with a specific date and duration.
- isInclusive - *Boolean optional*.

Example

```
{
    "period": "2018",
    "isInclusive": true
}
```

SimpleComponentValue

Object. SimpleComponentValue contains a simple value for a component, and if that value is from a code list, the ability to indicate that child codes in a simple hierarchy are part of the value set of the component for the region.

- value - *String*.
- lang - *String optional*. IETF Language Tag according to RFC 5646 documentation for specifying locals in HTTP - *String*. The localised name.
- cascadeValues - *Boolean optional* or constant *string* `excluderoot`. CascadeValues identifies if the value should be cascaded.
- validFrom - *String optional*. A timestamp from which the value is valid. Values must follow the ISO 8601 syntax for combined dates and times, including time zone.
- validTo - *String optional*. A timestamp from which the value is superseded. Values must follow the ISO 8601 syntax for combined dates and times, including time zone.

Copyright © SDMx TWG 2020

Example

```
{
    "value": "NRP0",
    "lang": "en",
    "cascadeValues": "excluderoot",
    "validFrom": "2021-09-01",
    "validTo":"2021-09-30"
}
```

CubeRegionKey

Object. CubeRegionKey provides a set of values for a dimension for the purpose of defining a data cube region. A set of distinct value can be provided, or if this dimension is represented as time, and time range can be specified.

- id - *String*.
- include - *Boolean optional*. The include attribute indicates whether the values provided for the referenced component are to be included are excluded from the region in which they are defined.
- removePrefix - *Boolean optional*. The removePrefix attribute indicates whether codes should keep or not the prefix, as defined in the extension of codelist.
- validFrom - *String optional*. A timestamp from which the set of values is valid. Values must follow the ISO 8601 syntax for combined dates and times, including time zone.
- validTo - *String optional*. A timestamp from which the set of values is superceded. Values must follow the ISO 8601 syntax for combined dates and times, including time zone.
- timeRange - *Object optional*. A *TimeRangeValue* object.
- values - Non-empty *array optional* of *String*s and *SimpleComponentValue* objects. Only one of timeRange or values properties is allowed.

Example

```
{
    "id": "EXR_TYPE",
    "include": true,
    "removePrefix": false,
    "validFrom": "2021-09-01",
    "validTo":"2021-09-30",
    "values": {
        "NRP0", "NN00", "NRD0", "NRU1"
        # SimpleComponentValue object #
    }
}

{
    "id": "EXR_TYPE",
    "include": true,
    "removePrefix": false,
    "validFrom": "2021-09-01",
    "validTo":"2021-09-30",
    "timeRange": {
        # TimeRangeValue object #
    }
}
```

dataKeySet

Object. dataKeySet defines a collection of full or partial data keys (dimension values).

- isIncluded - *Boolean*.
- keys - Non-empty *array* of *dataKey* objects. Data Key contains a set of dimension values which identify a full set of data.

Copyright © SDMx TWG 2020

Example

```
{
    "isIncluded": true,
    "keys": [
        {
            # DataKey object #
        }
    ]
}
```

dataKey

Object. DataKey is a region which defines a distinct full or partial data key. The key consists of a set of values, each referencing a dimension and providing a single value for that dimension. The purpose of the key is to define a subset of a data set (i.e. the observed value and data attribute) which have the dimension values provided in this definition. Any dimension not stated explicitly in this key is assumed to be wild carded, thus allowing for the definition of partial data keys.

- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.
- include - *Boolean*.
- validFrom - *String optional*. A timestamp from which the region is valid. Values must follow the ISO 8601 syntax for combined dates and times, including time zone.
- validTo - *String optional*. A timestamp from which the region is superseded. Values must follow the ISO 8601 syntax for combined dates and times, including time zone.
- keyValues - Non-empty *array* of *dataKeyValue* objects.
- components - Non-empty *array* of *dataComponentValueSet* objects.

Example

```
{
    "include": "true",
    "validFrom": "2021-09-01",
    "validTo":"2021-09-30",
    "keyValues": [
        {
            # dataKeyValue object #
        }
    ],
    "components": [
        {
            # dataComponentValueSet object #
        }
    ]
}
```

dataKeyValue

Object. DataKeyValue provides a dimension value for the purpose of defining a distinct data key. Only a single value can be provided for the dimension.

- id - *String*.
- include - *Boolean optional*. The include attribute indicates whether the values provided for the referenced component are to be included are excluded from the region in which they are defined.
- removePrefix - *Boolean optional*. The removePrefix attribute indicates whether codes should keep or not the prefix, as defined in the extension of codelist.
- value - *String*.

Copyright © SDMx TWG 2020

Example

```
{
    "id": "EXR_TYPE",
    "include": true,
    "removePrefix": false,
    "value": "NRP0"
}
```

dataComponentValueSet

Object. dataComponentValueSet defines the structure for providing values for a data attributes, measures, or metadata attributes. If no values are provided, the component is implied to include/excluded from the region in which it is defined, with no regard to the value of the component. Note that for metadata attributes which occur within other metadata attributes, a nested identifier can be provided. For example, a value of CONTACT.ADDRESS.STREET refers to the metadata attribute with the identifier STREET which exists in the ADDRESS metadata attribute in the CONTACT metadata attribute, which is defined at the root of the report structure.

- id - *String*.
- include - *Boolean optional*. The include attribute indicates whether the values provided for the referenced component are to be included are excluded from the region in which they are defined.
- removePrefix - *Boolean optional*. The removePrefix attribute indicates whether codes should keep or not the prefix, as defined in the extension of codelist.
- timeRange - *Object optional*. A *TimeRangeValue* object.
- values - Non-empty *array optional* of *String*s and *datacomponentvalue* objects. Only one of timeRange or values properties is allowed.

Example

```
{
    "id": "EXR_TYPE",
    "include": true,
    "removePrefix": false,
    "values": {
        "DIM"
        # DataComponentValue object #
    }
}

{
    "id": "EXR_TYPE",
    "include": true,
    "removePrefix": false,
    "timeRange": {
        # TimeRangeValue object #
    }
}
```

DataComponentValue

Object. DataComponentValue contains a simple value for a component, and if that value is from a code list, the ability to indicate that child codes in a simple hierarchy are part of the value set of the component for the region.

- cascadeValues - *Boolean optional* or constant *string* `excluderoot`. CascadeValues identifies if the value should be cascaded.
- lang - *String optional*. IETF Language Tag according to RFC 5646 documentation for specifying locals in HTTP - *String*. The localised name.
- value - *String*.

Example

```
{
    "cascadeValues": "excluderoot",
    "lang": "en",
    "value": "NRP0"
}
```


Copyright © SDMx TWG 2020

metadataTargetRegion

Object. metadataTargetRegion defines the structure of a metadata target region. A metadata target region must define the report structure and the metadata target from that structure on which the region is based. This type is based on the abstract RegionType and simply refines the key and attribute values to conform with what is applicable for target objects and metadata attributes, respectively. See the documentation of the base type for more details on how a region is defined.

- annotations - *Array optional*. Provides a list of annotation objects. See the section annotation.
- links - *Array optional*. A collection of links to additional resources. See the section link.
- include - *Boolean optional*. The include attribute indicates that the region is to be included or excluded within the context in which it is defined. For example, if the regions is defined as part of a content constraint, the exclude flag would mean the data identified by the region is not present.
- components - *Array optional* of *MetadataAttributeValueSet* objects containing a reference to a component (data attribute, metadata attribute, or measure) and provides a collection of values for the referenced component. This serves to state that for the key which defines the region, the components that are specified here have or do not have (depending on the include attribute of the value set) the values provided. It is possible to provide a component reference without specifying values, for the purpose of stating the component is absent (include = false) or present with an unbounded set of values. As opposed to key components, which are assumed to be wild carded if absent, no assumptions are made about the absence of a component. Only components which are explicitly stated to be present or absent from the region will be know. All unstated components for the set cannot be assumed to absent or present.
- validFrom - *String optional*. A timestamp from which the region is valid. Values must follow the ISO 8601 syntax for combined dates and times, including time zone.
- validTo - *String optional*. A timestamp from which the region is superseded. Values must follow the ISO 8601 syntax for combined dates and times, including time zone.

Example

```
{
    "include": true,
    "validFrom": "2021-09-01",
    "validTo":"2021-09-30",
    "components": [
        {
            # MetadataAttributeValueSet object #
        }
    ]
}
```

MetadataAttributeValueSet

Object. MetadataAttributeValueSet defines the structure for providing values for a metadata attribute. If no values are provided, the attribute is implied to include/excluded from the region in which it is defined, with no regard to the value of the metadata attribute. Note that for metadata attributes which occur within other metadata attributes, a nested identifier can be provided. For example, a value of CONTACT.ADDRESS.STREET refers to the metadata attribute with the identifier STREET which exists in the ADDRESS metadata attribute in the CONTACT metadata attribute, which is defined at the root of the report structure.

- id - *String*.
- include - *Boolean optional*. The include attribute indicates whether the values provided for the referenced component are to be included are excluded from the region in which they are defined.
- removePrefix - *Boolean optional*. The removePrefix attribute indicates whether codes should keep or not the prefix, as defined in the extension of codelist.
- timeRange - *Object optional*. A *TimeRangeValue* object.
- values - Non-empty *array optional* of *String*s and *SimpleComponentValue* objects. Only one of timeRange or values properties is allowed.

Copyright © SDMx TWG 2020

Example

```
{
    "id": "EXR_TYPE",
    "include": true,
    "removePrefix": false,
    "values": {
        "NRP0", "NN00", "NRD0", "NRU1"
        # SimpleComponentValue object #
    }
}

{
    "id": "EXR_TYPE",
    "include": true,
    "removePrefix": false,
    "timeRange": {
        # TimeRangeValue object #
    }
}
```

releaseCalendar

Object. The ReleaseCalendar describes information about the timing of releases of the constrained data. All of these values use the standard “P7D” - style format.

- offset - *String*. Offset is the interval between January first and the first release of data within the year.
- periodicity - *String*. Periodicity is the period between releases of the data set.
- tolerance - *String*. Tolerance is the period after which the release of data may be deemed late.

Example

```
{
    "offset": "30",
    "periodicity": "12",
    "tolerance": "2019"
}
```

customTypeScheme

Object. CustomTypeScheme provides the details of a custom type scheme, in which user defined operators are described.

See *Common SDMX artefact properties*.

See the schema file for more information.

vtlMappingScheme

Object. VtlMappingScheme provides the details of a VTL mapping scheme, in which VTL mappings are described.

See *Common SDMX artefact properties*.

See the schema file for more information.

namePersonalisationScheme

Object. NamePersonalisationScheme provides the details of a name personalisation scheme, in which name personalisations are described.

See *Common SDMX artefact properties*.

See the schema file for more information.

rulesetScheme

Object. RulesetScheme provides the details of a ruleset scheme, in which rulesets are described.

See *Common SDMX artefact properties*.

See the schema file for more information.

Copyright © SDMx TWG 2020

transformationScheme

Object. TransformationScheme provides the details of a transformation scheme, in which transformations are described.

See *Common SDMX artefact properties*.

See the schema file for more information.

userDefinedOperatorScheme

Object. UserDefinedOperatorScheme provides the details of a user defined operator scheme, in which user defined operators are described.

See *Common SDMX artefact properties*.

See the schema file for more information.

ERROR

Object optional. Used to provide a error message in addition to RESTful web services HTTP error status codes. The following pieces of information are to be provided:

- code - *Number*. Provides a code number for the error message. Code numbers are defined in the SDMX 2.1 Web Services Guidelines.
- title - *String optional*. A short, human-readable (best-language-match) summary of the problem that SHOULD NOT change from occurrence to occurrence of the problem, except for purposes of localization.
- titles - *Object optional*. A list of short, human-readable localised summaries (see *names*) of the problem that SHOULD NOT change from occurrence to occurrence of the problem, except for purposes of localization.
- detail - *String optional*. A human-readable (best-language-match) explanation specific to this occurrence of the problem. Like title, this field's value can be localized. It is fully customizable by the service providers and should provide enough detail to ease understanding the reasons of the error.
- details - *Object optional*. A list of human-readable localised explanations (see *names*) specific to this occurrence of the problem. Like titles, this field's value can be localized. It is fully customizable by the service providers and should provide enough detail to ease understanding the reasons of the error.
- links - *Array optional*. *Links* field is an array of *link* objects. If appropriate, a collection of links to additional external resources for the error.

See the section on localised text elements on how the message deals with languages.

Example


```
{
    "code": 150,
    "title": "Invalid number of items in the item parameter",
    "titles": {
        "en": "Invalid number of items in the item parameter",
        "fr": "Nombre invalide d'items dans le paramètre 'item'"
    }
}
```


Copyright © SDMx TWG 2020

Linking mechanism

LINK

Object optional. A link to an external resource.

- href - *String*. Absolute or relative URL of the external resource.
- rel - *String*. Relationship of the object to the external resource. See semantics below.
- urn - *String optional*. The urn holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
- uri - *String optional*. The uri attribute holds a URI that contains a link to additional information about the resource, such as a web page. This uri is not an SDMX resource.
- title - *String optional*. A human-readable (best-language-match) description of the target link.
- titles - *Object optional*. A list of human-readable localised descriptions (see *names*) of the target link.
- type - *String optional*. A hint about the type of representation returned by the link.
- hreflang - *String optional*. The natural language of the external link, the same as used in the HTTP Accept-Language request header.

See the section on localised text elements on how the message deals with languages.

Example

```
{
    "href": "https://registry.sdmx.org/ws/rest/datastructure/ECB/ECB_EXR1",
    "rel": "self",
    "uri": "http://www.xyz.org/pdf/0123456789"
}

{
    "href": "https://registry.sdmx.org/ws/rest/dataflow/ECB.DISS/BSI_PUB/1.0",
    "rel": "dataflow",
    "urn": "urn:sdmx:org.sdmx.infomodel.datastructure.dataflow=ECB.DISS:BSI_PUB(1.0)"
}

{
    "href": "https://registry.sdmx.org/FusionRegistry/ws/rest/provisionagreement/ESTAT/PA_NAMAIN_IDC_N",
    "rel": "provisionagreement"
}

{
    "href": "https://registry.sdmx.org/help.html",
    "rel": "help",
    "title": "Documentation about the SDMX Global Registry",
    "titles": {
        "en": "Documentation about the SDMX Global Registry",
        "fr": "Documentation concernant le 'SDMX Global Registry'"
    },
    "type": "text/html",
    "hreflang": "en"
}
```

Collections of links can be attached to various elements in SDMX-JSON.

Similarily with standards such as HTML5 and Atom, link elements in SDMX-JSON *must* define a *URL* (the `href` attribute) and a *semantic* (the `rel` attribute). This allows clients to follow the links they care about and ignore the ones whose semantic they are not interested in. In addition, links in SDMX-JSON *may* define a `title` (a human-friendly description of the target link) and a `type` (a hint about the type of representation returned by the link). Please refer to the list of Media Types and Subtypes assigned and listed by the IANA for additional information about expected values for the `type` attribute.

Copyright © SDMx TWG 2020

SDMX-JSON offers a list of predefined semantics, but implementers are free to extend it. The list of predefined semantics comes from the list of SDMX artefacts that can be returned by SDMX RESTful web services, semantics defined in RFC5988 and additional items deemed to be useful in the context of statistical data dissemination. These semantics are:

- SDMX artefacts: dataStructure, metadataStructure, categoryScheme, conceptScheme, codelist, hierarchicalCodelist, organisationScheme, agencyScheme, dataProviderScheme, dataConsumerScheme, organisationUnitScheme, dataflow, metadataflow, reportingTaxonomy, provisionAgreement, structureSet, process, categorisation, contentconstraint, attachmentconstraint, category, concept, code, organisation, agency, dataProvider, dataConsumer, organisationUnit, reportingCategory, Data
- RFC5988: alternate, copyright, glossary, help, index.
- Miscellaneous: calendar (link to a release calendar), source (information about the source of data), request (the SDMX RESTful query that triggered the SDMX-JSON response).

The *URL* captured in the `href` attribute can be *absolute* or *relative*. **It is recommended to use absolute URLs in case the SDMX-JSON message is archived.**

Copyright © SDMx TWG 2020

Localised text elements

Localised best-language-match text strings (static properties matched through “Lookup”):

The first best language match according to the user’s preferred language choices expressed through the HTTP content negotiation (Accept-Language header parameter) is to be provided for each localised text element. The message does however not indicate the returned language per localised text element.

This language matching type is called “Lookup”, see https://tools.ietf.org/html/rfc4647#section-3.4.

Example

```
"name": "Frequency"
```

Localised text objects (variable properties matched through “Filtering”):

All available language matches according to the user’s preferred language choices expressed through the HTTP content negotiation (Accept-Language header parameter) is to be provided for each localised text element.

This language matching type is called “Filtering”, see https://tools.ietf.org/html/rfc4647#section-3.3.

Example

```
"names": {
    "en": "Frequency",
    "fr": "Fréquence"
}
```

The localised text object needs to be present whenever the related localised best-language-match text strings is present, and especially whenever a localised text is mandatory in the SDMX Information model. Note that localised text (and the knowledge about the locale) is mandatory in structure messages when artefacts are being submitted for storage to a registry or to other databases. The localised text object is important for use cases where multiple languages are required or where the information on the language used is required.

In case that there is no language match for a particular localisable element, it is optional to:

- return the element in a system-default language or alternatively to not return the element
- indicate available alternative languages for the element’s maintainable artefact through links to underlying localised resources

It is recommended to indicate all languages used anywhere in the message for localised elements through http Content-Language response header (languages of the intended audience) and/or through a “contentLanguages” property in the meta tag. The main language used can be indicated through the “lang” property in the meta tag.

Copyright © SDMx TWG 2020

Security Considerations

This document defines a response format for SDMX RESTful Web Services in JSON and it raises no new security considerations. SDMX Web Services Guidelines includes the security considerations associated with its usage.

Copyright © SDMx TWG 2020

Extending SDMX-JSON

The objects defined in SDMX-JSON are “open”, i.e. they can be extended with properties not defined in this specification. Providers of SDMX-JSON messages are therefore welcome to add support for features not covered in this specification. Whenever appropriate, providers who opt to do so are invited to inform us, so that future versions of SDMX-JSON may integrate these extensions, thereby improving interoperability.

The snippet below shows an example of an `error` object, extended with a `wsCustomErrorCode`:

```
"errors": [
    {
        "code": 150,
        "title": "Invalid number of items in the item parameter",
        "titles": {
            "en": "Invalid number of items in the item parameter",
            "fr": "Nombre invalide d'items dans le paramètre 'item'"
        }
        "wsCustomErrorCode": 39272
    }
]
```


Copyright © SDMx TWG 2020

References

NORMATIVE REFERENCES

[iso8601] *ISO 8601:2004 Data elements and interchange formats — Information interchange — Representation of dates and times*, ISO, 2004

[json] *The JSON Data Interchange Format*, Standard ECMA-404, ECMA International, October 2013

[sdmxim21] *Section 2: Information Model: UML Conceptual Design*, SDMX Standards 2.1, SDMX, July 2011

[sdmxws21] *Section 7: Guidelines for the Use of Web Services*, SDMX Standards 2.1, SDMX, March 2013 The latest release can be consulted here: https://github.com/sdmx-twg/sdmx-rest/

Copyright © SDMx TWG 2020

7.2.3 Data Message

SDMX-JSON Data Message

As one of the standard SDMX data exchange formats, SDMX-JSON Data Message is a JSON (JavaScript Object Notation) based data exchange message format designed for and therefore responding to the main use case of data discovery and visualisation on the web.

JSON is a widely used standardised lightweight data-interchange format. SDMX-JSON with a simple and dense but generic message format that supports different types of data structures will ease writing software for the web that easily consumes SDMX data resources. It should allow for an efficient client-server exchange with few roundtrips, small message sizes and fast parsing. The format may be used in a wide variety of programming languages and application programs. Testing and development of the format has focused on JavaScript applications running in web browsers.

The current SDMX-JSON Data Message format 2.0.0 supports the SDMX 3.0 Information Model and works together with the SDMX 3.0 RESTful Web Services API for data queries.

The SDMX-JSON format is streamable from a supporting web service in order to limit the amount of server resources.

See: - docs: for the descriptive documentation - samples: for message samples - tools: for the normative JSON schema usable for validation purposes

Copyright © SDMx TWG 2020

Introduction to SDMX-JSON Data Message 2.0.0

Let’s first start with a brief introduction of the SDMX information model.

In order to make sense of some statistical data, we need to know the concepts associated with them. For example, on its own the figure 1.2953 is pretty meaningless, but if we know that this is an exchange rate for the US dollar against the euro on 23 November 2006, it starts making more sense.

There are two types of concepts: dimensions and attributes. Dimensions, when combined, allow to uniquely identifying statistical data. Attributes on the other hand do not help identifying statistical data, but they add useful information (like the unit of measure or the number of decimals). Dimensions and attributes are known as “components”.

The measurement of some phenomenon (e.g. the figure 1.2953 mentioned above) is known as an “observation” in SDMX. Sometimes, observations can also have several measures, e.g. an estimated value can be complemented with the value corresponding to the upper confidence limit and the value corresponding to the lower confidence limit of the esimation. Observations, when exchanged, are grouped together into a “data set”. However, there can also be an intermediate grouping. For example, all exchange rates for the US dollar against the euro can be measured on a daily basis and these measures can then be grouped together, in a so-called “time series”. Similarly, you can group a collection of observations made at the same point in time, in a “cross-section” (for example, the values of the US dollar, the Japanese yen and the Swiss franc against the euro at a particular date). Of course, these intermediate groupings are entirely optional and you may simply decide to have a flat list of observations in your data set.

The SDMX information model is much richer than this limited introduction; however the above should be sufficient to understand the sdmx-json format. For additional information, please refer to the SDMX documentation.

Samples, tools and other SDMX-JSON resources are available in the public Github repository.

Before we start, let’s clarify a few more things about this guide:

- New fields may be introduced in later versions. Therefore consuming applications should tolerate the addition of new fields with ease.
- The ordering of fields in objects is undefined. The fields may appear in any order and consuming applications should not rely on any specific ordering. It is safe to consider a nulled field and the absence of a field as the same thing.
- Not all fields appear in all contexts. For example response with error status messages may not contain fields for data, dimensions and attributes.

Copyright © SDMx TWG 2020

Field Guide to SDMX-JSON Data Message 2.0.0 Objects (aligned with SDMX 3.0.0)

MESSAGE

Message is the top level object and it contains the data as well as the structural metadata needed to interpret those data.

- meta - *Object optional*. A *meta* object that contains non-standard meta-information and basic technical information about the message, such as when it was prepared and who has sent it.
- data - *Object optional*. *Data* contains the message's "primary data".
- errors - *Array optional*. *Errors* field is an array of *error* objects. When appropriate provides a list of error messages in addition to RESTful web services HTTP error status codes.

The members data and errors MUST NOT coexist in the same message.

Example

```
{
  "meta": {
      # meta object #
  },
  "data": {
      # data object #
  },
  "errors": [
      {
      # error object #
      }
  ]
}
```

META

Object optional. Used to include non-standard meta-information and basic technical information about the message, such as when it was prepared and who has sent it. Any members MAY be specified within `meta` objects.

- schema - *String optional*. Contains the URL to the schema allowing to validate the message. This also allows identifying the version of SDMX-JSON format used in this message. **Providing the link to the SDMX-JSON schema is recommended.**
- id - *String*. Unique string that identifies the message for further references.
- test - *Boolean optional*. Indicates whether the message is for test purposes or not. False for normal messages.
- prepared - *String*. A timestamp indicating when the message was prepared. Values must follow the ISO 8601 syntax for combined dates and times, including time zone.
- contentLanguages - *Array optional*. Array of strings containing the identifier of all languages used anywhere in the message for localized elements, and thus the languages of the intended audience, representing in an array format the same information than the http Content-Language response header, e.g. "en, fr-fr". See IETF Language Tags. The array's first element indicates the main language used in the message for localized elements. **The usage of this property is recommended.**
- name - *String optional*. Human-readable (best-language-match) name for the transmission.
- names - *Object optional*. Human-readable localised *names* for the transmission.
- sender - *Object*. *Sender* contains information about the party that is transmitting the message.
- receivers - *Array optional* of *Receiver* objects thats contain information about the party that is receiving the message. This can be useful if the WS requires authentication.
- links - *Array optional*. *Links* field is an array of *meta link* objects. If appropriate, a collection of links to additional external resources for the header.

See the section on localised text elements on how the message deals with languages.

Copyright © SDMx TWG 2020

Example

```
"meta": {
  "schema": "https://raw.githubusercontent.com/sdmx-twg/sdmx-json/master/data-message/tools/schemas/sdmx-json-data-schema.json",
  "copyright": "Copyright 2017 Statistics hotline",
  "id": "b1804c51-1ee3-45a9-bb75-795cd4e06489",
  "prepared": "2018-01-03T12:54:12",
  "test": false,
  "contentLanguages": [ "en", "fr-fr" ],
  "name": "Transmission name",
  "names": {
    # name object #
  },
  "sender": {
    # sender object #
  },
  "receivers": [
    {
      # receiver object #
    }
  ],
  "links": [
    {
      # link object #
    }
  ]
}
```

sender

Object. Information about the party that is transmitting the message. Sender contains the following fields:

- id - *String*. A unique identifier of the party.
- name - *String nullable*. A human-readable (best-language-match) name of the sender.
- names - *Object optional*. A list of human-readable localised *names* of the sender.
- contacts - *Array optional*. A collection of *contacts*. Provides contact information for the party in regard to the transmission of the message.

See the section on localised text elements on how the message deals with languages.

Example

```
"sender": {
  "id": "ECB",
  "name": "European Central Bank",
  "names": { "en": "European Central Bank",
        "fr": "Banque Centrale Européenne" },
  "contacts": [
    {
      # contact objects #
    }
  ]
}
```

names

Object containing all appropriate localised names, one per object property:

- One or more of: IETF Language Tag according to RFC 5646 documentation for specifying locals in HTTP - *String*. The localised name.

See the section on localised text elements on how the message deals with languages.

Example

```
{
  "en": "This is an English name",
  "en-GB": "This is a British name",
  "fr": "C'est un nom français"
}
```


Copyright © SDMx TWG 2020

contact

Object. A collection of contact details. Each object in the collection may contain the following field:

- id - *String*. Identifier for the resource.
- name - *String optional*. Human-readable (best-language-match) name of the contact.
- names - *Object optional*. Human-readable localised *names* of the contact.
- department - *String optional*. Human-readable (best-language-match) name of the organisational structure for the contact.
- departments - *Object optional*. Human-readable localised *names* of the organisational structure for the contact.
- role - *String optional*. Human-readable (best-language-match) name of the responsibility of the contact.
- roles - *Object optional*. Human-readable localised *names* of the responsibility of the contact.
- telephones - *Array optional*. An array of telephone numbers for the contact.
- faxes - *Array optional*. An array of fax numbers for the contact person.
- uris - *Array optional*. An array of uris. Each uri holds an information URL for the contact.
- emails - *Array optional*. An array of email addresses for the contact person.
- x400s - *Array optional*. An array of X.400 addresses for the contact person.

See the section on localised text elements on how the message deals with languages.

Example

```
{
  "id": "HOTLINE",
  "name": "Statistics hotline",
  "names": { "en": "Statistics hotline",
        "fr": "Service d'assistance téléphonique des Statistiques" },
  "department": "Statistics hotline",
  "departments": { "en": "Statistics hotline",
          "fr": "Service d'assistance téléphonique des Statistiques" },
  "role": "Statistics hotline",
  "roles": { "en": "Statistics hotline",
        "fr": "Service d'assistance téléphonique des Statistiques"},
  "telephones": [ "+00 0 00 00 00 00" ],
  "faxes": [ "+00 0 00 00 00 01" ],
  "uris": [ "www.xyz.org" ],
  "emails": [ "statistics@xyz.org" ]
}
```

receiver

Object optional. Information about the party that is receiving the message. This can be useful if the WS requires authentication. Receiver contains the same fields as sender.

meta link

See the section on linking mechanism for all information on links.

DATA

Object optional. Header contains the message's "primary data".

- structures - *Array optional*. *Structures* field is an array of *Structure* objects that contain the information needed to interpret the data available in the message, such as the list of concepts used. Since SDMX 3.0.0, SDMX restful web service requests can query data for several structures. Each returned structure is presented separately. Also the underlying data are presented in separate data sets.
- dataSets - *Array optional*. *DataSets* field is an array of *dataSet* objects. That's where the data (i.e.: the observations) will be.

Copyright © SDMx TWG 2020

Example

```
"data": {
  "structures": [
    {
      # structure object #
    }
  ],
  "dataSets": [
    {
      # dataSet object #
    }
  ]
}
```

STRUCTURE

Object optional. Provides the structural metadata necessary to interpret the data contained in the message. It tells you which are the components (`dimensions` , `measures` and `attributes`) used in the message and also describes to which level in the hierarchy (`dataSet` , `dimensionGroup` , `series` , `observations`) these components are attached.

- links - *Array optional*. *Links* field is an array of *structure link* objects. A collection of links to structural metadata or to additional information regarding the structure. **At least the link to the Data Structure Definition, Dataflow or Data Provision Agreement to which the data relates is required.**
- dimensions - *Object*. Describes the *dimensions* used in the message as well as the levels (`dataSet` , `series` , `observations`) at which these `dimensions` are presented.
- measures - *Object optional*. Describes the *measures* used in the message. *Measures* are always presented at the `observations` level. For backward-compatibility, the `measures` object can be omitted if there is only one measure with the ID “OBS_VALUE”. In this case, the measure values (of an indeterministic type) are written directly into the dataSet. SDMX 3+.0.0 implementations should always use the `measures` object. In case an SDMX 3+.0.0 data structure definition has no measures, the `measures` object must be present but empty.
- attributes - *Object*. Describes the *attributes* used in the message as well as the levels (`dataSet` , `dimensionGroup` , `series` , `observations`) at which these *attributes* are presented.
- annotations - *Array optional*. *Annotations* field is an array of *annotation* objects. If appropriate, provides a list of `annotations` that can be referenced by `structure` , `component` , `component value` , `dataSets` , `series` and `observations` .
- dataSets - *Array optional*. *dataSets* field is an array of integers. It contains the indexes of the *dataSet* objects in the *dataSets* array of the *data* object, which contain the data for this *structure*. If omitted, then all data included in the message are assumed to be described by the *structure*.

Example

```
{
  "links": [
    {
      # link object #
    }
  ],
  "dimensions": {
    # dimensions object #
  },
  "measures": {
    # measures object #
  },
  "attributes": {
    # attributes object #
  },
  "annotations": [
    {
      # annotation object #
    }
  ],
  "dataSets": [0]
}
```

structure link

See the section on linking mechanism for all information on links.

Copyright © SDMx TWG 2020

dimensions, measures, attributes

Object. Describes the dimensions/measures/attributes used in the message as well as the levels (`dataSet` , `dimensionGroup` , `series` , `observations`) at which these dimensions/measures/attributes are presented.

- dataSet - *Array optional*. *DataSet* field is an array of *component* objects. Optional array to be provided if components (only dimensions or attributes) are presented at the `dataSet` level. It is recommended, when technically feasible, to present all dimensions and attributes at the `dataSet` level for which the message contains only 1 single value, e.g. dimensions with single selections in the query filter and attributes that do not very with any of the dimensions.
- dimensionGroup - *Array optional*. *DimensionGroup* field is an array of *component* objects. Optional array to be provided if components (only attributes) are presented at the `dimensionGroup` level.
- series - *Array optional*. *Series* field is an array of *component* objects. Optional array to be provided if components (only dimensions or attributes) are presented at the `series` level.
- observation - *Array optional*. *Observation* field is an array of *component* objects. Optional array to be provided if components (dimensions, measures or attributes) are presented at the `observation` level. When using the SDMX API, then the dimension(s) specified in the parameter “dimensionAtObservation” would be presented at `observation` level. If “dimensionAtObservation=AllDimensions” then all dimensions, except those with only one value possibly presented at the `dataSet` level, would be presented at `observation` level. Measures must always be presented at observation level.

Example

```
"dimensions": {
  "dataSet": [
    {
      # component object #
    }
  ],
  "series": [
    {
      # component object #
    }
  ],
  "observation": [
    {
      # component object #
    }
  ]
},
"measures": {
  "observation": [
    {
      # component object #
    }
  ]
},
"attributes": {
  "dataSet": [
    {
      # component object #
    }
  ],
  "dimensionGroup": [
    {
      # component object #
    }
  ],
  "series": [
    {
      # component object #
    }
  ],
  "observation": [
    {
      # component object #
    }
  ]
}
```

component

Object optional. A component represents a `dimension` , a `measure` or an `attribute` used in the message. It contains basic information about the component (such as its `name` and `id`) as well as the list of `values` used in the message for this particular component. Dimensions must always present their values in the `values` array because the dataSets will only be able to use the corresponding indexes of these values in the `values` array. Measures should present their values in the `values` array when they are coded. If they are non-coded then the measure values are directly written into the dataSets. Attributes should present their values

Copyright © SDMx TWG 2020

in the `values` array at least when they are coded, or if they are presented at dataset, group or series level (in order to avoid repetition). If they are non-coded and presented at observation level then instead of using the component's `values` array, the attribute values can be directly written into the dataSets.\ Each of the components may contain the following fields:

- id - *String*. Identifier for the component.
- name - *String optional*. Human-readable (best-language-match) name for the component.
- names - *Object optional*. Human-readable localised *names* for the component.
- description - *String optional*. Human-readable (best-language-match) description for the component.
- descriptions - *Object optional*. Human-readable localised descriptions (see *names*) for the component.
- keyPosition - *Number*. **This field is mandatory for `dimensions` but not to be supplied for `measures` or `attributes`.** Indicates the position of the `dimension` in the Data Structure Definition, starting at 0. It needs to be provided also for the special `dimensions` such as Time or Measure dimensions. The information in this field is consistent with the order of `dimensions` in the "key" parameter string (i.e. D.USD.EUR.SP00.A) for data queries in the SDMX API.
- roles - *Array* of *Strings optional*. Defines the component role(s), if any. Roles are represented by the id of a concept defined as SDMX cross-domain concept. Several of the concepts defined as SDMX cross-domain concepts are useful for data visualisation, such as for example, the series title ("TITLE"), the unit of measure ("UNIT_MEASURE"), the number of decimals to be displayed ("DECIMALS"), the country or geographic reference area ("REF_AREA", e.g. when using maps), the period of time to which the measured observation refers ("REF_PERIOD"), the time interval at which observations occur over a given time period ("FREQ"), etc. It is strongly recommended to identify any component that can be useful for data visualisation purposes by using the appropriate SDMX cross-domain concept as role.
- isMandatory - *Boolean optional*. **Only for `measures` and `attributes`.** If `true` then that `measure` or `attribute` is mandatory in the exchange of complete datasets. The default is `false`.
- relationship - *Object*. **This field is mandatory for `attributes` but not to be supplied for `measures` or `dimensions`.** *Attribute relationship* describes how the value of this attribute varies with the values of other components. This relationship expresses the attachment level of the attribute as defined in the data structure definition. Depending on the message context (especially the data query) an attribute value can however be attached physically in the message at a different level.
- format - *Object optional*. *Format* describes the allowed components representation (permitted type and cardinality of the values). It contains essential information to determine the location of the component values and the related necessity of using indexes for referencing. It is only used when the component values are not defined by an enumerated list of identifiable items (codelist).
- default - *String* or *Number optional*. Defines a default `value` for the component (**valid for `attributes` only!**). If no value is provided in the data part of the message then this value applies.
- links - *Array optional*. *Links* field is an array of *component link* objects. If appropriate, a collection of links to additional information regarding the component.
- annotations - *Array optional*. *Annotations* is a collection of indices of the corresponding *annotations* for the component. Indices refer back to the array of *annotations* in the structure field.
- values - *Array optional*. *Values* field is an array of *component value* objects. Note that `dimensions` and `attributes` presented at `dataSet` level can only have one single component value. ***Values* are mandatory for `dimensions`**, but *optional* for `measures` and `attributes`. Whenever the *values* field is provided, the component values in the dataSets will always contain the corresponding array indexes, otherwise they will contain the values themselves.

See the section on localised text elements on how the message deals with languages.

Copyright © SDMx TWG 2020

Example

```
{
  "id": "FREQ",
  "name": "Frequency",
  "names": { "en": "Frequency",
      "fr": "Fréquence" },
  "description": "The time interval at which observations occur over a given time period.",
  "descriptions": { "en": "The time interval at which observations occur over a given time period.",
        "fr": "L'intervalle de temps avec lequel les observations sont faites sur une période donnée." },
  "keyPosition": 0,
  "role": [ "FREQ" ],
  "format": {
    # format object #
  },
  "links": [
    {
      # link object #
    }
  ],
  "annotations": [ 2, 35 ],
  "values": [
    {
      # component value object #
    }
  ]
}

{
  "id": "OBS_VALUE",
  "name": "Observation value",
  "names": { "en": "Observation value",
      "fr": "Valeur d'observation" },
  "role": [ "PRIMARY" ],
  "format": {
    # format object #
  }
}

{
  "id": "SOURCE",
  "name": "Source",
  "names": { "en": "Source",
        "fr": "Source" },
  "description": "The source of the data depending on the time period.",
  "descriptions": { "en": "The source of the data depending on the time period.",
            "fr": "La source des données dependante de la période de temps." },
  "relationship": {
    # attribute-relationship object #
  },
  "format": {
    # format object #
  },
  "default": "MAFF_Agricultural Statistics",
  "links": [
    {
      # link object #
    }
  ],
  "annotations": [ 2, 35 ],
  "values": [
    {
      # component value object #
    }
  ]
}
```

component link

See the section on linking mechanism for all information on links.

attribute relationship

Object. The attribute's relationship defines the relationship between an attribute and other data structure definition components as defined in the data structure definition. It provides the original "attachment level", but depending on the message context

Copyright © SDMx TWG 2020

(especially the data query) an attribute value can however be presented physically in the message at a different level. The attribute relationship serves also to define the scope, meaning to which measures an attribute applies.

- dataflow - Empty *Object optional*. This means that the value of the attribute **varies** per dataflow. It is the data modeller's responsibility to design or use non-overlapping dataflows that do not have observations in common, otherwise the integrity of dataflow-specific attribute values is not assured by the model, e.g. when querying those data through its DSD.
- dimensions - *Array* of *Strings optional*. One or more ID(s) of (a) local dimension(s) in the data structure definition on which the value of this attribute **depends**.
- observation - Empty *Object optional*. This means that value of the attribute can **vary** with each observation.
- primaryMeasure - *String optional*. Deprecated value having the same meaning than the relationship `observation`. For backward-compatibility only.
- measures - *Array* of *Strings optional*. One or more ID(s) of (a) local measure(s) in the data structure definition to which the values of this attribute **apply**. This is only for informational and presentational purposes. If the `measures` relationship is not present then the attribute values apply to whole observations.

Exactly one of `dataflow`, `dimensions` and (`observation` or `primaryMeasure`) is required. Note that relationships defined in data structure definitions through `attachmentGroups` or a `group` are to be resolved by the server conveniently for the client into above `dimensions`.

Example

```
{
  "dataflow": {}
}

{
  "dimensions": [
    "FREQ", "CURRENCY"
  ]
}

{
  "observation": {}
}

{
  "measures": [
    "MEAS1", "MEAS2"
  ]
}

{
  "primaryMeasure": "OBS_VALUE"  # Deprecated, for backward-compatibility only.
}
```

format

Copyright © SDMx TWG 2020

Object. The format object defines the representation for a component. It describes the possible content for component values, which could be text (including XHTML and multi-lingual values), a simple value or multiple values.

Copyright © SDMx TWG 2020

- minOccurs - *Non-negative integer optional* **Only for `measures` and `attributes`.** Indicates the minimum number of value that must be reported for the component. If missing than there is no lower limit on its occurrences. If > 1 then the component values will use the `values` property of the component value instead of `id` and `name` or `value`. The default is 1.
- maxOccurs - *Positive integer/String optional* **Only for `measures` and `attributes`.** Indicates the maximum number of values that can be reported for the component. If set to the string "unbounded" than there is no upper limit on its occurrences. If > 1 then the component values will use the `values` property of the component value instead of `id` and `name` or `value`. The default is 1.
- dataType - *String optional*. Describes the type of data format allowed for the representation of the component. Only the following dataTypes are supported: "String", "Alpha", "AlphaNumeric", "Numeric", "BigInteger", "Integer", "Long", "Short", "Decimal", "Float", "Double", "Boolean", "URI", "Count", "InclusiveValueRange", "ExclusiveValueRange", "Incremental", "ObservationalTimePeriod", "StandardTimePeriod", "BasicTimePeriod", "GregorianTimePeriod", "GregorianYear", "GregorianYearMonth", "GregorianDay", "ReportingTimePeriod", "ReportingYear", "ReportingSemester", "ReportingTrimester", "ReportingQuarter", "ReportingMonth", "ReportingWeek", "ReportingDay", "DateTime", "TimeRange", "Month", "MonthDay", "Day", "Time", "Duration", "GeospatialInformation" and "XHTML". `Dimensions` do not support the type "XHTML". Time `dimensions` (having the id and role "TIME_PERIOD") only support the types "ObservationalTimePeriod", "StandardTimePeriod", "BasicTimePeriod", "GregorianTimePeriod", "GregorianYear", "GregorianYearMonth", "GregorianDay", "ReportingTimePeriod", "ReportingYear", "ReportingSemester", "ReportingTrimester", "ReportingQuarter", "ReportingMonth", "ReportingWeek", "ReportingDay", "DateTime", "TimeRange". The default data type is "String" except for time `dimensions`, which takes "ObservationalTimePeriod" as default. If used then the component values will use instead of `id` and `name`:
  - if `minOccurs` and `maxOccurs` are equal to 1: the `value` property,
  - otherwise: the `values` property.
- isSequence - *Boolean optional*. Indicates whether the values are intended to be ordered, and it may work in combination with the interval, startValue, and endValue attributes or the timeInterval, startTime, and endTime, attributes. If this attribute holds a value of 'true', a start value or time and a numeric or time interval must supplied. If an end value is not given, then the sequence continues indefinitely.
- interval - *Integer optional*. Specifies the permitted interval (increment) in a sequence. In order for this to be used, the isSequence attribute must have a value of 'true'.
- startValue - *Number optional*. Is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates the starting point of the sequence. This value is mandatory for a numeric sequence to be expressed.
- endValue - *Number optional*. Is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates that ending point (if any) of the sequence.
- timeInterval - *String optional*. Indicates the permitted duration in a time sequence. It complies with the time duration specification of the ISO 8601 standard. In order for this to be used, the isSequence attribute must have a value of 'true'.
- startTime - *String optional*. Is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates the start time of the sequence. This value is mandatory for a time sequence to be expressed. It must be a valid standard time period (gYear, gYearMonth, date, dateTime and SDMX time periods).
- endTime - *String optional*. Is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates that ending point (if any) of the sequence. It must be a valid standard time period (gYear, gYearMonth, date, dateTime and SDMX time periods).
- minLength - *Positive integer optional*. Specifies the minimum length of the value in characters.
- maxLength - *Positive integer optional*. Specifies the maximum length of the value in characters.
- minValue - *Number optional*. Is used for inclusive and exclusive ranges, indicating what the lower bound of the range is. If this is used with an inclusive range, a valid value will be greater than or equal to the value specified here. By default, the minValue is assumed to be inclusive.
- maxValue - *Number optional*. Is used for inclusive and exclusive ranges, indicating what the upper bound of the range is. If this is used with an inclusive range, a valid value will be less than or equal to the value specified here. By default, the maxValue is assumed to be inclusive.
- decimals - *Positive integer optional*. Indicates the number of characters allowed after the decimal separator.
- pattern - *String optional*. Holds any standard regular expression.

Copyright © SDMx TWG 2020

- isMultiLingual - *Boolean optional*. **Only for `measures` and `attributes`.** This indicates for a text format of type “String” or “XHTML”, whether the uncoded component value should allow for multiple values in different languages. The default is `false`. If `true`, the component values will use instead of `id` and `name`:
  - if `minOccurs` and `maxOccurs` are equal to 1: the `value` property,
  - otherwise: the `values` property.
- sentinelValues - *Array* of *Objects optional*. When present, the sentinelValues array indicates that sentinel values are defined for the data format. Each *sentinelValue* object indicates a reserved value in an otherwise open value domain that holds a specific meaning. For example, a value of -1 can be defined to indicate a non-applicable value.

Example

```
{
  "minOccurs": 2,
  "maxOccurs": 3,
  "dataType": "String",
  "minLength": 4,
  "maxLength": 4,
  "pattern": "^[A-Za-z][A-Za-z0-9_-]*$",
  "isMultilingual": true
}

{
  "minOccurs": 2,
  "maxOccurs": 2,
  "dataType": "Double",
  "isMultilingual": false,
  "sentinelValues": [
    {
      # sentinel value object #
    }
  ]
}
```

sentinelValue

Object. It defines a reserved value (within the value domain of the data format) along with its meaning.

- value - *Number* or *String optional*. The sentinel value (within the value domain of the data format) being described.
- name - *String optional*. Human-readable (best-language-match) name (or meaning) of the sentinel value.
- names - *Object optional*. Human-readable localised *names* (or meanings) of the sentinel value.
- description - *String optional*. Human-readable (best-language-match) description for the sentinel value.
- descriptions - *Object optional*. Human-readable localised descriptions (see *names*) for the sentinel value.

Example

```
{
  "value": "-1",
  "name": "Non-response",
  "names": { "en": "Non-response",
      "fr": "Non-réponse" },
  "description": "Description for non-response.",
  "descriptions": { "en": "Description for non-response.",
        "fr": "Description de non-réponse." }
}
```

component value

Copyright © SDMx TWG 2020

Object optional. A particular value for a component in a message.

- id - *String*. Unique identifier for a coded component value.
- name - *String optional*. Human-readable (best-language-match) name for a coded component value.
- names - *Object optional*. Human-readable localised *names* for a coded component value.
- value - *String*, *Number*, *Integer*, *Boolean* or localised *Strings optional*. Only for non-coded unique (non-localised or localised) component values, e.g. for non-coded dimensions.
- values - *Array optional* of *String*, *Number*, *Integer*, *Boolean*, localised *Strings* and recursive *Array* of such arrays. Only for non-coded multivalued (non-localised or localised) component values. Only nested multi-valued metadata attribute values (according to the nesting defined in the Metadata Structure Definition) can use arrays of arrays.
- description - *String optional*. Human-readable (best-language-match) description for the component value. The description is typically longer than the text provided for the name field.
- descriptions - *Object optional*. Human-readable localised descriptions (see *names*) for the component value. A descriptions is typically longer than the text provided for the name field.
- start, end - *String optional*. Start and end are instances of time that define the actual Gregorian calendar period covered by the values for the time dimension. The algorithm for computing start and end fields for any supported reporting period is defined in the SDMX Technical Notes. These fields should be used only when the component value represents one of the values for the time dimension. Values are considered as inclusive both for the start field and the end field. Values must follow the ISO 8601 syntax for combined dates and times, including time zone. These fields are useful for visualisation tools, when selecting the appropriate point in time for the time axis. Statistical data, can be collected, for example, at the beginning, the middle or the end of the period, or can represent the average of observations through the period. Based on this information and using the start and end fields, it is easy to get or calculate the desired point in time to be used for the time axis.
- parent - *String optional*. Contains the ID for the parent of the component value (the component value of the parent might also be present in the message if according observations are also returned).
- order - *Integer optional*. Contains the original order number of the component value enabling the reconstruction of the ordered component value hierarchy. Note that, to allow for a streamed message generation, the orders of observations and of component values in the component value array are not significant.
- links - *Array optional*. *Links* field is an array of *component value link* objects. If appropriate, a collection of links to additional information regarding the component value.
- annotations - *Array optional*. *Annotations* is a collection of indices of the corresponding *annotations* for the component value. Indices refer back to the array of *annotations* in the structure field.

See the section on localised text elements on how the message deals with languages.

Copyright © SDMx TWG 2020

Example

```
{
  "id": "2010",
  "name": "2010",
  "names": { "en": "2010",
      "fr": "2010" },
  "description": "Description for 2010.",
  "descriptions": { "en": "Description for 2010.",
        "fr": "Déscription pour 2010." },
  "start": "2010-01-01T00:00Z",
  "end": "2010-12-31T23:59:59Z",
  "parent": "T",
  "order": 56,
  "links": [
    {
      # link object #
    }
  ],
  "annotations": [ 5, 49 ]
}

{
  "value": "123.4"
}

{
  "value": "AAA"
}

{
  "value": { "en": "Value", "fr": "Valeur" }
}

{
  "values": [1234, 567.8]
}

{
  "values": ["AAA", "BBB"]
}

{
  "values": [
    { "en": "Value 1", "fr": "Valeur 1" },
    { "en": "Value 2", "fr": "Valeur 2" }
  ]
}

{
  "values": [
    ["CONTACT 1 Name 1","CONTACT 1 Name 2"],
    ["CONTACT 2 Name 1","CONTACT 2 Name 2"]
  ]
}
```

component value link

See the section on linking mechanism for all information on links.

annotation

Object optional. An `annotation` object can be referenced through its `annotations` array index by `structure`, `component`, `component value`, `dataSets`, `series` and `observations`. It contains the following optional information:

- id - *String optional*. ID provides a non-standard identification of an annotation. It can be used to disambiguate annotations.
- title - *String optional*. Provides a non-localised title for the annotation.
- type - *String optional*. Type is used to distinguish between annotations designed to support various uses. The types are not enumerated, and these can be freely specified by the creator of the annotations. The definitions and use of annotation types should be documented by their creator.
- value - *String optional*. A non-localised value text of the annotation.
- text - *String optional*. A human-readable (best-language-match) text of the annotation.
- texts - *Object optional*. A list of human-readable localised texts (see *names*) of the annotation.
- links - *Array optional*. *Links* field is an array of *annotation link* objects. If appropriate, a link to an additional external resource which may contain or supplement the annotation.

See the section on localised text elements on how the message deals with languages.

Copyright © SDMx TWG 2020

Example

```
{
  "title": "Sample annotation",
  "type": "reference",
  "value": "123456",
  "text": "Sample annotation text",
  "texts": {
    "en": "Sample annotation text",
    "fr": "Exemple de texte d'annotation"
  },
  "id": "74747",
  "links": [
    {
      # link object #
    }
  ]
}
```

annotation link

See the section on linking mechanism for all information on links.

DATASET

Object. That's where the data (i.e.: the `observations`) will be.

In typical cases, the file will contain only one `dataSet`. However, in some cases, such as when retrieving, from an SDMX 2.1 web service, what has changed in the data source since in particular point in time, the web service might return more than one `dataSet`. Since SDMX 3.0.0, SDMX restful web servic requests can query data for several structures. In this case, the response includes one or more separate data sets per returned structure.

Several levels may appear in a `dataSet` object, depending on the way data is organized. A `dataSet` may contain a flat list of `observations`. In this scenario, we have 2 main levels in the data part of the message: the `dataSet` level and the `observation` level. A `dataSet` may also organize observations in logical groups called `series`. These groups can represent time series or series by other dimensions (also called cross-sections). In this scenario, we have 3 main levels in the data part of the message: the `dataSet` level, the `series` level and the `observation` level.

`Dimensions` and `attributes` may be presented at any of these main levels. However, attributes attached to only specific dimensions maybe presented also at an intermediate `dimensionGroup` level inbetween the `dataSet` level and the `series` level. `Measures` are always presented at the `observation` level. In case the `dataSet` is a flat list of `observations`, `observations` will be found directly under a `dataSet` object. In case the `dataSet` represents series of time or series of another dimension, the `observations` will be found under the `series` elements.

Copyright © SDMx TWG 2020

The `dataSet` properties are:

- structure - *Integer optional*. *Structure* contains the index of the *structure* in the *structures* array of the *data* object, which describe the data in this *dataSet*. If omitted, then the data in this *dataSet* are assumed to be described by the first *structure* (default: 0).
- action - *String optional*. Action provides a list of actions, describing the intention of the data transmission from the sender's side.
  - `Append` - this is an incremental update for an existing `dataSet` or the provision of new data or documentation (attribute values) formerly absent. If any of the supplied data or metadata is already present, it will not replace these data.
  - `Replace` - data are to be replaced, and may also include additional data to be appended.
  - `Delete` - data are to be deleted.
  - `Information` (default) - data are being exchanged for informational purposes only. When used to update a system, the `Append` action is assumed.
- reportingBegin - *String optional*. The start of the time period covered by the message.
- reportingEnd - *String optional*. The end of the time period covered by the message.
- validFrom - *String optional*. The validFrom indicates the inclusive start time indicating the validity of the information in the data.
- validTo - *String optional*. The validTo indicates the inclusive end time indicating the validity of the information in the data.
- publicationYear - *String optional*. The publicationYear holds the ISO 8601 four-digit year.
- publicationPeriod - *String optional*. The publicationPeriod specifies the period of publication of the data in terms of whatever provisioning agreements might be in force (i.e., "2005-Q1" if that is the time of publication for a `dataSet` published on a quarterly basis).
- links - *Array optional*. *Links* field is an array of *dataSet link* objects. If appropriate, a collection of links to additional information regarding the dataSet.
- annotations - *Array optional*. *Annotations* is a collection of indices of the corresponding *annotations* for the dataSet. Indices refer back to the array of *annotations* in the structure field.
- attributes - *Array optional*. *Attributes* is a collection of *attribute values or indices of the corresponding values*, depending on the presence of the `values` array in the component definition, of all attributes presented at the `dataSet` level. Indexes correspond to the indexes of the `values` array of the respective *component* object within the `structure.attributes.dataSet` array, and are always *zero* because attributes presented at the `dataSet` level can only have one value. This is typically the case for `attributes` that always have the same value for all the `observations` available in the `dataSet`. In order to avoid repetition, that value can simply be presented at the `dataSet` level. When an attribute has no value for a specific `dataSet`, then *null* is used instead of the value or its index.
- dimensionGroupAttributes - *Object optional*. *DimensionGroupAttributes* is a collection of *attribute values or indices of the corresponding values*, depending on the presence of the `values` array in the component definition, of all attributes presented at the `dimensionGroup` level.
- series - *Object optional*. A collection of *series* objects, to be used when the `observations` contained in the `dataSet` are presented in logical groups (time series or cross-sections), e.g. when using the SDMX API with the parameter "dimensionAtObservation=TIME_PERIOD" (default option) or with the "dimensionAtObservation" parameter with an ID of any other specific `dimension`. This element must **not** be used in case the `dataSet` presents a flat view of `observations`.
- observations - *Object optional*. *Observations* is a collection of *measure and/or attribute values or indices of the corresponding values*, depending on the presence of the `values` array in the component definition, of all measures and of all attributes presented at the `observation` level. It also contains the indexes of annotations applying to observations. This collection is used in case when a `dataSet` is presented as a flat view of `observations`, e.g. when using the SDMX API with the parameter "dimensionAtObservation=AllDimensions". All `dimensions`, except those with only one `value` possibly presented at the `dataSet` level, would be presented at `observation` level. Alternatively, in case the `observations` are to be presented in logical groups (time series or cross-sections), the *series* element is to be used instead.

For information on how to handle the indexes for `dimensions`, `measures`, `attributes` and `annotations` see the section dedicated to handling indexes.

Copyright © SDMx TWG 2020

Example

```
{
  "structure": 0,
  "action": "Information",
  "reportingBegin": "2012-05-04",
  "reportingEnd": "2012-06-01",
  "validFrom": "2012-01-01T10:00:00Z",
  "validTo": "2013-01-01T10:00:00Z",
  "publicationYear": "2005",
  "publicationPeriod": "2005-Q1",
  "links": [
    # links array #
  ],
  "annotations": [ 3, 42 ],
  "attributes": [
    0,
    null,
    456.7,
    "This is a string value",
    {"en": "English Text", "fr": "Texte français"},
    [123.4, 456.7], ["A", "B"],
    [{"en": "English Text 1", "fr": "Texte français 1"}, {"en": "English Text 2", "fr": "Texte français 2"}] ],
  "dimensionGroupAttributes": {
    # dimensionGroupAttributes object #
  },
  "series": {
    # series object #
  }
}

{
  "structure": 0,
  "action": "Information",
  "reportingBegin": "2012-05-04",
  "reportingEnd": "2012-06-01",
  "validFrom": "2012-01-01T10:00:00Z",
  "validTo": "2013-01-01T10:00:00Z",
  "publicationYear": "2005",
  "publicationPeriod": "2005-Q1",
  "links": [
    # links array #
  ],
  "annotations": [ 3, 42 ],
  "attributes": [
      0,
      null,
      456.7,
      "This is a string value",
      {"en": "English Text", "fr": "Texte français"},
      [123.4, 456.7],
      ["A", "B"],
      [{"en": "English Text 1", "fr": "Texte français 1"}, {"en": "English Text 2", "fr": "Texte français 2"}] ],
  "dimensionGroupAttributes": {
    # dimensionGroupAttributes object #
  },
  "observations": {
    # observations object #
  }
}
```

dataSet link

See the section on linking mechanism for all information on links.

dimensionGroupAttributes

Object optional. Collection of *values or value indexes* of all attributes presented at the `dimensionGroup` level, in form of JSON *name/value pairs*. Values are presented if the attribute definition doesn't contain the `values` array.\ Indexes of values are presented if the attribute definition does contain the `values` array.

The `dimensionGroupAttributes` object contains a JSON *name/value pair* for each *partial dimension value combination* for which a `dimensionGroup` level attribute value exists.

The *name* in the JSON *name/value pair* represents the unique (partial) dimension value combination to which the dimensionGroup attribute values are attached. "Partial" combination means that DimensionGroup attributes are not attached to all dimensions but only to a subset. This combination is the concatenation of the indexes of the corresponding `values` of the `dimensions`, separated by a colon character (":"), e.g. "::4:0:". Note that dimension values to which the attribute value is attached take the indexes from the `values` array of the respective *component* objects within the `structure.dimensions` object, whereas the other dimensions (which are not part of the partial key) have no value (are left empty). All dimensions have therefore their well-defined position in the *name* combination independently from the level at which dimensions are presentated elsewhere in the message.\ In the example "::4:0:", the data has altogether 5 dimensions (including the time dimension), and the related attribute values are

Copyright © SDMx TWG 2020

attached only to the 3rd and 4th dimensions, for which the dimension value indexes are 4 and 0. The dimension 1, 2 and 5 have no related value and their positions in the combination are left empty.

The *value* in the JSON *name/value pair* is an array containing for the given `dimensionGroup` combination:

- first: the corresponding values of **all** *attributes* presented in the `structure.attributes.dimensionGroup` array or the indexes of these values, depending on the presence of the `values` array in the component definition,
- and last: the indexes of the values of all corresponding `annotations`.

Elements for `annotations` are only included if there are corresponding `annotations`. **If `annotations` are present, then all corresponding `attributes` must be included.** Otherwise, if there are no corresponding `annotations`, then beginning from the end of the array, the corresponding `attributes` can be omitted if:

- the `attribute` has no value (e.g. for optional attributes) or
- the `attribute` value corresponds to the default value.

If an attribute that has to be listed in the array does not have (a) value(s) then *null* is used instead of a value.

The *name/value pair* is only present if there is at least one corresponding `attribute` or `annotation` value.

The data type for the array `attribute` elements must be *integer*, *number*, *string*, *object of localised strings* (see: *names*), arrays of these 4 types or *null*. Indexes for `attribute` values are of type *integer* and correspond to the indexes in the `values` array of the respective `component` object within the `structure.attributes.dimensionGroup` array.\ The data type for the array `annotation` elements is *integer*. Indexes for `annotation` values correspond to the indexes in the array of `annotations` in the `structure` element.

For information on how to handle the indexes for `dimensions`, `attributes` and `annotations` see the section dedicated to handling indexes.

Copyright © SDMx TWG 2020

Example

For this example, to ease understanding, let's consider a CSV format with horizontal time series (with header row):

```
DIM1,DIM2,DIM3,MEAS1,MEAS2,ATTR1_OBS,ATTR2_DIMGROUP,ATTR3_DIMGROUP
DIM1_VALUE_1,DIM2_VALUE_1,DIM3_VALUE_1,10,20,ATTR1_VALUE_1,ATTR2_VALUE_1,ATTR3_VALUE_1
DIM1_VALUE_1,DIM2_VALUE_1,DIM3_VALUE_2,11,21,ATTR1_VALUE_2,ATTR2_VALUE_2,ATTR3_VALUE_1
DIM1_VALUE_1,DIM2_VALUE_2,DIM3_VALUE_1,12,22,ATTR1_VALUE_3,ATTR2_VALUE_1,ATTR3_VALUE_1
DIM1_VALUE_1,DIM2_VALUE_2,DIM3_VALUE_2,13,23,ATTR1_VALUE_4,ATTR2_VALUE_2,ATTR3_VALUE_1
```

Note

Attribute `ATTR1_OBS` is attached at observation level and thus depends on all dimensions. Attribute `ATTR2_DIMGROUP` is attached to the dimensions `DIM1` and `DIM3` only. Its values only vary with the values of those dimensions. Attribute `ATTR3_DIMGROUP` is attached to the dimension `DIM1` only. Its values only vary with the values of this dimension.

In SDMX-JSON, dimension values are replaced by their indices. Attribute values may be replaced by their indices, which is **not** done in this example:

```
"dimensionGroupAttributes": {
"0::": { null, 0 },
"0::0": { "ATTR2_VALUE_1" },
"0::1": { "ATTR2_VALUE_2" }
},
"observations": {
"0:0": [10, 20, "ATTR1_VALUE_1"],
"0:1": [11, 21, "ATTR1_VALUE_2"],
"1:0": [12, 22, "ATTR1_VALUE_3"],
"1:1": [13, 23, "ATTR1_VALUE_4"]
}

/*
dimensionGroup 1: "0::" corresponds to the 1 index for "DIM1":"DIM1_VALUE_1". All other dimensions are discarded.
        The attributes for this dimension group are: "ATTR2_DIMGROUP": null (no correspondance), "ATTR3_DIMGROUP": "ATTR3_VALUE_1"

dimensionGroup 2: "0::0" corresponds to the 2 indexes for "DIM1":"DIM1_VALUE_1" and "DIM3":"DIM3_VALUE_1".
The "DIM2" dimension is discarded.
        The attributes for this dimension group are: "ATTR2_DIMGROUP": "ATTR2_VALUE_1", "ATTR3_DIMGROUP":
        no correspondance and omitted since last array element

dimensionGroup 2: "0::1" corresponds to the 2 indexes for "DIM1":"DIM1_VALUE_1" and "DIM3":"DIM3_VALUE_2".
The "DIM2" dimension is discarded.
        The attributes for this dimension group are: "ATTR2_DIMGROUP": "ATTR2_VALUE_2", "ATTR3_DIMGROUP":
        no correspondance and omitted since last array element

Observation 1: "0:0" corresponds to the 2 indices for "DIM2":"DIM2_VALUE_1", "DIM3":"DIM3_VALUE_1"
(DIM1 is omitted since presented at dataSet level to avoid its repetition for each observation)
                The measures for this observation are:
            "MEAS1": 10
            "MEAS2": 20
                The attributes for this observation are:
                "ATTR1_OBS": "ATTR1_VALUE_1"
Observation 2: "0:0" corresponds to the 2 indices for "DIM2":"DIM2_VALUE_1", "DIM3":"DIM3_VALUE_2"
(DIM1 is omitted since presented at dataSet level to avoid its repetition for each observation)
                The measures for this observation are:
            "MEAS1": 11
            "MEAS2": 21
                The attributes for this observation are:
                "ATTR1_OBS": "ATTR1_VALUE_2"
Observation 3: "0:0" corresponds to the 2 indices for "DIM2":"DIM2_VALUE_2", "DIM3":"DIM3_VALUE_1"
(DIM1 is omitted since presented at dataSet level to avoid its repetition for each observation)
                The measures for this observation are:
            "MEAS1": 12
            "MEAS2": 22
                The attributes for this observation are:
                "ATTR1_OBS": "ATTR1_VALUE_3"
Observation 4: "0:0" corresponds to the 2 indices for "DIM2":"DIM2_VALUE_2", "DIM3":"DIM3_VALUE_2"
(DIM1 is omitted since presented at dataSet level to avoid its repetition for each observation)
                The measures for this observation are:
            "MEAS1": 13
            "MEAS2": 23
                The attributes for this observation are:
                "ATTR1_OBS": "ATTR1_VALUE_4"
*/

"dimensions": {
"dataSet": [
    {
    "id": "DIM1",
    "name": "Dimension 1",
    "names": { "en": "Dimension 1" },
    "values": [
        {
        "id": "DIM1_VALUE_1",
        "name": "Dimension 1 - Value 1 with index 0",
        "names": { "en": "Dimension 1 - Value 1 with index 0" }
        }
    ]
```


Copyright © SDMx TWG 2020

```
    }
],
"series": [],
"observation": [
    {
    "id": "DIM2",
    "name": "Dimension 2",
    "names": { "en": "Dimension 2" },
    "values": [
        {
        "id": "DIM2_VALUE_1",
        "name": "Dimension 2 - Value 1 with index 0",
        "names": { "en": "Dimension 2 - Value 1 with index 0" }
        },
        {
        "id": "DIM2_VALUE_2",
        "name": "Dimension 2 - Value 2 with index 1",
        "names": { "en": "Dimension 2 - Value 2 with index 1" }
        }
    ]
    },
    {
    "id": "DIM3",
    "name": "Dimension 3",
    "names": { "en": "Dimension 3" },
    "values": [
        {
        "id": "DIM3_VALUE_1",
        "name": "Dimension 3 - Value 1 with index 0",
        "names": { "en": "Dimension 3 - Value 1 with index 0" }
        },
        {
        "id": "DIM3_VALUE_2",
        "name": "Dimension 3 - Value 2 with index 1",
        "names": { "en": "Dimension 3 - Value 2 with index 1" }
        }
    ]
    }
]
},
"attributes": {
"dataSet": [],
"dimensionGroup": [
    {
    "id": "ATTR2_DIMGROUP",
    "name": "Attribute 2",
    "names": { "en": "Attribute 2" },
    "format": {
        "dataType": "String"
    }
    },
    {
    "id": "ATTR3_DIMGROUP",
    "name": "Attribute 3",
    "names": { "en": "Attribute 3" },
    "values": [
        {
        "id": "ATTR3_VALUE_1",
        "name": "Attribute 3 - Value 1 with index 0",
        "names": { "en": "Attribute 3 - Value 1 with index 0" }
        }
    ]
    }
],
"series": [],
"observation": [
    {
    "id": "ATTR1_OBS",
    "name": "Attribute 1",
    "names": { "en": "Attribute 1" },
    "format": {
        "dataType": "String"
    }
    }
]
}
```


!!! note For attributes “ATTR2_DIMGROUP” and “ATTR1_OBS”, the values are omitted in the attribute definitions and thus presented directly in the dataSets’ `dimensionGroupAttributes` and `observations` (instead of indexes).

series

Object optional. Collection of series in form of JSON *name/value* pairs, when the `observations` contained in the `dataSet` are used into logical groups (time series or cross-sections). Each underlying series is represented as a JSON *name/value pair* (“name”: “value”) in the `series` object.

The *name* in the JSON *name/value pair* represents the unique series identifier, which is the concatenation of the indexes of the `values` of all `dimensions` presented at `series` level (i.e. indexes of the fields in the `values` array of the respective *component* object within the `structure.dimensions.series` array) separated by a colon character (“:”), e.g. “0:0:4:10:2”.

Copyright © SDMx TWG 2020

The *value* in the JSON *name/value pair* is an object containing:

- annotations - *Array optional*. *Annotations* is a collection of indices of the corresponding *annotations* for the series. Indices refer back to the array of `annotations` in the structure field.
- attributes - *Array optional*. *Attributes* is a collection of *attribute values or indexes of the corresponding values*, depending on the presence of the `values` array in the component definition, of all `attributes` presented at the `series` level. Indexes correspond to the indexes in the `values` array of the respective `component` object within the `structure.attributes.series` array. This is typically the case for `attributes` that always have the same value for all the `observations` available in the series. In order to avoid repetition, that value can simply be presented at the `series` level. When an attribute has no value for a specific series, then *null* is used instead of the index.
- observations - *Object optional*. *Observations* is a collection of *measure and/or attribute values or indices of the corresponding values*, depending on the presence of the `values` array in the component definition, of all measures and of all attributes presented at the `observation` level. It also contains the indexes of annotations applying to observations. The `dimension` specified in the “dimensionAtObservation” parameter of the SDMX API is used to identify the observations of a series. By default that dimension is the time dimension (with ID “TIME_PERIOD”).

For information on how to handle the indexes for `dimensions`, `measures`, `attributes` and `annotations` see the section dedicated to handling indexes.

Copyright © SDMx TWG 2020

Example

For this example, to ease understanding, let's consider a CSV format with horizontal time series (with header row):

```
DIM1,DIM2,Value for 2016,Value for 2017,ATTR1,ATTR2,ANNOT,ATTR3 for 2016,ATTR3 for 2017
DIM1_VALUE_1,DIM2_VALUE_1,1.5931,1.5925,ATTR1_VALUE_1,""en:English Text 1;fr:Texte français 1"";""en:English Text 2;fr:Texte français 2""",,ATTR3_VALUE_1,
DIM1_VALUE_1,DIM2_VALUE_2,40.3426,40.3000,ATTR1_VALUE_2,,ANNOT_VALUE1,ATTR3_VALUE_1,ATTR3_VALUE_1
```

In SDMX-JSON, using "dimensionAtObservation=TIME_PERIOD" (default) the observations are presented in a similar way, grouped by time series (with the TIME_PERIOD dimension at observation level), but dimension values are replaced by their indices. Attribute values may also be replaced by their indexes, which - in this example - is done for ATTR1 and ATTR3 but **not** for ATTR2:

```
"series": {
  "0:0": {
    "attributes": [
      0,
      [{"en": "English Text 1", "fr": "Texte français 1"}, {"en": "English Text 2", "fr": "Texte français 2"}]
      ],
    "observations": {
      "0": [1.5931, 0],
      "1": [1.5925]
    }
  },
  "0:1": {
    "annotations": [0],
    "observations": {
      "0": [40.3426, 0],
      "1": [40.3000, 0]
    }
  }
}

/*
Series 1: "0:0" corresponds to the 2 indices for "DIM1":"DIM1_VALUE_1", "DIM2":"DIM2_VALUE_1"
          The attributes for this series are: "ATTR1":"ATTR1_VALUE_1",
          "ATTR2":""en:English Text 1;fr:Texte français 1"";""en:English Text 2;fr:Texte français 2"""
          This series has 2 observations:
          Observation 1: "0" corresponds to the index for "TIME_PERIOD":"2016"
                         The value for this observation is: 1.5931
                         The attribute for this observation is: "ATTR3":"ATTR3_VALUE_1"
          Observation 2: "1" corresponds to the index for "TIME_PERIOD":"2017"
                         The value for this observation is: 1.5925
                         The attribute for this observation is: "ATTR3":"ATTR3_VALUE_2"
                         (because this is the default value)
Series 2: "0:1" corresponds to the 2 indices for "DIM1":"DIM1_VALUE_1", "DIM2":"DIM2_VALUE_2"
          The annotation for this series is: "ANNOT":"ANNOT_VALUE1"
          The attributes for this series are: "ATTR1":"ATTR1_VALUE_2"
          (omitted because this is the default value and since the only and last attribute)
          This series has 2 observations:
          Observation 1: "0" corresponds to the index for "TIME_PERIOD":"2016"
                         The value for this observation is: 40.3426
                         The attribute for this observation is: "ATTR3":"ATTR3_VALUE_1"
          Observation 2: "1" corresponds to the index for "TIME_PERIOD":"2017"
                         The value for this observation is: 40.3000
                         The attribute for this observation is: "ATTR3":"ATTR3_VALUE_1"
*/

"dimensions": {
  "dataSet": [],
  "series": [
    {
      "id": "DIM1",
      "name": "Dimension 1",
      "names": { "en": "Dimension 1" },
      "values": [
        {
          "id": "DIM1_VALUE_1",
          "name": "Dimension 1 - Value 1 with index 0",
          "names": { "en": "Dimension 1 - Value 1 with index 0" }
        }
      ]
    },
    {
      "id": "DIM2",
      "name": "Dimension 2",
      "names": { "en": "Dimension 2" },
      "values": [
        {
          "id": "DIM2_VALUE_1",
          "name": "Dimension 2 - Value 1 with index 0",
          "names": { "en": "Dimension 2 - Value 1 with index 0" }
        },
        {
          "id": "DIM2_VALUE_2",
          "name": "Dimension 2 - Value 2 with index 1",
          "names": { "en": "Dimension 2 - Value 2 with index 1" }
        }
      ]
    }
  ],
  "observation": [
    {
      "id": "TIME_PERIOD",
```


Copyright © SDMx TWG 2020

```
      "name": "Time Period",
      "names": { "en": "Time Period" },
      "values": [
        {
          "id": "2016",
          "name": "2016",
          "names": { "en": "2016" }
        },
        {
          "id": "2017",
          "name": "2017",
          "names": { "en": "2017" }
        }
      ]
    }
  ]
},
"attributes": {
  "dataSet": [],
  "series": [
    {
      "id": "ATTR1",
      "name": "Attribute 1",
      "names": { "en": "Attribute 1" },
      "default": "ATTR1_VALUE_2",
      "values": [
        {
          "id": "ATTR1_VALUE_1",
          "name": "Attribute 1 - Value 1 with index 0",
          "names": { "en": "Attribute 1 - Value 1 with index 0" }
        },
        {
          "id": "ATTR1_VALUE_2",
          "name": "Attribute 1 - Value 2 with index 1",
          "names": { "Attribute 1 - Value 2 with index 1" }
        }
      ]
    },
    {
      "id": "ATTR2",
      "name": "en": "Attribute 2",
      "names": { "en": "Attribute 2" },
      "format": {
        "maxOccurs": 2,
        "dataType": "String",
        "isMultiLingual": true
      }
    }
  ],
  "observation": [
    {
      "id": "ATTR3",
      "name": "Attribute 3",
      "names": { "en": "Attribute 3" },
      "default": "ATTR3_VALUE_2",
      "values": [
        {
          "id": "ATTR3_VALUE_1",
          "name": "Attribute 3 - Value 1 with index 0",
          "names": { "en": "Attribute 3 - Value 1 with index 0" }
        },
        {
          "id": "ATTR3_VALUE_2",
          "name": "Attribute 3 - Value 2 with index 1",
          "names": { "en": "Attribute 3 - Value 2 with index 1" }
        }
      ]
    }
  ]
},
"annotations": [
  {
    "title": "Annotation 1 - with index 0",
    "type": "example",
    "text": "Sample annotation text",
    "texts": { "en": "Sample annotation text" },
    "id": "ANNOT_VALUE1"
  }
]
```


Note

For attribute “ATTR2”, the values are omitted in the attribute definition and thus presented directly in the dataSets’ `observations` (instead of indexes).

observations

Object optional. Collection of observations in form of JSON *name/value pairs*. Each underlying observation is represented as a JSON *name/value pair* in the `observations` object.

Copyright © SDMx TWG 2020

The *name* in the JSON *name/value pair* represents the unique observation identifier, which is the concatenation of the indexes of the `values` of all `dimensions` presented at `observation` level (i.e. indexes of the fields in the `values` array of the respective *component* object within the `structure.dimensions.observation` array) separated by a colon character (":"). The `dimension` specified in the "dimensionAtObservation" parameter of the SDMX API is used to identify the observations. By default that dimension is the time dimension (with ID "TIME_PERIOD"). Thus, there is only one single index in the observation identifier, e.g. "2". With "dimensionAtObservation=allDimensions", when the data are represented as a flat view of observations, all dimensions with more than 1 value will be presented at observation level. Here, there can be several concatenated indexes in the observation identifier, e.g. "0:0:4:10:2".

The *value* in the JSON *name/value pair* is an array containing:

- first: the corresponding values of **all** *measures* (as presented in the `structure.measures.observation` array) or the indexes of these values, depending on the presence of the `values` array in the component definition,
- followed by: the corresponding values of **all** *attributes* presented in the `structure.attributes.observation` array or the indexes of these values, depending on the presence of the `values` array in the component definition,
- and last: the indexes of the values of all `annotations` of that observation.

Therefore, the array elements after the `measures` are for the `observation` level `attributes` and for `annotations` of that observation. Elements for `annotations` are only included if there are `annotations` for that observation. **If `annotations` are present for an observation, then all `attributes` defined at `observation` level must be included.** Otherwise, if the observation has no `annotations`, then beginning from the end of the array, `observation` level `attributes` can be omitted if:

- the `attribute` is not set for this observation (possible for optional attributes) or
- the `attribute` value for this observation corresponds to the default value.

The data type for the array `measure` and `attribute` elements is *integer*, *number*, *string*, *object of localised strings* (see: *names*), arrays of these 4 types or *null*. The last is for a reported `measure` measure value or for unused optional `attributes` when the attribute position in the array needs to be filled. Indexes for `measure` values are of type *integer* and correspond to the indexes in the `values` array of the respective `component` object within the `structure.measures.observation` array. Indexes for `attribute` values are of type *integer* and correspond to the indexes in the `values` array of the respective `component` object within the `structure.attributes.observation` array. The data type for the array `annotation` elements is *integer*. Indexes for `annotation` values correspond to the indexes in the array of `annotations` in the `structure` element.

For information on how to handle the indexes for `dimensions`, `measures`, `attributes` and `annotations` see the section dedicated to handling indexes.

Copyright © SDMx TWG 2020

Example

For this example, to ease understanding, let’s consider data in a flat CSV format (with header row):

```
DIM1,DIM2,MEAS1,MEAS2,ATTR1,ATTR2,ATTR3,ANNOT
DIM1_VALUE_1,DIM2_VALUE_1,105.6,120.8,ATTR1_VALUE_1;ATTR1_VALUE_2,ATTR2_VALUE_1,,ANNOT_VALUE1
DIM1_VALUE_1,DIM2_VALUE_2,105.9,120.2,ATTR1_VALUE_1,ATTR2_VALUE_2,,
```

In SDMX-JSON, the observations are presented in a similar flattened way, but dimension and attribute values are replaced by their indices:

```
"observations": {
  "0:0": [105.6, 120.8, ["ATTR1_VALUE_1","ATTR1_VALUE_2"], 0, null, 0],
  "0:1": [105.9, 120.2, ["ATTR1_VALUE_1"], 1]
}

/*
Observation 1: "0:0" corresponds to the 2 indices for "DIM1":"DIM1_VALUE_1", "DIM2":"DIM2_VALUE_1"
              The measures for this observation are:
          105.6
          120.8
              The attributes for this observation are:
                "ATTR1": "ATTR1_VALUE_1", "ATTR1_VALUE_2"
    "ATTR2": "ATTR2_VALUE_1" (0 is the index of the corresponding element in the attribute's "values" array)
                "ATTR2": null (null is used as placeholder before the next array position for the following annotation)
              The annotation for this observation is:
          "ANNOT": "ANNOT_VALUE1" (0 is the index of the corresponding element in the annotation array)
Observation 2: "0:1" corresponds to the 2 indices for "DIM1":"DIM1_VALUE_1", "DIM2":"DIM2_VALUE_2"
              The measures for this observation are:
          105.9
          120.2
              The attributes for this observation are:
                "ATTR1":"ATTR1_VALUE_1"
    "ATTR2":"ATTR2_VALUE_2" (1 is the index of the corresponding element in the attribute's "values" array)
*/

"dimensions": {
  "dataSet": [],
  "series": [],
  "observation": [
    {
      "id": "DIM1",
      "name": "Dimension 1",
      "names": { "en": "Dimension 1" },
      "values": [
        {
          "id": "DIM1_VALUE_1",
          "name": "Dimension 1 - Value 1 with index 0",
          "names": { "en": "Dimension 1 - Value 1 with index 0" }
        }
      ]
    },
    {
      "id": "DIM2",
      "name": "Dimension 2",
      "names": { "en": "Dimension 2" },
      "values": [
        {
          "id": "DIM2_VALUE_1",
          "name": "Dimension 2 - Value 1 with index 0",
          "names": { "en": "Dimension 2 - Value 1 with index 0" }
        },
        {
          "id": "DIM2_VALUE_2",
          "name": "Dimension 2 - Value 2 with index 1",
          "names": { "en": "Dimension 2 - Value 2 with index 1" }
        }
      ]
    }
  ]
},
"attributes": {
  "dataSet": [],
  "series": [],
  "observation": [
    {
      "id": "ATTR1",
      "name": "Attribute 1",
      "names": { "en": "Attribute 1" },
      "format": {
        "dataType": "String",
        "maxValue": 2
      }
    },
    {
      "id": "ATTR2",
      "name": "Attribute 2",
      "names": { "en": "Attribute 2" },
      "values": [
        {
          "id": "ATTR2_VALUE_1",
          "name": "Attribute 2 - Value 1 with index 0",
          "names": { "en": "Attribute 2 - Value 1 with index 0" }
        },
```


Copyright © SDMx TWG 2020

```
        {
          "id": "ATTR2_VALUE_2",
          "name": "Attribute 2 - Value 2 with index 1",
          "names": { "en": "Attribute 2 - Value 2 with index 1" }
        }
      ]
    },
    {
      "id": "ATTR3",
      "name": "Attribute 3",
      "names": { "en": "Attribute 3" },
      "default": "ATTR3_VALUE_1",
      "values": [
        {
          "id": "ATTR3_VALUE_1",
          "name": "Attribute 3 - Value 1 with index 0",
          "names": { "en": "Attribute 3 - Value 1 with index 0" }
        }
      ]
    }
  ]
},
"annotations": [
  {
    "title": "Annotation 1 - with index 0",
    "type": "example",
    "text": "Sample annotation text",
    "texts": { "en": "Sample annotation text" },
    "id": "ANNOT_VALUE1"
  }
]
```


Note

For attribute “ATTR1”, the values are omitted in the attribute definition and thus presented directly in the dataSets’ `observations` (instead of indexes).

ERROR

Object optional. Used to provide status messages in addition to RESTful web services HTTP error status codes. The following pieces of information should be provided:

- code - *Number*. Provides a code number for the status message if appropriate. Standard code numbers are defined in the SDMX 2.1 Web Services Guidelines.
- title - *String optional*. A short, human-readable (best-language-match) summary of the situation that SHOULD NOT change from occurrence to occurrence of the status, except for purposes of localization.
- titles - *Object optional*. A list of short, human-readable localised summaries (see *names*) of the status that SHOULD NOT change from occurrence to occurrence of the status, except for purposes of localization.
- detail - *String optional*. A human-readable (best-language-match) explanation specific to this occurrence of the status. Like title, this field’s value can be localized. It is fully customizable by the service providers and should provide enough detail to ease understanding the reasons of the status.
- details - *Object optional*. A list of human-readable localised explanations (see *names*) specific to this occurrence of the status. Like titles, this field’s value can be localized. It is fully customizable by the service providers and should provide enough detail to ease understanding the reasons of the status.
- links - *Array optional*. *Links* field is an array of *meta link* objects. If appropriate, a collection of links to additional external resources for the status message.

See the section on localised text elements on how the message deals with languages.

Example


```
{
  "code": 150,
  "title": "Invalid number of dimensions in the key parameter",
  "titles": { "en": "Invalid number of dimensions in the key parameter"
        "fr": "Nombre invalide de dimensions dans le paramètre 'key'" }
}
```


Copyright © SDMx TWG 2020

Linking mechanism

LINK

Object optional. A link to an external resource.

- href - *String* or . Absolute or relative URL of the external resource.
- rel - *String*. Relationship of the object to the external resource. See semantics below.
- urn - *String optional*. The urn holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
- uri - *String optional*. The uri attribute holds a URI that contains a link to additional information about the resource, such as a web page. This uri is not an SDMX resource.
- title - *String optional*. A human-readable (best-language-match) description of the target link.
- titles - *Object optional*. A list of human-readable localised descriptions (see *names*) of the target link.
- type - *String optional*. A hint about the type of representation returned by the link.
- hreflang - *String optional*. The natural language of the external link, the same as used in the HTTP Accept-Language request header.

See the section on localised text elements on how the message deals with languages.

Example

```
{
    "href": "https://registry.sdmx.org/ws/rest/datastructure/ECB/ECB_EXR1",
    "rel": "self",
    "uri": "http://www.xyz.org/pdf/0123456789"
}

{
    "href": "https://registry.sdmx.org/ws/rest/dataflow/ECB.DISS/BSI_PUB/1.0",
    "rel": "dataflow",
    "urn": "urn:sdmx:org.sdmx.infomodel.datastructure.dataflow=ECB.DISS:BSI_PUB(1.0)"
}

{
    "href": "https://registry.sdmx.org/FusionRegistry/ws/rest/provisionagreement/ESTAT/PA_NAMAIN_IDC_N",
    "rel": "provisionagreement"
}

{
    "href": "https://registry.sdmx.org/help.html",
    "rel": "help",
    "title": "Documentation about the SDMX Global Registry",
    "titles": { "en": "Documentation about the SDMX Global Registry" },
    "type": "text/html",
    "hreflang": "en"
}
```

Collections of links can be attached to various elements in SDMX-JSON.

Similarly with standards such as HTML5 and Atom, link elements in SDMX-JSON *must* define a *URL* (the `href` attribute) and a *semantic* (the `rel` attribute). This allows clients to understand the semantics of links, in order to follow those that are of interest to them. In addition, links in SDMX-JSON *may* define a `title` (a human-friendly description of the target link) and a `type` (a hint about the type of representation returned by the link). Please refer to the list of Media Types and Subtypes assigned and listed by the IANA for additional information about expected values for the `type` attribute.

Copyright © SDMx TWG 2020

SDMX-JSON offers a list of predefined semantics, but implementers are free to extend it. The list of predefined semantics comes from the list of SDMX artefacts that can be returned by SDMX RESTful web services, semantics defined in RFC5988 and additional items deemed to be useful in the context of statistical data dissemination. These semantics are:

- SDMX artefacts: dataStructure, metadataStructure, categoryScheme, conceptScheme, codelist, hierarchicalCodelist, organisationScheme, agencyScheme, dataProviderScheme, dataConsumerScheme, organisationUnitScheme, dataflow, metadataflow, reportingTaxonomy, provisionAgreement, structureSet, process, categorisation, contentconstraint, attachmentconstraint, category, concept, code, organisation, agency, dataProvider, dataConsumer, organisationUnit, reportingCategory, Data
- RFC5988: alternate, copyright, glossary, help, index.
- Miscellaneous: calendar (link to a release calendar), source (information about the source of data), request (the SDMX RESTful query that triggered the SDMX-JSON response).

The *URL* captured in the `href` attribute can be *absolute* or *relative*. **It is recommended to use absolute URLs in case the SDMX-JSON message is archived.**

Copyright © SDMx TWG 2020

Handling indexes

The purpose of using indexes is to avoid the repetition of space-consuming values of measures, attributes, dimensions and annotations. For the first 2 types, whenever their values are provided directly in the `values` array of the component definition itself, then the datasets must only use the corresponding element indexes in those arrays instead of the real values. Dimensions and annotations will always use the indexes.

Let's say that the following data content of a message needs to be processed:


```
{
  "structures": [
    {
      "id": "ECB_EXR_WEB",
      "links": [
        {
          "href": "https://sdw-wsrest.ecb.europa.eu/service/dataflow/ECB/EXR/1.0",
          "rel": "dataflow"
        }
      ],
      "dimensions": {
        "dataSet": [
          {
            "id": "FREQ",
            "name": "Frequency",
            "names": { "en": "Frequency" },
            "keyPosition": 0,
            "values": [
              {
                "id": "D",
                "name": "Daily",
                "names": { "en": "Daily" }
              }
            ]
          },
          {
            "id": "CURRENCY_DENOM",
            "name": "Currency denominator",
            "names": { "en": "Currency denominator" },
            "keyPosition": 2,
            "values": [
              {
                "id": "EUR",
                "name": "Euro",
                "names": { "en": "Euro" }
              }
            ]
          },
          {
            "id": "EXR_TYPE",
            "name": "Exchange rate type",
            "names": { "en": "Exchange rate type" },
            "keyPosition": 3,
            "values": [
              {
                "id": "SP00",
                "name": "Spot rate",
                "names": { "en": "Spot rate" }
              }
            ]
          },
          {
            "id": "EXR_SUFFIX",
            "name": "Series variation - EXR context",
            "names": { "en": "Series variation - EXR context" },
            "keyPosition": 4,
            "values": [
              {
                "id": "A",
                "name": "Average or standardised measure",
                "names": { "en": "Average or standardised measure" }
              }
            ]
          }
        ],
        "series": [
          {
            "id": "CURRENCY",
            "name": "Currency",
            "names": { "en": "Currency" },
            "keyPosition": 1,
            "values": [
              {
                "id": "NZD",
                "name": "New Zealand dollar",
                "names": { "en": "New Zealand dollar" }
              },
              {
                "id": "RUB",
                "name": "Russian rouble",
```


Copyright © SDMx TWG 2020

```
            "names": { "en": "Russian rouble" }
          }
        ]
      }
    ],
    "observation": [
      {
        "id": "TIME_PERIOD",
        "name": "Time period or range",
        "names": { "en": "Time period or range" },
        "values": [
          {
            "id": "2013-01-18",
            "name": "2013-01-18",
            "names": { "en": "2013-01-18" }
          },
          {
            "id": "2013-01-21",
            "name": "2013-01-21",
            "names": { "en": "2013-01-21" }
          }
        ]
      }
    ]
  },
  "attributes": {
    "dataSet": [],
    "series": [
      {
        "id": "TITLE",
        "name": "Series title",
        "names": { "en": "Series title" },
        "values": [
          {
            "name": "New Zealand dollar (NZD)",
            "names": { "en": "New Zealand dollar (NZD)" }
          },
          {
            "name": "Russian rouble (RUB)",
            "name": { "en": "Russian rouble (RUB)" }
          }
        ]
      }
    ],
    "observation": [
      {
        "id": "OBS_STATUS",
        "name": "Observation status",
        "names": { "en": "Observation status" },
        "values": [
          {
            "id": "A",
            "name": "Normal value",
            "names": { "en": "Normal value" }
          }
        ]
      }
    ]
  },
  "annotations": [
    {
      "title": "Sample series annotation title",
      "type": "example",
      "text": "Sample series annotation text",
      "texts": { "en": "Sample series annotation text" },
      "id": "ABC123456"
    },
    {
      "title": "Sample observation annotation title",
      "type": "example",
      "text": "Sample observation annotation text",
      "texts": { "en": "Sample observation annotation text" },
      "id": "XYZ98765"
    }
  ],
  "dataSets": [0]
 }
],
"dataSets": [
 {
  "structure": 0,
  "action": "Information",
  "series": {
   "0": {
    // 0 is the index of the first value of (series-level) CURRENCY dimension: "NZD"
    "annotations": [0], // 0 is the index of the first value of annotations: "ABC123456"
    "attributes": [0], // 0 is the index of the first value of the (first) (series-level) TITLE attribute: "New Zealand dollar (NZD)"
    "observations": {
     "0": [1.5931, 0], // "0" corresponds to the first value of (obs-level) TIME_PERIOD dimension: "2013-01-18"
     // 1.5931 is the corresponding observation value
     // 0 is the index of the first value of (obs-level) OBS_STATUS attribute: "A"
     "1": [1.5925, 0] // "1" corresponds to the second value of (obs-level) TIME_PERIOD dimension: "2013-01-21"
     // 1.5925 is the corresponding observation value
     // 0 is the index of the first value of (obs-level) OBS_STATUS attribute: "A"
```


Copyright © SDMx TWG 2020

```
          }
        },
        "1": {
          // 1 is the index of the second value of (series-level) CURRENCY dimension: "RUB"
          "attributes": [1], // 1 is the index of the second value of the (first) (series-level) TITLE attribute: "Russian rouble (RUB)"
          "observations": {
            "0": [40.3426, 0], // "0" corresponds to the first value of (obs-level) TIME_PERIOD dimension: "2013-01-18"
            // 40.3426 is the corresponding observation value
            // 0 is the index of the first value of (obs-level) OBS_STATUS attribute: "A"
            "1": [40.3, 0, 1] // "1" corresponds to the second value of (obs-level) TIME_PERIOD dimension: "2013-01-21"
            // 40.3000 is the corresponding observation value
            // 0 is the index of the first value of (obs-level) OBS_STATUS attribute: "A"
            // 1 is the index of the second value of annotations: "XYZ98765" (because there is no other obs-level attribute defined)
          }
        }
      }
    }
  ]
}
```

There is one `dataSet` in the message, and it contains two `series`.

```
"0": {
 "annotations": [0],
 "attributes": [0],
 "observations": {
  "0": [1.5931, 0],
  "1": [1.5925, 0]
 }
},
"1": {
 "attributes": [1],
 "observations": {
  "0": [40.3426, 0],
  "1": [40.3000, 0, 1]
 }
}
```

The `structure.dimensions` field tells us that, out of the 6 dimensions, 4 have the same value for the 2 series and are therefore attached to the `dataSet` level.

We see that, for the first series, we get the value 0:

```
"0": { ... }
```

From the structure.dimensions.series information, we know that CURRENCY is the (only) series dimension.

```
"series": [
 {
  "id": "CURRENCY",
  "name": "Currency",
  "names": { "en": "Currency" },
  "keyPosition": 1,
  "values": [
   {
    "id": "NZD",
    "name": "New Zealand dollar",
    "names": { "en": "New Zealand dollar" }
   }, {
    "id": "RUB",
    "name": "Russian rouble",
    "names": { "en": "Russian rouble" }
   }
  ]
 }
]
```

The value “0” identified previously is the index of the item in the collection of values for this component. In this case, the dimension value is therefore “New Zealand dollar”.

Now, for the first observation of the first series, we get the value 0:

```
 "0": [...],
```

From the `structure.dimensions.observation` information, we know that TIME_PERIOD is the (only) dimension at `observation` level.

```
"observation": [
 {
  "id": "TIME_PERIOD",
  "name": "Time period or range",
  "names": { "en": "Time period or range" },
  "values": [
```


Copyright © SDMx TWG 2020

```
    {
     "id": "2013-01-18",
     "name": "2013-01-18",
     "names": { "en": "2013-01-18" }
    }, {
     "id": "2013-01-21",
     "name": "2013-01-21",
     "names": { "en": "2013-01-21" }
    }
   ]
  }
 ]
```

The value “0” identified previously is the index of the item in the collection of values for this component. In this case, the dimension value is therefore “2013-01-18”.

Now, for the first (and only) attribute of the first observation of the first series, we get the value 0 (here the last value in array):

“0”: [1.5931, 0],

From the `structure.attributes.observation` information, we know that OBS_STATUS is the (only) attribute at `observation` level.

```
"observation": [
 {
  "id": "OBS_STATUS",
  "name": "Observation status",
  "names": { "en": "Observation status" },
  "values": [
   {
    "id": "A",
    "name": "Normal value",
    "names": { "en": "Normal value" }
   }
  ]
 }
]
```

The value 0 identified previously is the index of the item in the collection of values for this component. In this case, the attribute value is therefore “Normal value”.

The same logic applies for mapping the other observations, its attributes and annotations.

Copyright © SDMx TWG 2020

Localised text elements

Localised best-language-match text strings (static properties matched through “Lookup”):

The first best language match according to the user’s preferred language choices expressed through the HTTP content negotiation (Accept-Language header parameter) is to be provided for each localised text element. The message does however not indicate the returned language per localised text element.

This language matching type is called “Lookup”, see https://tools.ietf.org/html/rfc4647#section-3.4.

Example

```
"name": "Frequency",
```

Localised text objects (variable properties matched through “Filtering”):

All available language matches according to the user’s preferred language choices expressed through the HTTP content negotiation (Accept-Language header parameter) is to be provided for each localised text element.

This language matching type is called “Filtering”, see https://tools.ietf.org/html/rfc4647#section-3.3.

Example

```
"names": { "en": "Frequency",
    "fr": "Fréquence" },
```

The localised text object needs to be present whenever the related localised best-language-match text strings is present, and especially whenever a localised text is mandatory in the SDMX Information model. Note that localised text (and the knowledge about the locale) is mandatory in structure messages when artefacts are being submitted for storage to a registry or to other databases. The localised text object is important for use cases where multiple languages are required or where the information on the language used is required.

In case that there is no language match for a particular localisable element, it is optional to:

- return the element in a system-default language or alternatively to not return the element
- indicate available alternative languages for the element’s maintainable artefact through links to underlying localised resources

It is recommended to indicate all languages used anywhere in the message for localised elements through http Content-Language response header (languages of the intended audience) and/or through a “contentLanguages” property in the meta tag. The main language used can be indicated through the “lang” property in the meta tag.

Copyright © SDMx TWG 2020

Security Considerations

This document defines a response format for SDMX RESTful Web Services in JSON and it raises no new security considerations. SDMX Web Services Guidelines includes the security considerations associated with its usage.

Copyright © SDMx TWG 2020

Extending SDMX-JSON

The objects defined in SDMX-JSON are “open”, i.e. they can be extended by implementers with properties not defined in this specification. Providers of SDMX-JSON messages are therefore welcome to add support for features not covered in this specification. Whenever appropriate, providers who opt to do so are invited to inform us, so that future versions of SDMX-JSON may integrate these extensions, thereby improving interoperability.

The snippet below shows an example of an `error` object, extended with a `wsCustomErrorCode`:

```
    "errors": [
        {
            "code": 150,
            "title": "Invalid number of dimensions in the key parameter",
            "titles": { "en": "Invalid number of dimensions in the key parameter" },
            "wsCustomErrorCode": 39272
        }
    ]
```


Copyright © SDMx TWG 2020

References

NORMATIVE REFERENCES

[iso8601] *ISO 8601:2004 Data elements and interchange formats — Information interchange — Representation of dates and times*, ISO, 2004

[json] *The JSON Data Interchange Format*, Standard ECMA-404, ECMA International, October 2013

[sdmx-im] *SDMX Information Model v3.0.0*, https://github.com/sdmx-twg/sdmx-im, SDMX, September 2021

[sdmx-rest] *SDMX RESTful Web Services Specification v2.0.0*, https://github.com/sdmx-twg/sdmx-rest, SDMX, April 2011

[json api 1.0] *A specification for building APIs in Json*, http://jsonapi.org/, 2015

Copyright © SDMx TWG 2020

7.2.4 Metadata Message

SDMX-JSON Metadata Message

As one of the standard SDMX metadata exchange formats, SDMX-JSON Metadata Message is a JSON (JavaScript Object Notation) based referential metadata exchange message format designed for and therefore responding to the main use case of data discovery and visualisation on the web.

JSON is a widely used standardised lightweight data-interchange format. SDMX-JSON with a simple and dense but generic message format that supports different types of data structures will ease writing software for the web that easily consumes SDMX resources. The format may be used in a wide variety of programming languages and application programs. Testing and development of the format has focused on JavaScript applications running in web browsers.

The current SDMX-JSON Metadata Message format 2.0.0 supports the SDMX 3.0 Information Model and works together with the SDMX 3.0 RESTful Web Services API for referential metadata queries.

See: - docs: for the descriptive documentation - samples: for message samples - tools: for the normative JSON schema usable for validation purposes

Copyright © SDMx TWG 2020

Introduction to SDMX-JSON Metadata Message

See the SDMX-JSON Data Message docs for a brief introduction of the SDMX information model. For additional information on the SDMX information model, please refer to the SDMX documentation.

Samples, tools and other SDMX-JSON resources are available in the public Github repository.

Before we start, let's clarify a few more things about this guide:

- New fields may be introduced in later versions. Therefore consuming applications should tolerate the addition of new fields with ease.
- The ordering of properties in objects is undefined. The properties may appear in any order and consuming applications should not rely on any specific ordering. It is safe to consider a nulled property and the absence of a property as the same thing.
- Not all properties appear in all contexts. For example response with error status messages may not contain a data property.

Copyright © SDMx TWG 2020

Field Guide to SDMX-JSON 2.0 Metadata Message Objects (aligned with SDMX 3.0.0)

MESSAGE

Message is the top level object and it contains the requested information (referential metadata) as well as the meta-information describing the (technical) context of the message and, possibly, status information.

- meta - *Object optional*. A *meta* object that contains non-standard meta-information and basic technical information about the message, such as when it was prepared and who has sent it.
- data - *Object optional*. *Data* contains the message's "primary data".
- errors - *Array optional*. *Errors* field is an array of *statusMessage* objects. When appropriate provides a list of status messages in addition to RESTful web services HTTP error status codes.

The members data and status CAN coexist in the same message.

Example

```
{
  "meta": {
    # meta object #
  },
  "data": {
    # data object #
  },
  "errors": [
    {
      # statusMessage object #
    }
  ]
}
```

META

Object optional. Used to include non-standard meta-information and basic technical information about the message, such as when it was prepared and who has sent it. Any members MAY be specified within `meta` objects.

- schema - *String optional*. Contains the URL to the schema allowing to validate the message. This also allows identifying the version of SDMX-JSON format used in this message. **Providing the link to the SDMX-JSON schema is recommended.**
- id - *String*. Unique string that identifies the message for further references.
- test - *Boolean optional*. Indicates whether the message is for test purposes or not. False for normal messages.
- prepared - *String*. A timestamp indicating when the message was prepared. Values must follow the ISO 8601 syntax for combined dates and times, including time zone.
- contentLanguages - *Array optional*. Array of strings containing the identifier of all languages used anywhere in the message for localized elements, and thus the languages of the intended audience, representing in an array format the same information than the http Content-Language response header, e.g. "en, fr-fr". See IETF Language Tags. The array's first element indicates the main language used in the message for localized elements. **The usage of this property is recommended.**
- name - *String optional*. Human-readable (best-language-match) name for the transmission.
- names - *Object optional*. Human-readable localised *names* for the transmission.
- sender - *Object*. *Sender* contains information about the party that is transmitting the message.
- receivers - *Array optional* of *Receiver* objects thats contain information about the party that is receiving the message. This can be useful if the WS requires authentication.
- links - *Array optional*. *Links* field is an array of *link* objects. If appropriate, a collection of links to additional external resources for the header.

See the section on localised text elements on how the message deals with languages.

Copyright © SDMx TWG 2020

Example


```
"meta": {
  "schema": "https://raw.githubusercontent.com/sdmx-twg/sdmx-json/master/metadata-message/tools/schemas/sdmx-json-metadata-schema.json",
  "copyright": "Copyright 2017 Statistics hotline",
  "id": "b1804c51-1ee3-45a9-bb75-795cd4e06489",
  "prepared": "2018-01-03T12:54:12",
  "test": false,
  "contentLanguages": [ "en", "fr-fr" ],
  "name": "Transmission name",
  "names": {
    # name object #
  },
  "sender": {
    # sender object #
  },
  "receivers": [
    {
      # receiver object #
    }
  ],
  "links": [
    {
      # link object #
    }
  ]
}
```


sender

Object. Information about the party that is transmitting the message. Sender contains the following fields:

- id - *String*. A unique identifier of the party.
- name - *String nullable*. A human-readable (best-language-match) name of the sender.
- names - *Object optional*. A list of human-readable localised *names* of the sender.
- contacts - *Array optional*. A collection of *contacts*. Provides contact information for the party in regard to the transmission of the message.

See the section on localised text elements on how the message deals with languages.

Example


```
"sender": {
  "id": "ECB",
  "name": "European Central Bank",
  "names": {
    "en": "European Central Bank",
    "fr": "Banque Centrale Européenne"
  },
  "contacts": [
    {
      # contact objects #
    }
  ]
}
```


names

Object containing all appropriate localised names, one per object property:

- One or more of: IETF Language Tag according to RFC 5646 documentation for specifying locals in HTTP - *String*. The localised name.

See the section on localised text elements on how the message deals with languages.

Example


```
{
  "en": "This is an English name",
  "en-GB": "This is a British name",
  "fr": "C'est un nom français"
}
```


Copyright © SDMx TWG 2020

contact

Object. A collection of contact details. Each object in the collection may contain the following field:

- id - *String*. Identifier for the resource.
- name - *String optional*. Human-readable (best-language-match) name of the contact.
- names - *Object optional*. Human-readable localised *names* of the contact.
- department - *String optional*. Human-readable (best-language-match) name of the organisational structure for the contact.
- departments - *Object optional*. Human-readable localised *names* of the organisational structure for the contact.
- role - *String optional*. Human-readable (best-language-match) name of the responsibility of the contact.
- roles - *Object optional*. Human-readable localised *names* of the responsibility of the contact.
- telephones - *Array optional*. An array of telephone numbers for the contact.
- faxes - *Array optional*. An array of fax numbers for the contact person.
- uris - *Array optional*. An array of uris. Each uri holds an information URL for the contact.
- emails - *Array optional*. An array of email addresses for the contact person.
- x400s - *Array optional*. An array of X.400 addresses for the contact person.

See the section on localised strings on how the message deals with languages.

Example

```
{
  "id": "HOTLINE",
  "name": "Statistics hotline",
  "names": {
    "en": "Statistics hotline",
    "fr": "Service d'assistance téléphonique des Statistiques"
  },
  "department": "Statistics hotline",
  "departments": {
    "en": "Statistics hotline",
    "fr": "Service d'assistance téléphonique des Statistiques"
  },
  "role": "Statistics hotline",
  "roles": {
    "en": "Statistics hotline",
    "fr": "Service d'assistance téléphonique des Statistiques"
  },
  "telephones": [ "+00 0 00 00 00 00" ],
  "faxes": [ "+00 0 00 00 00 01" ],
  "uris": [ "www.xyz.org" ],
  "emails": [ "statistics@xyz.org" ]
}
```

receiver

Object optional. Information about the party that is receiving the message. This can be useful if the WS requires authentication. Receiver contains the same fields as *sender*.

meta link

See the section on *linking mechanism* for all information on links.

DATA

Object optional. Header contains the message's "primary data".

- metadataSets - *Array optional*. This field is an array of *metadataSet* objects. A metadata set contains a collection of reported metadata against a set of values for a given full or partial target identifier, as described in a metadata structure definition. The metadata set may contain reported metadata for multiple report structures defined in a metadata structure definition.

Copyright © SDMx TWG 2020

Example

```
"data": {
  "metadataSets": [
    {
      # metadataSet object #
    }
  ]
}
```

metadataSet

Object. Contains a collection of reported metadata against a set of values for a given full or partial target identifier, as described in a metadata structure definition. The metadata set may contain reported metadata for multiple report structures defined in a metadata structure definition.

- action - *String optional*. Action provides a list of actions, describing the intention of the data transmission from the sender's side.
  - `Append` - this is an incremental update for an existing `dataSet` or the provision of new data or documentation (attribute values) formerly absent. If any of the supplied data or metadata is already present, it will not replace these data.
  - `Replace` - data are to be replaced, and may also include additional data to be appended.
  - `Delete` - data are to be deleted.
  - `Information` (default) - data are being exchanged for informational purposes only, and not meant to update a system.
- publicationPeriod - *String optional*. The publicationPeriod specifies the period of publication of the data in terms of whatever provisioning agreements might be in force (i.e., "2005-Q1" if that is the time of publication for a `metadataSet` published on a quarterly basis).
- publicationYear - *String optional*. The publicationYear holds the ISO 8601 four-digit year.
- reportingBegin - *String optional*. The start of the time period covered by the message.
- reportingEnd - *String optional*. The end of the time period covered by the message.
- id - *String*. Identifier for the metadata set.
- agencyID - *String*. ID of the agency maintaining this metadata set.
- version - *String optional*. Version of this metadata set according to semantic versioning. If not specified then the metadata set is non-versioned.
- isExternalReference - *Boolean optional*. If set to "true" it indicates that the content of the metadata set is held externally.
- metadataflow - *String*. URN reference to the metadataflow definition (either *metadataflow* or *metadataProvisionAgreement* is required).
- metadataProvisionAgreement - *String*. URN reference to the metadata provision definition (either *metadataflow* or *metadataProvisionAgreement* is required).
- validFrom - *String optional*. The validFrom indicates the inclusive start time indicating the validity of the information in the data.
- validTo - *String optional*. The validTo indicates the inclusive end time indicating the validity of the information in the data.
- annotations - *Array optional*. *Annotations* is a collection of indices of the corresponding *annotations* for the metadata set.
- links - *Array optional*. *Links* field is an array of *link* objects. If appropriate, a collection of links to additional information regarding the metadata set.
- name - *String*. Human-readable (best-language-match) name of the metadata set.
- names - *Object optional*. Human-readable localised *names* of the metadata set.
- description - *String optional*. Human-readable (best-language-match) description of the metadata set.
- descriptions - *Object optional*. Human-readable localised descriptions (see *names*) of the metadata set.
- targets - Non-empty *array* of *String*. Each *target* holds a valid SDMX Registry URN (see SDMX Registry Specification for details) of the object to which the reported metadata apply. The same metadata set can be linked to multiple targets.
- attributes - Non-empty *array* of recursive *attribute* objects. Contains the reported metadata attribute values for the reported metadata and recursively their child metadata attributes.

Copyright © SDMx TWG 2020

See the section on localised text elements on how the message deals with languages.

Example

```
{
  "action": "Information",
  "publicationPeriod": "2018-Q1",
  "publicationYear": "2018",
  "reportingBegin": "1960",
  "reportingEnd": "2020",
  "id": "METADATASET",
  "agencyID": "ECB.DISS",
  "version": "1.0",
  "isExternalReference": false,
  "metadataflow": "urn:sdmx:org.sdmx.infomodel.metadatastructure.Metadataflow=ECB.DISS:MDF(1.0)",
  # "metadataProvisionAgreement": "urn:sdmx:org.sdmx.infomodel.registry.MetadataProvisionAgreement=ECB.DISS:MDPA(1.0.0)",
  "validFrom": "2018-04-01",
  "validTo": "2018-07-01",
  "annotations": [
    {
      # annotation object #
    }
  ],
  "links": [
    {
      # link object #
    }
  ],
  "name": "Metadata set",
  "names": {
    "en": "Metadata set",
    "fr": "Set de métadonnées"
  },
  "description": "This is the description of the metadata set",
  "descriptions": {
    "en": "This is the description of the metadata set",
    "fr": "Ceci est la description de l'ensemble des métadonnées"
  },
  "targets": ["urn:sdmx:org.sdmx.infomodel.datastructure.DataStructure=ECB:ECB_EXR1(1.0)"],
  "attributes": [
    {
      # attribute object #
    }
  ]
}
```

attribute

Object. Defines the structure for a reported metadata attribute. A value for the attribute can be supplied as either a single value, or multi-lingual text values (either structured or unstructured). An optional set of child metadata attributes is also available if the metadata attribute definition defines nested metadata attributes.

- id - *String*. ID for the reported metadata attribute.
- annotations - *Array optional*. *Annotations* is a collection of indices of the corresponding *annotations* for the reported metadata attribute.
- format - *Object optional*. *Format* describes the allowed metadata attribute representation. It is only used when the metadata attributes are not defined by an enumerated list of identifiable items (codelist).
- value - *String*, *Number*, *Integer*, *Boolean* or localised *String* (see *names*) *object*, *optional*. Value for the reported metadata attribute. Also HTML strings are supported.
- attributes - Non-empty *array* of recursive *attribute* objects. Contains the reported metadata attribute values for the reported metadata and recursively their child metadata attributes.

One of the properties is required: value, values, text or structuredText.

See the section on localised text elements on how the message deals with languages.

Copyright © SDMx TWG 2020

Example

```
{
  "id": "REPORT_ATTRIBUTE1",
  "format": {
    # format object #
  },
  "value": "CODED_TEXT",
  "annotations": [
    {
      # annotation object #
    }
  ],
  "attributes": [
    {
      # attribute object #
    }
  ]
}
```

```
  "value": "<p>An XHTML text</p>"
```

```
  "value": {
    "en": "<p>An XHTML text</p>"
  }
```

format

Copyright © SDMx TWG 2020

Object. The format object defines the representation for a component. It describes the possible content for component values, which could be text (including XHTML and multi-lingual values).

- dataType - *String optional*. Describes the type of data format allowed for the representation of the component. Only the following dataTypes are supported: “String”, “Alpha”, “AlphaNumeric”, “Numeric”, “BigInteger”, “Integer”, “Long”, “Short”, “Decimal”, “Float”, “Double”, “Boolean”, “URI”, “Count”, “InclusiveValueRange”, “ExclusiveValueRange”, “Incremental”, “ObservationalTimePeriod”, “StandardTimePeriod”, “BasicTimePeriod”, “GregorianTimePeriod”, “GregorianYear”, “GregorianYearMonth”, “GregorianDay”, “ReportingTimePeriod”, “ReportingYear”, “ReportingSemester”, “ReportingTrimester”, “ReportingQuarter”, “ReportingMonth”, “ReportingWeek”, “ReportingDay”, “DateTime”, “TimeRange”, “Month”, “MonthDay”, “Day”, “Time”, “Duration”, “GeospatialInformation” and “XHTML”. The default data type is “String”.
- isSequence - *Boolean optional*. Indicates whether the values are intended to be ordered, and it may work in combination with the interval, startValue, and endValue attributes or the timeInterval, startTime, and endTime, attributes. If this attribute holds a value of ‘true’, a start value or time and a numeric or time interval must supplied. If an end value is not given, then the sequence continues indefinitely.
- interval - *Integer optional*. Specifies the permitted interval (increment) in a sequence. In order for this to be used, the isSequence attribute must have a value of ‘true’.
- startValue - *Number optional*. Is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates the starting point of the sequence. This value is mandatory for a numeric sequence to be expressed.
- endValue - *Number optional*. Is used in conjunction with the isSequence and interval attributes (which must be set in order to use this attribute). This attribute is used for a numeric sequence, and indicates that ending point (if any) of the sequence.
- timeInterval - *String optional*. Indicates the permitted duration in a time sequence. It complies with the time duration specification of the ISO 8601 standard. In order for this to be used, the isSequence attribute must have a value of ‘true’.
- startTime - *String optional*. Is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates the start time of the sequence. This value is mandatory for a time sequence to be expressed. It must be a valid standard time period (gYear, gYearMonth, date, dateTime and SDMX time periods).
- endTime - *String optional*. Is used in conjunction with the isSequence and timeInterval attributes (which must be set in order to use this attribute). This attribute is used for a time sequence, and indicates that ending point (if any) of the sequence. It must be a valid standard time period (gYear, gYearMonth, date, dateTime and SDMX time periods).
- minLength - *Positive integer optional*. Specifies the minimum length of the value in characters.
- maxLength - *Positive integer optional*. Specifies the maximum length of the value in characters.
- minValue - *Number optional*. Is used for inclusive and exclusive ranges, indicating what the lower bound of the range is. If this is used with an inclusive range, a valid value will be greater than or equal to the value specified here. By default, the minValue is assumed to be inclusive.
- maxValue - *Number optional*. Is used for inclusive and exclusive ranges, indicating what the upper bound of the range is. If this is used with an inclusive range, a valid value will be less than or equal to the value specified here. By default, the maxValue is assumed to be inclusive.
- decimals - *Positive integer optional*. Indicates the number of characters allowed after the decimal separator.
- pattern - *String optional*. Holds any standard regular expression.
- isMultiLingual - *Boolean optional*. This indicates for a text format of type “String” or “XHTML”, whether the uncoded component value should allow for multiple values in different languages. The default is `false`.
- sentinelValues - *Array* of *Object*s *optional*. When present, the sentinelValues array indicates that sentinel values are defined for the data format. Each *sentinelValue* object indicates a reserved value in an otherwise open value domain that holds a specific meaning. For example, a value of -1 can be defined to indicate a non-applicable value.

Copyright © SDMx TWG 2020

Example

```
{
  "dataType": "String",
  "minLength": 4,
  "maxLength": 4,
  "pattern": "^[A-Za-z][A-Za-z0-9_-]*$",
  "isMultilingual": true
}

{
  "dataType": "Double",
  "isMultilingual": false,
  "sentinelValues": [
    {
      # sentinel value object #
    }
  ]
}
```

sentinelValue

Object. It defines a reserved value (within the value domain of the data format) along with its meaning.

- value - *Number* or *String optional*. The sentinel value (within the value domain of the data format) being described.
- name - *String optional*. Human-readable (best-language-match) name (or meaning) of the sentinel value.
- names - *Object optional*. Human-readable localised *names* (or meanings) of the sentinel value.
- description - *String optional*. Human-readable (best-language-match) description for the sentinel value.
- descriptions - *Object optional*. Human-readable localised descriptions (see *names*) for the sentinel value.

Example

```
{
  "value": "-1",
  "name": "Non-response",
  "names": { "en": "Non-response",
       "fr": "Non-réponse" },
  "description": "Description for non-response.",
  "descriptions": { "en": "Description for non-response.",
        "fr": "Description de non-réponse." }
}
```

annotation

Object optional. Provides all information about an annotation.

- id - *String optional*. ID provides a non-standard identification of an annotation. It can be used to disambiguate annotations.
- title - *String optional*. Provides a non-localised title for the annotation.
- type - *String optional*. Type is used to distinguish between annotations designed to support various uses. The types are not enumerated, and these can be freely specified by the creator of the annotations. The definitions and use of annotation types should be documented by their creator.
- value - *String optional*. Provides a non-localised value text for the annotation.
- text - *String optional*. A human-readable (best-language-match) text of the annotation.
- texts - *Object optional*. A list of human-readable localised texts (see *names*) of the annotation.
- links - *Array optional*. *Links* field is an array of *link* objects. If appropriate, a link to an additional external resource which may contain or supplement the annotation.

See the section on localised text elements on how the message deals with languages.

Copyright © SDMx TWG 2020

Example


```
{
  "id": "74747",
  "title": "Sample annotation",
  "type": "reference",
  "value": "123456",
  "text": "Sample annotation text",
  "texts": {
    "en": "Sample annotation text",
    "fr": "Exemple de texte d'annotation"
  },
  "links": [
    {
      # link object #
    }
  ]
}
```


metadataSet link

See the section on linking mechanism for all information on links. Providing links allowing accessing the underlying SDMX Data Structure Definition, Dataflow and/or Provision Agreements is recommended.

STATUSMESSAGE

Object optional. Used to provide status messages in addition to RESTful web services HTTP error status codes. The following pieces of information should be provided:

- code - *Number*. Provides a code number for the status message if appropriate. Standard code numbers are defined in the SDMX 2.1 Web Services Guidelines.
- title - *String optional*. A short, human-readable (best-language-match) summary of the situation that SHOULD NOT change from occurrence to occurrence of the status, except for purposes of localization.
- titles - *Object optional*. A list of short, human-readable localised summaries (see *names*) of the status that SHOULD NOT change from occurrence to occurrence of the status, except for purposes of localization.
- detail - *String optional*. A human-readable (best-language-match) explanation specific to this occurrence of the status. Like title, this field's value can be localized. It is fully customizable by the service providers and should provide enough detail to ease understanding the reasons of the status.
- details - *Object optional*. A list of human-readable localised explanations (see *names*) specific to this occurrence of the status. Like titles, this field's value can be localized. It is fully customizable by the service providers and should provide enough detail to ease understanding the reasons of the status.
- links - *Array optional*. *Links* field is an array of *link* objects. If appropriate, a collection of links to additional external resources for the status message.

See the section on localised strings on how the message deals with languages.

Example


```
{
  "code": 150,
  "title": "Invalid parameter",
  "titles": {
    "en": "Invalid parameter",
    "fr": "Paramètre invalide"
  }
}
```


Copyright © SDMx TWG 2020

Linking mechanism

LINK

Object optional. A link to an external resource.

- href - *String*. Absolute or relative URL of the external resource.
- rel - *String*. Relationship of the object to the external resource. See semantics below.
- urn - *String optional*. The urn holds a valid SDMX Registry URN (see SDMX Registry Specification for details).
- uri - *String optional*. The uri attribute holds a URI that contains a link to additional information about the resource, such as a web page. This uri is not an SDMX resource.
- title - *String optional*. A human-readable (best-language-match) description of the target link.
- titles - *Object optional*. A list of human-readable localised descriptions (see *names*) of the target link.
- type - *String optional*. A hint about the type of representation returned by the link.
- hreflang - *String optional*. The natural language of the external link, the same as used in the HTTP Accept-Language request header.

See the section on localised text elements on how the message deals with languages.

Example

```
{
  "href": "https://registry.sdmx.org/ws/rest/datastructure/ECB/ECB_EXR1",
  "rel": "self",
  "uri": "http://www.xyz.org/pdf/0123456789"
}

{
  "href": "https://registry.sdmx.org/ws/rest/dataflow/ECB.DISS/BSI_PUB/1.0",
  "rel": "dataflow",
  "urn": "urn:sdmx:org.sdmx.infomodel.datastructure.dataflow=ECB.DISS:BSI_PUB(1.0)"
}

{
  "href": "https://registry.sdmx.org/FusionRegistry/ws/rest/provisionagreement/ESTAT/PA_NAMAIN_IDC_N",
  "rel": "provisionagreement"
}

{
  "href": "https://registry.sdmx.org/help.html",
  "rel": "help",
  "title": "Documentation about the SDMX Global Registry",
  "titles": { "en": "Documentation about the SDMX Global Registry" },
  "type": "text/html",
  "hreflang": "en"
}
```

Collections of links can be attached to various elements in SDMX-JSON.

Similarly with standards such as HTML5 and Atom, link elements in SDMX-JSON *must* define a *URL* (the `href` attribute) and a *semantic* (the `rel` attribute). This allows clients to follow the links they care about and ignore the ones whose semantic they are not interested in. In addition, links in SDMX-JSON *may* define a `title` (a human-friendly description of the target link) and a `type` (a hint about the type of representation returned by the link). Please refer to the list of Media Types and Subtypes assigned and listed by the IANA for additional information about expected values for the `type` attribute.

Copyright © SDMx TWG 2020

SDMX-JSON offers a list of predefined semantics, but implementers are free to extend it. The list of predefined semantics comes from the list of SDMX artefacts that can be returned by SDMX RESTful web services, semantics defined in RFC5988 and additional items deemed to be useful in the context of statistical data dissemination. These semantics are:

- SDMX artefacts: dataStructure, metadataStructure, categoryScheme, conceptScheme, codelist, hierarchicalCodelist, organisationScheme, agencyScheme, dataProviderScheme, dataConsumerScheme, organisationUnitScheme, dataflow, metadataflow, reportingTaxonomy, provisionAgreement, structureSet, process, categorisation, contentconstraint, attachmentconstraint, category, concept, code, organisation, agency, dataProvider, dataConsumer, organisationUnit, reportingCategory, Data
- RFC5988: alternate, copyright, glossary, help, index.
- Miscellaneous: calendar (link to a release calendar), source (information about the source of data), request (the SDMX RESTful query that triggered the SDMX-JSON response).

The *URL* captured in the `href` attribute can be *absolute* or *relative*. **It is recommended to use absolute URLs in case the SDMX-JSON message is archived.**

Copyright © SDMx TWG 2020

Localised text elements

Localised best-language-match text strings (static properties matched through “Lookup”):

The first best language match according to the user’s preferred language choices expressed through the HTTP content negotiation (Accept-Language header parameter) is to be provided for each localised text element. The message does however not indicate the returned language per localised text element.

This language matching type is called “Lookup”, see IETF tools.

Example

```
"name": "Frequency",
```

Localised text objects (variable properties matched through “Filtering”):

All available language matches according to the user’s preferred language choices expressed through the HTTP content negotiation (Accept-Language header parameter) is to be provided for each localised text element.

This language matching type is called “Filtering”, see https://tools.ietf.org/html/rfc4647#section-3.3.

Example

```
"names": {
  "en": "Frequency",
  "fr": "Fréquence"
},
```

The localised text object needs to be present whenever the related localised best-language-match text strings is present, and especially whenever a localised text is mandatory in the SDMX Information model. Note that localised text (and the knowledge about the locale) is mandatory in structure messages when artefacts are being submitted for storage to a registry or to other databases. The localised text object is important for use cases where multiple languages are required or where the information on the language used is required.

In case that there is no language match for a particular localisable element, it is optional to:

- return the element in a system-default language or alternatively to not return the element
- indicate available alternative languages for the element’s maintainable artefact through links to underlying localised resources

It is recommended to indicate all languages used anywhere in the message for localised elements through http Content-Language response header (languages of the intended audience) and/or through a “contentLanguages” property in the meta tag. The main language used can be indicated through the “lang” property in the meta tag.

Copyright © SDMx TWG 2020

Security Considerations

This document defines a response format for SDMX RESTful Web Services in JSON and it raises no new security considerations. SDMX Web Services Guidelines includes the security considerations associated with its usage.

Copyright © SDMx TWG 2020

Extending SDMX-JSON

The objects defined in SDMX-JSON are “open”, i.e. they can be extended with properties not defined in this specification. Providers of SDMX-JSON messages are therefore welcome to add support for features not covered in this specification. Whenever appropriate, providers who opt to do so are invited to inform us, so that future versions of SDMX-JSON may integrate these extensions, thereby improving interoperability.

The snippet below shows an example of an `error` object, extended with a `wsCustomErrorCode`:

```
"errors": [
    {
        "code": 150,
        "title": "Invalid parameter",
        "titles": {
            "en": "Invalid parameter",
            "fr": "Paramètre invalide"
        },
        "wsCustomErrorCode": 39272
    }
]
```


Copyright © SDMx TWG 2020

7.3 SDMx-CSV (v2.1)

7.3.1 SDMX-CSV

SDMX-CSV is a new data exchange format for public data dissemination. SDMX-CSV format conforms to the RFC 4180 standard specification, and it supports the SDMX 2.1 Information Model. SDMX-CSV is streamable from a supporting web service in order to limit the amount of server resources. In addition, the format aims to ease understanding and use of the data messages by non-experts with a simple but generic message structure that supports different types of data structures. CSV is a widely used standardised simple format used to exchange data and is supported by many tools. Testing and development of the format has focused on statistical and analytical applications.

The SDMX-CSV Data Message works together with the SDMX RESTful Web Services API. It will assist developers to write software that requests data responses in CSV from SDMX RESTful API. The current version of SDMX-CSV Data Message focuses on easy data ingestion by industry-standard analytical or statistical software. SDMX RESTful API also supports reference metadata queries and structural metadata queries. Today, these are not supported by the SDMX-CSV format.

Copyright © SDMx TWG 2020

7.3.2 Data Message Field Guide

Introduction

SDMX-CSV Data Message is an SDMX data exchange format based on the RFC 4180. CSV is a widely used standardised and simple format to exchange data supported by many tools.

SDMX-CSV integrates with other specifications, i.e.:

- The SDMX API RESTful specification (e.g. content negotiation with mime-type to get SDMX-CSV representations, specific formats for responses, language selection through HTTP content negotiation)
- The RFC 4180 specification

RFC 4180: A COMMON FORMAT FOR CSV FILES

In order to benefit from best practices, SDMX-CSV is based on the rules defined in the RFC 4180, which defines a common format and MIME Type for CSV files. It is advised to read the (very short) RFC for a full list of requirements but, in a nutshell, the RFC defines rules such as:

- How the CSV file should be structured (the RFC specifies that all records must have an identical structure (determined column number), like when using an SDMX "flat" representation for data);
- When double-quotes should be used and how to escape them when needed;
- How spaces should be handled: Spaces are considered part of a field and should not be ignored;
- Which mime type should be used;
- What is the default character set, etc.

The SDMX-CSV format is flexible enough in its representation to support the needs of different target audiences:

- It is designed and optimised for the purpose of general public data dissemination of statistical data, and for usage in common statistical software.
- It allows using the messages to create pivot tables in spreadsheets applications.

Design principles for SDMX-CSV 2.0 Data Messages (aligned with SDMX 3.0.0)

- In order to ensure the identifiability of the data contained in the message, the header row containing the column headers is mandatory and its content is well-defined.
- After the mandatory header row, each row contains the information related to one specific observation or to one or more attributes attached to partial keys. For `Delete` actions a row can also concern several observations if dimensions are wildcarded.
- In RFC 4180, csv stands for "comma-separated values". However, while SDMX-CSV uses indeed the "comma" (%x2C) as the default field separator, it adopts the wider interpretation of csv as "character-separated values". It is recommended for implementers to provide SDMX-CSV messages according to the locale of the user (e.g. as indicated in the http Accept-Language header). It means that e.g. the semi-colon ';' (as used typically in specific regions or countries) is acceptable as separator. See also the related example below. Note that the separator used in a message can be determined by retrieving the character that follows the fixed first column header term *STRUCTURE* (which may be extended by a squared bracket term).

COLUMNS

- The first column is always used for the structure type: dataflow, data structure definition or data provision agreement.
- The next one or two columns are always used for the structure's identification.
- The next column is always used for the action to be performed.
- The next up to two columns are used for the series and/or observation key.
- Each Data Structure Definition (DSD) component (dimensions, measures, attributes including those defined through a referenced Metadata Structure Definition) included in the message is represented in one or two columns. SDMX web services should return the columns in the order of components as defined in (each of) the underlying Data Structure Definition(s), grouped by type of component, thus in case of data defined by different data structures: first the dimensions of the first data structure, then the

Copyright © SDMx TWG 2020

remaining dimensions of the second data structure and so forth, then the measures of the first data structure, then the remaining measures of the second data structure and so forth, then the attributes of the first data structure, then the remaining attributes of the second data structure and so forth. However, any order of these columns is valid for data uploads to SDMX-consuming systems.

- Only all those dimension columns have to be present, that are required to uniquely identify the concerned attributes and/or measures.
- Attributes can but do not need to be included even if they have a mandatory status.
- Measures can but do not have to be included.
- When an SDMX RESTful web service implements streaming, then it might not know, while generating the csv header row, which measures and attributes actually have values. Therefore, it can happen that all values presented in an attribute or measure column are left empty.
- Implementers have the possibility to add any other custom columns as required, e.g. updated, prepared, etc.

COLUMN HEADERS (FIRST ROW)

- The header field of the first column always contains the term `STRUCTURE`.
  - This field must be extended with a sub-field delimiter encapsulated in squared brackets "[]", e.g. `STRUCTURE[;]`, in case the message contains multi-valued or multi-language measure or attribute values.
- The header field of the second column always contains the term `STRUCTURE_ID`.
- If option `labels=name` (see *optional parameters*): An additional column is added right after the artefact identification column containing the term `STRUCTURE_NAME`.
- The header field of the next column should contain the term `ACTION`. For convenience, if this column is not present, a default action ("Information") is assumed for the whole message.
- The next up to two columns contain, if option `key=series|obs|both` (see *optional parameters*), in this order the terms `SERIES_KEY` and/or `OBS_KEY`.
- The other columns for components contain:
  - Default: The ID of the component reported in that column, e.g. `DIM1`.
  - If option `labels=both` (see *optional parameters*): The ID and the localised name of the component reported in that column separated by the term ": ", e.g. `DIM1: Dimension 1`.
  - If option `labels=name` (see *optional parameters*): An additional column is added right after the component identification column containing the localised name of the component reported in the previous column.
- Any other custom column contains a custom but unique term, e.g. `UPDATED`.

COLUMN CONTENT (ALL ROWS AFTER HEADER)

- The first column contains: `dataflow`, `datastructure` or `dataprovision`, depending on type of artefact for which the data contained in the row are defined: dataflow, data structure definition or data provision agreement.
- The second column contains:
  - Default: The artefact identification information for the data in the row in the form *AGENCY:ARTEFACT_ID(VERSION)*[1], e.g. `ESTAT:NA_MAIN(1.6.0)`.
  - If option `labels=both` (see *optional parameters*): The artefact identification information and its localised name separated by the term ": ", e.g. `ESTAT:NA_MAIN(1.6.0): National Accounts Main Aggregates`.
- If option `labels=name` (see *optional parameters*): An additional column is added right after the artefact identification column with the artefact's localised name, e.g. `National Accounts Main Aggregates`.

Copyright © SDMx TWG 2020

- The next column contains one character representing one of the current 4 action types:
  - “I”: Information - Data is for information purposes. If such data messages are loaded into an SDMX database, the action “A” (Append) is assumed.
  - “A”: Append - Data is for an incremental update of existing observations or partial-key attributes or for the provision of new data formerly absent. This means that only the information provided explicitly in the message should be altered. Any measure or attribute value that is to be added or changed must be provided. However, the absence of an observation value or a data attribute at any level does not imply deletion; instead it is simply implied that the value is to remain unchanged. Therefore, it is valid and acceptable to send a data message with an action of Append which (in addition to identifying structure columns) contains only identifying dimensions with some attribute values. In this case, whatever the attachment level of the attributes is, the values for the attributes will be updated. Note that it is not permissible to update measure or attribute values using incomplete identification information, e.g. without the structure ID or without the necessary dimensions (full key for measures, full key/partial key/none for attributes).
  - “R”: Replace - Data is for replacement. Existing observations are to be fully replaced. Existing attribute values are to be replaced. Observations or attribute values formally absent will be appended.
  - “D”: Delete - Data is to be deleted. ‘Delete’ is assumed to be an incremental deletion. The deletion is to take place at the lowest level of detail provided in the row. Concretely, if a ‘Delete’ row only contains the identification information of the structural artefact (dataflow, data structure definition or metadata provision agreement) without any dimension, measure and attribute values then all data for the given artefact will be deleted. If the row contains only the structure identification and partial dimension values then all observations and all attribute values relating to those dimension values will be deleted. If the row contains only the structure identification, partial dimension values as well as values for some of the related attributes then only these attribute values will be deleted. If the row contains only the structure identification and full dimension values then the related observation and all its observation-level attribute values will be deleted. Finally, if the row contains only the structure identification, full dimension values as well as values for some of the related measures and attributes then only these measure and observation-level attribute values will be deleted. To be deleted measure and attribute values must be non-empty, e.g. marked with the dash character “-“.
  - For convenience, if this column is absent then the action “Information” is assumed.
- The next up to two columns contain, if option `key=series|obs|both`, in this order the series keys and/or the observation keys (see *optional parameters*).
- The other columns for components contain:
  - Default: The ID(s) (if coded) or value(s) (if non-coded) for the component values reported in that column for the corresponding observation, e.g. `A`.
  - If option `labels=both` (see *optional parameters*): The ID(s) and the localised name separated by the term “: ” (if coded) or the value(s) (if non-coded) for the component values reported in that column for the corresponding observation, e.g. `A: A value name`.
  - If option `labels=name` (see *optional parameters*): An additional column is added right after the component identification column containing the localised name, e.g. `A value name`, of the component value reported in the previous column. It is empty if the value has no localised name.
  - For rows containing the information related to one specific observation, the related values for attributes attached to partial keys may have to be replicated.
  - For rows containing the information related to one or more attributes attached to partial keys, in addition to these attributes only the components that are part of the partial key need to be filled, all other components can be left empty. Also the columns not related to the attribute’s data structure (when data from different data structures are present) are to be left empty.
  - For rows containing information to be deleted, the deletion is assumed to take place at the lowest level of detail provided in the message. For that purpose, to be deleted measure or attribute values are non-empty, e.g. marked with the dash character “-“. Delete operations allow wildcarding dimensions by leaving the corresponding dimension field empty.
- The other custom columns contain any potentially localised custom content.

Copyright © SDMx TWG 2020

LOCALISATION

- HTTP content negotiation, see RFC 2616 - HTTP 1.1 Header Field Definitions
  - Always use this mime-type in the Accept header: `application/vnd.sdmx.data+csv; version=2.0.0`.
  - The client can indicate preferred languages through the Accept-Language header, e.g. `fr, en-gb;q=0.8, en;q=0.7`.
- Always localise all artefact names according to the preferred language. The first best language match according to the user's preferred language choices in the http Accept-Language header (or if that is not available than according to the system's default language order) is to be used for each localisable name element. The message does however not indicate the returned language per localisable name element. In case that there is no such language match for a particular localisable name element, it is optional to return the element in a system-default language or alternatively to not return the element. **It is recommended to indicate all languages used anywhere in the message for localised name elements through http Content-Language response header (languages of the intended audience).**
  Note: For multi-language values, all language versions are provided independently from the preferred language (see below).

MULTI-VALUED COMPONENTS AND NESTED METADATA ATTRIBUTES

- Some components (measures or attributes) allow for multiple values. Those multiple values are separated by a special sub-field separation character, e.g. `;`.
- This sub-field separation character has to be defined as first character in the squared bracket term of the header field of the first column, e.g. `STRUCTURE[;]`.
- Such components are indicated by having their IDs followed by empty squared brackets "[]", e.g. `ATTR4[]`.
- For coded multi-valued components, if option `labels=both` (see *optional parameters*) then each individual value is to be prefixed with its ID and the term ": ", e.g. `A: Value A;B: Value B`.
- Each metadata attribute is also to be presented in its own column(s), even if the metadata attributes are nested. In that case, the attribute IDs in the column headers are pre-fixed with the IDs of the related parent attribute(s) separated by a dot `.`, e.g. `CONTACT[].NAME[]`. All the values corresponding to one attribute are presented like a multi-valued component by respecting their position in the nested attribute tree, e.g. `name for contact 1;name for contact 2`. Parent branches in that attribute tree without a value for a specific attribute need to be indicated by leaving the corresponding multi-value sub-field empty, e.g. `name for contact 1;;name for contact 3`. Nested parent branches are indicated by using nested double quotes. Note that fields containing double quotes must themselves be encapsulated in double quotes and that nested inner double quotes need to be doubled recursively, e.g. `""name 1 for contact 1;name 2 for contact 1"";""name 1 for contact 2;name 2 for contact 2""`.

NON-CODED MULTI-LINGUAL COMPONENTS

- Some non-coded components (measures or attributes) allow for multi-lingual values. Those values are separated by a special sub-field separation character, e.g. `;`.
- This sub-field separation character has to be defined as first character in the squared bracket term of the header field of the first column, e.g. `STRUCTURE[;]`.
- Such components are indicated by having their IDs followed by the list of possible 2-letter ISO language codes separated by the sub-field separator and encapsulated squared brackets "[]", e.g. `ATTR2[en;fr]`.
- Each individual language value is to be prefixed with its 2-letter ISO language code and a colon character ":", e.g. `en:Value;fr:Valeur`. Thus, in distinction to the ID prefix for coded values when using the HTTP accept header `labels=both` (see *optional parameters*), the language prefix `xx:` doesn't have an extra space character.
- Note that multi-lingual components are always non-coded and therefore do not interfere with value IDs.

NON-CODED MULTI-LINGUAL MULTI-VALUED COMPONENTS

- Some non-coded components (measures or attributes) allow for multiple multi-lingual values. All individual values are separated by a special sub-field separation character, e.g. `;`.
- This sub-field separation character has to be defined as first character in the squared bracket term of the header field of the first column, e.g. `STRUCTURE[;]`.
- Such components are indicated by having its ID followed by the list of possible language codes separated by the sub-field separator and encapsulated squared brackets "[]", e.g. `ATTR2[en;fr;de]`.
- Each individual language value is to prefixed with its 2-letter ISO language code and a colon character ":", e.g. `en:Value1`.

Copyright © SDMx TWG 2020

- Each multi-lingual value set is to be encapsulated in double quotes, e.g. `"en:Value1;fr:Valeur1";"en:Value2;de:Wert2"`. However, note that fields containing double quotes must themselves be encapsulated in double quotes and that the inner double quotes need to be doubled, thus the complete example is `"""en:Value1;fr:Valeur1"";""en:Value2;de:Wert2"""`.

NON-CODED XHTML-VALUED COMPONENTS

- Some non-coded components (measures or attributes) allow for XHTML values.
- Each XHTML value is to be encapsulated in double quotes, e.g. `"<p>This is some ""metadata html""</p>"`. Remember that the inner quotes need to be doubled.
- The CSV format allows fields to contain line breaks if those fields are enclosed in double quotes. Thus XHTML values can also contain line breaks.

Optional parameters

Optional parameters can be added to the HTTP Accept header. They need to be separated by the character combination `"; "`.

- labels (id|name|both; default=id): This parameter applies to all Nameable SDMX Artefacts contained in the header and the body of the message:
  - If the parameter value is `id` then only the id/value of the artefacts is displayed.
  - If the parameter value is `name` then the id/value and the name of the artefacts are displayed in separate columns (see *columns*), the ID/value column always directly preceding its related localised name column.
  - If the parameter value is `both` then the concatenated id/value and localised name of the artefacts (see the section on localised names on how the message deals with languages) separated by `": "` are displayed. Note that the character combination `": "` could also be part of the artefact name and could therefore occur several times within the concatenated string.
- timeFormat (original|normalized; default=original):
  - If the parameter value is `original` then the time dimension (*TIME-PERIOD*) values are displayed in the SDMX *TIME_PERIOD* format as originally recorded.
  - If the parameter value is `normalized` then the time dimension (*TIME_PERIOD*) values are converted to the most granular ISO 8601 representation taking into account the highest frequency of the data in the message and the moment in time when the lower-frequency values were collected (which, e.g. at the ECB, is typically either at the beginning, middle or end of the reporting period). This eases comparisons and business analysis of multi-frequency values, e.g. in pivot tables. As an example, if annual and daily data are available in the message and the annual data were collected at the end of the reporting period, the formatted value for the annual period 2014 becomes 2014-12-31.
- keys (none|obs|series|both; default=none): Request the addition of column(s) for keys.
  - If the value is `none` (the default), no related column will be added.
  - If the value is `obs`, a new column OBS_KEY will be added after the ACTION column. The column will contain the combination of IDs/values for all the dimensions, order by their order in the data structure definition and separated by a dot character (.), e.g. M.USD.EUR.SP00.2020-01
  - If the value is `series`, a new column SERIES_KEY will be added after the ACTION column. The column will contain the combination of IDs/values for all the dimensions except the one(s) attached to the observation, ordered by their order in the data structure definition and separated by a dot character (.), e.g. M.USD.EUR.SP00
  - If the value is `both`, both a SERIES_KEY and an OBS_KEY columns must be added after the ACTION column, starting with the SERIES_KEY column.

Examples

Note: All examples assume the minimal HTTP Accept header: `application/vnd.sdmx.data+csv; version=1.0.0`

Copyright © SDMx TWG 2020

Ordinary case

```
STRUCTURE,STRUCTURE_ID,ACTION,DIM_1,DIM_2,DIM_3,OBS_VALUE,ATTR_2,ATTR_3,ATTR_1,UPDATED
dataflow,ESTAT:NA_MAIN(1.6.0),I,A,B,2014-01,12.4,Y,"Normal, special and other values",N,2021-01-22T13:15:41Z
dataflow,ESTAT:NA_MAIN(1.6.0),I,A,B,2014-02,10.8,Y,"Normal, special and other values",Y,2021-01-22T13:15:41Z
```

Note

- The following default parameter settings are automatically applied:
  - `labels=id`
  - `timeFormat=original`
- *UPDATED* is a custom column

Components in any order, missing component(s), component with multiple values

```
STRUCTURE[;],STRUCTURE_ID,ACTION,OBS_VALUE1,OBS_VALUE2,ATTR_3,ATTR_1[],DIM_2,DIM_1,DIM_3
dataflow,ESTAT:NA_MAIN(1.6.0),I,12.4,12.5,"Normal, special and other values",X;Y,B,A,2014-01
dataflow,ESTAT:NA_MAIN(1.6.0),I,10.8,10.9,"Normal, special and other values",X;Z,B,A,2014-02```
```

Components in any order and missing component, HTTP Accept header: `application/vnd.sdmx.data+csv; version=1.0.0; key=series`

```
STRUCTURE[;],STRUCTURE_ID,ACTION,SERIES_KEY,OBS_VALUE1,OBS_VALUE2,ATTR_3,ATTR_1,DIM_2,DIM_1,DIM_3
dataflow,ESTAT:NA_MAIN(1.6.0),I,A.B,12.4,12.5,"Normal, special and other values",N,B,A,2014-01
dataflow,ESTAT:NA_MAIN(1.6.0),I,A.B,10.8,10.9,"Normal, special and other values",Y,B,A,2014-02
```

Localisation: HTTP Accept header: `application/vnd.sdmx.data+csv; version=1.0.0; labels=both; key=both`, HTTP Accept-Language header: `fr-FR, en;q=0.7`

```
STRUCTURE[|];STRUCTURE_ID;ACTION;SERIES_KEY;OBS_KEY;DIM_1: Dimension 1;DIM_2: Dimension 2;DIM_3: Dimension 3;OBS_VALUE: Observation value;ATTR_2: Attribut
2;ATTR_3: Attribut 3;ATTR_1: Attribut 1
dataflow;ESTAT:NA_MAIN(1.6.0): Principaux agrégats des comptes nationaux;I;A.B;A.B.2014-01;A: Value A;B: Value B;2014-01: 2014-01;12,4;Y: Oui;Normal, special and
other values;N: Non
dataflow;ESTAT:NA_MAIN(1.6.0): Principaux agrégats des comptes nationaux;I;A.B;A.B.2014-02;A: Value A;B: Value B;2014-02: 2014-02;10,8;Y: Oui;Normal, special and
other values;Y: Oui
```

Note that in this example the client prefers French (fr) language with the France (FR) locale, but will also accept any type of English. Therefore, in the message the French language with the France locale is applied, transforming also the field separator from comma (,) to semicolon (;), and the decimal separator from dot (.) to comma (,).

HTTP Accept header: `application/vnd.sdmx.data+csv; version=1.0.0; labels=both; timeFormat=normalized`

```
STRUCTURE[;],STRUCTURE_ID,ACTION,DIM_1: Dimension 1,DIM_2: Dimension 2,DIM_3: Dimension 3,OBS_VALUE: Observation value,ATTR_2: Attribute 2,ATTR_3: Attribute
3,ATTR_1: Attribute 1
dataflow,ESTAT:NA_MAIN(1.6.0): National Accounts Main Aggregates,I,A: Value A,B: Value B,2014-01-01,12.4,Y: Yes,"Normal, special and other values",N: No
dataflow,ESTAT:NA_MAIN(1.6.0): National Accounts Main Aggregates,I,A: Value A,B: Value B,2014-02-01,10.8,Y: Yes,"Normal, special and other values",Y: Yes
```

HTTP Accept header: `application/vnd.sdmx.data+csv; version=1.0.0; labels=name`

```
STRUCTURE,STRUCTURE_ID,STRUCTURE_NAME,ACTION,DIM_1,Dimension 1,DIM_2,Dimension 2,DIM_3,Dimension 3,OBS_VALUE,Observation value,ATTR_1,Attribute 1,ATTR_2,Attribute
2,ATTR_3,Attribute 3
dataflow,ESTAT:NA_MAIN(1.6.0),National Accounts Main Aggregates,I,A,Value A,B,Value B,2014-01,2014-01,12.4,,Y,Yes,"Normal, special and other values",,N,No
dataflow,ESTAT:NA_MAIN(1.6.0),National Accounts Main Aggregates,I,A,Value A,B,Value B,2014-02,2014-02,10.8,,Y,Yes,"Normal, special and other values",,Y,Yes
```


Copyright © SDMx TWG 2020

Multi-valued components

```
STRUCTURE[;],STRUCTURE_ID,ACTION,DIM_1,DIM_2,DIM_3,OBS_VALUE,ATTR_1[],ATTR_2[],ATTR_3[]
dataflow,ESTAT:NA_MAIN(1.6.0),I,A,B,2014-01,12.4,Value X;Value Y,"M, N & O;P & Q",A;B;C
dataflow,ESTAT:NA_MAIN(1.6.0),I,A,B,2014-02,10.8,Value X;Value Y,"M, N & O;P & Q",A;C
```

Non-coded multi-lingual components, varying dataflows based on the same underlying data structure

```
STRUCTURE[;],STRUCTURE_ID,ACTION,DIM_1,DIM_2,DIM_3,OBS_VALUE,ATTR_1[en;fr]
dataflow,ESTAT:NA_MAIN(1.6.0),I,A,B,2014-01,12.4,en:Any Value;fr:N'importe quelle Valeur
dataflow,ESTAT:NA_MAIN(1.7.0),I,A,B,2014-02,10.8,"en:Value ""X"";fr:Valeur ""X"""
```

Varying structural artefacts based on same underlying data structure

```
STRUCTURE[;],STRUCTURE_ID,ACTION,DIM_1,DIM_2,DIM_3,OBS_VALUE,ATTR_1[en;fr]
dataflow,ESTAT:DF_NA_MAIN(1.6.0),I,A,B,2014-01,12.4,en:Any Value;fr:N'importe quelle Valeur
datastructure,ESTAT:DSD_NA_MAIN(1.7.0),I,A,B,2014-02,10.8,"en:Value ""X"";fr:Valeur ""X"""
dataprovision,ESTAT:DPA_NA_MAIN(1.8.0),I,A,B,2014-03,11.2,"en:Value ""Y"";fr:Valeur ""Y"""
```

Varying structural artefacts based on different underlying data structures

```
STRUCTURE[;],STRUCTURE_ID,ACTION,DIM_A1B1,DIM_A2,DIM_A3C2,DIM_B2,DIM_C1,DIM_C3,MEAS_A1B1C1,MEAS_C2,ATTR_A1,ATTR_B1
dataflow,ESTAT:DF_A(1.6.0),I,DIMVAL_A1B1,DIMVAL_A2,DIMVAL_A3C2,,,,"MEASVAL_A1B1C1",,"ATTRVAL_A1",
datastructure,ESTAT:DSD_B(1.7.0),I,DIMVAL_A1B1,,,DIMVAL_B2,,,"MEASVAL_A1B1C1",,,"ATTRVAL_B1"
dataprovision,ESTAT:DPA_C(1.8.0),I,,,DIMVAL_A3C2,,DIMVAL_C1,DIMVAL_C3,"MEAS_A1B1C1","MEAS_C2",,
```

Varying actions

```
STRUCTURE,STRUCTURE_ID,ACTION,DIM_1,DIM_2,DIM_3,OBS_VALUE,ATTR_1
dataflow,ESTAT:NA_MAIN(1.6.0),A,A,B,2014-01,12.4,X
dataflow,ESTAT:NA_MAIN(1.6.0),R,A,B,2014-02,10.8,Y
```

Data for a non-versioned[1] data structure definition

```
STRUCTURE,STRUCTURE_ID,ACTION,DIM_1,DIM_2,DIM_3,OBS_VALUE,ATTR_1
datastructure,AGENCY:DF_ID,I,A,B,2014-01,12.4,N
datastructure,AGENCY:DF_ID,I,A,B,2014-02,10.8,Y
```

Attributes attached to partial keys for a data provision agreement

```
STRUCTURE,STRUCTURE_ID,ACTION,DIM_2,DIM_3,ATTR_1
dataprovision,AGENCY:DPA_ID(1.0.0),I,B,2014-01,N
dataprovision,AGENCY:DPA_ID(1.0.0),I,B,2014-02,Y
```

Mixing rows for attributes attached to partial keys with rows for observations

```
STRUCTURE,STRUCTURE_ID,ACTION,DIM_1,DIM_2,DIM_3,MEAS_1,ATTR_1,ATTR_2
dataflow,AGENCY:DF_ID(1.0.0),I,A,B,2014-01,12.4,N,
dataflow,AGENCY:DF_ID(1.0.0),I,,B,,,,Y
```

Nested metadata attributes attached to partial keys

```
STRUCTURE,STRUCTURE_ID,ACTION,DIM_2,COLLECTION.METHOD[en;fr],CONTACT[],CONTACT[].NAME[]
dataflow,AGENCY:DF_ID(1.0.0),I,A,en:AAA;fr:BBB,Contact 1;Contact 2,"""Contact 1 Name 1;Contact 1 Name 2"";""Contact 1 Name 1;Contact 2 Name 2"""
dataflow,AGENCY:DF_ID(1.0.0),I,B,en:CCC;fr:DDD,Contact 1;Contact 2;Contact 3,"""Contact 1 Name 1;Contact 1 Name 2"";;""Contact 3 Name 1;Contact 3 Name 2"""
```


Copyright © SDMx TWG 2020

Non-coded XHTML-formatted values with line-breaks

```
STRUCTURE,STRUCTURE_ID,ACTION,DIM_1,DIM_2,DIM_3,OBS_VALUE,ATTR_1
dataflow,ESTAT:NA_MAIN(1.6.0),I,A,B,2014-01,12.4,"<p>This is some ""xhtml"" with a line
break</p>"
dataflow,ESTAT:NA_MAIN(1.6.0),I,A,B,2014-02,10.8,"<p>This is some other ""xhtml""</p>"
```

Deleting specific measure and attribute values: all non-empty values (e.g. marked with '-') are deleted

```
STRUCTURE,STRUCTURE_ID,ACTION,DIM_1,DIM_2,DIM_3,OBS_VALUE,ATTR_2,ATTR_3,ATTR_1
dataflow,ESTAT:NA_MAIN(1.6.0),D,A,B,2014-01,-,,,
dataflow,ESTAT:NA_MAIN(1.6.0),D,A,B,2014-02,,,-,
```

Deleting specific measure and attribute values with wildcarded dimensions: all non-empty values (e.g. marked with "-") are deleted for all dimension combinations where

- row 2: DIM2=A
- row 3: DIM2=B

```
STRUCTURE,STRUCTURE_ID,ACTION,DIM_1,DIM_2,DIM_3,OBS_VALUE,ATTR_2,ATTR_3,ATTR_1
dataflow,ESTAT:NA_MAIN(1.6.0),D,,A,,-,,,
dataflow,ESTAT:NA_MAIN(1.6.0),D,,B,,,,-,
```

Deleting whole observations with wildcarded dimensions: all observations are deleted for all dimension combinations where

- row 2: DIM2=A
- row 3: DIM2=B and DIM3=C

```
STRUCTURE,STRUCTURE_ID,ACTION,DIM_2,DIM_3
dataflow,ESTAT:NA_MAIN(1.6.0),D,A,,
dataflow,ESTAT:NA_MAIN(1.6.0),D,B,C,
```

Deleting all data for a data structure definition

```
STRUCTURE,STRUCTURE_ID,ACTION
datastructure,ESTAT:DSD_NA_MAIN(1.6.0),D
```

or

```
STRUCTURE,STRUCTURE_ID,ACTION,DIM_1,DIM_2,DIM_3
datastructure,ESTAT:DSD_NA_MAIN(1.6.0),D,,,
```

1. Note that since SDMX 3.0.0 the syntax *AGENCY:ARTEFACT_ID(VERSION)* allows omitting the version for non-versioned artefacts. In this case using *AGENCY:ARTEFACT_ID* is sufficient, e.g. `AGENCY:DF_ID` ↩↩

Copyright © SDMx TWG 2020

7.3.3 Metadata Message Field Guide

Introduction

SDMX-CSV Data Message is an SDMX data exchange format based on the RFC 4180. CSV is a widely used standardised and simple format to exchange data supported by many tools.

SDMX-CSV integrates with other specifications, i.e.:

- The SDMX API RESTful specification (e.g. content negotiation with mime-type to get SDMX-CSV representations, specific formats for responses, language selection through HTTP content negotiation)
- The RFC 4180 specification

RFC 4180: A COMMON FORMAT FOR CSV FILES

In order to benefit from best practices, SDMX-CSV is based on the rules defined in the RFC 4180, which defines a common format and MIME Type for CSV files. It is advised to read the (very short) RFC for a full list of requirements but, in a nutshell, the RFC defines rules such as:

- How the CSV file should be structured (the RFC specifies that all records must have an identical structure (determined column number), like when using an SDMX “flat” representation for data);
- When double-quotes should be used and how to escape them when needed;
- How spaces should be handled: Spaces are considered part of a field and should not be ignored;
- Which mime type should be used;
- What is the default character set, etc.

Design principles for SDMX-CSV 2.0 Metadata Messages (aligned with SDMX 3.0.0)

- In order to ensure the identifiability of the metadata contained in the message, the header row containing the column headers is mandatory and its content is well-defined.
- An SDMX-CSV referential metadata message contains metadata attribute values for one or more metadatasets reported for one or more metadataflows or metadata provision agreements.
- After the mandatory header row, each row contains the information related to one specific metadataset attached to one or more identifiable artefacts (targets).
- In RFC 4180, csv stands for “comma-separated values”. However, while SDMX-CSV uses indeed the “comma” (%x2C) as the default field separator, it adopts the wider interpretation of csv as “character-separated values”. It is recommended for implementers to provide SDMX-CSV messages according to the locale of the user (e.g. as indicated in the http Accept-Language header). It means that e.g. the semi-colon ‘;’ (as used typically in specific regions or countries) is acceptable as separator. See also the examples below. Note that the separator used in a message can be determined by retrieving the character that follows the header field of the first column which extended by a squared bracket term (see below).

COLUMNS

- The first column is always used for the underlying type of structure by which the metadataset is defined: metadataflow or metadata provision agreement.
- The next one or two columns are always used for the related structure identification.
- The next one or two columns are used for the metadataset identification.
- The next column is used for the action to be performed for the metadataset.
- The next column is used for the structure types of all targets of the metadataset.
- The next one or two columns are used for the identification of all targets of the metadataset.
- Each metadata attribute of the included metadataset(s) is represented in one or two columns. SDMX web services should return the columns in the metadata attribute order as defined in (each of) the underlying Metadata Structure Definition(s), thus in case of data defined by different metadata structures: first the metadata attributes of the first metadata structure, then the

Copyright © SDMx TWG 2020

remaining metadata attributes of the second metadata structure and so forth. However, any order of these columns is valid for metadata uploads to SDMX-consuming systems.

- Implementers have the possibility to add any other custom columns as required, e.g. publicationPeriod, publicationYear, reportingBegin, reportingEnd, prepared, etc.
- In the context of appending or deleting metadata, certain columns may be omitted, see below.

COLUMN HEADERS (FIRST ROW)

- The header field of the first column always starts with the term `MDSTRUCTURE`.
  - This field must be extended with a sub-field delimiter encapsulated in squared brackets "[]", e.g. `MDSTRUCTURE[;]`, in case the message contains metadatasets with multiple targets or with multi-instance or multi-language metadata attributes.
- The header field of the second column always contains the term `MDSTRUCTURE_ID`.
- If option `labels=name` (see *optional parameters*): An additional column is added right after the structure identification column containing the term `MDSTRUCTURE_NAME`.
- The header field of the next column always contains the term `METADATASET_ID`.
- If option `labels=name` (see *optional parameters*): An additional column is added right after the metadataset identification column containing the term `METADATASET_NAME`.
- The header field of the next column should contain the term `ACTION`. For convenience, if this column is not present, a default action ("Information") is assumed for the whole message.
- The header field of the next column contains the term `TARGET_TYPES`.
- The header field of the next column contains the term `TARGET_IDS`.
- If option `labels=name` (see *optional parameters*): An additional column is added right after the target identification column containing the term `TARGET_NAMES`.
- The other columns for components contain:
  - Default: The ID of the metadata attribute reported in that column prefixed by all corresponding nested parent metadata attributes separated by a dot "." in the form *METADATA_ID[.METADATA_ID]+*, e.g. `ATTRIBUTE_GRANDPARENT_ID.ATTRIBUTE_PARENT_ID.ATTRIBUTE_CHILD_ID`. Additional pairs of squared brackets `[]` are added at the end of the IDs of those metadata attributes that have multiple instances, e.g. `CONTACT[].NAME`, `CONTACT[].PHONE[]` or `CONTACT.PHONE[]`, and/or that contain localised values. In the latter case the brackets encapsulate the ISO 2-letter language codes that can be encountered in that column, separated by the special sub-field separation character, e.g. `;`, defined in the squared bracket term of the header field of the first column, e.g. `MDSTRUCTURE[;]`. Example of a localised child attribute: `PROCESS.STEP[en;fr]`, and for multiple instances: `PROCESS.STEP[][en;fr]`.
  - If option `labels=both` (see *optional parameters*): The full ID (as described above under 'Default') and the localised name of the metadata attribute reported in that column separated by the term ": ", e.g. `ATTRIBUTE_ID: ATTRIBUTE_NAME.
  - If option `labels=name` (see *optional parameters*): An additional column is added right after the metadata attribute identification column containing the localised name of the metadata attribute reported in the previous column.
- Any other custom column contains a custom but unique term, e.g. `publicationPeriod`.

COLUMN CONTENT (ALL ROWS AFTER HEADER)

- The first column contains: `metadataflow` or `metadataprovision`, depending on type of artefact for which the metadata contained in the message are defined: metadataflow or metadata provision agreement.
- The second column contains:
  - Default: The structure identification information in the form *AGENCY:ARTEFACT_ID(VERSION)*[1], e.g. `ESTAT:MDF(1.6.0)`.
  - If option `labels=both` (see *optional parameters*): The structure identification information and its localised name separated by the term ": ", e.g. `ESTAT:MDF(1.6.0): Metadataflow name`.
- If option `labels=name` (see *optional parameters*): An additional column is added right after the structure identification column with the structure's localised name, e.g. `Metadataflow name`.

Copyright © SDMx TWG 2020

- The next column contains the metadataset identification information in the form *AGENCY:ARTEFACT_ID(VERSION)*[1], e.g. `AGENCY:MD_SET(1.0.0)`.
  - If option `labels=both` (see *optional parameters*): The ID and the localised name of the metadataset separated by the term ": ", e.g. `ESTAT:MD_SET(1.0.0): Metadataset 1`.
- If option `labels=name` (see *optional parameters*): An additional column is added right after the metadataset identification column with the metadataset's localised name, e.g. `Metadata set name`.
- The next column contains one character representing one of the current 4 action types:
  - "I": Information - Metadata is for information purposes. If such metadata messages are loaded into an SDMX database, the action "A" (Append) is assumed.
  - "A": Append - Metadata is intended for an incremental update of existing metadatasets or the provision of new metadatasets formerly absent. This means that only the information provided explicitly in the message should be altered. Any metadata attribute value that is to be added or changed must be provided. However, the absence of a metadata attribute value at any metadata nesting level does not imply deletion; instead it is simply implied that the value is to remain unchanged. Therefore, it is valid and acceptable to send a metadata message with an action of 'Append' which (in addition to the required identification columns) contains only the column and value of a parent metadata attribute. In this case, only that value will be updated. The values of the child metadata attributes are not changed or deleted. Note that it is not permissible to update metadata attributes using incomplete identification information, e.g. without the metadataset or without the target identifier. In order to update a metadata attribute, the full identification information (all identification columns listed here) must always be provided. According to the SDMX 3.0 semantic versioning rules, it is not possible to update a semantically versioned metadataset.
  - "R": Replace - Existing metadatasets are to be fully replaced. Metadatasets formally absent will be added. According to the SDMX 3.0 semantic versioning rules, it is not possible to replace a semantically versioned metadataset.
  - "D": Delete - Metadata is to be deleted. 'Delete' is assumed to be an incremental deletion. The deletion is to take place at the lowest level of detail provided in the row. Concretely, if a 'Delete' row only contains the identification information of the structural artefact (metadataflow or metadata provision agreement) without a metadataset then all metadatasets for the given artefact will be deleted. If a 'Delete' row only contains up to the metadataset identification without metadata attributes then the given metadataset is to be deleted. Finally, if a row contains all complete identification information up to a non-versioned metadataset and some values for metadata attributes, then only the metadata attributes with values will be deleted from the non-versioned metadataset. To be deleted attribute values must be non-empty, e.g. marked with the dash character "-". According to the SDMX 3.0 semantic versioning rules, it is not possible to modify a semantically versioned metadataset.
  - For convenience, if this column is absent then the action "Information" is assumed.
- The next column contains the types of all the targets of the metadataset according to the resource names defined for Structural Metadata Queries, e.g. `dataflow`. Multiple targets are separated by the special sub-field separation character, e.g. `;`, defined in the squared bracket term of the header field of the first column, e.g. `MDSTRUCTURE[;]`. Example for multiple target types: `dataflow;dataflow`.
- The next column contains the identification information of all the targets of the metadataset in the form *AGENCY:ARTEFACT_ID(VERSION)* [1], separated by the sub-field separation character, e.g. `AGENCY:DF1(1.0.0);AGENCY:DF2(1.0.0)`.
  - If option `labels=both` (see *optional parameters*): The column contains the ID and the localised name of the targets separated by the term ": ", e.g. `AGENCY:DF(1.0.0): Dataflow name` or `AGENCY:DF1(1.0.0): Dataflow 1 name;AGENCY:DF2(1.0.0): Dataflow 2 name`.
- If option `labels=name` (see *optional parameters*): An additional column is added right after the target identification column with the target's localised name, e.g. `Dataflow name` or `Dataflow 1 name;Dataflow 2 name`.

Copyright © SDMx TWG 2020

- The other columns for metadata attributes contain:
  - Default: The ID(s) (if coded) or value(s) (if non-coded) for the metadata attribute reported in that column, e.g. `A`, `A;B` or `"<p>An XHTML text</p>"`.
  - If option `labels=both` (see *optional parameters*): The ID(s) and their localised name(s) for the metadata attribute separated by the term ": " (if coded) or the value(s) (if non-coded) for the metadata attribute reported in that column, e.g. `A: A value name`, `A: A value name;B: B value name` or `"<p>An XHTML text</p>"`.
  - If option `labels=name` (see *optional parameters*): An additional column is added right after the metadata attribute identification column containing the localised name, e.g. `A value name` or `A value name;B value name`, of the metadata attribute value reported in the previous column. It is empty if the value has no localised name.
  - All string/textual values (complete string between column-separating characters including ID's or language codes) should always be encapsulated in quotation marks, they must be if they contain commas or inner quotation marks. Quotation marks in strings/textual values must always be escaped by doubling the quotes.
  - When metadata from different metadata structures are present then the columns not related to the attribute's metadata structure are to be left empty.
- The other custom columns contain any potentially localised custom content.

LOCALISATION

- HTTP content negotiation, see RFC 2616 - HTTP 1.1 Header Field Definitions
  - Always use this mime-type in the Accept header: `application/vnd.sdmx.metadata+csv; version=2.0.0`.
  - The client can indicate preferred languages through the Accept-Language header, e.g. `fr, en-gb;q=0.8, en;q=0.7`.
- Always localise all artefact names according to the preferred language. The first best language match according to the user's preferred language choices in the http Accept-Language header (or if that is not available than according to the system's default language order) is to be used for each localisable name element. The message does however not indicate the returned language per localisable name element. In case that there is no such language match for a particular localisable name element, it is optional to return the element in a system-default language or alternatively to not return the name element. **It is recommended to indicate all languages used anywhere in the message for localised name elements through http Content-Language response header (languages of the intended audience).**
  Note: For multi-language metadata attribute values, all language versions are provided independently from the preferred language (see below).

MULTI-INSTANCE METADATA ATTRIBUTES

- Values from multiple instances of a metadata attribute within a metadataset are separated by the special sub-field separation character, e.g. `;`, defined in the squared bracket term of the header field of the first column, e.g. `MDSTRUCTURE[;]`.
- Such metadata attributes are indicated in the column header by having their ID followed by empty squared brackets "[]", e.g. `ATTR[]`.
- For coded multi-instance metadata attributes, if option `labels=both` (see *optional parameters*) then each individual value is to be prefixed with its ID and the term ": ", e.g. `A: Value A;B: Value B`.

NON-CODED MULTI-LINGUAL METADATA ATTRIBUTES

- Non-coded metadata attributes allow for multi-lingual values. Those values are separated by the special sub-field separation character, e.g. `;`, defined in the squared bracket term of the header field of the first column, e.g. `MDSTRUCTURE[;]`.
- Such metadata attributes are indicated in the column header by having their ID followed by the list of possible 2-letter ISO language codes separated by the sub-field separator and encapsulated squared brackets "[]", e.g. `ATTR[en;fr]`.
- Each individual language value is to be prefixed with its 2-letter ISO language code and a colon character ":", e.g. `en:Value;fr:Valeur`. Thus, in distinction to the ID prefix for coded values when using the HTTP accept header `labels=both` (see *optional parameters*), the language prefix `xx:` doesn't have an extra space character.

NON-CODED MULTI-LINGUAL MULTI-INSTANCE METADATA ATTRIBUTES

- When non-coded multi-lingual metadata attributes have multiple instances within a metadataset, then all individual values are included and separated by the special sub-field separation character, e.g. `;`, defined in the squared bracket term of the header field of the first column, e.g. `MDSTRUCTURE[;]`.

Copyright © SDMx TWG 2020

- Such metadata attributes are indicated in the column header by having their ID followed by squared brackets “[]” as well as by the list of possible language codes separated by the sub-field separator and encapsulated in additional squared brackets “[]”, e.g. `ATTR[][en;fr;de]`.
- Each individual language value is to prefixed with its 2-letter ISO language code and a colon character “:”, e.g. `en:Value1`.
- Not each value needs all language versions. In order to allow knowing to which value the different language items belong, each multi-lingual value set is to be encapsulated in double quotes, e.g. `"en:Value1;fr:Valeur1";"en:Value2;de:Wert2"`. However, note that fields with double quotes must themselves be encapsulated in double quotes and that the inner double quotes need to be doubled, thus the fully complete example is `"""en:Value1;fr:Valeur1"";""en:Value2;de:Wert2"""`.

NON-CODED XHTML-VALUED COMPONENTS

- Some non-coded metadata attributes allow for XHTML values.
- Each XHTML value is to be encapsulated in double quotes, e.g. `"<p>This is some ""metadata html""</p>"`. Remember that the inner double quotes need to be doubled.
- The CSV format allows fields to contain line break characters if those fields are enclosed in double quotes. Thus XHTML values can also contain line breaks, although HTML viewers will ignore them.

Optional parameters

The following optional parameter can be added to the HTTP Accept header. It needs to be separated by the character combination `"; "`.

- labels (id|name|both; default=id): This parameter applies to all Nameable SDMX Artefacts contained in the header and the body of the message:
  - If the parameter value is `id` then only the id of the Artefacts is displayed.
  - If the parameter value is `both` then the concatenated id and localised name of the Artefacts (see the section on localised names on how the message deals with languages) separated by `": "` are displayed. Note that the character combination `": "` could also be part of the Artefact name and could therefore occur several times within the concatenated string.
  - If the parameter value is `name` then the id/value and the name of the artefacts are displayed in separate columns (see *here*), the ID/value column always directly preceding its related localised name column.

Examples

Note: All examples assume the minimal HTTP Accept header: `application/vnd.sdmx.metadata+csv; version=1.0.0`

Ordinary case

```
MDSTRUCTURE,MDSTRUCTURE_ID,METADATASET_ID,ACTION,TARGET_TYPES,TARGET_IDS,ATTRIBUTE_1,ATTRIBUTE_1.CHILD,ATTRIBUTE_2
metadataflow,OECD:MDF(1.0.0),OECD:MDS(1.0.0),I,dataflow,OECD:DF(1.0.0),A STRING VALUE,"<p>An XHTML text with ""quotes""</p>",123
```

Note

The following default parameter settings are automatically applied:

- `labels=id`

??? example “Metadata attribute with multiple instances and multi-lingual values”

```csv
MDSTRUCTURE[;],MDSTRUCTURE_ID,METADATASET_ID,ACTION,TARGET_TYPES,TARGET_IDS,ATTRIBUTE_1,ATTRIBUTE_1.ATTRIBUTE_1_2[][en;fr],ATTRIBUTE_2[],ATTRIBUTE_3[]
metadataflow,OECD:MDF(1.0.0),OECD:MDS(1.0.0),I,dataflow,OECD:DF(1.0.0),CODE_ID,"""en:""""<p>An XHTML text</p>""""";fr:""""<p>Un texte XHTML</
p>""""";""en:""""<p>Another XHTML text</p>""""";fr:""""<p>Un autre texte XHTML</p>""""""","""Text with """"quotes""""""";""Another text""",123;456
```


Copyright © SDMx TWG 2020

??? example “Localisation: HTTP Accept header: `application/vnd.sdmx.metadata+csv; version=1.0.0; labels=both`, HTTP Accept-Language header: `fr-FR, en;q=0.7`, metadata attribute with multiple instances, metadata attributes with multi-lingual values”

```` csv
MDSTRUCTURE[|],MDSTRUCTURE_ID;METADATASET_ID;ACTION;TARGET_TYPES;TARGET_IDS;ATTRIBUTE_1: Attribut d'exemple 1;ATTRIBUTE_1.ATTRIBUTE_1_2[][en|fr]: Attribut
d'exemple 12;ATTRIBUTE_2[]: Attribut d'exemple 2
metadataflow;OECD:MDF(1.0.0): Metadataflow d'exemple;OECD:MDS(1.0.0): Metadataset d'exemple;I;dataflow;OECD:DF(1.0.0): Dataflow d'exemple;CODE_ID: Nom du
code;"""en:""""<p>An XHTML text</p>""""|fr:""""<p>Un texte XHTML</p>""""""|""en:""""<p>Another XHTML text</p>""""|fr:""""<p>Un autre texte XHTML</p>""""""""";
123,45|6,789
````

Note that in this example the client prefers French (fr) language with the France (FR) locale, but will also accept
any type of English. Therefore, in the message the French language with the France locale is applied, transforming
also the field separator from comma (,) to semicolon (;), and the decimal separator from dot (.) to comma (,).

??? example “Localisation: HTTP Accept header: `application/vnd.sdmx.metadata+csv; version=1.0.0; labels=name`, HTTP Accept-Language header: `en-US`, metadata attribute with multiple instances, metadata attributes with multi-lingual values, different targets and metadatasets”

```` csv
MDSTRUCTURE[;],MDSTRUCTURE_ID,MDSTRUCTURE_NAME,METADATASET_ID,METADATASET_NAME,ACTION,TARGET_TYPES,TARGET_IDS,TARGET_NAMES,ATTRIBUTE_1,Attribute
1,ATTRIBUTE_1.ATTRIBUTE_1_2[][en|fr],Attribute 12,ATTRIBUTE_2[],Attribute 2
metadataflow,OECD:MDF(1.0.0),Metadataflow name,OECD:MDS(1.0.0),Metadataset name,I,dataflow;dataflow,OECD:DF(1.0.0);OECD:DF(1.1.0),Dataflow name 1;Dataflow
name 2,CODE_ID,Code name,"""en:""""<p>An XHTML text</p>""""";fr:""""<p>Un texte XHTML</p>""""""";""en:""""<p>Another XHTML text</p>""""";fr:""""<p>Un autre texte
XHTML</p>""""""",123.45;6.789
metadataflow,OECD:MDF(1.0.0),Metadataflow name,OECD:MDS(1.1.0),Metadataset new name,I,codelist,OECD:CL(1.0.0),Codelist name,CODE_ID,Code
name,"""en:""""<p>Text 1</p>""""";fr:""""<p>Texte 1</p>""""""";""en:""""<p>Text 2</p>""""";fr:""""<p>Texte 2</p>""""""",0
````

Varying metadataflows

```
MDSTRUCTURE[;],MDSTRUCTURE_ID,METADATASET_ID,ACTION,TARGET_TYPES,TARGET_IDS,ATTRIBUTE_1,ATTRIBUTE_2[][en;fr;de]
metadataflow,OECD:MDF(1.0.0),OECD:MDS(1.0.0),I,dataflow,OECD:DF(1.0.0),CODE_ID,"""en:Value1;fr:Valeur1"";""en:Value2;de:Wert2"""
metadataflow,OECD:MDF(1.1.0),OECD:MDS(1.1.0),I,dataflow,OECD:DF(1.1.0),CODE_ID,"""en:Value1;fr:Valeur1"";""en:Value2;de:Wert2"""
```

??? example “Non-versioned metadataset for a non-versioned[1] data provision agreement”

```` csv
MDSTRUCTURE[;],MDSTRUCTURE_ID,METADATASET_ID,ACTION,TARGET_TYPES,TARGET_IDS,ATTRIBUTE_1,ATTRIBUTE_2[en;fr]
metadataprovision,OECD:MDP,OECD:MDS,I,dataflow,OECD:DF(1.0.0),CODE_ID,"en:Value1;fr:Valeur1"
````

Non-coded metadata attribute values with line-breaks

```
MDSTRUCTURE[;],MDSTRUCTURE_ID,METADATASET_ID,ACTION,TARGET_TYPES,TARGET_IDS,ATTRIBUTE_1[]
metadataflow,OECD:MDF(1.0.0),OECD:MDS(1.0.0),I,dataflow,OECD:DF(1.0.0),"""This text with a line
break"";""This is some other text</p>"""
```

??? example “Deleting all values of specific metadata attributes from a non-versioned metadatset: the values that correspond to the attribute identifiers are deleted”

```` csv
MDSTRUCTURE[;],MDSTRUCTURE_ID,METADATASET_ID,ACTION,TARGET_TYPES,TARGET_IDS,ATTRIBUTE_1[],ATTRIBUTE_1[].ATTRIBUTE_1_2[],ATTRIBUTE_2
metadataflow,OECD:MDF(1.0.0),OECD:MDS,D,dataflow,OECD:DF(1.0.0),-,-,-
````

Deleting a whole metadataset

```
MDSTRUCTURE[;],MDSTRUCTURE_ID,METADATASET_ID,ACTION
metadataflow,OECD:MDF(1.0.0),OECD:MDS(1.0.0),D
metadataflow,OECD:MDF(1.1.0),OECD:MDS(1.1.0),D
```


Copyright © SDMx TWG 2020

Deleting all metadatasets defined by a specific metadata artefact

```
MDSTRUCTURE[;],MDSTRUCTURE_ID,METADATASET_ID,ACTION
metadataflow,OECD:MDF(1.0.0),,D
metadataflow,OECD:MDF(1.1.0),,D
```

1. Note that since SDMX 3.0.0 the syntax *AGENCY:ARTEFACT_ID(VERSION)* allows omitting the version for non-versioned artefacts. In this case using *AGENCY:ARTEFACT_ID* is sufficient, e.g. `OECD:MDP` ↩↩↩↩

Copyright © SDMx TWG 2020